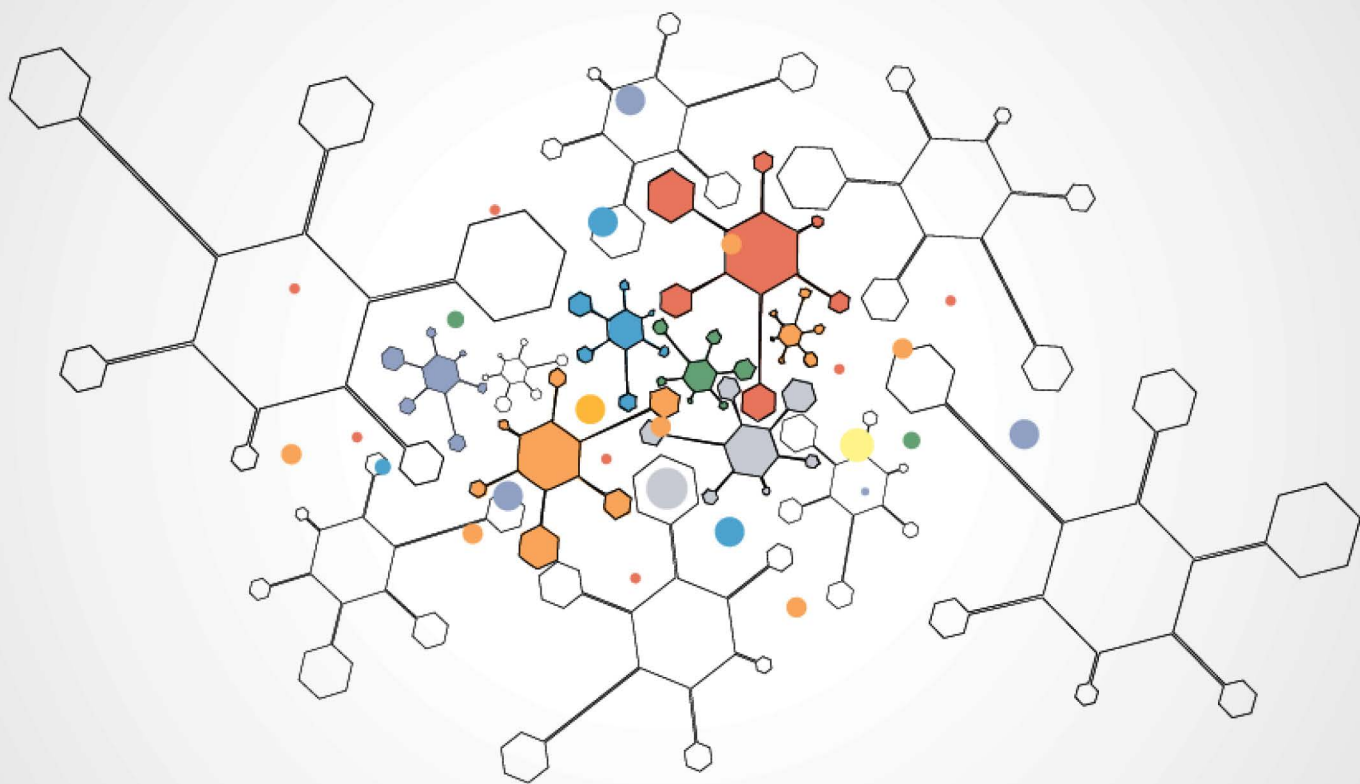


◁packt▷

# Microservices Design Patterns in .NET

SECOND EDITION



Making sense of microservices  
design and architecture using  
.NET 10 and C# 14

**TREVOR WILLIAMS**

# Microservices Design Patterns in .NET

Second Edition

Making sense of microservices design and architecture using  
.NET 10 and C# 14

**Trevor Williams**



# Microservices Design Patterns in .NET

Second Edition

Copyright © 2025 Packt Publishing

*All rights reserved.* No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

This book was written by author. Generative AI tools were used only to assist with phrasing and for some code samples. Packt does not accept AI-generated content that replaces expert authorship.

**Portfolio Director:** Ashwin Nair

**Relationship Leads:** Suman Sen and Nitin Nainani

**Project Manager:** Ruvika Rao

**Content Engineer:** Nithya Sadanandan

**Technical Editor:** Rohit Singh

**Copy Editor:** Safis Editing

**Indexer:** Hemangini Bari

**Proofreader:** Nithya Sadanandan

**Production Designer:** Alishon Falcon

**Growth Lead:** Priyadarshini Sharma

First published: December 2022

Second edition: December 2025

Production reference: 1111225

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83702-741-5

[www.packtpub.com](http://www.packtpub.com)

*To my family, your patience, love, and constant encouragement make the long nights and early mornings worth it. And to my students and the .NET community, thank you for the curiosity that keeps me building and teaching.*

*– Trevor Williams*

# Contributors

## About the author

**Trevor Williams** is a Jamaican-born software and systems engineer and the author of *Microservices Design Patterns in .NET* and *Effective .NET Memory Management*, as well as a contributor to the Azure Integration Guide for Business. He is an online instructor with more than 500,000 learners across courses on ASP.NET Core, EF Core, Azure, and microservices, and he teaches thousands more through his YouTube tutorials.

Trevor holds a master's degree in Computer Science (Software Development) and Microsoft Azure certifications. His work focuses on practical, production-ready .NET applications, data access, messaging, observability, and cloud deployments, as well as helping teams apply proven patterns to ship with confidence. He continues to publish articles, videos, and courses to make modern .NET development approachable for all.

*Writing a second edition is a team sport. I'm grateful to the Packt crew for their expert guidance from proposal to print. To the readers of the first edition, your reviews, issues, and emails directly shaped what we expanded here. Finally, my heartfelt thanks go to my family for their unwavering support and for giving me the space to complete this book successfully.*

## About the reviewer

**Vishnu VG** is a .NET programmer and software architect with more than 17 years of experience in the IT industry. Based in Thiruvananthapuram, Kerala, India, his expertise includes C#, .NET, .NET Core, cloud platforms such as AWS and Azure (he holds an AWS Associate certification), Linux, serverless architectures, and container technologies. He is also proficient in managing both on-premises and hybrid enterprise systems. Beyond his technical skill set, Vishnu is an active contributor to the developer community. Since 2021, he has been an AWS Community Builder, and he shares his knowledge through his YouTube channel and podcast, both titled *Coding Talks with Vishnu VG*.



# Table of Contents

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>Preface</b>                                                 | <b>xvii</b> |
| Free Benefits with Your Book .....                             | xxiv        |
| <b>Part I: Understanding Microservices and Design Patterns</b> | <b>1</b>    |
| <b>Chapter 1: Introducing Microservices – the Big Picture</b>  | <b>3</b>    |
| Approaches to software development .....                       | 4           |
| Modular monolith development pattern • 7                       |             |
| Microservices – why or why not? .....                          | 8           |
| Microservice versus monolith – scoping an application • 9      |             |
| Choosing the microservices approach • 12                       |             |
| Microservices design patterns .....                            | 15          |
| Microservices and .NET Core .....                              | 16          |
| .NET version • 17                                              |             |
| Development tools • 18                                         |             |
| Languages, frameworks, and libraries • 18                      |             |
| Summary .....                                                  | 20          |
| <b>Chapter 2: Applying Domain-Driven Design Principles</b>     | <b>23</b>   |
| Technical requirements .....                                   | 24          |
| Exploring DDD and its significance .....                       | 24          |
| The pros and cons of DDD • 27                                  |             |
| DDD and microservices • 30                                     |             |
| Designing entities and models .....                            | 31          |
| Entities and why we need them • 31                             |             |
| Practical uses of entities in code • 32                        |             |



|                                                                        |               |
|------------------------------------------------------------------------|---------------|
| Rich domain model versus anemic domain model • 34                      |               |
| <b>Understanding and using value objects .....</b>                     | <b>37</b>     |
| <b>Using the aggregate pattern .....</b>                               | <b>41</b>     |
| Aggregates and aggregate roots • 41                                    |               |
| Relationships in aggregates • 46                                       |               |
| Handling relationships that span aggregates • 46                       |               |
| <b>Summary .....</b>                                                   | <b>47</b>     |
| <br><b>Chapter 3: Synchronous Communication between Microservices</b>  | <br><b>51</b> |
| <b>Technical requirements .....</b>                                    | <b>52</b>     |
| <b>Introduction to synchronous communication .....</b>                 | <b>52</b>     |
| <b>Implementing HTTP API communication .....</b>                       | <b>54</b>     |
| Using SOAP for API development • 56                                    |               |
| Implementing a RESTful microservice • 60                               |               |
| Testing a RESTful microservice • 68                                    |               |
| Using HTTPClient • 70                                                  |               |
| <b>Implementing gRPC API communication .....</b>                       | <b>74</b>     |
| Creating a gRPC microservice • 76                                      |               |
| Communicating with a gRPC service • 79                                 |               |
| <b>Summary .....</b>                                                   | <b>82</b>     |
| <br><b>Chapter 4: Asynchronous Communication Between Microservices</b> | <br><b>85</b> |
| <b>Technical requirements .....</b>                                    | <b>86</b>     |
| <b>Introducing asynchronous communication .....</b>                    | <b>86</b>     |
| <b>Strategies for asynchronous services .....</b>                      | <b>87</b>     |
| Understanding message queues • 88                                      |               |
| Understanding message bus systems • 89                                 |               |
| Understanding eventual consistency • 91                                |               |
| Understanding the pub-sub pattern • 93                                 |               |
| <b>Implementing asynchronous communication .....</b>                   | <b>95</b>     |
| Choosing a message bus • 95                                            |               |

---

|                                                           |                |
|-----------------------------------------------------------|----------------|
| Installing RabbitMQ • 97                                  |                |
| Implementing a publisher • 99                             |                |
| Implementing a consumer • 101                             |                |
| <b>Challenges and possible solutions .....</b>            | <b>105</b>     |
| Message ordering • 105                                    |                |
| Duplicate messages • 107                                  |                |
| Additional considerations • 109                           |                |
| <b>Summary .....</b>                                      | <b>110</b>     |
| <br><b>Chapter 5: Working with the Aggregator Pattern</b> | <br><b>113</b> |
| <hr/>                                                     |                |
| <b>Technical requirements .....</b>                       | <b>114</b>     |
| <b>What is the Aggregator pattern? .....</b>              | <b>114</b>     |
| Developing a UI without an aggregator • 116               |                |
| Introducing the Aggregator pattern • 119                  |                |
| <b>Implementing the Aggregator pattern .....</b>          | <b>122</b>     |
| Creating an aggregator service • 123                      |                |
| Connecting a UI to the aggregator service • 128           |                |
| <b>Performance optimization and best practices .....</b>  | <b>132</b>     |
| Asynchronous processing • 132                             |                |
| Caching • 132                                             |                |
| Using compression • 134                                   |                |
| Error handling • 136                                      |                |
| Logging and monitoring • 136                              |                |
| <b>Summary .....</b>                                      | <b>137</b>     |
| <br><b>Chapter 6: Working with the CQRS Pattern</b>       | <br><b>139</b> |
| <hr/>                                                     |                |
| <b>Technical requirements .....</b>                       | <b>140</b>     |
| <b>Understanding the CQRS pattern .....</b>               | <b>140</b>     |
| What is the CQRS pattern? • 140                           |                |
| Benefits of the CQRS pattern • 142                        |                |
| Disadvantages of the CQRS pattern • 143                   |                |

|                                                              |                |
|--------------------------------------------------------------|----------------|
| <b>Implementing the CQRS pattern .....</b>                   | <b>144</b>     |
| Adding MediatR • 145                                         |                |
| Creating a command • 146                                     |                |
| Creating a query • 149                                       |                |
| <b>Best practices and recommendations .....</b>              | <b>152</b>     |
| Good naming conventions • 152                                |                |
| Organizing files and classes • 153                           |                |
| Speeding up read operations • 155                            |                |
| <b>Summary .....</b>                                         | <b>157</b>     |
| <br><b>Chapter 7: Applying Event-Sourcing Patterns .....</b> | <br><b>159</b> |
| <b>Technical requirements .....</b>                          | <b>160</b>     |
| <b>Understanding events and event sourcing .....</b>         | <b>160</b>     |
| What are events? • 160                                       |                |
| Key attributes of events • 161                               |                |
| <b>Event-sourcing patterns in microservices .....</b>        | <b>162</b>     |
| Pros and cons of event sourcing • 163                        |                |
| <b>Creating an event store .....</b>                         | <b>164</b>     |
| Scoping event storage using a relational database • 165      |                |
| Scoping event storage using a non-relational database • 166  |                |
| Implementing an event store with PostgreSQL • 169            |                |
| <b>Implementing event-sourcing patterns .....</b>            | <b>170</b>     |
| Domain events and event sourcing • 171                       |                |
| Implementing domain events in our application • 171          |                |
| Using events for read-only operations • 175                  |                |
| <b>Best practices and recommendations .....</b>              | <b>178</b>     |
| Designing events with care • 178                             |                |
| Using snapshots for performance optimization • 179           |                |
| Choosing an appropriate data store and ORM • 180             |                |
| <b>Summary .....</b>                                         | <b>181</b>     |

---

## **Part II: Database and Storage Design Patterns** **185**

---

### **Chapter 8: Database Design Strategies for Microservices** **187**

---

**Technical requirements** ..... 188

**Exploring database architecture patterns** ..... 188

Database per service • 189

Shared database • 190

**Developing a database** ..... 193

Relational databases • 194

Non-relational databases • 198

Choosing a database technology • 200

Choosing an ORM • 202

Choosing a database development technique • 203

**Best practices and recommendations** ..... 207

The polyglot persistence strategy • 207

Database sharding • 209

Eventual consistency • 213

**Summary** ..... 215

### **Chapter 9: Implementing Transactions across Microservices Using the Saga Pattern** **217**

---

**Technical requirements** ..... 218

**Exploring the Saga pattern** ..... 218

Issues and considerations • 219

**Understanding and implementing choreography** ..... 221

Rollback on failure • 223

Building a choreography saga pattern with MassTransit and RabbitMQ • 226

Pros and cons • 230

**Understanding and implementing orchestration** ..... 231

Rolling back on failure • 234

|                                  |     |
|----------------------------------|-----|
| Implementing orchestration • 235 |     |
| Pros and cons • 243              |     |
| Summary .....                    | 244 |

## **Part III: Resiliency, Security, and Infrastructure Patterns      247**

### **Chapter 10: Building Resilient Microservices      249**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| Technical requirements .....                                             | 250 |
| Understanding core failure scenarios between microservices .....         | 250 |
| Synchronous API failures and how to handle them • 251                    |     |
| Asynchronous API failures and how to handle them • 253                   |     |
| Implementing fault-handling policies in synchronous communication .....  | 254 |
| Adding a circuit breaker policy • 256                                    |     |
| Adding a caching layer • 257                                             |     |
| Reducing authentication errors • 262                                     |     |
| Implementing fault-handling policies in asynchronous communication ..... | 265 |
| Resiliency in message brokers and queues • 266                           |     |
| Using the outbox pattern • 267                                           |     |
| Summary .....                                                            | 270 |

### **Chapter 11: Implementing the API and BFF Gateway Patterns      273**

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| Technical requirements .....                                        | 274 |
| What is an API gateway? .....                                       | 274 |
| Advantages of an API gateway • 276                                  |     |
| Disadvantages of an API gateway • 278                               |     |
| Implementing an API gateway .....                                   | 279 |
| Using a reverse proxy • 280                                         |     |
| Implementing an API gateway using YARP • 281                        |     |
| Implementing an API gateway using Ocelot • 285                      |     |
| Backend for Frontend pattern .....                                  | 291 |
| Challenges of implementing microservices for multiple devices • 291 |     |

---

|                                                                                  |                |
|----------------------------------------------------------------------------------|----------------|
| BFF pattern using Ocelot • 292                                                   |                |
| <b>Security and rate limiting in API gateways .....</b>                          | <b>295</b>     |
| Implementing API gateway security • 296                                          |                |
| Implementing rate limiting in API gateways • 300                                 |                |
| <b>Summary .....</b>                                                             | <b>303</b>     |
| <br><b>Chapter 12: Micro Frontends — Extending Microservices to the Frontend</b> | <br><b>307</b> |
| <hr/>                                                                            |                |
| <b>Technical requirements .....</b>                                              | <b>308</b>     |
| <b>Introducing micro frontends and their benefits .....</b>                      | <b>308</b>     |
| Service-oriented architecture • 309                                              |                |
| Micro frontends and DDD • 311                                                    |                |
| Real-world implementations • 313                                                 |                |
| <b>Designing and developing micro frontends .....</b>                            | <b>314</b>     |
| Blazor versus JavaScript frameworks • 316                                        |                |
| Creating a Blazor WebAssembly micro frontend • 317                               |                |
| Micro frontends and API gateways • 321                                           |                |
| <i>How a gateway supports micro frontends • 322</i>                              |                |
| <b>Performance optimization for a smooth user experience .....</b>               | <b>324</b>     |
| Implementing ahead-of-time compilation • 324                                     |                |
| Lazy loading components • 325                                                    |                |
| Building a resilient micro frontend • 328                                        |                |
| <b>State management and communication between micro frontends .....</b>          | <b>331</b>     |
| Sharing user session state • 331                                                 |                |
| <b>Summary .....</b>                                                             | <b>336</b>     |
| <br><b>Chapter 13: Securing Microservices</b>                                    | <br><b>339</b> |
| <hr/>                                                                            |                |
| <b>Technical requirements .....</b>                                              | <b>340</b>     |
| <b>Understanding API security .....</b>                                          | <b>340</b>     |
| Essential application security concepts • 341                                    |                |
| Challenges in securing microservices • 343                                       |                |
| <b>Bearer tokens for securing communications .....</b>                           | <b>345</b>     |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| Understanding JWTs • 346                                            |            |
| Implementing access token security • 350                            |            |
| Limitations of bearer tokens • 353                                  |            |
| <b>Using OAuth and OpenID Connect to secure microservices .....</b> | <b>354</b> |
| Identity providers • 355                                            |            |
| Implementing token authentication • 357                             |            |
| Securing an API using IdentityServer • 366                          |            |
| Securing the Ocelot API gateway with IdentityServer • 369           |            |
| <b>Summary .....</b>                                                | <b>372</b> |

---

## **Part IV: Cloud Development Strategies 375**

---

### **Chapter 14: Container Hosting and Development Patterns 377**

---

|                                                     |            |
|-----------------------------------------------------|------------|
| Technical requirements .....                        | 378        |
| Using containers in development .....               | 378        |
| Understanding Docker • 380                          |            |
| Understanding container images • 381                |            |
| Pros and cons of containers • 383                   |            |
| <b>Working with Dockerfiles and images .....</b>    | <b>385</b> |
| Launching a containerized application • 389         |            |
| Using native .NET container support • 392           |            |
| Publishing containers to a container registry • 392 |            |
| <b>Understanding container orchestration .....</b>  | <b>396</b> |
| Adding Docker Compose to a project • 397            |            |
| Using Kubernetes • 403                              |            |
| <b>Summary .....</b>                                | <b>408</b> |

### **Chapter 15: Serverless Microservices Development 411**

---

|                                           |     |
|-------------------------------------------|-----|
| Technical requirements .....              | 412 |
| Introducing serverless architecture ..... | 412 |
| What is serverless computing? • 412       |     |

|                                                                       |                |
|-----------------------------------------------------------------------|----------------|
| Serverless versus containerized microservices • 414                   |                |
| Cloud providers and serverless technologies • 415                     |                |
| Advantages of serverless computing for microservices • 419            |                |
| Drawbacks of serverless computing for microservices • 420             |                |
| <b>Designing microservices with serverless in mind .....</b>          | <b>421</b>     |
| Choosing the right size for a function • 422                          |                |
| Dependency injection in serverless functions • 423                    |                |
| Durable orchestration design • 424                                    |                |
| Observability, logging, and monitoring • 427                          |                |
| Security considerations • 427                                         |                |
| Integration technologies and triggers • 428                           |                |
| <b>Building serverless microservices on Microsoft Azure .....</b>     | <b>429</b>     |
| Recommended local toolset for Azure Functions development • 431       |                |
| Building an appointment booking service on Azure Functions • 432      |                |
| <b>Summary .....</b>                                                  | <b>443</b>     |
| <br><b>Chapter 16: Observability and Monitoring with Modern Tools</b> | <br><b>445</b> |
| <b>Technical requirements .....</b>                                   | <b>446</b>     |
| <b>Implementing structured logging .....</b>                          | <b>446</b>     |
| Determining log requirements • 447                                    |                |
| Using the .NET logging API • 449                                      |                |
| Adding Serilog • 455                                                  |                |
| <b>Log aggregation and its uses .....</b>                             | <b>457</b>     |
| Integrating with Seq • 458                                            |                |
| Distributed log tracing • 460                                         |                |
| OTel and distributed tracing • 461                                    |                |
| <b>Implementing the sidecar pattern .....</b>                         | <b>464</b>     |
| Understanding service meshes • 467                                    |                |
| Pros and cons of sidecars and service meshes • 468                    |                |
| Sidecars with Dapr • 469                                              |                |



---

|                                       |            |
|---------------------------------------|------------|
| Using .NET Aspire .....               | 474        |
| Using Aspire for microservices • 475  |            |
| Summary .....                         | 482        |
| <b>Chapter 17: Wrapping It All Up</b> | <b>485</b> |

---

|                                                      |            |
|------------------------------------------------------|------------|
| Developing microservices .....                       | 486        |
| Managing data in distributed applications .....      | 487        |
| Implementing monitoring and service resilience ..... | 489        |
| Cloud-based development patterns .....               | 489        |
| Summary .....                                        | 490        |
| <b>Index</b>                                         | <b>493</b> |

---

|                                  |            |
|----------------------------------|------------|
| <b>Other Books You May Enjoy</b> | <b>508</b> |
|----------------------------------|------------|

---

# Preface

Welcome back! This second edition of *Microservices Design Patterns in .NET* builds upon the original text and reflects the evolution of the .NET and cloud-native ecosystems. You'll still learn about the core architectural patterns that make microservices effective, but now with deeper, production-ready techniques, refreshed code, and end-to-end scenarios. The main idea is the same: decompose complex domains into smaller, cohesive services that collaborate to deliver a unified user experience, while being transparent about the new complexities this introduces and the patterns that tame them.

Why .NET remains our foundation hasn't changed, but the platform itself has: modern .NET emphasizes performance, cross-platform reliability, and a rich toolchain that accelerates shipping high-quality distributed systems. In this edition, we expand into updated gateway patterns (including BFF), micro-frontends for domain-aligned UX, hardened security, container-hosting techniques for day-to-day development, serverless microservices on Azure, and unified observability with contemporary tooling. These topics complement the original arc (aggregation, CQRS, event sourcing, sagas, resilience, and health) and reflect how real teams build, secure, and operate microservices today.

We continue to focus on .NET for the following reasons:

- **Well-maintained ecosystem:** Microsoft and the community iterate quickly with clear guidance, strong docs, and first-class tooling
- **Performance:** Each release improves runtime and library performance, which is critical when service hops, serialization, and network latency dominate
- **Cross-platform:** We can build and run on Windows, macOS, or Linux, and deploy to containers, orchestrators, and serverless platforms alike
- **Breadth:** From web and messaging to identity, telemetry, and DevOps, .NET spans the full surface area you'll touch in a microservices program

**Our approach remains pragmatic:** We explain why each pattern is used, implement it in .NET, and show how the pieces fit together in a cohesive healthcare domain so you can apply them with confidence in your own systems. We'll cover container hosting and development patterns beyond docker run. You'll transition from single-service containers to cohesive local stacks, registries, and orchestrators, utilizing Docker Compose and Kubernetes patterns that can be used in both production environments and day-to-day iteration and development.

We'll also cover modern observability with OpenTelemetry, sidecars, and .NET Aspire. We go far beyond "enabling logging." We'll show how to standardize traces, metrics, and logs with OpenTelemetry; compare Jaeger and Prometheus; introduce sidecars and meshes where beneficial (and skip them where not); and wire it all together with .NET Aspire so that developers can see the entire system while coding.

We will revisit security for distributed boundaries. Building on token-based security, we connect identity, scopes/claims, service-to-service trust, and zero-trust network posture, striking a balance between usability and defense-in-depth across services and gateways.

These additions don't replace the core patterns from the first edition; they refine them. You still need to choose between synchronous and asynchronous communication deliberately, partition data with intention, and design for failure from the outset. The difference is that you'll now do so with stronger primitives and better tooling.

## Who this book is for

This book is for .NET developers, senior developers, software architects, and DevOps engineers who want a practical, pattern-driven path to designing, building, and operating microservices. If you're expanding your architectural toolbox and want clear, working examples in modern .NET, you're in the right place. Prior experience with ASP.NET Core APIs and C# will enable you to maximize the benefits of the material.

## What this book covers

*Anchor 1, Introducing Microservices – the Big Picture*, lays out the motivation, benefits, and trade-offs of moving from a monolith to microservices in .NET, framing the healthcare management system we'll build throughout the book.

*Anchor 1, Applying Domain-Driven Design Principles*, introduces bounded contexts, aggregates, aggregate roots, entities, and value objects, showing how DDD guides service boundaries and data modeling for healthcare domains.

*Anchor 1, Synchronous Communication between Microservices*, demonstrates REST and gRPC for request/response interactions, discusses latency and coupling risks, and implements resilient HTTP/gRPC clients in .NET for real-time scenarios (for example, cross-service lookups during appointment booking).

*Anchor 1, Asynchronous Communication between Microservices*, implements message-driven patterns with RabbitMQ/Azure Service Bus, Pub/Sub, and idempotent consumers to decouple services (for example, emitting events for emails and calendar entries after a booking).

*Anchor 1, Working with the Aggregator Pattern*, shows how to compose data from multiple services behind a single façade that feeds the UI efficiently while reducing chatty frontends and cross-service knowledge.

*Anchor 1, Working with the CQRS Pattern*, discusses separating reads from writes with MediatR and query models, enabling scale-appropriate stores and optimized read models in a clinical records workflow.

*Anchor 1, Applying Event-Sourcing Patterns*, talks about how to apply autonomy at the persistence layer, discussing patterns for cross-service relationships and consistency when each service owns its database.

*Anchor 1, Database Design Strategies for Microservices*, expands on schema design, replication, and query-side modeling across microservices, preparing the ground for reliable reporting and analytics in healthcare.

*Anchor 1, Implementing Transactions across Microservices Using the Saga Pattern*, is about coordinating long-running workflows (such as multi-step patient registration) using choreography/orchestration to maintain consistency without distributed transactions.

*Anchor 1, Building Resilient Microservices*, discusses implementing retries with backoff, circuit breakers, timeouts, and caching with Polly and related patterns so inter-service calls fail fast, shed load, and recover gracefully.

*Anchor 1, Implementing the API and BFF Gateway Patterns*, shows how to build an API gateway and backends-for-frontends to tailor endpoints per client (web/mobile), enforce cross-cutting policies, and simplify client integration.

*Anchor 1, Micro Frontends — Extending Microservices to the Frontend*, looks at decomposing the UI using micro-frontend techniques, allowing independent teams to ship features that compose cleanly within the healthcare portal.

*Anchor 1, Securing Microservices*, explores securing service-to-service and user-to-service calls with OAuth 2/OpenID Connect, scopes, and gateway enforcement to protect patient data.

*Anchor 1, Container Hosting and Development Patterns*, covers container-first development with Docker, local orchestration, and deployment patterns, as well as preparing services for modern platforms and dev-inner-loop workflows.

*Anchor 1, Serverless Microservices Development*, implements an appointment-booking workflow using Azure Functions, queues, and durable orchestrations, covering local development, testing, and operations.

*Anchor 1, Observability and Monitoring with Modern Tools*, discusses unifying logs, metrics, and traces using OpenTelemetry, Seq, and Jaeger, as well as introducing sidecars (Dapr) and .NET Aspire for end-to-end visibility and diagnostics.

*Anchor 1, Wrapping It All Up*, recaps the architecture and patterns, ties them back to real-world .NET microservices delivery, and outlines next steps for scaling and hardening a healthcare-grade platform.

## To get the most out of this book

You'll benefit from working knowledge of ASP.NET Core API development and basic data modeling. The chapters are hands-on and follow on from each other; running the samples and experimenting locally will accelerate your learning.

| Software/hardware covered in the book | Operating system requirements |
|---------------------------------------|-------------------------------|
| ASP.NET Core 10                       | Windows, macOS, or Linux      |
| C# 14                                 | Windows, macOS, or Linux      |
| .NET CLI                              | Windows, macOS, or Linux      |
| Visual Studio 2026                    | Windows                       |
| Visual Studio Code                    | Windows, macOS, or Linux      |
| Docker                                | Windows, macOS, or Linux      |
| Kubernetes                            | Windows, macOS, or Linux      |
| Microsoft Azure                       | N/A                           |

Most examples are shown using .NET CLI commands and Visual Studio Code. These commands can be used on any operating system and with alternative IDEs, such as Visual Studio/Visual Studio Code.

## Download the example code files

The code bundle for the book is hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition>. We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing>. Check them out!

## Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: <https://packt.link/gbp/9781837027415>.

## Conventions used

There are a number of text conventions used throughout this book.

**CodeInText**: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: “The **BaseEntity** class will take a generic type parameter, allowing us to set the **Id** type as needed.”

A block of code is set as follows:

```
public abstract class BaseEntity<TId>
{
    public TId Id { get; set; }
}
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
public record TimeSlot(DateTime Start, DateTime End)
{
    public TimeSlot
    {
        if (End <= Start)
        {
            throw new ArgumentException("End time must be
                after start time.");
        }
    }
}
```

Any command-line input or output is written as follows:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672
rabbitmq:4.0-management
```

**Bold:** Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: “To install the REST Client Extension, open VS Code and navigate to the **Extensions** view by clicking the square icon in the sidebar.”



Warnings or important notes appear like this.



Tips and tricks appear like this.

## Get in touch

Feedback from our readers is always welcome.

**General feedback:** If you have questions about any aspect of this book or have any general feedback, please email us at [customer@packt.com](mailto:customer@packt.com) and mention the book’s title in the subject of your message.

**Errata:** Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

**Piracy:** If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at [copyright@packt.com](mailto:copyright@packt.com) with a link to the material.

**If you are interested in becoming an author:** If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

## Share your thoughts

Once you've read *Microservices Design Patterns in .NET — Second Edition*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.



## Free Benefits with Your Book

This book comes with free benefits to support your learning. Activate them now for instant access (see the “*How to Unlock*” section for instructions).

Here’s a quick overview of what you can instantly unlock with your purchase:

### PDF and ePub Copies



### Free PDF and ePub versions



Access a DRM-free PDF copy of this book to read anywhere, on any device.



Use a DRM-free ePub version with your favorite e-reader.

### Next-Gen Web-Based Reader



### Next-Gen Reader



**Multi-device progress sync:** Pick up where you left off, on any device.



**Highlighting and notetaking:** Capture ideas and turn reading into lasting knowledge.



**Bookmarking:** Save and revisit key sections whenever you need them.



**Dark mode:** Reduce eye strain by switching to dark or sepia themes.

## How to Unlock

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

***Note:** Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW





---

# Part 1

---

## Understanding Microservices and Design Patterns

This part establishes the foundations for designing domain-aligned .NET microservices and the core interaction/data patterns you'll reuse throughout the book. You'll identify bounded contexts, choose between synchronous and asynchronous communication, and introduce composition and read/write segregation patterns that keep services cohesive and evolvable.

This part has the following chapters:

- *Anchor 2, Introducing Microservices – the Big Picture*
- *Anchor 2, Applying Domain-Driven Design Principles*
- *Anchor 2, Synchronous Communication between Microservices*
- *Anchor 2, Asynchronous Communication between Microservices*
- *Anchor 2, Working with the Aggregator Pattern*
- *Anchor 2, Working with the CQRS Pattern*
- *Anchor 2, Applying Event-Sourcing Patterns*



# 1

## Introducing Microservices – the Big Picture

Microservices are featured in every avenue of software development. This software development style has been touted to increase development speed and efficiency and improve software scalability and delivery. Microservices would have you assess that monolith, break it into smaller and more perceivable chunks or services, and develop and maintain the code base per service as independent units.



---

*“Microservices are small, autonomous services that work together”*

---

– Sam Newman, *Thoughtworks*

Traditionally, applications have been developed as a single unit or monolith. All the components are tightly coupled, and a change to one component threatens to have rippling effects on the code base and functionality. This makes long-term maintenance a major concern and can hinder developers from rolling out updates quickly.

This development technique is not unique to any stack and has become extremely popular in enterprise application development. While microservices are seen as a pattern, several sub-patterns are employed to ensure the code base is as effective as possible.

Microservices are typically developed as APIs via **HTTP (REST APIs)** or a **remote procedure call (gRPC)** that may or may not interact with each other. These services are then generally unified as a single application that allows users to complete operations without knowing all the underlying and moving parts.

The general notion is that each microservice is autonomous, has a limited scope, and aids in a collectively loosely coupled application. There are several industry-level success stories where microservices architecture is implemented. A popular application that boasts this approach is *Netflix*. This application started as a monolithic DVD rental system, and the company later embraced microservices to support its streaming services. This architecture allows Netflix to deploy thousands of services independently, enhancing scalability and resilience and enabling rapid feature development.

Other popular applications include *Uber*, *Amazon*, and *Etsy*.

This book is designed to cover the intricate details of various implementation styles and design patterns used in microservices. We will focus on implementing them using the .NET development stack, and you will learn how code can be written and deployed. You will learn about design and coding patterns, third-party tools and environments, and best practices for handling different scenarios and cross-cutting concerns that arise in microservices application development.

In this chapter, we will review the following:

- Traditional development approaches
- Building microservices
- Why design patterns are required
- Why .NET is a suitable framework for microservices development

Let us begin by exploring monoliths and why they are needed or need to be replaced in some scenarios.

### Free Benefits with Your Book

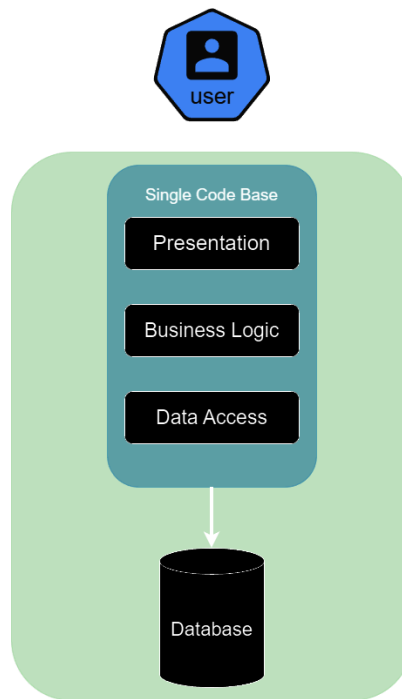
Your purchase includes a free PDF copy of this book along with other exclusive benefits. Check the *Free Benefits with Your Book* section in the Preface to unlock them instantly and maximize your learning experience.

## Approaches to software development

Before exploring microservices, it's essential to understand the architecture they evolved to replace: the monolith. A monolithic application, or "monolith," is a single, unified software system in which all components and functionalities are interwoven.

A monolith in .NET and C# encompasses a single application's **user interface (UI)**, business logic, and data access layers. *Figure 1.1* is a depiction of this structure, which typically includes several layers, as follows:

- **Presentation layer:** This is the part of the application users interact with. In .NET, it is often implemented as an ASP.NET MVC or Razor Pages application, potentially combined with a JavaScript client-side framework such as Vue or React. This layer handles HTTP requests, UI rendering, and user interaction.
- **Business logic layer:** This layer contains the application's core functionalities and rules. It is where business-related decisions, calculations, and workflows are managed. It acts as the application's brain, executing logic based on user inputs or data requirements.
- **Data access layer:** This layer handles communication with the database. It enables **create, read, update, and delete (CRUD)** operations and supports complex querying and transaction management. In .NET, this is often implemented using Entity Framework Core or ADO.NET.
- **Database:** Typically, monolithic applications use a single relational database, such as SQL Server, to persist data across various functional areas of the application. All modules access this central database, sharing schemas and structures.



*Figure 1.1 – A typical monolithic application with all application layers and a database*



By default, every ASP.NET project starts as an all-in-one project. As the code base grows, it is common to separate the different layers into independent projects. This helps enforce the separation of concerns as our project grows. This is a common design pattern called *n-tier architecture*, and it promotes the separation of concerns by organizing an application into logical layers, or “tiers,” where each tier is responsible for a specific aspect of the application’s functionality. The most typical implementation is a three-tier architecture, as seen in *Figure 1.1*. By adhering to the n-tier architecture in .NET, developers achieve better **modularity**, **maintainability**, and **testability** and form a strong foundation for building scalable and enterprise-grade .NET applications.

Since all functionalities are part of the same code base and runtime environment, components of a monolithic application are tightly interwoven. Monoliths are developed and deployed as a single unit, so any modification, no matter how minor, typically requires extensive regression testing and redeployment. This becomes increasingly burdensome as the code base grows.

As the application grows in scale and complexity, however, the monolithic structure presents some serious challenges:

- **Scaling difficulties:** Scaling requires duplicating the entire application rather than scaling individual components. If only one module needs more resources, the whole application must be scaled, leading to inefficiencies.
- **Limited technology flexibility:** Since all components share a single stack, developers need more flexibility in choosing technologies. For instance, if the data processing layer would benefit from a more specialized technology, it might be more challenging to introduce it without impacting other layers.
- **Deployment bottlenecks:** As mentioned previously, even minor updates require redeploying the entire application. This often results in bottlenecks where a minor feature or bug fix can lead to delays, as it requires complete integration testing and downtime for deployment.
- **Higher risk of code debt:** As the application grows, the code base can become unwieldy and difficult to manage, especially if not modularized properly. Over time, dependencies can pile up, creating a **spaghetti code** situation where it becomes difficult to determine how various parts interact and change behavior.

A modern development pattern, **modular monolith**, has emerged to combat many of these concerns. This method seeks to segment the parts of the application into logical segments (often based on modules). We will review this next.

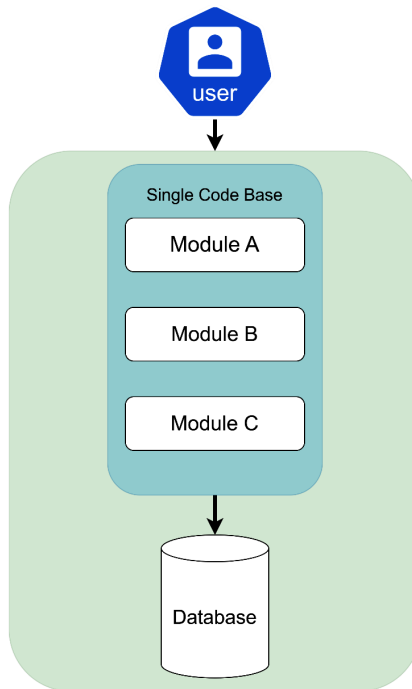
## Modular monolith development pattern

The modular monolith development pattern seeks to address the most common issues in a traditional monolith by introducing a structured approach to monolithic architecture. They divide the application's code base into distinct, self-contained modules, each responsible for a specific business domain or functionality.

Some of the tangible and immediate benefits that this approach introduces are as follows:

- **Separation of concerns:** Modular monoliths divide the application's code into modules based on the different business domains or implemented functionalities. This segregation improves code organization and readability.
- **Independent development and testing:** Development teams can work on different application parts concurrently without interfering with each other. Modules can now be developed and maintained independently.

This means the application might end up with several micro frontends or components, which interface with specific business logic and data access layers, as implemented by the module.



*Figure 1.2 – A modular monolith application with modules encapsulating specific business and data access logic and a database*

Figure 1.2 shows the logical separations that this pattern introduces as it offers a balanced approach, combining the simplicity and ease of deployment of monolithic applications with the structural benefits of modularity. However, we may still have scenarios requiring independent deployment, diverse technologies, or extensive scalability. The entire application must still be redeployed when changes are introduced, thus retaining the deployment bottleneck associated with monoliths. There is also not much more flexibility where diverse technologies are needed since we are still using one primary technology stack. Finally, we cannot individually scale any module where necessary. The entire application must be scaled.

At best, the modular monolith approach is a practical middle ground for applications that may eventually transition to a microservices architecture. Let us now review microservices development and some tangible benefits we can realize.

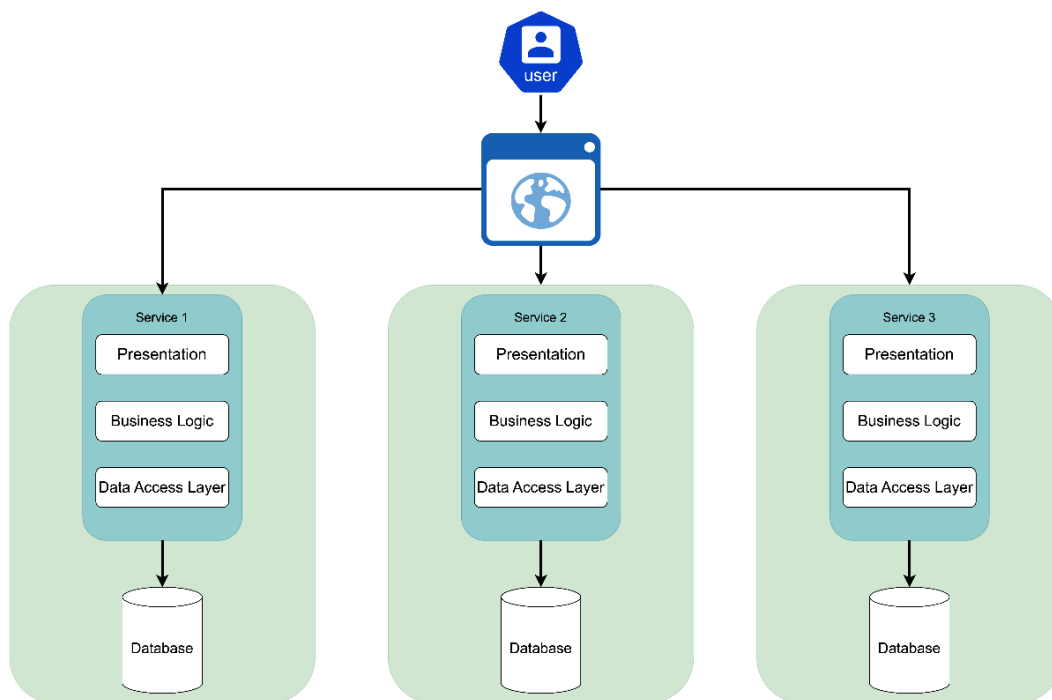
## Microservices – why or why not?

Microservices application development introduces an architectural style, breaking an application into small, independent services that communicate through well-defined APIs. Each service is designed based on specific business capabilities and is autonomous, leading to independent development, deployment, and scaling. This approach directly contrasts with monolithic architectures, where all components are interwoven into a tightly coupled unit. Some of the immediate benefits of this approach include the following:

- **Independence:** Each microservice is developed for a specific business need, promoting a precise alignment between technical implementations and business objectives. This independence also allows for independent development, testing, deployment, and scaling.
- **Decentralized data management:** Microservices often use their own database and, by extension, database technology. This allows the service to be developed using the most appropriate data storage solution.
- **Technology diversity:** Teams can select each service's most suitable technology stack. This code base diversity will have little to no impact on the entire application since the code bases will be independently implemented. This aids in fostering innovation and efficiency.

Each microservice, though potentially structured as a monolith internally, serves a distinct purpose within the application's ecosystem. These services, which may or may not include a dedicated UI, encapsulate the necessary business and data access logic for their specific functions. This contrasts sharply with monolithic architectures, where users interact through a single, unified presentation layer, integrating all application functionalities.

In contrast, a microservices architecture decouples these functionalities into independent services, each potentially with a presentation layer. We will review different techniques and patterns to manage this significant change, but it must be mentioned. *Figure 1.3* shows a high-level implementation of a microservices application where a user might access a single UI to access several underlying and independent services.



*Figure 1.3 – How a single UI may communicate with several microservices*

Now that we have a high-level understanding of the monolith and microservice development approaches, let's compare how they might help or hinder the development of an application.

## Microservice versus monolith – scoping an application

Scoping an application is a critical step in software development. It helps us define the project's boundaries, objectives, and deliverables. A well-defined scope and precise requirements ensure that all stakeholders understand what the application will achieve, its features, and the resources required. This clarity helps prevent scope creep, where unplanned features or changes are introduced, leading to potential delays and budget overruns.

Imagine building a health facility management web application to manage customer information, book appointments, generate invoices, and deliver patient test results. Some of the essential requirements are as follows:

- Customers need a user profile with demographic information and essential documents
- Customers must book an appointment with a particular doctor at a specific time based on the available timeslots in a doctor's schedule
- Once an appointment is made, the doctor's calendar slot will change from *available* to *booked*
- After a visit, an invoice is generated and sent to the customer
- The customer can pay for the visit online or in the office
- All this information will be stored for future reference
- Internal users and customers need a UI to complete tasks

These represent the core requirements of the system that will lay the foundation for the implementations and patterns we will explore in this book.

Developing a health facility management application requires careful scoping to ensure that all functional requirements are met efficiently. We need to clearly define the functional and non-functional requirements and different domains that must be addressed during the implementation. The identifiable modules are as follows:

- **User profile management:** Patients can create and manage profiles, including demographic details and essential documents
- **Appointment scheduling:** This feature enables patients to book appointments with specific doctors, view available time slots, and receive confirmations
- **Doctor's calendar management:** This service allows doctors to manage their schedules, reflecting real-time availability and booked appointments
- **Billing and invoicing:** After the visit, invoices are generated, options for online or in-office payments are offered, and records are maintained for future reference
- **UI for users:** This interface offers administrative staff and patients tools to manage appointments, patient information, billing, and other operational tasks
- **Data storage and retrieval:** This ensures that all information is securely stored and easily retrievable, supporting operational needs and compliance requirements

Now that we have identified the key modules that need to be developed, we can decide whether a monolithic or microservices architecture aligns best with the application's requirements. In scoping the architectural approach when creating this application, several factors must be considered:

- **Application complexity and size:** A monolithic architecture may be appropriate for small to medium-sized applications in small to medium-sized companies. However, if the project's scope is prone to expansion and we need to ensure scalability, a microservices architecture can offer better manageability and scalability.
- **Team structure and expertise:** A more minor development team may not benefit from a microservices approach. The required expertise may be lacking, and there is more room for direct collaboration and communication. In contrast, a larger team might boast more experience in handling distributed systems and may benefit from the autonomy and flexibility microservices provide.
- **Scalability requirements:** Cloud platforms offer the ability to scale in and out automatically, and a microservices application can take advantage of cloud scalability. Also, it is good to mention that as serverless containers become more popular in cloud systems, microservices would be an ideal choice.
- **Deployment and maintenance:** Microservices allow for independent service deployment, ensuring that each deployment does not affect other components or services, reducing the need for extensive testing. This can lead to faster updates and easier maintenance. Monolithic applications, while more straightforward to deploy initially, can become cumbersome to update and maintain as they grow.
- **Technology diversity:** Microservices offer the flexibility to employ diverse technologies and programming languages across different services, empowering teams to select the optimal tools for each task. For instance, the user profile management module might excel with a relational database such as SQL Server, leveraging its robust .NET adapters. Conversely, the payment module could benefit from a non-relational data store – for example, a document database such as MongoDB – necessitating a different stack, such as Node.js. This starkly contrasts with monolithic architectures, which typically constrain development to a single technology stack.
- **Organizational readiness:** Implementing a microservices architecture requires a cultural shift, often toward *Agile* and *DevOps* practices. Monitoring, deployment, and testing will need more robust approaches for successful implementation. Assessing the readiness to adopt these practices before choosing microservices is essential.

This could be implemented as a monolith, where one frontend can communicate with one backend, database, and deployment environment. This is a standard approach to a project, and we have been using this approach successfully on larger projects for years. But, as mentioned earlier, we have always faced challenges, and they become increasingly apparent as technology evolves and system and business needs change.

If we conceptualize how the application could be architected using microservices, we can scope standalone services for each identified module or domain. Using domain-driven design, which we will review in detail in the next chapter, we then scope out dependencies and determine where we might need to duplicate certain entities. For instance, *Customers* need representation in the *Booking and Appointments* database and *Payments*. This duplication is required if we use *separate databases per service* (which is strongly encouraged).

As we dive into the requirements and recommended approaches for developing using microservices, we will see that another level of complication is introduced when scoping the flow of operations that involve multiple services to play a part. For instance, when making a booking, we need to do the following:

- Retrieve the customer making the booking
- Ensure that the preferred time slot is available
- If available, generate the invoice
- Collect the payment
- Confirm the appointment

In a monolith, getting data from multiple tables is straightforward. You can write all the queries in one place and send the combined data to the user. With microservices, it's more complex. You need multiple services to talk to each other, which requires careful coordination.

Because of the different cross-cutting concerns surrounding microservices development, we must know the various design patterns and approaches to implementing our code and infrastructure. While we abstract seemingly complex operations and workflows into smaller and more perceivable chunks, we end up in the same position where the application must carry out a specific operation. We will still need to deal with the original requirements.

## Choosing the microservices approach

Before settling on a microservices-based application development approach, asking yourself, “Why?” and “Do I need it?” is always essential. Let us review some of the pros and cons of using

microservices. By choosing this approach, however, we can be sure of the following tangible benefits and challenges:

- **Individual service scalability:** It is easier to scale individual parts of the application and address specific performance gaps as they arise. If, for instance, a vaccine becomes widely available and customers are encouraged to book an appointment online, we will surely experience a large load during the first few weeks. Our customer microservice might not be affected, but we must scale our booking, appointments, and payment services.

We can scale vertically (scale up) and allocate more resources (CPU and RAM) to the hosting environment when the load increases, or we can scale horizontally (scale out) by spawning more service instances to be load balanced. Using the right hosting platforms and infrastructure allows us to automate this process.

- **Availability:** Availability means the probability of a system being operational at a given time. This metric goes hand in hand with the ability to scale and address the needed reliability of the underlying infrastructure and deployment method. Microservices will be deployed individually, removing, as much as possible, a single point of failure. A single point of failure is a part of a system that, if it fails, will stop the entire system from working.

While microservices help us to reduce single failure points, there are situations where services depend on each other, so one service being down can result in different services being unable to complete their operations. There is also the *gateway pattern*, which will aggregate all services behind one entry point. This gateway must always be online for our distributed services to remain available.

- **Development speed:** Given that the application has been broken into domains, developers can focus on ensuring that their features are being developed efficiently. This also contributes to how quickly features can be added, tested, and deployed.

It will now become a practical approach to have one team per domain, and it will be easier to scope the requirements for a domain and focus on fewer functional requirements for a piece of work. Each team can now independently own the service from development to deployment.

Of course, we have seen that services will still need to communicate, so we will still have to orchestrate the integration between the teams. While each team is independent, they will still need to make their code and documentation available and easy enough to access. Version control is also vital since the services will be updated over time, but this must be managed.



- **Efficient data storage:** While there are scenarios where you may use one database for multiple microservices, this is generally discouraged, and a *database-per-service* approach is preferred. Services must be loosely coupled to be developed, deployed, and scaled independently. This is best accomplished if each service has its own data storage.

It makes even more sense when you consider that the type of data being stored might influence the type of data storage used. One service might rely on transactions and need a relational database, while another might need an aggregated read-only store. Changes to one service's database do not affect any other services. This approach has challenges, where data must be synchronized across the services. Now, we need to consider *distributed transactions* across several unrelated data stores.

- **Monitoring:** Monitoring is one of the most important aspects of a distributed system. It allows us to ensure uptime and mitigate against failures proactively. We need to be able to view the health of our service instances.

Logging is also a very crucial tool for monitoring and troubleshooting. Each microservice typically writes logs containing information about errors, warnings, information, and debug messages. We must think about how to centralize logs and performance metrics, sparing us the task of going to each service's environment manually. Cloud providers such as **Azure**, **AWS**, and **Google Cloud Platform (GCP)** offer robust *distributed tracing tools* to help developers observe and diagnose issues in microservice-based applications. These tools are essential for gaining end-to-end visibility across asynchronous and synchronous interactions within a distributed system. These platforms often support **OpenTelemetry**, an open standard for telemetry data collection, making it easier to integrate tracing across heterogeneous systems and third-party platforms. By leveraging these tools, developers can analyze request flows, identify performance issues, and improve system reliability in production environments.

We must also consider *health checks* that tell us whether a microservice instance is failing. With health checks, we can see a quick snapshot of the service's health and have that data point returned to the dashboard.

- **Deployment:** Each microservice needs to be independently deployable and scalable. This includes all the security, data storage, and additional assets our services use. Ideally, each service will have its own physical hosting space. A virtualized infrastructure might be used where multiple physical servers are impossible, with appropriate resource allocations per service. The idea is that each microservice instance is isolated from the other and will not compete for resources.

Each microservice should be small and succinct enough to be packaged as a **container image** and later hosted in a **container**. A container encapsulates the details of the technology used to build a service and provides all CPU, memory, and microservice dependencies needed for operation. This makes it easy for the microservice to move between testing and production environments and ensures environment consistency.

**Docker** is the go-to container management system and works hand in hand with container orchestration services. These services can help start the correct containers at the proper time, handle storage considerations, and address potential container failures. These tasks are not practical to handle manually, so we enlist the services of Kubernetes, Docker Swarm, and Marathon to automate them.

Beyond containers, **serverless architectures** – enabled by platforms such as **AWS Lambda**, **Azure Functions**, and **Google Cloud Functions** – are gaining significant momentum in modern cloud deployments. While containers provide a powerful abstraction for packaging and deploying individual services with complete control over runtime environments, **serverless computing** focuses entirely on code execution without the need to manage infrastructure. This architectural style is especially appealing for lightweight microservices, background tasks, and real-time processing scenarios.

The more we explore the microservices development approach, the more we see where it addresses certain things while introducing a few more concerns. The microservices approach is not a silver bullet that fixes all your architectural challenges, and it is crucial to appreciate the complexities that it introduces. These concerns and complexities are generally addressed using design patterns, which can save time and energy. We will review the design patterns at a high level next.

## Microservices design patterns

Designing robust and efficient microservices architectures requires addressing common challenges such as code organization, inter-service communication, data consistency, and fault tolerance. Understanding and employing established design patterns that offer proven solutions and best practices to navigate these concerns is in our best interest. This book will explore the most common problems and examine the design pattern concepts that help us address these concerns:

- **Coding and integration patterns:** Familiarity with standard development concepts is crucial to adopting and implementing more complex development patterns. We will review patterns that can be implemented in a monolith setting but can bring unique advantages to each microservice and how the code base might be laid out based on the requirements.

We will also review communication and integration patterns that ensure consistency in data flow between the services. These patterns involve synchronous and asynchronous communication, and we will determine when either approach is best.

- **Data management:** We will also look at patterns that help manage data across our distributed services. It seems easy to give each service its own database until we ensure data consistency across the different data stores. Specific patterns, such as the **saga pattern**, are pivotal to maintaining confidence in what we see after each operation.
- **Resiliency, security, and infrastructure patterns:** These patterns seek to calm and comfort a brewing storm. With all the moving parts we have identified, it is vital to ensure that as many things as possible are automated and consistent in the deployment. We also want to balance *security* between system needs and a good user experience. These patterns help us ensure that our systems always perform at peak efficiency.
- **Cloud development patterns:** These patterns will cover infrastructure best practices to ensure our services can be hosted in cloud environments most efficiently and cost-effectively. We will focus on providing portability and reliability in these more complex environments.

Now that we know the application and its requirements and have decided to move forward with a microservices architecture approach, let's review why we will use .NET as the primary technology stack when developing the different components.

## Microservices and .NET Core

This book addresses implementing microservices and design patterns using .NET Core. We have mentioned that this architectural style is platform-agnostic and has been implemented using several frameworks. However, .NET Core simplifies microservices development and offers many benefits, including cloud integration, rapid growth, and cross-platform support.

.NET is a versatile, open source framework developed by Microsoft, designed for building modern, high-performance applications across various platforms, including Windows, macOS, and Linux. Its cross-platform capabilities, modular architecture, and robust performance make it an excellent choice for developing microservices. Some of the tangible benefits are as follows:

- **Cross-platform compatibility:** .NET supports multiple operating systems and can be deployed in virtually any environment. This allows for extreme flexibility and reduced infrastructure constraints.
- **High performance:** .NET is designed for speed and efficiency, thus allowing us to deliver high-performance applications. This performance benefit is crucial for microservices that must be highly performant and handle large requests.

- **Modular and lightweight:** .NET is designed to be modular, allowing us to include only the necessary components and packages needed to implement a service's functionality. This results in lightweight, maintainable, and portable microservices.
- **Containerization support:** .NET is compatible with containerization technologies such as Docker, simplifying microservices' deployment and orchestration and enhancing portability and consistency across development and production environments.
- **Comprehensive tooling and support:** The .NET ecosystem boasts a rich set of tools, documentation, and support from a vibrant and active community of developers. Microsoft provides tools, such as Visual Studio (Windows only) and Visual Studio Code (any OS), to provide powerful development environments for developers. There are capable alternative tools on the market, such as Rider (JetBrains).

Now that we have seen the main reasons why we will use .NET for our development tasks, let us review the different versions of .NET that can be used and will be used as we advance.

## .NET version

If you are already familiar with .NET development technologies, this section may not be beneficial. Still, it is good for us to be on the same page regarding versions that can be used. .NET has evolved significantly since its inception, introducing new features, performance enhancements, and extended platform support with each release. Microsoft follows a structured release cycle for their .NET versions, providing developers with predictable updates, support, and improvements. A new major version of .NET is released each November, delivering significant features, performance improvements, and updates. Each release is categorized as either **long-term support (LTS)** or **standard-term support (STS)**:

- **LTS releases:** Supported for a minimum of three years, LTS versions are ideal for applications requiring extended stability and support. *.NET 10.0 (LTS)* (<https://dotnet.microsoft.com/en-us/download/dotnet/10.0>) is the *latest Long-Term Support* release at the time of writing, offering extended support until November 2028, ensuring stability and access to the latest features and improvements.
- **STS releases:** Supported for 18 months, STS versions offer the latest features and improvements, suitable for applications that can adopt new technologies more rapidly. *.NET 9 (STS)* (<https://dotnet.microsoft.com/en-us/download/dotnet/9.0>), which was released on November 12, 2024, introduced significant enhancements aimed at improving developers' performance, productivity, and security. It is set to go out of support in May 2026.

Our code examples will be completed using .NET 10 and C# 14, to make use of the latest features where possible. Now that we have established the .NET versions, let's review our development tool options.

## Development tools

**Visual Studio (VS)** and **Visual Studio Code (VS Code)** are robust development environments from Microsoft. Each offers unique features tailored to different development needs. Both IDEs have free versions for educational, hobbyist, and open source development.

VS (<https://visualstudio.microsoft.com/vs/community/>) is a comprehensive **integrated development environment (IDE)** designed for large-scale, enterprise-level applications. It boasts a comprehensive UI that simplifies creating a wide array of templates for various .NET applications. VS facilitates quick project setup and access to powerful built-in debugging, profiling, and testing tools, streamlining the development process.

While Visual Studio is powerful and the flagship tool for .NET development, it presents some challenges. It is not cross-platform and requires significant system resources, making it less ideal for less powerful machines.

VS Code (<https://code.visualstudio.com/>) is a lightweight and fast alternative to VS. It is an extensible code editor suitable for various development tasks. It can run on Windows, macOS, and Linux, offering flexibility across different operating systems. It also has a vast collection of extensions, allowing you to customize the environment to fit specific development needs. The most formidable extension, within this context, is the C# Dev Kit. The C# Dev Kit transforms VS Code into a robust alternative to VS for .NET development. This extension introduces several features that bridge the gap between the lightweight nature of VS Code and the comprehensive capabilities of VS.

For .NET and microservices development, the choice between VS and VS Code depends on specific project requirements, operating system constraints, and the need for third-party integrations. We will alternate between these tools in particular parts of this book, but note that everything that can be accomplished in one IDE can also be done in the other.

Now that we know which IDEs are best suited for our .NET development, let's review the languages and libraries highlighted throughout the book.

## Languages, frameworks, and libraries

.NET is a unifying environment where one application can be developed using several frameworks, libraries, and languages. It supports various programming languages, enabling developers to choose the most suitable language for their application needs. These languages can be broadly categorized into those officially supported by Microsoft and additional languages supported by the broader community.

The officially supported languages are as follows:

- **C#:** The most popularly used .NET-supported language is a versatile, object-oriented language widely used for developing desktop, web, and mobile applications
- **F#:** A functional-first language supporting object-oriented and imperative programming paradigms, ideal for complex data processing tasks
- **Visual Basic .NET (VB.NET):** A relatively simple programming language suitable for building type-safe, object-oriented applications

Beyond the officially supported languages, the .NET ecosystem accommodates other languages, such as C++, Python, and PowerShell.

Thanks to the **Common Language Runtime (CLR)**, the execution engine of .NET, these languages can integrate with and reference each other. The CLR enables the integration of multiple programming languages within a single application. Developers can select the language that best aligns with their project's requirements, leveraging each of their strengths while benefiting from the shared features and libraries provided by the .NET platform.

Beyond languages, there are several frameworks embedded in the .NET ecosystem. These frameworks allow us to create different web, mobile, and desktop application project types. For instance, if we need to create an API (which we will need to do a lot as we advance), then we can use the ASP.NET Core API framework to spin up such a project. Similarly, if we need a Windows service to process background tasks, we have a Windows Service framework accordingly. While there is support for most languages in each framework, we will focus on using C# as the primary language.

Finally, we will focus on several popular and not-so-popular libraries that help with everyday development tasks. For instance, **Entity Framework Core (EF Core)** is an object-database mapper for .NET. It enables developers to work with a database using .NET objects, streamlining data access in microservices. Another one is **Polly**, which is used to implement resilience and transient-fault handling capabilities and enhance the robustness of microservices. These are just two examples, but many more will come in later chapters.

Given its versatility and extensive support, .NET is a fantastic candidate for developing robust and modern application-based microservices architectural principles. Now that we have reviewed the benefits of using .NET, let us summarize everything we have learned.

## Summary

We started by reviewing the traditional monolithic approach to developing an application. We established that this approach has worked for years and is not inherently incorrect. We also acknowledged its shortfalls and explained why there might be better ways to build a project.

We then explored how microservices will change an application's implementation and development style. The idea here is to make the code and components as loosely coupled as possible, making it easier to manage the different parts independently and increasing the ability to manage various parts of the application without affecting others. While this approach has its benefits, it does introduce complexities where the monolith does not and requires more expertise in implementations.

To properly implement a microservices architecture-based application, we must appreciate the cross-cutting concerns that must be addressed and the different approaches, or design patterns, that help us to do so effectively. Factors such as service communication, data consistency, security, and deployment are all important and must be addressed adequately during development.

Finally, we reviewed .NET as a development environment. We explored the versions, tools, and languages that are supported and recommended. .NET offers a stable and robust ecosystem with support for various application types and languages.

By now, I hope you understand microservices, why you may or may not be using this architectural style, and the importance of using design patterns. In each of the chapters of this book, we will explore the use of design patterns to develop an enterprise-ready and reliable system based on microservices, using .NET and various supporting technologies.

We will remain realistic, explore the pros and cons of our design decisions, and examine how various technologies help us connect the dots.

The next chapter will explore domain-driven design, a helpful development pattern for scoping our applications, and the resulting microservices.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW







# 2

## Applying Domain-Driven Design Principles

In the previous chapter, we looked at some key elements that comprise microservices. In this chapter, we will focus on more practical applications of these concepts, starting with **Domain-Driven Design (DDD)**. Implementing DDD helps us to scope an application built on microservices design principles. DDD is a software development approach introduced by Eric Evans in his seminal 2003 book, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. It emphasizes the importance of aligning software design with the core business domain. It advocates for close collaboration between technical teams and domain experts to create models that accurately reflect complex business processes and rules.

By applying DDD concepts, software teams can avoid developing systems that are misaligned with business objectives, harder to maintain, and less adaptable to change. Without DDD's emphasis on modeling software closely to the business domain, applications may fail to represent business processes accurately, leading to solutions that do not fully address user requirements. Also, teams that need a structured approach to encapsulating business logic may find adapting the software to evolving business requirements challenging, resulting in brittle systems that resist change.

DDD is particularly beneficial in microservices development because it emphasizes aligning software architecture with business domains. This alignment facilitates the creation of cohesive, scalable, and maintainable services. By applying the concepts of DDD to microservices development, we can craft services that directly meet the business objectives, are easier to manage, and can adapt to changing requirements.

In this chapter, we'll delve into the principles of DDD, specifically tailored to the unique requirements of a healthcare management system for a medical establishment. We'll explore the purpose and application of DDD in structuring and organizing complex systems to model the problem domain accurately.

This chapter is structured around three key topics:

- The significance of DDD
- The differentiation between aggregates and entities
- The use of value objects

Let's begin by understanding the significance of DDD and what it brings to software design.

## Technical requirements

We will review some C# code to understand how domain entities and design patterns are implemented. For the code referenced in the chapter, refer to the GitHub repository: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter02>.

Ensure that you have at least one of the following software installed on your machine to be able to execute this code (use the links to download and install the software):

- Visual Studio 2022 (Community Edition for non-commercial development): <https://visualstudio.microsoft.com/vs/>
- Visual Studio Code: <https://code.visualstudio.com/download>

## Exploring DDD and its significance

DDD aims to tackle complex software problems by focusing on the core domain and business needs. The primary objective is to model software that accurately mirrors the problem domain, making it easier for developers to understand the application's intricacies and adapt to new requirements.

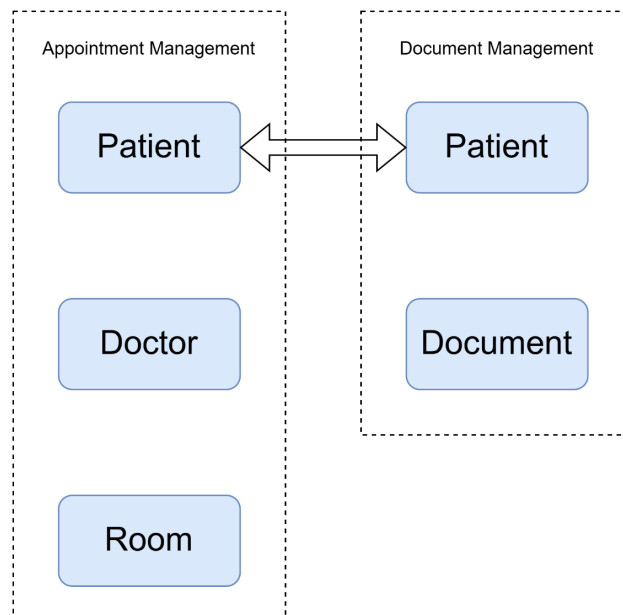
This software design approach encourages developers to assess processes and sub-processes and decipher all the atomic elements therein. Atomic means that one process might have many moving parts, and while they all combine to give one output, they have their own routines to carry out. Each sub-process can be seen as self-governing and attributed to a **domain**. This insight leads us to break up a monolith into independent microservices that do their own thing with their data. That is a domain.

Before we go further, let us take some time to explore some keywords and their definitions:

- **Entity:** An entity is a fundamental building block representing a distinct object within the domain, characterized by a unique identity and a life cycle. Entities are crucial for modeling concepts that require differentiation, even if their attributes are identical. Consider a Patient entity in a healthcare management system. Each patient has a unique identifier (e.g., patient ID) to distinguish them from others. Attributes such as name, address, and medical history may change, but the patient's identity remains constant.
- **Models:** Models are abstractions that define rules and behaviors of a domain and are used to solve domain problems. We organize information about the target domain into smaller pieces and call them models. A model is a central reference point in our design and development process and can then be grouped into logical modules and dealt with one at a time. A domain will generally contain too much information for just one model; for instance, while our healthcare management system needs to capture patient information, we do not need to know the patient's eye and hair colors. This might be an obvious example, but it could get far more complicated than that in a real scenario. Sifting through the relevant parts of domain knowledge will require close collaboration with domain experts, fellow designers, and developers.
- **Ubiquitous language:** This refers to language unique to a domain model and used by team members within activities related to the domain. We have already established that models for a domain need to be developed through collaboration with domain experts. Given the difference in skill sets and perceptions, there will be communication barriers. Developers tend to think and speak about concepts related to programming, such as inheritance and polymorphism. Unfortunately, domain experts don't usually know or care about that. Domain experts will use jargon and terms that developers will not understand. This gap in communication does not work well for any team.
- **Bounded context:** This defines boundaries in a system or subsystem that inform the work a particular team will carry out and the focal point of their efforts within their domain. A bounded context is a logical boundary of a domain where terms and rules apply. In defining a bounded context, it is vital to *intentionally mark its edges*, in both **code** and **communication**. This separation could be represented by distinct modules or microservices in an application or by clearly defined APIs or message contracts. This makes integration between contexts intentional and predictable.

This boundary's terms, definitions, and concepts constitute ubiquitous language. Establishing bounded contexts is a crucial step in DDD and is strategically employed in scoping large models within extensive teams. In DDD, we break down our larger models into bounded contexts and then define the existing relationships among them. Bounded context maps can be utilized in our healthcare management system to examine the relationships illustrated in the maps.

For instance, we have the context of document management and patient appointments. They both contain unrelated and related concepts. Documents exist solely in a document management system but may reference patients. Context mapping is a widely used strategy in DDD to represent the relationships between bounded contexts. *Figure 2.1* illustrates the relationship between two domains.



*Figure 2.1 – Appointment Management and Document Management domains, each with their own Patient entity, linked by a double arrow to show the shared concept of Patient.*

- **Value objects:** Value objects are fundamental domain elements defined solely by their attributes, not by identity. Unlike entities, they have **no unique identifier** and are considered equal if all their properties match. They are **immutable**, meaning their state cannot change after creation—any modification results in a new instance. Because of this, value objects are **interchangeable** when they hold the same values, making them ideal for representing concepts such as dates, money, or measurements.

- **Aggregates:** Aggregates are clusters of related entities and value objects treated as a single unit to maintain consistency and enforce business rules. Each aggregate has a **root entity** that is the only entry point for external access. All modifications go through the **aggregate root**, ensuring that domain **invariants** are upheld and changes remain **atomic** and **consistent**. This structure protects the integrity of the domain model and encapsulates internal complexity.

Let us step back from microservice design patterns and assess what it takes to build software. A domain is a category of business rules and operations. If your software is to be used in a bank, then the domain is banking; if it is used in a hospital, then the domain is healthcare. The domain is not the code or technology. It is the business logic, rules, and concepts that matter most to the people using the software. It could be banking, e-commerce, healthcare, education, and so on. Your goal as a developer is to understand this domain deeply and model it in code to reflect the business's thinking and workings.

So, the software you develop must complement the domain to solve the overall problem. The core concepts and elements of the domain must be present in the software's design and models. Healthcare applications typically involve complex data structures, relationships, and workflows. DDD's focus on **Domain-Specific Language (DSL)** helps bridge communication between developers and stakeholders, ensuring that all parties have a shared understanding.

For example, rather than having a generic Customer class, we can create specific representations, such as Patient or Doctor, each with unique properties and behaviors.

Now that we have a high-level understanding of DDD, let us review some of the benefits and drawbacks of introducing it to our application.

## The pros and cons of DDD

DDD requires a substantial upfront investment in understanding and modeling the domain in detail. It emphasizes focusing on smaller, individual domain components, which makes the resulting software more flexible. It breaks our application into smaller segments, facilitating easier modifications of parts and elements of the application with minimal side effects. The application's code is typically well organized and highly testable, with the business logic for the domain isolated within this code base. The principles of DDD can enhance your application implementation activities, even if you don't apply it from end to end for a project. DDD is particularly effective at breaking down complex business logic.

Some tangible benefits of using DDD include the following:

- **Enhanced communication:** Since DDD promotes a ubiquitous language, there tends to be more straightforward and consistent communication between team members (technical and non-technical), thus reducing misunderstandings. This common language bridges the gap between technical and business perspectives, ensuring that all parties have a mutual understanding of the domain concepts and terminologies. DDD encourages joint modeling sessions, such as event-storming workshops, where developers and domain experts collaboratively explore and define the domain. These sessions facilitate open dialogue, allowing for the clarification of requirements and the discovery of domain insights that might be overlooked.
- **Alignment with business objectives:** By focusing on the core domain, DDD ensures that the software design closely mirrors business processes, leading to solutions that effectively address user requirements. DDD emphasizes focusing on the most critical aspects of the business domain, allowing development efforts to concentrate on areas that provide the most value to the organization.
- **Improved flexibility and maintainability:** DDD's modular structure allows for more straightforward adaptation to changing business needs, resulting in more maintainable and scalable systems. By embedding business rules within domain entities and aggregates, DDD ensures that changes in business processes are localized. This encapsulation reduces the risk of unintended side effects when modifying code, enhancing maintainability.
- **Cleaner code base:** DDD encourages using best practices and design patterns, leading to a cleaner and more reliable code base. DDD divides complex systems into domain, application, and infrastructure layers, each with specific responsibilities. This separation enhances code readability and maintainability.

DDD is unsuitable for applications with simple requirements and business logic for creating, adding, updating, and deleting data. It is time-consuming and requires expert-level domain knowledge. Non-technical individuals will be needed and must be available throughout the project's duration.

Some of the downsides of DDD are as follows:

- **Resource intensity:** Implementing DDD requires significant time and effort to understand the domain, which can be resource-intensive. DDD requires a thorough exploration of the business domain, necessitating close collaboration between developers and domain experts to understand complex processes and rules. This in-depth analysis demands significant time and effort.

- **Complexity overhead:** DDD may introduce unnecessary complexity for simpler domains, making it less suitable for straightforward projects. DDD emphasizes detailed domain modeling, which can create numerous entities, value objects, and aggregates. Managing these components requires careful design and can increase the system's complexity, especially if the domain is not inherently complex. The use of repositories, factories, and services in DDD introduces additional layers of abstraction. While these patterns promote flexibility and testability, they can make the system harder to understand and navigate, especially for new team members. While bounded contexts and aggregates are essential patterns in DDD for managing complexity, they also introduce challenges. Bounded contexts help divide a large domain into smaller, well-defined areas where a particular domain model applies. This separation reduces ambiguity and allows teams to work independently, but it introduces integration complexity. Teams must define clear interfaces, such as APIs or messaging protocols, for communication across contexts, which can lead to duplication of similar models, coordination overhead, and additional effort in maintaining consistency. Similarly, aggregates enforce consistency and encapsulate business rules within well-defined boundaries. By ensuring all changes go through the aggregate root, aggregates protect the integrity of the domain. However, this rigor comes at a cost: designing the proper aggregate boundaries can be difficult. Overly large aggregates may degrade performance, while overly small ones can fragment domain logic. Aggregates also restrict access to internal entities, which may complicate implementation and limit flexibility. In essence, while bounded contexts and aggregates introduce structured complexity, they do so in service of building systems that are more robust, maintainable, and aligned with the real-world domain they represent.
- **Requires domain expertise:** Effective application of DDD necessitates deep domain knowledge, which may require collaboration with domain experts and additional training for the development team. This pattern necessitates deep domain expertise because it centers on modeling software that accurately reflects the complexities and nuances of the business domain. This approach requires close collaboration between developers and domain experts to ensure the software aligns with business processes and objectives.
- **Not ideal for every project:** In projects with minimal domain complexity or highly technical projects, DDD may not provide significant benefits and could lead to over-engineering. For projects where technical complexity outweighs domain complexity, the intricacies of DDD are often related to technical aspects such as performance optimization, system integration, or advanced algorithms rather than business processes. Applying DDD in these scenarios can lead to unnecessary complexity and may not provide significant benefits, as this approach is not tailored to address purely technical challenges.



In summary, while DDD offers substantial benefits in aligning software design with business needs and improving system maintainability, it also demands considerable resources. It may introduce unnecessary complexity in specific scenarios.

Now that we understand DDD at a high level, let's explore how the concepts it promotes relate to microservice design patterns.

## DDD and microservices

Applications designed using DDD principles are best implemented through microservices. By now, we can appreciate that microservices architecture promotes dividing a monolithic application into self-contained and independent services. So, to use the idea of boundaries and context, each microservice serves its bounded context. Each will have its models, language, business rules, and technology stack.

However, this is not a silver bullet since perfect alignment between a microservice and a bounded context might only sometimes be accurate. Still, some applications, including ours, are ideal candidates for microservices and DDD.

We can consider several scenarios where we can isolate certain services that are not the most obvious ones and wouldn't have been scoped initially as bounded contexts. Take, for example, email and alert systems. It is easy enough to place that logic and functionality in the web application, such that when an appointment is submitted, we send the confirmations to our patients and the alerts to the staff. This is reasonable, but we could also create separate message queue-based services that serve the sole purpose of delivering these messages. That way, the web application has even less responsibility, and we run less of a risk of inadvertently modifying UI logic when addressing an email or alert maintenance concern. We will review the benefits and implementations of message queue-based services in *Anchor 3, Asynchronous Communication between Microservices*.

Ultimately, because DDD suggests separating the contexts into standalone components, microservice architecture is the perfect architectural pattern to support this ambition. Considering that DDD serves as an initial guideline to define business rules with clear boundaries and minimal dependencies, each microservice will be developed to support that set of business rules and implement supporting services in the most efficient and loosely coupled manner.

Now that we can appreciate how DDD and microservices go hand in hand, let us review how entities and models are best employed and used in a DDD implementation.

## Designing entities and models

As previously discussed, entities are objects defined by their unique identity and life cycle, representing distinct elements within the domain. They are defined to encapsulate the intended behavior of the entity they represent. Therefore, we will end up defining methods to enforce specific business rules. Models, however, serve as abstract representations of the domain, encompassing entities, value objects, and their relationships. They are rich, behavior-driven representations of the core domain concepts, rules, and processes. A model encapsulates the way the domain “thinks” and operates, serving as a direct reflection of the business’s mental model.

This section delves into the characteristics and roles of entities and models, exploring their interplay and significance in designing robust, maintainable, and scalable software systems.

Let us start by reviewing entities and some considerations when creating them.

### Entities and why we need them

Decisions made in DDD are driven by behavior, but behaviors require objects. These objects are referred to as **entities**. An entity is a representation of data in your system, something that you need to be able to retrieve, manipulate, and store data on. Entities also typically have an identity key, most commonly an auto-incrementing integer or a **Globally Unique Identifier (GUID)** value.

In code, you want to create a base entity type to set the desired key type relative to the derived type. The following is an example of a `BaseEntity` class in C#:

```
public abstract class BaseEntity<TId>
{
    public TId Id { get; set; }
}
```

The `BaseEntity` class will take a generic type parameter, allowing us to set the `Id` type as needed. We also ensure that we set the class type as abstract to prevent independent instantiation of `BaseEntity`.

Having a mental map of the entities and data persistence strategy you intend to implement is one thing, but modeling and coding are often different activities when it is time to implement them in code. Given DDD’s unique demands, some patterns and techniques can be employed to implement specific technical attributes in our DDD-styled entities.

It is vital to establish a **central entity** or **aggregate root** for your system and then design all other parts around it. For instance, appointment booking is the most central operation of our system, so all other entities are just referenced. In another scope, patient record management is the most integral part, so we want to make that aspect as robust as possible.

These scenarios show that our decisions must be relative to the business objectives. One size does not fit all, and our design needs to consider what is best for our overall context. Beyond that, we need to understand the operations to be carried out before implementing domain events, repositories, factories, and any other business logic-related elements.

Now, let us look at the concrete uses of entities in our DDD-styled system. We must understand how entities should be employed and their actual uses in our system.

## Practical uses of entities in code

An entity is primarily defined by its identity and is essential to the domain model. We must design and implement entities carefully.

An entity represents data that might need to traverse multiple microservices and, as a result, needs to have an identity value that can uniquely identify it in any system. We generally use sequential integers for our ID values in a relational database, which is acceptable for a single service. Still, we cannot rely on that sequential value across multiple services and databases. For this reason, we usually employ a GUID, a randomly generated alphanumeric string. It is not sequential, but it is easier to count on consistency since we set it in code instead of relying on the database to set the value.

Consider the definition of a `Patient` entity, as seen in the following code example. This simple class inherits from the `BaseEntity` class and defines `Guid` as the preferred data type for the identity value:

```
public class Patient : BaseEntity<Guid>
{
    public string Name { get; private set; }
    public string Sex { get; private set; }
    public Guid? PrimaryDoctorId { get; private set; }
}
```

Choosing the appropriate entity identifier type in microservices architectures is crucial for ensuring system reliability and scalability. While both integers and GUIDs are commonly used, GUIDs offer the following:

- **Global uniqueness:** Since services operate independently and may generate identifiers concurrently, using GUIDs minimizes the risk of identifier collisions without centralized coordination.
- **Decentralized ID generation:** Each service can generate GUIDs locally without relying on a central authority, enhancing system resilience and reducing bottlenecks. This allows services to remain loosely coupled, reducing the risk of cascading failures and improving scalability.
- **Simplified data merging:** There will be cases where data from multiple sources or services must be combined, and GUIDs ensure that identifiers remain unique across datasets.
- **Enhanced security:** GUIDs are less predictable than sequential integers, making it more challenging for malicious users to guess valid identifiers. This adds a layer of security against potential enumeration attacks.

As mentioned, there are cases where the same entity will have representations across multiple services, leading to the need for the exact identification value to be referenced in the different services. In such a scenario, the same attributes and behaviors might not be the same in each microservice or bounded context. For instance, the Patient entity in the patient management microservice might contain all the key attributes and behaviors of the patient we would have scoped. Still, the same entity in the appointment booking microservice might only need minimal data and behaviors as required by the appointment booking process. The entity's data attributes will always be relative to the requirements of the microservices or bounded context.

Consider designing our data objects for an ORM tool, where we must consider the implications of all **Create, Read, Update, and Delete (CRUD)** operations against our data. Based on the general design pattern of our entity classes, we would place navigation objects inside both entities in the relationship, but this could lead to cascading issues if not appropriately managed. This is a crucial design decision, as it is sometimes better to remove the navigation object, sacrificing some of the magic of Entity Framework, and retain greater control and predictability of how our models will interact. By keeping only the foreign key reference, we can better enforce one-way aggregate relationships with non-root entities.

This is why we might only reference `PrimaryDoctorId` and not include the `PrimaryDoctor` navigation property. In the case of microservices, `PrimaryDoctor` might be an entity in a separate service and doesn't need a physical presence in the current patient management service.

Domain entities, at their most basic, represent the data attributes of a scoped entity but can have methods that implement behaviors and logic that are only beneficial for the specific domain or entity. Next, we will look at anemic and rich domain models and how we implement them.

## Rich domain model versus anemic domain model

In applications not built using the DDD approach, we tend to have simple classes that model an entity's data attributes. At this point, most of the entity's logic and behavior are implemented at the business logic layer, not in the model itself. We call these **anemic models**. In DDD, we begin to implement implicit behaviors in the form of methods, turning the model into a **rich model**.

It is good to appreciate the difference between an anemic model and a rich model. A rich model is more behavioral and fits what we have described, that is, a model that implements methods for tasks relative to the model within the domain. An anemic model is more data-centric and tends only to implement properties. Anemic models are usually implemented as child entities, where there is no special logic. The logic is implemented in the **aggregate root** or the business logic layers. Anemic domain models are implemented using procedural style programming. This means the model has no behaviors and only exposes properties for the data points it will store. We then tend to put all our behavior in service objects in the business layer and risk ending up with spaghetti code, thus losing the advantages a domain model provides.

Let us look at the Patient class as a slightly more complex but still anemic entity model:

```
public class Patient : BaseEntity<int>
{
    public Patient(string name, string sex, int? primaryDoctorId = null)
    {
        Name = name;
        Sex = sex;
        PrimaryDoctorId = primaryDoctorId;
    }

    public Patient(int id)
    {
        Id = id;
    }

    private Patient() // required for EF
    {
```

```
    }

    public string Name { get; private set; }
    public string Sex { get; private set; }
    public int? PrimaryDoctorId { get; private set; }

    public void UpdateName(string name)
    {
        Name = name;
    }

    public override string ToString()
    {
        return Name.ToString();
    }
}
```

Enforcing encapsulation in your class is a good idea by requiring values to be set upon the instantiation of the object. Your decision on how rich or anemic your model is depends on the use or general operations of the microservice. Anemic models might be perfect for simpler CRUD services, where DDD might be a stretch for what you need to design the system. It is more simply used to model our persistence models since we only use the model for data storage and CRUD purposes.

In the following code, we will see an example of the Appointment class being implemented as a rich domain model, including logic to handle certain key operations on the data. The example has been broken into smaller chunks of code to highlight the different general components of a rich data model:

```
public class Appointment : BaseEntity<Guid>
{
    public Appointment(Guid id,
        int appointmentTypeId,
        Guid scheduleId,
        int doctorId,
        int patientId,
        int roomId,
        DateTime start,
        DateTime end,
```

```
        string title,
        DateTime? dateTimeConfirmed = null)
    {
        Id = id;
        AppointmentTypeId = appointmentTypeId;
        ScheduleId = scheduleId;
        DoctorId = doctorId;
        PatientId = patientId;
        RoomId = roomId;
        Start = start;
        End = end;
        Title = title;
        DateTimeConfirmed = dateTimeConfirmed;
    }
```

At a minimum, we need to ensure that we use constructors to enforce object creation rules. We list the required values and assign them upon creation. It is common practice to include validation checks and/or default values at this stage:

```
public void UpdateRoom(int newRoomId)
{
    if (newRoomId == RoomId) return;

    RoomId = newRoomId;
}

public void UpdateStartTime(DateTime newStartTime)
{
    if (newStartTime == Start) return;

    Start = newStartTime;
}
```

```
public void Confirm(DateTime dateConfirmed)
{
    if (DateTimeConfirmed.HasValue) return;

    DateTimeConfirmed = dateConfirmed;
}
}
```

We also have some examples of behaviors that we implement in the method. Traditionally, you want to implement these in the business logic layer or a repository. For a rich data model, we equip it with the methods it needs to morph its data as needed. We may also implement our own validation rules in these methods.

Now that we understand what an entity is, the rules for creating it, and the general guidelines for implementing it, we can explore value objects and how they differ from entities in our domain model.

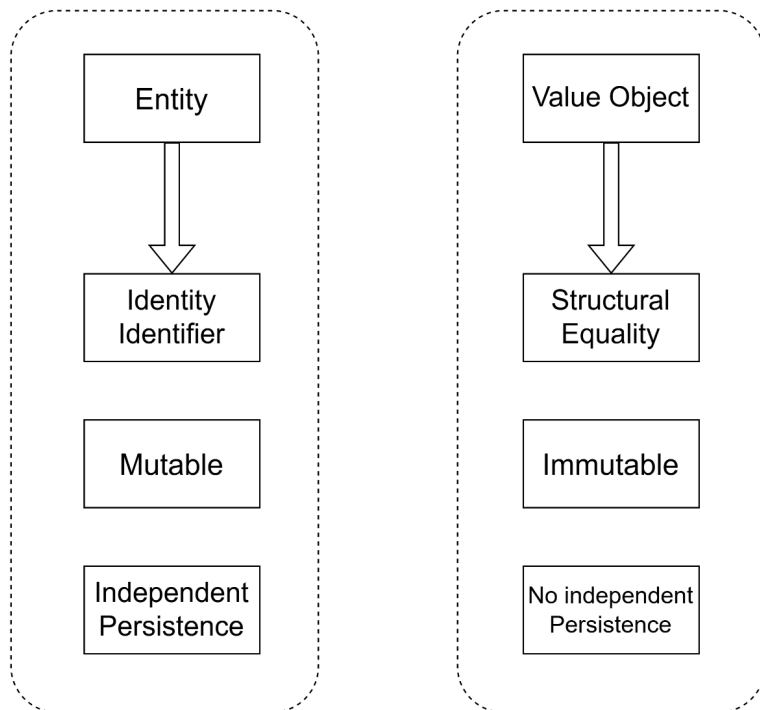
## Understanding and using value objects

We have observed the main attributes that entity objects should be identified by: continuity and identity, not necessarily their values. This brings us to consider what we call objects that are indeed defined by their values. These are **value objects**. They, too, have their place in the domain model as they are used to measure and quantify parts of the domain.

They do not boast identity keys in the same way that entities do, but their keys are formed through the composition of the values of all their properties, hence the name value objects.

Given that the data they store is so important in defining their identity and uniqueness in our system, these objects are immutable; that is, they never change once they are created. Understanding the differences between entity models and value objects is essential. *Figure 2.2* shows a comparison between entities and value objects.





*Figure 2.2 – Entities have an identity, are mutable, and are independently persisted, while Value Objects use structural equality, are immutable, and are not independently persisted.*

Remember that the value object's values should never change once an object has been created. Instead, another instance should be created with the new intended values when necessary. We can do so if these objects must be compared by comparing all the values. This gets easier and more practical since the introduction of **record types** in C# 10. Records differ from classes and structs in that record types are based on value-based equality for comparisons. Two record objects are considered equal if the record definitions are identical and the values in both records are equal for every field.

Value objects can have methods and behaviors, but their scope should be limited. Methods should only compute and never change the state of the value object or values therein, noting that it is immutable. Remember, we should create a new object if new values are required.

Let us delve a bit deeper into the basics of value objects. The truth is, we use them all the time in our development tasks, probably without noticing. A typical example of these would be string objects. A string in .NET and most other languages is a collection of characters or a `char` array. This character collection gives the string a value or a specific meaning. If we change one value

of the character array or reorder them, we change the whole meaning of the string. In .NET, it is relatively easy to augment these values through string manipulation methods that allow us to change the letter cases or extract a part of the string. In doing these operations, we don't change the value of the string, but we get a whole new object with the new values. As we said, immutable objects do not change in value, but a new object must be created if a change is desired.

When scoping value objects for your system, assessing all the information needed from the start is essential to making this value object airtight. An excellent example of making sure the information is correct is weight. It is easy to store data on a patient and state that they weigh 50. But the value 50 by itself is useless, considering how many possible units of measurement there are. So, in practicality, this value has no meaning without the unit. 50 lbs. (pounds) is an entirely different measurement from 50 kg. We also need to ensure that the Class or Record type used to store this information restricts which value can be updated at a time. For instance, changing the numeric value is acceptable as a person may have gained or lost weight. However, allowing the same flexibility to update the unit by itself can have a more profound impact on what the numeric value means regarding the weight change. It's better to design the type so that the unit can only be changed together with the numeric weight value, never on its own.

You can also consider appointment scheduling. We should only entertain accepting the start date and time with an accompanying end date and time. Suppose we set this appointment and start and end datetime combination in a record type. In that case, it will make it much easier to carry out equality checks for clashes, and we don't need to clutter our code with overloads and excess logic to ensure that the appointment times are acceptable for the system. We can define a `TimeSlot` record in C#. This record will encapsulate the start and end `DateTime` values, providing value-based equality checks and immutability. An `Appointment` class can then utilize this `TimeSlot` record to represent individual appointments. Below is an example of how this could be implemented.

```
public record TimeSlot(DateTime Start, DateTime End)
{
    public TimeSlot
    {
        if (End <= Start)
        {
            throw new ArgumentException("End time must be after start time.");
        }
    }
}
```

The `TimeSlot` record encapsulates the start and end times of an appointment. Using a record, we benefit from value-based equality and immutability, simplifying equality checks and ensuring that the time slot remains consistent once created. Our `Appointment` class will look like this (shortened for brevity):

```
public class Appointment
{
    public Guid Id { get; }
    public TimeSlot Slot { get; }
    // Shortened for brevity
    public Appointment(Guid id, TimeSlot slot /* Shortened for brevity */)
    {
        Id = id;
        Slot = slot;
        // Shortened for brevity
    }
}
```

The most crucial goal here, once again, is to ensure that the state of the value object is not changed after it is created. So whether you choose to use a `Record` or `Class` type, the values should be set through the constructor at the time of object creation, and the constructor must also perform all validation and invariant checks.

Here's how you can create and compare appointments using the `TimeSlot` record and the `Appointment` class:

```
var slot1 = new TimeSlot(
    new DateTime(2024, 11, 23, 14, 0, 0),
    new DateTime(2024, 11, 23, 15, 0, 0)
);

var slot2 = new TimeSlot(
    new DateTime(2024, 11, 23, 14, 0, 0),
    new DateTime(2024, 11, 23, 15, 0, 0)
);
```

```
var appointment1 = new Appointment(Guid.NewGuid(), "Doctor's Appointment",
    slot1);
var appointment2 = new Appointment(Guid.NewGuid(), "Meeting with Client",
    slot2);

bool areSlotsEqual = slot1 == slot2; // True, since records are compared
// by value
bool areAppointmentsEqual = appointment1.Equals(appointment2); // False,
// different IdsValues
```

It is also good practice to define the value objects as read-only types to guard against modifications beyond that. Remember, though, that with a class type, you must include appropriate logic for equality comparison. In contrast, a record type comes with that built in since it is based on value-based equality semantics.

Now that we have seen entities and value objects in action, let us focus on the aggregate pattern and how it is used in DDD.

## Using the aggregate pattern

The **aggregate pattern** is a specific software design pattern within DDD. It promotes the collection of related entities and aggregates them into a unit. Aggregates make it easier to define ownership of elements in large systems. They define consistency boundaries within which all changes must be made through a single, controlled entry point, the aggregate root. This design organizes the model and serves as a guardrail to prevent accidental data corruption and business rule violations.

By requiring that all state changes pass through the aggregate, developers are discouraged (and often technically prevented) from directly manipulating the internal state of nested entities or related data. This isolation ensures that internal invariants are not violated due to partial updates or inconsistent interactions across modules or services. After identifying the different contexts in the domain, we can extract the data we need from potentially multiple contexts and sources and model them.

## Aggregates and aggregate roots

An aggregate comprises one or more entities and value objects that interact in one way or another. This interaction then encourages us to treat them as a unit for data changes. We also want to ensure consistency in the aggregate before making changes.

Consider a Patient aggregate in a healthcare management system. We can identify the different components, such as the following:

- **Aggregate root:** Patient entity identified by a unique patient ID
- **Entities:** Medical Record, Appointment, and Prescription entities associated with the patient
- **Value objects:** Address, contact information, and insurance details value objects

All interactions with the patient's data, such as updating contact information or adding a new medical record, are performed through the Patient aggregate root. This approach ensures that all business rules related to the patient are enforced consistently.

Aggregates make enforcing specific rules for data and validation across multiple objects easier. For example, a patient can have multiple addresses but needs to have at least one to be in a valid state. These constraints are more easily applied from a higher root level. It is also easier to ensure that data changes according to the ACID principles. Treating each aggregate as a self-contained consistency boundary and ensuring that all changes to an aggregate are made through the aggregate root make it easier to ensure that changes are treated as a single unit of work. This ensures transactional guarantees, even if eventual consistency is used between aggregates or services.

The aggregate roots also help us maintain **invariants**, which are non-negotiable conditions that ensure the system is consistent. An invariant is a rule that defines valid states for a domain model and must be consistently enforced to maintain business correctness.

Let us take our appointment booking service and the related aggregate. An invariant would be a rule around dates, where an appointment cannot be scheduled in the past. This can be enforced by rejecting scheduling requests where the start time is less than the current time. A good way of determining an aggregate root is to consider whether deleting a record should trigger a cascade deletion of other objects in the hierarchy. Our aggregate root represents a cluster of associated objects treated as a unit for data-related changes.

*Figure 2.3* shows a relationship between a root and a non-root entity. Diagrams are a good way to visualize how different entities and objects relate. This will help us gain a broader understanding during the scoping exercise as we assess each model's role in the domain and how our data strategies will come to life.

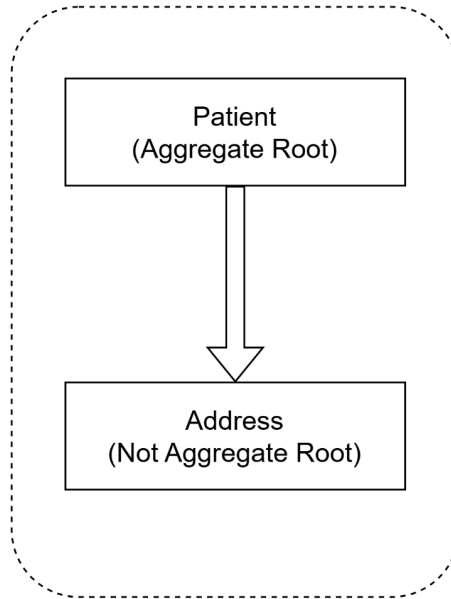


Figure 2.3 – Patient aggregate showing Patient as the aggregate root and Address as a related child entity within the same aggregate.

In our healthcare management system, the appointment scheduling service and functionality can be effectively modeled using the aggregate pattern, since an aggregate encapsulates related entities and value objects, ensuring consistency and enforcing business rules within its boundary.

The Appointment aggregate can be designed to include the following:

- **Aggregate root:** Appointment entity, identified by a unique AppointmentId
- **Entities:**
  - Patient: Represents the individual receiving care
  - Doctor: Represents the healthcare provider
- **Value objects:**
  - TimeSlot: Encapsulates the start and end times of the appointment
  - Location: Specifies where the appointment will take place

The following is an example implementation of the Appointment aggregate in C#:

```
public record TimeSlot(DateTime Start, DateTime End)
{
    public TimeSlot
```

```
{
    if (End <= Start)
    {
        throw new ArgumentException("End time must be after start time.");
    }
}

}

public record Location(string RoomNumber, string Building);

public class Appointment
{
    public Guid AppointmentId { get; }
    public Guid PatientId { get; }
    public Guid DoctorId { get; }
    public TimeSlot Slot { get; }
    public Location Location { get; }
    public string Purpose { get; private set; }

    public Appointment(Guid appointmentId, Guid patientId, Guid doctorId,
        TimeSlot slot, Location location, string purpose)
    {
        AppointmentId = appointmentId;
        PatientId = patientId;
        DoctorId = doctorId;
        Slot = slot;
        Location = location;
        Purpose = purpose;
    }

    public void Reschedule(TimeSlot newSlot)
    {
        // Business rules to check for conflicts, etc.
        Slot = newSlot;
    }

    public void ChangePurpose(string newPurpose)
```

```
{
    Purpose = newPurpose;
}
}
```

As we continue to build on the concept of an appointment, our entity model evolves to become an aggregate root with more and more fields and behaviors. Now, this appointment can be used in code as follows:

```
var appointmentId = Guid.NewGuid();
var patientId = Guid.NewGuid(); // Retrieved from patient records
var doctorId = Guid.NewGuid(); // Retrieved from doctor records

var slot = new TimeSlot(
    new DateTime(2024, 11, 25, 10, 0, 0),
    new DateTime(2024, 11, 25, 11, 0, 0)
);

var location = new Location("Room 101", "Building A");

var appointment = new Appointment(appointmentId, patientId, doctorId,
    slot, location, "Routine Check-up");

// Reschedule the appointment
var newSlot = new TimeSlot(
    new DateTime(2024, 11, 26, 14, 0, 0),
    new DateTime(2024, 11, 26, 15, 0, 0)
);
appointment.Reschedule(newSlot);

// Change the purpose of the appointment
appointment.ChangePurpose("Follow-up Consultation");
```

By modeling the appointment scheduling system as an aggregate, we create a robust structure that aligns with DDD principles, facilitating maintainability and scalability in the healthcare management system. Some of the benefits of this modeling include the following:

- **Consistency enforcement:** By encapsulating related entities and value objects, the aggregate ensures that all business rules and invariants are maintained within its boundary



- **Simplified data access:** External components interact with the aggregate root (Appointment), reducing the complexity of data access and manipulation
- **Transaction management:** Operations that modify the aggregate's state are handled as a single transaction, ensuring atomicity and consistency

Now, let us turn our sights to exploring relationships a bit more. We must look at other parts of the system and determine what a child, parent, or sibling should be.

## Relationships in aggregates

Since aggregates are clusters of related objects, we need to fully understand how those objects relate to one another. We think of relationships as bi-directional; *Object A* relates to *Object B*, and vice versa.

For example, a patient has an appointment, and we need to schedule an appointment for a patient. This way of thinking might contradict the principles of DDD, though, because we are trying to reduce couplings, and a two-way relationship can introduce unnecessary complexity and tight coupling. While Appointment might reference Patient by ID, Patient does not need a direct reference to Appointment. This approach keeps aggregates self-contained and simplifies their interactions.

In DDD, we want to promote the notion that one-way relationships will suffice. If we go for two-way relationships, which might seem plausible, we must ensure that the added complexity is justifiable. A relationship allows one object to access details related to the other. This means that we should be able to see all the details of the related appointment from the patient, but we do not need to see all the patient's details when going in the other direction.

A simple ID reference to the patient can suffice. If we introduce a bi-directional relationship, we create a direct dependency between appointments and patients, which isn't necessarily true. A good measure to use in defining our models is to ask, "*Can I define this object without needing the other?*"

An excellent guideline is to consider whether our aggregates should always flow in a single direction from the root to its dependents and never the other way around.

We have examined apparent and tightly knit relationships, but what happens when relationships are more widely spread? Let's discuss how we handle relationships that traverse aggregates.

## Handling relationships that span aggregates

Handling relationships spanning DDD aggregates requires strategies that preserve each aggregate's autonomy and integrity. Systems can maintain robustness and scalability by referencing other aggregates by identity, utilizing domain events, coordinating through application services,

and embracing **eventual consistency**. Eventual consistency is a strategy for maintaining data consistency across bounded contexts or services without requiring immediate synchronization. This technique allows us to avoid enforcing strict, real-time consistency across the entire system, which can lead to tight coupling and performance bottlenecks, and instead, different parts of the system can be updated asynchronously, typically using domain events. We will discuss this in more detail in *Anchor 3, Applying Event-Sourcing Patterns*.

We know that aggregates are boundaries between logical groupings in our applications. We enforce these boundaries by restricting direct references to objects in the aggregate if they are not the root. In our example of the patient and the address, we can safely make the patient record reference the address, making our address an entity or value object.

The critical thing to note in this association is that the only way to get the correct address for a patient is by traversing the patient's record. The address won't be referenced anywhere else. However, patients can be referenced from other aggregates, such as an appointment record or a document. So, it is essential to understand when a bit of data can be referenced directly, and we can use this to guide our core domain concepts.

We have investigated value objects and what distinguishes them from entities. We have also reviewed the best ways to implement them in C# code to ensure their unique characteristics.

## Summary

This chapter explored DDD, a software development methodology that aims to align software systems with complex business domains. DDD focuses on creating models that accurately reflect the domain by fostering collaboration between technical teams and domain experts. The chapter emphasized the importance of understanding and structuring domain elements to build systems that are adaptable, maintainable, and aligned with business goals. This approach is particularly beneficial for applications such as healthcare management systems, which involve intricate relationships, workflows, and rules.

The chapter began by introducing the foundational concepts of DDD, including entities, which are objects with unique identities and life cycles. Entities, such as *Patient* in a healthcare system, represent distinct objects whose attributes may change over time while their identity remains constant. DDD also relies heavily on models, which serve as abstractions to solve domain problems. Models help developers focus on the core aspects of the domain, filtering out unnecessary information. Effective modeling requires close collaboration with domain experts to ensure relevance and accuracy.

Another critical aspect of DDD is creating a ubiquitous language, a shared vocabulary that bridges the gap between domain experts and developers. This common language reduces misunderstandings and ensures consistency throughout the development process. The chapter also introduced bounded contexts, which define logical boundaries within the domain to isolate models, rules, and terms. These boundaries simplify system design by containing complexity and clarifying the scope of interactions between different system parts.

The chapter also delved into immutable value objects defined by their attributes rather than identity. Value objects, such as `TimeSlot` for scheduling appointments, enable simpler equality checks and enforce consistency without the overhead of managing unique identifiers. In contrast to entities, value objects are interchangeable when their values are identical. The chapter explained how immutability and proper validation are essential characteristics of value objects to ensure the integrity of the domain model.

A significant focus was placed on aggregates, clusters of related entities, and value objects treated as a single unit of consistency. Aggregates, such as `Appointment`, including entities such as `Patient` and `Doctor`, provide a structured way to manage related data and enforce business rules. By routing all interactions through the aggregate root, aggregates ensure consistency within their boundaries and simplify transaction management.

We also examined the benefits and challenges of DDD. Advantages include enhanced communication, alignment with business objectives, improved flexibility, and a cleaner code base. However, the methodology is resource-intensive, requires deep domain expertise, and can introduce unnecessary complexity in simpler systems. Despite these challenges, DDD provides a robust framework for designing scalable, maintainable, and domain-focused applications, making it particularly valuable for complex domains like healthcare.

In summary, DDD offers a systematic approach to tackling complexity by emphasizing collaboration, clarity, and consistency. This chapter laid the groundwork for understanding how DDD principles integrate with microservices and other design patterns, offering a pathway to building more effective and resilient software systems. In the next chapter, we will explore synchronous communication between microservices.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





# 3

## Synchronous Communication between Microservices

In the previous chapter, we explored domain-driven design, a method for scoping and development that aids us in determining how to represent our entities in code. One of the major takeaways is that data will be shared across microservices, necessitating communication to exchange details about related data.

Each microservice is generally given a scope of management functionality, which usually includes an isolated data store. At some point, another microservice will require data from another microservice, and this is where we facilitate inter-service communication.

Communication in a distributed system can occur at various levels and through multiple patterns. In this chapter, we will focus on synchronous communication, where one service directly calls another and waits for a response before proceeding. While this approach can simplify specific workflows and ensure immediate feedback between services, it also introduces significant challenges. These include increased latency, tighter coupling between services, and a greater risk to overall system availability. If one service is slow or unavailable, it can cause a chain reaction of failures. Such drawbacks can undermine the autonomy and resilience that microservices aim to achieve, making it essential to consider when and how to use synchronous communication carefully.

We will start by reviewing scenarios such as real-time data workflows, where synchronous communication is preferred or required, then dive into the details of mainstream communication protocols such as **gRPC** and **REST**.

After reading this chapter, we will be able to do the following:

- Understand synchronous communication
- Understand how HTTP works
- Understand, implement, and communicate with RESTful APIs
- Understand, implement, and communicate with gRPC APIs

We will begin by exploring synchronous communication and the advantages and disadvantages of this method.

## Technical requirements

The code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter03>.

## Introduction to synchronous communication

Synchronous communication refers to a pattern where a service requests another service and waits for an immediate response before proceeding. This approach is akin to traditional function calls, where the caller is blocked until the callee returns a result.

Synchronous communication involves direct, real-time interactions between services. Key characteristics include the following:

- **Immediate response:** The requesting service expects an instant reply, leading to a tightly coupled interaction
- **Blocking behavior:** The caller is halted until the response is received, which can impact performance if the callee is slow or unresponsive
- **Simplified error handling:** Errors can be detected and managed promptly, as the caller is immediately aware of the outcome

Accept that we are building a complex and distributed system. Navigating the general request-response cycle of a web service call, the appropriate protocols to be used, and how we handle failures or long-running processes can be challenging. There are two broad categories of communication: **synchronous** and **asynchronous**.

Beyond that, we must scope the nature of the operation and make a call accordingly. If the operation requires an immediate response, we use synchronous techniques, and for long-running processes that don't necessarily need an immediate response, we make it asynchronous.

As mentioned before, we need to ensure that our inter-service operations consider the following:

- **Performance:** We always consider performance when developing a solution. Individually, we need each service to be as performant as possible, but this requirement also extends to communication scenarios. We need to ensure that when one service calls another, the call is done using the most efficient method possible. Since we predominantly use **RESTful APIs**, **HTTP** communication would be the go-to method. We can also consider using **gPRC**, which allows us to call a REST API with higher throughput and less latency.

One critical concern with synchronous communication is the **cumulative latency** introduced as service calls traverse the network. Each downstream service call introduces a network **hop**. Every hop involves network transmission time, processing delays on the target service, and potential queuing if the service is under load. While each call may only add milliseconds of delay, the total latency can escalate rapidly as more services are involved. Designing communication paths with minimal hops and considering fallback mechanisms or timeouts becomes essential to maintaining performance and reliability.

- **Resilience:** We must ensure that our service calls are done via durable channels. Remember that hardware can fail, or there can be a network outage when the service call is executed. So, we must consider two patterns that will make our services resilient:
  - **Retry pattern:** Transient failures are temporary failures that can derail an operation's completion. They tend to go away by themselves, though, and we would prefer to retry the operation a few times instead of failing the application's operation completely. Using this pattern, we retry our service call a few times based on a configuration, and should we not have any success, we trigger a timeout. We must be more careful with operations that augment the data since the request may get sent and a transient failure might prevent a response. This doesn't mean the operation wasn't completed, and retrying might lead to unwanted outcomes.
  - **Circuit Breaker pattern:** This pattern limits the number of times we try to make the service call. Multiple calls may fail because of how long a transient failure takes to resolve itself, or the number of requests going to the service might cause a bottleneck in available system resources and allocations. So, we could configure this pattern to limit the time we spend trying to call one service.
- **Tracing and monitoring:** We have established that a single operation can span multiple services. This brings another challenge to monitoring and tracing activities through all the services from one originating point. At this point, we need to use an appropriate tool



to handle distributed logging and aggregate the logs into a central place for more leisurely perusal and issue tracking. Tools such as OpenTelemetry and Azure Monitor can help with this task. We will do a deep dive into this later in this book.

Synchronous communication offers some of these benefits as it is easier to implement and understand due to its straightforward request-response nature. We also benefit from real-time data retrieval and processing, which helps us know whether our operations are successful or have failed. In the case of a failure, the linear flow of execution aids in tracing and resolving issues.

In our healthcare management system, a simple query from the frontend must be done synchronously. Suppose we need to see all the doctors in the system to present a list to the user. In that case, we need to invoke a direct call to the doctors API microservice, which fetches the records from the database and returns the data.

This is a straightforward operation that can be performed by any system, virtually or in any language, but we must be mindful of the potential drawbacks of synchronous communication. This implementation makes services interdependent, making the system less flexible and more challenging to scale. If the calling service experiences delays, the caller is also delayed, affecting overall performance.

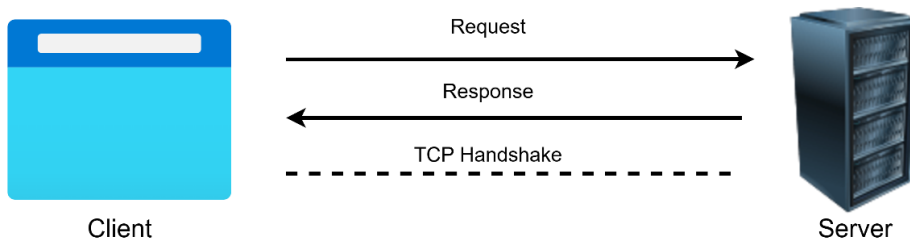
Now that we have an overview of synchronous communication and why we might need it, let us review HTTP communication and principles in more detail.

## Implementing HTTP API communication

HTTP operates on a request-response paradigm, where the client initiates a request and the server processes and returns a corresponding response.

Each HTTP request is independent, with no inherent memory of previous interactions. This stateless nature simplifies server design but requires mechanisms such as cookies or sessions to maintain state when necessary.

*Figure 3.1* outlines the general data flow of HTTP communication between a client and a server and the established **TCP handshake**.



*Figure 3.1 – HTTP communication between client and server via TCP handshake*

HTTP boasts a straightforward design and text-based format, and it is easy to implement and debug, contributing to its widespread adoption. It also supports various methods (e.g., GET, POST, PUT, and DELETE) that define specific actions to be performed on resources, enabling a wide range of operations. HTTP's simplicity, flexibility, and extensibility have established it as the cornerstone of web communication, allowing the seamless exchange of information that defines the modern internet.

HTTP requests and responses also include essential metadata about the communication between clients and services in the form of HTTP headers, which are key-value pairs enabling microservices to interpret and handle requests properly. In microservices-based systems, headers are commonly used for the following:

- **Authentication and authorization:** Headers such as Authorization carry tokens (e.g., JWTs, API keys, or OAuth tokens) that allow services to validate the caller's identity and permissions.
- **Content type declaration:** The Content-Type header specifies the media type of the request or response body (e.g., application/json, application/xml), enabling the receiving service to parse the payload correctly. Similarly, the Accept header tells the server what response format the client expects.
- **Correlation and tracing:** Headers such as X-Request-ID, traceparent, and baggage (used in distributed tracing) help propagate trace and correlation IDs across services for better observability and debugging.
- **Caching and compression:** Headers such as Cache-Control, ETag, and Content-Encoding influence caching policies and response compression to improve performance.
- **Custom metadata:** Microservices can also define and exchange custom headers to include context-specific data without altering the request body.

The next important part of this conversation comes in the form of a response or **HTTP response code**. HTTP defines standard response or status codes that we use to interpret the success or failure of our request. The categories of status codes are the following:

- **1xx (Informational):** Communicates protocol-level information
- **2xx (Success):** Indicates that the request was accepted and no errors occurred during processing
- **3xx (Redirection):** Indicates that an alternative route needs to be taken to complete the original request

- **4xx (Client Error):** General range for errors that arise from the request, such as poorly formed data (400) or a wrong address (404)
- **5xx (Server Error):** These errors indicate that the server failed to complete the task for some unforeseen reason

When constructing services, it is essential to document properly how requests should be formed and ensure that our responses match the actual outcomes.

**Representational State Transfer (REST)** and **Simple Object Access Protocol (SOAP)** are two prominent paradigms that utilize HTTP for web services. While both enable data exchange over networks, they differ significantly in design principles, data formats, and use cases. Let us compare these protocols in detail.

## Using SOAP for API development

The concept of **Application Programming Interfaces (APIs)** has been integral to software development since the early days of computing. APIs create bridges that enable communication between disparate software components and systems.

SOAP was developed in the late 1990s to exchange structured information and implement web services. It relies on XML for message format and typically uses HTTP/HTTPS for message negotiation and transmission. It also supports security (WS-Security) and transactions (enabling operation rollbacks), making it suitable for enterprise-level applications. However, SOAP's XML-based messaging could be verbose, leading to larger message sizes and increased bandwidth usage.

XML schemas define the structure of the request and response messages. Consider a web service that provides appointment details. A simplified schema for the appointment service might look like this:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.example.com/appointment" xmlns:app="http://
  www.example.com/appointment" elementFormDefault="qualified">
  <xs:element name="GetAppointmentDetails">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="AppointmentId" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:element name="GetAppointmentDetailsResponse">
    <xs:complexType>
```

```
<xs:sequence>
  <xs:element name="Appointment" type="app:AppointmentType"/>
</xs:sequence>
</xs:complexType>
</xs:element>
<xs:complexType name="AppointmentType">
  <xs:sequence>
    <xs:element name="AppointmentId" type="xs:string"/>
    <xs:element name="PatientId" type="xs:string"/>
    <xs:element name="DoctorId" type="xs:string"/>
    <xs:element name="StartTime" type="xs:dateTime"/>
    <xs:element name="EndTime" type="xs:dateTime"/>
    <xs:element name="RoomNumber" type="xs:string"/>
    <xs:element name="Building" type="xs:string"/>
    <xs:element name="Purpose" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
</xs:schema>
```

A client can send a SOAP request to obtain the details of a specific appointment. The client sends the following SOAP request to retrieve the details of an appointment with an identifier value:

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:app="http://www.example.com/appointment">
  <soap:Header/>
  <soap:Body>
    <app:GetAppointmentDetails>
      <app:AppointmentId>123e4567-e89b-12d3-a456-426614174000
    </app:AppointmentId>
    </app:GetAppointmentDetails>
  </soap:Body>
</soap:Envelope>
```

The following are the components of the SOAP request:

- `<soap:Envelope>`: The root element that defines the XML document as a SOAP message
- `<soap:Header>`: An optional element for header information; it is empty in this example
- `<soap:Body>`: Contains the request details

- `<app:GetAppointmentDetails>`: The specific operation being invoked, defined in the "http://www.example.com/appointment" namespace
- `<app:AppointmentId>`: The parameter for the operation specifying the appointment identifier

The server processes the request and responds with the appointment details:

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:app="http://www.example.com/appointment">
  <soap:Header/>
  <soap:Body>
    <app:GetAppointmentDetailsResponse>
      <app:Appointment>
        <app:AppointmentId>123e4567-e89b-12d3-a456-426614174000
        </app:AppointmentId>
        <app:PatientId>987e6543-e21b-34d3-c456-426614174111
        </app:PatientId>
        <app:DoctorId>654e3210-e54b-56d3-d456-426614174222</app:DoctorId>
        <app:StartTime>2024-12-01T10:00:00</app:StartTime>
        <app:EndTime>2024-12-01T11:00:00</app:EndTime>
        <app:RoomNumber>101</app:RoomNumber>
        <app:Building>Main</app:Building>
        <app:Purpose>General Consultation</app:Purpose>
      </app:Appointment>
    </app:GetAppointmentDetailsResponse>
  </soap:Body>
</soap:Envelope>
```

Let us review two critical tags in the preceding SOAP request body:

- `<app:GetAppointmentDetailsResponse>`: The response to the invoked operation
- `<app:Appointment>`: Contains the appointment details, including identifiers, time slots, location, and purpose

Web services can achieve platform-independent communication with strict adherence to agreed-upon message structures by utilizing SOAP with well-defined XML schemas. It has remained a popular choice for web service development due to its strong emphasis on standardization, reliability, and enterprise-grade features. Since it operates over a formal contract, this ensures

a strict message structure and type validation, which is especially beneficial in large, regulated industries such as finance and healthcare. It also supports built-in error handling, transaction management, WS-Security, and message-level encryption, making it suitable for scenarios that require robust security and transactional integrity.

The specification's extensiveness, however, could make implementation and maintenance challenging, and parsing XML could be resource-intensive, affecting performance. Some of the limitations of this implementation included the following:

- **Verbosity:** XML-based messaging can be verbose, leading to larger message sizes and increased bandwidth usage
- **Complexity:** The specification's extensiveness can make implementation and maintenance challenging
- **Performance:** Parsing XML can be resource-intensive, affecting performance

REST is a more efficient framework that has supplanted SOAP. It offers a more straightforward and lightweight approach to web services. REST leverages standard HTTP protocols, making it straightforward to implement and consume, and its statelessness and caching mechanisms enhance scalability.

Some of the reasons for choosing REST include the following:

- **Flexibility in data formats:** Unlike SOAP, which is limited to XML, REST supports multiple data formats, including JSON, XML, HTML, and plain text. JSON is lightweight and more straightforward to parse, enhancing performance and compatibility with web applications.
- **Performance and scalability:** REST's stateless nature and support for caching improve performance and scalability. REST reduces server load and simplifies scaling.
- **Lower bandwidth consumption:** REST's use of JSON allows it to consume less bandwidth than SOAP's XML-based messaging. This efficiency is crucial for high-traffic applications and mobile devices.
- **Better support for web clients:** REST's compatibility with web technologies and its stateless operations make it more suitable for web clients, including browsers and mobile applications.

An example of a JSON response from a RESTful service for the same appointments API would look like this:

```
{  
  "Appointment": {  
    "AppointmentId": "123e4567-e89b-12d3-a456-426614174000",
```

```
"PatientId": "987e6543-e21b-34d3-c456-426614174111",  
"DoctorId": "654e3210-e54b-56d3-d456-426614174222",  
"StartTime": "2024-12-01T10:00:00",  
"EndTime": "2024-12-01T11:00:00",  
"RoomNumber": "101",  
"Building": "Main",  
"Purpose": "General Consultation"  
}  
}
```

The Appointment object encapsulates all relevant details as key-value pairs in the JSON object. This JSON structure provides a more concise and readable format for the appointment details.

Now that we have a better idea of why REST is a preferred approach for HTTP-based service development, let's focus on REST and review how it can be used to develop a microservice.

## Implementing a RESTful microservice

REST has recently become the most used HTTP-based API communication and development framework. RESTful APIs expose methods that allow us to access underlying functionality via standard HTTP calls. A call or request typically consists of a **URL** or **endpoint**, a **verb** or **method**, and some data:

- **URL or endpoint:** The request URL is the address of the API or resource you are trying to interact with.
- **Verb or method:** The most used verbs are GET (to retrieve data), POST (to create a record), PUT (to update data), and DELETE (to delete data).
- **Data:** Data accompanies a call when necessary to complete a request. For instance, a POST request to the booking microservice would need to include the booking details that need to be created.

Let us start by creating a RESTful microservice in .NET. The following activity will be completed using mostly .NET CLI commands, which can be used regardless of your operating system and preferred IDE.

To make the project, we can run the following command in the terminal relative to a preferred directory:

```
dotnet new webapi -o AppointmentsApi  
cd AppointmentsApi
```

This command creates a new Web API project named `AppointmentsApi` and navigates into the newly created project directory. You can then use the `Code .` command to launch the project in **Visual Studio Code (VS Code)**, or you may proceed to open the solution in your preferred IDE.

We'll use the **Object Relational Mapper (ORM)**, **Entity Framework Core (EF Core)**, and a **SQLite database**. In later chapters, we will discuss in detail the thought process behind choosing an appropriate database and ORM for a service, but for now, SQLite and EF Core are good selections for this demo.

Add the necessary EF Core packages by running these commands:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

In the `Models` directory, create a new file named `Appointment.cs` and define the `Appointment`, `TimeSlot`, and `Location` classes and records as follows:

```
[Owned]
public record TimeSlot(DateTime Start, DateTime End);
[Owned]
public record Location(string RoomNumber, string Building);

public class Appointment
{
    public Guid AppointmentId { get; set; }
    public Guid PatientId { get; set; }
    public Guid DoctorId { get; set; }
    public TimeSlot Slot { get; set; }
    public Location Location { get; set; }
    public string Purpose { get; set; }

    public void Reschedule(TimeSlot newSlot)
    {
        Slot = newSlot;
    }

    public void ChangePurpose(string newPurpose)
    {

```



```
        Purpose = newPurpose;  
    }  
}
```

These are like the definitions in our previous chapter. Notice that a data attribute, [Owned], has been added to the value objects. This is required for EF Core to model entity types that can only ever appear on navigation properties of other entity types.

In the same directory, create a new file named `AppointmentContext.cs` in the `Data` directory to define the database context:

```
using AppointmentsApi.Models;  
using Microsoft.EntityFrameworkCore;  
public class AppointmentContext : DbContext  
{  
    public AppointmentContext(DbContextOptions<AppointmentContext> options)  
        : base(options)  
    {  
    }  
    public DbSet<Appointment> Appointments { get; set; }  
}
```

This context class registers the `Appointments` table as a C# `DbSet` and must be registered in the `Program.cs` file, with the dependency injection container, and configure the connection string as follows:

```
using Microsoft.EntityFrameworkCore;  
var builder = WebApplication.CreateBuilder(args);  
// Add services to the container.  
builder.Services.AddControllers();  
builder.Services.AddDbContext<AppointmentContext>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString(  
        "DefaultConnection")));  
var app = builder.Build();  
// Configure the HTTP request pipeline.  
if (app.Environment.IsDevelopment())  
{  
    app.UseDeveloperExceptionPage();  
}
```

```
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Finally, we need to ensure that the connection string is present in the `appsettings.json` file:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=AppointmentDb.db"
  },
  // Other settings...
}
```

Now that we have created the entity, the database context, and the project, we need to install the `dotnet ef` tool, and then we can perform migrations to generate the supporting database:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Now that we have completed the preliminary setup for our Appointments microservice and database, let's create our endpoints to allow data to be read and manipulated in this database.

ASP.NET Core features two types of API templates. The most used and more straightforward template requires specialized classes called **controllers**, which encapsulate HTTP and RESTful logic. Alternatively, you can opt for the Minimal API template, where additional classes are unnecessary, and all the HTTP logic can be written in the `Program.cs` file directly. The latter is more lightweight but might require a steeper learning curve to adapt to the pattern. We will use controllers for this demo, which .NET will enable us to generate through scaffolding.

First, we need to install the `dotnet-aspnet-codegenerator` tool with the following command:

```
dotnet tool install --global dotnet-aspnet-codegenerator
```

Then, we need an additional package to allow scaffolding in our web project. **Scaffolding** is a general term used for generating boilerplate or template code. We can use the following command to add the code generation design package to the project:

```
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design
```

After the successful installation of the code generation tool and package, we can execute our scaffolding activity with the following command:

```
dotnet aspnet-codegenerator controller -name AppointmentsController -async  
-api -m Appointment -dc AppointmentContext -outDir Controllers -dbProvider  
sqlite
```

This command generates a controller with **Create, Read, Update, Delete (CRUD)** actions tailored for the Appointment entity, facilitating API operations.

Here's a breakdown of the command:

- `controller`: Specifies that you want to generate a controller
- `-name AppointmentsController`: Sets the name of the controller to `AppointmentsController`
- `-async`: Generates asynchronous action methods
- `-api`: Indicates that the controller is for a Web API
- `-m Appointment`: Specifies the model class to use, in this case, `Appointment`
- `-dc AppointmentContext`: Defines the data context class, `AppointmentContext`
- `-outDir Controllers`: Sets the output directory for the generated controller
- `-dbProvider sqlite`: Sets the database provider to be used

Once we can confirm that our controller has been created, we can move to explore what is contained in this class. The `AppointmentsController` class supports CRUD operations for the Appointment entity. The controller is defined as follows:

```
[Route("api/[controller]")]  
[ApiController]  
public class AppointmentsController : ControllerBase
```

Let us take some time to assess the important parts of this code snippet:

- `[Route("api/[controller]")]`: Establishes the routing pattern, mapping HTTP requests to `api/appointments` to this controller
- `[ApiController]`: Indicates that this class is an API controller, enabling features such as automatic model validation and binding
- `ControllerBase`: The base class for API controllers provides essential RESTful and HTTP-related functionalities

The controller utilizes dependency injection to access `AppointmentContext`. Dependency injection is an integral part of clean coding and development practices. This method ensures that references to objects (such as a connection to the database) are disposed off properly after they have been used for a specific task. This is accomplished in the constructor's definition of the class:

```
private readonly AppointmentContext _context;

public AppointmentsController(AppointmentContext context)
{
    _context = context;
}
```

This approach ensures the controller can access the database context, facilitating data operations on `Appointments DbSet`.

Thanks to the *magic* of scaffolding, the controller comes ready-made with several **action methods** to handle HTTP requests. Notice that each method is decorated with an attribute that matches an HTTP verb:

- **GET:** Fetches all appointments asynchronously and returns them as an enumerable list:

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Appointment>>>
    GetAppointments()
{
    return await _context.Appointments.ToListAsync();
}
```

- **GET:** Retrieves a specific appointment by its unique identifier. If it is not found, it returns a 404 Not Found response:

```
[HttpGet("{id}")]
public async Task<ActionResult<Appointment>> GetAppointment(Guid id)
{
    var appointment = await _context.Appointments.FindAsync(id);

    if (appointment == null)
    {
        return NotFound();
    }
}
```

```
    }

    return appointment;
}
```

- **PUT:** Updates an existing appointment. Validates the ID, applies changes, and handles concurrency exceptions appropriately:

```
[HttpPut("{id}")]
public async Task<IActionResult> PutAppointment(Guid id,
    Appointment appointment)
{
    if (id != appointment.AppointmentId)
    {
        return BadRequest();
    }

    _context.Entry(appointment).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!AppointmentExists(id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }

    return NoContent();
}
```

- **POST:** This method adds a new appointment to the database and returns a 201 Created response with the location of the newly created resource:

```
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteAppointment(Guid id)
{
    var appointment = await _context.Appointments.FindAsync(id);
    if (appointment == null)
    {
        return NotFound();
    }

    _context.Appointments.Remove(appointment);
    await _context.SaveChangesAsync();

    return NoContent();
}
```

- **DELETE:** Deletes an appointment by its ID. If the appointment doesn't exist, it returns a 404 Not Found response:

```
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteAppointment(Guid id)
{
    var appointment = await _context.Appointments.FindAsync(id);
    if (appointment == null)
    {
        return NotFound();
    }

    _context.Appointments.Remove(appointment);
    await _context.SaveChangesAsync();

    return NoContent();
}
```

The class also includes a private method, `AppointmentExists`, to check the existence of an appointment. This method assists in verifying the presence of an appointment in a bid to handle concurrency and validation scenarios.

In general, the boilerplate code in `AppointmentsController` defines a well-structured API controller that uses ASP.NET Core's features to demonstrate best practices in routing, dependency injection, asynchronous programming, and error handling, providing a robust foundation for building RESTful services.

Now that we have our first microservice and database, we must interact with it for testing purposes. At this point, we need to use an appropriate application to simulate client interactions. We will review our testing options next.

## Testing a RESTful microservice

Testing is critical to developing robust and reliable RESTful microservices. In the context of our appointments API service, employing a combination of testing strategies and tools ensures that the service functions correctly and meets specified requirements. Testing often means that we will simulate a client's communication with our service and verify that the output of the API endpoint is in keeping with our expectations.

VS Code offers a powerful extension called **REST Client** that enables developers to compose and execute HTTP requests directly within the editor. Creating `.http` or `.rest` files allows you to streamline API testing and development without leaving your coding environment.

To install the REST Client extension, open VS Code and navigate to the **Extensions** view by clicking the square icon in the sidebar or pressing `Ctrl + Shift + X`. In the search bar, type `REST Client` and select the extension authored by Huachao Mao, as seen in *Figure 3.2*. Click **Install** to add the extension to your VS Code setup.

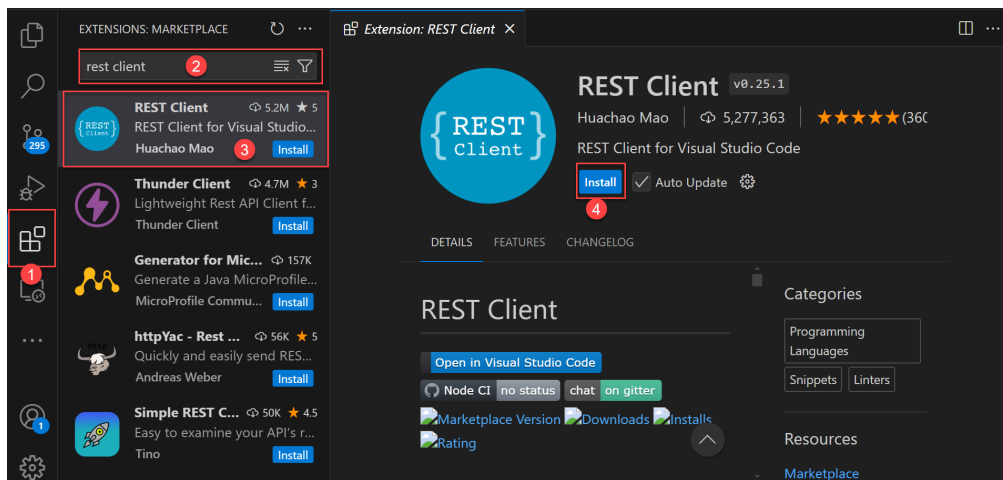


Figure 3.2 – Adding the REST Client extension to VS Code

Next, create a file in your project called `api-requests.http`. Within your `.http` file, you can define HTTP requests using a straightforward syntax. Each request can include the method, URL, headers, and body. We can populate the `.http` file like this to test the GET and POST endpoints:

```
# Retrieve all appointments
GET https://{ API ADDRESS}/api/appointments
Accept: application/json

###

# Create a new appointment
POST https://{ API ADDRESS}/api/appointments
Content-Type: application/json

{
  "patientId": "987e6543-e21b-34d3-c456-426614174111",
  "doctorId": "654e3210-e54b-56d3-d456-426614174222",
  "slot": {
    "start": "2024-12-01T10:00:00",
    "end": "2024-12-01T11:00:00"
  },
  "location": {
    "roomNumber": "101",
    "building": "Main"
  },
  "purpose": "General Consultation"
}
```

Note that you need to replace the sections marked as `{ API ADDRESS }` with the `applicationUrl` value found in `launchSettings.json` in your project. Now that we have test cases in our `.http` file, we can test the API. Use the `dotnet run` command to start the API and execute the POST endpoint by clicking the **Send Request** link above the request line. The response will be displayed in a new pane showing the status code, headers, and body. This should look like the following:

```
HTTP/1.1 201 Created
Connection: close
Content-Type: application/json; charset=utf-8
Date: Thu, 28 Nov 2024 15:53:27 GMT
Server: Kestrel
```



The HTTP response code is 201, a standard return type for a successful POST operation that creates a resource record. This is a manual test, and it can become tedious to test by hand in a more extensive application. Unit testing is a popular approach to automating API tests.

While traditional unit tests focus on isolated logic within classes or methods, they can also effectively test the behavior of controller actions, service layers, and input/output handling without running the entire application every time.

In testing an API, we simulate how a client will communicate with the API and the type of response it will get. When talking about synchronous API-to-API communication, the API making the call acts as a client. Let us review how we can facilitate this communication in our .NET API.

## Using HttpClient

We have access to a library called `HttpClient`, which provides methods that map to each HTTP verb, allowing us to bootstrap our calls and pass in the data we require for the operation. It is also a good idea to have a list or directory of endpoints to be used. This can be accomplished by listing addresses in `appsettings.json` and then having a class of constants where we define the behaviors or resources for the service.

Our `appsettings.json` would be decorated with the following block, which allows us to access the service address values from anywhere in our app:

```
"ApiEndpoints": {  
  "DoctorsApi": "DOCTORS_API_ENDPOINT",  
  "PatientsApi": "PATIENTS_API_ENDPOINT"  
}
```

The values per service configuration will be relative to the published address of the corresponding web service. This will serve as a base address that will be appended to an endpoint address. To allow `HttpClient` to be used in our .NET application, we need to register each base address and give it a name. Add the following code to the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);  
// Shortened for brevity  
// Register PatientsApiClient  
builder.Services.AddHttpClient<PatientsApiClient>(client =>  
{  
  client.BaseAddress = new  
    Uri(builder.Configuration["ApiEndpoints:PatientsApi"]);  
});
```

```
// Register DoctorsApiClient
builder.Services.AddHttpClient<DoctorsApiClient>(client =>
{
    client.BaseAddress = new
        Uri(builder.Configuration["ApiEndpoints:DoctorsApi"]);
});
var app = builder.Build();
// Shortened for brevity
```

This approach lets us register typed clients. Each client's logic is encapsulated in a class relative to the code needed to interact with a respective API, promoting the separation of concerns. This approach also centralizes configuration so the logic can be mocked or substituted during unit testing, enhancing testability.

The implementation for `PatientsApiClient` will look like this:

```
namespace AppointmentsApi.Services;
public record Patient(
    Guid PatientId,
    string FirstName,
    string LastName,
    DateTime DateOfBirth,
    string Gender,
    string ContactNumber,
    string Email
);

public class PatientsApiClient
{
    private readonly HttpClient _httpClient;

    public PatientsApiClient(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    // Example method to retrieve patient details
    public async Task<Patient> GetPatientAsync(Guid patientId)
```

```
{
    var response = await
        _httpClient.GetAsync($"api/patients/{patientId}");
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadFromJsonAsync<Patient>();
}
```

DoctorsApiClient will look like this:

```
namespace AppointmentsApi.Services;
public record Doctor(
    Guid DoctorId,
    string FirstName,
    string LastName,
    string Specialty
);

public class DoctorsApiClient
{
    private readonly HttpClient _httpClient;

    public DoctorsApiClient(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    // Example method to retrieve doctor details
    public async Task<Doctor> GetDoctorAsync(Guid doctorId)
    {
        var response = await
            _httpClient.GetAsync($"api/doctors/{doctorId}");
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<Doctor>();
    }
}
```

Both implementations have similar features:

- **HttpClient injection:** Because the typed clients are registered with the respective base URLs, `HttpClient` can safely be injected and will safely be disposed of after an operation. This is important to reduce the possibility of socket exhaustion, a side effect of `HttpClient` connections being opened and not closed properly.
- **Get method:** This method uses the injected `HttpClient` to make a GET request call to the endpoint. The client encapsulates functionality to *deserialize* the JSON returned by the called API and convert it to a C# object of a class modeled with matching fields and appropriate data types.
- **Models:** Both implementation classes contain records that act as models or data transfer objects. These models are based on the expected JSON response and are designed to match the data points by name and appropriate C# data types.

Both typed client classes can now be injected where needed. To keep this simple, we will inject it into the existing `AppointmentsController`:

```
public class AppointmentsController : ControllerBase
{
    private readonly AppointmentContext _context;
    private readonly PatientsApiClient _patientsApiClient;
    private readonly DoctorsApiClient _doctorsApiClient;

    public AppointmentsController(AppointmentContext context,
        PatientsApiClient patientsApiClient, DoctorsApiClient
        doctorsApiClient)
    {
        _context = context;
        _patientsApiClient = patientsApiClient;
        _doctorsApiClient = doctorsApiClient;
    }
    // Shortened for brevity...
}
```

Then, use the new functionality to enhance our GET method like this:

```
public async Task<ActionResult<Appointment>> GetAppointment(Guid id)
{
    var appointment = await _context.Appointments.FindAsync(id);
```

```
        if (appointment == null)
        {
            return NotFound();
        }

        // Use the clients to fetch patient and doctor details
        var patient = await _patientsApiClient.GetPatientAsync(id);
        var doctor = await _doctorsApiClient.GetDoctorAsync(id);

        // Combine data and return response
        AppointmentDetails appointmentDetails = new AppointmentDetails(

            id,
            patient,
            doctor,
            appointment.Slot.Start,
            appointment.Slot.End,
            appointment.Location.RoomNumber,
            appointment.Location.Building
        );

        return Ok(appointmentDetails);
    }
```

Our method will synchronously call the doctor and patient APIs to retrieve details about those entities and then compile a complete object with additional information.

This completes a general guideline on how a RESTful API might be developed, tested, and called using ASP.NET's API framework, Visual Studio and VS Code, and the HttpClient library. Now that we know synchronous communication with REST and HTTP, let's review how we set up gRPC communication between our services.

## Implementing gRPC API communication

As computing systems evolved, the need for communication between programs on different machines became apparent. In the 1960s and 1970s, **Remote Procedure Call (RPC)** emerged, enabling a program to execute a procedure on a remote server as if it were local. This abstraction

simplified network communication, allowing developers to invoke remote services with simple function calls. However, early RPC implementations often led to tight coupling between the client and server, making changes or updates challenging without affecting both ends. Some notable challenges we may face include:

- **Tight coupling:** RPC often makes changes or updates almost impossible without affecting both ends
- **Platform dependency:** Early RPC implementations were platform-specific, hindering system interoperability
- **Scalability issues:** RPC's synchronous nature can lead to performance bottlenecks, especially in high-latency networks

Essentially, a client sends a request to the server to execute a specific procedure with the provided parameters, and the server processes the request, performs the procedure, and returns the result to the client. Eventually, modern architectures transitioned to protocols such as gRPC, a high-performance RPC framework developed by Google. It offers improved performance and cross-platform support and addresses many of RPC's limitations.

Some of the tangible benefits of using gRPC include the following:

- **Compactness:** Produces smaller payloads, reducing network bandwidth usage
- **Faster serialization and deserialization:** **Protocol Buffers (Protobuf)**'s binary format allows quicker processing than REST's text-based JSON
- **Server streaming:** The server can send multiple responses to a single client request, which is beneficial for delivering continuous data streams
- **Client streaming:** The client sends requests to the server, which processes them and returns a single response
- **Bidirectional streaming:** Both the client and server exchange messages simultaneously, enabling real-time communication

gRPC is fully supported by ASP.NET Core, making it a great candidate for our .NET Core-based microservices solution. Given its contract-based nature, it naturally enforces specific standards and expectations that we try to emulate when creating our REST API service classes with interfaces.

It also boasts cross-language interoperability. gRPC can automatically generate client and server code in several programming languages, including C#, Java, Go, Python, Node.js, Ruby, Kotlin, and Swift. This enables teams to build microservices in different languages while maintaining a consistent and type-safe contract across the entire system. This makes gRPC an excellent choice

for **polyglot microservices architectures**, where different teams may use different stacks but still need reliable and structured communication.

In transitioning from RESTful APIs to gRPC, we are introduced to **HTTP/2**, which presents several benefits that enhance performance, efficiency, and scalability. gRPC operates over HTTP/2, a significant advancement over **HTTP/1.1**, offering the following:

- **Multiplexing:** Allows multiple requests and responses to be sent concurrently over a single TCP connection, reducing latency and improving throughput
- **Binary framing and compression:** This method utilizes a compact binary format for data transmission, leading to faster parsing and reduced bandwidth usage compared to the text-based JSON format used in RESTful APIs

Adopting gRPC with HTTP/2 offers substantial benefits over traditional RESTful APIs, including improved performance, efficient data serialization, support for advanced communication patterns, strong typing, built-in support for deadlines and timeouts, and enhanced security features. These advantages make gRPC a compelling choice for modern, high-performance, scalable service architectures.

Now, let us review how to set up a Documents microservice, which will facilitate retrieving a patient's medical documents stored in the system.

## Creating a gRPC microservice

We start by creating a new ASP.NET Core gRPC service project. Run the following command in a directory of your choice:

```
dotnet new grpc -o DocumentsService
```

Next, we define the service contract using Protobuf in a **proto** file. This contract specifies the service methods and message types. Protobuf acts as the **Interface Definition Language (IDL)** that specifies the **service contract** and the structure of the messages exchanged between clients and servers. Services and messages are defined in **.proto** files, and they describe the following:

- **Service contracts:** Which remote procedures are available, their names, and what input/output they expect
- **Message structures:** The schema of the request and response data

Add the following documents **.proto** file to the Protos folder in the project:

```
syntax = "proto3";  
option csharp_namespace = "DocumentsService.Protos";  
package DocumentSearch;
```

```
service DocumentService {  
    rpc GetAll (PatientId) returns (DocumentList);  
    rpc Get (DocumentId) returns (Document);  
}  
  
message Empty{}  
  
message Document {  
    string patientId = 1;  
    string name = 2;  
    string Id = 3;  
}  
  
message DocumentList{  
    repeated Document documents = 1;  
}  
  
message DocumentId {  
    string Id = 1;  
}  
  
message PatientId {  
    string Id = 1;  
}
```

This proto file outlines the properties and behaviors available and exposed by the server (or broadcasting microservice). We have defined a method to retrieve all documents for a patient ID and one to get a document by ID. Now that we have defined our prototype, we must implement our methods in an actual service class.

Add the `documents.proto` file to the project and configure it in the `.csproj` file:

```
<ItemGroup>  
    <Protobuf Include="Protos\documents.proto" GrpcServices="Server" />  
</ItemGroup>
```

Now that we have laid the groundwork, we must create the implementation class for the contract:

```
public class DocumentServiceImpl : DocumentService.DocumentServiceBase  
{
```



```
// Simulating a data store
private static readonly List<Document> Documents = new List<Document>
{
    new Document { Id = "1", PatientId = "123", Name = "Document1" },
    new Document { Id = "2", PatientId = "456", Name = "Document2" }
};

public override Task<DocumentList> GetAll(PatientId request,
    ServerCallContext context)
{
    var documentList = new DocumentList();
    documentList.Documents.AddRange(Documents.Where(
        q => q.Id == request.Id));
    return Task.FromResult(documentList);
}

public override Task<Document> Get(DocumentId request,
    ServerCallContext context)
{
    var document = Documents.Find(doc => doc.Id == request.Id);
    if (document == null)
    {
        throw new RpcException(new Status(StatusCode.NotFound,
            "Document not found"));
    }
    return Task.FromResult(document);
}
}
```

Notice that we have models, such as `Document`, that we have yet to create explicitly. In gRPC services, data structures and service contracts are defined based on the contents of the proto file. The `Document` message, as specified in your `.proto` file, will be automatically translated into a corresponding C# class during the build process. This generated class will include properties that mirror the fields defined in the `Document` message. According to the Protobuf specification, the class also implements the interfaces and methods required for serialization and deserialization.

Now that we have defined the implementation, we must register the service in the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddGrpc();
var app = builder.Build();
app.MapGrpcService<DocumentServiceImpl>();
app.Run();
```

Now that we have configured our document services, we can run the `dotnet run` command. Let's move forward with adding the needed integrations to our appointments service.

## Communicating with a gRPC service

Now, we set up our RESTful API to communicate with our gRPC service. To enable the RESTful API to make gRPC calls to the gRPC web service, we must first add the following support package:

```
dotnet add package Grpc.Net.Client
dotnet add package Google.Protobuf
dotnet add package Grpc.Tools
```

Next, ensure that the `.proto` file defining the gRPC service is included in your project. Place it in a suitable directory, such as `Protos`. You may update the namespace of the `proto` file to the following value:

```
option csharp_namespace = "AppointmentsApi.Protos";
```

After doing this, modify `.csproj` to include the `Protobuf` file and specify that it should generate client code:

```
<ItemGroup>
  <Protobuf Include="Protos\documents.proto" GrpcServices="Client" />
</ItemGroup>
```

Given our changes to the filesystem, building the project before moving forward is a good idea. Run the `dotnet build` command. The build process uses the `Grpc.Tools` package to generate the `DocumentService` types:

- `AppointmentsApi\obj\Debug\[TARGET_FRAMEWORK]\Protos\Documents.cs`: The `Protobuf` code that populates, serializes, and retrieves the request and response message types
- `AppointmentsApi\obj\Debug\[TARGET_FRAMEWORK]\Protos\DocumentsGrpc.cs`: Contains the generated client classes

Now that we have configured the API project with the client and models needed to communicate with the gRPC service, we can refactor the GET endpoint and DTO to include the documents attached to the patient.

The DTO will be defined like this:

```
public record AppointmentDetails(  
    /* Shortened for brevity */  
    DocumentList Documents  
);
```

We have added a field for DocumentList, one of the models defined in proto. The GET method will look like this:

```
[HttpGet("{id}")]  
public async Task<ActionResult<Appointment>> GetAppointment(  
    Guid id)  
{  
    /* Shortened for brevity */  
    // Use gRPC service to retrieve document information  
    // on patient  
    using var channel = GrpcChannel.ForAddress("GRPC_ENDPOINT");  
    var client = new  
        DocumentService.DocumentServiceClient(channel);  
    var documents = await client.GetAllAsync(new PatientId{  
        Id = patient.PatientId.ToString()});  
  
    // Combine data and return response  
    AppointmentDetails appointmentDetails = new  
        AppointmentDetails(  
            /* Shortened for brevity */  
            documents  
        );  
    return Ok(appointmentDetails);  
}
```

The preceding code example initializes the gRPC client using a hardcoded service URL ("GRPC\_ENDPOINT"). While this works for development, a more flexible and production-ready approach is

to configure the endpoint address dynamically. This can be achieved by defining the gRPC service URL in the `appsettings.json` file under a custom section:

```
"GrpcEndpoints": {
  "DocumentService": "https://localhost:5170"
},
```

You can then inject the `IConfiguration` interface and retrieve the value when setting up the channel:

```
var grpcAddress = _configuration["GrpcEndpoints:DocumentService"];
using var channel = GrpcChannel.ForAddress(grpcAddress);
var client = new DocumentService.DocumentServiceClient(channel);
```

This approach adheres to cloud-native best practices, enabling environment-specific configurations without modifying the code base and ensuring that your microservice remains portable, scalable, and easy to deploy across different stages of the software life cycle.

Now, we are instantiating a **stub** or **proxy**. This stub provides the same methods as the remote service, allowing the client to invoke them as if they were local. We are using our stub to call the method to retrieve the list of documents for the patient who has already been retrieved and is associated with the appointment. *Figure 3.3* shows the general flow of communication between the gRPC client and server.

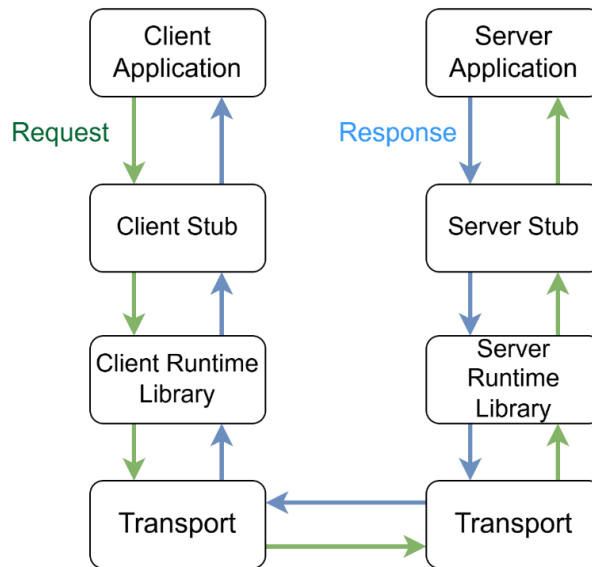


Figure 3.3 – The gRPC client/server flow

The general flow of the sequence is as follows:

- **Serialization:** The client-side stub serializes the request message into a binary format using Protobuf
- **Transmission:** The serialized message is transmitted over the network to the server, typically via HTTP/2, which offers benefits such as lower latency and support for parallel request multiplexing
- **Server processing:** The server receives the message, deserializes it, processes the request, and prepares a response
- **Response handling:** The server serializes the response message and sends it back to the client, where the stub deserializes it into a usable object

We have created RESTful and gRPC microservices and communicated with them synchronously. Following these examples, you can implement a microservice using the more appropriate technology based on the service's needs. Let us jump to summarize what we have done.

## Summary

This chapter dove into the nuances of communication within microservices architectures, focusing on the necessity and implementation of synchronous communication. We began by reviewing synchronous communication, where a service requests data from another and waits for an immediate response, resembling traditional function calls. While this approach is straightforward and beneficial for real-time data retrieval, it introduces complexities such as tight coupling, potential performance bottlenecks, and cascading failures if one service becomes unresponsive.

We then explored synchronous communication technologies such as RESTful HTTP communication and gRPC, exploring their roles in building robust APIs. As the cornerstone of web communication, HTTP is detailed for its simplicity, extensibility, and stateless nature, which align well with REST principles. RESTful APIs, leveraging HTTP, offer flexibility through JSON formatting, scalability with caching, and low-bandwidth consumption. We then provided a step-by-step guide on how ASP.NET Core creates a RESTful microservice in .NET, covering database integration, controller setup, and testing methodologies using tools such as VS Code's REST Client extension. After completing the creation of a RESTful microservice, we investigated using the HTTP client library of ASP.NET Core to facilitate synchronous communication with other RESTful microservices.

We then pivoted to gRPC and outlined its advantages over REST, highlighting features such as HTTP/2 support for multiplexing, compact binary serialization through Protobuf, and advanced communication patterns (e.g., streaming and bidirectional communication).

We then used ASP.NET Core to create a gRPC microservice. We went from defining the service contract in a proto file to implementing service logic and integrating with a RESTful API, illustrating how easy it is to communicate with a gRPC service.

By comparing REST and gRPC in terms of efficiency, scalability, and adaptability, the chapter equipped you with the knowledge to choose the appropriate communication protocol for various scenarios in microservices development.

Now, we can move on to asynchronous microservice communication.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





# 4

## Asynchronous Communication Between Microservices

In the previous chapter, we examined synchronous communication between microservices and explored tangible use cases. Now, we are going to examine asynchronous communication, which is pivotal in enabling scalability, resilience, and flexibility in distributed systems.

Unlike synchronous communication, which relies on direct, real-time interaction between services, asynchronous communication allows services to interact without requiring an immediate response. This decoupling of services ensures that each service can operate independently, enhancing the overall robustness of the system.

This chapter explores the foundational concepts, patterns, and technologies that underpin asynchronous communication in microservices architectures. We will also discuss the trade-offs associated with asynchronous communication. While it brings numerous benefits, it introduces complexities such as eventual consistency, message delivery guarantees, and the challenges of debugging distributed systems.

We will address how to mitigate these challenges using strategies such as idempotent message processing, message retry mechanisms, and robust logging and tracing tools.

By the end of this chapter, we will be able to do the following:

- Understand what asynchronous communication is and when we should use it
- Configure a message bus
- Implement pub-sub communication



This chapter provides practical guidance on implementing asynchronous communication using popular tools and frameworks. From RabbitMQ to cloud-native solutions such as Azure Service Bus, we will demonstrate how to select and leverage the right tools for your microservices ecosystem.

By the end of this chapter, you will be equipped with the knowledge and techniques to effectively integrate asynchronous communication into your microservices applications, enhancing their scalability, reliability, and performance.

## Technical requirements

The code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter04>.

You will also need to install **RabbitMQ** and **Erlang** to continue the coding activities in this chapter.

## Introducing asynchronous communication

Let us revisit some vital definitions and concepts before moving forward. There are two messaging patterns that we will employ between microservices:

- **Synchronous communication:** We covered this pattern in the previous chapter, where one service calls another directly and waits for a response. We use **HTTP** or **gRPC** for this kind of communication.
- **Asynchronous communication:** In this pattern, microservices communicate using messages, reducing the lag associated with synchronous communication. The service does not wait for a response when a message is sent.

There are times when one microservice needs to invoke a task in another to complete an operation, but the outcome of the task doesn't need to be known. An example is sending confirmation emails after our health facility management system has booked an appointment. Imagine the user waiting for the UI to complete its loading operation and show the confirmation. In between them clicking **Submit** and seeing the confirmation, the booking service needs to complete the following operations:

- Create an appointment record
- Send an email to the patient who booked the appointment
- Create a calendar entry for the system

Despite our best efforts, the booking service attempting to complete these operations will take some time and might lead to a less pleasing user experience. We could also argue that the responsibility of sending emails should not be native to the booking service. In the same way, calendar management should stand on its own. So, we could refactor the booking service's tasks to the following:

- Create appointment record
- Synchronously call the doctors service to get the doctor's details
- Synchronously call the patients service to get the patient's details
- Synchronously call an Email service to send an email to the patient who booked the appointment
- Synchronously call the calendar management service to create a calendar entry for the system

We have refactored the booking service to do fewer operations and offload the intricacies of the non-appointment booking operations onto other services. This is a good refactor, but we retain and potentially amplify the main flaw in this design. We will still wait to complete one potentially time-consuming operation before moving on to the next one, which runs the same risk. At this point, we can consider how important waiting on a response to these actions is relative to us entering the booking record in our database, which is the most crucial operation in terms of allowing the user to know the outcome of the booking process.

In this setting, we can use an asynchronous communication pattern to ensure that the major operation is completed and that the other operations, like sending the email and entering the calendar entry, can eventually happen without compromising our user experience.

Let us first explore strategies for handling asynchronous communication.

## Strategies for asynchronous services

Implementing asynchronous communication in a microservices application is pivotal for achieving scalability, resilience, and flexibility. By decoupling services and enabling non-blocking interactions, systems can handle varying loads and maintain responsiveness.

Asynchronous communication in microservices can be achieved through various patterns and protocols:

- **Message queues:** Message brokers facilitate communication between services. Services send messages to queues, which are then processed by consuming services. This approach decouples services and allows for flexible message handling. For example, Amazon **Simple Queue Service (SQS)** is a fully managed message queuing service that enables decoupled communication between distributed components. One benefit of these services is that they are **First-in-First-out (FIFO)** queues. They preserve the exact order of messages and ensure each message is processed once. FIFO queues are particularly useful in financial applications, order processing systems, or any scenario where the sequence of events must be preserved.

Other message queue implementations include **RabbitMQ**, **Azure Queue Storage**, and **ActiveMQ**, each offering varying features related to delivery guarantees, ordering, throughput, and integration capabilities.

- **Publish-subscribe (pub-sub) model:** Services publish messages to topics, and multiple subscribers receive these messages. This model is adequate for event-driven architectures where various services must react to specific events.
- **Event streaming:** Implement continuous streams of events that services can consume in real time. This method is suitable for applications requiring immediate processing of data streams.

Message queues enable decoupled communication between services by storing messages until consumers process them. Selecting the appropriate technology depends on specific application requirements, such as message throughput, latency, scalability, and integration capabilities. Evaluating these factors ensures that the chosen solution aligns with the system's architectural goals and operational constraints.

We will take a closer look at message queues next.

## Understanding message queues

A message queue is usually implemented as a bridge between two systems, a publisher and a consumer. When a publisher places messages in the queue, a consumer processes the information in the message as soon as it becomes available. Queues enforce a FIFO delivery method, so the processing order can always be guaranteed. This is usually implemented in a one-to-one communication so that a specific queue will be provisioned for each subscribing application.

Payment systems tend to use this pattern heavily, where the actual sequence of submitted payments matters, and they need to ensure the resilience of the instructions. If you think about it, payment systems generally have a very low failure rate. Most of the time, when we submit a payment request, we can rest assured that it will be completed successfully at some point in the future.

Figure 4.1 shows a publisher interacting with several message queues.

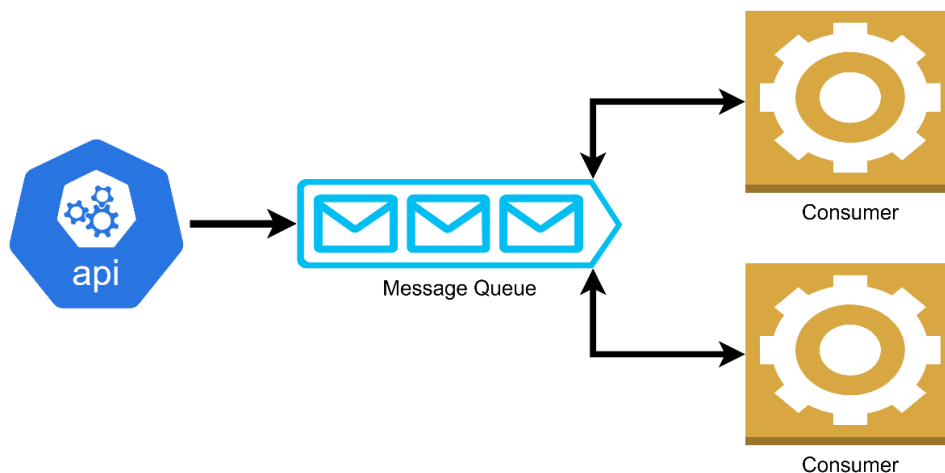


Figure 4.1 – A publisher communicating with multiple queues

With message queues, the consumer tends to work on a polling model. This means that the consumer is designed to check for messages based on a predefined interval, and when there is a message, it will attempt to process it. It is good practice to delete the message from the queue when there are multiple consumers to one queue, as this helps to prevent the same message from being grabbed by another consumer and potentially leading to double processing.

While message queues have their uses and bring a certain level of reliability to our system design, they can be inefficient and introduce some cons we might need to prepare to live with in a distributed system. There may also be situations where multiple consumers require a copy of the same message. In that event, we focus on a more distributed messaging setup, like a message bus. We will discuss this next.

## Understanding message bus systems

A *message*, *event*, or *service bus* provides interfaces where multiple or competing subscribers can process one published message. This is advantageous when publishing the same data to various applications or services. We do not need to connect to numerous queues to send a message; we can have one connection, complete one send action, and not worry about the rest.

Figure 4.2 shows a publisher interacting with a message that has several consumers.

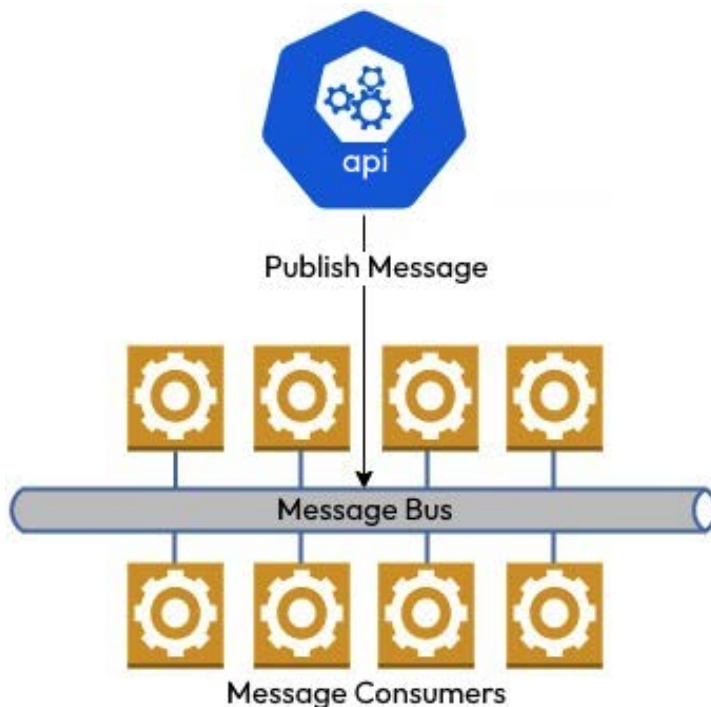


Figure 4.2 – Message bus with many consumers

Returning to our scenario where booking an appointment has several operational concerns, we can use a message bus to distribute the data to the relevant services and allow them to complete their operations on their own time. Instead of making direct calls to the other APIs, our appointment booking API will create a message and place it on the message bus. The email and calendar services are subscribed to the message bus, which processes the message accordingly.

This pattern has several advantages, including decoupling and application scalability. This aids in making the microservices even more independent from each other and reduces limitations associated with adding more services in the future. It also adds stability to the overall interaction of our services since the message bus acts as a storage intermediary for the data needed to complete an operation. If a consuming service is unavailable, the message will be retained, and the pending messages will be processed. If all services are running and messages are backing up, we can scale the number of instances of the services to reduce the message backlog more quickly.

You may encounter several types of messages, but we will focus on two in this chapter: **command** and **event** messages. Command messages request some action to be performed. So, our message to the calendar services sends instructions for creating a calendar entry. Given the nature of these commands, we can take advantage of this asynchronous pattern and have the messages be picked up eventually. That way, even large numbers of messages will be processed.

Event messages more simply announce that some action took place. Since these messages are generated after an action, they are worded in the past tense and can be sent to several microservices. In this case, the message to the email service can be seen as an event, and our email service will relay that information accordingly. This message type generally has just enough information to let the consuming services know what action was completed.

Command messages are generally aimed at microservices that need to post or modify data. This message must be consumed and actioned before the expected data will be available for some time. This is one of the key downsides to the asynchronous messaging model, termed **eventual consistency**. We will to explore this in more detail and discover the best approaches.

## Understanding eventual consistency

One of the biggest challenges in microservices design is data management and developing a strategy for keeping data in sync. Sometimes, this means having multiple copies of the same data in several microservice databases. **Eventual consistency** is a notion in distributed computing systems that accepts that data will be out of sync for some time. This type of constraint is only acceptable in distributed systems and fault-tolerant applications.

It is easy enough to manage one dataset in one database, as is the case with a monolithic application, since the data will always be up to date for any other part of the application to be able to access. A single-database approach to our application guarantees **ACID** transactions, which we discussed earlier, but we are still faced with the challenge of **concurrency** management. Concurrency refers to the fact that we might have multiple versions of the same data available at different points. This challenge is easier to manage in a single database application but presents a unique challenge in a distributed system.

It is reasonable to assume that when data changes in one microservice, data will change in another, leading to some inconsistency between the data stores for a period. The **CAP theorem** introduces the notion that we cannot guarantee all three of a distributed system's main attributes, that is, **consistency**, **availability**, and **partition tolerance**:

- **Consistency:** Every read operation against a data store will yield the current and most up-to-date version of the data, or an error will be raised if the system cannot guarantee it is the latest.
- **Availability:** Data will always be returned for a read operation, even if this is not the latest guaranteed version.
- **Partition tolerance:** This means that the systems will operate even when there might be transient errors that would stop a system under normal circumstances. Imagine if we have minor connectivity issues between our services and messaging systems. Data updates will be delayed, but this principle will suggest we must choose between **availability** and **consistency**.

Choosing consistency or availability is a significant decision moving forward. Since microservices are generally always available, we must be careful with our choices and how strictly we impose our constraints around the system's consistency policy: eventual or strong.

There are scenarios where strong consistency is optional since all the work performed by an operation is completed or rolled back. These updates are either lost (if rolled back) or propagated to the other microservices on their own time without having a detrimental effect on the application's overall operations. If this model is selected, we can gauge our user experience through sensitization and letting them know that updates are only sometimes immediate across the different screens and modules.

Using the pub-sub model is the number one way to implement this kind of event-driven communication between services, where they all communicate via a messaging bus. After each operation is completed, each microservice publishes an event message to the message bus, which other services eventually pick up and process.

We usually use event-driven communication using a pub-sub model to implement eventual consistency. When data is updated in one microservice, you can publish a message to a central messaging bus, and other microservices that have copies of the data can receive a notification by subscribing to the bus. Because the calls are asynchronous, the individual microservices can continue to serve requests using the copy of the data that they already have, and the system needs to tolerate that while it might not be consistent right away, meaning that the data may not all be in sync immediately, eventually it will be consistent across the microservices. Of course,

implementing eventual consistency can be more complex than just firing and forgetting messages to a message bus. Later in this book, we'll look at ways to mitigate the risks.

We have now explored using messaging via queues and the challenges of coping with eventual consistency. Let us focus on the pub-sub pattern, the primary method of ensuring services consume and act upon messages.

## Understanding the pub-sub pattern

The pub-sub pattern has gained considerable popularity and acclaim and is widely used in distributed systems. Essentially, this pattern revolves around publishing data, contextually called **messages**, to an intermediary messaging system (often referred to as an **event bus** or **message broker**), which can be described as resilient and reliable, and then having subscribing applications monitor this intermediary system. Subscribers express interest in specific event types and receive relevant messages from the broker. This decoupling enhances system modularity and scalability, as services can evolve independently without direct dependencies.

Implementing the pub-sub pattern involves the following:

- **Event definition:** Creating event classes that encapsulate the data to be transmitted
- **Publishing events:** Services generate and send events to the message broker
- **Subscribing to events:** Services register to receive specific events from the broker
- **Event handling:** Upon receiving events, subscribers execute appropriate business logic

**Event definition** refers to the precise specification of events that services emit to signify significant state changes or actions. These events are the primary communication between decoupled services, enabling them to react to changes in other system parts without direct dependencies. For instance, a new appointment booking can be defined as an event. This event will trigger the creation and disbursement of one or more messages containing the appropriate information.

Key components of an event definition include the following:

- **Event schema:** This is a structured representation of the event's data, detailing the attributes and their respective data types. For instance, an `AppointmentCreated` event might include attributes such as `AppointmentId`, `PatientId`, and `DoctorId`. Defining an explicit schema ensures that all services interpreting the event consistently understand its structure and content.
- **Event type:** This categorizes the nature of the event. Explicitly naming event types allows services to subscribe to and handle only the events relevant to their operations.



- **Event versioning:** As systems evolve, event structures may need modifications. Implementing versioning strategies ensures backward compatibility, allowing services to process events according to their support version, thereby preventing disruptions.
- **Event metadata:** Supplementary information, such as timestamps, source identifiers, and correlation IDs, provides context about the event's origin and relationship to other events or processes. This metadata is crucial for tracing event flows and debugging.

**Publishing events** involves a service emitting notifications about significant state changes or occurrences to inform other services within the system. When a noteworthy action occurs, the responsible service generates an event encapsulating relevant data. This event typically includes details such as the event type, timestamp, and associated metadata. The generated event is dispatched to an event bus or message broker, and subscriber services that have expressed interest in specific event types receive the published events from the event bus. A service registers interest in particular event types with an event broker or bus. This registration ensures that the service is notified when relevant events occur. Upon receiving a subscribed event, the service executes predefined business logic to process the event's data. This processing can involve updating the internal state, triggering additional workflows, or emitting new events. This summarizes what **subscribing to events** involves.

It is crucial to understand the trade-offs associated with *message delivery guarantees*. These guarantees define how often a message may be delivered to a consumer and directly impact system reliability, performance, and complexity.

There are three common types of delivery guarantees:

- **At-most-once delivery:** A message is delivered *only once*. It may be lost, but it will never be duplicated. This is useful in systems where occasional message loss is acceptable or downstream services are idempotent and re-processing is not viable. This model favors performance over reliability. The messaging system does not include retry logic.
- **At-least-once delivery:** A message is delivered *one or more times*. It will not be lost, but duplicates may occur. This is used when every message must be processed, such as in billing systems or order processing workflows. Consumers must implement *idempotency* to handle duplicate messages gracefully. This is the most used guarantee in distributed systems. Many cloud-native messaging services, such as Azure Service Bus, Amazon SNS/SQS, and Apache Kafka, offer delivery at least once by default.
- **Exactly once delivery:** A message is delivered *only once*, with no duplicates or loss. Example usage is in high-stakes financial transactions, mission-critical command delivery, or other domains where duplicates or loss would cause systemic issues. This is the most challenging

guarantee to implement, often requiring distributed transaction support or advanced coordination mechanisms between the broker and consumer. Few systems offer actual exactly once semantics. Kafka provides *exactly once processing* through *idempotent producers*, *transactional writes*, and *consumer offset management*.

Finally, **event handling** refers to the processes and mechanisms by which services respond to events. Essentially, these are notifications that signify changes in state or the occurrence of specific actions within the system. Effective event handling ensures services react appropriately to events, maintaining system coherence and enabling responsive behaviors.

## Implementing asynchronous communication

Now that we better understand why services might communicate asynchronously and the patterns that may be used, we can modify our existing appointment service to communicate with a notification service to dispatch appointment creation emails.

For this implementation, we will make the following changes:

- Install and configure a service bus using RabbitMQ
- Modify the appointments service to produce messages
- Implement a notifications service to consume and process messages

Since we need an intermediary service to facilitate our messaging, we will start by reviewing our options.

## Choosing a message bus

As mentioned previously, a message bus is a communication mechanism used in distributed systems to facilitate the exchange of messages between decoupled services, components, or applications. When selecting a messaging broker for .NET microservices, consider scalability requirements, integration capabilities, operational complexity, and cost to determine the most suitable option for your application's needs. Some alternatives and their pros and cons include the following:

- **Apache Kafka:** A distributed streaming platform for high-throughput and low-latency message processing:
  - **Pros:** Handles large volumes of data efficiently; suitable for real-time analytics and event streaming
  - **Cons:** More complex to set up and manage; may be overkill for simple messaging needs

- **Azure Service Bus:** A fully managed enterprise message broker with rich features:
  - **Pros:** Seamless integration with Azure services; supports advanced messaging scenarios
  - **Cons:** Tied to the Azure ecosystem; potential cost considerations
- **Amazon SQS:** AWS's fully managed message queuing service:
  - **Pros:** Scalable and easy to use; integrates well with other AWS services
  - **Cons:** Limited to queue-based messaging; lacks some advanced features of other brokers
- **NATS:** A lightweight, high-performance messaging system:
  - **Pros:** Simple design; low latency; suitable for cloud-native applications
  - **Cons:** Limited persistence options; fewer features than more comprehensive brokers
- **ActiveMQ:** An open source message broker that supports multiple protocols:
  - **Pros:** Supports a wide range of protocols; flexible and mature
  - **Cons:** Can be resource-intensive; may require more maintenance
- **RabbitMQ:** An open source message broker that facilitates asynchronous communication between distributed systems by implementing several protocols:
  - **Pros:** RabbitMQ supports multiple protocols and is highly interoperable
  - **Cons:** Not as performant for higher workloads and does not support event streaming

For this activity, we will proceed with RabbitMQ, known for its straightforward installation and configuration. It provides an intuitive management interface accessible to developers and administrators. It also excels in complex routing scenarios and supports various messaging patterns, including one-to-one, one-to-many, and pub-sub models. Finally, it boasts a large, active community and extensive documentation, facilitating troubleshooting and knowledge sharing.

It enables applications to exchange messages reliably and efficiently, decoupling the sender and receiver, which enhances scalability and resilience. The major components that sum up how it works are as follows:

- **Producers and consumers:** Applications that send messages are called producers, while those that receive messages are consumers
- **Exchanges and queues:** Producers send messages to exchanges, which route them to queues based on defined rules. Consumers then retrieve messages from these queues
- **Bindings:** Associations between exchanges and queues that determine how messages should flow
- **Message routing:** RabbitMQ supports various exchange types (direct, topic, fanout, headers) to control message routing mechanisms

Now that we have reviewed our options and settled on using RabbitMQ as our message bus, let us review the setup steps.

## Installing RabbitMQ

To install RabbitMQ, go to its downloads page (<https://www.rabbitmq.com/docs/download>) and select the version that best suits your operating system's needs. Note that RabbitMQ depends on Erlang (<https://www.erlang.org/downloads>), and you must ensure this is also installed.

For a platform-agnostic experience, you may use the Docker (<https://docs.docker.com/get-started/get-docker/>) image. Docker can be installed relatively quickly and is available for all operating systems.

I will proceed with the Docker option, as the RabbitMQ image encapsulates the libraries and dependencies (like Erlang) that RabbitMQ needs to run. After installing and running Docker, open your command prompt window and use the following command to download the image and initialize a RabbitMQ container:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672
rabbitmq:4.0-management
```

This retrieves the latest (at the time of writing) RabbitMQ 4.0.x image and then runs the container. It also adds the default RabbitMQ port mapping to your machine. This is the same port that will be used regardless of the preferred installation method.

Once your setup is finished, ensure that RabbitMQ (native) or the RabbitMQ container (Docker) is running and navigate to `http://localhost:15672/`. You can authenticate using `guest` as the username and password values. This will launch the management dashboard, as seen in *Figure 4.3*. These are the default credentials whether you use the native or container versions.

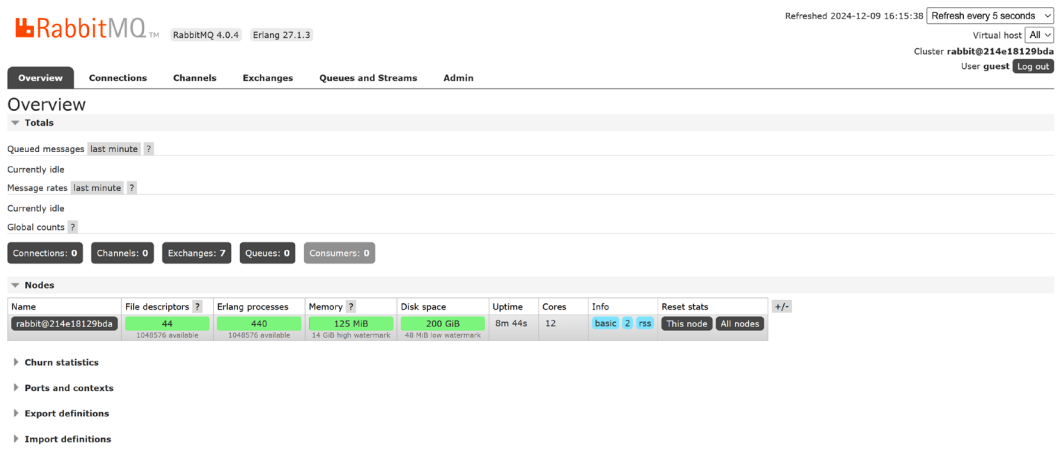


Figure 4.3 – RabbitMQ management dashboard

The **RabbitMQ management plugin** provides this user-friendly web interface. It enables administrators to efficiently monitor and manage various aspects of RabbitMQ servers and clusters. This dashboard, accessible via a web browser, offers the following:

- Real-time metrics on message rates, queued messages, and node statuses allow administrators to assess the system’s health
- Detailed information on each queue, including message counts (ready and unacknowledged), consumers, and bindings
- Insight on all active connections and channels, providing details such as the connected client, protocol, and associated channels

While the management dashboard is a powerful tool for monitoring and managing RabbitMQ, it is recommended that you integrate RabbitMQ with external monitoring tools for long-term monitoring and alerting.

Now that we have configured our messaging system, let us modify our appointment service to dispatch messages as needed.

## Implementing a publisher

Now that our messaging system is configured, we need to modify the appointments service to place a message in a queue each time an appointment is made. This message must contain enough information that the consuming service can use to complete the email operation appropriately.

We will use a package called `MassTransit` to help with this operation. `MassTransit` is an open source package that simplifies the development of message-based applications. It provides a consistent abstraction over various message transports, including `RabbitMQ`, `Azure Service Bus`, and `Amazon SQS`.

We can start by adding the required packages to the appointments service by running these commands:

```
dotnet add package MassTransit
dotnet add package MassTransit.AspNetCore
dotnet add package MassTransit.RabbitMQ
```

Next, we can define a class representing a message as a JSON object that will be created and placed on the queue. The message will contain enough information for the consumer to complete the process. `MassTransit` requires message types to be classes, records, or interfaces:

```
public class AppointmentCreated
{
    public Guid AppointmentId { get; set; }
    public Guid PatientId { get; set; }
    public Guid DoctorId { get; set; }
    public DateTime AppointmentDate { get; set; }
}
```

To configure `MassTransit` to use `RabbitMQ`, we will make the following modifications to the `Program.cs` file:

```
using MassTransit;
var builder = WebApplication.CreateBuilder(args);
// Other service configurations
builder.Services.AddMassTransit(x =>
{
    x.UsingRabbitMq();
});
// Other service configurations
```

The additional benefit of this configuration is that we can now inject the `IPublishEndpoint` service in any class that needs to publish a message to RabbitMQ. Since we will do this from our controller, we can modify the controller and POST action like this:

```
public class AppointmentsController : ControllerBase
{
    /* Other fields */
    private readonly IPublishEndpoint _publishEndpoint;

    public AppointmentsController(/* Other services */,
        IPublishEndpoint publishEndpoint)
    {
        _publishEndpoint = publishEndpoint;
    }

    [HttpPost]
    public async Task<ActionResult<Appointment>>
        PostAppointment(Appointment appointment)
    {
        /* Process appointment and insert in database */
        // Publish AppointmentCreated to RabbitMQ
        await _publishEndpoint.Publish<AppointmentCreated>(new
        {
            appointment.AppointmentId,
            appointment.PatientId,
            appointment.DoctorId,
            appointment.Slot.Start
        });
        return CreatedAtAction("GetAppointment", new { id =
            appointment.AppointmentId }, appointment);
    }
    /* Other endpoints */
}
```

We have modified our POST method to persist the incoming appointment in the database and call the generic `Publish` method on the `IPublishEndpoint` interface. We use our `AppointmentCreated` class to define the type of event we will publish.

We have configured our service to produce messages and place them on a RabbitMQ exchange. Now, we can configure the consumer service to receive the messages and complete the asynchronous processing.

## Implementing a consumer

Our consumer service for this activity will notify the related patient anytime an appointment has been created. This service must subscribe to RabbitMQ and respond to incoming messages accordingly. It must also synchronously communicate with the Doctor and Patient APIs to retrieve contact details.

By adding `MassTransit` to our `AppointmentsApi`, we can automatically monitor a message queue. It can handle incoming messages from a queue without requiring a separate service. It is excellent because we can retrofit the existing service to dispatch notifications in the background. Using this service to manage appointments and dispatch notifications can be seen as violating the principles governing microservices since this service will have two functions. It is also a gray area since the notifications are appointment-related, and we want to ensure they are dispatched asynchronously.

If all services require notifications to be dispatched, then a central `NotificationsService` would be a more appropriate approach. We will modify the `AppointmentsApi` without complicating the situation to monitor the RabbitMQ exchange and dispatch emails as needed.

If the consumer and publisher are in different projects or solutions, a shared library must be a central reference point for the reliant services. The same namespace must be used for both the publisher and consumers.

We need to define a consumer class that implements `IConsumer<T>`, where `T` is the type of message you expect to receive in this case. `T` is `AppointmentCreated`. We will call this consumer class `AppointmentCreatedConsumer` and implement it like this:

```
public class AppointmentCreatedConsumer(IEmailService _emailService,
    PatientsApiClient patientsApiClient, DoctorsApiClient
    doctorsApiClient) : IConsumer<AppointmentCreated>
{
    public async Task Consume(ConsumeContext<AppointmentCreated> context)
    {
        var message = context.Message;

        Console.WriteLine($"Retrieve Doctor details");
        Console.WriteLine($"Send Email to doctor ");
    }
}
```



```

        var doctor = await
            doctorsApiClient.GetDoctorAsync(message.DoctorId);

        Console.WriteLine($"Retrieve patient details");
        Console.WriteLine($"Send Email to patient ");
        var patient = await
            patientsApiClient.GetPatientAsync(message.PatientId);

        var emailContent = $"Dear {patient.FirstName}
            {patient.LastName},\n\n Your appointment with Dr.
            {doctor.FirstName} {doctor.LastName} is confirmed for
            {message.AppointmentDate}.\n\n Best regards,
            \nHealth Care Management System";

        await _emailService.SendEmailAsync(patient.Email,
            "Appointment Confirmation", emailContent);
    }
}

```

We are implementing the `IConsume<T>` interface, which requires defining the `Consume` method. Once a message is received, it will execute the contained code, where we retrieve the doctor and patient details, formulate the email, and dispatch it using our email service. This email service uses `MailKit` to handle the email operation and is defined like this:

```

using MailKit.Net.Smtp;
using MimeKit;
using System.Threading.Tasks;

public interface IEmailService
{
    Task SendEmailAsync(string to, string subject, string body);
}

public class EmailService : IEmailService
{
    public async Task SendEmailAsync(string to, string subject,
        string body)
    {

```

```

        var message = new MimeMessage();
        message.From.Add(new MailboxAddress("HealthCare System",
            "no-reply@healthcare.com"));
        message.To.Add(new MailboxAddress("Patient", to));
        message.Subject = subject;
        message.Body = new TextPart("plain") { Text = body };

        using var client = new SmtpClient();
        await client.ConnectAsync("smtp.mailserver.com", 587, false);
        await client.AuthenticateAsync("smtp_username", "smtp_password");
        await client.SendAsync(message);
        await client.DisconnectAsync(true);
    }
}

```

Sensitive details such as `smtp_username` and `smtp_password` should be abstracted to the configuration or secrets file. They are referenced here for simplicity's sake.

Now that we have configured our consumer and email service, we can finally modify `Program.cs` to register the email service and modify the MassTransit config to look like this:

```

// Other service configurations
builder.Services.AddTransient<IEmailService, EmailService>();
builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<AppointmentCreatedConsumer>();
    x.UsingRabbitMq((context, cfg) =>
    {
        // Configure the receive endpoint
        cfg.ReceiveEndpoint("appointment_created_queue", e =>
        {
            e.ConfigureConsumer<AppointmentCreatedConsumer>(context);
        });
    });
});
// Other service configurations

```

With these modifications, we can verify that calling the API’s POST method will affect the database and place a message on a queue in RabbitMQ. The consumer will attempt to process the message as soon as it is available. *Figure 4.4* shows the portal after it has received an AppointmentCreated message.

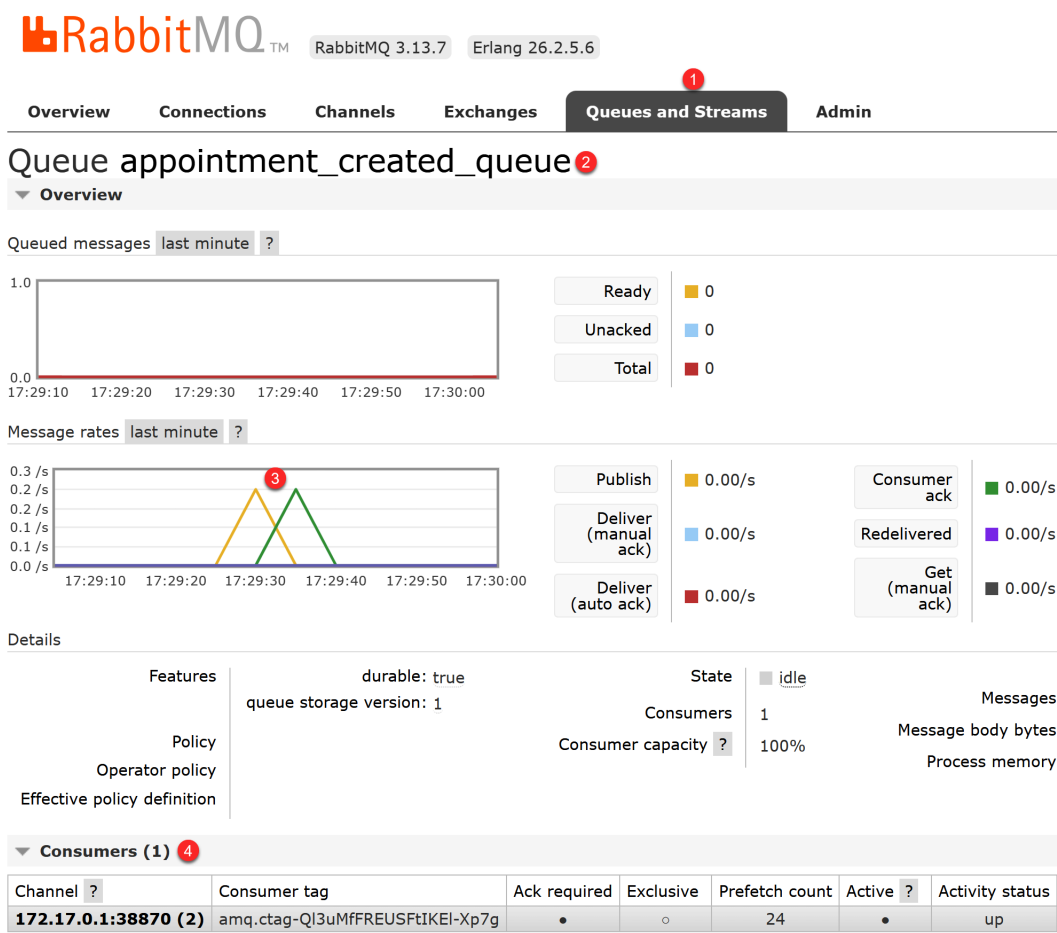


Figure 4.4 – Automatic creation of a message exchange and queue to store the newly created message

This completes a standard pub-sub implementation. Remember, your design decisions are crucial and heavily depend on your project’s needs. It is standard practice to create a microservice that distributes alerts and notifications on behalf of the other microservices. This method reduces the responsibilities of the individual services and allows for truly asynchronous data processing, putting you at the forefront of your project’s success.

As mentioned before, asynchronous communication introduces several challenges and cross-cutting concerns. Understanding and addressing these challenges is key to successful implementation. We will discuss some of them and equip you with possible ways to address them next.

## Challenges and possible solutions

Implementing a pub-sub architecture offers significant benefits in decoupling system components and enhancing scalability. However, it also presents several challenges impacting system performance, reliability, and maintainability.

One primary challenge is ensuring message ordering. In pub-sub systems, messages may not arrive in the sequence they were sent. This disorder can lead to inconsistencies if messages need to be processed in a particular sequence. To address this, developers can include sequence numbers or timestamps within the message payload, allowing subscribers to reorder messages upon receipt. We will investigate a solution to this next.

### Message ordering

Implementing such sequencing requires careful design to ensure that all messages are appropriately tagged and that subscribers have the logic to reorder them correctly. First, ensure that the `AppointmentCreated` event includes a property for the sequence number. This allows each message to carry information about its position in the sequence. We will introduce a timestamp for this example:

```
public class AppointmentCreated
{
    // Shortened for brevity.
    public DateTime Timestamp { get; set; }
}
```

Ensure you set a valid time value when creating the `AppointmentCreated` message to be placed in the queue:

```
await _publishEndpoint.Publish<AppointmentCreated>(new
{
    // Other assignments
    DateTime.UtcNow
});
```

When creating a new AppointmentCreated message, set Timestamp to the current UTC to ensure consistency across different systems. In the consumer, maintain a record of the last processed timestamp for each appointment. Before processing a new message, check whether its timestamp is later than the last processed one to ensure messages are handled in the correct order. I advise storing this value using non-volatile storage (file or database) to ensure it is never lost when the service restarts. For simplicity, we will use a local variable. We can modify the Consume method to look something like this:

```
// Dictionary to track the last processed timestamp for each appointment
private static readonly ConcurrentDictionary<Guid, DateTime>
    LastProcessedTimestamps = new();

public async Task Consume(ConsumeContext<AppointmentCreated> context)
{
    var message = context.Message;
    // Retrieve the last processed timestamp for this appointment
    var lastTimestamp =
        LastProcessedTimestamps.GetOrAdd(message.AppointmentId,
            DateTime.MinValue);

    // Check if the message is newer than the last processed message
    if (message.Timestamp > lastTimestamp)
    {
        // Process message and send email.
        // Shortened for brevity.
        // Update the last processed timestamp
        LastProcessedTimestamps[message.AppointmentId] =
            message.Timestamp;
    }
    else
    {
        // implement logic to handle out-of-order messages, such as
        // logging or storing for later processing
    }
}
```

Decide on a strategy for managing out-of-order messages. You might implement a retry mechanism or temporarily store them for reordering. Limiting concurrency and setting a low prefetch count can help maintain order but may impact throughput. Evaluate the trade-off between message ordering and performance based on your application's requirements. MassTransit can be modified to assist with this. The following lines can be added to the configuration in `Program.cs`:

```
builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<AppointmentCreatedConsumer>();
    x.UsingRabbitMq((context, cfg) =>
    {
        // Configure the receive endpoint
        cfg.ReceiveEndpoint("appointment_created_queue", e =>
        {
            e.PrefetchCount = 1; // Fetch one message at a time
            e.UseConcurrencyLimit(1); // Process one message at a time
            e.ConfigureConsumer<AppointmentCreatedConsumer>(context);
        });
    });
});
```

Another significant issue is message duplication. The same message may get delivered more than once, leading to redundant processing and potential data anomalies. Let us review possible mitigations for this next.

## Duplicate messages

Unreliable network conditions can lead to message duplication. For instance, if a message acknowledgment from the consumer to the broker is lost due to a network issue, the broker may assume the message was not processed and resend it. Similarly, if the consumer fails to acknowledge within a specified timeout period, the broker might deliver the message again. Many messaging systems prioritize ensuring messages are delivered at least once, even if it means occasional duplicates. This approach favors reliability over strict single delivery.

To mitigate this, it's essential to design **idempotent** message handlers. These are message handlers that can process the same message multiple times without causing any adverse effects. Alternatively, incorporating unique message identifiers enables subscribers to track and discard duplicates effectively. Implementing idempotency or duplicate detection mechanisms adds complexity to the system but is crucial for maintaining data integrity.

Another way to manage this issue is to introduce a message ID tracked in a database or cache. This is checked each time to appropriately identify and deal with a potential duplicate. In our case, we can add the appointment ID to the AppointmentCreated message and track the processed messages:

```
public class AppointmentCreated
{
    // Shortened for brevity
    public Guid MessageId { get; set; }
}
```

When creating a new AppointmentCreated message, assign the Guid value from the database record to the MessageId property to ensure uniqueness:

```
await _publishEndpoint.Publish<AppointmentCreated>(new
{
    // Other assignments
    MessageId = appointment.AppointmentId
});
```

In the consumer, maintain a record of processed message IDs to detect and ignore duplicates. Once again, storing the IDs using a database or cache is best, but we will use ConcurrentDictionary for this example. Our Consume method will check whether the ID of the current message is in the list and stop execution if true:

```
private static readonly ConcurrentDictionary<Guid, bool>
    ProcessedMessageIds = new();

public async Task Consume(ConsumeContext<AppointmentCreated> context)
{
    var message = context.Message;
    // Check if the message has already been processed
    if (ProcessedMessageIds.ContainsKey(message.MessageId))
    {
        Console.WriteLine($"Duplicate message detected:
            {message.MessageId}. Ignoring.");
        return; // Exit without processing
    }

    // Proceed with processing the message
}
```

```
// Mark the message as processed  
ProcessedMessageIds[message.MessageId] = true;  
}
```

Dealing with duplicate messages can significantly reduce the processing time between message consumption and increase efficiency.

Some other issues might arise; we will look at them at a high level next.

## Additional considerations

The more a system grows, the more it becomes distributed. At this point, the issues we face and must deal with cannot all be solved by code. Although we are developers, we must always consider the infrastructural implications of our development efforts. Some of these infrastructural challenges include the following:

- **Scalability:** As the number of publishers and subscribers grows, the system may encounter performance bottlenecks, resulting in increased latency or message loss. It is advisable to employ distributed message brokers that support load balancing and horizontal scaling to enhance scalability. Partitioning topics and implementing consumer groups can also distribute the load more evenly. However, these solutions require a robust infrastructure and careful configuration to operate effectively under varying loads.
- **Fault tolerance:** Failures in any part of the pub-sub system can lead to message loss or system downtime. Implementing redundancy and failover mechanisms, such as having multiple instances of publishers and subscribers and using persistent storage for messages, can mitigate these risks. While these measures improve reliability, they also introduce additional complexity and resource requirements.
- **Security:** Unauthorized access to messages can result in data breaches, compromising sensitive information. Implementing robust authentication and authorization mechanisms is essential to control access to topics. Encrypting messages in transit and at rest further protects data from unauthorized access. However, these security measures can introduce performance overhead and require ongoing management to handle keys and access controls.
- **Logging and tracing:** Due to its asynchronous and distributed nature, monitoring and debugging in a pub-sub system can be complex. Tracking message flow and diagnosing issues require comprehensive logging and monitoring tools. Implementing correlation IDs can help trace message paths across components, facilitating easier debugging. Setting up such monitoring infrastructure necessitates additional development effort and can increase system complexity.



While the pub-sub architecture provides a flexible and scalable communication model, it introduces challenges that require careful consideration and implementation of appropriate solutions. Addressing these challenges is vital for building a robust and efficient pub-sub system. Next, let us summarize all that we have learned in this chapter.

## Summary

This chapter provided an in-depth examination of asynchronous communication in microservices architecture, emphasizing its importance for scalability, resilience, and flexibility. It contrasted synchronous communication, where services interact directly and require immediate responses, with asynchronous communication, which allows services to operate independently, improving system robustness.

We began with the challenges of synchronous designs, such as user experience delays and tightly coupled service responsibilities, and introduced asynchronous patterns as solutions. Scenarios such as booking systems in health management applications demonstrate the practicality of offloading non-critical operations such as email notifications or calendar entries to asynchronous workflows. By focusing on key operations, such as booking appointments and handling other tasks asynchronously, the system ensures responsiveness and a better user experience.

Key strategies for asynchronous communication were outlined, including message queues, pub-sub models, and event streaming. Each pattern was explained with use cases: message queues are pivotal for decoupled communication with guaranteed order, pub-sub models enable event-driven architectures, and event streaming suits real-time data processing. We used RabbitMQ and MassTransit to complete step-by-step examples of how publishers and consumers can be developed to integrate systems such as appointment notifications and email services.

Finally, we reviewed the significant challenges of asynchronous systems, such as message ordering, duplication, and eventual consistency. Solutions include using sequence numbers, timestamps, and unique identifiers to deduplicate messages and ensure order. The chapter also covered infrastructural considerations such as scalability, fault tolerance, and security. For instance, RabbitMQ was recommended for its robust features, including exchange-to-queue bindings and support for advanced routing patterns. Alternatives such as Apache Kafka and Azure Service Bus were briefly compared, noting their respective strengths in high-throughput processing and cloud integration.

After completing this chapter, you should have the theoretical and practical knowledge to integrate asynchronous communication into your microservices and improve scalability and reliability.

Now that we have covered two ways that microservices might communicate, let's explore the Aggregator pattern, where we seek to help a UI manage multiple inter-service dependencies.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW





# 5

## Working with the Aggregator Pattern

As interactions between each discrete service in your microservices application become complex, the challenges of handling data retrieval and aggregation become increasingly intricate.

Consider a scenario where a client application requires information that spans multiple services, such as user profiles, order histories, and product recommendations. Without a cohesive strategy, the client would need to initiate separate requests to each service, manage the responses, and handle any errors that arise. This approach increases the complexity of client-side logic and leads to inefficiencies, including heightened latency and increased network overhead.

The Aggregator pattern offers an elegant solution to this challenge. It introduces an intermediary service consolidating data from various microservices and presenting a unified response to the client. This abstraction simplifies client interactions, reduces the number of network calls, and centralizes error handling, enhancing overall system performance. Variations of this pattern include API gateways and **Backend for Frontend (BFF)** patterns, which seek to condense data from multiple microservices and package it for dissemination to one or more clients.

In this chapter, we will delve into the nuances of the Aggregator pattern in .NET microservices development. We will explore:

- The theory behind the Aggregator pattern
- Implementing the Aggregator pattern
- Best practices and optimizations

By the end of this chapter, you will have a comprehensive understanding of how to leverage the Aggregator pattern to streamline service interactions and enhance the functionality of your microservices-based applications. We will begin by reviewing the pattern and its pros and cons.

## Technical requirements

Code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET--Second-Edition/tree/main/Chapter05>.

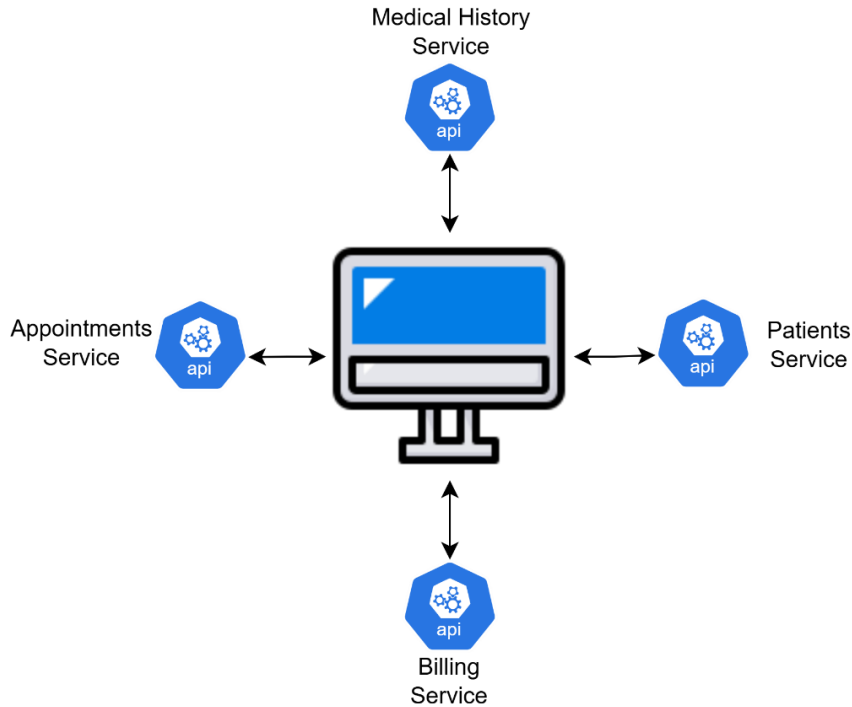
## What is the Aggregator pattern?

The Aggregate pattern is a powerful tool in microservices architecture that facilitates data consolidation from multiple services into a single response. By implementing this pattern, we can create efficient, scalable, and user-friendly APIs that abstract the complexities of underlying microservices, improving system performance and simplifying client interactions.

Our healthcare management systems will access data from several services with each UI request. Consider a UI that allows a clinician to access a patient's comprehensive profile, including personal information, medical history, current medications, upcoming appointments, and billing details. Each of these data segments might reside in separate services:

- **Patient service:** Manages personal and demographic information.
- **Medical history service:** Stores past medical records and treatment history.
- **Appointment service:** Handles scheduling and appointment details.
- **Billing service:** Invoices and payment information.

*Figure 5.1* depicts how a UI might attempt to call several services.



*Figure 5.1 – A UI communicating with multiple services while loading one page*

When a user accesses the patient dashboard, the UI initiates requests for each service. These requests must occur in parallel or sequentially, depending on the architecture, to avoid delays. For example, the flow of requests made by the UI might look like this:

1. Send a GET request to the Patient Service API (`/api/patient/{id}`).
2. Send a GET request to the Medical History Service API to retrieve the patient's medical records (`/api/medical-history/{patientId}`).
3. Send a GET request to the Appointments Service API to retrieve the patient's appointments (`/api/appointments/{patientId}`).
4. Send a GET request to the Billing Service API to retrieve the patient's payment information (`/api/billing/{id}`).

Also, remember that each request might require specific authentication tokens, headers, and query parameters tailored to the service being accessed. This pattern is reminiscent of the synchronous communication patterns we explored in a previous chapter. Given the number of network roundtrips, each service call introduces some lag and latency, and recall that they all add up to impact the UI's performance.

Now that we understand how a client application might need to interact with several services, let's review how this implementation might look in a Blazor WebAssembly application.

## Developing a UI without an aggregator

We will use a **Blazor WebAssembly (Blazor WASM)** application to demonstrate how a UI must interact with several services. Any technology can be used to develop a frontend, each with its unique set of pros and cons. Blazor is a modern frontend web framework that leverages HTML, CSS, and C# to accelerate web app development. It enables you to create reusable components that run on both the client and server, delivering fast, dynamic, and engaging web experiences.

We will discuss more details of preferred and recommended UI frameworks later in this book, but for now, Blazor allows us to:

- Run entirely in the browser, providing fast and interactive user interfaces without requiring constant communication with the server.
- Develop a **Single-Page Application (SPA)** that enables smooth navigation and reduces full-page reloads, resulting in a better user experience.
- Develop for any modern browser without needing plugins. WASM runs on Windows, macOS, Linux, and mobile platforms.
- Call REST APIs or gRPC endpoints directly.

Let us create a new Blazor WASM project using the following command:

```
dotnet new blazorwasm -n HealthcareDashboard
cd HealthcareDashboard
```

Since the app will call multiple REST APIs, we must add `HttpClient` using dependency injection. Use this command to add the project dependency:

```
dotnet add package Microsoft.Extensions.Http
```

We can then add all the endpoints for the services that the UI must use. By default, this project type does not come with an `appsettings.json` file. So, in the `wwwroot` folder, create a new file called `appsettings.json` and define the base URLs for the microservices like this:

```
{
  "ApiEndpoints": {
    "AppointmentsApi": "https://appointments.api/",
    "PatientsApi": "https://patients.api/",
    "MedicalHistoryApi": "https://medicalhistory.api/",
```

```
    "BillingApi": "https://billingdetails.api/"  
  }  
}
```

Remember to change the API addresses to the actual address of your service. We implement service client classes for each microservice to promote maintainability, scalability, and modularity. In the project root, create a Services folder and add the implementation for each service. Here is an example of the PatientServiceClient class:

```
public record Patient(  
    Guid PatientId,  
    string FirstName,  
    string LastName,  
    DateTime DateOfBirth,  
    string Gender,  
    string ContactNumber,  
    string Email  
);  
  
public class PatientServiceClient(HttpClient httpClient)  
{  
    private readonly HttpClient _httpClient = httpClient;  
  
    public async Task<Patient?> GetPatientAsync(Guid patientId)  
    {  
        return await  
            _httpClient.GetFromJsonAsync<Patient>($"/{patientId}");  
    }  
}
```

In Program.cs, register HttpClient instances for each service and inject the client classes:

```
var config = builder.Configuration.GetSection("ApiEndpoints");  
  
// Register HttpClient for each service  
builder.Services.AddHttpClient<PatientServiceClient>(client =>  
    client.BaseAddress = new Uri(config["ApiEndpoints:PatientsApi"]));  
// Other service client registrations
```



Now that we have wired up our services and clients, we can implement the user interface. In the Pages folder, create a Dashboard.razor component and add the following code:

```
@page "/dashboard/{PatientId:Guid}"
@using HealthcareDashboard.UI.Services
@inject PatientServiceClient PatientService
@* Other services to inject *@

<h3>Patient Dashboard</h3>
@if (isLoading)
{
    <p>Loading data...</p>
}
else
{
    <div>
        <h4>Patient Full Name</h4>
        <p>@patient?.FirstName @patient?.LastName</p>
        <h4>Patient Profile</h4>
        <p><strong>Gender:</strong> @patient?.Gender</p>
        <p><strong>Email:</strong> @patient?.Email</p>
        <p><strong>Contact:</strong> @patient?.ContactNumber</p>
        @* Additional profile details *@
    </div>
}

@code {
    [Parameter]
    public Guid PatientId { get; set; }
    private Patient? patient;
    @* Additional properties *@
    private bool isLoading = true;

    protected override async Task OnParametersSetAsync()
    {
        try
        {
            isLoading = true;
```

```
        patient = await PatientService.GetPatientAsync(PatientId);
        /* More synchronous service calls via service clients */
    }
    catch (Exception ex)
    {
        throw;
    }
    finally
    {
        isLoading = false;
    }
}
```

This component can be navigated to via the URL template /Dashboard/{Patient ID as GUID}. Once the parameter value is detected, the `OnParametersSetAsync` method executes several API calls to retrieve the information from multiple APIs. The UI will be loading until this entire operation is completed. As mentioned before, if one of the services has an issue, the whole UI will be stuck for a while.

Another thing to consider is that the UI could be responsible for too many things. We need to consider how the UI should handle if one of the service calls fails. Do we fail the entire load operation or complete a partial load?

Another primary concern is how tightly coupled the UI is to the services. We will need to modify several aspects of the UI code with each modification or additional service. Imagine if each service needs additional headers and has specific security requirements.

Introducing an **Aggregator service** can offload these responsibilities. We will discuss this further in the next section.

## Introducing the Aggregator pattern

The UI would call the aggregator to handle all microservice interactions, aggregate the data, and return a consolidated response. This would simplify the UI logic, reduce latency, and centralize error handling, making the system more efficient and maintainable. This is the first step in implementing the **Aggregator pattern**, which abstracts the complexity of multiple service interactions and provides a streamlined and efficient communication channel. *Figure 5.2* shows how the UI will communicate with an aggregator, which then communicates with the underlying services and returns the relevant data.

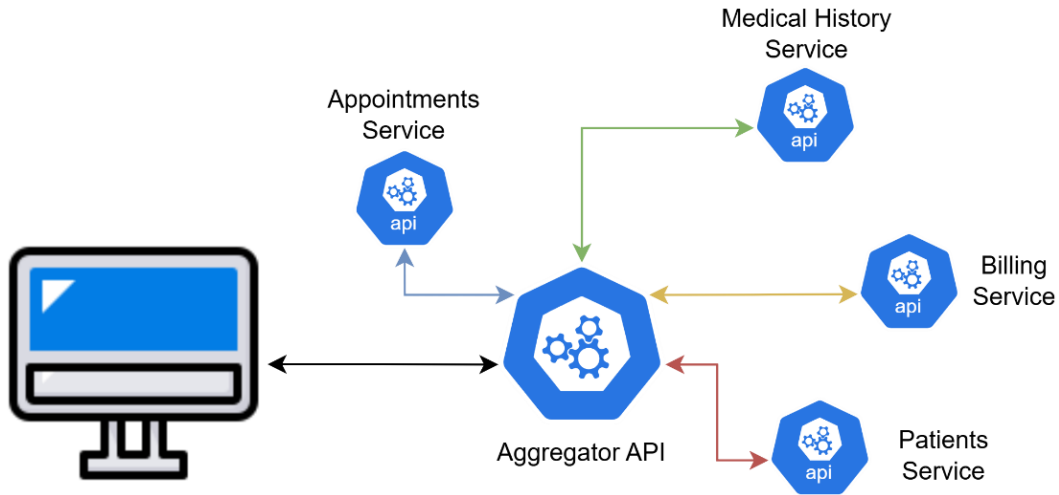


Figure 5.2 – Shows a UI communicating with an Aggregator service, which then calls the other APIs

This method reduces the number of client-server interactions and centralizes error handling and data formatting, resulting in a more efficient and user-friendly system. The UI sends a single request to the aggregator service for a consolidated patient dashboard. The UI does not need to know the individual services' endpoints, data formats, or authentication mechanisms. Let us review some of the tangible benefits that we can obtain from implementing this pattern:

- **Security benefits:**
  - The UI communicates only with the aggregator, minimizing direct exposure to microservices.
  - Authentication and authorization can be centralized, ensuring consistent enforcement of security policies.
  - Sensitive credentials or tokens required by individual services are hidden from the UI.
  - Fewer API endpoints are directly accessible from the UI, reducing the system's overall attack surface.
  - The aggregator can sanitize and validate data received from microservices before sending it to the UI.
  - End-to-end encryption can be enforced for data flows between the UI, aggregator, and microservices.

- **Optimization benefits:** Reduced network overhead since the UI makes fewer requests to different services. A single request to the aggregator service reduces the number of network round-trips.
  - Requests to microservices can be handled in parallel by the aggregator, reducing the time spent waiting for responses.
  - Caching can be implemented in the aggregator, serving repeated or frequently requested data faster than querying individual services.
- **Failure management benefits:**
  - The aggregator can detect failures in individual microservices and return partial data to the UI, ensuring the application remains functional even when some services are down. Since the aggregator intercepts the data before it is forwarded, additional quality and sanitization logic can be included to assess and adjust inconsistencies in the data.
  - The UI does not need to handle individual service errors. The aggregator consolidates error handling and provides a unified response with detailed error information when necessary.
  - Retry mechanisms can be centralized in the aggregator to handle transient failures.
  - The aggregator provides a single point for tracking and logging requests, errors, and performance metrics, simplifying debugging and incident response.
- **Data transformation benefits:**
  - The aggregator can transform data from multiple services into a cohesive response tailored to the UI's needs.

Example: combining patient, appointment, and billing services responses into a `PatientDashboard` object simplifies UI rendering.
  - The aggregator shields the UI from changes in individual microservice schemas. If a microservice updates its API, the aggregator handles the adaptation, maintaining consistency for the UI.

Example: If the Medical History service changes a field from `diagnosis` to `condition`, the aggregator translates this for the UI, avoiding disruptions.
  - The aggregator can add calculated fields or enrich responses with additional information.

Example: Adding a `fullName` field by concatenating `firstName` and `lastName` from the Patient service.

While these are compelling benefits of using an aggregator, it is essential to note that this aggregator can become a single point of failure. If the aggregator goes down, all clients relying on it lose access to every underlying data source, regardless of whether those sources remain healthy. While the aggregator introduces potential risk, it opens significant **scalability possibilities** when designed correctly. Here are some design factors to consider:

- **Horizontal scaling:** Employ a frontend load balancer that evenly distributes incoming traffic across multiple aggregator nodes, automatically adjusting to instance health and capacity.

Define dynamic scaling rules based on real-time metrics: CPU usage, request latency, and queue depths. This ensures the aggregation layer grows to meet demand, then contracts during lulls to optimize costs.

- **Asynchronous processing:** Leverage message queues for long-running or non-latencysensitive operations. Requests are enqueued and processed by worker pools, smoothing spikes in traffic and preventing request storms from overwhelming downstream APIs.
- **Sharding and partitioning:** If the aggregator must maintain in-flight tracking or correlation state, partition workloads by customer, geographical region, or API category. Each shard scales independently and minimizes contention.

Combining redundancy, careful state management, and intelligent loadshedding techniques, an API aggregator can evolve from a potential bottleneck into a robust, highly scalable core of your distributed system.

Now that we have seen the theory behind why the aggregator service can lead to a more efficient UI, let us review an implementation.

## Implementing the Aggregator pattern

We will develop a UI for this implementation to communicate with an aggregator service. This aggregator service will implement logic to communicate with the existing appointments service and the additional services (in theory). Since we have a working knowledge of developing microservices using ASP.NET Core, I will leave it to you to create the additional microservices to complete this example.

Let us begin by creating the aggregator service.

## Creating an aggregator service

The aggregator service will serve as a single access point for our client UI, which means it is simply another API that will call other APIs. These calls will happen synchronously, a technique we have already explored. So, we need to create a new API project that will need access to HTTP and JSON processing libraries. We can use the following commands to make the project and add the dependencies:

```
dotnet new webapi -o AggregatorService
cd AggregatorService
dotnet add package Microsoft.Extensions.Http
dotnet add package Newtonsoft.Json
```

In the `appsettings.json` file, ensure you have a section outlining the services and their endpoints:

```
{
  /* Other configs */
  "ApiEndpoints": {
    "AppointmentsApi": "https://appointments.api/",
    "PatientsApi": "https://patients.api/",
    "MedicalHistoryApi": "https://medicalhistory.api/",
    "BillingApi": "https://billingdetails.api/"
  }
}
```

Now we can add a `Models` folder and add DTOs that represent data we are expecting from the APIs and the aggregated model that should be returned to the UI:

```
namespace AggregatorService.Models;

public class PatientDashboard
{
    public PatientDto? Patient { get; set; }
    public List<AppointmentDto>? Appointments { get; set; }
    // Additional details
}

public class PatientDto
{
    public Guid Id { get; set; }
```

```

    public string Fullname { get; set; } = string.Empty;
    public DateTime DateOfBirth { get; set; }
    public string Gender { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
    public string ContactNumber { get; set; } = string.Empty;
}

public class AppointmentDto
{
    public Guid Id { get; set; }
    public DateTime AppointmentDate { get; set; }
    public string DoctorName { get; set; } = string.Empty;
}

// Additional DTOs

```

Now, we can create a Services folder and add our service clients. For this step-by-step process, I will focus on the service client for the appointments service. The service clients for the additional services will be very similar in implementation.

Since we are concerned with the patient's list of appointments, we will return a list of appointments from the API call based on the patient ID passed with the API call:

```

public class AppointmentsServiceClient(HttpClient httpClient)
{
    private readonly HttpClient _httpClient = httpClient;

    public async Task<List<AppointmentDto>>>
        GetAppointmentsAsync(int patientId)
    {
        var appointments = await
            _httpClient.GetFromJsonAsync<List<AppointmentDto>>>
                ($"api/appointmentsforpatient/{patientId}");
        return appointments;
    }
}

```

Note that this endpoint does not currently exist in the appointments API. So, we will also need to create a matching endpoint. We can add the following action to `AppointmentsController` to ensure coverage:

```
// GET: api/Appointments/GetAppointmentsByPatientId/5
[HttpGet("GetAppointmentsByPatientId/{patientId}")]
public async Task<ActionResult<IEnumerable<AppointmentByPatientId>>>
    GetAppointmentsByPatientId(Guid patientId)
{
    // Get appointments for a patient
    var appointments = await _context.Appointments.Where(a =>
        a.PatientId == patientId)
        .Select(q => new AppointmentByPatientId(q.AppointmentId,
            string.Empty, q.Slot.Start))
        .ToListAsync();

    // Get doctor details for each appointment in parallel
    var tasks = appointments.Select(async appointment =>
    {
        var doctor = await _doctorsApiClient
            .GetDoctorAsync(appointment.AppointmentId);
        appointment.DoctorName = doctor.LastName;
    });

    await Task.WhenAll(tasks);    return appointments;
}

// New DTO for this operation
public class AppointmentByPatientId(Guid appointmentId,
    string doctorName, DateTime date)
{
    public Guid AppointmentId { get; set; } = appointmentId;
    public string DoctorName { get; set; } = doctorName;
    public DateTime Date { get; set; } = date;
}
```



In this case, we need to find the patient's appointments and get the doctor's details as required. This leads us to complete several API calls, which are done concurrently to speed up the process. We also introduce a new DTO, which we use class and not record for. This is because we want to allow for mutability after initialization. With record, the values cannot be changed after an object is created, so be careful which you use and when.

Feel free to implement endpoints as necessary. As methods, requirements, and implementations change, existing services will continue to evolve to meet the needs at hand.

Don't forget to register the service clients in Program.cs:

```
// Register AppointmentServiceClient
builder.Services.AddHttpClient<AppointmentServiceClient>(client =>
{
    client.BaseAddress = new
        Uri(builder.Configuration["ApiEndpoints:AppointmentsApi"]);
});
// Other service clients registrations
```

Now, we can add a controller where we will implement our endpoints. Create a Controllers folder (if none exists) and add a new file, AggregatorController.cs. Here, we can add the code and orchestrate all the API calls. Here is a shortened version of this code:

```
[ApiController]
[Route("api/[controller]")]
public class AggregatorController( /* Other service client injections */
    PatientServiceClient _patientService,
    AppointmentServiceClient _appointmentService) : ControllerBase
{
    [HttpGet("patient-dashboard/{patientId}")]
    public async Task<ActionResult<PatientDashboard>>
        GetPatientDashboard(int patientId)
    {
        var patientTask = _patientService.GetPatientAsync(patientId);
        var appointmentsTask =
            _appointmentService.GetAppointmentsAsync(patientId);
        // Additional tasks
        await Task.WhenAll(patientTask, appointmentsTask,
            /* Additional tasks */);
    }
}
```

```
var patient = await patientTask;
if(patient is null) return NotFound();

var appointments = await appointmentsTask;
var dashboardData = new PatientDashboard
{
    Patient = patient,
    Appointments = appointments
    /* Other property assignments */
};
return Ok(dashboardData);
}
```

Although we are focusing on the appointments service client and communication, here is an implementation of the patient service client. One data model is used to retrieve the data from the API, and then it is transformed as needed before it is returned. The notable change here is returning the patient's full name:

```
public record Patient(
    Guid PatientId,
    string FirstName,
    string LastName
    /* Other properties */
);

public class PatientServiceClient
{
    private readonly HttpClient _httpClient;

    public PatientServiceClient(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<PatientDto?> GetPatientAsync(int patientId)
    {
        var patient = await _httpClient
```

```

        .GetFromJsonAsync<Patient>($"api/patients/{patientId}");
        if(patient is null) return null;

        var patientDto = new PatientDto
        {
            Id = patient.PatientId,
            Fullname = $"{patient.FirstName} {patient.LastName}",
            /* Other property assignments */
        };

        return patientDto;
    }
}

```

Now that the aggregator service has been prepared, let's review some of the changes required for the UI.

## Connecting a UI to the aggregator service

Now, we can modify our UI to interact only with the aggregator service. The first change we can make is to the `appsettings.json` file, where we will only need the address of the aggregator service:

```

"ApiEndpoints": {
  "AggregatorServiceApi": "https://AggregatorService.api/"
}

```

We will need a new model representing the aggregated and transformed data the aggregator returns. Create a `Models` folder if none exists, and create a new class called `PatientDashboardModel`. Add the following to this class:

```

namespace HealthcareDashboard.UI.Models;
public class PatientDashboardModel
{
    public PatientDto? Patient { get; set; }
    public List<AppointmentDto?> Appointments { get; set; }
    // Additional properties
}

public class PatientDto

```

```

{
    public Guid Id { get; set; }
    public string Fullname { get; set; } = string.Empty;
    public DateTime DateOfBirth { get; set; }
    public string Gender { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string ContactNumber { get; set; } = string.Empty;
}

public class AppointmentDto
{
    public Guid Id { get; set; }
    public DateTime AppointmentDate { get; set; }
    public string Doctor { get; set; } = string.Empty;
}
// Additional DTO definitions

```

Now that we have this model, we can create a service client for the new aggregator service. This will look like this:

```

public class AggregatorServiceClient(HttpClient httpClient)
{
    private readonly HttpClient _httpClient = httpClient;

    public async Task<PatientDashboardModel?>
        GetPatientDashboardAsync(Guid patientId)
    {
        return await
            _httpClient.GetFromJsonAsync<PatientDashboardModel?>
                ($"/aggregator/patient-dashboard/{patientId}");
    }
}

```

This new service must be registered in `Program.cs`, and the existing service registrations may be removed:

```

builder.Services.AddHttpClient<AggregatorServiceClient>(client =>
    client.BaseAddress = new
        Uri(config["ApiEndpoints:AggregatorServiceApi"]));

```

Now that we have adjusted the backend, we can refactor the **Dashboard** component to make one service call and use the returned object for its data:

```
@page "/dashboard/{PatientId:Guid}"
@using HealthcareDashboard.UI.Models
@using HealthcareDashboard.UI.Services
@inject AggregatorServiceClient AggregatorService

<h3>Patient Dashboard</h3>

@if (isLoading)
{
    <p>Loading data...</p>
}
else if(patientProfile is null)
{
    <p>No data found for the patient</p>
}
else
{
    <div>
        <h4>Patient Full Name</h4>
        <p>@patientProfile?.Patient?.Fullname</p>
        <h4>Patient Profile</h4>
        <p><strong>Gender:</strong> @patientProfile?.Patient?.Gender</p>
        <p><strong>Email:</strong> @patientProfile?.Patient?.Email</p>
        <p><strong>Contact:</strong>
            @patientProfile?.Patient?.ContactNumber</p>

        <h4>Appointments</h4>
        <ul>
            @foreach (var appointment in patientProfile?.Appointments)
            {
                <li>@appointment.AppointmentDate.ToShortDateString()
                    with @appointment.Doctor</li>
            }
        </ul>
        @* Additional profile sections *@
    </div>
}
```

```
}
@code {
    [Parameter]
    public Guid PatientId { get; set; }
    private PatientDashboardModel? patientProfile = new
        PatientDashboardModel();
    private bool isLoading = true;

    protected override async Task OnParametersSetAsync()
    {
        try
        {
            isLoading = true;
            patientProfile = await
                AggregatorService.GetPatientDashboardAsync(PatientId);
        }
        catch (Exception ex)
        {
            throw;
        }
        finally
        {
            isLoading = false;
        }
    }
}
```

Now, the component only needs to conduct one service call and can have properties needed for the single return object.

This activity concludes that introducing an aggregator can lead to cleaner and less coupled UI-to-microservice communication. However, the aggregator itself can become complicated since it now needs to shoulder the responsibility of efficiently orchestrating service calls. Let us review some of the best practices next.

## Performance optimization and best practices

The aggregator service is introduced to optimize how the UI interacts with the application's microservices. We must never forget that the system is only as good as its weakest link. Since this new service is the link between the UI and the rest of the services, it needs to be well-built and able to handle various scenarios.

As such, it is essential to implement best practices as much as possible and practical and ensure that all service interactions are handled efficiently. One quick win is asynchronous programming, which comes out of the box in .NET. We will review why this helps us next.

### Asynchronous processing

Asynchronous programming allows a program to perform non-blocking operations, enabling multiple tasks to run concurrently. This paradigm is beneficial when dealing with waiting operations, such as network requests to APIs.

In a synchronous program, tasks are executed sequentially. For a task such as an API call, the program halts execution until the call completes. This blocking behavior can make the program idle while waiting for the response.

Asynchronous programming allows a program to initiate a task, continue executing other tasks while waiting, and resume the task once the response is ready. This is achieved using constructs such as `async/await` in modern programming languages such as C#.

We use this in our `AggregatorController.cs` implementation, where we concurrently execute API call tasks, significantly reducing the time it takes to complete the calls. A single thread can handle thousands of concurrent requests because it isn't blocked while waiting for I/O operations.

While asynchronous programming helps to speed up API calls, there are times when we do not need to call the API for every data request. In cases where the data does not change very often, we can consider introducing caching. We will review this next.

### Caching

Caching stores a copy of data in a temporary storage layer (cache) to serve future requests faster. Instead of repeatedly fetching the same data from the source (e.g., microservices), the aggregator service retrieves it from the cache.

Caching works by serving up data from the source on the first request. Once the data is retrieved and added to the cache store with an expiration period, for any subsequent requests, if the cached data has not expired, it is used to fulfill the request without calling the source.

A simple method of implementing caching is **in-memory caching**, where data is cached in the application's memory. This method is volatile, and the data will be lost when the application restarts, so it is not a long-term solution. We will explore more details of caching and implementations in *Anchor 4, Building Resilient Microservices*, so I will spare the configuration details for now. However, after implementing caching in the aggregator service, we can modify `AggregatorController.cs` to check the cache before making an API call. In the following example, we do this for the patient dashboard information:

```
[HttpGet("patient-dashboard/{patientId}")]
public async Task<ActionResult<PatientDashboard>>
    GetPatientDashboard(int patientId)
{
    // Define the cache key
    var cacheKey = $"PatientDashboard_{patientId}";

    // Check if the data exists in the cache
    if (_cache.TryGetValue(cacheKey, out PatientDashboard
        cachedDashboard))
    {
        // Return cached data
        return Ok(cachedDashboard);
    }

    // If not in cache, fetch data from microservices
    // Store the data in the cache with an expiration time
    _cache.Set(cacheKey, dashboard, TimeSpan.FromMinutes(5));

    return Ok(dashboard);
}
```

This is a condensed snippet of how caching works in a method where data would be retrieved. More robust caching techniques involve distributed caching, where the cache is not volatile. As mentioned before, we will review these details in a later chapter. Next, we look at a low-hanging fruit that can lead to sending smaller packets with each response.



## Using compression

Compression is a technique for reducing the size of data transmitted over a network. It improves the performance and efficiency of applications, especially web services and APIs. Compressing response payloads allows applications to save bandwidth, reduce latency, and enhance the overall user experience.

Compression reduces the size of the response payload, improving bandwidth usage and response times, especially for large data.

.NET provides built-in support for several compression algorithms:

- **Gzip:** A popular algorithm for web applications due to its balance between compression ratio and speed
- **Brotli:** A newer algorithm with better compression ratios than **Gzip**, especially for static assets
- **Deflate:** Another algorithm, like Gzip, with a slightly different implementation

To enable compression, install the package for response compression with the following command:

```
dotnet add package Microsoft.AspNetCore.ResponseCompression
```

Then, you can enable compression in `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);  
// Add response compression services  
builder.Services.AddResponseCompression(options =>  
{  
    options.EnableForHttps = true; // Enable compression for HTTPS  
    //responses  
    options.Providers.Add<GzipCompressionProvider>();  
    options.Providers.Add<BrotliCompressionProvider>();  
});  
  
var app = builder.Build();  
// Use response compression middleware  
app.UseResponseCompression();  
app.MapControllers();  
app.Run();
```

This can be especially useful for an aggregator service that returns large payloads after aggregating data from multiple services. After implementing compression, payloads may see up to 200% size reductions.

`ResponseCompressionMiddleware` follows this pipeline internally:

1. **Client request:** The client sends an `Accept-Encoding` header indicating which compression methods it supports (e.g., `gzip`, `br`, `deflate`).
2. **Middleware match:** The middleware checks the following:
  - Does the response MIME type match the allowed list?
  - Is the `Accept-Encoding` header present and supported?
  - Is the HTTP method eligible?
  - Are there any excluded paths or content types?
3. **Compression provider selection:** The `ResponseCompressionMiddleware` iterates through configured compression providers (in order) and matches the first supported encoding with the client's `Accept-Encoding`.
4. **Compression stream wrapping:** It wraps the `HttpResponse.Body` stream with a corresponding compression stream:
  - `GZipStream` – Provides functionality for compressing and decompressing streams using the Gzip data format, which is based on the **DEFLATE** algorithm (RFC 1952).
  - `BrotliStream` – Used to compress or decompress streams using the Brotli compression algorithm, which provides superior compression ratios compared to Gzip, especially for text-based content such as HTML, CSS, and JavaScript.
  - `DeflateStream` – Used for compressing and decompressing streams using the DEFLATE algorithm (RFC 1951). It is a low-level compression stream compared to `GZipStream`.
5. **Header injection:** Sets response headers:
  - `Content-Encoding` (e.g., `gzip`)
  - `Vary: Accept-Encoding` (important for caching proxies)
6. **Response execution:** Data written to the response body is routed through the compression stream before being sent to the client.

Now, let us look at error-handling methods.

## Error handling

Error handling is critical to building reliable and efficient systems, especially in an aggregator service. This service integrates data from multiple microservices, each of which may encounter failures. Without robust error-handling mechanisms, a single failure can propagate to the Aggregator service, affecting its ability to deliver results to the client. In an Aggregator service, error handling manages failures from dependent microservices, network issues, and data inconsistencies.

Graceful degradation ensures that when a microservice fails, the aggregator service can still return partial results or fallback data, maintaining some level of functionality. More advanced patterns to handle retries or gracefully halt a faulty execution include retry and circuit breaker patterns, which are discussed in detail in later chapters.

Let us focus on how HTTP calls can be managed with native try-catch logic. With API calls, we can expect one of three kinds of errors:

- **Transient errors:** Temporary issues such as network timeouts or rate limiting. These can often be resolved by retrying.
- **Permanent errors:** Issues such as 404 Not Found or 401 Unauthorized. Retrying is usually unnecessary.
- **Unexpected errors:** Unexpected conditions, such as deserialization errors or null references.

Do not worry too much about the details of the errors and how they will be handled. We will explore this in *Anchor 3, Building Resilient Microservices*. Regardless of the main reason for the failure, an exception is thrown when `HttpClient` gets an error response. This error can be caught, and a default value can be returned. This way, the error from one service does not affect other service calls.

It is important to align on a standard error-handling method with your team and do what is best for your solution. There may be cases where one error should terminate the entire process, and there are times when we should be graceful and carry on with the more essential service calls, even if one or more fail.

Of course, we want to be aware of our system's failures and failure points, so logging and monitoring are always good ideas. We will investigate this next.

## Logging and monitoring

Logging and monitoring are foundational practices for ensuring any software system's reliability, performance, and maintainability. These practices enable developers to track system behavior, detect and diagnose issues, and make informed decisions to optimize the service.

Logging is the practice of recording system events and messages during a program's execution. Logs provide a chronological record of operations, errors, and warnings, offering valuable insights into the application's behavior and performance. Logs can be informational, detailed for debugging, or contain error details.

.NET provides a powerful and extensible logging framework integrated into the `Microsoft.Extensions.Logging` namespace. This built-in mechanism supports structured logging and multiple logging providers and is highly customizable, making it suitable for applications of any size.

The logging service can be injected into any class using the following template, where `ClassName` is the name of the class the logger is being injected into:

```
ILogger<ClassName> logger
```

Log messages can then be written at different points in code in like this:

```
logger.LogInformation("I am a log message");
```

Each log message type has a corresponding method.

Monitoring is continuously observing a system's performance, availability, and health. Monitoring tools collect and analyze metrics, enabling proactive identification and resolution of potential issues. .NET provides several built-in monitoring mechanisms to help developers and administrators understand their applications' performance, health, and behavior. These tools are integrated into the framework, offering seamless ways to collect metrics, diagnose issues, and ensure the reliability of applications in both development and production environments.

Later in the book, we will review more robust scenarios and implementations. Let us summarize what we have learned about the aggregator pattern.

## Summary

This chapter reviewed the Aggregator pattern, a critical design approach for managing interactions between microservices. It highlighted the challenges of direct client-to-microservice communication, such as increased latency, complex client-side logic, and tightly coupled dependencies, particularly in systems such as healthcare applications. These issues are mitigated by introducing an aggregator service, and the complexity is shifted from the UI to a central service, streamlining data consolidation and interaction.

We began with a theoretical overview of the pattern, outlining its role in simplifying communication by mediating between the UI and multiple microservices. This approach centralizes error handling, enhances security by reducing the exposure of individual microservices, and improves performance by reducing the number of API calls required by the client.

We then implemented a traditional client/API setup, where the UI directly calls individual microservices. We saw that the UI must handle parallel API requests to services such as patient, appointment, and billing APIs, be prepared to manage custom authentication and header request needs for each service, and address potential failures independently. This tightly couples the UI with backend changes, increasing maintenance overhead. With the aggregator pattern, the UI only communicates with a single endpoint, significantly simplifying its responsibilities.

We then transitioned into a detailed guide on implementing the pattern using a .NET API. Key steps included creating modular service clients for interacting with individual microservices, designing DTOs to represent aggregated data, and leveraging asynchronous programming to handle concurrent API calls efficiently. We also reviewed how to refactor the UI to rely solely on the aggregator service.

We emphasized best practices and optimizations for the aggregator service, including asynchronous programming, caching, compression, error handling, and logging and monitoring.

The aggregator pattern is indispensable for building scalable, efficient, resilient microservices-based applications. By shifting complexity from the UI to the aggregator, developers can create systems that are easier to maintain and optimize. This foundational chapter established a roadmap for leveraging the pattern effectively, offering practical guidance and theoretical insights to implement it within a .NET environment.

We will tackle the CQRS pattern next.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW



# 6

## Working with the CQRS Pattern

We now know that microservices require some foresight during the planning phase and that we must employ the best patterns and technology to support our decisions. In this chapter, we will explore another pattern that has gained much acclaim for helping us write clean and maintainable code: the **Command Query Responsibility Segregation (CQRS)** pattern, which is an extension of the **CQS** pattern.

The CQRS pattern is a fundamental architectural principle that plays a pivotal role in the design and implementation of modern microservices. At its core, CQRS divides the responsibilities of reading and writing data into two distinct paths, enabling developers to optimize each independently. This segregation allows for better scalability, maintainability, and performance in applications where complex business logic and high-throughput requirements are paramount.

In a traditional monolithic or layered architecture, the same data model is often used for retrieving (querying) and modifying (commanding) data. While this approach works for more straightforward applications, it can lead to inefficiencies and rigidity as the system becomes complex. For example, specific scenarios may require intricate queries that result in over-fetching or under-fetching data, while other scenarios demand efficient, atomic updates to the system's state. This means that an update or write operation should not affect all data points of a table and should be done in a specific and efficient manner. Attempting to optimize a single model for these diverse needs can quickly result in a bloated and cumbersome design.

At that point, we begin to use words such as behavior and scenarios. We begin to consider structuring our code in a manner that allows us to isolate behaviors and quickly determine whether this behavior is simply a request for data or will augment data by the end of the operation.

By the end of this chapter, we will be able to do the following:

- Understand the benefits of the CQRS pattern and why it is used for microservices development
- Implement commands in the CQRS pattern
- Implement queries in the CQRS pattern

As we delve into this chapter, we will explore the principles and practices of implementing the CQRS pattern within a microservices architecture. You will gain insights into designing command and query models where appropriate.

Let us start by exploring this pattern and why it can be a game-changer for your microservices application.

## Technical requirements

The code used in this chapter can be found in the project repository hosted on GitHub at <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter06>.

## Understanding the CQRS pattern

The CQRS pattern offers significant benefits in the context of microservices. Microservices are inherently distributed and autonomous, with each service owning its data and encapsulating its business logic. By adopting CQRS, a microservice can independently scale its read and write operations, aligning with the specific workload patterns it encounters. Now, let us spend some time understanding the CQS and CQRS patterns.

## What is the CQRS pattern?

The CQRS pattern is derived from the CQS pattern, which stands for **Command-Query Segregation**. This pattern states that every method should perform an action (a command) or return data (a query), but never both. This principle laid the groundwork for cleaner and more predictable code bases by ensuring that methods had a single responsibility. However, CQS was primarily focused on improving code-level design rather than addressing architectural concerns in large-scale systems.

Figure 6.1 shows a typical CQS architecture.

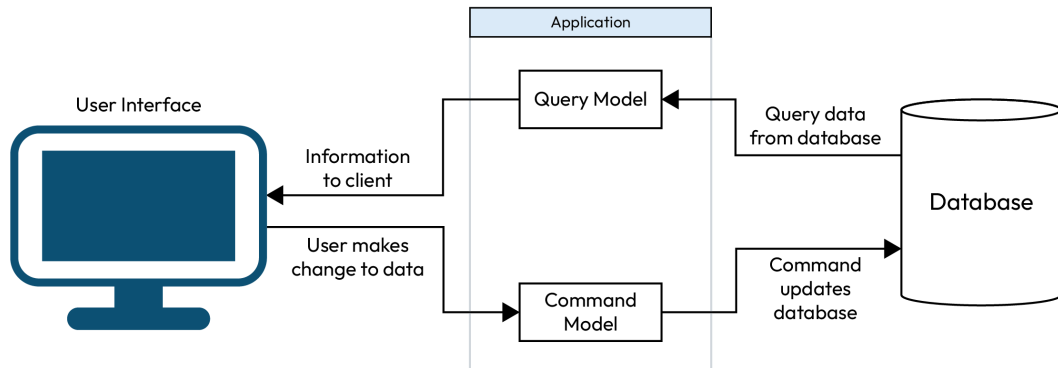


Figure 6.1 – The application will interact with models for read operations and models for write operations, known as commands

Looking more closely, it can be argued that CQS only accounts for one data store, meaning we read/write operations against the same database.

CQRS extends the CQS principle to the architecture level, solving the challenges posed by using a single model for queries and commands. A shared model often leads to inefficiencies in traditional monolithic or layered architectures as the system scales. Complex queries might require over-fetching or under-fetching data, while commands demand precise, atomic state changes. Optimizing a single model for these diverse requirements often results in a cumbersome and fragile design.

The concepts of over- and under-fetching relate to how much data is processed in a request. It is generally good practice in standard database querying that when you want data, you attempt to retrieve the exact data you require instead of getting too much or too little for the context. Given this best practice, we want to begin this at the software level and get the data we need to complete a request. Getting more data than necessary introduces unnecessary latency and can result in excessive data processing.

The CQRS pattern addresses this challenge by introducing a clear separation of concerns. Commands are responsible for actions that change the system's state, such as creating, updating, or deleting resources. Queries, however, handle data retrieval, delivering the information required in the most efficient format possible. This separation enables developers to tailor the underlying models and infrastructure to the unique requirements of each operation.



CQRS suggests having separate data stores, one for write operations and another for read operations. For instance, the write model might employ a transactional database optimized for consistency, given its strict adherence to the **Atomicity, Consistency, Isolation, and Durability (ACID)** principles. In contrast, the read model could leverage a denormalized, cache-friendly structure designed for speed, such as a cache (Redis, Memcached, etc.) or NoSQL (MongoDB, Cosmos DB, etc.) data store. Implementing multiple data stores is not always an option, nor is it a must.

Figure 6.2 shows the CQRS architecture with two databases.

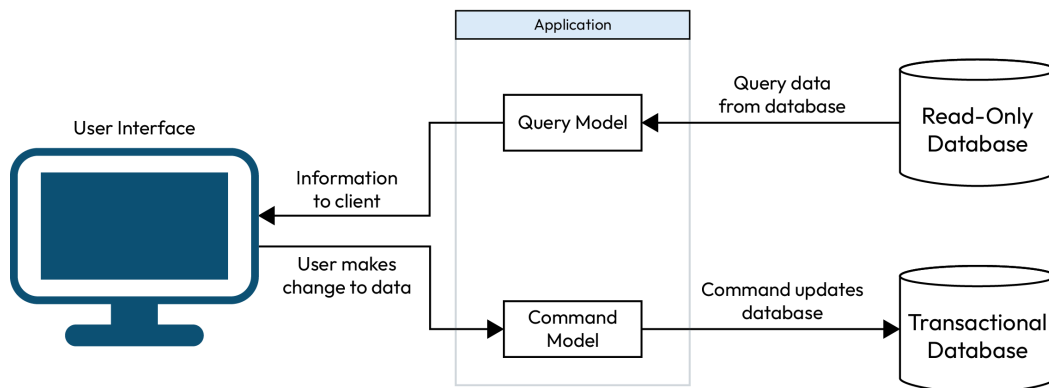


Figure 6.2 – The query model represents read operation-optimized representations of the data from the transactions database

CQRS has gained much acclaim since its introduction in development and is touted as a staple of microservice design. However, it is not unique to or required for microservices. It also adds a new level of complexity to the development effort, requiring more specific classes and code to be written. This can lead to project bloat, increased code complexity, and overhead in development effort.

Let us review some tangible benefits of implementing this seemingly complex pattern in our application.

## Benefits of the CQRS pattern

CQRS is about splitting a single model into two variations, one for reads and one for writes. The end goal, however, is an improved system architecture. The first benefit of this approach is **scalability**.

Assessing your system's read/write workload is essential. Read operations are more intensive than write operations because one read may require data from several tables, resulting in joins and additional filters, which reduce data retrieval speed and efficiency. Each request might have its own requirements based on the data's appearance. One school of thought encourages employing a data store dedicated to and optimized for reading operations. This allows us to scale read operations separately from write operations.

An example of a dedicated read store is a data warehouse, where some form of data transformation pipeline constantly transforms data from the write data store, a normalized *relational database*. Another commonly used technology is NoSQL databases such as MongoDB or Cosmos DB. The data constructs provided by the data warehouse or NoSQL databases represent a denormalized, read-only version of the data from the relational data store.

The second benefit is **performance**. Though it goes hand in hand with scalability, there are different dynamics that we consider. Using separate data stores isn't always a viable option, so there are other techniques that we can employ to optimize our operations that wouldn't be possible with a unified data model. For instance, we can apply caching to queries involving read operations. We can also employ database-specific features and fine-tuned SQL statements for our requests.

Another benefit that we can reap from this pattern is **simplicity**. This sounds contradictory, given that we mentioned complications earlier in this chapter, but it depends on the lens you use to assess the rewards you will reap in the long run. Commands and queries have different needs, and using one data model to suit both needs is unreasonable. CQRS forces us to consider creating specific data models for each query or command, which leads to more maintainable code. Each new data model is responsible for one operation, and modification therein will have little to no impact on other aspects of our program.

It is said that *when you have a hammer, everything looks like a nail*; when you hear of a new pattern, you may feel the need to use it everywhere. Please consider this pattern carefully before using it and ensure its value in the application is justified.

We have seen here that using CQRS in our projects has a few benefits. But where there are pros, there are cons. GQRS introduces several challenges that must be considered carefully. Let us review some of the downsides to employing this design pattern.

## Disadvantages of the CQRS pattern

Every pattern we consider must be investigated thoroughly for its value and potential drawbacks. We always want to make sure that the benefits outweigh the disadvantages and that we won't live to regret these significant design decisions. CQRS is not ideal for applications that will perform simple CRUD operations.

As stated, it is behavior- or scenario-driven, so you can justify the use cases before you take the plunge. The pattern is based on the concept of carrying out a command or completing a query, and in effect, it will lead to a perceived duplication of code since it promotes the idea of separation of concerns and encourages commands and queries to have dedicated models.

On a personal note, I have seen development leads with the best intentions implementing CQRS in projects that didn't require it. The project was overcomplicated, confusing new developers about where things went.

As we have seen, multiple data storage areas are justified when using CQRS, arguably its most complete implementation. Once we introduce numerous data stores, we introduce data consistency problems. We must implement event-sourcing techniques and include service-level agreements to inform users of the potential gap between our read/write operations.

We also must consider the additional infrastructure and general operation costs when we have multiple databases. With various databases come numerous potential points of failure and the need for further monitoring and fail-safe techniques to ensure the system runs as smoothly as possible, even in the face of outages.

CQRS can add value to a project when it is applied sensibly. When your project has complex business logic or a distinct need to separate data stores and read/write operations, then CQRS will shine as an appropriate architectural choice.

This pattern is not unique to any programming language, and .NET has excellent support, which we will explore in the implementation activities later in the chapter. We have reviewed some of this pattern's drawbacks. Let's now explore implementing CQRS in our healthcare microservices application.

## Implementing the CQRS pattern

Assessing your service's specific needs and characteristics is critical before implementing the CQRS pattern. Implementing CQRS indiscriminately can introduce unnecessary complexity without yielding significant benefits. To illustrate this, consider the healthcare management system we have been developing. This system encompasses microservices that manage patient and doctor data, documents, appointments, billing, and medical history.

Assessing your services ensures that you apply CQRS where it adds the most value. Factors to consider include the complexity of business logic, the frequency and nature of read and write operations, and the service's scalability requirements. By carefully evaluating each service in your architecture, you can avoid over-engineering while reaping the benefits of CQRS in the proper contexts. In the following sections, we will explore how to implement the pattern effectively for those services that truly benefit from it.

Services such as **Appointment booking**, **Patient Record Management**, and **Medical History** are strong candidates for the CQRS pattern. The appointment booking service may need to handle frequent updates to availability while providing users with real-time, optimized views of available

slots. Similarly, the patient record management service must ensure consistency when modifying sensitive medical information while enabling efficient retrieval of a patient's history. The medical history service benefits from CQRS by separating real-time data aggregation from query-intensive reporting requirements, allowing it to scale independently for high-performance data processing.

Conversely, services that handle authentication, notifications, and document delivery are unlikely to benefit significantly from CQRS. These services typically involve straightforward operations with minimal need for complex queries or state changes. For instance, the authentication service handles login and token validation, which are inherently atomic operations that do not require distinct command and query models. Similarly, the notifications service focuses on sending messages based on predefined templates, and the document delivery service provides files that do not involve intricate data retrieval or modification logic.

For our implementation, we will focus on applying this pattern to the appointment booking service, where we will enhance the usage of application logic and abstract the logic from the controllers. Let us begin by looking at the MediatR library and why it is a good option for our implementation in .NET Core.

## Adding MediatR

The most popular and arguably most effortless way of implementing the CQRS pattern in .NET is MediatR. This lightweight library facilitates in-process messaging and is commonly used to implement the CQRS pattern in .NET applications. It promotes a clean separation of concerns by decoupling handlers from their callers.

Some of the key features of this package include the following:

- Simplifies command and query handling by leveraging the mediator design pattern
- Reduces dependencies between application layers
- Supports pipeline behaviors for cross-cutting concerns such as logging and validation

The mediator design pattern is a behavioral design pattern that facilitates communication between different components or objects in a system without them having direct dependencies on each other. Instead, these components (colleagues) interact through a central mediator object, encapsulating the communication logic. This pattern helps reduce coupling between objects, making the system more modular and easier to maintain.

Let us add MediatR to our appointment booking service by using the following command:

```
dotnet add package MediatR
```

After that, we need to register it in the `Program.cs` file. Add the following line in the service registration section. Adding the following code will allow .NET Core to identify MediatR models and business logic, or handler, implementations in the code:

```
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly())
);
```

Also, please note that we are modifying a project that already includes an implementation of `MassTransit`. Be sure to add the use of MediatR to classes where MediatR interfaces are being injected, as there are some overlaps with `MassTransit` interface names as well.

Now that we have added the MediatR library, let us implement a command.

## Creating a command

As you'll remember, a command is expected to carry out actions that will modify the data in the data store, otherwise called a write operation. Given that we use the mediator pattern to govern our operations, we need a command model and a handler.

Our model is relatively simple to implement. It tends to be a standard class or record, but with MediatR present, it will implement a new type called `IRequest`. `IRequest` will associate this model class with an associated handler, which we will investigate briefly.

Let us start by creating a folder called `Commands` in the project root. This folder will house subfolders to store the different assets required for each command type. So, make another folder inside `Commands` called `CreateAppointments`.

Now, we will create a new command class called `CreateAppointmentCommand.cs`, as follows:

```
public record CreateAppointmentCommand(
    Guid DoctorId, Guid PatientId, Location Location,
    TimeSlot Slot, string Purpose) : IRequest<Appointment>;
```

We use a record type in this example, but it could also be a class if that is what you'd prefer to use. Notice that we inherit from `IRequest<Appointment>`, which outlines the following to MediatR:

- This command should be associated with a handler with a corresponding return type
- The handler associated with this command is expected to return an `Appointment` object value

Commands don't always need to return a type. You don't need to specify a data type alongside `IRequest` if you don't expect a return type.

Now that we have a command model, let's implement the corresponding handler. Our handler is where the logic for carrying out the task usually sits. In some implementations, and without a rich data model, you can carry out all the validations and additional tasks required to ensure that the task is handled correctly. This way, we can better isolate the expected business logic when a particular command is sent for execution.

Our handler for our command to create an appointment will be called `CreateAppointmentHandler.cs` and can be defined like this:

```
public class CreateAppointmentHandler(
    AppointmentContext _context,
    IPublishEndpoint _publishEndpoint) :
    IRequestHandler<CreateAppointmentCommand, Appointment>
{
}
```

We inject the database context and publisher service from the original API controller using the default constructor. We are also inheriting from `IRequestHandler`, which implements a method called `Handle`, which defaults to the outlined return type of `IRequest<>` attached to the command model. It will automatically generate a parameter called `request`, which will be of the command model's data type.

Your editor will prompt you to implement the `Handle` method, which you may populate with the following code:

```
var newAppointment = new Appointment
{
    AppointmentId = Guid.NewGuid(),
    DoctorId = request.DoctorId,
    PatientId = request.PatientId,
    Location = request.Location,
    Slot = request.Slot,
    Purpose = request.Purpose
};
_context.Add(newAppointment);
await _context.SaveChangesAsync();

//Perform post-creation hand-off to services bus.
await _publishEndpoint.Publish<AppointmentCreated>(new
```

```
{
    newAppointment.AppointmentId,
    newAppointment.PatientId,
    newAppointment.DoctorId,
    newAppointment.Slot.Start,
    DateTime.UtcNow,
    MessageId = newAppointment.AppointmentId
});
return newAppointment;
```

Here, we do the following:

- Take the data from the request object sent via the request command object and use it to populate the newAppointment object
- Create a new appointment record in the database
- Add an AppointmentCreated object for database and service bus operations
- Return the newAppointment object, which matches the data type indicated by the IRequest<Appointment> inheritance on the command

Do note that if we did not require a return value, we would have to return `Unit.Value`. This is the default void representation of MediatR.

Now that we have a command that creates a new appointment, let's refactor our controller to use the mediator and no longer implement all the business logic. First, we need to inject `IMediator` to add a dependency for MediatR:

```
public class AppointmentsController(
    IMediator _mediator, /* other dependencies */):
    ControllerBase
```

Then, we can refactor the POST method to look like this:

```
public async Task<ActionResult<Appointment>>
PostAppointment(CreateAppointmentCommand createAppointmentCommand)
{
    var appointment = await _mediator.Send(
        createAppointmentCommand);
    return CreatedAtAction("GetAppointment", new {
        id = appointment.AppointmentId }, appointment);
}
```

Now, we can rely on the handler to complete all the functionality required to create an appointment, and the API controller no longer needs the implementation details. You can observe that each method of authoring the code amounts to the same thing: we need to compose the appropriate model type with the proper values before sending it off to our mediator. The mediator will have already figured out which model matches which handler and will proceed to carry out its operation.

Commands and queries generally follow a similar implementation. The next section will implement a query model/handler pair using our mediator and CQRS patterns.

## Creating a query

A query is expected to search for data and return a result. This search might be complicated or straightforward. However, we implement this pattern as an easy way to segregate the query logic from the request's originator (such as the controller) and the command logic.

This type of separation increases a team's ability to maintain either side of the application without excessive conflicts. We will similarly use the mediator pattern to govern how we carry out these operations, and we will need a query model and a handler.

Our model is simple as we can leave it empty or include properties that will play a part in the process to be carried out in the handler. We inherit `IRequest<>` and define a return type since we are writing a query and expect data to be returned.

Let us look at two examples of queries: one that will retrieve all the appointments in the database and another that will retrieve appointments only by ID. We can define `GetAppointmentsQuery.cs` like this:

```
public record GetAppointmentsQuery(): IRequest<List<Appointment>>;
```

We can define `GetAppointmentByIdQuery.cs` like this:

```
public record GetAppointmentByIdQuery(string Id):  
    IRequest<AppointmentDetails>;
```

You can place these new classes in `Queries`, a new folder alongside the `Commands` folder. In the `Queries` folder, create subfolders to house the assets related to the operations: `GetAppointments` and `GetAppointmentById`.

Creating more files and references increases the need to organize classes and folders. We will discuss more patterns and recommendations in the next section of the chapter, but for now, we can move the existing DTOs that are now being used by the handlers and models to a central folder called `DTOs`. Also, ensure that you update your namespaces.



The `GetAppointmentsQuery` model doesn't need any properties since it will just be used as an outline for the handler to make an association. `GetAppointmentByIdQuery` has an `Id` property because the ID will be needed for the handler to execute the task at hand correctly. The difference in return types is also a crucial point to note, as that sets the tone for what the handler will be able to return.

We need to ensure that our query models are crafted specifically for the type of data we expect to retrieve from the matching handler.

Now, let's look at implementing these handlers. Our query handlers will execute the expected queries and return the data as defined by `IRequest<>`. As previously outlined, the ideal usage of this pattern will see us using a separate data store where the data being queried is already optimized for return. This would make our query operation efficient and reduce the need for data transformation and sanitization. An alternative ORM to **Entity Framework Core (EF Core)**, such as **Dapper**, may sometimes provide variably faster query response times.

In our example, we are neither implementing a separate data store nor changing the ORM, so we will inject the database context into the handler for our query to get all the appointments. This handler will be called `GetAppointmentsHandler.cs` and can be implemented like this:

```
public class GetAppointmentsHandler(AppointmentContext _context) :
    IRequestHandler<GetAppointmentsQuery, List<Appointment>>
{
    public async Task<List<Appointment>> Handle(
        GetAppointmentsQuery request, CancellationToken cancellationToken)
    {
        return await _context.Appointments.ToListAsync();
    }
}
```

This handler's definition is simple: we retrieve a list of appointments. When we need an appointment by ID, it implies that we need the appointment details. This will call for a more complex query involving joins and/or synchronous API calls to get the details of related records. We can "handle" these operations in our new handler, `GetAppointmentByIdHandler`:

```
public class GetAppointmentByIdHandler(AppointmentContext _context,
    PatientsApiClient _patientsApiClient,
    DoctorsApiClient _doctorsApiClient) :
    IRequestHandler<GetAppointmentByIdQuery, AppointmentDetails>
{
```

```
public async Task<AppointmentDetails> Handle(
    GetAppointmentByIdQuery request, CancellationToken cancellationToken)
{
    var appointment = await _context.Appointments.FindAsync(request.Id);
    if (appointment == null)
    {
        return null;
    }
    // Use the Http clients to fetch patient and doctor details
    // Use GRPC service to retrieve document information on patient
    // Combine data and return response
    return appointmentDetails;
}
```

This is a succinct version of the handler's implementation since much of the code is derived from the `GetAppointment` action in the `AppointmentsController.cs` file. Notice that the relevant services are injected along with the database context, and all the complex logic needed to return the details is abstracted from the controller.

Now, we can refactor the `GetAppointments` and `GetAppointment` actions in the API controller to call these handlers:

```
public async Task<ActionResult<IEnumerable<Appointment>>> GetAppointments()
{
    return await _mediator.Send(new GetAppointmentsQuery());
}

public async Task<ActionResult<Appointment>> GetAppointment(string id)
{
    var appointmentDetails = await _mediator.Send(
        new GetAppointmentByIdQuery(id));
    return appointmentDetails == null ? NotFound() : Ok(appointmentDetails);
}
```

Note that the `GetAppointmentById` handler might return `null` if the record cannot be found. For this reason, we check whether the return value is `null` and return the appropriate HTTP response.

Now, you have the foundation required to implement CQRS in your API. Apart from isolating read and write operations, another tangible benefit is that it leads to cleaner and more coherent controllers. The more we abstract the business logic away from the API layer, the cleaner the code becomes and the easier it is to isolate and test business logic.

Let us review some best practices and recommendations for implementing this pattern.

## Best practices and recommendations

Best practices are generally a matter of philosophy and preference. Software development aims to deliver working software, but that often comes at the expense of code readability and testability. This pattern, like many others, seeks to increase the testability of the code, but it might come at the expense of readability if it is not done correctly.

The first thing we want to review is naming conventions. Good naming standards can help you and your team when implementing this pattern.

### Good naming conventions

A naming convention is a standardized set of rules or guidelines for naming elements in code, such as classes, methods, variables, and files. These conventions aim to improve the code base's readability, maintainability, and overall quality.

When implementing the CQRS pattern, clear and consistent naming conventions become critical due to the separation of commands and queries. Poor naming can lead to confusion, errors, and reduced productivity. Here are the key reasons why good naming conventions matter:

- Good naming conventions make it immediately clear whether a particular class, method, or file belongs to the command or query side of CQRS.

For example, use imperative verbs to indicate actions (e.g., `CreateAppointmentCommand`, `UpdatePatientCommand`, and `GetPatientsQuery`).

- Developers can quickly understand code elements' role and purpose without extensive documentation or exploration.
- Consistently named elements are easier to locate and modify when changes are required.
- Naming conventions help align code with domain concepts, making the code more intuitive and reflective of business requirements.

By adhering to good naming conventions, you create a foundation for a maintainable, scalable, and developer-friendly implementation of the CQRS pattern. Another point of contention is the number of files needed to implement and sustain this pattern. Let us discuss some organizational techniques to help reduce attrition.

## Organizing files and classes

The CQRS pattern emphasizes separating the responsibilities of commands and queries, and proper organization of files and classes is critical for maintaining this separation. Coupled with the **Single Responsibility Principle (SRP)**, good organization ensures that each component in the CQRS architecture has a clear and focused purpose. You can employ several techniques to assist with the organization of the files and folders:

- **Separate command and query folders:** Create distinct folders or namespaces for commands and queries to reinforce the separation of responsibilities. As in our previous implementation, we created a folder for commands and one for queries, with subfolders containing the handler and model classes for the operation.
- **Group by feature:** Organize files by feature or domain rather than technical type (e.g., commands, queries, and handlers). This approach aligns with domain-driven design and makes it easier to navigate the code base. It is alternatively referred to as **vertical slice architecture**. The following is an example of this folder hierarchy:

```
- Features
  - Appointments
    - Commands
      - CreateAppointmentCommand.cs
      - CreateAppointmentHandler.cs
    - Queries
      - GetAppointmentByIdQuery.cs
      - GetAppointmentByIdHandler.cs
```

- **Adhere to SRP:** SRP is a school of thought that promotes clear separation and independence for code blocks. A method should have a single task, and a class should implement one thing. This means that, by these rules, you should avoid placing multiple class definitions in one class file.

SRP is the *S* in **SOLID**, and it is a mainstay of good development practices. But when implementing CQRS, as the project grows, it becomes difficult to navigate that many files. Sometimes, you may find that the model, handler, and sometimes DTO classes are all

defined in one file. This reduces the number of files created and allows you to see all the related assets in one place. Here is an example of this implementation if we refactor the `GetAppointmentsByPatientId` method code to be in CQRS:

```
public class AppointmentByPatientId(Guid appointmentId,
    string doctorName, DateTime date)
{
    // Shortened for brevity
}

public record GetAppointmentByPatientIdQuery(string PatientId):
    IRequest<List<AppointmentByPatientId>>;

public class GetAppointmentsByPatientIdhandler(
    AppointmentContext _context, DoctorsApiClient _doctorsApiClient) :
    IRequestHandler<GetAppointmentByPatientIdQuery,
    List<AppointmentByPatientId>>
{
    public async Task<List<AppointmentByPatientId>> Handle(
        GetAppointmentByPatientIdQuery request, CancellationToken
        cancellationToken)
    {
        // Get appointments for a patient
        // Get doctor details for each appointment in parallel
        return appointments;
    }
}
```

Note that the DTO, request model, and handler are all defined in the same class file. This alleviates the navigation effort when changes are required to one or more of these code blocks. You and your team may decide which approach is best.

Let us now review some recommendations and implementation options for efficient read operations.

## Speeding up read operations

Since CQRS separates reads and writes, optimizing read operations is critical for improving the performance of query-heavy applications. The following are tangible techniques to achieve faster read performance:

- **Use EF Core's no-tracking queries:** By default, EF Core tracks changes to entities retrieved from the database, which incurs performance overhead. Use `AsNoTracking()` to execute read-only queries that do not require tracking. This reduces memory usage and speeds up query execution for read-only operations. Here is an example of a query written using `AsNoTracking()`:

```
await _context.Appointments.AsNoTracking().ToListAsync();
```

- **Secondary (denormalized) data stores:** We have used SQLite, a lightweight relational database technology. Relational databases are known for their organization and ability to ensure data integrity, but usually at the expense of read performance. In a case where we have several services that need to provide individual details (such as fetching appointment details), we must consider a consolidated version of the data in a separate store. This consolidation is referred to as **denormalization**.

Consider creating read-optimized, denormalized data models tailored for specific query use cases. Consider using separate databases, tables, or materialized views to pre-aggregate and store data in a format optimized for querying. This can significantly minimize the computational cost of joins, API calls, and aggregations at runtime. This approach also introduces the need for synchronization code and jobs that need to run in the background to ensure that the read-only data store reflects the current state of the data. This is one of the major factors that this implementation must contend with, and it adds another level of complexity to the project.

- **Use ADO.NET:** As mentioned before, it is a commonly used alternative ORM to the flagship EF Core. EF Core is exceptionally performant and improves with each new iteration. It is still an abstraction layer with materialization operations and some performance costs.

ADO.NET remains a fundamental part of the .NET ecosystem, providing a low-level API for database interaction. It offers robust, flexible, and high-performance data access features and is still supported for scenarios where fine-grained control over database operations is needed. It allows you to execute raw SQL queries directly, giving complete control over

the SQL sent to the database. An example of a query to retrieve all appointment records from the database would look like this:

```
var appointments = new List<Appointment>();
using (var connection = new SqlConnection(
    _context.Database.GetDbConnection().ConnectionString))
{
    await connection.OpenAsync(cancellationToken);
    using (var command = new SqlCommand("SELECT * FROM Appointments",
        connection))
    {
        using (var reader = await command.ExecuteReaderAsync(
            cancellationToken))
        {
            while (await reader.ReadAsync(cancellationToken))
            {
                var appointment = new Appointment
                {
                    Id = reader.GetInt32(reader.GetOrdinal("Id")),
                    // Map other properties as needed
                };
                appointments.Add(appointment);
            }
        }
    }
}
```

While ADO.NET is powerful, it can involve more boilerplate code than EF Core. Using an ORM for most operations and ADO.NET for critical performance optimizations can be a balanced approach for large projects. If you want a fully modernized experience or additional abstractions, consider combining ADO.NET with libraries such as Dapper, which adds lightweight ORM capabilities without sacrificing control.

- **Use Dapper:** Next, we have Dapper. Dapper is a micro-ORM for .NET that offers high-performance data access. It is particularly well suited for the query side of CQRS, where complex joins or projections may be required. It is lightweight and fast, ideal for scenarios with performance-critical queries. It allows direct execution of SQL queries and provides

granular control over database interactions. It also requires the authoring of SQL queries but offers some mapping and data projection capabilities, making it a bit easier than using ADO.NET.

An example of the preceding ADO.NET code written using Dapper would look like this:

```
using (var connection = new SqlConnection(
    _context.Database.GetDbConnection().ConnectionString))
{
    await connection.OpenAsync(cancellationToken);
    var query = "SELECT * FROM Appointments";
    var appointments = await connection.
QueryAsync<Appointment>(query);
    return appointments.ToList();
}
```

Dapper is often viewed as a middle ground between ADO.NET and EF Core, offering a blend of performance and ease of use. It abstracts away much of the repetitive code required in ADO.NET and maps database rows to .NET objects automatically. It also has fewer abstraction layers than EF Core, making it more performant.

The choice between Dapper, ADO.NET, and EF Core depends on the project's specific requirements, such as performance, maintainability, and the team's familiarity with SQL or ORM tools.

CQRS is a pattern that should not be taken lightly. It introduces many advantages but is better suited for more complex projects and experienced development teams. Now that we have reviewed it, let us summarize what we have learned.

## Summary

This chapter discussed the **CQRS** pattern and its implementation, benefits, and drawbacks, particularly within microservices architectures. At its core, CQRS divides system responsibilities into distinct paths for reading (queries) and writing (commands). This separation enhances scalability, maintainability, and performance in systems with complex business logic.

In microservices, CQRS aligns with the principles of autonomy and scalability. It allows services to independently scale read and write operations and optimize data storage and retrieval mechanisms for specific use cases. This flexibility addresses inefficiencies inherent in traditional monolithic systems, where a single model is used for queries and commands.



Some of the benefits of CQRS include improved scalability through separate read stores, enhanced performance by leveraging techniques such as caching and fine-tuned queries, and simplicity in code maintenance by isolating commands and queries. Amid the benefits, however, be cautious that it can lead to increased complexity, perceived code duplication, and potential data consistency issues, mainly when multiple data stores are used. Remember to evaluate the complexity of your systems and the specific requirements of their services before adopting CQRS.

We then reviewed the implementation of CQRS using the MediatR library. Commands represent write operations, while queries retrieve data, and both follow a clear separation of concerns. The MediatR library facilitates decoupling and modularity by acting as a mediator between handlers and callers. The implementation involves defining command and query models and their associated handlers and ensuring that the logic is abstracted from controllers for cleaner and more testable code.

After seeing how CQRS works, we discussed naming conventions and file organization practices to help emphasize clarity, maintainability, and adherence to the **SRP**.

We looked at simple ways to optimize read operations, such as no-tracking queries in EF Core. We then looked at more advanced techniques, such as using a secondary denormalized data store. Alternative ORMs such as Dapper were recommended. These approaches enhance performance by minimizing unnecessary overhead and tailoring data retrieval mechanisms to specific application needs.

In conclusion, while CQRS offers substantial benefits for complex systems, its adoption must be justified by a project's specific requirements. Proper evaluation, planning, and adherence to best practices are essential for leveraging this pattern's full potential. This chapter provided a comprehensive foundation for understanding and implementing CQRS effectively in modern software development.

Since a major feature of this pattern is creating read-only stores, and we mentioned the need to ensure that read and write stores are synchronized, we will explore event-sourcing patterns as a solution to this challenge in the next chapter.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW



# 7

## Applying Event-Sourcing Patterns

The previous chapter explored the CQRS pattern, which encourages us to create a clear separation between code and data sources that govern read and write operations. With this kind of separation, we risk having inconsistencies in our data between operations, which introduces the need for additional techniques to ensure data consistency.

We must contend with the typical microservices pattern, where each service is expected to have its own data store. Remember that there will be situations where data needs to be shared between services. Some mechanisms must adequately transport data between services to remain in sync.

Event sourcing is touted as a solution to this issue, where a new data store is introduced that keeps track of all the command operations as they happen. The records in this data store are considered events and contain enough information for the system to track what happens with each command operation. These records are called events and act as an intermediary store for event-driven or asynchronous services architecture. They can also act as an audit log as they will store all the necessary details for replaying changes being made against the domain.

This chapter will explore the event sourcing pattern and justify its use as a solution to our potentially out-of-sync databases.

After reading this chapter, we will be able to do the following:

- Understand what events are
- Understand event-sourcing patterns in microservices
- Understand how to create an event store
- Understand how to use an event store for reads in CQRS

Now, we will start our journey by understanding what events are and what they are used for.

## Technical requirements

The code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter07>.

You will also need to use PostgreSQL for a portion of the implementation of this concept. You may download it at <https://www.postgresql.org/download/>.

## Understanding events and event sourcing

**Event sourcing** is a powerful architectural pattern that stores a system's state as a series of immutable events. These events represent changes to the system over time rather than storing the current state directly. By creating and storing events, developers can achieve a more granular level of understanding about the evolution of a system's state, making it particularly beneficial for applications that require robust audit trails, state reconstruction, or real-time processing.

### What are events?

An event, within the context of software development, refers to an immutable record of an occurrence within the system, typically resulting from the successful processing of a command. They are usually stored in JSON or alternative universal formats (e.g., Avro, Protobuf, etc.) to ensure system-agnostic compatibility and support versioning. They are then used to carry out additional actions in the background. For instance, in our healthcare management system, an event could be tied to operations such as the following:

- **AppointmentCreated:** A patient books an appointment
- **DoctorAdded:** A new doctor has been added to the system
- **PaymentProcessed:** A payment is completed
- **DiagnosisAdded:** A new record has been added to a patient's medical history

Events are immutable, meaning they cannot be altered once recorded. This immutability is fundamental to ensuring the integrity and reliability of events. Events bring several advantages to modern software development:

- **Auditability:** By recording every change as an event, you can recreate a complete history of system activity, which is invaluable for compliance and troubleshooting.
- **Decoupling:** Events promote loose coupling between components. Event producers do not need to know about the consumers, enabling the independent evolution of system parts. This is usually accomplished using message brokers, as we saw in *Anchor 4, Asynchronous Communication between Microservices*.

- **Scalability:** Event-driven systems can scale more effectively by distributing event processing across multiple consumers or microservices. This means that the number of consumers can be increased on demand when “too many” messages can be processed. Increasing the number of consumers will ensure efficiency on demand.
- **Reproducibility:** Since events are immutable, you can replay them to reconstruct the system’s state at any given point, particularly useful for debugging and testing. For context, events being **immutable** means they cannot be modified after they are created. This is usually accomplished through versioning, cryptographic hashing, or using an append-only data store.
- **Real-time processing:** Events enable real-time updates and actions. For instance, a payment event can trigger an inventory update or customer notification.
- **Flexibility in data modeling:** Events store the “why” behind state changes, providing richer context and enabling new insights as your application evolves.

In several cases, events are used for analytics purposes, notifications, and even to establish an audit trail of all data operations in the system. Given the versatility of how events can be used, their structure must be established and consistent to ensure that they can fulfill the system’s needs. Let us review some of the defining characteristics of events.

## Key attributes of events

Events can be used to build a foundation of any application’s core functionality. While the concept can be suitable for many situations, we need to understand some key attributes of events, adequately scope the need for their introduction, and uphold specific standards in our implementations.

An event typically contains the following attributes:

- **Event name:** A descriptive name indicating the type of event
- **Timestamp:** The exact time when the event occurred
- **Event ID:** A unique identifier for the event
- **Aggregate ID:** An identifier that ties the event to a specific entity or process
- **Payload:** Additional data capturing the event’s details (e.g., appointment ID, patient details, etc.)
- **Metadata:** Optional data providing context, such as the event’s source (such as the model or data type name) or the schema’s version

**Aggregate ID** is not intrinsically specific to the aggregates we mentioned in *Anchor 4, Applying Domain-Driven Design Principles*. It generally refers to the ID of the entity being tracked when the event is created. Therefore, event stores can be used in systems that do not implement DDD and, by extension, in non-microservices-based applications.

Remember that events represent facts or what has happened in the system, and as a result, business logic should not change these facts. Now that we know what events are and how they might be used, let's review why you need to implement event-sourcing patterns in your microservices application.

## Event-sourcing patterns in microservices

Applications built with a microservices architecture are structured to have loosely coupled and independent services. The next chapter, *Database Design Patterns for Microservices*, discusses the thought process needed when scoping database design. However, each microservice could have an independent data store, which presents a unique challenge to keep the data in sync between services.

This challenge increases when we consider that we must compromise on our **ACID** principles, namely, the **A**, which means **atomic**. We cannot guarantee that all our write operations will be completed as a unit. The atomic principle dictates that all data operations should complete or fail as a unit. Given the allowance for different technologies to be used for the data stores, we cannot guarantee that.

Given all these factors, we can turn to the event sourcing pattern, which allows us to persist events that track all the activities occurring against data in each service. This pattern is beneficial for asynchronous communication between services, where we can keep track of all changes in the form of **events**. These events can act as the following:

- **Persistent events:** Events contain enough detail to inform and recreate domain objects
- **Audit logs:** Events are generated with each change so that they can double as audits
- **Entity state identification:** We can use events to view an entity's point-in-time state on demand

The idea of tracking changes against the entities is called **replaying**. We can replay events in two steps:

1. Grab all or partial events stored for a given aggregate.
2. Iterate through all events and extract the relevant information to reconstruct the aggregate instance.

Event sourcing involves querying records using `AggregateId` and `Timestamp` to sort and identify event records. Event replays require iterating through all the events, grabbing information, and then changing the state of the target aggregate.

Now, let us review some of the pros and cons of event sourcing.

## Pros and cons of event sourcing

Event replays and how we conduct our updates depend on whether the aggregate is a *domain aggregate*. If the aggregate relies on domain services for manipulation, we need to make clear that replays are not about repeating or redoing command operations. A command is an operation that changes data in a system, which can create and store an event. This operation also has the potential to be a long-running operation with event data-generating side effects, which we may not want.

Event replays examine the data, perform logic to extract information, and apply the events to reconstruct the aggregate. Overall, stored events may be processed differently than the application employing the technique.

Also, events are data stored below the current state. This means we can reuse them to build any projection of the data we need. Ad-hoc projections of the data can be used to read data stored in a CQRS project structure, data analytics, business intelligence, and even artificial intelligence and simulations. Contextually, suppose we have a stream of events and can extract a specific subset. In that case, we can replay them and perform ad-hoc calculations and processes to generate custom and potentially new information. Events are constant and will always be the same now and later. This is an additional benefit in that we can always be sure to count on the data for consistency.

While event sourcing offers numerous benefits, it also introduces certain complexities. Developers need to understand the intricacies of event sourcing, which may involve a steeper learning curve than traditional approaches. We must navigate challenges such as the following:

- Managing changes to the structure of events. This will require careful versioning and backward compatibility considerations.
- Complex querying because the state is not stored directly. Therefore, querying the current state may require specialized patterns or additional computation.
- Handling failures and ensuring consistency when errors occur. In this case, you may need retry and replay mechanisms to ensure that events are stored correctly.

- Performance tuning since event replaying can quickly become an intensive operation based on the number of logged events. To address this, we can take snapshots of the aggregate state and business entities that have recently been amended and use these snapshots as a stored version of the record, sparing the need to iterate through many events.

Now that we have examined some of the more serious implications of this pattern and techniques that can be used to reduce its impact on our application, let's review how to create an appropriate and efficient event store.

## Creating an event store

We must understand an **event store** before we get to the scoping phase of creating one. A simple search on the topic might yield many results from various sources, each citing varied definitions. For this book, we will conclude that an event store is *an ordered, easily queryable, and persistent source of long-term records representing events against entities in a data store*.

Recall that an event record will have `AggregateId`, `Timestamp`, `EventType`, and `Payload` representing the state of the data at that time. An application will persist event records in a data store. This data store has an API or some interface that allows adding and retrieving events for an entity or aggregate. The event store might also behave like a message broker, allowing other services to subscribe to events as the source publishes them.

Figure 7.1 shows a typical event store architecture.

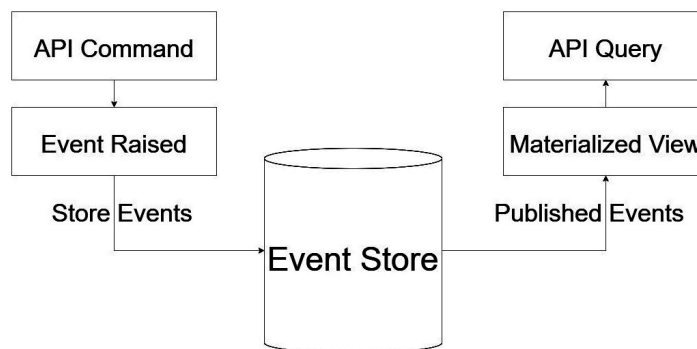


Figure 7.1 – An event store sits between the command and query handlers

Armed with an overview of what an event is and what it should be used for, we can itemize some of the more critical points to consider when scoping an appropriate design for our event store:

- Since events should be immutable, the data store should be *append-only*.
- Each change that takes place in the domain needs to be recorded in the event store.

- Since events need to contain specific details relevant to the recorded event, we must maintain some flexibility with the data structure.
- An event might contain data from multiple sources in the domain. We must leverage possible relationships between tables to grab all the details required to log the event.

Armed with these requirements, we can decide whether to use a relational or non-relational data store. If we use a relational data store, we would do well to have denormalized tables modeled from the event data we intend to store. This denormalized data table would represent an aggregated view of data points and be easier to read instead of joining several tables to get the data. An alternative and more efficient manner to handle event storage is using a non-relational or NoSQL database. This will allow us to store the relevant event data as documents in a far more dynamic manner.

Let us explore some options regarding storing events in a relational database.

## Scoping event storage using a relational database

Relational databases are popular because they are strict and make it easy to enforce data quality standards. The drawback is that they leave little room for flexibility, arguably the most significant need when storing events.

One workaround is to implement denormalized tables, where related data is stored in complete form and not just as a reference ID. This helps to capture all the details of an event and create specialized read models of the events. However, this approach can lead us down a rabbit hole of modeling several tables based on several events. This might not be sustainable in the long run since we would need to introduce new tables with each newly scoped event and constantly change designs as the events evolve.

An alternative would be to create a singular log table with columns matching the data points outlined previously for an event. For example, this table and the matching data types would be as follows:

| Column name   | Data type   |
|---------------|-------------|
| Id            | int         |
| AggregateId   | Guid        |
| Timestamp     | DateTime    |
| EventType     | Varchar     |
| Payload       | String/JSON |
| VersionNumber | Int         |

*Table 7.1 – Typical outline of an event store table*



We can add constraints to our records by introducing a `UNIQUE` index on the `AggregateId` and `VersionNumber` columns. `VersionNumber` represents the state version of an aggregate. It is increased each time a new event is applied to that aggregate. Conceptually, this number reflects the *number of changes* that have occurred. When saving a new event to the event store, the expected aggregate version is compared with the current version in storage.

This will help us with speedier queries and ensure we do not repeat any combination of these values.

Payload here would be a serialized representation of the data associated with the event record. This serialization is best done in a human-readable format such as JSON. Several relational database technologies allow us to query and manipulate JSON, such as the following:

- **PostgreSQL** offers **JSON** (text-based) and **JSONB** (binary-optimized JSON) column types. This is ideal when you need frequent reads and writes on JSON fields with complex queries such as filtering or partial updates.
- **MySQL** provides a native JSON column type. It suits systems requiring moderate querying and more straightforward JSON manipulation.
- **Microsoft SQL Server** supports JSON storage in standard `NVARCHAR` columns while offering robust functions for querying and manipulation. It supports **JSON** query and update operations with `JSON_QUERY`, `JSON_VALUE`, and `FOR JSON` clauses.
- **Oracle Database** supports a native **JSON** type, version 21c, which is effective for enterprise systems requiring robust indexing.

The type of database technology employed does play a part in how flexibly and efficiently we can store and retrieve data. Let us look at how we can model our NoSQL data stores.

## Scoping event storage using a non-relational database

**Non-relational databases**, specifically **document databases**, are characterized by their ability to store unstructured data effectively. Let's talk about what this means:

- Schemas are flexible and can evolve. Columns are not mandated in the design phase, so extending and contracting the data based on the immediate need is easy enough.
- Data types are not strictly implemented, so the data structure can evolve at any point without detrimental effects on previously stored records.
- Data can be nested and can contain sequences. This is significant since we do not need to spread related data across multiple documents. One document can represent a denormalized representation of data from several sources.

Popular examples of document stores are **MongoDB**, **Microsoft Azure Cosmos DB**, and **Amazon DynamoDB**. Outside of the specific querying and integration requirements for each database option, the concepts of how documents are formed and stored are the same.

We can outline the document's properties similarly to how a table would look. However, in a document data store, the data is stored in JSON format (unless specifically requested or implemented otherwise). An event entry would look like this:

```
{
  "type": "AppointmentCreatedEvent",
  "aggregateId": "aggregateId-guid-value",
  "data": {
    "doctorId": "doctorId-guid-value",
    "customerId": "customerId-guid-value",
    "dateTime": "recorded-date-time",
  },
  "timestamp": "2022-01-01T21:00:46Z"
}
```

Another advantage to using a document data store is that we can more efficiently represent a record with its event history in a materialized view. That could look like this:

```
{
  "type": "AppointmentCreatedEvent",
  "aggregateId": "aggregateId-guid-value",
  "doctorId": "doctorId-guid-value",
  "customerId": "customerId-guid-value",
  "dateTime": "recorded-date-time",

  "history": [
    {
      "type": "AppointmentCreatedEvent",
      "data": {
        "doctorId": "doctorId-guid-value",
        "customerId": "customerId-guid-value",
        "dateTime": "recorded-date-time",
        ...
      },
    },
  ],
}
```

```
    "timestamp": "2022-01-01T21:00:46Z"
  },
  {
    "type": "AppointmentUpdatedEvent",
    "data": {
      "doctorId": "different-doctorId-guid-value",
      "customerId": "customerId-guid-value",
      "comment": "Update comment here"
    },
    "timestamp": "2022-01-01T21:00:46Z",
    ...
  },
  ...
],
"createdAt": "2022-01-01T21:00:46Z",
...
}
```

This data representation can be advantageous for retrieving a record with all its events, especially if we intend to display this data on a user interface. We have generated a view that acts as both a snapshot of the aggregate data's current state and the stream of events that have affected it. This form of data aggregation helps us reduce some complexity and simplify the concept of event retrieval.

We want our event store to be as performant as possible, so you can consider adding compound and text indexes to the schema to optimize query performance and using partitions to scale horizontally when there is excessive load. You can also use event versioning and schema migration strategies to manage changes to event structures in NoSQL databases.

These optimizations are crucial for handling large event streams. To manage this efficiently, consider storing events in partitioned collections or tables using the aggregate ID as a partition key. This approach ensures that queries for a single aggregate are scoped to a specific partition, reducing the read footprint. You can also implement a paginated event read mechanism to stream events in chunks, improving memory efficiency.

Now that we have scoped our storage options, let us review implementing an event store using PostgreSQL.

## Implementing an event store with PostgreSQL

We will use PostgreSQL to create an event store for events associated with appointment creation. We will implement this new database using code-first and EF Core. To support PostgreSQL, we need the `Npgsql` package. Run the following command in the terminal at the root of the project directory:

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

Now, we can define a new data model in the `Data` folder called `EventEntity`. It will look like this:

```
public class EventEntity
{
    [Key]
    public Guid EventId { get; set; }
    public Guid AggregateId { get; set; }
    public string EventType { get; set; }
    public DateTime EventTimestamp { get; set; }
    [Column(TypeName = "jsonb")]
    public string Payload { get; set; }
}
```

We annotate the `EventId` property to show that it serves as the table's primary key and include an EF Core annotation over the `Payload` column to map to the PostgreSQL JSONB column type. Now, we can define a new data context representing the event database. We call it `EventStoreDbContext` and define it like this:

```
public class EventStoreDbContext(DbContextOptions<EventStoreDbContext>
    options) : DbContext(options)
{
    public DbSet<EventEntity> Events { get; set; }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    { modelBuilder.Entity<EventEntity>().ToTable("appointment_events");
    }
}
```

Now, you can update the `appsettings.json` file with the following line, filling in your specific server, user name, and password details. In production workloads, store secrets in environment variables or a secret vault:

```
{
    "ConnectionStrings": {
```

```
// existing connection string
"EventStoreDatabase": "Host=localhost;Database=event_store;
    Username=yourusername;Password=yourpassword"
}
}
```

We will also add the configuration to the Program.cs file:

```
builder.Services.AddDbContext<EventStoreDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString(
        "EventStoreDatabase")));
```



While we are proceeding with a code-first approach, the event store might not be unique per service and might serve as an event store for several microservices. In this case, you may design and model the database per service (or perform database scaffolding) as needed.

We will proceed with a data store that only serves the appointment booking microservice for simplicity. To create the database, we need to develop and execute a migration. You can do this with the following commands:

```
dotnet ef migrations add InitializeEventStoreDatabase --context
EventStoreDbContext
dotnet ef database update --context EventStoreDbContext
```

Now that we have created our event store, let's review how to implement event-sourcing patterns in our API.

## Implementing event-sourcing patterns

Event sourcing is implemented to provide a single point of reference for the history of what has happened within an aggregate and, by extension, a bounded context. Earlier in this book, we discussed the use of DDD as a design pattern that helps us scope the different services that might be required as we develop our microservices application.

Events can be employed in implementing this pattern to help us model expected outcomes within our bounded contexts. Events are scoped based on the ubiquitous language established within the bounded context and informed by decisions within the domain.

Within the domain, aggregates are responsible for creating domain events, and our domain events are usually raised based on the outcome of some user action or command. It is important to note that domain events are not raised based on actions such as the following:

- Button clicks, mouse moves, page scroll events, or simple application exceptions. Events should be based on the established ubiquitous language of the bounded context. Recall from our chapter on DDD that having a common language per domain helps teams ensure they are on the same page. These bounded contexts are essential to properly defining what should be an event.
- Events from other systems or outside of the current context. It is essential to establish the boundaries between each domain context properly.
- Simple user requests to the system. At this point, a user request is a command. The event is raised based on the outcome of the command.

Let us better understand why domain events are integral to implementing event-sourcing patterns.

## Domain events and event sourcing

Event sourcing uses domain events to store the states an aggregate has gone through. We have already seen that event sourcing requires us to store the record ID, a timestamp, and details that help us understand what the data looked like at that moment. Properly implementing domain events within a bounded context will lay a foundation for a successful implementation of event sourcing, so proper scoping and implementation are essential.

Implementing **domain events** in code can be relatively simple using the `MediatR` library, which was integral in our CQRS pattern implementation. Let us consider introducing domain events to our appointment booking system. As far as we have explored, several activities need to be completed when an appointment is booked in our system. We might also need to extend our system's capabilities to support the idea that changes might be required to the original appointment and should be tracked.

Let us adjust our application to implement domain events.

## Implementing domain events in our application

Domain events in our appointment booking microservice represent significant actions or decisions in the domain, such as scheduling, rescheduling, or canceling an appointment. We must store these events sequentially to implement event sourcing, allowing state reconstruction and a complete audit trail. We can extend our `MediatR` implementation to publish and handle these events.

We start by defining classes that represent these domain events. You must decide how much or little information you would require for the event to be handled effectively. For instance, some events might only need the appointment's ID value. Be sure to scope this correctly and send as much information as required. You do not want to send only the ID and then need to query for additional details and risk potentially many event handlers trying to fetch details from just an ID value.

You may define the following classes in a new `Events` folder in the existing `Models` folder:

```
public class AppointmentCreatedEvent : INotification
{
    public Guid AppointmentId { get; set; }
    public DateTime AppointmentDate { get; set; }
    public Guid PatientId { get; set; }
    public Guid DoctorId { get; set; }
    public TimeSlot Slot { get; set; }
    public Location Location { get; set; }
}

public class AppointmentCancelledEvent : INotification
{
    public Guid AppointmentId { get; set; }
    public string Reason { get; set; }
    public DateTime CancelledAt { get; set; }
}
```

Both models inherit from `INotification`, an interface courtesy of `MediatR`, representing a notification message in a publish-subscribe pattern. It is primarily used to define domain events or messages that do not require a response but are broadcast to multiple handlers who may act on them. Each notification can have one or more handlers who respond to it when it is published. For instance, when an appointment gets created, we might have a handler dispatching an email alert separate from one updating a **SignalR** hub. Handlers implement the `INotificationHandler<T>` interface, where `T` is the notification type.

A notification is more of a command than a query, so feel free to add the following handler to the `Commands` folder. You might also consider creating a new folder called `EventHandlers`. You may review some file and folder recommendations for our CQRS activities to guide you but do what's best for your project and team.

Now, we define the following handler to dispatch our email alert to the patient:

```
public class NotifyEmailAppointmentCreatedHandler (
    IEmailService _emailService) :
    INotificationHandler<AppointmentCreatedEvent>
{
    public async Task Handle(AppointmentCreatedEvent notification,
        CancellationToken cancellationToken)
    {
        // Send Email notifications
        return Task.CompletedTask;
    }
}
```

Our event type, `AppointmentCreatedEvent`, is used as the target type for `INotificationHandler`. This is all it takes to add specific logic sequences to a raised event. It also helps us separate concerns and better isolate bits of code associated with raised events. In the same way, if we wanted to define an event handler for SignalR operations, we could define a second handler for the same event type:

```
public class NotifySignalRHubAppointmentCreatedHandler :
    INotificationHandler<AppointmentCreatedEvent>
{
    public Task Handle(AppointmentCreated notification,
        CancellationToken cancellationToken)
    {
        // SignalR awesomeness here

        return Task.CompletedTask;
    }
}
```

With this adjustment, we can publish an event from the POST action in the API controller like this:

```
public async Task<ActionResult<Appointment>>
PostAppointment(CreateAppointmentCommand createAppointmentCommand)
{
    var appointment = await _mediator.Send(createAppointmentCommand);
    await _mediator.Publish(new AppointmentCreatedEvent{
        AppointmentId = appointment.AppointmentId,
```



```

AppointmentDate = appointment.Slot.Start,
PatientId = appointment.PatientId,
Location = appointment.Location,
Slot = appointment.Slot
});
return CreatedAtAction("GetAppointment",
    new { id = appointment.AppointmentId }, appointment);
}

```

Now, we use the Publish method from the MediatR library to raise an event, and all handlers defined to watch for the specified event type are called into action.

These refactors to our code will introduce even more code and files, but they help us maintain a distributed and loosely coupled code base. Still, we need to implement a handler that stores the data related to the event. We can create a new handler called `SaveAppointmentCreatedEventDetailsHandler`, which will find more details about the appointment and store the event in our Postgres database table. We can define it like this:

```

public class SaveAppointmentCreatedEventDetailsHandler (
    EventStoreDbContext _eventStoreDbContext, /* Additional dependencies*/
    : INotificationHandler<AppointmentCreatedEvent>
{
    public async Task Handle(AppointmentCreatedEvent notification,
        CancellationToken cancellationToken)
    {
        // Use the clients to fetch document, patient and doctor details -
        // shortened for brevity
        // Define a data object with the relevant details to be stored in the
        // payload
        AppointmentDetails appointmentDetails = new (
            /* Detail properties as needed */
        );
        // Save the details to the event store
        EventEntity @event = new ()
        {
            AggregateId = notification.AppointmentId,
            EventId = Guid.NewGuid(),
            EventType = nameof(AppointmentCreatedEvent),
            Payload = JsonSerializer.Serialize(appointmentDetails), // serialize

```

```
//data as JSON
    EventTimestamp = DateTime.UtcNow
};
_eventStoreDbContext.Events.Add(@event);
await _eventStoreDbContext.SaveChangesAsync();
}
}
```

Our events table will be updated with the relevant information in the background every time we create a new appointment.

Other scenarios that you can consider include the following:

- **AppointmentCancelledEvent:** We haven't implemented this, but consider a flag on the Appointment entity that indicates its state (pending, completed, canceled, etc.) and logs each change. You might also need to adjust the payload model (still retaining one standard) to include additional details that might not be captured in the primary data store.
- **AppointmentUpdatedEvent:** What if a patient requests a new time or the location/doctor assigned is changed?

Every event type must include a payload version of the record being manipulated. This way, we can always know the record's state at that time.

Bear in mind that this all happens in the same transaction. While events in CQRS can be seen as a potential alternative to asynchronous communications (for instance, sending an email), the operations all happen in real time. They can burden the API with additional computations in the background. Be deliberate in implementing events and deciding what can be handed off to an external service and message queue versus what needs to be handled internally.

With this activity, we have reviewed how to store domain events cleanly. Now, we need to review read-only operations. Let us review this next.

## Using events for read-only operations

Records stored in the event store can power read-only queries in a system. By leveraging the flexible nature of events, we can shape our payloads to store enough data to fulfill some of our querying needs.

The payload we are storing is modeled from the detailed information returned when the `GetAppointment` endpoint is called. Therefore, we can consider refactoring

GetAppointmentByIdHandler to use the event store to return the details and remove the need for several API calls. It would look like this:

```
public class GetAppointmentByIdHandler(EventStoreDbContext _
eventStoreDbContext) : IRequestHandler<GetAppointmentByIdQuery,
AppointmentDetails>
{
    public async Task<AppointmentDetails> Handle(
        GetAppointmentByIdQuery request, CancellationToken cancellationToken)
    {
        var appointmentEvent = await _eventStoreDbContext.Events
            .Where(e => e.AggregateId.ToString() == request.Id)
            .OrderByDescending(e => e.EventTimestamp)
            .FirstOrDefaultAsync();
        // Additional logic required to parse data based on the event type and
        // potentially modified payload format
        // return serialized payload data, or null if nothing is found
        return appointmentEvent == null ? null : JsonSerializer.Deserialize<
            AppointmentDetails>(appointmentEvent.Payload);
    }
}
```

Our refactored handler now relies on the event store, which helps us implement the fundamental concept of separate read-and-write data stores. We can query our events for the most recent one, parse the payload if necessary (different events might have slightly modified models, though consistency is recommended), and parse the data to be returned.

We can also rely on the event type to help us filter the events that we wish to return. If we are to refactor GetAppointmentsByPatientIdHandler to return appointments that a patient has scheduled, then it would look like this:

```
public class GetAppointmentsByPatientIdHandler(EventStoreDbContext _
eventStoreDbContext) : IRequestHandler<GetAppointmentByPatientIdQuery,
List<AppointmentByPatientId>>
{
    public async Task<List<AppointmentByPatientId>> Handle(
        GetAppointmentByPatientIdQuery request,
        CancellationToken cancellationToken)
    {
        var patientJson = JsonSerializer.Serialize(new { Patient = new {
```

```

        request.PatientId } });
var events = await _eventStoreDbContext.Events
    .Where(e => e.EventType == "AppointmentCreatedEvent" &&
        EF.Functions.JsonContains(e.Payload, patientJson))
    .AsNoTracking()
    .ToListAsync();

List<AppointmentByPatientId> appointments = [];
foreach (var @event in events)
{
    var appointment = JsonSerializer.Deserialize<AppointmentDetails>(
        @event.Payload);

    if (appointment != null && appointment.StartTime < DateTime.UtcNow)
        continue;

    appointments.Add(new AppointmentByPatientId(
        appointment.AppointmentId,
        appointment.Doctor.FirstName + " " + appointment.Doctor.LastName,
        appointment.StartTime
    ));
}
return appointments.OrderBy(q => q.Date).ToList();
}
}

```

We start by forming a JSON representation of the Patient-PatientId key-value pair, as it would appear in the payload that we have stored. Then, we query the events store for all records where the payload contains that key-value pair string. `EF.Functions.JsonContains` is a unique EF Core method that allows us to efficiently interrogate a JSONB column in PostgreSQL for specific matches. Once we have filtered the events based on their type and the patient details, we can sift through and find only events with a future date. This is possible after deserializing the JSON payload into a C# object.

As mentioned before, event sourcing has its pros and cons. The most apparent con in this example is that we must load all the events to find the future-dated ones. This is not the most efficient querying technique, but we are limited by the fact that the JSONB column has limited query and filter support. Also, an event store is optimized for write operations. Using it for complex read queries can lead to performance bottlenecks. Read models are better suited for read operations.

Now that we understand how events can be created, stored, and interrogated, let's examine some best practices and recommendations for implementing event-sourcing patterns.

## Best practices and recommendations

Implementing event sourcing effectively requires careful consideration of design principles, storage, performance, and operational challenges. A critical aspect of this pattern is the design of events themselves. Events should accurately reflect business processes and be named to convey domain significance. Let us review this first.

### Designing events with care

Events are the foundation of event sourcing, so it is crucial to design them carefully. Each event should represent a meaningful, immutable change in the domain. For example, instead of storing generic changes such as `AppointmentUpdated`, use specific events such as `AppointmentScheduled`, `AppointmentRescheduled`, or `AppointmentCancelled`. This specificity makes events more understandable and aligns them closely with business requirements. To ensure forward compatibility, include a version number in each event and consider how to handle schema evolution, such as adding or deprecating properties.

Designing events with forward and backward compatibility is essential to building a resilient and evolvable event-sourced system. Backward compatibility ensures that new versions of your application can still understand and process events created by older system versions. Forward compatibility, though more difficult, allows older versions of your system to ignore or safely process events generated by newer versions. These principles are critical in environments where multiple services may be deployed independently and evolve at different rates. When introducing new fields, make sure the changes are additive, meaning they don't remove or rename existing fields, and defaults are provided for older consumers. Event processors should tolerate missing data and ignore unknown fields they don't recognize.

A common approach to support compatibility is to **version** your event schema explicitly. This can be done by including a `Version` field in the event object, allowing the event processor to choose the correct logic based on the version number. This ensures that your system remains robust in the face of change. This is particularly crucial when replaying historical events to rebuild the state, as it guarantees consistent and accurate behavior even years after an event was initially written.

Another critical practice is ensuring consistency between events and the underlying state of the system. Events should persist atomically with changes to the primary database, ensuring the

system remains consistent despite failure. Event handlers must also be idempotent to avoid unintended side effects when events are replayed or processed multiple times. This guarantees that the same event produces the same outcome, regardless of how often it is handled.

Ensure that events are written with version checks inside a database transaction. This ensures consistency and guards against race conditions. A typical pattern is to load the current aggregate version, compare it to the expected version, and insert the new event only if the versions match, all within a single transaction.

Once we have sorted out the design of events, we need to put mechanisms in place to increase efficiency when retrieving them. A snapshot is a widely touted solution to the inefficiencies that event-read operations introduce.

## Using snapshots for performance optimization

Reading thousands of events can be expensive. Snapshots provide a mechanism to store an aggregate's state at a specific time. When rebuilding the state, start from the latest snapshot and replay only subsequent events. This significantly reduces the computational overhead. For example, create a snapshot of the aggregate's state after every 100 events. A snapshot is a serialized representation of an aggregate's current state at a specific point in time. When creating a snapshot, the system saves the aggregate's state and references the last event applied.

Snapshots significantly improve the performance of state reconstruction and read operations by reducing the number of events that need to be read. You can create snapshots based on application-specific criteria, such as the frequency of changes or the event log size. A step-by-step design of a snapshotting mechanism could look like this:

1. **Define the snapshot structure:** Define a database table that will store the snapshot of your aggregate's state. The EF Core model would look like this:

```
public class AggregateSnapshot
{
    public Guid AggregateId { get; set; }
    public int Version { get; set; }
    public string PayloadSnapshot { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

2. **Determine snapshot frequency:** Decide when to create snapshots. A common approach is to create a snapshot after a certain number of events.

3. **Implement snapshot creation:** Monitor the number of events since the last snapshot within your aggregate. When the threshold is reached, generate a snapshot.
4. **Restoring from a snapshot:** When loading an aggregate, first, attempt to retrieve the latest snapshot and then apply any events that have occurred since that snapshot.

When implementing snapshots in an event-sourced system, one critical consideration is determining how frequently to create them. While snapshots significantly improve the performance of event replay, they also introduce additional storage overhead. Striking the right balance between performance and storage efficiency is essential.

More frequent snapshots can significantly speed up state reconstruction, especially for aggregates with high event volumes. This is particularly beneficial for time-sensitive operations, such as displaying current appointment statuses or generating real-time reports. With snapshots in place, the system only needs to replay recent events after the last snapshot, reducing latency and CPU usage.

However, creating snapshots too frequently can lead to excessive storage consumption, especially if snapshots are stored as full serialized representations of aggregates. This can inflate the database size and impact backup and recovery times. It also introduces the need for careful cleanup strategies to remove outdated snapshots that are no longer necessary.

Another critical factor is the choice of storage and query mechanisms.

## Choosing an appropriate data store and ORM

We discussed earlier the importance of scoping our storage solution. We compared relational and non-relational data storage methods and saw the pros and cons of each approach. Choosing a durable and scalable event store is crucial for reliability. PostgreSQL, with its advanced JSONB support, offers a comprehensive storage method. Coupled with EF Core, there are some limitations to how queries can be written to interrogate the payload's content. It requires custom schemas and logic to handle event streams and ordering and may need optimization for high-throughput event storage and retrieval.

Alternatively, consider using a purpose-built system such as **EventStoreDB**, which is tuned specifically for storing events. It includes default features that help with proper indexing on frequently queried fields such as `AggregateId` or `EventType`, ensuring that queries remain performant even as the event store grows. Additionally, implementing retention policies or archiving older events can help manage storage costs and maintain system performance. However, it may require time to learn its specific APIs and concepts, and it is still growing. It has a smaller community compared to more established databases.

Adhering to these practices enables organizations to build robust, scalable, and maintainable systems that leverage the full potential of event sourcing. Now that we have explored this concept and implementation, let us review what we have learned.

By keeping the read-check and insert within the same transaction, no other writer can interfere between reading the current version and writing the new event. To prevent duplicate version writes, it is also essential to have a version column indexed in the `appointment_events` table and included in your primary key or uniqueness constraint.

Also, a retention policy should be considered, where the length of events in the primary event store is restricted. Not all historical events need to be retained indefinitely for operational use. For instance, after a specific period, older events may no longer be relevant for business processes but might still be valuable for auditing or compliance. In such cases, the policy might specify that only events from the last 2 or 3 years remain in the live store.

Archiving involves moving or compressing these older events into a separate, cost-effective storage solution. For example, archived events can be exported to blob storage, a data lake, or compressed and stored in cold-tier database tables. This keeps the primary event store lean and fast while preserving historical data for future reference, audits, or analytics.

Finally, do not neglect testing. Unit tests validate domain aggregates' behavior and ability to apply and generate events correctly. This helps confirm that the domain model evolves correctly in response to events. Integration tests ensure that events can persist and be retrieved accurately from the underlying event store. Tests verify interactions with the database, including version control, serialization/deserialization, and transactional integrity.

This technique guarantees atomicity and enforces optimistic concurrency, aligning well with the principles of event sourcing.

## Summary

This chapter thoroughly explored the event sourcing pattern, emphasizing its role in modern software architecture, particularly within microservices. Event sourcing records all state changes as a series of immutable events, offering insights into the data's evolution and enabling robust features such as audit trails, state reconstruction, and real-time processing.

An event records a change in the system, such as creating an appointment or processing a payment. Events possess key attributes such as a unique identifier, timestamp, and payload, making them immutable and reliable. This immutability ensures data integrity and supports reproducibility, auditability, and decoupling between components, enabling independent evolution of system modules.



After defining an event, we proceeded to review its implementation. Individual services maintain their data stores, creating data synchronization and consistency challenges. Event sourcing mitigates this by using an event store, which acts as a centralized repository of all changes, offers persistent logs, and facilitates communication between asynchronous systems. Events in such systems serve multiple purposes: persisting data, maintaining audit logs, and allowing for the reconstruction of the entity's state through replaying events.

We thoroughly reviewed implementing an event store, highlighting the importance of append-only data stores for immutability, flexibility in data structures for evolving requirements, and options for using relational or non-relational databases. Relational databases such as PostgreSQL offer features such as JSONB columns for efficient querying, while NoSQL databases provide the advantage of unstructured data storage, allowing for greater adaptability.

Tools such as MediatR were recommended for handling domain events within bounded contexts. The chapter also provided practical code examples for implementing event sourcing using technologies such as .NET and PostgreSQL, including setting up an event store, defining domain events, and handling them through notifications and handlers.

While event sourcing offers significant benefits, it also introduces complexities, such as managing schema evolution, ensuring idempotency in event handling, and addressing performance concerns during event replay. Techniques such as snapshotting were suggested to optimize performance by periodically storing the aggregate's state, reducing the computational overhead of replaying events.

We then concluded with best practices, emphasizing careful event design, consistent storage mechanisms, and thoughtful selection of tools such as PostgreSQL or specialized systems such as EventStoreDB. These practices aim to build scalable, maintainable systems that fully leverage event sourcing's potential for robust data management and system reliability.

It is important to remember that event sourcing adds complexity and that the benefits should be weighed against that added complexity.

In the next chapter, we will examine data management strategies for microservices in more detail.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





---

# Part 2

---

## Database and Storage Design Patterns

This part focuses on persistence choices for autonomous services. You'll design storage per service, apply polyglot persistence where it's beneficial, partition data for scalability with sharding, and maintain cross-service consistency using the Saga pattern.

This part has the following chapters:

- *Anchor 2, Database Design Strategies for Microservices*
- *Anchor 2, Implementing Transactions across Microservices Using the Saga Pattern*



# 8

## Database Design Strategies for Microservices

In the previous chapter, we explored the concepts of event sourcing and event stores. Event sourcing patterns help us reconcile changes made to our data stores across our microservices. Now, we'll pivot to the broader scope of data management in a microservices architecture.

This is a critical challenge and a key enabler of system scalability, resilience, and maintainability. Unlike monolithic applications, where data is typically managed in a centralized database, microservices encourage a decentralized data ownership model. Each service is responsible for data storage, retrieval, and consistency mechanisms, which introduce data integrity, transactions, and inter-service communication complexities.

This chapter explores various strategies for managing data in a microservices environment. We will examine different approaches, such as database per service, shared databases, and event-driven data propagation, highlighting their advantages, trade-offs, and best practices.

Additionally, we will discuss key techniques for ensuring that microservices remain loosely coupled while supporting business processes requiring cross-service data consistency.

After reading this chapter, you will be able to do the following:

- Evaluate and choose the best database development pattern
- Develop databases to meet your microservices application requirements
- Understand more advanced database development patterns

By the end of this chapter, you will understand how to design a robust data management strategy tailored to your microservices architecture, enabling your system to scale effectively while maintaining data integrity and performance.

To begin exploring data management strategies, let's review common database architecture patterns employed in microservices.

## Technical requirements

This chapter does not have strict technical requirements. We review SQL technologies such as **SQL Server**, **PostgreSQL**, and **SQLite**, and **NoSQL** technologies such as **MongoDB** and **Cosmos DB**. It is good to research these in depth to truly appreciate how they can be used in your context and project.

This chapter's code reviews coding concepts explored using **ASP.NET Core**, **SQLite**, and **Entity Framework Core**, so no reference code is needed.

## Exploring database architecture patterns

Effective database design is fundamental to the success of microservices architecture. Unlike traditional monolithic applications that rely on a single, centralized database, microservices require careful planning. A core characteristic of microservices architecture is loose coupling between services. We need to maintain each service's independence and individuality and allow them to interact with the data they need when needed autonomously.

We want to ensure that one service's data manipulation does not inhibit another service's ability to use its data. In the infrastructure, we would also consider having each database in its hosting environment to facilitate maximum uptime for each service. This way, one box outage would not take down all the hosted services. This is called **database per service**.

When implementing database per service, you can consider **polygot persistence** as an extension, where you choose the most suitable database for the developed service. Different microservices will have different needs, and the most appropriate data store must be selected to support the service.

There is also a scenario where resources are limited, and we cannot facilitate multiple databases and server hosting environments. Then, we must consider having one database with adequate logical separations at the database level. This approach works well when a single database technology can be used for multiple microservices. This way, we can quickly identify which database object belongs to which service. The two popular flavors of this approach are **tables per service** and **schema per service**.

There are additional concepts such as the following:

- **Event sourcing:** We have already examined this extensively. It provides auditability, scalability, and possible performance boosts.
- **API composition:** We access data via API calls, as we examined in *Anchor 3, Synchronous Communication between Microservices*.

Let us start by exploring some nuances governing the database per-service approach.

## Database per service

Databases are the foundation of any application. A good or poor database design can determine how efficient your application is, how easily it can be extended, and how efficiently you can write your code.

The database-per-service pattern allows us to choose the best database for the operation that each microservice might complete. A more homogenous technology stack that all developers can identify with and easily maintain would be ideal. Attempting to remain homogenous, however, has led to shortcuts and extensive integration projects in the past, where the need to use one technology overshadowed the opportunity for the best technology for the problem being addressed.

A more specific term for this approach is **polyglot persistence**. This approach allows developers to select the best database type (SQL, NoSQL, Graph DBs, etc.) for a given microservice rather than forcing a single, one-size-fits-all database solution. This will be discussed in more detail later in the chapter. Each microservice will be left to define its data access layers and parameters, and one database failure will not inhibit the operation of the other services. This gets tricky when data ownership is considered since data flows through each service, and sometimes, some details need to be duplicated across different services.

The general idea, however, is that the best database technology is used based on the service's technology and overall business requirements. While this pattern helps to address several concerns, it can also introduce many more:

- **Additional cost:** When discussing having fewer infrastructural dependencies between our services, we introduce more robust networking software and hardware, servers, and licenses for the supporting software.

Using a cloud platform might offset some of the infrastructural and software costs, but even this approach will have a modest price tag.



- **Heterogenous development stack:** This is an advantage from the perspective of attempting to meet the business needs of the microservice as appropriately as possible. The bigger picture, however, comes when we need to source talent to maintain certain technologies that might have been used. Cross-training between teams is recommended but not consistently effective, and a company can risk having a microservice built by past staff members that none of the current ones can maintain.
- **Data synchronization:** We have already mentioned the issue of **eventual consistency**, which leads us to implement contingencies to deal with data being out of sync across multiple databases. Properly executing this introduces additional code and infrastructure overhead.
- **Transactional handling:** We cannot ensure **Atomicity, Consistency, Isolation, and Durability (ACID)** transactions across databases, leading to inconsistent data between data stores. This will require another coding pattern called the **Saga pattern**, which we will discuss further in the next chapter.
- **Communication failure:** Because one microservice cannot directly access another's database, we may need to introduce synchronous microservice communication to complete an operation. This introduces more potential failure points in the operation at hand.

It is always important to remember that every pattern has pros and cons. We should never implement a pattern solely because it is recommended. We should constantly correctly assess the problems that must be addressed and choose the most appropriate solutions and patterns to ensure complete coverage and the best budget.

Now that we have discussed some of the dos and don'ts of the database-per-service pattern, let's discuss using a shared database for our services.

## Shared database

There are times when microservices are developed using identical technology stacks. Take the examples in this book so far, where we have used ASP.NET Core and SQLite for multiple microservice implementations. In this situation, sharing resources for cost-cutting and long-term management is reasonable.

Given that loose coupling is an advantage to the individuality of microservices, careful consideration must be given to what resources are shared between services. When the approach is taken to have microservices share resources, it is best practice to establish clear boundaries between the services.

Efficient resource allocation ensures that each microservice operates optimally without adversely affecting others sharing the same database. Each microservice should maintain its database connection pool to manage connections efficiently and prevent resource contention:

- **Isolation:** By having separate connection pools, services avoid interfering with each other's database connections, enhancing stability.
- **Configuration:** Tune parameters such as maximum pool size, connection timeout, and idle time based on the service's load and performance requirements. For instance, a high-throughput service may require a larger pool size than a service with sporadic database interactions.
- **Monitoring:** Implement monitoring to track connection pool usage and identify potential issues such as connection leaks or saturation.

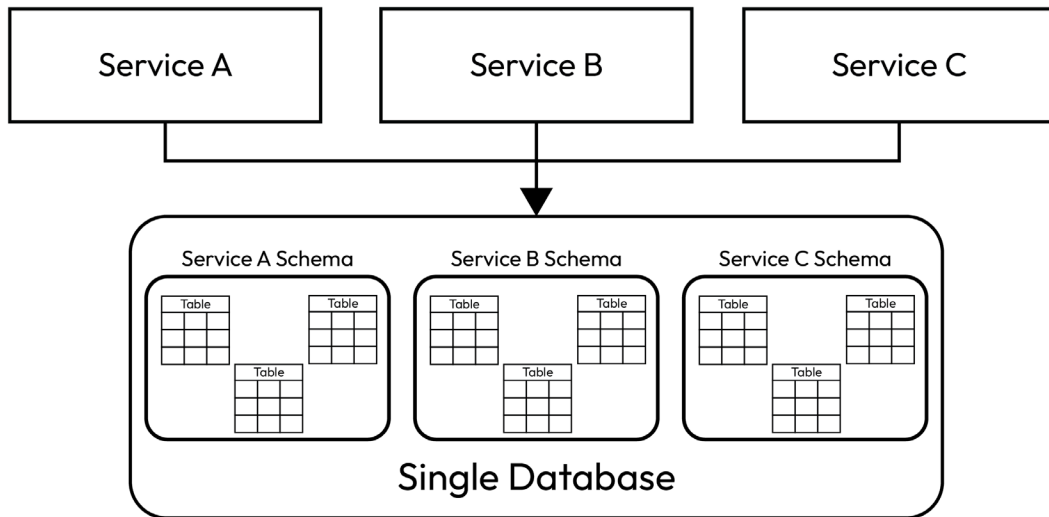
We can also implement resource quotas at the database level to prevent any single service from monopolizing shared resources:

- **User-level quotas:** Assign database users with specific CPU usage, memory, and I/O operations quotas for each service.
- **Workload management:** Utilize database features to prioritize workloads, ensuring critical services maintain performance during peak times
- **Alerts and throttling:** To maintain overall system health, set up alerts for quota breaches and implement throttling mechanisms

Relative to database design, this doesn't mean you need to have different databases, but you can develop clear naming conventions and resource allocations for each service. In the case of using relational databases, some techniques that can be used include the following:

- **Tables-per-service:** We can define tables that are optimized for the data to be stored for each service. We will model these tables in the microservice code and ensure we only include them. In this model, it is common that some tables are denormalized representations of data that can be found in other tables to increase query efficiency and service the specific needs of the service they are created for, which requires the data in this format.
- **Schema-per-service:** Relational databases allow us to specify schema values to categorize our tables. A schema is an organizational unit in a database that helps us categorize tables. We can use these to organize tables per service. Like the tables-per-service pattern, the tables in each schema are optimized based on the needs of the matching microservice. We also have a better opportunity to tune access rights and restrictions per schema and reduce the security administrative overhead.

These conventions aid in reducing confusion, simplifying onboarding for new developers, and streamlining database maintenance tasks. *Figure 8.1* shows multiple services in one database.



*Figure 8.1 – Multiple services share one database, but schemas are created for each service to preserve segregation and data autonomy*

Implementing tables and schemas per service requires the fewest resources since it would be the same as building one application on top of one database. The downside is that we retain a single point of failure and do not adequately scale the varied service demands. Another critical benefit of this pattern is that it makes it easy to implement transactions. We mentioned earlier that a distributed system will require more advanced patterns, such as the Saga pattern. Since we are using one database, using a single ORM to manage an operation that may span multiple tables used by numerous services is more manageable.

Although technically feasible, this model significantly compromises service independence and leads to tightly coupled components that are difficult to scale, evolve, or deploy independently:

- **Increased coupling:** One of the defining goals of microservices is service autonomy. A shared database violates this principle by creating tight coupling at the data level since multiple services depend on the same tables or data structures. A schema change required by one service could inadvertently break another, and business logic embedded in database triggers or stored procedures may serve multiple services, complicating reasoning and maintenance. At this point, services become dependent on each other's internal implementation details, defeating the purpose of abstraction and encapsulation.

- **Performance bottlenecks and resource contention:** As the number of services increases, or if a particular service has higher traffic volumes, the database becomes a critical bottleneck. We already mentioned that setting resource thresholds for each service is essential. Still, a tangible risk is that the system might experience degraded performance across services due to resource contention. If one service overwhelms the database, others may suffer downtime even if they are functionally unrelated.
- **Reduced deployment agility:** Services should be independently deployable in a properly decoupled microservices architecture. However, a shared database requires schema changes to be carefully coordinated across all dependent services to avoid runtime errors. This restricts the deployment of a single service and often requires a synchronized release plan with other teams. This can significantly slow down the software delivery lifecycle and negate the productivity gains typically associated with microservices.

Now that we have explored some database design patterns and strategies, let's review approaches to ensure good database design.

## Developing a database

Developing a capable database is a fundamental skill for a modern developer. This role was once dedicated to the database developer in a team whose sole purpose was to do all things database. The developer would write code to interact with the database based on the application's needs.

More recently, the role of a typical application developer has evolved into what is now called a **full stack developer** role. Modern developers need as much application development knowledge as they do database development knowledge. It is now very common to see teams of two to three developers who work on a microservice team and can develop and maintain the user interface, application code, and database.

Developing a database transcends one's comfort level with the technology being used. It is also essential to consider the team's experience with the technology. Many developers neglect to consult the business and fully understand the business requirements before implementing the technology. This often leads to poor design, rework, and additional maintenance during the application's lifetime. Before settling on a technology, you should thoroughly and adequately assess the system's needs, entities, business processes, and data security and retention policies.

Since we are discussing microservices, we have the unique opportunity to build smaller applications and data stores for tranches of the entire application. This makes it easier for us to ingest and analyze the storage needs of the service that we are focusing on and reduces the

overall complexity of building an extensive database as a catch-all, thus reducing the margin for error during the design phase. As discussed previously, we can also make better choices about the most appropriate database, influencing our design considerations.

Some services need relational integrity based on the nature of the data they process; others need to produce results faster than they need to be correct, and some only require a **key-value** store. The following section assesses the pros and cons of using relational databases.

## Relational databases

Relational databases have been a mainstay for years. For good reasons, they have dominated the database technology landscape for some time. They are built on rigid principles that complement clean and efficient data storage while ensuring accuracy in what is stored.

The strict rules ensure **data integrity**, **consistency**, and **reliability**. The foundation of these rules is the ACID principles, designed to maintain database reliability, even in multi-user or distributed environments.

- **Atomicity:** This ensures that each database transaction is all or nothing. If any part of the transaction fails, the entire transaction is rolled back to maintain data integrity.

**Example:** In our health care management system, when a patient books an appointment and needs to pay a fee, we must ensure that the booking is saved and the payment is successful. If either fails, we roll back the entire operation and save nothing to the database. This is accomplished through a construct called **transactions**.

- **Consistency:** This ensures that a transaction brings the database from one valid state to another, adhering to all defined rules, constraints, and validations.

**Example:** A database schema enforcing a constraint that prevents a patient from being added without a first and last name. Transactions violating this rule are rejected.

- **Isolation:** This ensures that concurrent transactions execute independently without interfering with each other, preventing data anomalies.

**Example:** Due to proper transaction isolation, two users attempting to book the same appointment slot for the same doctor should be unable to reserve it simultaneously.

- **Durability:** This ensures that once a transaction is committed, it remains permanently stored, even in a system crash.

**Example:** After confirming an appointment, the details should remain intact and retrievable even if the system restarts.

Some of the most popular relational database management systems include the following:

- **SQL Server:** Microsoft's flagship relational database management system – individuals and enterprises use it widely for application development.
- **MySQL/MariaDB:** MySQL is a traditionally open source and free-to-use database management system mainly used for PHP development. MariaDB was forked from the original MySQL code base and is maintained by a community of developers to maintain the free-to-use policy.
- **PostgreSQL:** A free and open source database management system that is robust enough to handle a wide range of workloads, from individual projects to data warehouses.
- **Oracle Database:** Oracle Corporation's flagship database management system is designed to handle a wide range of operations, including real-time transaction processing, data warehousing, and mixed workloads.
- **IBM DB2:** This is developed by IBM on top of one of the most reliable systems for high transaction and traffic enterprise settings. This database supports relational and object-relational structures.
- **SQLite** is a free and lightweight database storage option for a quick and easy database. Unlike most alternatives that require a server, often a dedicated machine, SQLite databases can live in the same filesystem as the app they are being integrated into, making them an excellent choice for mobile-first apps.

Relational databases store data in tables, each representing an entity. The *relationships* between these entities define how the data is connected. Sound design principles encourage you to have a *primary key* column in each table, which will always have a unique value, to identify a record uniquely. This primary key value is then referenced by other tables in the form of a *foreign key*, which helps to reduce the number of times data is repeated across tables.

Through *normalization*, we assess where data is shared between entities and introduce *foreign keys* to ensure data is efficiently shared across multiple tables. Each foreign key represents a reference or *index* created between each table. Once this association is made between the primary and foreign keys, we create a relationship that introduces a *constraint* or restriction on what values are possible in the foreign key column. This is also referred to as *referential integrity*.

Relationships can be further defined by their *cardinality*. This refers to the nature by which the primary key values will be referenced in other tables. The most common cardinalities are as follows:

- **One-to-one:** A primary key value is referenced once in another table as a foreign key. For instance, if a patient can only have one address on record, the table storing the address cannot have multiple references to the same patient.
- **One-to-many:** The primary key value can be referenced multiple times in another table. For instance, if a customer has made multiple orders, then one customer’s ID is referenced numerous times in the orders table.
- **Many-to-many:** A primary key can be referenced multiple times as a foreign key in another table, and that table’s primary key can be referenced multiple times in the original table. This can be confusing, and implementing it will directly violate the referential integrity we are fighting to maintain. A *linker table* is introduced as an intermediary to associate the different combinations of key values from either table.

Using our healthcare management system as an example, if we revisit how appointments are tracked relative to the customer who has made them, we will see that we implemented a reference point between the customer’s ID, their primary key, and the appointment. This ensures we do not repeat the customer’s information whenever an appointment is booked. An appointment can also not be created for a customer who is not already in the system.

Figure 8.2 shows a typical relationship.

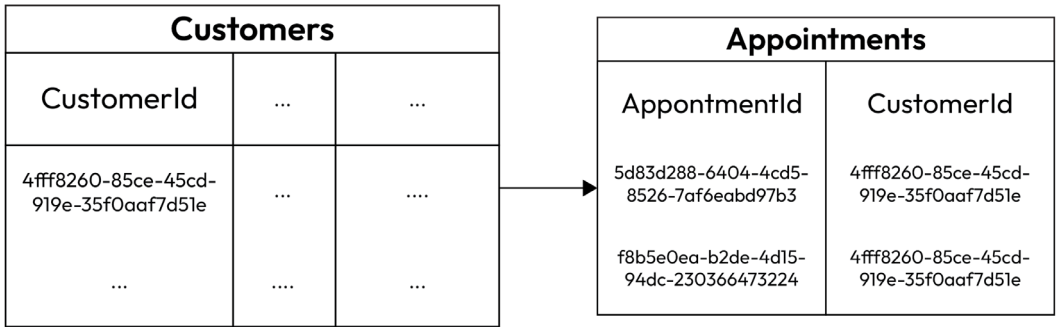


Figure 8.2 – An example of a one-to-many relationship where one customer is referenced many times in the appointments table

Many-to-many relationships occur often, and it is essential to recognize them. Building on the example of customers and their appointments, we can expand and see that we also need to associate a customer with a room for an appointment. The same room is not always guaranteed,

leading us to realize that many customers will book appointments, which might happen in many rooms. If we try a direct association, then details of the customer or the room must be repeated to reflect the possible combinations of customers, rooms, and appointments.

Ideally, we would have a table storing rooms and the associated details, a table storing customers, and an appointments table sitting between the two, seeking to associate them on the day as needed.

Figure 8.3 shows a typical many-to-many relationship.

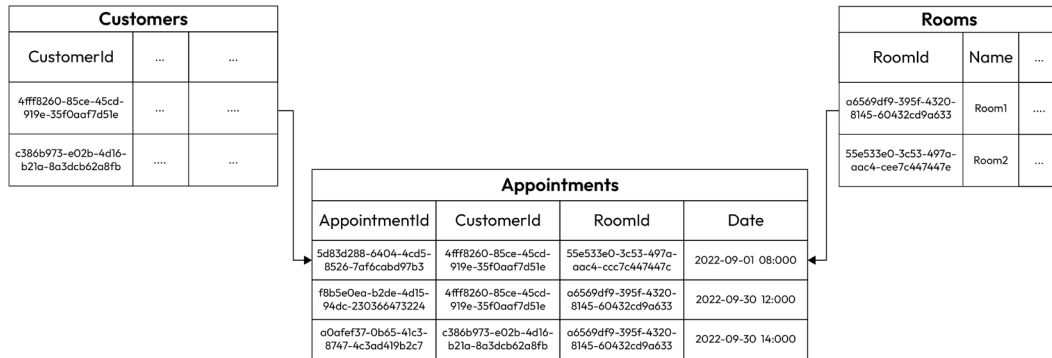


Figure 8.3 – An example of a many-to-many relationship where we associate many records from two tables using an intermediary linker table

We have already discussed ACID principles and why they are essential. Relational databases are designed to ensure that these principles can be observed as a default mode of operation. This makes changing the layout and references of tables relatively tricky, especially if the changes involve changing how tables are related to each other.

Another potential drawback of relational database storage is performance with large datasets. Relational databases are typically efficient in data storage, retrieval, and speed. They are built to manage large workloads, so the technology is hardly at fault, but our design will either complement speed or inhibit it.

These are trade-offs between proper relational database design and upholding relational integrity. Design principles favor normalization, where data bits are spread across multiple tables. This is good until we need the data and traverse multiple tables with potentially thousands of records to get it. Considering this deficiency, NoSQL or document storage databases became increasingly popular as they sought to store data in one place, making the retrieval process much faster.

We will discuss the use of non-relational data stores next.



## Non-relational databases

Non-relational databases significantly altered the way we viewed data storage principles and were widely adopted by developers everywhere. That might be a naive assessment of their impact, but the fact remains that they introduced a dimension to data storage that was not very popular or widely used. They proposed a more flexible and scalable data storage technique that favored a more agile development style.

Agile development hinges on morphing a project as we go along. This means the excess scoping and planning recommended when using a relational data store would not be necessary upfront. Instead, we could start with the idea of a system and begin using a non-relational data store to accommodate that tiny part. As the system evolves, so can the data store, with minimal data loss or attrition risk.

NoSQL databases are designed from the ground up to support horizontal scalability and high availability. This makes them especially well-suited to microservices environments where services may need to scale independently and elastically in response to load.

One key mechanism that enables NoSQL systems to scale horizontally is sharding, the practice of partitioning data across multiple nodes (or shards), each responsible for a subset of the dataset. For example, **MongoDB** and **Cassandra** automatically distribute data across shards based on a shard key. At the same time, **Couchbase** and **Amazon DynamoDB** use partitioning and consistent hashing to distribute data and ensure even load balancing.

This allows independent scaling, so new nodes can be added to the cluster to handle increased traffic or storage needs without downtime. Replication and distributed consensus mechanisms ensure that data remains available even in the event of node failures, and queries can be routed directly to the shard responsible for the requested data, improving response times.

Non-relational databases are also called **NoSQL databases**, or *not only SQL* databases, and they favor a non-tabular data storage model. The most popular NoSQL databases use document-style storage, where each record is stored in one document containing all the details needed for that record. This directly violates the principles of normalization, which would have us spread the data to reduce possible redundancy. However, the primary advantage of this model is the speed with which we can write and retrieve data since all the details are in one place.

Using the example of our health care appointment management system, if we were to store an appointment in a document, it would look like this.

```
{
  "AppointmentId": "5d83d288-6404-4cd5-8526-7af6eabd97b3",
  "Date": "2022-09-01 08:000",
  "Room": {
    "Id": "4fff8260-85ce-45cd-919e-35f0aaf7d51e",
    "Name": "Room 1"
  },
  "Customer": {
    "Id": "4fde8261-85df-65cd-819f-45f0bae9f52e",
    "FirstName": "John",
    "LastName": "Higgins"
  }
}
```

Unlike the relational model, where these details were split across multiple tables and referenced, we can include all the details needed to assess what an appointment entails fully.

Types of NoSQL databases include the following:

- **Document database:** Stores data as **JavaScript Object Notation (JSON)** objects. This JSON document outlines fields and values and can support a hierarchy of JSON objects and collections all in one document. Popular examples include *MongoDB*, *Cosmos DB*, and *CouchDB*.
- **Key-value database:** This type stores data in simple key-value pairs. Values are usually stored as string values and do not natively support storing complex data objects. They are typically used as quick lookup storage areas, such as for application caching. A popular example of this is *Redis*.
- **Graph database:** This type stores data in nodes and edges. A node stores information on an object, such as a person or place, and the edges represent relationships between the nodes. These relationships link how data points relate to each other, as opposed to how records relate. A popular example of this is *Neo4j*.

The clear advantages of using NoSQL document databases come from how data is stored. Unlike the relational model, we do not need to traverse many relationships and tables to get a fully human-readable data record. As the preceding example shows, we can store all the details in one document and scale from there. This is handy when we need performant read operations and do not want to compromise system performance with complex queries.

A clear disadvantage, however, comes from how storage is handled. Document databases do very little, if anything at all, to reduce the risk of data redundancy. We will end up repeating details of what would have been related data, which means that maintenance of this data, in the long run, could become a problem. Using our customer records as examples, if we needed to add a new data point for each customer who booked an appointment, we would need to traverse the records to make the one change. This would be a much easier and more efficient operation using a relational model.

Now that we have discovered some advantages and disadvantages of relational and non-relational database stores, let's discuss scenarios in which either would be a better choice.

## Choosing a database technology

When choosing the best technology for anything, we are faced with several factors:

- **Maintainability:** How easy is this technology to maintain? Does it have excessive infrastructural and software requirements? How often are updates and security patches produced for the technology? Ultimately, we want to ensure we do not regret our selection weeks into starting the project.
- **Extensibility:** To what extent can I use this technology to implement my software? What happens when the requirements change and the project needs to adapt? We want to ensure that our technology is not too rigid and can support the dynamics of the business needs.
- **Supporting technologies:** Is the technology stack I use the best fit? We are often forced to take shortcuts and implement methods, sometimes contrary to best practices, to facilitate matching technologies that are not the best fit for each other. With more libraries being produced to support integrations of heterogeneous technology stacks, this is becoming less of an issue. However, it is something we want to consider from day one.
- **Comfort level:** How comfortable are you and the team with the technology? Using new technology and expanding your scope and experience is always lovely, but it is crucial to recognize your limitations and knowledge gaps. Training can just as quickly address these, but we always want to ensure we can handle the technology we choose and are less prone to surprises in the long run.
- **Maturity:** How mature is the technology? In some development circles, technology lives for months at a time. We don't want to embrace new technology the day it is released without doing our due diligence. Seek to choose more mature technologies that have been tried and proven, with strong enterprise or community support and documentation.

- **Appropriateness:** Lastly, how appropriate is the technology for the application? We want to use the best technology possible to do the job. However, this sometimes gets compromised when considering the bigger picture, including budget constraints, project type, team experience, and the business's risk appetite. Still, we want to ensure that our chosen technology can adequately address the project's needs as much as possible. Consider read/write throughput and latency relative to the data requirements of each service.

Now, let us narrow this to database development. We have discussed the pros and cons of the different types of database models and the other technology providers for each. While we are not limited to the options outlined above, they serve as guidelines to help us make assessments from the most unbiased point of view possible.

In microservices, we want to choose the best database for the service's needs. There are no hard and fast rules regarding which storage mechanism should be used, but there are recommendations that can help guide you during the system design.

You can consider using relational databases in the following scenarios:

- **Working with complex reports and queries:** Relational databases can run efficient queries across large datasets and are a much better storage option for generating reports.
- **Working with a high-transaction application:** Relational databases are a good fit for heavy-duty and complex transaction operations. They do a good job of ensuring data integrity and stability.
- **You require ACID compliance:** Relational databases are based on ACID principles, which can help protect your data and ensure its accuracy and completeness.
- **Your service will not evolve rapidly:** If your service doesn't have changing requirements, a database should be easy to design and maintain in the long run.

You can consider using a non-relational database in the following scenarios:

- **Your service constantly evolves:** You need a flexible data store that can adapt to new requirements without too much disruption.
- **If you anticipate that data may not always be clean or meet a certain standard:** Given our flexibility with non-relational data stores, we accommodate data of varying levels of accuracy and completeness.
- **You need to support rapid scaling:** This point goes hand in hand with the need to evolve the data store based on the business's needs so we can ensure that the database can be changed with minimal code changes and lower costs.

Now that we have explored some of the significant considerations regarding choosing a database technology, let's review our options for interacting with a database from code.

## Choosing an ORM

Choosing an **ORM** is an essential part of the design process. This lays a foundation for how our application will communicate with our database. The abbreviation **ORM** is short for **Object Relational Mapping**. Every language supports some form of ORM. Sometimes, this is built into the language, and sometimes, developers and architects contract an external package or library to support database-related operations. We have already explored two popular ORMs in .NET, **Entity Framework Core** and **Dapper**. The Stack Overflow team developed Dapper, which is known for being extremely fast, lightweight, and easy to integrate into high-performance and straightforward applications. While Entity Framework Core offers a comprehensive suite of features, including change tracking, lazy loading, and automated migrations, Dapper provides a more focused, performant alternative, especially when you want to maintain control over your SQL and maximize execution speed. It executes raw SQL and maps the result directly to C# objects with minimal overhead. Benchmarks consistently show that Dapper's performance is very close to raw ADO.NET.

**Entity Framework Core (EF Core)** offers built-in support for some NoSQL databases, notably Azure Cosmos DB. This integration allows developers to work with NoSQL data stores using EF Core's familiar patterns and abstractions. On the other hand, Dapper does not natively support NoSQL databases in the same seamless manner as EF Core does with providers such as Azure Cosmos DB or others. EF Core is the more suitable choice if you're looking for structured support for NoSQL databases within an ORM framework.

Where neither EF Core nor Dapper has direct support for the selected database technology, it is best to confirm the existence of a stable and mature support library for your chosen database. For instance, if you are using MongoDB, the `MongoDB.Driver` is the official NuGet package provided by MongoDB for integrating MongoDB with .NET applications. It is a mature and actively maintained library, offering comprehensive support for MongoDB's features within the .NET ecosystem.

Entity Framework Core has undergone constant improvement and is currently open source. It provides excellent interfaces and abstractions that reduce the need for specific code to be written based on the database being accessed. This is a significant feature as it allows us to reuse code across different database technologies and helps us to be flexible regarding the database technology being used. We can easily change the database technology without affecting the main application and its operations. Barring the presence of any obvious bias on why we should use Entity Framework in our .NET projects, it does help that we can keep our technology stack homogenous.

We have already explored at length how to install EF Core in our .NET project and what it takes to create a database and have it modeled in our application. We will skip those details here.

Through EF Core, we can use **LINQ** syntax in C# and execute a query. EF Core will attempt to generate the most efficient SQL syntax, carry out the operation, and return the data in the form of objects fashioned by the class that models the table. This process is called **materialization**. We can then interact with the records as standard C# objects. The following example shows these things:

- How a database context can be initialized (representing a connection to the database)
- How a table can be queried using LINQ
- How the results can be examined to extract relevant data

```
// Initialize a connection to the database
using var context = new ApplicationDatabaseContext();

// Go to the database and retrieve a list of patients
var patients = await context.Patients.ToListAsync();

// Iterate through the list and print to the screen
foreach (var patient in patients)
{
    Console.WriteLine($"{patient.FirstName} {patient.LastName}");
}
```

EF Core and LINQ's abstraction allows us to write C# code without worrying about the type of database being queried. This means the same code works similarly for SQL Server, SQLite, and Azure Cosmos DB. EF Core gives us a complete way to interact with our database and carry out our operations without needing to break much from our C# syntax. To go beyond the preceding example, EF Core has full support for dependency injection, which makes connection management and garbage collection almost a non-issue.

A fundamental step not addressed above concerns database development techniques, which govern how to create or use an existing database in our project.

## Choosing a database development technique

This section will discuss the possible approaches to materializing database support for our application using EF Core.

There are two popular types of development techniques:

- **Code-first:** This technique allows us to model the database fluently alongside our application code. We create the data models using standard C# classes and manage the changes to the data models and eventual database using *migrations*.
- **Schema-first:** Also called **database-first**, this technique involves creating the database using the usual database management tools. We then scaffold the database into our application. Using EF Core, we will get a database context class representing the database and all its objects, classes per table, and view.

Up to this point, all our code examples involving EF Core used migrations and the relevant .NET CLI commands to create a database based on the database context and entity model classes in our code. Therefore, we have already seen how to create and apply a migration. The concept of migrations is not unique to EF Core but is present in several other frameworks in other languages. It is a simple process that tracks all the changes being made to a data model as we write our applications and the tables and objects evolve. A migration will evaluate the changes being applied to the database and generate commands (which eventually become SQL scripts) to effect those changes against the database.

Once a migration is created, a new file is generated that contains a ledger of all the changes that are to be made to the database. This includes changes to the data context, models or attributes, and constraints. This generated code will look different based on the database engine that EF Core is configured to target. For instance, **constraints** and **data types** will look differently for a migration targeting SQLite than one targeting PostgreSQL. Also, note that not all configurations in EF Core can be applied to all database engines.

The migration instruction class file contains Up and Down methods that outline what changes will be applied and what will be undone if the migration is reversed. This makes it convenient for us to ensure that the intended changes are carried out and that we can remove a migration and make corrections before these changes are applied with this command:

```
dotnet ef migrations remove
```

The `database update` command that was used to execute the first migration and create the database is the same command that is used to apply subsequent migrations. The `remove` command will only work if the last migration hasn't been applied to the database. If it has been used, you must revert the database to the previous migration before removing it.

To roll back your database to a previous migration in EF Core, follow these steps:

1. List all migrations to determine the specific migration you want to revert to:

```
dotnet ef migrations list
```

2. Use the database update command to revert the database to the specified migration:

```
dotnet ef database update <MigrationName>
```

Rolling back migrations can lead to data loss, especially if the reverted migrations included operations such as dropping tables or columns. Ensure you have appropriate backups before performing rollbacks.

Once a migration is applied to a database, a table called `__EFMigrationsHistory` is created (on the first run) or updated (on subsequent migrations). This table is checked each time to determine which migrations have already been applied. This ensures that only pending migrations are executed, preventing redundant operations and potential errors.

The main advantages of the code-first approach are as follows:

- **Development speed:** You can write domain classes without designing the database upfront. This benefits those with a programming background who may not have extensive database design experience.
- **Control over code:** It provides complete control over the entity classes, allowing for cleaner code without the clutter of auto-generated content. Developers can define and manage their classes precisely as needed.
- **Version control and migrations:** Changes to the model are tracked through code, facilitating version control. EF Core's migration feature allows for seamless updates to the database schema as the model evolves, ensuring consistency between the code and the database.
- **Flexibility in complex relationships:** The code-first approach enables developers to express complex relationships using code, which can be particularly useful when the database schema is not yet defined or is expected to evolve.

While these can be compelling reasons to use the code-first approach, handling migrations and database updates has a steep learning curve. Developers need a solid understanding of C# and EF Core's configurations to manage migrations effectively and ensure the code accurately reflects the desired database schema. Migrations are designed to ensure that you always know the database's version and the current state of the changes applied.



Given the automation they support, it is easy to use them during development and quickly manage changes. However, this can become complicated when working in a team and several people simultaneously make migration changes. The changes can fall out of sync and lead to hours of triaging. When in production, this becomes more complicated, and it is recommended that you manage how migrations are correctly applied.

For developers who prefer SQL scripts (like me), EF Core allows us to generate idempotent migration scripts to ensure that database schema updates can be applied safely, regardless of the database's current state. An idempotent script includes checks that prevent the re-application of already executed migrations, making it suitable for deploying to multiple environments at different migration stages:

```
dotnet ef migrations script --idempotent --output "path/to/your/script.sql"
```

Certain database providers may have specific requirements or limitations when generating idempotent scripts. For example, issues have been reported with PostgreSQL in some EF Core versions.

While the code-first approach in EF Core empowers developers with greater control, flexibility, and efficiency, especially during the initial stages of application development, when there is already a database, we do not have the liberty to use this approach. In this case, we want to model the existing database, its entities, and constraints as they are. The process of modeling an existing database in a project is called **scaffolding**. EF Core supports scaffolding a database with this command:

```
dotnet ef dbcontext scaffold "Your_Connection_String" Microsoft.EntityFrameworkCore.Sqlite -o Models
```

This command states that we wish to generate a database context (dbcontext) based on the database located at the provided connection string. Based on the technology of the target database (SQL Server, Oracle, etc.), this connection string will be different. In this example, we are scaffolding an SQLite database into our application, so we specify the appropriate database provider package and output the files generated to a directory in our project called `Models`.

This command can be more complicated if multiple projects are in the solution. We must specify the startup project and the project that contains the database context and model files. This is a more detailed version of the command:

```
dotnet ef dbcontext scaffold "Your_Connection_String" Microsoft.  
EntityFrameworkCore.Sqlite --project YourDataProject --startup-project  
YourStartupProject --output-dir Models --context-dir Data --context  
YourDbContextName -force
```

This command must be executed each time a change is made to the database, along with the `-force` flag, to ensure that existing files are overwritten accordingly. Given that the changes will likely happen at the database level, there is no innate tracking mechanism to track which version the database is at. Therefore, code-first is a very popular option for database development as its use of migrations helps us solve this problem.

EF Core contains rich features that make database development and management more manageable for a developer who might not be a database developer or expert. It is widely used, well-documented, maintained, and generally recommended as the ORM when using a supported database in .NET development.

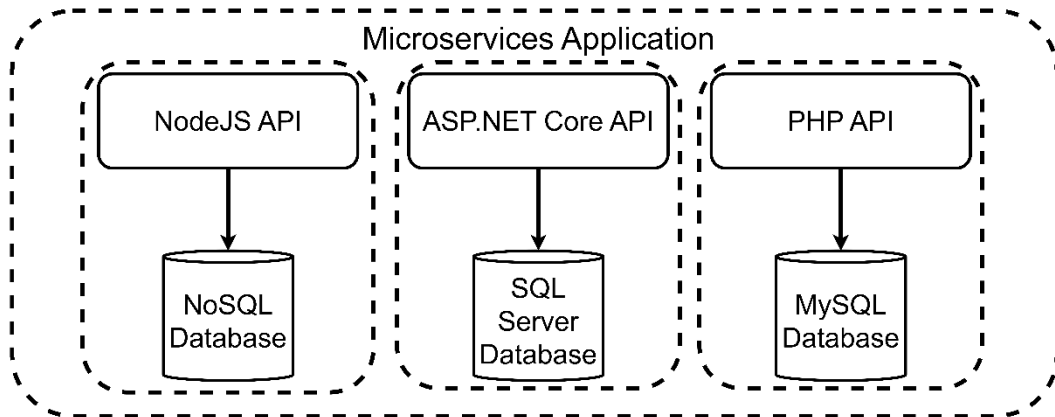
Now that we have examined some hands-on concepts regarding database development, let's explore some best practices and considerations for databases in microservices applications.

## Best practices and recommendations

Developing databases within a microservices architecture requires careful planning and adherence to best practices to ensure scalability, maintainability, and data consistency. We have already looked at higher-level concepts, such as when to consider a database-per-service or single database technique. A significant takeaway from the database-per-service approach is that each service might require a different database technology. This is known as **polyglot persistence** and is one of the major talking points of microservices development. Let us dive more deeply into the best ways to support this need.

### The polyglot persistence strategy

The polyglot persistence strategy uses multiple, diverse data storage technologies within the same application, each chosen to suit specific requirements. Each service can utilize the database technology that best fits its data storage and retrieval needs in a microservices architecture. *Figure 8.4* shows multiple services to individual databases.



*Figure 8.4 – Multiple services can be built using different technologies alongside the most appropriate development framework technology*

Microservices allow multiple teams to choose the best technology stack to implement the solution and, by extension, the best type of database technology to complement the technology and the problem. Some development frameworks work best with certain database technologies, making selecting the entire stack for a particular microservice easier.

Some of the tangible benefits of this approach include the following:

- **Optimized data management:** It allows specialized databases tailored to specific data models and access patterns, enhancing performance and efficiency
- **Flexibility:** It enables services to evolve independently, adopting new storage technologies as requirements change
- **Resilience:** It isolates data-related issues within a single service, preventing them from affecting the entire system

As already discussed, several challenges must be overcome when implementing a database per service, and this might become even more complicated when supporting several technologies. Some serious implementation considerations are the following:

- **Data consistency:** Managing consistency across different databases can be challenging. Implementing strategies such as **event sourcing** can help. Combined with polyglot persistence, it enables robust auditing and temporal data management. It facilitates polyglot persistence by decoupling the **write model** from the **read model**, allowing projections into one or more query-optimized databases, each potentially using different technologies.

- **Operational complexity:** Maintaining multiple database systems increases operational overhead. As mentioned, ensure your team has the expertise to manage diverse technologies. This now means maintaining several environments and devising different deployment strategies to handle each technology's different needs.
- **Data integration:** Aggregating data from different sources will require additional tools and services. Since different databases store data differently, aggregating heterogeneous data requires transformation logic to normalize and reshape it into a unified format. Effective data integration often requires dedicated tools and services such as **Extract, Transform, Load (ETL)** tools, data pipelines (Apache Kafka, Azure Event Hubs, etc.) to stream events across services, and data warehouses (Snowflake, Azure Synapse, Amazon Redshift) to centralize data for querying.

Consider our healthcare application comprising multiple microservices, each responsible for distinct functionalities:

- **Patient records service:** This service manages patient demographics, medical histories, and treatment plans. A relational database is best for storing this structured data with complex relationships, ensuring data integrity and supporting ACID transactions.
- **Appointment scheduling service:** This service handles scheduling, rescheduling, and canceling patient appointments. To date, we have used a relational database for this. Still, we could consider using a *NoSQL database* for rapid read-and-write operations since all the appointment details can be stored in one document record.
- **Documents service:** This stores and retrieves sizable medical documents such as X-rays, MRIs, and CT scans. It can leverage an object storage system such as **Amazon S3** or **Azure Blob** storage to manage large binary files, providing scalable storage and efficient retrieval.

While polyglot persistence solves the issue of choosing the best solution for a problem, it does not address common scalability issues that we may face as our application's user base grows. Database sharing is a commonly touted technique for supporting scaling needs. We will discuss this next.

## Database sharding

Database sharding addresses the challenge of scaling a single database to handle large volumes of data and high traffic loads. **Sharding** is a form of horizontal partitioning in which a database is divided into smaller, more manageable pieces called **shards**. Each shard contains a subset of the data, and together, they comprise the entire dataset. This strategy enhances performance and

scalability by distributing the data and the associated load across multiple servers. This approach provides benefits such as the following:

- **Improved performance:** The system can handle more read and write operations concurrently by distributing data across multiple shards, reducing bottlenecks
- **Enhanced scalability:** Sharding allows adding more shards to accommodate growing data volumes, facilitating horizontal scaling
- **Isolation:** Failures or heavy loads in one shard do not directly impact others, enhancing fault tolerance

Suppose our healthcare system is a globally used software (SaaS) service that manages millions of patient records across various regions. In that case, we can implement sharding based on geographical areas to optimize data access and system performance. A cloud platform that facilitates global reach and stability would best support sharding adequately.

While several platforms offer built-in sharding capabilities, several standard sharding techniques are used independently of any specific vendor or cloud platform. The first one is **range-based sharding**. Data is partitioned based on a continuous range of a particular column, typically an identifier or a timestamp. This is the easiest method to implement and increases the performance of queries that target a specified range.

Another method is **hash-based sharding**, in which a hash function is applied to a sharding key (e.g., user ID, tenant ID) to determine which shard the data should go to. With a well-designed system, even data distribution can be achieved. This method is complicated to scale, making it more challenging to perform range queries.

While several capable cloud platforms support this need, I will focus on Microsoft Azure's facilities. For relational databases, Azure provides the **Elastic Database** tools to facilitate sharding. These tools enable applications to scale out by distributing data across multiple databases, known as shards. A key component is the **Shard Map Manager**, which maintains metadata on data distribution across shards, allowing applications to route requests to the appropriate database based on the **sharding key**.

A sharding key is a specific field or set of fields within a dataset that determines how data is partitioned across multiple shards in a sharded database architecture. The shard key's value dictates the data distribution, ensuring that related data is stored together within the same shard, which can enhance query performance and scalability. Choosing an appropriate shard key is critical, as it determines how data is distributed across shards. The key should ensure an even distribution of data and access patterns to prevent hot spots.

To set this up in a project, follow these steps:

1. Set up a dedicated database to act as the Shard Map Manager and store information about shard locations and key ranges.
2. Create and manage shard maps that define how data is partitioned across shards by range or list mappings.
3. Utilize the Elastic Database client library to route queries to the correct shard based on the sharding key.

```
// Configure Shard Map Manager
var shardMapManager = ShardMapManagerFactory.GetSqlShardMapManager(
    shardMapManagerConnectionString,
    ShardMapManagerLoadPolicy.Lazy);

// Get the shard map
ListShardMap<int> shardMap =
    shardMapManager.GetListShardMap<int>("CustomerShardMap");

// Get the shard based on the customer ID
Shard shard = shardMap.GetShard(customerId);

// Create a connection to the shard
using (SqlConnection conn =
    shard.OpenConnection(shardConnectionString,
        ConnectionOptions.Validate))
{
    // Perform database operations
}
```

Note that this sharding technique's integration with EF Core is limited, so additional steps must be taken to create database context objects dynamically using the appropriate connection string. Typically, you would have to store the connecting strings to the shard mapping database and the different shards, usually in `appsettings.json` or using Azure Key Vault or environment variables. Then, you create a database context file representing the general database structure of any given tenant database:

```
public class TenantDbContext : DbContext
{
    public TenantDbContext(DbConnection connection) : base(new
```

```
DbContextOptionsBuilder<TenantDbContext>()
    .UseSqlServer(connection).Options)
{
}

// Define your DbSet here
public DbSet<Patient> Patients { get; set; }
}
```

To open a connection to a shard, we can implement a special utility service that will dynamically load the correct database, based on the requesting tenant:

```
public class TenantService
{
    public void PerformTenantOperation(int tenantId)
    {
        var shard = ShardingService.GetShard(tenantId);
        using (var connection = shard.OpenConnection(
            Configuration.GetConnectionString("ShardMapManager"),
            ConnectionOptions.Validate))
        {
            using (var context = new TenantDbContext(connection))
            {
                // Perform data operations
                var patients = context.Patients.ToList();
            }
        }
    }
}
```

This general guideline can help you to integrate EF Core with your database sharding in Azure SQL.

Azure also supports sharding in NoSQL databases through Azure Cosmos DB. Azure Cosmos DB is a globally distributed, multi-model NoSQL database service that inherently supports sharding through partitioning. When creating a container in Cosmos DB, you define a **partition key** that determines how data is distributed across partitions, effectively implementing sharding.

.NET supports an Azure Cosmos DB client library. `Microsoft.Azure.Cosmos` supports easy querying based on the partition key. In this example, `PatientId` is the partition key, distributing patient records across partitions to optimize performance and scalability:

```
// Create a new container with a partition key
await cosmosClient.GetDatabase("PatientsDB")
    .CreateContainerIfNotExistsAsync(
        new ContainerProperties
        {
            Id = "PatientRecords",
            PartitionKeyPath = "/PatientId"
        },
        throughput: 400);
```

These Azure services can help us implement sharding strategies tailored to relational and NoSQL databases, ensuring their applications remain responsive and scalable under varying workloads.

Another central talking point about managing consistent data across several databases is eventual consistency. We will look at this and how to manage it next.

## Eventual consistency

**Eventual consistency** is a common strategy for balancing data changes in a distributed system. It ensures that, given enough time without new updates, all replicas of a data item will converge to the same value. This model accepts temporary inconsistencies, guaranteeing that consistency will eventually be achieved.

Eventual consistency is suitable for scenarios where the application can tolerate temporary inconsistencies, such as changing a doctor assigned to an appointment. Make sure to properly scope which scenario fits this loose expectation and when data updates must be handled in real time.

Managing eventual consistency requires careful design and the implementation of specific patterns and practices. While **CQRS** and **event sourcing** are prominent strategies to handle consistency, other techniques help ensure data integrity across distributed components.

A popular approach is the **outbox pattern**. This addresses the challenge of ensuring atomicity between database operations and message publishing. When a service updates its database and needs to publish an event to notify other services, there's a risk of inconsistency if the event fails to publish after the transaction is committed. With this pattern, we implement a **transactional**



**outbox table** in the service's database to temporarily store events as part of the same transaction that modifies the business data. A separate process reads pending events from the outbox table and publishes them to the message broker.

This pattern combines several other patterns and is an excellent way to manage data consistency in **asynchronous service communications**. In this scenario, one service publishes data to a message broker, and other services need that data to complete their operations. The intermediary data store helps to track which messages were successfully delivered and can assist with retries where necessary.

Azure offers robust support for handling eventual consistency if you use cloud-based databases. This flexibility is particularly evident in Azure Cosmos DB, Microsoft's globally distributed, multi-model database service. Azure Cosmos DB provides five well-defined consistency levels, enabling developers to choose the most appropriate trade-off between consistency, availability, and performance for their applications:

- **Strong consistency:** This guarantees that read operations return the most recent committed write.
- **Bounded staleness:** This ensures that read operations lag behind writes by no more than a specified time interval or number of versions.
- **Session consistency:** This provides consistency within a client session, ensuring read-your-own-writes behavior.
- **Consistent prefix:** This ensures that read operations never see out-of-order writes; they may be incomplete, but are always in order.
- **Eventual consistency:** This option offers the weakest consistency guarantee. Reads may return out-of-order or stale data, but replicas will eventually converge.

While eventual consistency can improve performance and availability, it's essential to consider that a read operation may read outdated data. Ensure that your application is designed to handle such scenarios. In multi-master configurations, concurrent writes can lead to conflicts. Thankfully, Azure Cosmos DB provides mechanisms for conflict resolution, including custom policies and procedures.

Now that we have explored several points governing good database design and development, let's summarize what we have learned.

## Summary

This chapter explored various approaches to managing data in a microservices architecture. Unlike traditional monolithic applications that rely on a centralized database, microservices advocate for a decentralized model where each service manages its data. This architectural approach introduces challenges such as data integrity, inter-service communication, and transactions, which are addressed through different design strategies.

Different database management approaches exist, including **database-per-service**, **shared databases**, and **event-driven data propagation**. The database-per-service pattern allows each microservice to have its database, ensuring isolation and scalability. This model enables **polyglot persistence**, where each microservice can use the most appropriate database technology (SQL, NoSQL, Graph DBs, etc.). However, this approach increases complexity, costs, and maintenance efforts. Alternatives include **shared database** models, where multiple services share a single database but use separate schemas or tables to maintain logical separation.

We then reviewed the advantages and trade-offs of relational (SQL) and non-relational (NoSQL) databases. Relational databases (such as SQL Server, MySQL, and PostgreSQL) adhere to **ACID** principles (**Atomicity, Consistency, Isolation, and Durability**), making them ideal for strong data integrity applications. NoSQL databases (MongoDB, Azure Cosmos DB, Redis) offer flexibility and scalability, favoring speed over strict consistency. The choice between these models depends on the specific requirements of the microservice, such as the need for complex transactions or rapid scalability.

The selected database technology directly affects the choice of **object-relational mapping (ORM)**. Tools such as **Entity Framework Core (EF Core)** and Dapper facilitate database interactions in .NET applications. We focused on EF Core since it has more native support for both relational and non-relational databases and has a rich feature set to support two main database development approaches: code-first (where developers define database schemas in code and use migrations for updates) and schema-first (where an existing database is scaffolded into an application). Code-first offers better control over migrations, while schema-first is useful when working with legacy databases.

After discussing general topics, we pivoted to some best practices and recommendations, where more advanced database techniques are required:

- Polyglot persistence allows microservices to choose the best database technology. This ensures that the developed service maximizes its potential to meet business needs and perform as efficiently as possible.
- Database sharding was introduced as a technique for distributing data across multiple databases or servers. This strategy improves performance and fault tolerance, ensuring that high-traffic microservices remain operational. Cloud services such as Azure Cosmos DB and Azure Elastic Database tools offer built-in support for sharding and consistency models, helping to optimize microservice performance.
- The Eventual Consistency model allows data across services to sync over time rather than enforcing immediate consistency. Strategies such as **Command Query Responsibility Segregation (CQRS)** and event sourcing help manage data synchronization. The Outbox pattern was also introduced to ensure reliable event publishing with less risk of data loss.

Designing a database strategy for microservices requires careful planning, balancing scalability, maintainability, and data consistency. This chapter gave you a comprehensive guide to selecting the right database models, persistence strategies, and scalability techniques to ensure efficient microservices architecture.

A major talking point in this chapter was transactions and transaction management. This is especially difficult in a distributed system, and we will tackle this challenge next by reviewing the **Saga pattern**.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW



# 9

## Implementing Transactions across Microservices Using the Saga Pattern

In the previous chapter, we explored database design strategies and their significance in ensuring data consistency, scalability, and maintainability within microservices architectures. Up until now, we have examined various techniques, including shared databases, database per service, CQRS, and event sourcing, which are crucial for managing data in distributed systems. While these patterns help establish a solid foundation for handling microservices data, they also introduce a critical challenge: ensuring data consistency across multiple services when executing a business transaction.

Traditional monolithic applications rely on **Atomicity, Consistency, Isolation, and Durability (ACID)** transactions, where a single database ensures that a transaction is either fully completed or fully rolled back. However, implementing such atomic transactions becomes complex in a microservices-based system, where each service has its own database. A single business operation might involve multiple microservices, and if any step fails, the system must either compensate for the failure or roll back changes to maintain consistency.

This is where the **Saga pattern** comes into play. This pattern manages distributed transactions without relying on a single database transaction. Instead of ACID guarantees, it ensures consistency through coordinated, compensating transactions across multiple services.

In this chapter, we will explore the following topics:

- What the Saga pattern is and why it is essential for microservices
- The two primary implementation approaches: choreography and orchestration
- How to implement the Saga pattern tools such as MassTransit and Entity Framework
- Best practices and recommendations for implementations

By the end of this chapter, you will understand how to implement resilient, distributed transactions across microservices in .NET, ensuring that your applications remain consistent, scalable, and fault-tolerant.

## Technical requirements

Code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET--Second-Edition/tree/main/Chapter09>.

## Exploring the Saga pattern

As discussed previously, your microservices application is best implemented using the Database-Per-Service pattern. The major drawback to this pattern choice is that we cannot guarantee that our databases will always remain in sync if one of the operations fails.

We need a mechanism that spans multiple services and can implement transactions across the various data stores, such as a two-phase commit, which requires all data stores to commit or roll back transactions simultaneously. This would be perfect, except that some NoSQL databases and message brokers are incompatible with this model.

Imagine a new patient being registered with our healthcare center. This process will require the patient to provide their information and some essential documents and book an initial appointment, which requires payment. These actions require four different microservices to get involved; thus, four different data stores will be affected.

We can refer to an operation that spans multiple services as a **saga**. Essentially, a saga is a sequence of local transactions. Each transaction updates the data target database and generates a message or event that triggers the next operation in the saga. If one of the local transactions fails along the chain, the saga will execute rollbacks across the databases affected by the preceding transactions.

A saga generally implements three types of transactions:

- **Reversible:** These are transactions that another transaction with the opposite effect can reverse.
- **Retriable:** These transactions are guaranteed to succeed and are implemented after pivot transactions.
- **Pivot:** As the name suggests, the success or failure of these transactions is pivotal to the continuation of the saga. If the transaction commits, then the saga runs until it is completed. These transactions can be placed as a final compensable transaction or the first retrievable transaction of the saga. They may also be implemented as neither.

Figure 9.1 shows the Saga pattern, where each step of the saga consists of a transaction (unique to the service) that must be completed before the next step in the saga can begin.

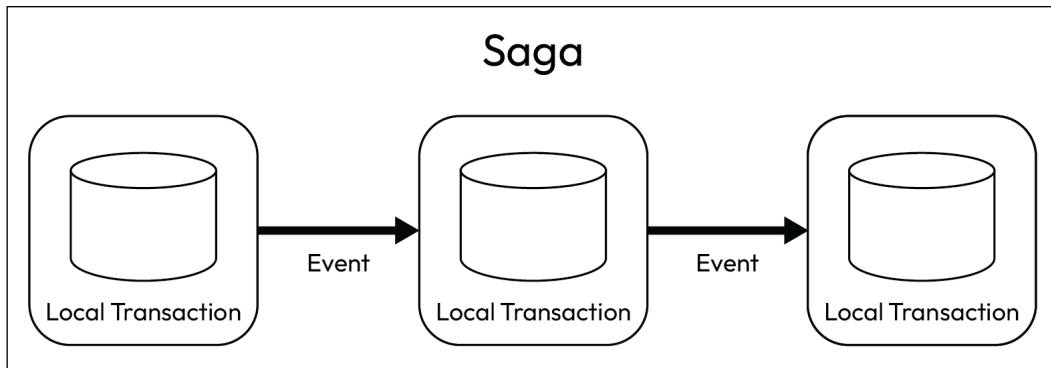


Figure 9.1 – Each local transaction sends a message to the next service in the saga until the saga is completed

Every pattern has advantages and disadvantages, and it is essential to consider all angles to plan an approach adequately. Let us review some known issues and considerations that must be taken into account when implementing this pattern.

## Issues and considerations

Given that we, up until this chapter, would have written off the possibility of implementing ACID transactions across our data stores in a microservices architecture, this pattern is not easy to implement. It requires absolute coordination and understanding of our application's moving parts.

This pattern is also difficult to debug. Given that we are implementing a single function across autonomous services, we have introduced a new point of failure and a new touchpoint, and special effort must be made to track and trace where the failure may have occurred. This complexity increases with each step of the saga's participating services.

We need to ensure that our saga can handle transient failures in the architecture. These are errors that happen during an operation and might not be permanent. Recall from previous chapters the need to ensure idempotency. Every operation protected by retry logic should ensure that the changes it invokes occur only once. This is a key part of implementing transactions and retries. Regardless, we must include retry logic to ensure that a single failure in an attempt does not prematurely end the saga. In doing so, we must also ensure that our data is consistent with each retry.

This pattern is certainly not without its challenges. Although the Saga pattern offers a pragmatic way to coordinate business workflows in a distributed system, its very strengths introduce a set of structural and operational drawbacks that every team needs to factor into their design trade-offs. Some of these drawbacks are as follows:

- The pattern abandons the comfort of ACID guarantees in favor of eventual consistency. That means all downstream services must be designed to tolerate temporary divergence and to present coherent states to users who might read data before the saga has fully converged.
- It requires extra logic, such as versioning, read-model isolation, or compensating flags, to prevent users from acting on stale information, and it demands clear communication of “in-progress” states at the API level.
- It increases in overall complexity. Each local transaction must be idempotent and capable of undoing itself, or of executing a separate compensation without corrupting historical data. The more steps the saga contains, the more permutations of success, retry, and failure paths must be covered in tests.
- Sagas introduce latency because each step waits for network calls, queueing, and persistence before the next can begin. If long-running transactions hold business resources, the system must keep those locks lightweight or risk artificial resource starvation. High-throughput scenarios exacerbate back-pressure problems when many concurrent sagas retry simultaneously, potentially leading to “retry storms” that destabilize the message broker or downstream APIs.

Teams that understand and prepare for the limitations and challenges of sagas can still reap the pattern's advantages, resilience, loose coupling, and independent service evolution while avoiding unpleasant surprises in production. It will undoubtedly help ensure that data is more consistent across loosely coupled services. Sagas are usually coordinated using either **orchestration** or **choreography**. Both methods have pros and cons. Let's begin by exploring choreography.

## Understanding and implementing choreography

**Choreography** is a method of coordinating sagas where participating services use messages or events to notify each other of completion or failure. In this model, the event broker sits between the services but does not control the flow of messages or the flow of the saga. This means there is no central point of reference or control, and each service monitors a message that serves as a confirmation trigger for it to initiate its operation.

In a choreography saga, each participant is autonomous, reacting only to the domain events it subscribes to and, when its local transaction succeeds or fails, emits a new event that will be observed by whichever service needs the result. For that relay to remain reliable, every event must serve two distinct purposes:

- It must supply *just enough* business data for the next service to do its work
- It must carry a correlation identifier that lets every message in the chain be stitched back to the single business request that spawned the saga

Getting these two concerns right is the difference between an elegant, observable workflow and a brittle, tightly coupled mess of messages. If an event contains more data than the immediate recipient needs, some problems surface:

- The payload becomes larger than necessary, increasing network bandwidth and deserialization costs
- It leaks implementation details that entice other teams to depend on fields that were never intended for them
- When another service later decides to alter its internal model (for example, renaming a field or adding a new optional value), dozens of downstream consumers can break unexpectedly

By restricting the payload to the subset of facts the next hop truly uses, you uphold the boundary between microservices and slow the ripple effect of change.

Conversely, suppose the event supplies less than required. In that case, the recipient will need to call another service or query another data store to fetch the missing context, turning an otherwise asynchronous flow into a chatty, synchronous one.



A choreography saga also needs to know which messages belong together. A single user action may spawn dozens of events, retries, and compensations over minutes or hours. By assigning a correlation ID (often called `SagaId`, `CorrelationId`, or `TraceId`) now the saga begins and including that token in every subsequent event, you create a breadcrumb trail that tracing systems, log aggregators, and human debuggers can follow end to end.

Correlation IDs serve three practical purposes:

- **Tracing and observability:** Distributed-tracing middleware (e.g., **OpenTelemetry**) can visualize the entire saga as a single span containing child spans for each service, making it obvious where latency or failures originate.
- **Idempotency and deduplication:** A consumer can keep a lightweight record of messages it has processed for a given ID. If the same event is delivered twice (because the producer retried), the consumer discards the duplicate without corrupting its local state.
- **Business auditability:** Product owners and compliance auditors often need to answer what exactly happened to payment #123. Storing the correlation ID with each database change and message log enables the reconstruction of the entire narrative.

Figure 9.2 shows the choreography flow:

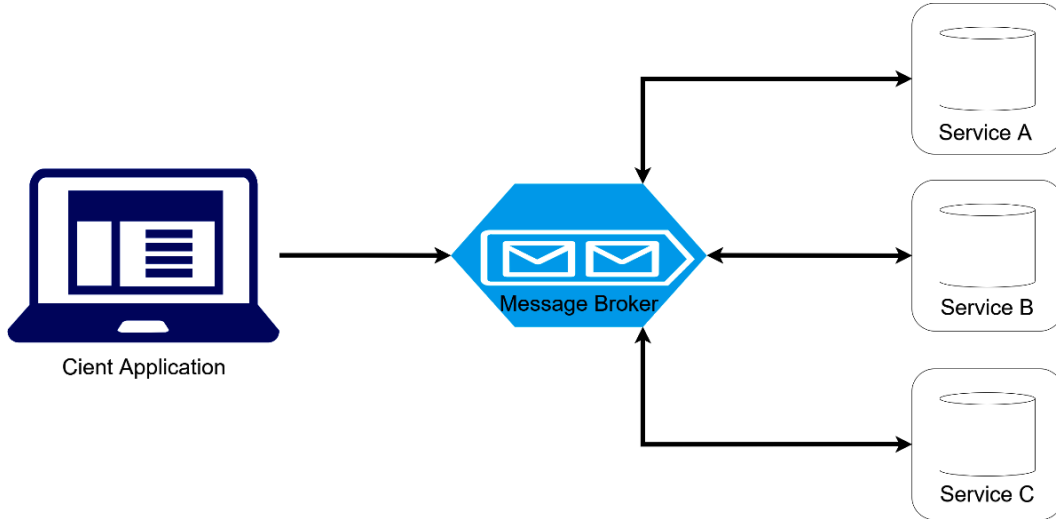


Figure 9.2 – An application request sends a message to the queue to inform the first service in the saga to begin, and messages flow between all participating services

The main takeaway from the choreography model is that there is no central point of control. Each service will monitor events and determine whether to act. The message's contents will inform the service what it should do, and if it acts, it will reply with a message stating the success or failure of its action. If the last service of the saga is successful, then no message is produced and the saga will end.

A well-behaved saga transforms a multi-step workflow (such as registering a patient, charging a fee, booking an appointment, and saving documents) into a sequence of autonomous but co-ordinated local transactions. If we were to visualize this process using our user registration and appointment booking example mentioned earlier in this chapter, we would have a flow like this:

1. The user submits a registration and appointment booking request (client request).
2. The registration service stores the new user's data and then publishes an event with relevant appointment and payment details. This event could be referred to, for example, as `USER_CREATED`.
3. The payment service listens for `USER_CREATED` events and attempts to process a payment as necessary. When successful, it produces a `PAYMENT_SUCCESS` event.
4. The appointment booking service processes `PAYMENT_SUCCESS` events and proceeds to add the appointment information as expected. This service makes the booking arrangements and produces a `BOOKING_SUCCESS` event for the next service.
5. The document upload service receives the `BOOKING_SUCCESS` event and proceeds to upload the documents and add a record to the document service data store.

This example shows that we can track the processes along the chain. If we wanted to know the details of each leg and the outcome, we could have the registration service listen to all events and create state updates or logs of the progress along the saga. It will also be able to communicate the success or failure of the saga back to the client.

What happens, though, when a service fails? How do we mitigate or reap the benefit of the saga's ability to reverse changes that have already occurred? Let's review that next.

## Rollback on failure

Sagas are necessary because they allow us to roll back the changes that have already happened when something fails. The service will publish an event stating that a local transaction failed. We need additional code in the preceding service to react with the rollback procedures accordingly. For example, if our payment service operation failed, the flow would look like this:

1. The appointment booking service failed to confirm the booking and published a `BOOKING_FAILED` event.

2. The payment service receives the `BOOKING_FAILED` event and proceeds to issue a refund to the client. This would be a remediation step.
3. The preceding registration service will see the `BOOKING_FAILED` event and notify the client that the booking was unsuccessful.

In this situation, we are not entirely reversing every step since we retain the user's registration information for future reference. What is essential, though, is that the next service in the saga, which uploads the documents, is not configured to listen for the `BOOKING_FAILED` event. Therefore, it will have no effect unless it receives a `BOOKING_SUCCESS` event.

A choreography saga succeeds only when every participant publishes its success or failure event in a timely fashion. The most insidious fault, therefore, is not an explicit `BOOKING_FAILED` event but the **absence** of the `BOOKING_SUCCESS` or `BOOKING_FAILED` message altogether. This can occur if the appointment booking service commits its local database transaction and then crashes before emitting the event, or if its message broker connection stalls while the process remains active. Because no single component manages the end-to-end process, other services wait, uncertain whether the booking is still in progress, has failed silently, or has succeeded but failed to communicate. Mitigating this requires explicit timeout logic external to the business services themselves.

A common technique is to introduce a lightweight *watchdog* or *process monitor* service that subscribes to all saga events and keeps a deadline ledger keyed by the correlation ID. Whenever it observes a `USER_CREATED` event, for example, it records that a follow-up `PAYMENT_SUCCESS` or `BOOKING_FAILED` message must be seen within, say, five minutes. If the deadline passes, the watchdog emits a synthetic timeout event, such as `BOOKING_TIMEOUT`, that triggers compensation just as a real failure would.

Another safeguard is the *transactional outbox pattern*. The booking service writes its intent to publish `BOOKING_SUCCESS` to the same database transaction that saves the booking record, and a background outbox dispatcher guarantees that the event is emitted even if the service crashes. In practice, teams can combine both the outbox pattern and the watchdog for at-least-once delivery and liveness, to close the gap that pure choreography leaves open.

We can also note that our remediation steps are relative to the operation. Our payment service is likely a wrapper around a third-party payment engine that will also write a local database record of the payment operation. Given the saga's lack of completion, it will not remove the payment record but mark it as a refunded payment or cancel it in its remediation steps.

While this is not a like for like reflection of a local transaction would undo a database write, a rollback might look different for each service based on the business rules or nature of the operation. We also note that our rollback did not encompass every service, as our business rules recommend retaining user registration information for future reference.

The choreography pattern assumes that every service can undo its work; yet, nothing guarantees that the compensating step will succeed on the first attempt. For instance, a payment refund may fail due to the payment provider being offline, the user's card network being down, or a concurrency conflict occurring within the payment database. If the compensation cannot be completed, the saga is now in its most precarious state, where forward work has been rolled back in some services (appointments are canceled) but not in others (money is still captured).

Robust systems address this in layers. At the infrastructure level, the refund command is placed on a **retry policy with exponential backoff** so that transient outages clear automatically. After a specified number of retries, the message is moved to a dead-letter queue, triggering an alert. Finally, the watchdog described earlier maintains a compensation outstanding flag for each saga. Suppose the flag remains set beyond a service-level agreement (for instance, thirty minutes for financial settlements). In this case, the monitor escalates to a dedicated human-resolution queue, where support staff can inspect the correlation ID, verify partial rollbacks, and execute a manual refund if necessary.

By acknowledging that compensations themselves are fallible and by providing automated retries, dead-letter handling, and human fallback, a choreography saga can recover gracefully even when its safety net tears.

In a reliable message bus, duplicate delivery is not a bug but a design premise, so every handler, including those that reverse work, must behave correctly if invoked more than once. When the payment service receives a `BOOKING_FAILED` event, it issues a refund. However, if network jitter forces the booking service to republish the same failure, the payment logic will fire again unless it is guarded. The solution is to store a deduplication key, composed of the correlation ID and the event type (or a monotonic event sequence), in an idempotency table within the payment database. Before processing, the handler checks whether that key already exists; if so, it acknowledges the message and returns without initiating a second refund. Because refunds usually interact with an external gateway, an additional level of idempotency, often an **idempotency key** in the gateway's API, is prudent to stop the call from crediting the customer multiple times if the payment service itself retries after a crash. Treating compensations as first-class, idempotent operations prevents a recovery path from becoming a new source of inconsistency.

Now that we have reviewed the theory of the moving parts, let's examine this pattern in practice.

## Building a choreography saga pattern with MassTransit and RabbitMQ

This saga will focus on the communication between the appointment booking and payment services. Since we have already created the appointment booking service, you may proceed with that work. The relevant setup for the payment service will be shared as needed.

Using a choreography saga pattern, we will build an interaction between the appointment booking and payment processing microservices. The system will do the following:

1. Book an appointment.
2. Trigger a payment request.
3. If the payment succeeds, the saga is complete.
4. If the payment fails, the appointment booking is rolled back.

We must include a shared class library to house standard event models that are used across each service. The following command will create and add this class library to the solution file:

```
dotnet new classlib -n Shared
dotnet sln add Shared/Shared.csproj
```

Then, we need to create the payment service, add the shared project reference to it, and the appointment booking service:

```
dotnet new webapi -n PaymentsApi --use-controllers
dotnet sln add PaymentsApi/PaymentsApi.csproj
dotnet add PaymentsApi/PaymentsApi.csproj reference Shared/Shared.csproj
dotnet add AppointmentsApi/AppointmentsApi.csproj reference Shared/Shared.csproj
```

Each microservice needs MassTransit, RabbitMQ, and EF Core. Run these commands inside each microservice folder:

```
dotnet add package MassTransit.AspNetCore
dotnet add package MassTransit.RabbitMQ
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Both (all) services will reference this shared project, and the event types must be present here so each service can identify them. We need an `AppointmentCreated` and `PaymentProcessed` class to represent the events our respective services will send to the message queue. These models can be defined in a class file called `Events.cs`:

```
public record AppointmentCreated(Guid CorrelationId,, Guid
    AppointmentId, Guid PatientId, DateTime AppointmentDate);
public record PaymentProcessed(Guid CorrelationId,, Guid
    AppointmentId, Guid PatientId, decimal Amount, bool Success);
```

These models are defined as and used for the following:

- `AppointmentCreated`: Published when an appointment is created
- `PaymentProcessed`: Published by the payment service, indicating whether the payment was successful or failed

We need to add a new column to the `Appointment` entity class to indicate whether an appointment has been canceled:

```
public class Appointment
{
    // Existing properties
    public bool IsCanceled { get; set; } = false;
    // Code
}
```

After adding this column, run a new migration and database update to ensure the column is present:

```
dotnet ef migrations add AddIsCanceledToAppointments --project
AppointmentsApi/AppointmentsApi.csproj --context AppointmentContext
dotnet ef database update --project AppointmentsApi/AppointmentsApi.csproj
--context AppointmentContext
```

Since we are building on the appointment booking microservice from previous chapters, we know that once an appointment is created, its details will be placed on the RabbitMQ instance that we have already set up. We will add another consumer to watch for messages that the payment processor service might produce in the event of a failure. Create a new class in the `Consumers` folder called `PaymentProcessedConsumer.cs`. It will have the following contents:

```
public class PaymentProcessedConsumer(AppointmentContext _dbContext) :
    IConsumer<PaymentProcessed>
```

```

{
    public async Task Consume(ConsumeContext<PaymentProcessed> context)
    {
        var message = context.Message;
        Console.WriteLine(
            $"[Appointment] CorrelationId={message.CorrelationId}: Payment " +
            $"{(message.Success ? "succeeded" : "failed")} for Appointment " +
            $"{message.AppointmentId}");

        if (!message.Success)
        {
            var appointment = await
                _dbContext.Appointments.FindAsync(message.AppointmentId);
            if (appointment != null)
            {
                appointment.IsCanceled = true;
                await _dbContext.SaveChangesAsync();
            }
        }
    }
}

```

In the payment processor service, we need to create a consumer that will consume the AppointmentCreated event that is published to the queue. Create a folder called Consumers and, in it, a file called AppointmentCreatedConsumer, which will be defined like this:

```

public class AppointmentCreatedConsumer : IConsumer<AppointmentCreated>
{
    public async Task Consume(ConsumeContext<AppointmentCreated> context)
    {
        var message = context.Message;

        Console.WriteLine($"CorrelationId={message.CorrelationId}: Payment
            | Processing payment for Appointment ID:{message.AppointmentId}");

        // Simulate payment failure 50% of the time
        var paymentSuccess = new Random().Next(0, 2) == 1;
    }
}

```

```
        await context.Publish(new PaymentProcessed(message.CorrelationId,
            message.AppointmentId, message.PatientId, 100.00m,
            paymentSuccess));
    }
}
```

In a real-life scenario, this process would connect to the payment gateway and process the payment according to the system's needs. In this example, we simulate a potential failure and reinsert the confirmation message into the queue.

Similar to the configurations we created for the appointments service to communicate with RabbitMQ, we need to add the following to the `Program.cs` file of the payment service:

```
builder.Services.AddMassTransit(mt =>
{
    mt.AddConsumer<AppointmentCreatedConsumer>();
    mt.UsingRabbitMq((context, cfg) =>
    {
        cfg.Host("localhost", "/", h =>
        {
            h.Username("guest");
            h.Password("guest");
        });

        cfg.ReceiveEndpoint("payment-service-appointment-created", e =>
        {
            e.ConfigureConsumer<AppointmentCreatedConsumer>(context);
        });
    });
});
```

Now, after attempting to process a payment, a `PaymentProcessed` object is placed on the queue and will be consumed by `PaymentProcessedConsumer` in the appointment booking service. When the `Success` property of the `PaymentProcessed` object is `false`, the appointment booking service will flag the appointment as canceled. This is a simplified example of how failure might be handled, and you may need to perform more complex data cleanup, such as removing reference records retroactively.



Regardless of the complexity of the remediation efforts, the fundamental concept remains that choreography relies on asynchronous communication between all services involved in the saga. There may be pains with implementing this version of the saga, so let us review the pros and cons of this choreography implementation.

## **Pros and cons**

In a choreography-based saga, apparent strength lies in its simplicity and autonomy of service. Each microservice is responsible only for its local transaction and for emitting events that downstream services consume, which keeps coupling low and allows individual teams to evolve their services independently. Because each step can publish its success or failure asynchronously, you can even fan out a single initiating event, such as `BOOKING_CONFIRMED`, to multiple consumers in parallel. This parallelism can dramatically reduce end-to-end latency when there is no ordering dependency between those operations.

However, that very flexibility becomes a liability once implicit service dependencies creep in. If two or more consumers rely on the same event, this can lead to race conditions, inconsistent states, or subtle failures. In such cases, you either need to shard your events into finer-grained types or impose extra logic on the consumers to delay their work until prerequisite events have arrived. Both approaches reintroduce complexity into what was intended to be a simple, autonomous flow.

Observability is another area where pure choreography can strain an organization's tooling. Because the saga unfolds as a series of messages across multiple queues and process boundaries, tracing a single business transaction requires a robust distributed-tracing system (such as OpenTelemetry), centralized logs annotated with correlation IDs, and ideally, some form of event-stream visualization or workflow dashboard. Without those, even a five-step saga can turn into a tangle of invisible handoffs, making root-cause analysis and recovery exercises painfully slow. The more participants and compensating paths you add, the more critical it becomes to invest in these cross-cutting monitoring and tracing capabilities.

Taken together, choreography shines in straightforward, loosely coupled flows where latency and autonomy are paramount; however, it can become fragile and opaque as the number of services, implicit dependencies, and failure modes increases. At that point, introducing a central orchestrator can help by making the entire workflow explicit and easier to observe and control.

For this reason, we will review another saga pattern, orchestration, which implements a central point of control.

## Understanding and implementing orchestration

When we think of orchestration, we think of coordination. An orchestra is a coordinated group of musicians working together to produce a unified sound. Each musician plays their part, but they are led by a conductor who guides them along the same path.

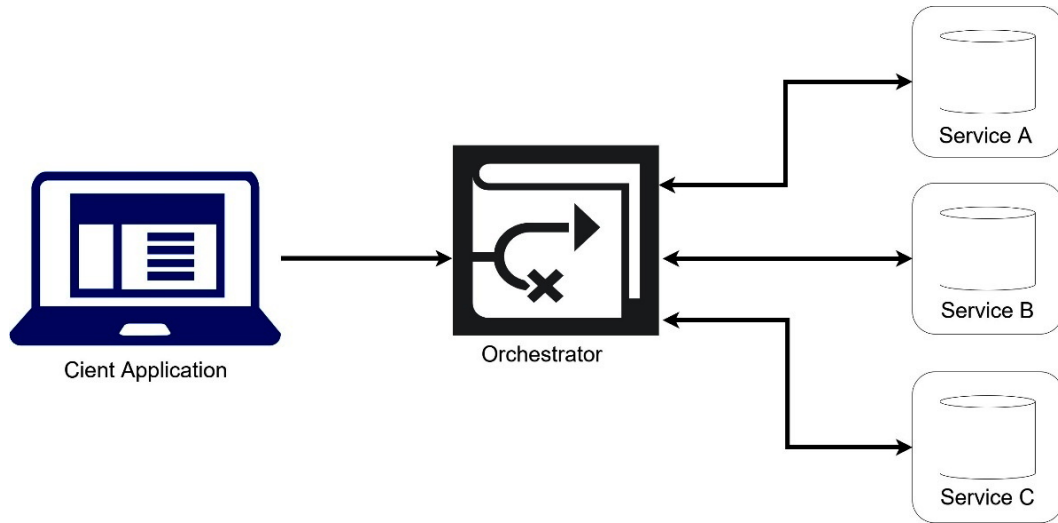
The orchestration method of implementing a saga is not significantly different in terms of requiring a central point of control, much like a conductor. All services are monitored by a central point of control to ensure they perform their role effectively and report any failures accordingly. The central control is referred to as an *orchestrator*, a microservice that sits between the client and all other microservices. It handles all the transactions, telling participating services when to complete an operation based on feedback received during the saga. The orchestrator executes the request, tracks and interprets the request's state after each task, and handles the remediation operations as necessary.

While you can hand-craft a saga orchestrator as a simple microservice that listens for events and invokes downstream services, many teams opt instead for purpose-built workflow engines that provide durability, state management, retry policies, and monitoring out of the box. This is where platforms that are designed to model complex, long-running processes as code shine. Some examples are the following:

- **Temporal:** An open source, code-centric workflow engine designed for writing reliable, long-running applications. Temporal exposes rich observability through built-in metrics and a web UI that lets you inspect in-flight workflows, view their event histories, and trigger ad hoc retries or compensations. Because workflows are ordinary code, teams can leverage their existing unit-testing frameworks to write automated tests for complex branching logic and failure scenarios, making Temporal a popular choice for microservices orchestration in polyglot environments.
- **Cadence:** The upstream, early version of the Temporal server, initially developed at Uber. It includes features such as hierarchical namespaces for multi-tenant isolation, flexible task routing, and pluggable persistence backends (e.g., Cassandra or MySQL). Cadence also supports workflow versioning, cron scheduling of workflows, and external signal handling, enabling human-in-the-loop approvals or manual interventions in an otherwise automated saga. Although Temporal has largely overtaken Cadence in community momentum, Cadence remains in production for many large-scale users and can be an appealing option when teams value its tried-and-tested stability and its lighter operational footprint.

- **AWS Step Functions:** A fully managed orchestration service provided by **Amazon Web Services (AWS)**. It allows you to define serverless workflows as state machines using either the Amazon States Language (a JSON or YAML-based DSL) or through higher-level SDKs such as the AWS CDK. A Step Functions workflow consists of states, tasks, choices, parallel branches, wait states, and retries. With the **Express Workflows** tier, Step Functions also supports high-throughput, short-lived workflows at a fraction of the cost, making it easy to glue together microservices in a pay-as-you-go model without standing up any orchestration infrastructure yourself.
- **Azure Durable Functions:** An extension of Azure Functions that provides first-class support for writing stateful serverless workflows in C# or JavaScript. You author orchestrator functions that call out to activity functions; Durable Functions transparently checkpoints their progress to Azure Storage so that you can write synchronous-looking code that executes asynchronously under the covers. Out-of-the-box patterns include function chaining, fan-out/fan-in parallelism, human interaction via external events, and time-based durable timers. Because orchestrations run in a managed Azure environment, you benefit from automatic scaling, built-in monitoring, and seamless integration with other Azure services. Durable Functions also support function orchestration retries and exception handling, allowing you to implement compensating actions or compensation scopes in case of downstream failures.
- **Camunda:** An open source workflow and decision automation platform built around the BPMN 2.0 and DMN standards. Unlike code-first engines, Camunda emphasizes a model-driven approach: business analysts can draw process diagrams in a graphical editor and deploy them to a lightweight process engine. For microservices teams, Camunda integrates via a worker-pattern SDK or an external task mechanism, allowing services to claim work items and complete them asynchronously. In addition to orchestration, Camunda's decision engine (DMN) enables you to externalize business rules and execute them declaratively. This combination of visual modeling, standards compliance, and robust tooling makes Camunda well suited for organizations seeking both developer agility and business-user visibility into complex orchestrations.

*Figure 9.3* shows the orchestrator flow:



*Figure 9.3 – An application request sends a message to the orchestrator, which begins to coordinate and monitor the subsequent calls to the participating services*

Let us revisit our appointment booking operation from the perspective of the orchestration saga implementation:

1. The user submits a registration and appointment booking request (client request).
2. The orchestrator centrally stores the data needed for the services participating in the saga.
3. The orchestrator calls the registration service, which stores the new user's data and responds with a success message.
4. The orchestrator then sends a request to the payment service and listens for the outcome of the payment operation.
5. The orchestrator moves to trigger the appointment booking service, which processes the appointment booking accordingly and responds to the orchestrator.
6. The orchestrator finally triggers the document upload service, uploads the documents, and adds a record to the document service data store.
7. The orchestrator then responds to the client, updating the state of the operation and responding to the client with the overall result.

Fundamentally, the orchestrator occupies a central role not merely as a passive observer of events, but as the active conductor that issues commands to each participant service and then waits for their replies in the form of events. Technically, those commands can be implemented in one of two fundamental ways: synchronous REST or asynchronous messaging.

While it may seem straightforward to have the orchestrator perform an HTTP POST to the appointment service and immediately block until it returns 200 OK, this tight coupling sacrifices many of the resilience benefits that motivated the use of a saga in the first place. A more robust approach is for the orchestrator to send a command message into a queue or topic that the appointment service subscribes to.

So, instead of services communicating via HTTP, they emit events that describe their outcomes. That unidirectional flow, from orchestrator to service, and from service to orchestrator, separates intent from result. It also enables the orchestrator to manage long-running processes. Rather than holding an open HTTP connection for minutes while a human approves an insurance claim, the orchestrator can pause in its state machine until it receives the next relevant event. Under the covers, most workflow engines or custom orchestrators treat each incoming event as a trigger to resume the saga's state machine, evaluate compensation branches, and then send the next command or conclude the process.

By explicitly distinguishing commands from events, an orchestrated saga built on messaging achieves loose coupling, resilience to partial failures, and clear observability through correlation IDs. It thereby preserves the autonomy of each microservice while still allowing a single, consistent workflow to span multiple bounded contexts.

The orchestrator is the central authority of the saga, deciding whether to proceed to the next step. Let us review what a rollback operation might look like.

## **Rolling back on failure**

Rolling back is the most essential part of implementing a saga, and like the choreography pattern, we are governed by the business rules of the operation and individual service operations.

When a downstream step in an orchestrated saga fails, the orchestrator's first task is to recognize that failure event and mark the overall workflow as having encountered an error. It then looks up which earlier step or steps must be undone to restore a consistent state. For each of those, the orchestrator emits a compensating command onto the message bus, and the relevant service consumes that command, executes its internal rollback logic, and then publishes an event when it's done.

The orchestrator, upon seeing the event, knows that the compensation has succeeded and can transition its saga instance to a final, successful state. At that point, it can emit a terminal notification or update the status store so that clients or operators understand that the original booking request was not processed and that any money taken has been refunded. Throughout this process, every command and event carries the same correlation identifier, ensuring full end-to-end traceability of both the forward and reverse paths of the business transaction.

The main takeaway is that the services will respond with failure to a central point, which will then coordinate the rollback operations across the various services. Reusing the failure scenario previously discussed, our orchestration would resemble the following:

1. The appointment booking service reports failure to the orchestrator.
2. The orchestrator receives the failure notice and calls the payment service to issue a refund.
3. The orchestrator will trigger additional clean-up operations needed for data and state that may have been stored at the beginning of the operation.
4. The orchestrator will notify the client of the operation's failure.

A rollback here is arguably easier to implement. This is not because we are changing how or what the services do, but because we can ensure the order in which the remediations will occur if that order is essential. We can accomplish that without introducing too much more complexity to the flow.

This saga orchestrator is the central controller of the saga process. It receives a command, listens for events, and determines the next course of action. `MassTransit` creates a saga state machine to manage this process. A *Saga State Machine* listens for incoming commands and tracks the status of each saga instance. Specific steps control each step of the saga.

Now, let us review a practical implementation of a saga using an orchestrator.

## Implementing orchestration

This example demonstrates the implementation of appointment booking and payment processing using the orchestrator saga pattern with `MassTransit` and `RabbitMQ`. It follows the same process as the previous choreography example, but services interact through a centralized saga orchestrator instead of communicating directly through events.

The orchestrator sends commands to each service to instruct them on what to do and then listens for events to learn the outcome and decide the next step. This clear separation between intent (commands) and result (events) is the hallmark of orchestrated saga communication.

In this example, we have the following:

1. `AppointmentRequested`: The user requests an appointment.
2. `SagaStateMachine`: This creates a saga instance and triggers the payment.
3. `PaymentRequested`: The payment microservice processes the payment.
4. `PaymentSucceeded` or `PaymentFailed`: The payment microservice publishes an event.

The Saga State Machine will react in these two ways:

- If it *succeeds*, the saga transitions to Completed (appointment is confirmed)
- If *failed*, the saga transitions to Failed (appointment is rolled back/canceled)

MassTransit's implementation of the orchestration saga requires some database storage. Each saga instance is represented by a state machine that advances in response to incoming messages. These workflows often span minutes or hours, and the orchestrator process itself may be restarted, scaled out, or failover. Therefore, you cannot rely on in-memory objects to keep track of your saga's progress. So, MassTransit integrates with durable storage, ensuring that every transition and any interim data are written to a backing store. This lets the orchestrator pick up exactly where it left off when the next event arrives, ensuring that retried or duplicate messages are handled idempotently and providing a complete audit trail of the saga's life cycle.

We can add it to the Shared project, so it is accessible to all referencing projects:

```
dotnet add package MassTransit.EntityFrameworkCore
```

We start by adding the following event models in the shared directory:

```
// Command to request appointment booking
public record AppointmentCreated(Guid AppointmentId,
    Guid PatientId, DateTime AppointmentDate);

// Command from saga orchestrator to payment service
public record PaymentRequested(Guid AppointmentId, decimal Amount);

// Events published by payment service
public record PaymentSucceeded(Guid AppointmentId);
public record PaymentFailed(Guid AppointmentId);

// Saga concluding events
public record AppointmentConfirmed(Guid AppointmentId);
public record AppointmentCanceled(Guid AppointmentId);
```

These models serve the following purposes:

1. AppointmentCreated: The appointment service sends this to the saga orchestrator to start the booking saga.
2. PaymentRequested: The saga orchestrator instructs the payment service to charge the user.

3. `PaymentSucceeded` | `PaymentFailed`: The payment service publishes its outcome.
4. `AppointmentConfirmed` | `AppointmentCanceled`: Saga announces the outcome.

For this example, we will focus on the `AppointmentCanceled` event and define a consumer in the `Consumers` folder of the `appointments` service as follows:

```
public class AppointmentCanceledConsumer : IConsumer<AppointmentCanceled>
{
    private readonly AppointmentContext _dbContext;

    public AppointmentCanceledConsumer(AppointmentContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task Consume(ConsumeContext<AppointmentCanceled> context)
    {
        var appointment = await _dbContext.Appointments
            .FindAsync(context.Message.AppointmentId);
        if (appointment != null)
        {
            appointment.IsCanceled = true;
            await _dbContext.SaveChangesAsync();
            Console.WriteLine($"[AppointmentService] Appointment
                               {appointment.AppointmentId} canceled.");
        }
    }
}
```

Please note that, while we focus on the outcome of an appointment booking, the concept remains that each service will be responsible for handling relevant failure events and rolling back as needed.

For this type of saga, we need to track the outcome of the various steps, which introduces the need for a new database. We have already explored the details of setting up a database using EF Core, so we will skip the details of these steps and proceed to add a new database. We will create a new entity model and database context in a new folder within the `Shared` project called `Data`:

```
public class BookingSagaState : SagaStateMachineInstance
{

```



```

public Guid CorrelationId { get; set; } // For correlating messages
public Guid AppointmentId { get; set; }
public string CurrentState { get; set; } = "None";
}

```

Below is the EF Core DbContext that represents the database to be used for the saga storage.

```

public class BookingSagaDbContext : DbContext
{
    public BookingSagaDbContext(DbContextOptions<BookingSagaDbContext>
        options) : base(options) { }

    public DbSet<BookingSagaState> BookingStates { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<BookingSagaState>(cfg =>
        {
            cfg.HasKey(x => x.CorrelationId);
        });

        base.OnModelCreating(modelBuilder);
    }
}

```

MassTransit's EF repository expects a CorrelationId primary key for the saga state.

Now, we create the saga state machine. We can place this in a folder called SagaOrchestrator and call the class BookingStateMachine.cs:

```

public class BookingStateMachine :
    MassTransitStateMachine<BookingSagaState>
{
    public BookingStateMachine()
    {
        // Define the states
        InstanceState(x => x.CurrentState);

        Event(() => AppointmentRequestedEvent, x =>
        {

```

```
x.CorrelateById(ctx => ctx.Message.AppointmentId);
x.SelectId(ctx => ctx.Message.AppointmentId); // Saga Key
});

Event(() => PaymentSucceededEvent, x => x.CorrelateById(ctx =>
    ctx.Message.AppointmentId));
Event(() => PaymentFailedEvent, x => x.CorrelateById(ctx =>
    ctx.Message.AppointmentId));

Initially(
    When(AppointmentRequestedEvent)
        .Then(ctx =>
            {
                ctx.Instance.AppointmentId = ctx.Data.AppointmentId;
                Console.WriteLine($"[Saga] Received
AppointmentRequested for {ctx.Data.AppointmentId}");
            })
        .TransitionTo(BookingInitiated)
        .Publish(ctx => new PaymentRequested
            (ctx.Data.AppointmentId, 99.99m)) // Request Payment
    );

During(BookingInitiated,
    When(PaymentSucceededEvent)
        .Then(ctx =>
            {
                Console.WriteLine($"[Saga] Payment succeeded for
{ctx.Instance.AppointmentId}");
            })
        .Publish(ctx => new AppointmentConfirmed
            (ctx.Instance.AppointmentId))
        .TransitionTo(BookingCompleted)
        .Finalize(),

    When(PaymentFailedEvent)
        .Then(ctx =>
            {
```

```

        Console.WriteLine($"[Saga] Payment failed for
            {ctx.Instance.AppointmentId}");
    })
    .Publish(ctx => new AppointmentCanceled
        (ctx.Instance.AppointmentId))
    .TransitionTo(BookingFailed)
    .Finalize()
);

// Set the states as final for cleanup
SetCompletedWhenFinalized();
}

// States
public State BookingInitiated { get; private set; }
public State BookingCompleted { get; private set; }
public State BookingFailed { get; private set; }

// Events
public Event<AppointmentRequested> AppointmentRequestedEvent {
    get; private set; }
public Event<PaymentSucceeded> PaymentSucceededEvent {
    get; private set; }
public Event<PaymentFailed> PaymentFailedEvent { get; private set; }
}

```

On `AppointmentRequested`, we store `AppointmentId` and publish `PaymentRequested`. When a successful payment is made, an `AppointmentConfirmed` event is published. In the event of a `PaymentFailed` event, an `AppointmentCanceled` event is published.

Now that we have defined the events, state machine, states, and database context for the saga, let us register these assets in the `Program.cs` file. First, we register the new database context:

```

builder.Services.AddDbContext<BookingSagaDbContext>(options
    =>options.UseNpgsql(builder.Configuration.GetConnectionString
        ("SagaDatabase")));

```

For simplicity, we will use a PostgreSQL database and this is the connection string, which we will add to the `appsettings.json` file:

```
"ConnectionStrings": {  
    ...  
    "SagaDatabase":  
        "Host=localhost;Database=sagadb;Username=postgres;Password=password"  
}
```

Next, we need to register the state machine and database context in the MassTransit configuration. We will modify the config to look like this. While we are at it, we will also register `AppointmentCanceledConsumer`:

```
builder.Services.AddMassTransit(x =>  
{  
    // Existing consumers  
    x.AddConsumer<AppointmentCanceledConsumer>();  
    x.AddSagaStateMachine<BookingStateMachine, BookingSagaState>()  
        .EntityFrameworkRepository(r =>  
        {  
            r.ConcurrencyMode = ConcurrencyMode.Pessimistic;  
            r.AddDbContext<DbContext, BookingSagaDbContext>();  
            r.UsePostgres();  
        });  
    x.UsingRabbitMq((context, cfg) =>  
    {  
        // Configure the receive endpoint  
        // Configure saga endpoints  
        cfg.ConfigureEndpoints(context);  
    });  
});
```

`AddSagaStateMachine<BookingStateMachine, BookingSagaState>()` instructs MassTransit to host the saga machine with an EF-based repository. The `cfg.ConfigureEndpoints(context)` call automatically wires up a queue for the saga's endpoints.

For simulation, we can also implement MassTransit and PaymentRequestedConsumer in the payment service:

```
public class PaymentRequestedConsumer : IConsumer<PaymentRequested>
{
    public async Task Consume(ConsumeContext<PaymentRequested> context)
    {
        var success = new Random().Next(0, 2) == 1;
        Console.WriteLine($"[PaymentService] Payment requested for
            Appointment {context.Message.AppointmentId}. Success?
            {success}");

        if (success)
        {
            await context.Publish(new
                PaymentSucceeded(context.Message.AppointmentId));
        }
        else
        {
            await context.Publish(new
                PaymentFailed(context.Message.AppointmentId));
        }
    }
}
```

Similarly, we register MassTransit and this consumer in the Program.cs file of the payment service like this:

```
// Add services to the container.
builder.Services.AddMassTransit(cfg =>
{
    cfg.AddConsumer<PaymentRequestedConsumer>();

    cfg.UsingRabbitMq((context, config) =>
    {
        config.Host("localhost", "/", h =>
        {
            h.Username("guest");
            h.Password("guest");
        }
    }
    );
});
```

```
    });  
  
    config.ConfigureEndpoints(context);  
  });  
});
```

Using sagas with MassTransit offers several advantages, enabling us to track and retry each step independently, thereby compensating for failed operations. The state machine will allow us to track the progress of each operation, and since the services communicate through messages, they can evolve independently while maintaining process integrity. Now that we have seen an implementation, let us discuss the pros and cons of using this pattern.

## Pros and cons

One of the most compelling strengths of the orchestrator-based saga pattern is the degree of control it affords over the entire lifecycle of a business process. Because the orchestrator explicitly issues each command in turn, you have precise command over the order of operations, the shape of the data flowing between steps, and the exact error-handling logic at each junction. This sequential, message-driven choreography means the orchestrator can apply conditional branching and enforce global timeouts or retry policies centrally, without scattering that logic across every microservice.

That level of centralized oversight comes at a cost, however. Even though the orchestrator relieves each participant of having to watch for every other component's events, you still must design and thoroughly test compensating transactions for every potential failure. Rolling back a successful payment, for instance, means the orchestrator must both publish a `RefundPayment` command and then wait for a `PaymentRefunded` event before it can mark the overall saga as failed. While the orchestrator centralizes the workflow, it does not make compensation simple by itself. You still need idempotent refund logic, reconciliation with the payment gateway, and database updates in the payment service, as well as accommodations for partial failures during the refund process.

Technically, this sequential flow is enforced by the orchestrator's state machine, where each incoming event triggers a transition to a new state, and only in that state does the orchestrator emit the following command. MassTransit wires up dedicated endpoints for both commands and events, and the orchestrator's saga repository persists its current state in a database, allowing it to safely pause between steps, survive restarts, and handle duplicate messages idempotently.

It is essential to remember that while orchestration reduces direct service-to-service coupling, every service must still adhere to the commands and events defined by the saga. The appointment booking service, for example, must implement a handler for `BookAppointmentCommand` that respects the expected input contract and emits either `BookingSucceeded` or `BookingFailed`. In this sense, services remain autonomous in their internal logic but must conform to a shared choreography contract established by the orchestrator.

Because the tight integration with MassTransit's `SagaStateMachine<T>` and `SagaStateMachineInstance` can make your saga logic feel coupled to MassTransit's APIs, one mitigation strategy is to isolate your core business workflow definitions into a separate library or layer. There, you define pure, framework-agnostic interfaces and data models. The MassTransit-specific state machine and EF repository then become an infrastructure concern that merely wires those definitions into the message bus and the database. That separation eases the pain of swapping out the messaging framework or evolving your saga logic independently of the orchestrator's plumbing.

Be sure to scope your need for a saga properly. If your saga has very few steps, a whole state machine might be overkill. Sometimes, a more straightforward approach, such as building an orchestrator service and tracking states in a table, may suffice. Sometimes, there is a concise *request* → *response* → *done or fail* scenario that can be implemented, where the library has many advanced features such as timers, concurrency strategies, and advanced transitions that might be unused yet still require learning.

While the built-in EF concurrency support is beneficial, it can add complexity if you're not already familiar with concurrency tokens. You might have to configure your repository carefully for production-scale loads.

Now that we have explored the implementation strategies for the saga, let us review what we have learned in this chapter.

## Summary

We began by emphasizing the importance of data consistency in a microservices architecture, where each service typically has its own dedicated database. Traditional ACID transactions become difficult in this setting because a single business operation can span multiple services. The Saga pattern, introduced as a solution to this challenge, splits a larger transaction across multiple local transactions. Each microservice involved handles its step independently and communicates success or failure through events, thus allowing for compensating actions when one of the steps fails.

A key part of the chapter covered the concept of coordination, which can be implemented through either choreography or orchestration. In the choreography approach, each service subscribes to relevant events and publishes new events after it finishes its local transaction. If anything goes wrong, those services that need to roll back listen for failure events and perform local compensations. This approach keeps services autonomous but can become increasingly complex as the number of participants and possible failure scenarios grows, making it harder to monitor and debug. The example illustrated how a user registration and appointment booking process might function, with participating services exchanging and processing events.

We then reviewed a practical example using MassTransit and RabbitMQ, explaining the difficulties of growing complexity in a choreographed saga. When more services and steps are added, one can easily end up with a complicated mesh of events and remedial actions. This is a significant reason to consider orchestration as an alternative.

In an orchestrated saga, a dedicated microservice known as the orchestrator sits at the center of the system and makes decisions about the workflow. Rather than having services emit events on their schedule, the orchestrator controls each step, calling individual services in a sequence, listening for success or failure, and then acting accordingly. If a failure occurs during any step, the orchestrator can determine the best course for rolling back or compensating. While this model is often easier to debug and maintain, it does introduce a single point of control that could become a bottleneck if not carefully designed. It also requires the orchestrator to know precisely how each service participates and what needs to happen next, which can add complexity to the central logic.

Throughout the chapter, best practices and recommendations were provided, highlighting how sagas are not a trivial matter. Always be aware that implementing this pattern increases overall code complexity and requires thorough planning of error handling, retry logic, and compensating operations. However, the benefits of consistency and reliability can be significant, as the application will be better equipped to handle partial failures and maintain consistent distributed data.

Through examples and explanations, this chapter aimed to make it clear how sagas, whether through choreography or orchestration, provide a robust strategy for tackling the challenges of distributed transactions in a microservices environment.

Now that we have explored sagas and how they work and are implemented, in the next chapter, we will move on to explore techniques for building more resilient microservices.



## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW



---

# Part 3

---

## Resiliency, Security, and Infrastructure Patterns

Here, you'll harden your services and their edges: add caching and failure-handling policies, introduce gateway and BFF patterns, extend microservice boundaries into the UI with micro-frontends, and protect every hop with token-based security.

This part has the following chapters:

- *Anchor 2, Building Resilient Microservices*
- *Anchor 2, Implementing the API and BFF Gateway Patterns*
- *Anchor 2, Micro Frontends: Extending Microservices to the Frontend*
- *Anchor 2, Securing Microservices*



# 10

## Building Resilient Microservices

In the previous chapter, we explored the **saga pattern**, which allowed us to orchestrate long-running business transactions across multiple services without compromising consistency. Sagas provide a crucial layer of fault tolerance for business workflows, but they are just the beginning when designing genuinely **resilient** systems.

Modern applications must be built to expect failure. In such systems, failure is not an anomaly but an operational certainty. Whether it's a network timeout, a downstream service crash, a temporary infrastructure hiccup, or an unresponsive third-party API, your application must be engineered to handle failure gracefully. This chapter is dedicated to helping you design **resilient microservices** using .NET technologies that can withstand and recover from these challenges in production.

**Resilience** refers to a system's ability to recover from failures and continue functioning, even in a degraded state. In microservices, resilience goes beyond exception handling. It encompasses timeouts, retries, circuit breakers, bulkhead isolation, failover strategies, monitoring, caching, and other related features. These mechanisms work together to ensure that faults are contained and that recovery paths are automated and well defined.

Do note that the resiliency measures we will discuss are supported here at the application level. While cloud environments, such as Microsoft Azure, AWS, or Google Cloud, do provide many infrastructure-level resiliency features (such as auto-retries, connection pools, service meshes, and failover), they often do not replace the need for application-level patterns, such as those discussed in this chapter.

In this chapter, we will cover the following topics:

- Understanding core failure scenarios between microservices
- Implementing fault-handling policies in synchronous communication
- Implementing fault-handling policies in asynchronous communication

These topics are explored through practical examples using our healthcare management system, grounded in real-world scenarios. You'll see how to apply these concepts in code to create functional, durable, and production-ready services. By the end of this chapter, you'll have a resilient services toolkit that will help you confidently deliver reliable, scalable, and responsive systems in the face of inevitable failure.

Let us begin by exploring how microservices communicate and what can go wrong.

## Technical requirements

Code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET--Second-Edition/tree/main/Chapter10>.

We use Redis for our distributed caching exercises later in this chapter, so you'll need a running instance to follow along. If you haven't already downloaded and installed it, you can get started here: <https://redis.io/downloads/>.

## Understanding core failure scenarios between microservices

In any distributed system, the potential for failure increases with the number of networked components. This reality is magnified in microservices architectures, where services must collaborate over network boundaries to fulfill even the simplest business use cases. While it is essential to mitigate network and infrastructure-level failures, these application-level measures can go a long way in ensuring that your application operates in a fault-tolerant state.

Even when operating in cloud environments such as Microsoft Azure, AWS, or Google Cloud, it is essential to understand that many infrastructure-level resiliency features do not replace the need for application-level patterns.

For example, Azure Application Gateway or AWS API Gateway can provide retry logic and timeouts at the networking level. However, this does not protect you from logical or transient failures that occur inside your application logic or between tightly coupled services. Therefore, even in

cloud-hosted systems, implementing these patterns at the application level adds an essential layer of fault tolerance, allowing for more fine-grained control over how services behave under failure conditions.

Our healthcare system microservices require a lot of collaboration, which often involves multiple internal microservices and integrates with external services. Let us consider a healthcare management system composed of the following core microservices:

- **Patient service:** Manages patient demographics and records
- **Appointment service:** Schedules appointments with physicians
- **Notification service:** Sends emails and SMS updates
- **Integration service:** Connects to third-party services such as insurance APIs for eligibility and authorization

Each of these services will perform its set of operations, and there are times when an operation requires a call to another web service. Several things can go wrong, and it is essential to plan and develop with the possible failure points in mind.

Services communicate either synchronously or asynchronously, and both methods have their pros and cons, as well as potential failure points. We will begin by discussing some of the failure points of synchronous API communication.

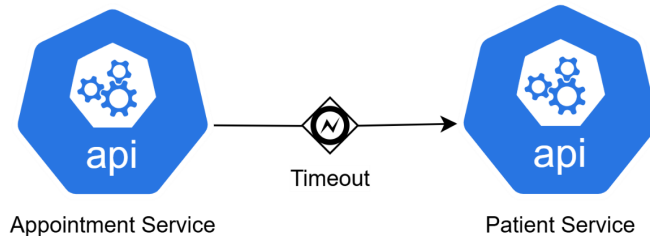
## Synchronous API failures and how to handle them

Several reasons can cause errors when services communicate. Some of these errors may be transient, meaning they occur due to a temporary outage, while others are more permanent, resulting from a more serious outage. Either way, we must anticipate these scenarios and write our code accordingly. First, let us review timeout failures.

Let us simulate a call between the appointment service and the patient service to get the details of the patient booking an appointment. Due to a high processing load, the request takes longer than expected and exceeds the configured timeout. This could be due to a request taking longer to process in the target service and exceeding the maximum wait period configured for the calling service. This might lead to the following outcomes:

- The client receives a 504 Gateway Timeout error
- Patients cannot book appointments, leading to frustration
- If unhandled, it may trigger retries and amplify the load

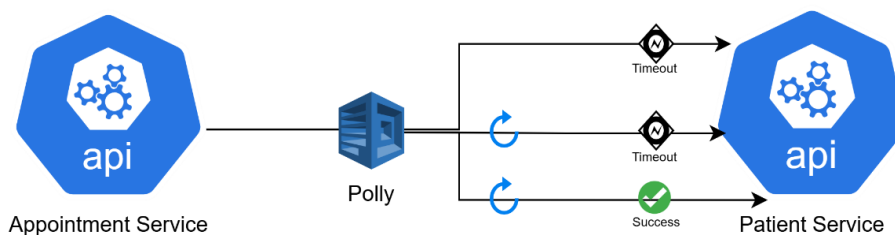
This is a common reason for failures in service-to-service calls, and in theory, we can mitigate this with explicit and reasonable timeouts configured in our service call code. There is no direct way to resolve a timeout, but we can choose to handle the fallout from the failed service call in a graceful manner. This timeout from a service-to-service call is depicted in *Figure 10.1*.



*Figure 10.1 – The service call times out, leading to an unexpected error*

Another possible reason for timeouts is **service unavailability**. This may be due to the target service undergoing maintenance or being temporarily unavailable, or to internal issues with the target microservice, such as the database becoming inaccessible during the request. Adding a cache can, to some extent, help mitigate the likelihood of frequent database outages and speed up response times, also reducing the risk of timeouts.

We can implement **circuit breakers** to stop further calls until the system stabilizes. Packages exist to assist with this implementation, such as **Polly** (a .NET library for transient-fault handling and resilience), which provides retry and circuit breaker policies. This allows us to configure timeout and retry logic to deal with issues like this. Retry logic becomes especially useful for transient failures, as it allows us to more accurately guarantee that a service call attempt will be successful in subsequent calls. In *Figure 10.2*, we see how Polly can assist in implementing retries.



*Figure 10.2 – Polly orchestrates service calls and retries after a failure*

The stakes increase when third-party services are introduced, since we cannot access or see the possible reasons for an outage. Similarly, we must implement fallback mechanisms to log a manual verification task, apply retry policies, and set realistic timeout limits. Recording degraded mode operations for later reconciliation would also be helpful.

Another potential failure point that is sometimes overlooked is poor **authentication** handling. Authentication is the first line of defense in any secured system. In a microservices architecture, each service operates independently, and different services may require distinct authentication strategies. We will discuss authentication strategies, but if we consider this a failure point, we see where interservice communication may sometimes fail between services, resulting in a 401 or 403 response code. This is most often due to the calling service failing to attach the required API key or authentication token. In other cases, it might be that the attached token has expired, and the service needs to retrieve a fresh token and then retry the initial request.

Now, let us review why asynchronous communication might fail and the potential mitigations.

## Asynchronous API failures and how to handle them

As previously discussed, asynchronous communication is crucial for decoupling services, enhancing scalability, and improving responsiveness, particularly in systems that must tolerate variable processing loads or operate across complex workflows. This mode of communication introduces a different class of failure scenarios that differ significantly from those in synchronous APIs.

Let us consider that the appointment service sends a message to the notification service after a successful booking. There is a tangible risk of message loss, which may occur if a broker fails or is misconfigured, messages do not persist due to the use of non-durable queues, or the consumer crashes before acknowledging a message. One way to address this is to use **durable queues** (e.g., RabbitMQ with persistence), ensure proper **acknowledgment** patterns (*Complete*, *Abandon*, or *Dead Letter*), or persist critical messages to a transactional outbox before enqueueing.

There is also the risk of timeouts, as a network failure can lead to poor or interrupted communication. A retry is reasonable, but it can lead to acknowledgment loss and cause the consumer to reprocess the same message multiple times. Ensure idempotency in message handlers and add **deduplication tokens**, such as message and correlation IDs. This way, we can store processed message IDs in a database or cache. We saw how useful this can be when implementing a choreography saga.

The **outbox pattern** is another popular method for handling system message loss. It ensures that changes to application state and message sending are handled atomically. Instead of publishing a message directly to a message broker within the same transaction as a database update, the outbox pattern stores the message in a dedicated **outbox table** in the local database as part of the transaction. A separate background worker, often referred to as an outbox processor or dispatcher, periodically scans the outbox table and publishes the messages to the actual message broker. *Figure 10.3* depicts this flow.



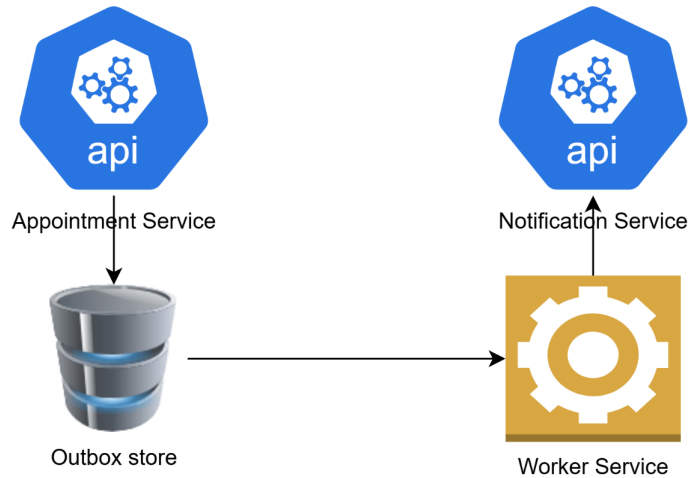


Figure 10.3 – Messages are stored in a database before the worker service processes them

This pattern solves the **dual write problem**, the risk of writing to a database and a message broker separately, where one operation may succeed and the other may fail, resulting in data inconsistency.

Now that we have explored possible failure points in synchronous and asynchronous service communication, we will review code implementations that address the different gaps. We will begin by addressing synchronous communication failure points.

## Implementing fault-handling policies in synchronous communication

For this implementation, we will build on the existing appointment service project, which currently has code that synchronously calls several services. We will begin by preparing it with Polly.

Polly is a framework that allows us to add a layer of resilience to our applications. It acts as a layer between two services, stores the details of an initiated request, and monitors the response time and/or the response code. It can be configured with parameters that determine what constitutes a failure, and we can further configure the type of action we would like to take. This action can take the form of a retry or cancellation of the request.

To add it to our .NET Core application and allow us to write extension code for our `HttpClient` objects, we start by adding these packages via NuGet:

```
dotnet add package Polly
dotnet add package Microsoft.Extensions.Http.Polly
```

Now, in our `Program.cs` file, we can configure our typed HTTP client to use our extension code for its Polly-defined policies. Add the following using statements:

```
using Polly;
using Polly.Extensions.Http;
```

We can then define the registration of our typed client like this:

```
builder.Services.AddHttpClient<PatientsApiClient>(client =>
{
    client.BaseAddress =
        new Uri(builder.Configuration["ApiEndpoints:PatientsApi"]);
}).AddPolicyHandler(GetRetryPolicy());
```

We have added a policy handler to our HTTP client, which will automatically be invoked for all calls made using this client. We must now define a method called `GetRetryPolicy()` to build our policy:

```
static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()
{
    return HttpPolicyExtensions.HandleTransientHttpError()
        .OrResult(r => !r.IsSuccessStatusCode)
        .Or<HttpRequestException>()
        .WaitAndRetryAsync(5, retryAttempt =>
            TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
            (exception, timeSpan, context) => {
                // Add logic to be executed before each retry, such as
                // logging or reauthentication
            });
}
```

It may not seem straightforward due to the builder pattern, but it is simple to understand and flexible enough to be customized to your needs. Firstly, we define the return type of the method as `IAsyncPolicy<HttpResponseMessage>`, which corresponds to the return type of our calls from the HTTP client. We then allow the policy to observe for transient HTTP errors, which the framework can determine by default, and we can extend that logic with more conditions, such as observing the value of `IsSuccessStatusCode`, which returns true or false for the success of an operation, or even whether `HttpRequestException` has been returned.

These few parameters cover the general worst-case scenarios of an HTTP response. We then set the parameters for our retries. We want to retry at most five more times, and each retry should be performed at a rolling interval starting two seconds after the previous call. This way, we allow a little time between each retry. This is the concept of the **backoff**. Finally, we can define the action we would like to take between each retry. This could include error handling or re-authentication logic.

Retry policies can negatively impact your system, leading to high concurrency and resource contention. We must ensure a solid policy and define our delays and retries efficiently. A carelessly configured retry policy may open the application to significant performance issues.

Given the possibility of implementing something that could have adverse effects in this manner, we now need a defense mechanism that will mitigate this risk and break the retry cycle if the errors persist. The best defense strategy is the circuit breaker, which we will configure using Polly next.

## Adding a circuit breaker policy

We should handle faults that take a bit longer to resolve and define a policy that abandons retry calls to a service when we have concluded that it is unresponsive for longer than hoped. What we don't want to do is continue retrying service calls without some form of exit condition. This would be like implementing an infinite loop if the target service remains unresponsive, and possibly cause an inadvertent **denial-of-service** attack on your service. For this reason, we implement the circuit breaker pattern, which acts as an orchestrator for our service calls.

We can introduce a circuit breaker between the client and the server. In our microservices application, the service making the call is the client, and the target service is the server receiving the request. Initially, it allows all calls to go through. We call this the **closed state**. If an error or delayed response occurs, the circuit breaker opens. Once the circuit breaker is open, subsequent calls will fail faster. This will shorten the time spent waiting for a response. It will wait for a configured timeout period and then allow the calls again if the target service has recovered. If there is no improvement, the circuit breaker will break the transmission.

We can continue from our code that added the retry policy using Polly by defining a circuit breaker policy and adding it as an HTTP handler to our client. We would modify the client's registration in `Program.cs` like this:

```
builder.Services.AddHttpClient<PatientsApiClient>(client =>
{
    client.BaseAddress =
```

```
        new Uri(builder.Configuration["ApiEndpoints:PatientsApi"]);  
    }).AddPolicyHandler(GetRetryPolicy())  
    .AddPolicyHandler(GetCircuitBreakerPolicy());
```

Now, we can add the `GetCircuitBreakerPolicy()` method like this:

```
static IAsyncPolicy<HttpResponseMessage> GetCircuitBreakerPolicy()  
{  
    return HttpPolicyExtensions  
        .HandleTransientHttpError()  
        .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));  
}
```

This policy defines a circuit breaker that opens the circuit when there have been five consecutive retry failures. Once that threshold is reached, the circuit will break for 30 seconds and automatically fail all subsequent calls, which will be interpreted as HTTP failure responses.

With these two policies in place, you can orchestrate your service retries and significantly mitigate the impact of unplanned outages on your application and the end user's experience. The circuit breaker policy also protects from any potential adverse effects of the retry policy.

Now, let us add a caching layer to reduce some of the load on the database for read requests.

## Adding a caching layer

**Caching** can be a valuable part of the resiliency strategy. We can implement a caching strategy that falls back on this cache if a service is unavailable. This means that we can use the caching layer as a fallback data source and create an illusion to the end user that all services are up and running. This cache would be periodically updated and maintained each time data in the source service's database is modified. This will help keep the cached data fresh.

Of course, with this strategy, we need to accept the implications of having potentially stale data. If the source service is offline and the supporting database is being updated, possibly by other jobs, then the cache will eventually become a stale data source. The more measures we implement to ensure its freshness, the more complexity we introduce to our application.

Notwithstanding the potential pros and cons of adding a caching layer, we can see where it will be a great addition to our microservices application and reduce the number of errors a user might see stemming from transient and even longer-term failures.

The most effective way to implement a caching layer is as a distributed cache. This means that all systems in the architecture can access the central cache, which exists as an external and stand-alone service. This implementation can increase speed and support scaling. We can use the **Redis cache** as our distributed cache technology, and we will investigate how we can integrate this into an ASP.NET Core application.

Redis is a popular caching database technology. It is an open source, in-memory data store that can also be used as a message broker. It can be installed on a local machine for local development efforts, but is sometimes deployed on a central server for more distributed systems. It is a key-value store that uses a unique key to index a value, and no two values can have the same key. This makes it very easy to store and retrieve data from this data store. In addition, values may be stored in very simple data types such as strings, numbers, and lists, but **JSON** is popularly used for more complex object types. This way, we can serialize and deserialize this string in our code and proceed as needed.

Extensive support for the Redis cache is also available on cloud providers such as Microsoft Azure and AWS. For this exercise, you may install the Redis cache locally or use a Docker container.

If you already have Docker installed from our previous activities, proceed to use the following commands to pull the Redis image and instantiate a container:

```
docker pull redis
docker run --name redis-cache -d -p 6379:6379 redis
```

You can run this command and send the ping command to verify that the container is running. You should receive a PONG response:

```
docker exec -it redis-cache redis-cli
```

To start using the Redis cache in our project, we need to run the following command:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

We then need to register our cache in `Program.cs` like this:

```
// Register the RedisCache service
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
        builder.Configuration.GetConnectionString("RedisConnection");
});
```

You may configure your connection string in either `appsettings.json` or in application secrets. It will look something like this:

```
"ConnectionStrings": {  
  // Other connections  
  "RedisConnection": "localhost:6379,abortConnect=false"  
},
```

Now, we can read from and write to the cache as needed in our application. Generally, we want to write to the cache when data is augmented. New data should be written to the cache; we can either remove the old version and create a new version for modified data or delete the data from the cache.

We can create a singular `CacheProvider` interface and implementation as a wrapper around our desired cache operations. Our interface will look like this:

```
public interface ICacheProvider  
{  
    Task ClearCache(string key);  
    Task<T> GetFromCache<T>(string key) where T : class;  
    Task SetCache<T>(string key, T value, DistributedCacheEntryOptions  
        options) where T : class;  
}
```

Our implementation looks like this:

```
public class CacheProvider : ICacheProvider  
{  
    private readonly IDistributedCache _cache;  
  
    public CacheProvider(IDistributedCache cache)  
    {  
        _cache = cache;  
    }  
  
    public async Task<T> GetFromCache<T>(string key) where T : class  
    {  
        var cachedResponse = await _cache.GetStringAsync(key);  
        return cachedResponse == null ? null :  
            JsonSerializer.Deserialize<T>(cachedResponse);  
    }  
}
```

```
}

public async Task SetCache<T>(string key, T value,
    DistributedCacheEntryOptions options) where T : class
{
    var response = JsonSerializer.Serialize(value);
    await _cache.SetStringAsync(key, response, options);
}

public async Task ClearCache(string key)
{
    await _cache.RemoveAsync(key);
}
}
```

This code allows us to interact with our distributed caching service and retrieve, set, or remove values based on their associated key. This service can be registered in our **Inversion of Control (IoC)** container and injected into our controllers and repositories. The idea here is that we can use the `GetFromCache` method when we need to read the values from the cache. The key allows us to narrow down the entry we are interested in, and the generic `T` parameter will enable us to define the desired data type. If we need to update the data in the cache, we can clear the cache record associated with the appropriate key and then use `SetCache` to place new data with an associated key. We will parse the latest data to JSON to ensure we do not violate the supported data types while maintaining the ability to store complex data. When we add new data, we need to call `SetCache` and add the new data.

We also want to ensure that we maintain the data's freshness as much as possible. A popular pattern involves clearing the cache and making a fresh entry each time data is entered or updated.

We can use these bits of code in our application and implement a caching layer to improve performance, resiliency, and stability. Now, let's add this to our `Appointments` controller.

First, modify the constructor to accept `IDistributedCache`, which supports Redis out of the box via `Microsoft.Extensions.Caching.StackExchangeRedis`:

```
public class AppointmentsController(
    // Other services
    IDistributedCache _cache) : ControllerBase
```

We will now modify the `GetAppointment` method to use the cache. First, we will check the cache using the `GetFromCache` method. If it is not found, we will retrieve it from the database, build the `AppointmentDetails` object, serialize it, and store it in the cache:

```
[HttpGet("{id}")]
public async Task<ActionResult<Appointment>> GetAppointment(Guid id)
{
    var cacheKey = $"appointment-details:{id}";
    var cached = await _cache.GetStringAsync(cacheKey);

    if (!string.IsNullOrEmpty(cached))
    {
        var cachedDetails =
            JsonSerializer.Deserialize<AppointmentDetails>(cached);
        return Ok(cachedDetails);
    }

    var appointment = await _context.Appointments.FindAsync(id);
    if (appointment == null)
    {
        return NotFound();
    }

    var patient = await _patientsApiClient.GetPatientAsync(id);
    var doctor = await _doctorsApiClient.GetDoctorAsync(id);

    using var channel = GrpcChannel.ForAddress
        (_configuration["GrpcEndpoints:DocumentService"]);
    var client = new DocumentService.DocumentServiceClient(channel);
    var documents = await client.GetAllAsync(new PatientId
        { Id = patient.PatientId.ToString() });

    var appointmentDetails = new AppointmentDetails(
        id,
        patient,
        doctor,
        appointment.Slot.Start,
        appointment.Slot.End,
```



```
        appointment.Location.RoomNumber,  
        appointment.Location.Building,  
        documents  
    );  
  
    var serialized = JsonSerializer.Serialize(appointmentDetails);  
    await _cache.SetStringAsync(cacheKey, serialized,  
        new DistributedCacheEntryOptions  
    {  
        AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10) // TTL  
    });  
    return Ok(appointmentDetails);  
}
```

The benefits here are significant because while we set the **time to live (TTL)** to 10 minutes, this can be much longer for data that does not change frequently. Additionally, since we utilize three data sources in this call, adding a cache will enhance the performance of subsequent queries for the details of the same appointment record.

Now that we have made our service more performant and reliable with caching, let's review how to reduce the likelihood of failures due to authentication errors.

## Reducing authentication errors

Securing web services is a common-sense practice, so this is an expectation for both internal and third-party APIs. Each microservice should authenticate itself or act on behalf of a user or system identity. Understanding the flow ensures that developers don't unintentionally drop or overwrite necessary authentication headers when making service calls.

Integrating authentication handling into the core of your service architecture and treating it as a first-class resiliency concern drastically reduces the chance of silent failures or inconsistent behavior. Your services will be better prepared to interact safely. Writing responsible authentication code is more than just passing tokens and checking permissions; it's about building a security-conscious development culture that anticipates failure, prevents misuse, and ensures trust across your distributed system.

The first common way to secure an API is with an API key. The API key is usually passed as an HTTP header with the request. If this key is missing or incorrect, the response is expected to be 401 - Unauthorized. Attaching this key before the request is easy enough when using the

HttpClient framework. A common way to ensure that all requests for a registered client are outfitted with all the necessary information to make a successful call is to use a delegate handler. This delegate handler acts as an interceptor, allowing you to enrich the request before it gets sent. In the following code, we will intercept a request, attach the API key header and value, and then forward the request:

```
public class ApiKeyHandler(IConfiguration _configuration) :
    DelegatingHandler
{
    private string _headerName = "x-api-key";
    private string _apiKey = _configuration["ApiKey"];
    protected override Task<HttpResponseMessage>
        SendAsync(HttpRequestMessage request,
            CancellationToken cancellationToken)
    {
        if (!request.Headers.Contains(_headerName))
        {
            request.Headers.Add(_headerName, _apiKey);
        }
        return base.SendAsync(request, cancellationToken);
    }
}
```

Once the handler is created, it must be registered as a standalone service and attached to the relevant client. Let us say that this API key is to be used by an external or third-party service, and we have an ExternalApiService class to handle these requests:

```
builder.Services.AddTransient<ApiKeyHandler>();
builder.Services.AddHttpClient<ExternalApiService>(client =>
{
    client.BaseAddress =
        new Uri(builder.Configuration["ThirdPartyApi:BaseUrl"]);
})
.AddHttpMessageHandler<ApiKeyHandler>();
```

For demo purposes, the ExternalApiService class can be defined as this:

```
public class ExternalApiService(HttpClient _httpClient)
{
```

```
public async Task<string> GetSecureDataAsync()
{
    var response = await _httpClient.GetAsync("/secure-data");
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync();
}
```

The alternative, possibly more popular, and secure method of securing APIs is through a **JSON Web Token (JWT)**. This is the preferred method for internal services, and we will discuss this implementation in detail later in the book. A JWT is a compact, URL-safe token used for authentication and authorization. It is issued to a client after a successful login operation. JWTs are perfect for authentication and authorization in microservices, as APIs are stateless and don't track authentication sessions.

The JWT contains information that helps identify the user or client accessing a service. If the token is valid, then access is granted. We can use a handler to attach a JWT to each request and help avoid 401 – Unauthorized errors:

```
using System.Net.Http.Headers;
namespace AppointmentsApi.Handlers;
public class JwtAuthHandler(IConfiguration _configuration) :
    DelegatingHandler
{
    protected override Task<HttpResponseMessage>
        SendAsync(HttpRequestMessage request,
            CancellationToken cancellationToken)
    {
        var token = _configuration["JwtSettings:Token"];
        if (!string.IsNullOrEmpty(token))
        {
            request.Headers.Authorization =
                new AuthenticationHeaderValue("Bearer", token);
        }
        return base.SendAsync(request, cancellationToken);
    }
}
```

We will register this handler similarly, but attach it to `DoctorsApiClient` for demo purposes. The token will be connected to all requests to the Doctors API:

```
builder.Services.AddTransient<JwtAuthHandler>();
builder.Services.AddHttpClient<DoctorsApiClient>(client =>
{
    client.BaseAddress =
        new Uri(builder.Configuration["ApiEndpoints:DoctorsApi"]);
})
.AddHttpMessageHandler<JwtAuthHandler>());
```

There are other factors that might still lead to failures, such as the following:

- **Expired JWT:** Tokens are issued with a limited validity period. If a token is present but expired, the code must be written so that a new token is retrieved and used in a subsequent call. This way, the initial failure does not affect the end user.
- **Forbidden:** This is a 403 – Forbidden response, which means that the token authenticates correctly (identifies the calling service or client), but the app does not have the permissions required to access the requested resource. There is little one can do to prevent this from affecting other services.

Now that we have reviewed methods for mitigating synchronous failures, let's review some implementations that can help us manage the failure points of asynchronous communication.

## Implementing fault-handling policies in asynchronous communication

We just reviewed retry policies, circuit breaker patterns, and caching, and explored how we can make synchronous API calls more resilient while failing more gracefully. Recall that asynchronous communication does not occur directly between services but requires an intermediary message delivery system, such as a message broker.

You may recall that message brokers are designed to be durable during message transmission and have mechanisms that can assist in persisting messages until a consumer processes them. Let us take some time to review some of the mechanisms that are available in not only RabbitMQ but also other message brokers.

## Resiliency in message brokers and queues

Decoupling services for asynchronous communication presents several challenges. We have already explored many of them in detail, but now we can focus on how we can mitigate data loss due to delivery failures, ensure reliability under load, and maintain consistency across distributed operations.

Transient faults, such as momentary network delays or unavailable services, are common in distributed systems. Because of this, it is prudent for us to introduce retry policies on our message consumers. Since we are using `MassTransit`, we can review how easy it is to configure retry policies on consumers.

Consider `AppointmentCreatedConsumer` that handles appointment confirmation messages; we can configure a retry policy during registration. In the `MassTransit` configuration section of `Program.cs`, we can add the following section:

```
services.AddMassTransit(x =>
{
    x.AddConsumer<AppointmentCreatedConsumer>(cfg =>
    {
        cfg.UseMessageRetry(r =>
        {
            r.Interval(3, TimeSpan.FromSeconds(5)); // Retry 3 times with
            // 5s interval
        });
    });

    x.UsingRabbitMq((context, cfg) =>
    {
        cfg.ConfigureEndpoints(context);
    });
});
```

This retry policy ensures that if a consumer throws an exception, the message will be retried three times at five-second intervals before being considered failed.

Then, we have the concept of a **dead-letter queue (DLQ)**. Since message delivery is not guaranteed in asynchronous systems, a DLQ is a special queue designed to catch messages that cannot be successfully processed, even after multiple retries. They serve as a safety net, allowing system operators and developers to inspect, analyze, and potentially reprocess failed messages without losing critical information or crashing the system.

Given the previous configuration, `MassTransit` automatically moves a failing message to an error or DLQ. For example, a failure in `AppointmentCreatedConsumer` will result in the message being redirected to the `appointment-created_error` queue. This queue can then be monitored or drained for diagnostic purposes. It is typical to create a dedicated consumer to log or reprocess messages manually in the error queue. This is especially useful in production environments where failed messages need to be investigated or replayed.

`MassTransit` is designed to provide a unified abstraction over multiple message brokers. Therefore, the retry, dead-lettering, outbox, and consumer configurations work across multiple transport providers, but their behavior varies depending on the underlying broker and its capabilities.

For instance, Azure Service Bus retry and DLQ functionality are mapped differently. Azure natively has DLQs in the form of the `$DeadLetterQueue` subqueue, but `MassTransit` emulates `_error` queues as well. Also, message TTL and session support are mapped to Azure constructs. In general, there is good support but nuanced behavior due to Azure's built-in features.

`MassTransit` also works well with Amazon SQS and basic retry policies, but SQS doesn't support some advanced patterns. DLQs must be configured manually in AWS and are not automatically created by `MassTransit`. Visibility timeout and max receive count are governed by AWS, not `MassTransit` retry configuration.

Generally, it is important to understand how the message broker or queue technology of choice works and to figure out whether fault-tolerance can be enforced in code or whether it must be manually configured in the platform.

We have seen how we can attempt to ensure that failed messages are backed up after several retries, but there are times when we need less volatile storage to maintain the history of messages to be processed. This is where we need to employ the outbox pattern. Let us review this pattern next.

## Using the outbox pattern

The outbox pattern is a technique for ensuring reliable and consistent communication between services. It utilizes the local database as a **durable buffer** for messages that require publication. This pattern guarantees **atomicity**: either the database update and the message write happen together, or neither does.

First, we need to create an `OutboxMessage` entity to represent the events that must be published:

```
public class OutboxMessage
{
    public Guid Id { get; set; }
```

```

    public string EventType { get; set; } = default!;
    public string Payload { get; set; } = default!;
    public DateTime OccurredOnUtc { get; set; }
    public bool Processed { get; set; }
    public DateTime? ProcessedOnUtc { get; set; }
}

```

Add DbSet for OutboxMessages to DbContext:

```

public class ApplicationDbContext : DbContext
{
    public DbSet<OutboxMessage> OutboxMessages { get; set; } = default!;
    // ... other DbSets and configurations
}

```

Now, let us modify the POST method to add the message to the outbox store instead of publishing to the message broker:

```

[HttpPost]
public async Task<ActionResult<Appointment>>
    PostAppointment(Appointment appointment)
{
    // Existing code to add record to database.
    // Shortened for brevity.
    var outboxMessage = new OutboxMessage
    {
        Id = Guid.NewGuid(),
        EventType = nameof(AppointmentCreated),
        Payload =
            JsonSerializer.Serialize(appointmentCreatedEvent),
        OccurredOnUtc = DateTime.UtcNow,
        Processed = false
    };
    _context.OutboxMessages.Add(outboxMessage);
    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
    return CreatedAtAction("GetAppointment", new { id =
        appointment.AppointmentId }, appointment);
}

```

Now, we can wrap this entire operation in a transaction to ensure that everything is either successful or unsuccessful.

Next, we create a hosted service that periodically checks for unprocessed outbox messages and publishes them to the message broker:

```
public class OutboxProcessor(IServiceProvider _serviceProvider,
    ILogger<OutboxProcessor> _logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken
        stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var scope = _serviceProvider.CreateScope();
            // Initialize required services, like the db context
            var messages = await dbContext.OutboxMessages
                .Where(m => !m.Processed)
                .OrderBy(m => m.OccurredOnUtc)
                .Take(20)
                .ToListAsync(stoppingToken);
            foreach (var message in messages)
            {
                try
                {
                    await messagePublisher.Publish(message.Payload);
                    message.Processed = true;
                    message.ProcessedOnUtc = DateTime.UtcNow;
                }
                catch (Exception ex)
                {
                    // Optionally implement retry logic or move to a
                    // dead-letter queue
                }
            }
            await dbContext.SaveChangesAsync();
        }
    }
}
```



Register `OutboxProcessor` as a hosted service in the `Program.cs`:

```
builder.Services.AddHostedService<OutboxProcessor>();
```

We can ensure that database changes and event publications are consistent. If the message broker is temporarily unavailable, messages remain in the outbox and are retried later. The outbox table also serves as a log of events to aid monitoring and debugging.

Now that we have explored techniques for increasing resilience in the calling and target services, let us summarize what we have learned.

## Summary

In this chapter, we explored techniques and tools to enhance distributed systems' fault tolerance, stability, and performance. Using a healthcare management system as the domain, it contextualized real-world scenarios where failures in service communication, whether synchronous or asynchronous, can have critical implications. It identified potential failure points and provided actionable strategies and implementation code to mitigate them effectively.

We began by emphasizing the inevitability of failure in distributed systems, highlighting that modern applications must be designed to recover gracefully from faults rather than assume uninterrupted operations. In microservices architectures, where multiple services work independently and communicate across network boundaries, failures such as timeouts, unavailability, and authentication breakdowns can easily disrupt workflows. We also categorized these failures and demonstrated how resilient design mitigates their impact using tools such as Polly for transient fault handling, Redis for distributed caching, and the outbox pattern for reliable event propagation.

One of the first types of failure explored was synchronous API communication failures, such as network timeouts or unavailability of a downstream service. We learned how retry policies using Polly can handle transient faults, allowing requests to be retried with exponential backoff. A circuit breaker pattern was introduced to guard against retry storms or indefinite loops. This prevents overloading services that are already struggling and shortens the time spent waiting for failing services to recover.

We then explored the importance of caching for read-heavy endpoints. We provided a detailed Redis implementation and outlined how to wrap cache logic in a reusable service layer. Caching significantly reduces pressure on services and databases while enabling the system to serve stale but usable data when the source service is unavailable. Although stale data introduces trade-offs, we understood that performance and reliability gains often outweigh the risks, especially when the cache is maintained and invalidated carefully.

In addressing authentication-related failures, we examined how API keys and JWTs can be automatically injected into HTTP requests using custom delegating handlers. This approach centralizes and secures identity propagation, reducing redundancy and the likelihood of missed headers. Additionally, we learned how services should respond to 401 (Unauthorized) and 403 (Forbidden) responses, as well as how expired tokens can be automatically refreshed to prevent cascading failures across dependent services.

The outbox pattern, a reliable method for ensuring message consistency in asynchronous service communication, was discussed and implemented. When services write domain data and outgoing messages within the same database transaction, the outbox pattern ensures atomicity; neither succeeds nor fails. A background worker service processes unsent messages in the outbox table and dispatches them to the message broker, avoiding the classic “dual write” problem. The implementation was detailed step by step, from defining the entity and adding it to DbContext, to publishing messages and updating their status.

The chapter is a comprehensive guide for .NET developers looking to build resilient, production-ready microservices. It strikes a balance between conceptual clarity and hands-on implementation, making it both accessible and practical. By incorporating retry strategies, circuit breakers, distributed caching, responsible authentication, and asynchronous messaging guarantees, you were equipped with the patterns needed to create reliable systems that degrade gracefully and recover quickly, exactly what modern microservices demand.

Now, we can proceed with implementing API gateways.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW





# 11

## Implementing the API and BFF Gateway Patterns

We have established that building a microservices application requires tracking multiple API endpoints. In *Anchor 5*, we examined the aggregator pattern as a potential solution for consolidating and unifying numerous endpoints into a single access point. While this method is helpful, it requires additional code and ongoing maintenance efforts.

The general idea is to keep access to our services as simple as possible as other APIs will call some of these endpoints, and some will integrate directly into the client applications that interact with the microservices.

This is where we consider implementing an API gateway. An API gateway serves as a unified interface for external clients, abstracting the complexity of the underlying microservices while streamlining requests and handling cross-cutting concerns, such as authentication, rate limiting, and logging.

The **Backend for Frontend (BFF)** pattern creates distinct gateways optimized specifically for different frontend applications, such as web, mobile, or third-party integrations. This ensures that each client receives the data they need in the most efficient format possible.

This chapter explores both the API and BFF gateway patterns. After reading this chapter, you will be able to do the following:

- Understand API gateways and why they are important
- Implement an API gateway using industry-leading technology and methods
- Properly implement the BFF pattern
- Understand security and rate limiting in API gateways

You will learn the key differences, advantages, and implementation considerations of each approach. You will gain the insights necessary to select and apply these patterns effectively through clear examples and best practices, enhancing your system's resilience, scalability, and maintainability.

Let us begin by exploring API gateways in more detail.

## Technical requirements

The code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter11>.

## What is an API gateway?

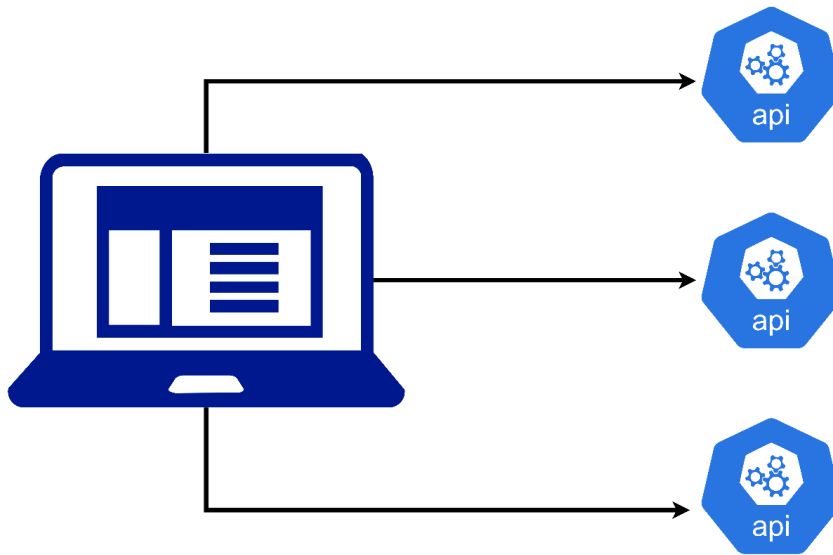
To understand the API gateway pattern and its necessity, we must review the fundamentals of service-oriented architecture and how these solutions are architected. The service-oriented architecture includes three main layers in an application:

- **Client:** Also referred to as a fronted client, this is what the user sees, and is designed to consume data from an API. Its functionality is generally limited to the API's available functions, and a frontend developer can leverage several techniques to expose functionality to the end user.
- **Server:** Also referred to as the backend, this section houses the API and the business logic. The client app is driven by the services made available in the backend.
- **Database:** The database is the heart of this application since it stores all the data used by the API backend and is displayed on the frontend. Recall that in microservices, multiple databases may be used by various services.

This application layout is popular in monolithic applications, where all necessary frontend functionality is contained in a single API. This effective development method has been instrumental in creating many successful and robust applications. We have, however, explored the downsides of a monolithic approach, where the API might become bloated and difficult to maintain in the long run.

While microservices architecture results in an application built with a service-oriented approach, we must recognize that our client will need to monitor several backends or APIs and be smart enough to orchestrate calls for each user request. In this setup, the client takes on the task of sequencing requests, managing dependencies, handling retries, and combining partial responses. This not only complicates the frontend logic but also tightly integrates the client with the internal backend structure. As a result, even minor changes to service contracts or routing can cause ripple effects, requiring frontend updates. Additionally, this approach increases latency and reduces resilience, since clients are now responsible for coordinating calls across distributed services that can fail or degrade independently. This is a significant load for the part of the application that should be the least complex.

*Figure 11.1* shows the client and microservices:



*Figure 11.1 – The client app needs to be aware of all the microservices' endpoints and retain knowledge of how each one works*

This is where we introduce an API gateway. This gateway will sit between our services and the client application, simplifying communication between the two and providing a crucial layer of abstraction between the microservices and the client UI. It acts as the unified front door to a distributed system. For the client, it will expose a single base URL, which the client will interact with and view as a unified API service; to the microservices, it will act as a conduit, forwarding requests from the client to the appropriate microservice. Let us review the advantages of introducing an API gateway.

## Advantages of an API gateway

When a client's request is received, the gateway interprets it, transforms the data if necessary, and then forwards it to the corresponding microservice. We can implement orchestration and result aggregation for situations where multiple microservices may need to be called and return more accurate data representations to the client as required.

This may sound redundant, considering that the aggregator pattern also facilitates these actions; however, using an API gateway often yields superior results. When orchestration and aggregation logic reside in a dedicated aggregator service, it introduces additional complexity. When new service interactions are required, developers must manually extend the aggregator's logic, which can potentially increase the risk of code duplication and maintenance overhead. An API gateway centralizes orchestration logic, simplifying the handling of multiple backend requests, efficiently aggregating responses, and isolating complex logic from individual microservices. This results in cleaner, more maintainable service implementations.

Using an API gateway for orchestration and result aggregation enforces a clear separation of responsibilities. Microservices focus on domain-specific logic without concern for how their responses are aggregated, while the API gateway orchestrates service interactions, aggregates results, and formats client-specific responses. This distinction helps maintain the purity and simplicity of the microservices, keeping them highly cohesive, independently deployable, and easily maintainable.

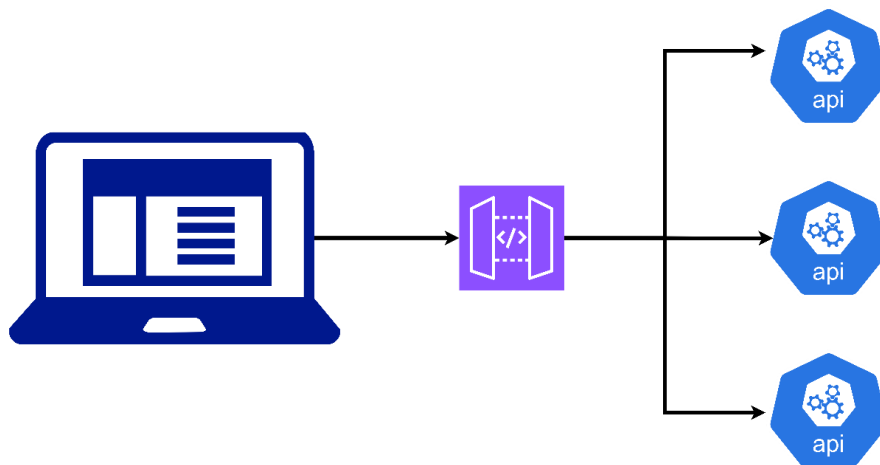
Our API gateway also has built-in capabilities for centralizing the following tasks for our microservices:

- **Centralized logging:** From the API gateway, we can centrally log all traffic to our various endpoints and track success and error responses from downstream services. This is advantageous because it spares us the need to implement redundant request/response logging in each service, helping to avoid an overly dispersed or “chatty” log collection. Using an API gateway allows us to centralize our implementation, prioritize what gets written to the log, and better catalog the outcomes of synchronous operations. We can also use the gateway to track and log statistics and response times of calls to the downstream services.
- **Caching:** Caching acts as a temporary data store that comes in handy when the primary data source is offline or when we need to reduce the number of times the service is called. We can utilize a caching layer in the gateway to enhance our application's performance and improve its resilience. With proper coordination and customization, we can utilize caching for high-speed read operations on high-traffic endpoints and even use it to handle partial failures, where we use cached data for a response when a service is unavailable.

- **Security:** Securing microservices can be a tedious and technical task. While individual services may have specific authorization rules, implementing standard security measures across multiple services can result in increased development overhead. We can centralize security measures at the gateway level using an API gateway. This can remove the burden from the microservice to authenticate and authorize access to resources, as the gateway will manage most of these requirements. We can also implement IP whitelisting at this level and limit access to an approved list of IP addresses.
- **Service monitoring:** We can configure our API gateway to perform proactive health checks on downstream services. As previously discussed, health checks or probes help us ascertain the status of our services. Since the gateway will need to forward requests, it is vital to determine the health of a downstream service before attempting an operation. As the gateway can determine the health of a service, it can be configured to handle failures and partial failures gracefully, for example, by routing around unhealthy instances.
- **Service discovery:** Our gateway must know the addresses of all services and how to transform and forward requests as needed. For this, a register of all the downstream services is required. This allows the gateway to efficiently locate and route requests to available service instances, effectively wrapping around the individual service endpoints and exposing a singular, stable address to the client application.
- **Rate limiting:** We sometimes want to limit the number of requests that can be sent quickly, from the same source, with the suspicion that such an activity might be a **denial-of-service (DoS)** attack on a service endpoint. We can implement generic rules that govern how often endpoints can be accessed using the API gateway.
- **Scalability:** An API gateway facilitates scalability by supporting features such as horizontal scaling, load balancing across service instances, and integration with auto-scaling capabilities. Manually coded aggregators require additional development overhead to handle these scenarios effectively. The gateway's flexibility in handling new client-specific demands or introducing new endpoints without extensive coding greatly enhances the adaptability of the microservices ecosystem.

Once again, the most essential aspect of the gateway's implementation is that it takes much of the responsibility away from our client, making scaling and diversifying client code much easier. *Figure 11.2* shows the client, a gateway, and microservices.





*Figure 11.2 – With the gateway introduced, the client app now has one endpoint and doesn't need to know the underlying services*

Now that we have seen how the API gateway helps us centralize access to several API endpoints and makes it easier for the client application to integrate API operations, let us review some of the disadvantages of using this pattern.

## Disadvantages of an API gateway

While the advantages are clear and irrefutable, we must also be aware of the downsides of introducing an API gateway. An API gateway is a central hub for all incoming client requests. If not designed for redundancy, fault tolerance, and high availability, it can become a single point of failure, potentially leading to significant downtime if it experiences issues. Therefore, you must implement redundancy and failover strategies, typically by deploying multiple gateway instances behind a load balancer.

Relying on an API gateway might encourage tight coupling to gateway-specific features, potentially undermining the microservices architecture's decentralized and autonomous nature. This risk arises if domain-specific logic or business rules leak into the gateway instead of remaining within the microservices. Separate concerns between gateway responsibilities and microservices. Keep the gateway logic minimalistic, pushing domain-specific logic into individual microservices.

Using an existing commercial or off-the-shelf gateway may limit flexibility, as it forces architectural constraints dictated by the gateway's design choices. This can impact your system's custom or unique requirements, leading to vendor lock-in and making future migration complex and costly. Evaluate whether custom-built API gateway solutions or plugins are necessary to meet your unique requirements.

Let us now review the ways an API gateway could be implemented.

## Implementing an API gateway

While API gateways significantly simplify communication and enhance system resilience, careful planning and informed architectural choices are critical. Awareness of the pros and cons enables architects to proactively address potential pitfalls, achieving the optimal balance of efficiency, scalability, and maintainability within their microservices architectures.

It is worth distinguishing between an API gateway and a service mesh. An API gateway is primarily concerned with managing external client access to a microservices system. It sits at the edge of the application, handling concerns such as routing, authentication, rate limiting, response aggregation, and protocol translation as requests enter from outside the system. In contrast, a service mesh is designed to manage internal communication between services. It operates within the mesh of microservices, handling low-level concerns such as service discovery, load balancing, observability, mTLS encryption, and retry policies through sidecar proxies.

API gateways and service meshes are complementary components where the gateway handles ingress traffic and client abstraction, while the mesh manages east-west traffic and internal service behaviors. Keeping this distinction clear helps teams apply the correct pattern to the most appropriate part of the system architecture.

When selecting an appropriate API gateway technology, you should consider the following factors:

- **Select the right API gateway technology:** Choosing an appropriate API gateway solution is foundational. Popular options include cloud-managed gateways such as Amazon API Gateway, Azure API Management, and Kong. Evaluate gateways based on your application's specific requirements, including scalability, ease of integration, security features, cost, and developer experience.
- **Define clear routing and request aggregation rules:** Clearly define routing rules to determine how incoming client requests map to backend microservices. The gateway can also orchestrate multiple service calls and aggregate results, returning a consolidated response to the client. This approach reduces network round-trip times and simplifies client-side logic.
- **Optimize performance and scalability:** Implement caching, asynchronous processing, and load-balancing strategies to enhance the performance and scalability of your API gateway. Utilize horizontal scaling to effectively handle fluctuating loads, minimize latency, and maintain high availability.
- **Monitoring and observability:** Integrate comprehensive monitoring, logging, and tracing capabilities within your gateway implementation to facilitate real-time tracking, quick issue diagnosis, and informed decision-making based on detailed metrics and logs.

In *Anchor 4*, we provided a detailed review of the implementation of the aggregator pattern. This led us to create an aggregator service that exposed a single endpoint and orchestrated several service calls as needed. The primary benefit of this approach is that we can fully control and manage how services get called and which ones are exposed via this aggregator service.

This is an excellent approach for a smaller application that might not have the most advanced cross-cutting needs and may have too many microservices to manage. However, as each service evolves and more are added, the maintenance effort increases, and this service needs to be adjusted frequently, contributing to development and administrative overhead. As the services become bloated with more and more concerns, it becomes a thick API gateway.

A **thick API gateway**, also known as a **heavy API gateway**, is an implementation where the gateway handles more substantial business logic and complex operations rather than simply routing requests or aggregating responses. Unlike a **thin API gateway**, which primarily manages straightforward request routing, authentication, and minimal aggregation, a thick API gateway can take on significant responsibilities, such as complex orchestration and choreography between services, and result aggregation and transformation. Thin API gateways can also be referred to as **reverse proxies**.

Let's take some time to appreciate what a reverse proxy is and how it can be utilized in the context of API gateways.

## Using a reverse proxy

A **reverse proxy** is an intermediary server that sits between client applications and backend servers. It is tasked with forwarding client requests to appropriate backend resources. Acting on behalf of the backend servers, a reverse proxy retrieves data and returns it to the client, providing an abstraction that isolates clients from internal infrastructure details. A reverse proxy primarily serves several key functions, including the following:

- **Load balancing:** Evenly distributes incoming requests among multiple backend servers, ensuring efficient resource usage, optimizing performance, and enhancing system reliability and fault tolerance
- **Security and protection:** Shields backend servers from direct external exposure, minimizing the risk of attacks such as **distributed denial-of-service (DDoS)**, malicious request handling, and unauthorized access attempts
- **Caching:** This method stores copies of frequently accessed data or responses, significantly reducing response time and lowering the load on backend servers

- **SSL/TLS termination:** This function handles encryption and decryption tasks, removing computational overhead from backend servers and centralizing the management of SSL certificates
- **Compression:** This process compresses responses from backend servers before sending them to clients, improving bandwidth utilization and reducing latency

A reverse proxy is a deliberately *thin-edge* component that does only transport-level work, while presenting a single, friendlier entry point to the outside world. The term *façade* is used more often than *API gateway* when describing them because they stop at networking concerns and, barring extremely advanced and flexible systems, do not inject business-aware logic, such as **JSON Web Token (JWT)** validation, quota enforcement, payload transformation, or service aggregation. Since they are so lean, the overhead that they introduce is minimal, making them ideal when raw throughput is paramount.

Popular examples of reverse proxies are **NGINX**, **HAProxy**, and **Apache HTTP Server**. These solutions are configured manually and often require service restarts to apply updates. Out of the box, these solutions do not get very far in filling the role of an API gateway, which is why we introduce yet another, **Yet Another Reverse Proxy (YARP)**.

## Implementing an API gateway using YARP

**YARP** is a modern, highly extensible, and configurable reverse proxy built using .NET technology. Unlike traditional standalone reverse proxies, YARP is not a dedicated server application; rather, it is a middleware library that integrates seamlessly into .NET applications. One of its most significant advantages is the ability to customize. YARP allows developers to tailor proxy behavior to their specific application requirements with minimal friction, using familiar tools and patterns.

YARP also offers numerous extension points, enabling the injection of custom logic at various stages of the request and response life cycle. This means developers can incorporate additional features or modify default behaviors using standard C# code, leveraging existing knowledge and skills. Its direct forwarding model supports scaling across multiple domains and backends, a rare capability among traditional reverse proxies. Additionally, Microsoft's active development and maintenance of YARP ensure it remains aligned with the latest industry standards and best practices.

YARP is backed by comprehensive documentation, which you can access on *Microsoft Learn*, and a wealth of examples to support onboarding and implementation, making it accessible even for developers new to reverse proxy concepts. Finally, YARP and its documentation are open source, inviting contributions, peer reviews, and community feedback, which drives continuous improvement.

Now that we understand YARP and its capabilities, let's implement it in our project. We will continue to develop using our project and solution from the previous chapter. Start by adding a new web project and adding it to the solution:

```
dotnet new webapi -n Gateway.Yarp
dotnet sln add Gateway.Yarp/Gateway.Yarp.csproj
```

Then, we can go into the new project folder and add the `Yarp.ReverseProxy` package:

```
cd Gateway.Yarp
dotnet add package Yarp.ReverseProxy
```

Now, we must register the YARP service configurations and middleware in the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy().LoadFromConfig
    (builder.Configuration.GetSection("ReverseProxy"));
var app = builder.Build();
app.UseAuthentication();
app.MapReverseProxy();
app.Run();
```

`MapReverseProxy()` adds the proxy pipeline as the app's root. We are loading the configuration settings from the `appsettings.json` file, so we need to add the following:

```
{
  "ReverseProxy": {
    "Routes": {
      "appointmentsRoute": {
        "ClusterId": "appointmentsClu",
        "Match": { "Path": "/appointments/{**catch-all}" }
      },
      "documentsRoute": {
        "ClusterId": "documentsClu",
        "Match": { "Path": "/documents/{**catch-all}" }
      },
      "patientsRoute": {
        "ClusterId": "patientsClu",
        "Match": { "Path": "/patients/{**catch-all}" }
      },
      "doctorsRoute": {
```

```
    "ClusterId": "doctorsClu",
    "Match": { "Path": "/doctors/{**catch-all}" }
  },
  "Clusters": {
    "appointmentsClu": {
      "Destinations": {
        "d1": { "Address": "https://appointments-svc/" }
      }
    },
    "documentsClu": {
      "Destinations": {
        "d1": { "Address": "https://documents-svc/" }
      }
    },
    "patientsClu": {
      "Destinations": {
        "d1": { "Address": "https://patients-svc/" }
      }
    },
    "doctorsClu": {
      "Destinations": {
        "d1": { "Address": "https://doctors-svc/" }
      }
    }
  }
}
```

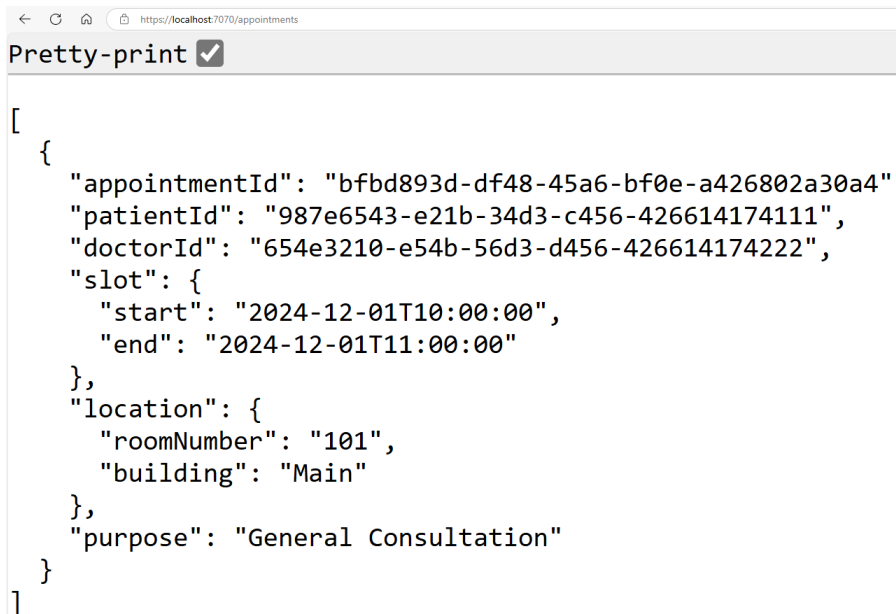
The Routes section of the configuration defines an unordered set of route definitions, each mapping an incoming request to one or more backend service instances. Every route must specify a RouteId value, a unique identifier, and a ClusterId value, which links the route to a corresponding entry in the Clusters section.

It must also include a Match element, either a list of hostnames or a path pattern expressed using the familiar ASP.NET Core route template syntax. When the proxy receives a request, it evaluates these matches in order of specificity; the most specific template wins unless an explicit Order value is supplied, in which case lower numbers take precedence.

Each route can be further enriched with policies such as custom headers, authorization requirements, CORS rules, and other cross-cutting concerns exposed through the `RouteConfig` options. Once a request satisfies the matching criteria, the proxy applies the declared policies and forwards the call to the cluster referenced by `ClusterId`.

The `Clusters` section, in turn, contains an unordered collection of named clusters. A cluster represents a logical upstream and primarily holds a set of named destinations, storing individual service instances and their addresses, any of which is eligible to handle traffic for the associated routes. During request processing, YARP combines the route's rules with the cluster's configuration to select an appropriate destination, completing the hand-off from edge to microservice.

To confirm the configuration of the addresses, you may replace the cluster config address for the appointments API with the actual launch address from the related project. For example, you may end up with `https://localhost:7206/api`. Now, you can run the appointments and YARP gateway projects using `dotnet run` in their respective directories. Send a request to the gateway's base URL, appended with `/appointments`, using a browser or an appropriate API testing tool. *Figure 11.3* shows the outcome of making this request using a standard browser.



*Figure 11.3 – The result of using the YARP gateway address to route to the appointments API endpoint*

Now, we can see that using YARP can provide a more native reverse proxy and complete control over the routes and request forwarding. This is good, but not great. We cannot perform certain transformations and fulfill some security requirements, such as token forwarding. We require API gateway technology developed specifically for these purposes. Next, we will examine Ocelot, a .NET library that provides additional API gateway features.

## Implementing an API gateway using Ocelot

**Ocelot** is an open source API gateway developed on the .NET Core platform. It is a simple gateway implementation that unifies communication with microservices through abstraction, as we expect from a gateway. The Ocelot API gateway transforms incoming HTTP requests and forwards them to the appropriate microservice address based on preset configurations.

It is a popular API gateway technology that can be easily integrated into a .NET Core application using the NuGet package manager. Its configurations can be outlined in JSON format, and here, we define **upstream** and **downstream** routes. *Upstream* routes refer to the service addresses exposed to the client, while *downstream* routes are the actual addresses of the mapped microservices. We can also define the allowed protocols for each upstream service route, allowing for robust control over the type of traffic we are willing to accept on a route.

Ocelot is well documented and extendable. It has built-in features such as the following:

- **Built-in cache management:** Ocelot's cache management lets you store responses from downstream services in memory (or a distributed cache) so that identical requests don't always incur the full round-trip.
- **Rate limiter:** Rate limiting protects downstream services by throttling the number of requests each client (or IP, or API key, depending on your policy) can make over a given time window.
- **Support for native .NET Core logging integrations:** Ocelot integrates with the standard Microsoft logging framework. That means any logging provider you already use in your .NET Core apps works out of the box.
- **Support for JWT authentication:** JWT authentication lets Ocelot validate tokens issued by your identity provider (e.g., IdentityServer4, Auth0, or Microsoft Entra ID) before forwarding requests downstream.
- **Retry and circuit breaker policies (using Polly):** Resiliency patterns are critical when dealing with flaky networks or unstable services.



- **Aggregating:** Aggregation lets you combine multiple downstream calls into a single response.
- **Request transformations:** Ocelot can manipulate both the incoming request before it's sent downstream and the outgoing response before it's returned to the client.

Let us set up an Ocelot API gateway application together. Start by creating a new ASP.NET Core API project and adding it to the existing solution file:

```
dotnet new webapi -n Gateway.Ocelot
dotnet sln add Gateway.Ocelot/Gateway.Ocelot.csproj
```

Then, we can go into the new project folder and add the Ocelot package:

```
dotnet add package Ocelot
dotnet add package Ocelot.Provider.Polly
```

Now that we have our package installed, we must begin outlining our routing configurations. We can create a new configuration file called `ocelot.json`. In this JSON file, we will outline all *upstream* and *downstream* routes. This configuration will look something like this:

```
{
  "Routes": [
    {
      "Key": "appointmentsRoute",
      "DownstreamPathTemplate": "/api/v1/appointments/{everything}",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost:7206",
          "Port": 443
        }
      ],
      "UpstreamPathTemplate": "/appointments/{everything}",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
    },
    {
      "Key": "patientsRoute",
      "DownstreamPathTemplate": "/api/v1/patients/{everything}",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
```

```
{
  "Host": "patients-svc",
  "Port": 443
},
{
  "UpstreamPathTemplate": "/patients/{everything}",
  "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
},
// Other service configurations
{
  "Key": "appointmentSummary",
  "UpstreamPathTemplate": "/portal/appointments/{appointmentId}",
  "UpstreamHttpMethod": [ "GET" ],
  "Aggregator": "AppointmentSummaryAggregator"
},
{
  "Aggregates": [
    {
      "Aggregator": "AppointmentSummaryAggregator",
      "RouteKeys": [
        "appointmentsRoute",
        "patientsRoute",
        "doctorsRoute"
      ]
    }
  ]
},
{
  "GlobalConfiguration": {
    "BaseUrl": "https://gateway.microservices-health.com",
    "RequestIdKey": "x-correlation-id",
    "QoSOptions": {
      "ExceptionsAllowedBeforeBreaking": 3,
      "DurationOfBreak": 3000,
      "TimeoutValue": 10000
    }
  }
}
```

This configuration file is straightforward. Once we understand the pattern, we can extend it as needed for the rest of our services. The sections are explained here:

- **Routes:** This is the parent section of our JSON configuration, where we define the upstream and downstream configurations.
- **DownstreamPathTemplate:** This section outlines the address at which the microservice can be found.
- **DownstreamScheme:** This outlines the protocols that we will use to communicate with the microservice being defined.
- **DownstreamHostAndPorts:** The host address and port are defined in this section.
- **UpstreamPathTemplate:** We outline the path that we expose to the client apps. By calling this defined route, Ocelot will automatically reroute the request to the service specified in **DownstreamPathTemplate**. You can also notice in the previous example that we can rename the route if necessary. The Customers API endpoint, initially found in the downstream API, can only be reached via a Patients endpoint address.
- **UpstreamHttpMethod:** Here, we define the methods we will accept as legitimate requests from a client.

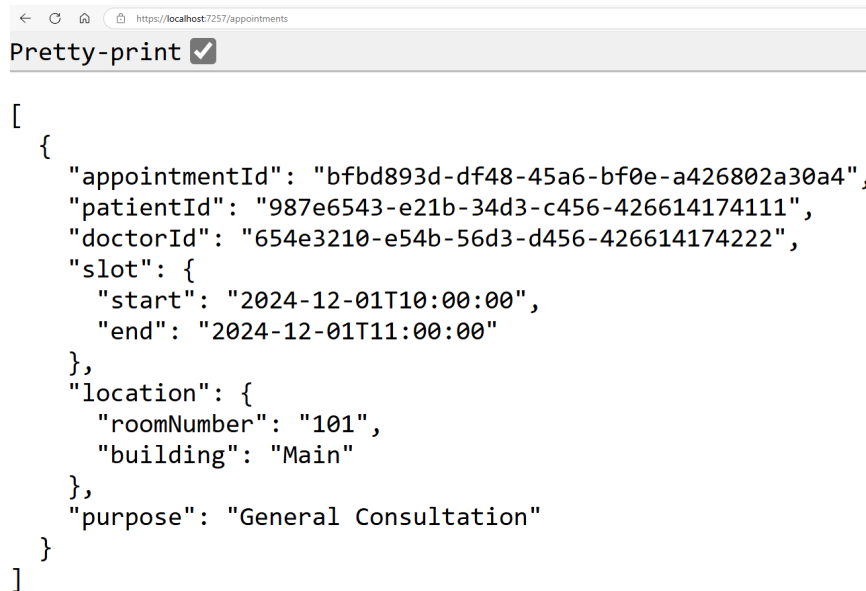
The **QoSOptions** block wires Ocelot's routes and aggregates them into **Polly-powered resilience policies**. It allows you to specify how the gateway should behave when downstream services slow down or fail. The three properties shown enable a **circuit breaker** plus a **per-request timeout**:

- **ExceptionsAllowedBeforeBreaking:** How many consecutive *handled exceptions* (network failures, 5xx responses, or timeouts) will Ocelot tolerate per destination before it "trips" the breaker? Based on the configuration, after the third failure in a row, the circuit for that downstream host is switched to **Open**; subsequent calls are short-circuited immediately, sparing clients additional latency and further load on the backend.
- **DurationOfBreak:** How long does the circuit remain **Open** before Polly moves it to **Half-Open** and allows a test request to probe recovery? This is measured in milliseconds, so in our configuration of 3000, the gateway waits three seconds, then lets one request through. If it succeeds, the circuit resets to **Closed** (regular operation); if it fails, the breaker reopens, and the timer restarts.
- **TimeoutValue:** This is a hard deadline per request. If a downstream call hasn't returned within this window, Ocelot cancels the HTTP task and registers it as a failure. Any request that takes more than 10 seconds is aborted and counted toward **ExceptionsAllowedBeforeBreaking**, so the circuit trips quickly when a service is sluggish rather than only when it is down.

We can modify `Program.cs` by adding these lines to add the `ocelot.json` file to our global configuration at application startup. Then, we register the Ocelot service and middleware:

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;
using Ocelot.Provider.Polly;
var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddJsonFile("ocelot.json", optional: false,
    reloadOnChange: true);
builder.Services.AddOcelot(builder.Configuration)
    .AddPolly();
var app = builder.Build();
await app.UseOcelot();
app.UseHttpsRedirection();
app.Run();
```

With these few configurations, we can now use the gateway URL as the API URL in our client apps. Let us attempt to retrieve the appointment records using the API gateway URL. We can browse the base URL and append `/appointments`, using a browser or API testing tool. *Figure 11.4* shows the expected result after using a browser.



*Figure 11.4 – Shows the result of Ocelot forwarding a request to the downstream appointments service*

Since we are already familiar with building an aggregator service, let us explore how different this experience would be if we used Ocelot for the implementation. We have already made provisions for aggregations in the `ocelot.json` configuration. The first part of the configuration defines an endpoint for the aggregation, and then we define the routes to be included in the aggregation logic. Now, we need to add the business logic that governs the aggregation. The following class contains this logic:

```
using Gateway.Ocelot.Models;
using Newtonsoft.Json;
using Ocelot.Middleware;
using Ocelot.Multiplexer;
using System.Net;
using System.Text;
namespace Gateway.Ocelot.Aggregators;
public sealed class AppointmentSummaryAggregator : IDefinedAggregator
{
    public async Task<DownstreamResponse>
        Aggregate(List<HttpContext> contexts)
    {
        var appointment = await contexts[0].Items.DownstreamResponse()
            .Content.ReadFromJsonAsync<AppointmentDetailsDto>();
        // Service calls for Doctor and Patient info
        var summary = new
        {
            appointment.AppointmentId,
            appointment.StartTime,
            Doctor = new { doctor.DoctorId, doctor.FirstName,
                doctor.LastName, doctor.Specialty },
            Patient = new { patient.PatientId, patient.FirstName,
                patient.LastName, patient.Gender }
        };
        var json = JsonConvert.SerializeObject(summary);
        var headers = new List<Header> { new("Content-Type",
            ["application/json"]) };
        var httpContent = new StringContent(json, Encoding.UTF8,
            "application/json");
        return new DownstreamResponse(httpContent,
            HttpStatusCode.OK,
```

```
headers,  
"OK");  
}  
}
```

When a request is sent to the aggregator URL, Ocelot converts the single request into multiple **parallel** downstream calls and then combines the results, without the caller being aware of the internal fan-out. If a downstream call fails *before* your aggregator runs, Ocelot injects a failed response object with the corresponding status code. The aggregator decides whether to return partial data or a fallback or to propagate the error.

This exercise has led us to create a versatile and resilient API gateway. We combined some of the resilience planning techniques of the previous chapter with aggregation techniques from *Anchor 3*. Given Ocelot's flexibility, we can see where gateway rules can be fine-tuned to meet the needs of the downstream and upstream services. This versatility comes in handy when planning for different frontend application requirements. This results in different backend configurations for each frontend. Let's review this concept next.

## Backend for Frontend pattern

As microservices grow and serve diverse clients, such as mobile apps, web applications, and third-party integrations, a single, generic API may not effectively meet every consumer's specific needs. This is where the BFF pattern becomes a crucial design strategy. The BFF pattern proposes creating *separate backend services* explicitly tailored to the needs of different frontend clients. Instead of a universal API that forces clients to adapt to a one-size-fits-all contract, each frontend application interacts with a dedicated BFF service, designed around its unique requirements.

Before we review how BFF works, let's take a moment to appreciate some of the challenges that implementing it can address.

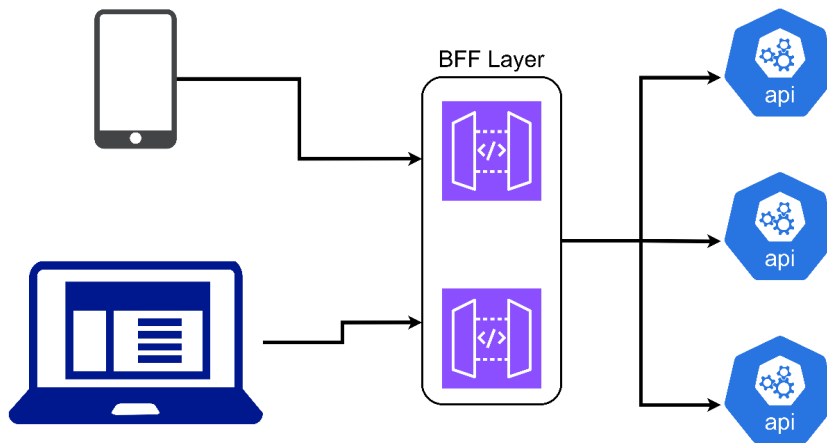
## Challenges of implementing microservices for multiple devices

Microservices naturally promote the separation of concerns at the business and domain levels. However, when different frontends must consume these services, several challenges arise:

- **Over-fetching and under-fetching:** Tailor BFF APIs to deliver only the data each frontend requires, reducing bandwidth and parsing complexity
- **UI-specific aggregations:** BFFs orchestrate multiple microservice calls, aggregating and formatting data for the client in the optimal structure

- **Authentication complexity:** Centralized session management and token handling inside the BFF, simplifying the frontend
- **Frontend-driven business logic leakage:** Encapsulate UI-specific adaptations in the BFF instead of polluting microservices or the client app
- **Versioning and change management:** Independently evolve BFFs per frontend, minimizing impact across unrelated consumers

The BFF pattern enables us to provide a service-per-device API approach by precisely defining our API functionality based on the user experience we aim for a user to have on a particular UI. This makes it easier for us to develop, maintain, and adjust our API based on the client's requirements and simplifies the process of delivering functionality across multiple clients. *Figure 11.5* shows a BFF setup.



*Figure 11.5 – Each client app references an endpoint to a gateway specially configured to optimize API traffic for the target device type*

We can use multiple implementations of Ocelot to implement the BFF pattern. Docker is an excellent candidate for hosting multiple Ocelot instances, and we can outline specific configuration files per gateway instance. This approach enables us to use a single base instance of the gateway implementation while spawning specific instances relative to the required BFF instance.

Let us review how this pattern can be implemented using Ocelot.

## BFF pattern using Ocelot

We have already seen that we can configure Ocelot as our API gateway. We can further enhance our implementation by deploying our Ocelot app in Docker images that match our desired BFF

instance. In short, we can mount several instances of Ocelot, each with its configuration file. We can provide each Docker container containing an Ocelot instance with its own set of rules by having several configuration files.

We will containerize the `Gateway.Ocelot` project and spin up multiple BFF instances (e.g., one for web and one for mobile) using **Docker Compose**. Start by adding a Dockerfile to `Ocelot.Gateway`:

```
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081

FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["Gateway.Ocelot/Gateway.Ocelot.csproj", "Gateway.Ocelot/"]
RUN dotnet restore "./Gateway.Ocelot/Gateway.Ocelot.csproj"
COPY . .
WORKDIR "/src/Gateway.Ocelot"
RUN dotnet build "./Gateway.Ocelot.csproj" -c $BUILD_CONFIGURATION
    -o /app/build

FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./Gateway.Ocelot.csproj" -c $BUILD_CONFIGURATION
    -o /app/publish /p:UseAppHost=false

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "Gateway.Ocelot.dll"]
```

We will discuss Docker in more detail in *Anchor 3, Container Hosting and Development Patterns*, so I will spare the exact details of what each line of this file does. This Dockerfile uses a four-stage build to produce a lean, runnable Ocelot gateway image on .NET 9. It builds the web project and then publishes the build files to an app directory accessed by the container.



Now, we can copy the existing `ocelot.json` to two files, one for each frontend. You can tweak each file as needed, adding different aggregators, rate limits, or multiple route subsets. At the **solution root**, add a `docker-compose.yml` file that builds the image once and launches two services:

```
version: "3.8"
services:
  web-bff:
    image: healthcare/ocelotgw:latest
    build:
      context: ./Chapter 11/Gateway.Ocelot
    container_name: web-bff
    environment:
      - ASPNETCORE_ENVIRONMENT=Production # or Development
    ports:
      - "8080:80" # expose Web BFF
    volumes:
      - ./Chapter 11/Gateway.Ocelot/ocelot.WebPortal.json:/app/ocelot.json

  mobile-bff:
    image: healthcare/ocelotgw:latest
    build:
      context: ./Chapter 11/Gateway.Ocelot
    container_name: mobile-bff
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    ports:
      - "8081:80" # expose Mobile BFF
    volumes:
      - ./Chapter
        11/Gateway.Ocelot/ocelot.MobilePortal.json:/app/ocelot.json

  appointments-svc:
    image: appointments-api:latest
    # ... your existing service definition

  patients-svc:
    image: patients-api:latest
    # ...
```

```
doctors-svc:  
  image: doctors-api:latest  
# ...
```

The BFF pattern provides a powerful way to streamline frontend-backend communication in .NET microservices applications. By building dedicated, lightweight backend services tailored to specific client needs, you achieve faster, more maintainable, and more secure interactions across your system.

One of the key responsibilities of an API gateway is to act as a central enforcement point for **security** and **traffic control policies**. This includes managing authentication, authorization, and rate limiting, all of which are critical for protecting downstream services from abuse, unauthorized access, and overload. In the next section, we explore how these features can be implemented in both YARP and Ocelot.

## Security and rate limiting in API gateways

API security is not a feature; it is a *design requirement*. In microservices architecture, APIs form the connective tissue of your architecture, and any gap in their security can become a high-impact vulnerability. APIs act as both an access layer and a trust boundary between systems; therefore, securing APIs is not optional.

One of the core security responsibilities of an API is to ensure that only legitimate users and systems can access specific endpoints and perform authorized actions. API security frameworks typically rely on protocols such as OAuth 2.0 and OpenID Connect, which often use JWTs to enforce user identity and role-based permissions. This emphasizes JWT as a token format used by the protocols. When we explore API security in more detail in *Anchor 4, Securing Microservices*, we will examine how to configure an API gateway as a secure access point to downstream services.

Two of the most effective security measures are to enforce access control through API keys and JWT-based authentication. These two techniques offer distinct forms of protection and can be used in combination, depending on the application's security requirements. API keys are simple tokens, typically passed via headers or query strings, that authenticate a client to the gateway. They are best used for server-to-server communication or simple applications where token-based authentication is not feasible. JWTs, on the other hand, are compact, self-contained tokens used to represent user identity and claims, commonly employed in user-facing systems leveraging OAuth 2.0 or OpenID Connect.

When we introduce an API gateway, it becomes the ideal chokepoint to enforce **security** and **rate limiting**, centralizing these cross-cutting concerns and relieving downstream services from repetitive implementations. Let us review how we can harden our API gateway.

## Implementing API gateway security

ASP.NET Core offers a comprehensive set of built-in security mechanisms that enable developers to secure their web APIs, Razor Pages, MVC applications, and microservices with ease, reducing reliance on third-party tooling. These features are designed to address common threats, including unauthorized access, **cross-site request forgery (CSRF)**, data tampering, and **man-in-the-middle (MITM)** attacks. By leveraging these out-of-the-box mechanisms, we can implement strong authentication, authorization, and data protection policies across their application architecture with minimal friction.

ASP.NET Core provides a flexible **authentication** system that supports various identity providers and token-based schemes out of the box. The framework defines a pluggable authentication pipeline, allowing developers to register one or more authentication handlers. The **authorization** system complements authentication by enforcing access control rules based on user identity, roles, claims, or custom policies. Authorization can be applied globally, per controller, or per action.

Since YARP is a middleware-driven reverse proxy built with ASP.NET Core, this means all its routing behavior can be enhanced using the pipeline of ASP.NET Core's built-in authentication and authorization features. This makes it straightforward to add API key validation or JWT validation before proxying requests to downstream services.

To validate API keys in a YARP gateway, you can implement custom middleware that inspects the request headers and rejects any request missing a valid key. Start by adding a static key validation middleware in your Program.cs:

```
// Service registrations
var app = builder.Build();
// Custom middleware to validate API key
app.Use(async (context, next) =>
{
    const string apiKeyHeader = "X-API-Key";
    var validApiKey = builder.Configuration["ApiKey"]; // Load from
    // appsettings or environment
    if (!context.Request.Headers.TryGetValue(apiKeyHeader,
        out var providedKey) ||
```

```
        providedKey != validApiKey)
    {
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.Response.WriteAsync("Unauthorized: API key is
            missing or invalid.");
        return;
    }
    await next.Invoke();
});
app.MapReverseProxy();
app.Run();
```

This modification to the middleware intercepts the request before it is forwarded to the proxy routes and ensures the presence of a valid API key. The route is evaluated to determine the intended destination of the incoming request, and then the appropriate API key is loaded from the configuration. The modified configuration file could look something like this:

```
{
  "ApiKeys": {
    "appointments": "key-for-appointments",
    "patients": "key-for-patients",
    "doctors": "key-for-doctors"
  },
  ...
}
```

This example is provided for simplicity, but you should store the API key value securely in environment variables or a secrets manager. Only requests that pass the API key check are forwarded to the downstream services.

To enforce JWT validation at the YARP gateway, the same ASP.NET Core authentication infrastructure is used. JWTs are typically issued by an identity provider (e.g., IdentityServer, Microsoft Entra ID, or Auth0) and must be validated for integrity, the issuer, and claims. Once again, I will spare you the details of JWT authentication until *Anchor 3, Securing Microservices*, where we delve into the topic in more detail. For now, we will examine some high-level concepts of implementing JWT authentication using the YARP API gateway.

First, you need to add a package to support JWT authentication using this command:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Then, you need to configure JWT authentication in your gateway's `Program.cs`:

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.Authority = "https://auth.healthmanagement.com";
        options.Audience = "api-gateway";
        options.RequireHttpsMetadata = true;
    });
builder.Services.AddAuthorization();
builder.Services.AddReverseProxy().LoadFromConfig
    (builder.Configuration.GetSection("ReverseProxy"));
```

Then, you wire the authentication and authorization middleware into the pipeline before `MapReverseProxy()`:

```
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapReverseProxy();
app.Run();
```

With this configuration in place, every request to the gateway will be authenticated using a JWT. The token must be included in the authorization header as a bearer token (i.e., `Authorization: Bearer <token>`). If the token is missing, expired, or malformed, or lacks the required audience or claims, the request will be rejected with a 401 or 403 status code, and the downstream service will not be contacted.

Ocelot simplifies security management by embedding authentication and authorization configuration directly in the `ocelot.json` file. This configuration enables easy declaration of which upstream endpoints require tokens and the necessary scopes or claims to access them.

Since we are using the out-of-the-box ASP.NET Core security configurations, we can reuse the API key and JWT configurations outlined in `Program.cs`. The `appsettings.json` file, however, will look significantly different for JWT authentication rules. The configuration will look like this:

```
{
  "Routes": [
    {
      "Key": "appointmentsRoute",
```

```

    "DownstreamPathTemplate": "/api/v1/appointments/{everything}",
    "DownstreamScheme": "https",
    "DownstreamHostAndPorts": [
      { "Host": "localhost", "Port": 7206 }
    ],
    "UpstreamPathTemplate": "/appointments/{everything}",
    "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ],
    "AuthenticationOptions": {
      "AuthenticationProviderKey": "Bearer",
      "AllowedScopes": [ "appointments.read", "appointments.write" ]
    }
  }
]
}

```

This configuration ensures that any request to `/appointments` must include a valid JWT bearer token containing one of the required scopes. If the token is missing, expired, or invalid, the request is rejected with a 401 Unauthorized or 403 Forbidden response, and it does not reach the downstream service.

The `Program.cs` file will look like this:

```

var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddJsonFile("ocelot.json", optional: false,
    reloadOnChange: true);
builder.Services.AddOcelot(builder.Configuration)
    .AddPolly();
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
await app.UseOcelot();

app.UseHttpsRedirection();
app.Run();

```

By default, Ocelot will forward the incoming `Authorization` header to downstream services. This behavior allows microservices to perform additional authorization checks if needed or log user activity for audit purposes. If a downstream service does not require the token, or if you need to transform or remove it, you can configure header transformations within Ocelot's routing rules.

## Implementing rate limiting in API gateways

**Rate limiting** is a technique used to control the number of requests a client can make to an API or service within a specific period. It serves as a traffic regulation mechanism to ensure fair resource usage, protect backend systems from overload, and mitigate abuse or DoS attacks. In modern microservices and distributed systems, rate limiting is a critical part of API design and security strategy.

At a high level, rate limiting involves defining **limits** and **time windows** and applying them to specific users, IP addresses, or API keys. These limits determine the number of requests allowed and when additional requests should be blocked or delayed. The most common rate-limiting algorithms include the following:

- **Fixed window:** In the fixed window algorithm, the system tracks the number of requests in a fixed interval (e.g., one minute). Once the limit is exceeded, all subsequent requests within that window are rejected.
- **Sliding window:** Sliding window counters provide a more accurate view of request rates by using rolling intervals. Instead of resetting the counter at the start of each minute, the window moves with time, giving a smoother rate enforcement.
- **Token bucket:** The token bucket algorithm allows requests if there are tokens in the bucket. Tokens are added at a fixed rate. If the bucket runs out, requests are delayed or denied. This approach supports burst traffic better than fixed windows.
- **Leaky bucket:** This algorithm smooths traffic by processing requests at a fixed rate, regardless of burst size. Excess requests are queued or dropped if the buffer fills.

When a client exceeds the rate limit, the server should respond with an appropriate status code and message:

- **HTTP 429 Too Many Requests:** A standard response indicating that the user has reached the rate limit
- **Retry-After header:** Indicates when the client can attempt the request again

Although not part of the core framework, ASP.NET Core enables developers to easily integrate rate limiting using middleware or third-party packages, such as `AspNetCoreRateLimit`. Let us add this package using the following command:

```
dotnet add package AspNetCoreRateLimit
```

The configuration block shown (`IpRateLimiting`) is specifically for `AspNetCoreRateLimit`, not YARP. While YARP is an API gateway, `AspNetCoreRateLimit` is a separate middleware that can be used with YARP (or any ASP.NET Core application). Still, it's not inherent to YARP's configuration for rate limiting. YARP has native rate-limiting capabilities.

To implement IP-based rate limiting using the `AspNetCoreRateLimit` library, we can add the following configuration section to the `appsettings.json` file:

```
"IpRateLimiting": {  
  "EnableEndpointRateLimiting": true,  
  "StackBlockedRequests": false,  
  "RealIpHeader": "X-Forwarded-For",  
  "GeneralRules": [  
    {  
      "Endpoint": "*/appointments/*",  
      "Period": "1s",  
      "Limit": 10  
    },  
    ...  
  ]  
}
```

It defines IP-based rate-limiting rules, which restrict how frequently a client (identified by IP address) can access specific API endpoints within specified time intervals.

`"EnableEndpointRateLimiting": true` enables per-endpoint rate limiting. Instead of applying a global limit to all requests, the system evaluates the limits individually for each API endpoint specified in the *General Rules* section. This allows you to apply stricter or looser rate limits depending on the sensitivity or performance impact of each API path.

`"StackBlockedRequests": false` determines how blocked requests are counted. When set to false, the system does not count blocked requests against the rate limit window. This helps prevent cascading rejections from causing extended lockouts.

If this were set to true, blocked requests would continue to be counted, potentially extending the denial period.



"RealIpHeader": "X-Forwarded-For" instructs the rate limiter to extract the client's IP address from the X-Forwarded-For HTTP header. This is necessary when the application is running behind a reverse proxy or load balancer, which may forward requests on behalf of the original client's IP address. Without this setting, the server might incorrectly use the proxy's IP address for rate tracking.

"GeneralRules": [ ... ] contains individual rate limit rules per endpoint. Each rule applies to a specified API route pattern and defines a time window and a maximum number of requests that are allowed within that window per IP address.

Finally, we need to modify the Program.cs file to register the rate-limiting service and middleware:

```
builder.Services.AddMemoryCache();
builder.Services.Configure<IpRateLimitOptions>
    (builder.Configuration.GetSection("IpRateLimiting"));
builder.Services.AddInMemoryRateLimiting();
builder.Services.AddSingleton<IRateLimitConfiguration,
    RateLimitConfiguration>();
var app = builder.Build();
app.UseIpRateLimiting();
app.MapReverseProxy();
```

One notable service being included is the memory cache. The rate-limiting middleware in ASP.NET Core requires caching to store and update metadata about the number of requests each client (typically identified by IP address, API key, or token) has made to specific endpoints within defined time windows. Without caching, the middleware would have no way of remembering how many requests a client has already made and therefore couldn't enforce limits.

On the other hand, Ocelot provides **built-in rate-limiting** capabilities via the RateLimitOptions configuration in ocelot.json. Here's how to use it:

```
{
  "Routes": [
    {
      "Key": "limitedPatientsRoute",
      "DownstreamPathTemplate": "/api/v1/patients/{everything}",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [{ "Host": "patients-svc", "Port": 443 }],
      "UpstreamPathTemplate": "/patients/{everything}",
      "UpstreamHttpMethod": [ "GET" ],
```

```
    "RateLimitOptions": {  
      "EnableRateLimiting": true,  
      "Period": "10s",  
      "Limit": 5  
    }  
  }  
]  
}
```

This configuration establishes a route named `limitedPatientsRoute`, which serves as a proxy to a downstream microservice responsible for patient data. `DownstreamPathTemplate` indicates that any request matching this route will be forwarded to the `/api/v1/patients/{everything}` downstream path. The `{everything}` token is a wildcard that captures any remaining portion of the URL, allowing for dynamic routing. These will be rewritten accordingly and forwarded to the appropriate endpoint on the downstream service.

`UpstreamPathTemplate` defines the route that external clients will use. When a client makes a GET request to `/patients/{anything}`, Ocelot interprets the URL, matches it to this route, and forwards the call to the correct downstream address. The `UpstreamHttpMethod` array explicitly restricts the route to accept only HTTP GET requests. Any attempt to access this path using POST, PUT, or DELETE will be rejected by the gateway, thereby reinforcing method-level API governance.

The most critical part of this configuration lies in the `RateLimitOptions` section. Rate limiting is enabled for this route, and it is configured to allow a maximum of five requests per client every 10 seconds. If a client attempts to exceed this threshold, Ocelot will immediately respond with a 429 Too Many Requests status code, without forwarding the request to the downstream service.

Now that we understand how we can implement security and rate limiting in API gateways, let's review what we have learned in this chapter.

## Summary

In this chapter, we dove deep into the critical role of API gateways and the BFF pattern in modern microservices architectures. First, we revisited why building a microservices application can quickly become overwhelming, with clients needing to track multiple API endpoints. To address this, we can introduce an API gateway to simplify client interaction by exposing a single, unified API while handling complex backend orchestration, security, logging, and aggregation concerns.

We explored what an API gateway is and that it does more than just route traffic. It can centralize cross-cutting concerns, such as authentication, rate limiting, caching, monitoring, and service discovery. This makes the client app much simpler and more maintainable because it only needs to interact with one gateway, rather than needing to know the details of every microservice.

As helpful as API gateways are, they aren't a silver bullet. If they aren't properly designed for redundancy, they can become single points of failure. If overused, they can also risk creating tight coupling or vendor lock-in. These downsides underscored the need for meticulous architectural and operational planning.

Selecting the right technology is essential when implementing API gateways. We need a tool that allows us to define explicit routing and aggregation rules, optimize for performance, and ensure strong observability. We also discussed the dangers of building thick gateways (overloaded with business logic) and how, ideally, a gateway should stay thin, mainly dealing with routing and policy enforcement.

We examined reverse proxies and API gateways, highlighting their similarities and differences. We explored how reverse proxies, such as NGINX and HAProxy, and newer .NET native solutions, such as YARP, fit into this story. YARP is a flexible, middleware-based solution that makes implementing thin, fast, programmable proxies in .NET applications easy. We also explored implementing YARP in a real-world project by setting up routes and clusters to forward traffic to the various microservices.

While YARP provided us with raw routing power, we realized that sometimes we need more advanced forwarding and manipulation features, and that's where **Ocelot** comes in. Ocelot is a full-fledged .NET API gateway that supports caching, authentication, request aggregation, rate limiting, retry, and circuit breaker policies. Setting up Ocelot involved creating a project, defining upstream and downstream routes in an `ocelot.json` file, and wiring it into the application pipeline. We implemented an example aggregator with Ocelot, showing how multiple service responses could be combined into a single response for the client.

Finally, we explored the BFF pattern. This pattern becomes particularly useful when client applications, such as web apps, mobile apps, and third-party systems, often have unique data and interaction requirements. Instead of trying to create one API that fits everyone poorly, the BFF pattern creates a specialized backend for each client type. Each BFF acts like a mini-gateway, optimized for its frontend.

We experimented with using Docker to create multiple Ocelot instances. We used Docker Compose to spin up two BFF containers, one for the web portal and one for the mobile app, each with its routing configuration mounted via volumes. This flexible architecture allows teams to evolve frontend experiences independently without impacting others.

Finally, we examined the significance of API security and rate limiting. Security is not optional in microservices. We reinforced all our gateways using ASP.NET Core's out-of-the-box authentication and authorization capabilities. That, coupled with rate limiting, ensures downstream services are protected from abuse or unintended high-frequency calls.

This chapter provided practical, real-world strategies for controlling complexity in microservices communication. Using API gateways, reverse proxies such as YARP, and full BFF patterns with Ocelot and Docker, you can now design simpler, more resilient, and easier-to-maintain systems for clients as they grow. You are now much better equipped to select and implement the right communication patterns based on the specific needs of your microservices and client applications.

Now that we have seen how to implement unique backends for different clients, the next chapter will focus on developing various frontends using other frameworks and technologies to suit each microservice.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require one.*

UNLOCK NOW





# 12

## Micro Frontends — Extending Microservices to the Frontend

Having explored API gateways as unified access points for backend services, we'll now shift our attention to the frontend, where the user experience is ultimately delivered. This chapter examines how frontend applications can evolve alongside microservices, leveraging micro frontends to enhance scalability, flexibility, and maintainability.

There are situations where, despite backend services needing to collaborate and share data, the frontend experience must differ based on business domains or team boundaries. This necessitates a unified frontend shell integrating multiple services or entirely separate user interfaces tailored for each microservice. Micro frontends enable us to achieve this, supporting team autonomy while maintaining a cohesive user experience.

Modern web applications continue to grow in complexity, driven by increasing user expectations, rapid feature delivery, and diversified platform requirements. Traditional monolithic frontend architectures often struggle to meet these demands, resulting in slower deployments, tightly coupled code bases, and coordination bottlenecks. Just as microservices revolutionized the backend by allowing systems to scale and evolve independently, micro frontends extend these principles to the frontend, enabling large user interfaces to be broken down into smaller, independently developed, deployed, and maintained components.

This architectural approach is especially valuable in organizations where multiple teams work on different domains or features. Each team can own its slice of the frontend and backend, adhering to DDD principles throughout the entire stack.

We will explore micro frontends, what they are, why they matter, and how they can be designed and implemented effectively. We will also discuss key topics, including UI composition techniques, performance optimization, inter-frontend communication, and state management.

By the end of this chapter, you should be able to do the following:

- Understand the core concepts and benefits of micro frontends
- Design micro frontends with the most suitable technology
- Optimize frontend performance and user experiences
- Manage state and share data between micro frontends

Let us begin by reviewing core concepts and the benefits of micro frontends.

## Technical requirements

Code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET--Second-Edition/tree/main/Chapter12>.

## Introducing micro frontends and their benefits

Before we discuss micro frontends, let's take a moment to understand what a frontend is and its responsibilities. The frontend, or **presentation layer**, is the part of the application that users directly interact with and see. It is responsible for rendering the **user interface (UI)**, handling user inputs, and orchestrating interactions between the user and the backend services. Think of the frontend as a business's storefront. While much of the inventory, logistics, and operations happen behind the scenes (in the backend), the storefront is where customers browse, interact, and ultimately make decisions. A well-designed and responsive frontend often distinguishes between user satisfaction and frustration.

Modern frontend development is built on a foundation of core web standards, supplemented by frameworks and tools that enable developers to create scalable, interactive applications. The core technologies used in web frontend development are as follows:

- **HyperText Markup Language (HTML):** HTML is the skeleton of any web page. It defines the content structure and semantic meaning. HTML allows you to create headings, paragraphs, images, links, forms, tables, and any visual element you can imagine through the power of a *canvas*.

- **Cascading Style Sheets (CSS):** CSS styles HTML elements with colors, layouts, typography, and animations. It ensures that the application functions properly and presents a polished and professional appearance. Popular CSS tools and frameworks include preprocessors such as **Syntactically Awesome Style Sheets (Sass)** or **Less**, as well as **Tailwind CSS**, **Bootstrap**, and **Material UI**.
- **JavaScript:** JavaScript enables dynamic behavior in web applications. It runs in the browser, responds to user actions, manipulates the **Document Object Model (DOM)**, and communicates with APIs. Modern JavaScript development often involves TypeScript, a typed superset of JavaScript that enhances reliability, as well as frameworks such as **React**, **Angular**, and **Vue.js**. A modern frontend application is expected to display dynamic information, typically informed by a backend.

As we have discussed in this book, the most common and basic type of software architecture is a monolith, where the presentation layer is built directly on top of the business logic and data access layers, all within a single project. This structure means the UI is tightly coupled with the backend, often sharing the same code base and deployment cycle. UI components are directly linked to backend services, making independent updates challenging, and adopting new technologies or frameworks is especially difficult, as these changes will affect the entire system.

Considering these frontend development challenges, we have adopted **service-oriented architecture (SOA)**. We will discuss this next.

## Service-oriented architecture

**Service-oriented architecture** introduces a paradigm in which the application is divided into a discrete service layer, where specific business functionalities are implemented, typically using protocols such as **HTTP/REST** or **SOAP**. This enables a more modular and loosely coupled development cycle, allowing different technologies and frameworks to be used to meet both frontend and backend needs.

Frontend applications can be developed separately, consuming services via APIs, and they can scale independently based on service load and usage patterns. Building frontends for microservices is rooted in SOA principles, where a frontend can consume one or more web services, as illustrated in *Figure 12.1*.



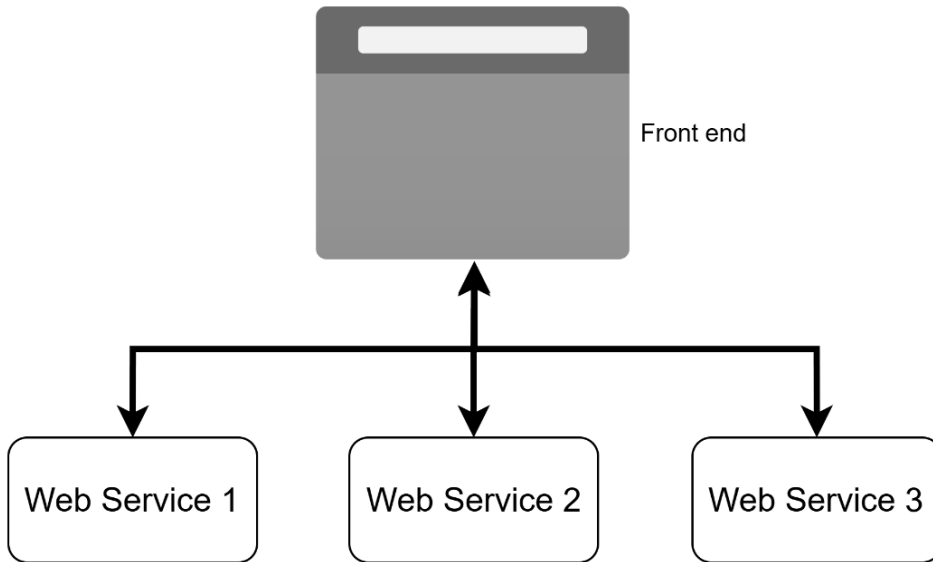


Figure 12.1 – A frontend can consume one or more microservices

Not every frontend is web-based, however. When we hear frontend, we often think of web applications immediately. However, a frontend is any user-facing interface that allows a person to interact with a software system, whether it is a website, *a mobile app*, *a desktop application*, or even *a voice assistant* or *command-line interface*.

In a distributed, service-oriented architecture, frontend clients can exist across multiple platforms, all communicating with the same backend services via APIs. The key principle is that frontends are about interaction and can occur in many contexts and through different technologies. There are situations when a particular business need or set of features is best delivered through a specific type of frontend.

This approach results in the creation of a **macrofrontend**. While the term **micro frontend** is well-established to describe independently deployable UI fragments aligned to specific business domains, the idea of a **macrofrontend** can be used informally to refer to the entire frontend layer when it is treated as a single, cohesive unit. In essence, a macro-frontend is what we get when the UI is built and deployed as a monolith, often by a central team, regardless of how modular the backend services are. This approach may still involve consuming multiple APIs or representing numerous domains, but it does so within a single application and release cycle. In contrast, a micro frontend represents a much smaller scope: it is a focused UI module owned by a specific team, developed independently, and deployed according to its own cadence.

In an application as domain-rich as our health management system, it is natural for different services to have different requirements for their user interfaces. Each micro frontend corresponds to a bounded context, such as Appointments or Patients, and is often integrated into a host shell. While a macro-frontend may be easier to build early in a project, micro frontends enable greater team autonomy, faster releases, and better scalability as the application grows in complexity. Since each service exposes its functionality through standardized APIs (REST or gRPC), the frontend needs to be flexible in its design and deployment.

We will review how DDD can help guide our frontend development strategy next.

## Micro frontends and DDD

As you may recall, **Domain-Driven Design (DDD)** encourages us to divide an enterprise system into bounded contexts, or autonomous subdomains that own their models, language, and lifecycle. When developing the backend, a **microservice** typically serves a single bounded context. In contrast, a micro frontend in the UI world carries the same responsibility: it presents the user-facing slice of that context and evolves at the same pace as the service it represents.

When every cross-functional team owns both its microservice and its micro frontend, we gain the following:

- **Team autonomy:** Each team ships the UI, API, and database migrations independently, without a “monolithic” frontend release train
- **Business alignment:** Features change in response to business needs; code and teams are organized around capabilities rather than layers
- **Technology freedom:** Teams can choose the framework that will be most productive for their bounded context (e.g., Blazor, React, Angular) while the shell integrates them seamlessly
- **Deploy-for-value:** Deployments can be more concise, and bug fixes and new releases can be rolled out quickly or rolled back without affecting the entire application

Now we see that the first pro for micro frontends is that they aid in scoping applications based on DDD requirements, and we can carve a large domain into bounded contexts, but that partitioning is only half the story. By composing these UIs thoughtfully, we ensure that the user experience stays aligned with domain goals and that the backend services remain loosely coupled with the delivery technology. As seen in *Figure 12.2*, each UI “slice” can be built, deployed, and evolved in lockstep with its corresponding service, while the overall application preserves a consistent look and feel.

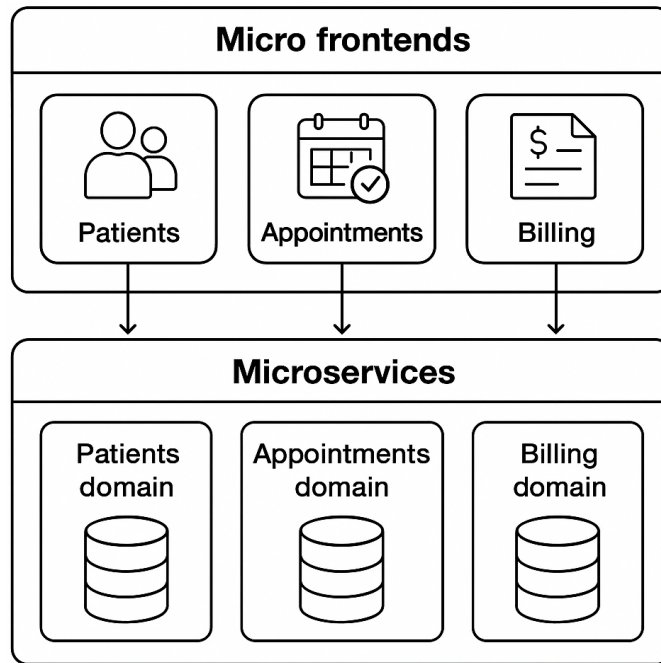


Figure 12.2 – Demonstrates how different UI slices can be built to serve each domain of the application

If we were to conceptualize a shared UI, we could think of the implementation like this:

1. Developing a shell application that provides global navigation to the various parts of the application, which represent the domains or features (e.g., Patients, Appointments, Billing), shared authentication flows, and standard design tokens (colors, typography, form styles). This shell is responsible for user login/logout, applying overall styling, and orchestrating which micro-UI to load when the user navigates to a specific domain. Under the hood, it can leverage either server-side composition (e.g., Razor Pages with embedded Blazor components) or client-side composition (a Blazor WebAssembly or React-based router that dynamically imports domain UIs).
2. Each domain has a dedicated UI, which lives in its repository or package, exposing a well-defined integration contract. That contract might be a JavaScript or WebAssembly entry point, a custom web component tag, or a server-rendered Razor component. Inside each domain UI, the team implements forms, tables, and interactive flows that consume their service's API. For the appointment scheduling UI, simple HTTP GET/POST calls (`HttpClient.GetFromJsonAsync<Appointment[]>`) can be used; however, the Document Management UI may invoke gRPC-Web streams to fetch large PDF scans efficiently.

3. Mapping UI modules to business domains also clarifies responsibility and ownership. The Patient Records team, for instance, can independently decide how to paginate record lists, cache images for offline review, or implement audit-trail overlays, without needing to coordinate with the Billing team's UI releases. Shared concerns, such as global error handling, standard authentication tokens, and telemetry instrumentation, are provided by the shell or a shared UI library, ensuring cross-cutting consistency without compromising team autonomy.

From a deployment standpoint, each domain UI can follow its release cycle. The Appointment UI may deploy hourly updates to resolve edge-case conflicts, while the Billing UI requires weekly releases following regulatory review. Using module federation or dynamic imports, the shell automatically picks up the latest version of each domain UI at runtime, ensuring that users always see the newest functionality without requiring a full-site redeploy.

Finally, beneath the surface of this modular structure, shared components (such as buttons, dialogs, and form fields) and a unified CSS framework (e.g., Bootstrap, Tailwind-based tokens, or a component library) visually unify the different UIs. This system can then be deployed either as a private NuGet package for Blazor or as an NPM package for JavaScript frameworks. By combining domain-aligned micro-UI modules with a standard design system and a flexible composition strategy, a health management application can deliver specialized experiences for patients, clinicians, and administrators, all while preserving scalability, maintainability, and team agility.

This is not the most popular pattern, but it is useful and has been implemented in several real-world situations. Let us review some of them next.

## Real-world implementations

Several real-world companies and publicly documented case studies have successfully implemented micro frontends to scale their frontend architectures across teams and domains. These implementations are highly aligned with the architectural patterns explored in this chapter on micro frontends and can further ground your discussion in practical relevance.

The first study is **Spotify**, which is often cited as a pioneer in micro frontends. It implemented a system where each feature in the Spotify web app, such as playlists, search, or user profiles, is managed by a separate team and developed as a self-contained frontend module. These modules are loaded dynamically and integrated into the main shell application. Spotify utilized this pattern to enhance feature velocity by enabling autonomous teams to deploy their components independently, experiment with various frameworks within the same application, and scale both frontend development and team ownership.

IKEA is another well-known example of transitioning from a monolithic frontend to micro frontends, allowing distributed teams to manage different parts of its e-commerce and content platform. It initially used `iframes` and later moved to a more integrated approach, using a single SPA and module federation.

Finally, DAZN adopted a micro frontend approach to enable faster delivery of product features across international markets. It developed an internal micro frontend framework to handle the lifecycle, communication, and integration between frontend components.

Now that we understand the pros and cons of developing a unified UI versus splitting the functionality across several UIs, and have seen some practical examples, let's examine some implementation ideas.

## Designing and developing micro frontends

Designing and developing micro frontends necessitates meticulous planning of domain boundaries, user interface composition strategies, inter-module communication, shared styles, and deployment models. Within a .NET ecosystem, various frontend technologies and patterns are available to facilitate the efficient implementation of micro frontends. The selection of the appropriate frontend framework significantly depends on the application's performance expectations, architectural style, platform reach, team skill set, and long-term maintainability objectives. The most prominent alternatives comprise ASP.NET Core MVC, Razor Pages, Blazor Server, Blazor **WebAssembly (WASM)**, and .NET MAUI. Each of these options possesses distinct strengths and trade-offs that warrant careful evaluation.

ASP.NET Core MVC remains a robust option for server-rendered web applications where **search engine optimization (SEO)**, accessibility, and traditional page-based navigation are prioritized. It is particularly well-suited for enterprise applications that necessitate strong back-office workflows, printable views, and demand minimal interactivity beyond forms and data grids. The **controller-view pattern** is familiar to many developers, and its long-standing presence ensures a mature ecosystem. However, MVC begins to exhibit limitations in contemporary single-page application scenarios. The management of partial updates, real-time features, or highly interactive user interfaces becomes cumbersome, often necessitating the augmentation of JavaScript and compromising the separation between server-side and client-side logic.

Razor Pages enhances the **Model-View-Controller (MVC)** framework by streamlining the architecture of page-driven applications. It integrates both controller and view logic into a single `.cshtml.cs` page model, thereby minimizing overhead for smaller applications or components. This approach is characterized by its cleanliness and efficiency, particularly in the context of forms,

as well as in **Create, Read, Update, Delete (CRUD)** operations and administrative interfaces. Nevertheless, akin to MVC, it is fundamentally server-bound and is not optimally configured for extensive client-side interactivity or offline functionalities.

Blazor Server signifies a notable transformation in web application development. It empowers developers to construct interactive single-page applications utilizing C# and Razor components, with the execution occurring on the server through **SignalR**. This architecture benefits from comprehensive access to server resources and provides rapid initial load times, as the browser receives only a minimal client interface. The programming model is both unified and robust, especially when integrating with existing .NET Core services. Nonetheless, Blazor Server applications exhibit sensitivity to network latency and scalability limitations; each user session maintains a persistent connection to the server, consequently consuming server resources. In scenarios involving high concurrent user loads or when users are connecting from unreliable networks, this model may introduce challenges.

.NET MAUI expands the reach of .NET into cross-platform development, enabling teams to create mobile and desktop applications using a shared code base. It facilitates native UI rendering for *iOS*, *Android*, *macOS*, and *Windows*, serving as an optimal solution when native performance and device integration are essential. Although robust, MAUI is not inherently oriented towards web applications. Its intended use diverges from conventional web applications and is most suitably reserved for instances where mobile or desktop delivery takes precedence.

Blazor WASM occupies a unique position in this ecosystem. It delivers client-side execution to .NET developers by compiling C# into WebAssembly, enabling fully interactive **Single-Page Applications (SPAs)** without the need for JavaScript. It facilitates the shared use of DTOs, services, validation logic, and authentication across both the backend and frontend. Developers benefit from the same tools, debugging experience, and dependency injection model they use on the server. Blazor WASM applications are inherently modular and readily support a micro frontend architecture. They can be built and deployed independently, integrate with REST or gRPC-based APIs, and even operate offline as **Progressive Web Apps (PWAs)**.

There are, of course, trade-offs. Blazor WASM's initial download size has historically been larger than that of JavaScript alternatives, though recent version improvements have mitigated this. It also lacks the vast component and plugin ecosystem that JavaScript frameworks offer, although this is steadily improving as the community and vendors invest in the platform. Additionally, since the runtime executes inside the browser, certain APIs and capabilities are either restricted or must be accessed through JavaScript interop.

Even with its challenges, Blazor WebAssembly stands out as an exciting option for modern web applications, especially when both the frontend and backend are built with .NET and your team excels at C#. It enables seamless full stack development using a single language, which significantly simplifies the process and reduces unnecessary repetition. For microservices and micro frontend architectures, Blazor WASM offers fantastic support for independent domain UI composition, static hosting, and detailed deployment pipelines. These features make it a great choice as a frontend framework for domain-driven systems, where different teams can manage their specific areas and enjoy the flexibility to deploy updates without disrupting other parts of the UI.

Ultimately, Blazor WebAssembly aligns with the same architectural principles that drive modern .NET development: modularity, separation of concerns, and scalable, service-oriented design. It empowers teams to build rich, maintainable user experiences while staying entirely within the .NET ecosystem, making it not just a viable choice, but a strategic one for organizations embracing cloud-native and distributed system paradigms.

Now that we have examined the .NET-based frontend frameworks, let us review how Blazor WASM fares against JavaScript-based alternatives.

## Blazor versus JavaScript frameworks

When deciding between Blazor WebAssembly and JavaScript-based frameworks such as **React**, **Angular**, or **Vue**, the choice often hinges on ecosystem alignment, developer skills, and architectural priorities. Blazor WASM stands out for .NET developers seeking a seamless, full stack experience using only C#. It enables teams to reuse models, validation logic, and service contracts across the client and server without the need to switch contexts to JavaScript or **TypeScript**. This is especially beneficial in organizations already invested in ASP.NET Core, as Blazor integrates smoothly with existing APIs, middleware, and identity providers.

In contrast, JavaScript frameworks possess a well-established and vast ecosystem. For instance, React and Angular enjoy substantial community backing, robust tooling, and a plethora of third-party libraries and UI components are accessible. They typically offer faster initial load times and greater compatibility with low-level browser APIs and frontend-specific workflows. This qualifies them as the strong choice for projects requiring optimized performance, access to advanced client-side features, or developers experienced with the JavaScript/TypeScript stack.

However, the trade-offs are not solely technical but also organizational. The utilization of a JavaScript framework within a .NET ecosystem frequently introduces dual-stack complexity. Teams are required to maintain distinct skill sets, construct pipelines, and manage deployment workflows,

thereby increasing cognitive and operational overhead. Blazor WASM facilitates tighter integration and shared tooling, thereby streamlining the onboarding process and reducing fragmentation across backend and frontend teams.

Nevertheless, Blazor WASM is comparatively newer, featuring a more limited component ecosystem and a performance profile that may not consistently match the raw speed of JavaScript bundles for large-scale enterprise apps. Although ongoing improvements in WebAssembly and **Ahead-of-Time (AOT)** compilation continue to mitigate this disparity, developers must carefully consider whether their application necessitates advanced user interface responsiveness or whether it would benefit more from architectural cohesion and simplicity in development.

In scenarios such as enterprise systems, internal tools, or full stack microservices platforms, particularly those utilizing gRPC, SignalR, or shared authentication, Blazor WASM frequently offers superior developer velocity and long-term maintainability. Conversely, when developing highly dynamic consumer-facing products characterized by rapid UI innovation cycles and extensive device diversity, JavaScript frameworks may present more immediate advantages. The decision, ultimately, should be predicated on the team's strengths, the application's performance and interoperability requirements, and the relative importance placed on a unified technology stack in contrast to the specific advantages of frontend power and flexibility.

We will walk through a complete example of building a Blazor WASM micro frontend that encapsulates three related bounded contexts in our Health Management System: Appointments, Doctors, and Patients.

## Creating a Blazor WebAssembly micro frontend

At this juncture, we will delineate the procedure for constructing a Blazor WASM micro frontend that encompasses three interconnected bounded contexts within our health management system: Appointments, Doctors, and Patients. Each domain signifies a distinct segment of the business, corresponding with its respective backend microservice, whereby this frontend will independently utilize each of its APIs. The culminating product will be a self-sufficient Blazor WASM application that integrates harmoniously into a broader micro frontend shell.

Let us start by creating a new Blazor WASM project using the following command:

```
dotnet new blazorwasm -o HealthPortal.Frontend --no-https
```

This command scaffolds a basic client-side Blazor application that runs entirely in the browser via WASM. Navigate into the project directory:

```
cd HealthPortal.Frontend
```



When you create a new Blazor WASM application, several files and folders are generated by default. Here's a detailed explanation of the most critical components:

- `/Pages`: This folder contains all the Razor components that serve as routeable pages. Each `.razor` file in this folder corresponds to a navigable route within the application. `Index.razor` is the default landing page of the application. The default template also includes sample components called `Counter.razor` and `Weather.razor`.

For our implementation, we can remove the sample components and replace them with the following:

- `Appointments.razor`: The component that fetches and displays appointment data
- `Doctors.razor`: The component for displaying and managing doctor information
- `Patients.razor`: The component for managing patient records
- `/Shared`: This folder contains UI elements that are used across multiple pages. `MainLayout.razor` defines the main layout for the application, typically including a header, sidebar, and main content area. `NavMenu.razor` contains the navigation links displayed in the sidebar or top menu.
- `/wwwroot`: This folder contains static assets, including images, CSS files, JavaScript files, and the service worker for **Progressive Web Application (PWA)** features. It functions similarly to the public directory in many frontend frameworks. This project template includes the **Bootstrap CSS framework** by default.
- `App.razor`: This acts as the root component for the Blazor application. It sets up routing using the Router component, rendering the appropriate page based on the URL.
- `Program.cs`: This is the entry point for the application. It configures services (such as `HttpClient`), dependency injection, and component initialization.
- `wwwroot/index.html`: This is the host HTML file for the Blazor WebAssembly app. It contains the `<script>` tag to bootstrap the Blazor runtime and references the compiled application bundle.

Now we can create our new Razor components in the corresponding folders with the following command. For brevity, we will focus on the `Appointments.razor` component.

```
dotnet new razorcomponent -n Appointments -o Pages/Appointments
```

Feel free to remove the components that were previously mentioned as samples. In the root project directory, create a `Models` folder. Add the following model definitions:

```
public class Appointment
{
    public int Id { get; set; }
    public int PatientId { get; set; }
    public int DoctorId { get; set; }
    public DateTime Date { get; set; }
    public string Status { get; set; }
}
```

These models should match the data payloads that are expected from the respective APIs.

We use the `HttpClient` library that was first discussed in *Anchor 5, Synchronous Communication between Microservices*. You may consider creating typed clients, using the example from *Anchor 4*; however, for this example, we can use a more straightforward approach to registering the clients. This approach requires that we add the `Microsoft.Extensions.Http` package using this command:

```
dotnet add package Microsoft.Extensions.Http
```

We can then add the following code to `Program.cs`:

```
var builder = WebAssemblyHostBuilder.CreateDefault(args);
// Service registrations
builder.Services.AddHttpClient("AppointmentsAPI", client =>
{
    client.BaseAddress = new Uri("https://appointments-service-url/");
});
// other HTTP client registrations
await builder.Build().RunAsync();
```

Each `HttpClient` registration creates an alias for a client instance. With this, we can make a *disposable* and short-lived instance for a specific service. Each component will inject `IHttpClientFactory`, which is an abstraction from the `HttpClient` library that allows us to create a client instance as needed. This will be used to fetch data from the appropriate API, and we will render the results in a table or list.

```
@page "/appointments"
[inject] IHttpClientFactory ClientFactory
[using] HealthPortal.Frontend.Models
```

```
<h3>Appointments</h3>

@if (appointments == null)
{
    <p><em>Loading...</em></p>
}
else
{
    <table>
        <thead>
            <tr><th>ID</th><th>Date</th><th>Status</th></tr>
        </thead>
        <tbody>
            @foreach (var appt in appointments)
            {
                <tr>
                    <td>@appt.Id</td>
                    <td>@appt.Date.ToShortDateString()</td>
                    <td>@appt.Status</td>
                </tr>
            }
        </tbody>
    </table>
}

@code {
    private List<Appointment>? appointments;

    protected override async Task OnInitializedAsync()
    {
        var client = ClientFactory.CreateClient("AppointmentsAPI");
        appointments = await
            client.GetFromJsonAsync<List<Appointment>>(string.Empty);
    }
}
```

`@page "/appointments"` declares the route that maps this Razor component to the URL path `/appointments`. When a user navigates to this path in the browser, this component will be rendered. `@inject IHttpClientFactory ClientFactory` injects an instance of the `IHttpClientFactory` into the component. This factory pattern allows the app to create configured `HttpClient` instances, often with domain-specific base URLs or authentication headers. `@using HealthPortal.Frontend.Models` brings the `Appointment` model class into scope, enabling you to use the `Appointment` type without needing a fully qualified namespace.

Razor components facilitate the use of Razor syntax, a specialized syntax that enables the seamless integration of HTML and C# code. By utilizing the `@` symbol, developers can interpolate C# constructs and variables within standard HTML. Our component presents a table that is dynamically generated based on the data processed through a `foreach` loop.

Finally, `@code { }` defines the component's logic block. This is where fields, methods, and lifecycle hooks are defined. `private List<Appointment>? appointments` declares a nullable list to hold the data is fetched asynchronously from the API. This is set to `null` initially, as the data won't exist at component initialization. `OnInitializedAsync()` is a lifecycle method in Blazor. It is automatically called after the component is initialized but before it is rendered. This is the ideal place to initiate asynchronous operations, such as API calls. Inside this method, we create an appropriate client, which, based on the name, is already bound to the `Appointments` API base address. This approach allows for dependency-injected clients that are properly configured (e.g., base address, headers). `await client.GetFromJsonAsync<List<Appointment>>(string.Empty);` sends a **GET** request to the base address of the `Appointments` API and deserializes the JSON response into a list of `Appointment` objects.

This is a standard example of how a Blazor component can be created and configured to interact with API calls. This example can easily be transferred to most of .NET's frontend frameworks, including **Razor**, **MVC**, and **.NET MAUI**. Regardless of the framework, this approach has several disadvantages. We generally want to limit the amount of knowledge that the frontend has of the services. In other words, the fewer details we expose to the frontend about how each service call needs to be performed, the better. This is where gateways can help reduce the amount of code the frontend needs to have. We will investigate this next.

## Micro frontends and API gateways

As microservices continue to evolve within enterprise applications, the API gateway emerges as a pivotal architectural component, serving not only the backend but also facilitating the frontend's interaction with distributed service endpoints. In conventional frontend architecture, a central-

ized user interface may invoke backend APIs directly, utilizing absolute URLs or domain-specific routing logic integrated within the frontend architecture. Nevertheless, this approach tightly couples the user interface to the service topology, rendering deployments fragile and augmenting the complexity associated with managing authentication, routing, and service discovery.

Applying micro frontend architecture can be quite a challenge, especially when each frontend module aligns with its own bounded context and corresponding microservice. It's essential to remember that each micro frontend must communicate independently with its backend microservice. Without a gateway in place, every frontend would have to understand the location, authentication rules, and protocol details of its backend service. This realization highlights the potential pitfalls that can hinder the goal of achieving decoupling and flexibility.

The API gateway addresses this issue by serving as an intermediary layer between micro frontends and their underlying services. It delivers a cohesive and consistent entry point for all frontend modules, simplifying service discovery, load balancing, protocol translation, and security enforcement. In the context of Blazor WebAssembly, where browser clients operate in a sandboxed environment and depend on `HttpClient` or `gRPC-Web` for backend communication, the gateway guarantees that every request, irrespective of the module, passes through a secure, standardized channel.

## How a gateway supports micro frontends

Let's say the Appointments, Doctors, and Patients micro frontends each need to fetch data from their respective APIs. With a gateway in place, instead of calling `https://appointments-api.example.com/api/appointments`, each frontend would call a unified endpoint such as `https://gateway.example.com/api/`.

The gateway, configured using YARP, Ocelot, or another reverse proxy, would route requests to the appropriate microservice using its internal service discovery mechanism. This allows each micro frontend to operate independently of the physical or logical location of its backend. Initially, our `Program.cs` file had the following:

```
builder.Services.AddHttpClient("AppointmentsAPI", client =>
{
    client.BaseAddress = new Uri("https://appointments-url/");
});
```

To support gateway routing, refactor it to the following:

```
builder.Services.AddHttpClient("Gateway", client =>
{
```

```
client.BaseAddress = new Uri("https://gateway.example.com/api/");
});
```

This change abstracts away the backend microservice URLs. If the location of the downstream services changes, you only need to update the routing in the gateway configuration, not in every frontend consumer. We are building on the examples from *Anchor 3, Implementing the API and BFF Gateway Patterns*, so feel free to reference the use of the gateway URLs from those examples.

Now we can refactor the `@code{}` section of the `Appointments.razor` component to use the appropriate endpoint call.

```
@code {
    private List<Appointment>? appointments;
    protected override async Task OnInitializedAsync()
    {
        var client = ClientFactory.CreateClient("Gateway");
        // This will resolve to:
        // https://gateway.example.com/api/appointments/
        appointments = await
            client.GetFromJsonAsync<List<Appointment>>("/appointments");
    }
}
```

Now we see that Blazor's component-based development style allows us to isolate domain-related logic and functionality quite easily. Unfortunately, we end up coupling the UI to the microservices by adding all the domain-related functionality in a single application. Leveraging multiple frontend frameworks strategically within your overall architecture maximizes each domain's user experience and developer productivity:

- **Blazor WASM** excels in rich, client-driven experiences with shared logic
- **.NET MAUI** handles native and offline experiences on mobile/desktop
- **ASP.NET Core MVC/Razor Pages** provide stable, regulatory-friendly server rendering
- **Blazor Server** delivers performant real-time UI experiences

By combining frameworks or using the most appropriate framework for the UI's requirements, thoughtfully tailored to the unique needs of each bounded context, you deliver robust, maintainable, and user-friendly applications that align with the microservices architectural style.

After confirming the desired style, ensure that the UI is as performant and stable as possible. We will discuss optimizations and best practices for smooth experiences next.

## Performance optimization for a smooth user experience

Performance plays a vital role in micro frontend architectures, where multiple independent UI modules come together to create a seamless user experience. Since each frontend module is developed and deployed independently, it's essential to adopt thoughtful strategies that keep load times short, bundle sizes small, runtime responsiveness high, and resources well-managed. Let's explore some best practices, helpful tips, and specific examples designed for .NET frontend frameworks such as Blazor WASM, Blazor Server, ASP.NET Core MVC, Razor Pages, and .NET MAUI.

The UI is often the only part of the application the user sees and directly interacts with, and the distributed nature of its backend inherently constrains it. Each feature a user interacts with may involve multiple API calls to different services, often deployed on separate servers with independent failure modes.

Therefore, we need to ensure that we are making the best use of the target device's resources and are loading the UI in the most efficient manner possible. Unlike monoliths, where a single backend often provides aggregate data in a UI-friendly format, microservices rarely do this out of the box. The UI must handle multiple asynchronous API calls, disparate service responses, error propagation, and fallback UI rendering, as well as variations in latency, availability, and schema. While these factors can be mitigated in theory, user-facing issues are quick to expose cracks in the design.

Let us begin by taking some measures to ensure that our frontend application is as performant as possible. We will start by implementing **ahead-of-time** compilation.

### Implementing ahead-of-time compilation

We have already unpacked why Blazor WASM is a good selection for a frontend framework. It is lightweight, robust, and supports domain segregation with ease. The fact remains, however, that this Blazor UI may need to interact with several services, and different error scenarios must be handled. Several key considerations are essential to ensure that your architecture remains scalable, maintainable, performant, and aligned with best practices for microservices.

Because Blazor WASM ships a stripped-down version of the .NET runtime and application binaries to the browser, it can result in significantly larger initial downloads compared to JavaScript frameworks such as React or Vue. This payload includes not only your application's compiled assemblies but also the .NET runtime, localization files, satellite assemblies, and referenced packages. The larger the payload, the longer it takes for the app to load, especially on slower networks or mobile devices.

Careful management of asset sizes and loading strategies is critical. We can implement **Ahead-of-Time (AOT)** compilation. This build process compiles your application code to WASM ahead of runtime, rather than relying on the .NET **Intermediate Language (IL) interpreter** in the browser. This improves runtime performance and reduces the time the browser needs to *warm up* your app after it has been downloaded.

With AOT, instead of shipping IL assemblies and interpreting them at runtime, the Blazor build tool compiles C# code directly into WASM. This reduces startup time and CPU load in the browser, especially for computationally intensive or frequently executed logic. To enable AOT compilation, update your .csproj file:

```
<PropertyGroup>
  <PublishTrimmed>true</PublishTrimmed>
  <PublishAot>true</PublishAot>
</PropertyGroup>
```

`PublishTrimmed` removes unused IL and metadata during the build process. This process is also known as **tree shaking**, a form of dead code elimination. For example, if you reference a large NuGet package but only use one class or method from it, trimming ensures that only the required code is included in the final binary.

`PublishAot` activates the AOT toolchain to compile assemblies into native WASM binaries. One potential drawback is that AOT increases the size of the build artifacts on disk and can extend build times; however, it reduces CPU usage and improves runtime performance in the browser.

Another technique for speeding up Blazor WASM apps is through code splitting and lazy loading assemblies. We will review this next.

## Lazy loading components

Code splitting allows your Blazor app to divide functionality into separate bundles or assemblies that are loaded on demand. This approach is crucial in micro frontend scenarios where a user may not require all domain-specific features simultaneously. This technique is ideal for less frequently used features and software modules (e.g., admin tools), domain-specific modules in a shell-hosted micro frontend system, or generally large libraries or third-party controls.

This concept is like **module federation**, a method for assembling frontends using independently constructed and deployed modules. First introduced in the JavaScript ecosystem through Webpack 5, it enables teams to create self-sufficient frontend modules that can be dynamically loaded into a host application during runtime, eliminating the need to bundle them at build time.



Traditionally, deploying a change to one part of a UI required rebuilding and redeploying the entire frontend application. With module federation, each domain-specific UI can be developed, tested, and released in isolation, drastically improving agility. This is especially critical in regulated or fast-moving industries. Furthermore, they can be dynamically loaded by a host shell application at runtime, rather than at compile time. In Blazor, a federated module might be a Razor component library or .dll assembly representing a bounded UI context. Instead of hardcoding dependencies into a monolithic WASM app, a shell loads and instantiates these assemblies on demand using tools such as LazyAssemblyLoader.

Let us review how we can implement this in our solution. Start by adding a shared **Razor Class Library (RCL)**, called `HealthPortal.Contracts`, to the solution using the following commands.

```
dotnet new razorclasslib -n HealthPortal.Contracts
dotnet sln HealthCare.Microservices.sln add HealthPortal.Contracts/
HealthPortal.Contracts.csproj
```

RCLs are a standard way to share components between multiple Blazor apps, or between a Blazor web app and a .NET MAUI hybrid app that renders Blazor components. Now, add the reference to this new project with this command.

```
dotnet add HealthPortal.Frontend/HealthPortal.Frontend.csproj reference
HealthPortal.Contracts/HealthPortal.Contracts.csproj
```

For this demo, we will lazy load the `Appointments.razor` component. So, move the `Appointments` directory from the `HealthPortal.FrontEnd` project and place it in the newly created RCL project. You must move the corresponding `Appointments` model class files. Ensure that you add the relevant NuGet packages to the RCL library to resolve any missing reference errors.

Now, once you have moved this component, proceed to modify the `App.razor` component in the frontend project. We need to add logic that allows the RCL component to be dynamically loaded when a navigation request requires it.

```
@using Microsoft.AspNetCore.Components.WebAssembly.Services
@using System.Reflection
@inject LazyAssemblyLoader LazyAssemblyLoader
<Router AppAssembly="@typeof(App).Assembly"
AdditionalAssemblies="assemblies" OnNavigateAsync="NavigateAsync">
  @* Shortened for brevity *@
</Router>
@code {
```

```

private List<Assembly> assemblies = new();
private async Task NavigateAsync(NavigationContext context)
{
    if (context.Path.StartsWith("appointments"))
    {
        var assemblyName = context.Path.Substring(6);
        var assembly = await LazyAssemblyLoader.LoadAssembliesAsync(
            new string[] { "HealthPortal.Contracts.wasm" });
        if (assembly != null)
        {
            assemblies.AddRange(assembly);
        }
    }
}

```

We start by injecting the `LazyAssemblyLoader` service into the component via dependency injection. The Blazor framework automatically registers a singleton service for `LazyAssemblyLoader`. The `LazyAssemblyLoader.LoadAssembliesAsync` method uses JS interop to fetch assemblies via a network call. Then we add `AdditionalAssemblies="assemblies" OnNavigateAsync="NavigateAsync"` to the Router tag to indicate that additional assemblies will be loaded based on a list of assemblies. This list is dynamically populated in the `NavigateAsync` method, which is an event fired every time a navigation attempt occurs in the application.

Finally, we check for specific route requests in the `NavigateAsync` method, and if the pattern matches a request for our component, appointments in this case, then we add the assembly to the list. Note that the name of the assembly is the name of the RCL project with the extension `.wasm`.

While this example shows one component being loaded dynamically, it also stands to reason that multiple elements could be loaded with a routing request. This is an excellent way to ensure that only what is needed is loaded per request.

To complete this setup, we need to add an `ItemGroup` section to the `csproj` file of the frontend project.

```

<ItemGroup>
    <BlazorWebAssemblyLazyLoad Include="HealthPortal.Contracts.wasm" />
    <!-- Add more assemblies as needed -->
</ItemGroup>

```

Blazor's Router component specifies which assemblies Blazor searches for routable components and handles rendering the component for the current route. Its `OnNavigateAsync` method is used together with lazy loading to load the appropriate assemblies for user-requested endpoints.

Now that we see how to make our Blazor WASM project more efficient, let us review how to make it more resilient.

## Building a resilient micro frontend

It is essential to understand what can go wrong from the UI perspective and mitigate as much as possible. Let us proceed to review some best practices that we can apply not only to Blazor but also to other .NET frontend frameworks.

We already know that several frontend frameworks are available. Users interacting with distributed systems rarely distinguish between frontend or backend errors; any service failure, delay, or timeout is experienced as a failure of the overall system. Therefore, whether you are building your frontend using Blazor WebAssembly, Blazor Server, ASP.NET Core MVC, Razor Pages, or even .NET MAUI, implementing resilient communication patterns is essential to ensure a seamless, responsive, and trustworthy user experience.

We also need efficient ways to handle errors. We may need to employ similar resilient service communication techniques to those discussed in *Anchor 5, Building Resilient Microservices*. Resilient API communication is not just about the transport layer; it must also be reflected in the user experience. A simple way to mitigate errors during component loading is what we did in our prior example. We use an `if` statement to dynamically update the component's content based on the presence of data. This way, we can avoid possible null exceptions being thrown and manage what the user sees as they wait for data (that may never arrive due to an error from the API).

By extension, Blazor allows us to ensure that every frontend component anticipates service failure and renders alternative UI states, therefore, gracefully handling and recovering from errors independently. We can implement **error boundary components** to encapsulate risky or independent operations:

```
<ErrorBoundary>
  <ChildContent>
    <Appointments />
  </ChildContent>
  <ErrorContent>
    <p>An error occurred while loading appointments.</p>
  </ErrorContent>
</ErrorBoundary>
```

This prevents issues in one component from cascading across your entire UI.

Other techniques that we can employ, that are not unique to Blazor, include the following:

- **Retry policies and timeout handling:** Retries help handle transient failures, short-lived outages, dropped connections, or timeouts. Rather than failing immediately, the frontend can automatically retry the request with a delay. Polly, the de facto resilience library for .NET, supports sophisticated retry strategies. Blazor Server, MVC, Razor Pages, and MAUI can directly register `HttpClient` instances with Polly, as we have already explored. With Blazor WASM, however, since it runs in the browser, Polly doesn't work. Instead, basic retry logic must be implemented manually using try-catch blocks and async delays:

```
int retries = 3;
while (retries <= 3)
{
    try
    {
        var data = await Http.GetFromJsonAsync<T>("endpoint");
        break;
    }
    catch when (retries > 0)
    {
        await Task.Delay(1000);
    }
}
```

- **Circuit breakers and rate limiting:** Every request should have a timeout boundary to avoid infinite hangs. In Blazor Server or MVC, a backend `HttpClient` can enforce this using Polly. MAUI supports both Polly and `CancellationTokenSource`, making it flexible for all communication types. Blazor WASM employs a manual implementation using `CancellationTokenSource` because it cannot use Polly:

```
using var cts = new CancellationTokenSource(5000);
await Http.GetFromJsonAsync<T>("endpoint", cts.Token);
```

- **Caching and stale data strategy:** This reduces the number of backend hits and provides a smoother user experience if data fails to load from the primary source. We can use in-memory or persistent caching for data that doesn't change frequently, such as user profiles and static lookup lists. MAUI apps can cache data to SQLite or local files; Blazor Server and MVC apps can use the memory cache or Redis via `IMemoryCache` and `IDistributedCache`.

Blazor WASM can utilize the browser's `localStorage` or `sessionStorage`. If it is running as a PWA, it can leverage features built into Blazor WASM to allow caching of assets and responses, significantly improving load performance and offline functionality. Use service workers for caching API responses and static assets to enhance resilience:

```
// Example service worker configuration in wwwroot/service-worker.js
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request).then(response => {
            return response || fetch(event.request);
        })
    );
});
```

- **Logging and frontend observability:** It is essential to be able to observe and monitor your frontend apps. This helps us identify slow responses, failed requests, retry attempts, and circuit breaker states. Common tools include the following:
  - Application Insights
  - OpenTelemetry
  - Custom logging (Serilog, NLog)

All .NET frontends can emit logs. MVC apps log via `ILogger<T>` and support correlation IDs, and MAUI and Blazor can inject custom telemetry services. We will discuss this in more depth in *Anchor 3, Observability and Monitoring with Modern Tools*.

Resilient microservice communication is a common challenge for all .NET frontend frameworks. The retry, timeout, fallback, caching, and observability patterns are similar, yet their implementations differ depending on the execution context. It is essential to design your frontend to expect and effectively manage service interruptions, not solely as a backend issue, but as a crucial part of the user experience.

By embedding resilience throughout your UI, you foster a system that degrades predictably, recovers smoothly, and builds user trust over time. Now we also need to consider that micro frontends sometimes need to communicate with each other. Let us now discuss how we can manage state and ensure effective frontend communication between operations.

## State management and communication between micro frontends

The concept of **state** refers to any data that represents the status of an application or a user's interaction with it. This includes everything from authentication tokens, form inputs, selected items in a list, scroll positions, loaded API data, and UI view preferences, such as themes. It can also refer to more complex constructs, such as a shopping cart or the steps in a workflow. The **management** of this state, or how it is stored, updated, shared, and persisted across sessions, components, and even services, is referred to as **state management**. In modern frontend applications, especially those built on top of microservices or following a micro-frontend architecture, state management becomes both critical and increasingly complex.

In micro frontend architectures, state can be **local** or specific to a component, such as an input value or tab selection; **shared** or available to multiple components; or **remote**, where it is stored on the server or retrieved from APIs. Managing these tiers of state requires careful planning. For example, shared state might live in the following:

- A **singleton service** injected via .NET dependency injection
- **Browser storage** (localStorage/sessionStorage) for persistence across modules
- **Query strings** to persist across navigation
- A backend cache if shared between devices or across users

While sharing state is vital to the cohesive operation of frontend components, several implications arise with each layer:

- Volatile or sensitive data *should not* be stored in the browser
- Session data might need encryption or expiration policies
- UI-specific state (e.g., scroll position) should reset on navigation but be retained during component updates

Now that we have a basic understanding of what it means to manage and share state between UI components or micro frontends, let us look at sharing the user session state in a Blazor WASM application.

### Sharing user session state

From the user's viewpoint, a responsive and user-friendly application “remembers” their actions. This includes maintaining unsaved form input when users navigate away, keeping filters on a dashboard, or retaining patient context across healthcare modules.

Effective state management ensures continuity, whereas poorly managed states result in frustration, repetitive input, or disorienting UI transitions. In component-based micro frontend architectures, individual modules often handle their data and behavior. Without a unified approach to managing shared state (e.g., the currently logged-in user, the selected domain entity), inconsistencies arise.

Blazor components are isolated by default, as they encapsulate both their UI and logic, and do not share state unless it is explicitly passed in as a parameter or injected as a service. It is common for several components (such as headers, dashboards, profile widgets, and menus) to need access to the *same session data*. Without a shared state container, components would have to duplicate logic to retrieve session data, resulting in inefficiency. Additionally, a change in one component does not automatically propagate to others, which can result in an inconsistent user interface and experience.

This issue is pervasive relative to authentication and user session management. Login/logout functionality is generally encapsulated in components, so a user logging in does not automatically change the state of other components in the app. The solution is to create a singleton state container, `UserSessionState`, that all components can inject and reactively bind to.

Let us begin by defining the `UserSessionState` class. This class stores user session data and exposes a simple event mechanism, allowing components to respond to state changes.

```
public class UserSessionState
{
    public string? Username { get; private set; }
    public string? Role { get; private set; }
    public bool IsAuthenticated => !string.IsNullOrEmpty(Username);

    public event Action? OnChange;

    public void SetUser(string username, string role)
    {
        Username = username;
        Role = role;
        NotifyStateChanged();
    }

    public void ClearUser()
    {

```

```
        Username = null;  
        Role = null;  
        NotifyStateChanged();  
    }  
  
    private void NotifyStateChanged() => OnChange?.Invoke();  
}
```

SetUser() updates the state and notifies subscribers, specifically the UI components. ClearUser() resets the session as it is called on logout, and OnChange is an event that notifies all consumers to *rerender* when the state changes.

In Program.cs, register the service as a singleton, so that all components get the same instance.

```
builder.Services.AddSingleton<UserSessionState>();
```

Now we can create a new component that handles the login operation. We will review application security and authentication later, so this code is for demonstration purposes only. We will call this component Login.razor.

```
@page "/login"  
@inject UserSessionState Session  
<h3>Login</h3>  
<input @bind="username" placeholder="Username" />  
<input @bind="password" placeholder="password" type="password" />  
<button @onclick="LogIn">Log In</button>  
  
@code {  
    private string username = string.Empty;  
    private string role = string.Empty;  
  
    private void LogIn()  
    {  
        Session.SetUser(username, role);  
    }  
}
```



This component accepts a username and roles, setting the values in the application's session state. Upon button click, the `SetUser()` method is called to update the application's authentication state. With that in place, we can modify the existing `NavMenu.razor` component to change its state based on the session state.

```
@inject UserSessionState Session
<div class="top-row ps-3 navbar navbar-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">HealthPortal.Frontend</a>

    <div class="d-flex align-items-center">
      @if (Session.IsAuthenticated)
      {
        <span class="text-white me-3">
          Welcome, @Session.Username (@Session.Role)</span>
        <button class="btn btn-sm btn-outline-light"
          @onclick="Logout">Logout</button>
      }
      else
      {
        <span class="text-white me-3">Not logged in</span>
      }

      <button title="Navigation menu" class="navbar-toggler"
        @onclick="ToggleNavMenu">
        <span class="navbar-toggler-icon"></span>
      </button>
    </div>
  </div>
</div>

@* Existing menu items *@
</div>

@code {
  private bool collapseNavMenu = true;

  private string? NavMenuCssClass => collapseNavMenu ? "collapse" :
```

```
        null;

        private void ToggleNavMenu()
        {
            collapseNavMenu = !collapseNavMenu;
        }

        protected override void OnInitialized()
        {
            Session.OnChange += StateHasChanged;
        }

        private void Logout()
        {
            Session.ClearUser();
        }

        public void Dispose()
        {
            Session.OnChange -= StateHasChanged;
        }
    }
```

We start with `@inject UserSessionState Session`, which brings the shared session state into the component. We then add a new section that conditionally renders user information beside the navbar toggler. The logout button allows the user to clear the session data. In the `@code` section, we have added `Session.OnChange += StateHasChanged` to the `OnInitialized()` method to ensure the component updates reactively when session state changes. `Dispose()` removes the event subscription to prevent memory leaks.

Now that we have an idea of the techniques that can be used to develop UI apps for a microservices-based application, let's review what we have discussed in this chapter.

## Summary

This chapter presented a comprehensive and methodical treatment of frontend architecture in a distributed microservices ecosystem, with a particular focus on implementing solutions using Blazor WebAssembly in the .NET ecosystem. It highlighted the need for scalable and modular frontends that can align with microservice principles such as bounded contexts, team autonomy, and independent deployment cycles.

We started by articulating the role of the frontend in a modern application, explaining its responsibilities and foundational web technologies such as HTML, CSS, and JavaScript. The evolution from monolithic UI structures to more modular, service-oriented architectures was traced, culminating in the introduction of macro-frontends and micro-frontends. We reviewed how **Domain-Driven Design (DDD)** naturally complements the micro frontend model by providing clear boundaries around user-facing functionality, and how each team can benefit from independently developing, owning, and deploying domain-specific micro frontends.

We focused on using Blazor **WebAssembly (WASM)**, while acknowledging that there are suitable alternatives within the .NET ecosystem. After comparing other options, such as ASP.NET Core MVC, Razor Pages, Blazor Server, and .NET MAUI, we weighed the benefits and limitations of each in the context of microservices. Blazor WASM's suitability is underscored by its ability to deliver rich, interactive user interfaces in the browser without relying on JavaScript, its tight integration with .NET backend services, and its support for dynamic module loading and independent deployment.

We then examined the steps to create a Blazor WASM frontend project by walking through the creation of a modular Blazor WASM frontend for the health management application, step by step. This practical walk-through covered project structure, component creation, data fetching using `HttpClientFactory`, and integration with backend services via gateway routing. Special attention was given to aligning frontend components with backend microservices, utilizing techniques such as lazy assembly loading and module federation to import and render UI components dynamically.

We then explored why having an API gateway makes frontend development much easier by abstracting service endpoints, simplifying communication, and centralizing cross-cutting concerns. This is particularly beneficial when the frontend needs to communicate securely and efficiently with multiple backend services.

Performance optimization was addressed through techniques such as **Ahead-of-Time (AOT)** compilation, tree shaking, and lazy loading. We demonstrated how reducing the initial payload size and leveraging runtime assembly loading can dramatically improve the startup performance and scalability of Blazor WASM apps. These strategies were complemented by discussions on resilient frontend communication, including retry policies, circuit breakers, caching, and observability, ensuring that frontend modules remain responsive and fault-tolerant even when backend services fail.

Finally, we reviewed state management and inter-module communication, which are essential in any micro frontend system. After understanding what is meant by *state*, we explored why it's critical for usability and how it should be managed across independently loaded components. We also illustrated how to implement a shared session state in Blazor WASM using a singleton pattern, enabling components to reactively update their UI in response to login or logout actions.

Overall, the chapter struck a balance between theoretical underpinnings and actionable guidance, demonstrating how to design, build, and manage a modular frontend architecture that aligns with microservice principles. It addressed common pitfalls, emphasized scalability and maintainability, and advocated for technologies and patterns that enable UI development to evolve in tandem with backend systems. The result was a compelling case for using Blazor WebAssembly in enterprise-grade .NET applications where flexibility, performance, and team autonomy are paramount.

Now, we will explore implementing security measures in our microservices application.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





# 13

## Securing Microservices

Security is a fundamental requirement of any modern software solution. With microservices enabling rapid delivery and scalability, the surface area of our applications has expanded, exposing more potential vulnerabilities. This makes securing every endpoint, message exchange, and data store critical. Security must be woven into the fabric of every microservice, not retrofitted as an afterthought.

However, implementing robust security in microservices comes with challenges. The very attributes that make microservices appealing (independence, decentralization, and polyglot flexibility) also complicate traditional security models. Unlike monolithic applications, where a single login might suffice, a microservices-based system must authenticate and authorize requests across a constellation of services. Each service must not only verify the identity of the user or client but also determine whether the requested operation is permissible, all while maintaining low latency and high availability.

In this chapter, we explore how to secure microservices effectively using modern techniques, focusing on token-based authentication and integration with identity providers. The journey begins with understanding the principles of bearer token security, including how tokens are issued, validated, and used to propagate identity and claims across service boundaries. We then transition to practical implementation in ASP.NET Core, using industry-standard tools and libraries to authenticate users, validate tokens, and secure service-to-service communication.

The challenge does not stop at simple user authentication. In a microservices ecosystem, services must often act on behalf of a user or communicate autonomously. We will explore how to protect these interactions, enforce scopes and claims, and adopt centralized identity providers to simplify and standardize authentication across the entire system.

In this chapter, you will do the following:

- Understand security and methods of implementation
- Understand how bearer token security underpins modern microservice authentication and authorization strategies
- Learn how to implement secure APIs using ASP.NET Core and configure services to validate and propagate tokens responsibly
- Gain hands-on experience integrating an identity provider to secure your microservices landscape and streamline authentication across distributed services

Securing microservices is a continuous and evolving responsibility. Let us now explore how to design and build security into every interaction within our system, ensuring that our architecture is not only scalable and efficient but also resilient against threats.

## Technical requirements

Code references used in this chapter can be found in the project repository hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET--Second-Edition/tree/main/Chapter13>.

## Understanding API security

Before we dive into API and microservices security, let's take a moment to review the general term "security" and why it is an important concept to be familiar with as a modern developer.

Application security refers to the discipline of implementing safeguards and best practices to protect software applications from threats and vulnerabilities throughout their life cycle, from development through deployment and maintenance. Its purpose is to ensure that data handled by the application is protected from unauthorized access, alteration, or destruction, and that the application functions reliably, even in the face of malicious attempts to exploit its behavior.

In a traditional monolithic system, security is typically centralized and easier to enforce due to the single deployment unit. In contrast, in a microservices-based architecture, application security becomes inherently more complex. Each microservice represents an independent process with its data, interfaces, and potential vulnerabilities. Therefore, security must be applied not only at the edge (e.g., the API gateway or web UI) but also across internal communication boundaries between services.

Security in a monolithic application, although often more centralized than in microservices, remains a multifaceted responsibility. Since a monolithic architecture consolidates all features, business logic, and data access into a single deployable unit, it can be easier to implement security because it offers fewer moving parts and a smaller surface area for external attacks compared to distributed systems. This, however, does not mean it is inherently simpler. It demands a disciplined approach across several layers of the application stack to protect against common vulnerabilities and ensure the confidentiality, integrity, and availability of data and services.

Now, let us explore some of the most important concepts when implementing security measures in our applications, whether they are monoliths or microservices-based.

## Essential application security concepts

Securing your application against attacks such as **Distributed Denial of Service (DDoS)**, brute-force login attempts, and unauthorized access enhances overall system stability and reduces the potential impact of compromises. In the long run, users and stakeholders gain confidence in the system's integrity and reliability. In secure applications, privacy and data ownership are respected, which enhances brand reputation and fosters customer retention.

Some of the most critical aspects of application security are as follows:

- **Authentication:** At the heart of any secure application lies the ability to verify who is trying to access the system. Authentication is the process of establishing identity, typically using credentials such as a username and password. In a monolithic application, authentication is usually implemented once and shared across the entire code base.
  - The most common authentication method in web applications is cookie-based authentication. Upon successful login, the application creates a session or issues a **signed cookie** that is stored on the client and sent with every subsequent request. This token or session identifier maps to a server-side session that maintains the authenticated user's context.
- **Authorization:** Once a user is authenticated, the next layer of defense is authorization, determining what that user is allowed to do. In a monolithic application, authorization is often enforced using role-based access control (**RBAC**), where users are assigned to roles (e.g., admin, doctor, nurse, clerk, etc.), and access to system features or resources is granted based on these roles. For example, in a healthcare system, a user might be allowed to view only patient records that belong to their assigned department. These constraints can be encoded using custom **claims** (e.g., department ID) and evaluated through a policy engine.



- **Data confidentiality and integrity:** Protecting data at rest and in transit is another foundational requirement. Sensitive data, such as passwords, health records, payment details, or **personally identifiable information (PII)**, must be encrypted or hashed. Passwords, for instance, should be hashed using a secure algorithm to defend against rainbow table attacks. For data in transit, HTTPS (TLS) must be enforced to prevent **man-in-the-middle (MITM)** attacks. This involves securing all HTTP endpoints with TLS certificates and ensuring that clients are redirected to the secure endpoint.

Another aspect of application security is the context in which the application operates. While the critical aspects should always be considered, additional security measures may or may not be necessary, depending on the industry or the application's general purpose. For instance, our health management system will have different security requirements compared to a school management system or an online banking system.

In this context, we must consider protecting sensitive data such as patient records, financial transactions, and PII, which is especially crucial in regulated industries such as healthcare. Different sectors have governing regulatory requirements that also dictate some of the measures that must be taken. Applications in domains such as healthcare (**Health Insurance Portability and Accountability Act (HIPAA)**), finance (**Payment Card Industry Data Security Standard (PCI DSS)**), or general data protection (**General Data Protection Regulation (GDPR)**) must meet stringent security standards. A secure-by-design approach simplifies compliance and avoids penalties.

While security is essential, as mentioned before, there is a delicate balance between security and usability. From the user's perspective, we generally need to introduce additional steps and usability challenges. Security often introduces friction into user workflows. Examples include **multi-factor authentication (MFA)**, session timeouts, and frequent re-authentication. Striking the right balance between usability and protection is a non-trivial task. While solutions attempt to be user-friendly, not every user appreciates the extra steps that are sometimes required.

For the developers, securing applications often involves additional layers of indirection that complicate system design and add maintenance overhead. Developers must possess in-depth knowledge of security principles and implementation details, including OAuth 2.0 flows, secure token storage, and encryption algorithms. Security misconfigurations, such as open ports, weak **Cross-Origin Resource Sharing (CORS)** settings, or overly permissive roles, are among the top vulnerabilities in cloud-native and microservices applications. These often result from complex setups and human error. Poorly implemented security can lead to more risk than having no security at all.

Application security is both a foundational concern and a multifaceted challenge in modern software systems. As we transition to distributed architectures, such as microservices, the responsibility for securing data and interactions increases proportionally. Now that we have a general understanding of the need for application security and some of the challenges we must navigate, let's review some specific challenges related to microservices.

## Challenges in securing microservices

Securing microservices poses a unique set of challenges that go far beyond the scope of traditional application security. While monolithic applications typically centralize authentication, authorization, and access control logic, microservices architectures decentralize those concerns across multiple services. This distribution complicates enforcement, visibility, and consistency. Moreover, because microservices often span different networks, execution environments, and deployment units, the surface area for potential attacks is significantly increased.

Since each microservice is an autonomous unit that typically exposes one or more endpoints via HTTP or gRPC, these endpoints may be public or internal. Still, each one is a potential attack vector. Unlike a monolith that may sit behind a single authentication gateway, microservices may need to authenticate and authorize not only external clients but also internal service-to-service requests. This creates a distributed trust boundary that must be managed with precision.

In our healthcare management system, for instance, we have services for patient profiles, appointment scheduling, billing, document storage, and notifications. Each of these services needs to validate the identity of the requester and enforce access control rules accordingly. A request to generate an invoice should only succeed if the calling user or service is authorized to view billing information for the specified patient. Even seemingly harmless services, such as the document upload endpoint, must ensure that uploaded content cannot be exploited to inject malware or overwrite unauthorized files.

Beyond that, regulations such as HIPAA require stringent security controls around patient data, including encryption in transit, auditing access logs, and ensuring that only authorized personnel can perform sensitive operations. These requirements are often challenging to meet in a system composed of independently deployed services with their own data stores and business logic.

One of the foremost challenges is distributed authentication. With users and services potentially accessing the system from multiple devices and networks, verifying identity is of paramount importance. Relying on session cookies tied to a central web server, as is common in monolithic applications, is insufficient. Microservices require stateless authentication mechanisms that can propagate identity claims across services without sharing session state.

Securing service-to-service communication is yet another obstacle. Unlike frontend clients that can be protected behind firewalls and load balancers, internal services often communicate over private networks, container orchestrators, or cloud-native platforms. This communication must be encrypted and authenticated. Any failure to enforce strict identity verification between services could allow rogue or compromised services to inject malicious requests into the system, potentially compromising the security of the entire system.

To address these challenges, several security patterns and technologies have emerged. Among the most widely adopted are API key authentication, Basic Authentication, and OAuth 2.0 with bearer tokens:

- **API key authentication:** One of the simplest forms of authentication. A unique key is issued to each client, which must be sent with each request to access the service. While easy to implement, this method has significant limitations. API keys are often static and lack user context. They provide minimal traceability and are vulnerable to leakage if logged or improperly secured. Rotating API keys without service disruption is also challenging, especially in distributed systems.
- **Basic Authentication:** Transmits a Base64-encoded username and password with each request, which is another legacy approach. While acceptable for internal tools or quick prototypes, it is generally discouraged for production microservices. Credentials are typically long-lived, and any compromise of the transport layer exposes user secrets. Even when combined with HTTPS, this method fails to provide fine-grained access control or auditability.
- **OAuth 2.0 with bearer tokens:** A more robust and modern alternative. OAuth separates the process of identity verification (authentication) from authorization, allowing tokens to be issued by a centralized **identity provider (IdP)**. In this model, a user authenticates with the IdP and receives a signed **JSON Web Token (JWT)**. The client then presents this token to microservices, which validate its signature and read claims to determine access rights. JWTs provide a compact, stateless method for propagating identity and authorization metadata. Claims may include the user's ID, roles, scopes, or even custom attributes, such as department or license tier. Services can verify the authenticity of tokens without needing to query a central database, making this method highly scalable.

Securing microservices is a complex endeavor that demands meticulous attention to detail and a profound understanding of distributed systems. The challenges of authentication, authorization, transport security, and secrets management are exacerbated by the decentralized nature of microservices. While several security methods exist, each comes with its trade-offs, and no single approach is sufficient on its own.

In the next section, we will take a more in-depth look at JWTs and explore how to secure our APIs using this security method.

## Bearer tokens for securing communications

In a typical web application, most of these factors can be implemented through form authentication, where we request uniquely identifying information and verify our database for a match. When a match is found, we instantiate a temporary storage mechanism that will identify the user as authenticated in our system. This temporary storage construct can come in the following forms:

- **Sessions:** A way to store information in a variable that can be used across a website. Unlike typical variables that lose their value with each request, a session retains its values for a period until it is either expired or destroyed. Session variables are typically stored on the server, and one or many session variables are created each time a user authenticates successfully. Session variables can store information such as username, role, and other relevant details. With too many users logging in simultaneously, using session variables can lead to memory issues on a less robust server.
- **Cookies:** An alternative to sessions, where a small file is created and stored on the user's device. It serves a similar purpose for storing information between requests as well as tracking a user's authenticated state. Each time a request is sent from the user's device, this cookie is sent, and the server web application uses its information to determine whether actions can be taken, and which actions, if so. Cookies are sometimes preferred over sessions, as they reduce the server's load and place more responsibility on the user's device.

Both options work fantastically when we are sure that we will be dealing with a web application that maintains a **state**. A session means that we retain user information between requests and remember who is logged in, along with their basic information, for the duration of their website usage. But what happens when we need to authenticate against APIs? An API, by nature, does not maintain a state. It does not attempt to retain the knowledge of the users accessing it, since APIs are designed for sporadic access from any channel at any point. For this reason, we implement bearer or access tokens.

- **Tokens** are self-contained pieces of information, typically cryptographically signed, that can convey authentication and authorization claims. They are passed along with API requests and verified by the receiving service without the need to consult a central authority or session store. This model supports scalability, statelessness, and performance

However, managing authentication using tokens presents several challenges. Because JWTs are typically self-contained and non-revocable, they must be short-lived to minimize the window of compromise. This introduces the need for token refresh mechanisms and potentially complex flows involving refresh tokens and silent re-authentication. Additionally, care must be taken to avoid leaking tokens in logs or error responses.

Let's take some time to review JWTs, including how they are created, used, and implemented in an API.

## Understanding JWTs

A JWT is a widely used construct in authentication and authorization scenarios for **stateless** APIs. Bearer tokens are based on an open industry standard of authentication that has made it easy for us to share authenticated user information between a server and a client. When an API is accessed, a temporary state is created for the duration of the request-response cycle. This means that when the request is received, we can determine the originating source of the request and can decode additional header information as needed. Once a response is returned, we no longer have a record of the request, its origin, or who submitted it.

A **bearer token** is a type of access token that authorizes the bearer (i.e., the party that possesses the token) to access a protected resource. The name “bearer” originates from the concept that whoever bears the token is assumed to be the rightful party and is granted access without requiring further proof. Bearer tokens are issued after a successful authentication request. We receive a request to our authentication API endpoint and use the information to check our databases, as previously described. Once a user is verified, we compile several data points, such as the following:

- **Subject:** This is usually a unique identifier for the user, such as the user ID from the originating database.
- **Issuer:** This is usually a name that is associated with the service that has generated the token for issuance.
- **Audience:** This is usually a name that is associated with the client application that will be consuming the token.
- **Username:** This is the user's unique system name. Usually used for login.
- **Email address:** This is the user's email address.
- **Role:** This is the user's system role that determines what they are authorized to do.



```
JwawN0dXJlIjogImh0dHA6Ly9leGFtcGx1LnVbS9qYW51ZG91L211LmpwZyIKfQ.
rHQjEmBqn9Jre00LykYNnspA10Qql2rvx4FsD00jw1B0Sym4NzpgvPKsDjn_wMkHxcp-
6CilPcoKrWHcipR2iAjzLvDNAReF97zoJqq880ZD1bwY82JDauCXELVR906_B0w3K-E7y-
M2macAAgNCUwtik6SjoSUZRcf-05lygIyLENx882p6MtmwaL1hd6qn5RZ0Q0TLr0Yu-
0532g9Exxcm-ChymrB4xLykpDj31UivJt63eEGGN6DH5K6o33TcxkIJNrCD4XB1CK-
KumZvCedgHHF3IAK4dVEDSUoG1H9z4pP_eWYNXvqQ0jGs-rDaQzUHL6cQQWNiDpW0l_1xX-
jQEvQ&state=af0ifjsldkj
```

Most development frameworks include tools and libraries that can decrypt bearer tokens during the application's runtime. Since bearer tokens are based on an open standard, support for decoding tokens is widely available. This allows us to write generic and consistent code to handle tokens being issued by an API. Each API implementation may include different tokens tailored to the specific needs of the application, but there are certain standards that we can always rely on.

During development, however, we may want to test a token to see the contents in a more human-readable form that we can expect to be present. For this reason, we rely on third-party tools that decode and display the contents of a token, allowing us to reference specific bits of information as needed. Tools such as [jwt.io](https://jwt.io) provide us with the ability to paste a token, as seen in the preceding example, and view the information in a more human-readable format.

Figure 13.1 shows the contents of a bearer token after it has been decoded.

PAYLOAD: DATA

```
{
  "iss": "http://server.example.com",
  "sub": "248289761001",
  "aud": "s6BhdRkqt3",
  "nonce": "n-0S6_WzA2Mj",
  "exp": 1311281970,
  "iat": 1311280970,
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe",
  "gender": "female",
  "birthdate": "0000-10-31",
  "email": "janedoe@example.com",
  "picture": "http://example.com/janedoe/me.jpg"
}
```

Figure 13.1 – Once we decode the token, we can access all the token's information and know how to reference different claims in the client application

All the information placed into a bearer token represents a key-value pair. Each of the keys in *Figure 13.1* serves a purpose:

- **iss**: This represents the **issuer** value. The issuer typically refers to the application that has issued the token. This could be the name of the API that generates the token after a user is successfully authenticated.
- **sub**: This represents the **subject** value. This value may vary, but it is used to identify the user by a standard system value, such as a username or user ID. The application consuming the JWT (both client and API) will rely on this value when attempting to perform user-specific operations.
- **aud**: This represents the **audience** value. This is the name given to an application that has requested and received a token. This value can be used to validate that a valid application has a valid token.
- **nonce**: This represents a unique value that should always change with each token to prevent replay attacks. This value is always new, ensuring that no two tokens issued to the same user are the same. This can sometimes be called a **jti** claim.
- **exp**: This represents the expiration date of the token. The value is in the form of a Unix epoch, which is a numerical representation of a moment in time.
- **iat**: This represents the date and time of issuance.

The token is presented in the HTTP `Authorization` header in subsequent API requests. The header looks like this:

```
Authorization: Bearer <token>
```

The simplicity of bearer tokens is also a liability. They operate on the principle of possession. If a token is intercepted, leaked, or stolen, any party holding that token can impersonate the original user. Bearer tokens are thus highly vulnerable to token theft, and additional mechanisms, such as HTTPS and short token lifetimes, are required to mitigate this risk.

This is where we introduce access tokens. An access token is a broader concept defined in the OAuth 2.0 framework. We will discuss this framework in more detail later. For now, access tokens represent the authorization granted to a client (such as a web application or API consumer) to access specific resources on behalf of a user. Access tokens are issued by an authorization server, such as IdentityServer, Microsoft Azure Entra ID, or Auth0, after the client has successfully authenticated and received user consent.



Access tokens can be bearer tokens, but not all bearer tokens are used as access tokens. Access tokens typically follow strict standards (e.g., RFC 6749 and RFC 6750) and are accompanied by metadata such as the following:

- **Scope:** Specifies what access is allowed (e.g., `read:appointments,write:invoices`)
- **Audience:** The intended recipient of the token (e.g., `appointments-api`)
- **Issuer:** The authority that generated the token
- **Expiration time:** Ensures that tokens are short-lived
- **Subject:** The identity of the user or client

The reason access tokens are often preferred ahead of generic bearer tokens is that they formalize and structure the access control model. They are tightly bound to the OAuth 2.0 specification and integrate seamlessly with IdPs that support scopes, delegated authorization, and resource servers.

Now that we have explored why we need access and bearer tokens and how they are used, let's review how to implement token security in our ASP.NET Core API application.

## Implementing access token security

ASP.NET Core offers native authentication and authorization support through its `Identity` library. This library provides direct integration with Entity Framework Core, enabling us to create standard user management tables in the target database. We can also further specify the authentication methods that we prefer and define policies that define authorization rules throughout the application.

This robust library has built-in support for the following:

- **User registration:** The user manager library provides functions that simplify user creation and management. It features functions that cover the most common user management operations.
- **Login, session, and cookie management:** The sign-in manager library has functions that can manage user authentication and session management scenarios.
- **Two-factor authentication:** Identity Core allows us to implement multi-factor authentication natively with email or SMS. This can be easily extended.
- **Third-party identity providers:** Social logins are essential for any modern web application, and Identity Core makes it easy to integrate this feature into your application.

Securing an API using bearer tokens ensures that each API call requires a valid token in the header section of the request. An HTTP header allows for additional information to be provided with an HTTP request or response.

In our case of securing an API, we require that each request include an Authorization header containing the bearer token. Our API will assess the incoming request headers, retrieve the token, and validate it against the predefined configurations. If the token doesn't meet the standards or has expired, an HTTP 401 Unauthorized response will be returned. If the token meets the requirements, then the request will be fulfilled. This built-in mechanism makes it easy and maintainable to support wide-scale and robust authentication and authorization rules in our application.

In this example, we will secure our existing appointments microservice. You can navigate to the appropriate directory and use the following CLI commands to add these new packages:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

The first package supports direct integration between Entity Framework Core and Identity Core. The second package contains extended methods that allow us to implement token generation and validation rules in our API configuration.

We must also modify AppointmentContext to inherit from IdentityDbContext:

```
public class AppointmentContext : IdentityDbContext
{
    /* Shortened for brevity */
}
```

This adjustment enables us to generate a new migration that will now add the Identity Core tables to our database, providing out-of-the-box support for authentication and authorization-related activities. Use the following commands to perform the migration and update the database accordingly:

```
dotnet ef migrations add AddedIdentityTables
dotnet ef database update
```

We will also need to ensure that our API is aware of our intention to support authorization, so we must add the following lines to the Program.cs file:

```
builder.Services.AddAuthorization();
builder.Services
    .AddIdentityApiEndpoints<IdentityUser>()
    .AddEntityFrameworkStores<AppointmentContext>();
```

This not only registers the authorization services but also initializes support for our identity-based services, registers default routes that map to identity-related operations, and specifies the data content that should be used by identity for data storage.

We also need to include our middleware to map the identity routes. After the call to `builder.Build()`, place the following code:

```
app.MapIdentityApi<IdentityUser>();
```

This finalizes the mapping of identity-related API endpoints. We can then confirm that the authorization middleware is also being initialized:

```
app.UseAuthorization();
```

With these modifications in place, we get a suite of identity-related endpoints that facilitate user management. To test, we will register a user. Add the following block to the `api-requests.http` file in the project:

```
###
# Register
POST http://localhost:5107/register
Content-Type: application/json
{
  "username": "test@email.com",
  "email": "test@email.com",
  "password": "P@ssword123"
}
```

This represents the register endpoint and the request payload for registering a new user with the email address `test@test.com` and a password of `P@ssword123`. You may also complete this test using Postman or a similar API testing tool. This endpoint returns a `200 OK` response, and you can confirm that a new user is now in the `AspNetUsers` table.

Now, we can test the login using the following request body:

```
###
# Login to the system
POST http://localhost:5107/login
Content-Type: application/json
{
  "email": "test@test.com",
```

```
"password": "P@ssword123"
}
```

This returns a 200 OK response, with the following information:

```
{
  "tokenType": "Bearer",
  "accessToken": "CfDJ8ImTP4HvH2ZPqpmH9gjeoDExk6-pNg...",
  "expiresIn": 3600,
  "refreshToken": "CfDJ8ImTP4HvH2ZPqpmH9gjeoDFOGsza2..."
}
```

We now have a bearer token that will expire in one hour and a refresh token, which can be used with the /refresh endpoint to obtain a fresh access token.

If you need to implement customized authentication endpoints, you can create your API controller and endpoint methods and implement custom logic tailored to your specific needs.

While this out-of-the-box solution is convenient and easy to implement, it is not ideal for several scenarios. We will review the limitations next.

## Limitations of bearer tokens

JWTs are typically validated locally, without requiring contact with the issuing authority. This is excellent for performance but introduces fragility in dynamic environments. For example, if a user's role changes or if their access is revoked, services have no way of knowing until the token expires. This poses a risk in systems where access to protected resources must be updated in real time. Revocation is essentially unsupported unless a centralized revocation list is implemented, which reintroduces a form of shared state, undermining the stateless design that made JWTs attractive in the first place. In effect, bearer token validation per service provides speed at the cost of real-time access control.

Another feature of tokens is that they are inherently self-contained and do not provide a native mechanism for implementing delegation. For example, consider a scheduling application within our healthcare management system that needs to contact the billing service on behalf of a doctor who has just created a new patient visit. With bearer tokens, the service must either forward the original user token, which may expose broader permissions than needed, or act on its authority, which breaks the principle of least privilege. There is no built-in mechanism for asserting *"This service is acting on behalf of this user, for this specific purpose."* In the absence of delegation-aware security models, service-to-service interactions become opaque and potentially over-permissive, making it challenging to enforce fine-grained, context-aware authorization.

When each microservice is responsible for handling its authentication logic, a significant burden is placed on the service itself. Each service must implement token parsing, validation, expiration checks, signature verification, and, if applicable, scope enforcement. While this logic may seem straightforward at first, it must be replicated consistently across every service in the system. Bearer token authentication per service also fails to provide centralized identity management, auditing, or user consent. In regulated industries such as healthcare, users must explicitly consent to data access or sharing. Services using independent token validation have no way to track or enforce such consent, nor do they have a unified view of the user's identity across services.

These challenges manifest quickly as microservices-based applications scale beyond a few endpoints and roles, increasing the need for a more robust, standards-based identity and authorization framework that centralizes control while supporting distributed enforcement. This is where **Open Authorization (OAuth) 2.0** and **OpenID Connect** come into play. We will review these next.

## Using OAuth and OpenID Connect to secure microservices

Having established the limitations of per-service bearer token authentication, we now turn to a more robust and industry-accepted approach: implementing centralized authentication and authorization using **OAuth 2.0** and **OpenID Connect (OIDC)**. These protocols form the foundation of modern identity and access control in distributed systems, enabling secure, scalable, and flexible authorization flows across multiple clients and services.

The emergence of OAuth 2.0 and OIDC was not merely a technological shift, but a response to the changing nature of application identity, user consent, and delegated access. These protocols now serve as the de facto standards for secure, federated identity in the modern web and are core to securing microservices, mobile clients, and API-driven ecosystems.

The first version of the OAuth protocol was released in 2007 by a group of developers from Twitter, Ma.gnolia (now known as Gnolia), and other platforms. OAuth 1.0 was designed to solve the “delegated authorization” problem, allowing a third-party app to act on behalf of a user without requiring their username or password. It used cryptographic signing (HMAC-SHA1) to secure API requests and required applications to register and be issued client keys and secrets.

OAuth 2.0 was released in 2012 by the IETF OAuth Working Group (RFC 6749), introducing a more developer-friendly and extensible model. It separates the act of authorization from authentication. It allows a client to obtain an access token, which it can use to call protected APIs. This token represents a set of permissions (scopes) granted by the user or resource owner.

The protocol also defines several **grant types** (or flows), which determine how the client obtains the token:

- **Authorization Code:** Used by web apps; exchanges a temporary code for a token
- **Implicit:** Used by browser-based apps (now discouraged due to security concerns)
- **Resource Owner Password Credentials:** Allows passing a username/password directly (discouraged)
- **Client Credentials:** Used for machine-to-machine authentication
- **Refresh Token:** Used to obtain a new access token without requiring re-authentication of the user

While OAuth 2.0 provided a powerful solution for access control and delegation, it intentionally did not address authentication. This gap led to the widespread misuse of OAuth as a login protocol, for which it was never intended.

To bridge the gap between authentication and authorization, the OpenID Foundation introduced OIDC in 2014. OIDC is an identity layer built on top of OAuth 2.0 that provides standard methods for logging in, retrieving user identity, and obtaining profile information. OIDC utilizes OAuth's authorization flows to authenticate the user and then issues an ID token, a JWT that contains claims about the authenticated user. It also supports discovery, allowing clients to fetch configuration metadata about the IdP, and dynamic client registration, enabling clients to register with IdPs at runtime.

OAuth 2.0 and OIDC represent the culmination of more than a decade of work in the identity and access management space. They address the limitations of early login models, enable secure delegation of access, and support robust identity federation.

Now, let's review popular OAuth and OIDC solutions, along with their pros and cons.

## Identity providers

An IdP is a specialized service that authenticates users and issues digital identities in the form of tokens. It is the trusted authority responsible for the following:

- Verifying the identity of users or clients
- Managing user credentials, profiles, and roles
- Issuing, validating, and revoking authentication tokens
- Enabling **single sign-on (SSO)** across systems and applications

The core value of an IdP lies in **centralized identity management**. By outsourcing authentication to a single, hardened endpoint, applications no longer need to manage user credentials, password policies, or account lockout mechanisms. This reduces the attack surface and enables teams to invest in a single, secure identity service, rather than duplicating efforts across multiple services.

By abstracting authentication, applications become agnostic to the authentication mechanism. You can replace or upgrade the identity system without needing to rewrite internal services. This decoupling enhances maintainability, particularly as systems become increasingly complex.

Modern IdPs adhere to open protocols, such as OAuth 2.0, OIDC, and **Security Assertion Markup Language (SAML)**. This enables broad compatibility with third-party tools, cloud services, and mobile platforms. The use of well-defined protocols also fosters interoperability, auditing, and compliance with industry security standards. IdPs enable SSO across multiple applications and services. Users authenticate once and receive tokens that can be used across various components of the system. This is particularly valuable in enterprise or regulated environments, where users may need to interact with multiple services without having to reauthenticate for each one.

Identity management providers generally fall into two primary categories:

- **Enterprise IdPs** – Designed to manage identity and access within an organization, offering centralized control over authentication, authorization, and user life cycle. These are commonly used in corporate environments to enforce policies, integrate with internal systems, and support enterprise-grade security requirements.
- **Social IdPs** – Manage user identities tied to personal accounts on platforms such as Google, Facebook, or other cloud-based services. These providers are frequently used to enable quick and convenient login for consumer-facing applications without requiring the user to create a new account.

IdPs include cloud-based options such as Okta and Microsoft Entra ID (formerly Azure Active Directory), on-premises solutions such as Duende IdentityServer, and social providers such as Google and Facebook.

We will use IdentityServer as our implementation example. **IdentityServer** is an open source framework built on ASP.NET Core that enhances the capabilities of Identity Core, enabling developers to implement OIDC and OAuth 2.0 standards in their web applications. It complies with industry standards and features built-in support for token-based authentication, SSO, and API access control within your applications. Though it is a commercial product, a community edition is accessible for small organizations or personal projects.

The recommended implementation style of IdentityServer would have us do the following:

- Create a new microservice for authentication
- Create a new database for just our authentication-related tables (optional)
- Configure scopes to be included in the token information
- Configure our services to know which scopes are allowed to access them

Figure 13.2 shows the IdentityServer authentication flow.

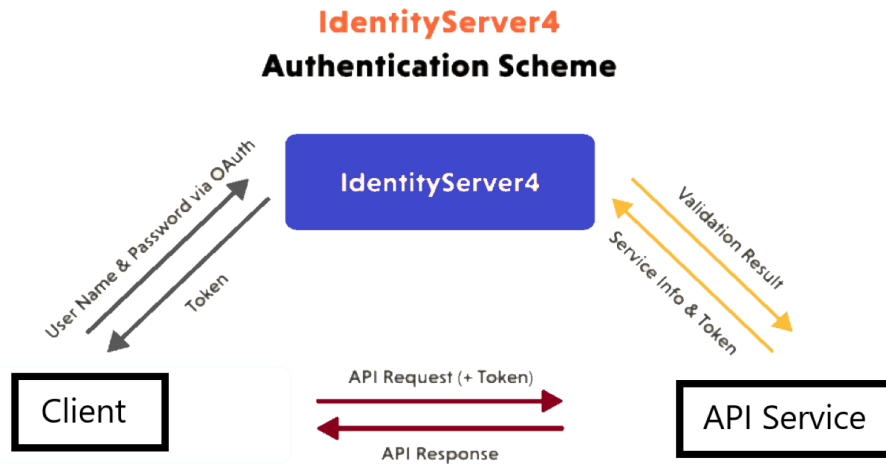


Figure 13.2 – This depicts how IdentityServer sits between a client and service and handles the flow of authentication and token exchange

Now, let's explore creating a new service and configuring it to serve as our central authority for authentication and authorization in our microservices application.

## Implementing token authentication

Duende offers us some quick-start ASP.NET Core project templates that are easy to create in our solution. These quick-start templates bootstrap the minimum requirements needed to bootstrap IdentityServer functionality in an ASP.NET Core project. The general steps involved in setting up an IdentityServer service are as follows:

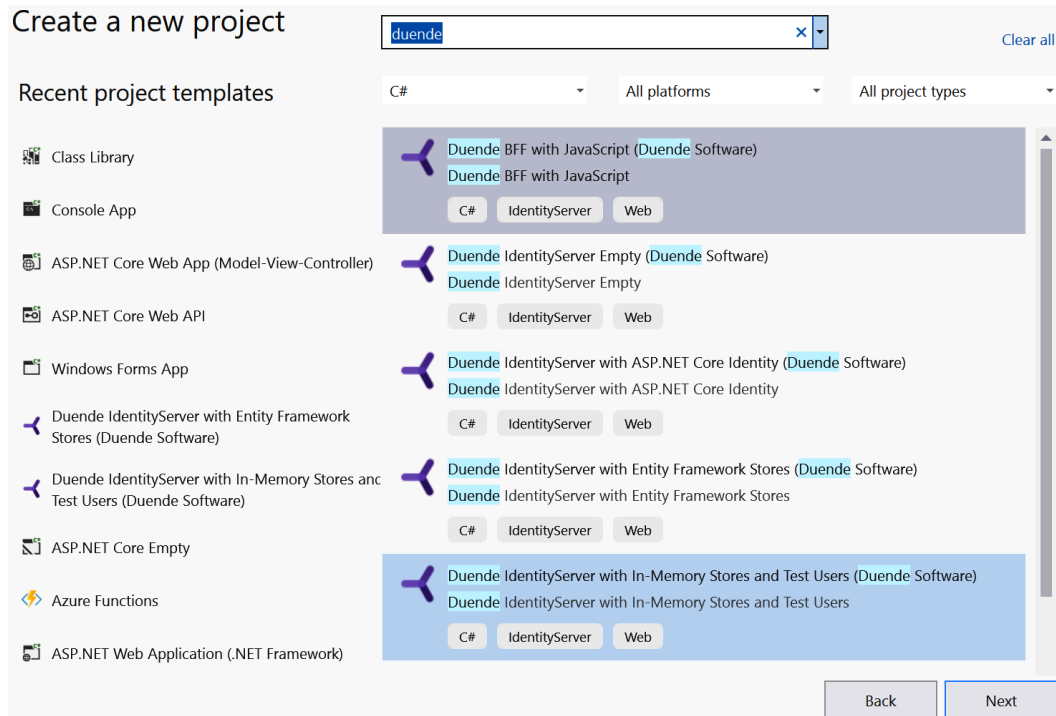
1. Add Duende IdentityServer support to a standard ASP.NET Core project
2. Add data storage support, preferably using Entity Framework configurations
3. Add support for ASP.NET Identity Core
4. Configure token issuing for client applications
5. Secure client applications with IdentityServer



To get started, we need to use the dotnet CLI and run the following command:

```
dotnet new install Duende.IdentityServer.Templates
```

That command will now give us access to new project templates prefixed with Duende IdentityServer. *Figure 13.3* depicts what we can expect to see in Visual Studio once these templates are installed.



*Figure 13.3 – We get a variety of project templates that help us speed up the IdentityServer implementation process*

This will also register new CLI commands, as follows:

- `isapi`: IdentityServer with in-memory storage (API-first)
- `isui`: IdentityServer with UI and in-memory configuration
- `isef`: IdentityServer with UI and EF Core stores

Using our healthcare microservices application, let us start off by adding a new *Duende IdentityServer with Entity Framework Stores* project to handle our authentication. Using the following CLI command in the existing solution folder, create a new project called `HealthCare.Auth` using this command:

```
dotnet new isef -n IdentityServerEf -o HealthCare.Auth
```

When prompted to seed the project, select ‘Y’ or ‘yes’. Now we have a preconfigured `IdentityServer` project with several moving parts. It is essential for us to understand the significant components and appreciate how we can utilize them to meet our needs. Let us conduct a high-level review of the file and folder structure that we get out of the box:

- `wwwroot`: A standard folder that is shipped with ASP.NET Core web application templates. It stores static assets, such as JavaScript and CSS files, that are used on the website.
- `Migrations`: Stores preset migrations that will be used to populate the data store with supporting tables. This is handy as it removes the need for us to create the databases.
- `Pages`: Stores default Razor pages used to support the UI requirements of user authentication operations. Out of the box, we get login, register, grants, and some user data management pages.
- `appsettings.json`: Standard `appsettings.json` file that contains logging and database connection configurations. We can modify this connection string to reflect our requirements better.
- `buildschema.bat`: Contains Entity Framework commands using `dotnet` command-line commands (`dotnet ef`) that will run migration scripts that have been included in the `Migrations` folder. We will use these commands to create our databases.
- `Config.cs`: A static class that serves as a configuration authority. It is used to outline `IdentityResources`, `Scopes`, and `Clients`:
  - `IdentityResources`: Map to scopes that grant access to identity-related information. The `OpenId` method supports the expected *subject* (or *sub*) claim value; the `Profile` method supports additional claim information such as `given_name` and `family_name`. We can also extend the default offerings and include additional details such as the user’s *roles*.
  - `Scopes`: Can be used to outline permissions that can be included in the token when it is issued.
  - `Clients`: Third-party clients that we expect to use the `IdentityServer` as a token issuing authority.

- `HostingExtension.cs`: Contains service and middleware registration extension methods. These methods are then called in the `Program.cs` file during startup.
- `Program.cs`: Primary program execution file in an ASP.NET Core application.
- `SeedData.cs`: Contains default methods that will ensure that data migrations and seeding operations are carried out at application startup.

IdentityServer uses two database contexts: a configuration store context and an operational store context. As a result, two database contexts are being created in the `HostingExtension.cs` file. The IdentityServer libraries are registered using the following code:

```
var isBuilder = builder.Services
    .AddIdentityServer(options =>
    {
        options.Events.RaiseErrorEvents = true;
        options.Events.RaiseInformationEvents = true;
        options.Events.RaiseFailureEvents = true;
        options.Events.RaiseSuccessEvents = true;
        options.EmitStaticAudienceClaim = true;
    })
    .AddTestUsers(TestUsers.Users)
    .AddConfigurationStore(options =>
    {
        options.ConfigureDbContext = b =>
        b.UseSqlite(connectionString, dbOpts => dbOpts
            .MigrationsAssembly(typeof(Program).Assembly.FullName));
    })
    .AddOperationalStore(options =>
    {
        options.ConfigureDbContext = b =>
        b.UseSqlite(connectionString, dbOpts =>
            dbOpts.MigrationsAssembly(typeof(Program).Assembly.FullName));
        options.EnableTokenCleanup = true;
        options.RemoveConsumedTokens = true;
    });
```

We are adding `TestUsers` to the configuration and then adding `ConfigurationStoreDbContext` and `OperationalStoreDbContext`. Other settings govern how alerts and tokens are handled. The defaults are generally solid, but you may modify them based on your specific needs.

The default connection string and Entity Framework Core libraries give us support for an SQLite database. This can be changed to whatever the desired data store may be, but we will continue with SQLite for the purposes of this exercise. If you opted to seed the database at creation time, you will see a file called `IdentityServer.db`. This SQLite database file contains all the required tables.

Suppose you prefer to use a different database technology. In that case, you will need to install the appropriate EF Core packages, generate new migrations, and update the database with the following commands:

```
dotnet ef database update --context PersistedGrantDbContext
dotnet ef database update --context ConfigurationDbContext
```

Let us start by configuring `IdentityResources` in the `Config.cs` file. This will allow us to set the information that we intend to support in our tokens. We can modify the `IdentityResources` method like this:

```
public static IEnumerable<IdentityResource> IdentityResources =>
    new IdentityResource[]
    {
        new IdentityResources.OpenId(),
        new IdentityResources.Profile(),
        new IdentityResource("roles", "User role(s)",
        new List<string> { "role" })
    };
```

We have added the list of roles to the resources list. Based on the claims being accounted for, we need to ensure that our users contain all their expected data, as well as the list of claims they are expected to have. Bear in mind that claims are the information that a client application will have via the token, since it is the only way a client can track which user is online and what they can do.

Now, we can refine the list of scopes that we are supporting by modifying the `ApiScopes` method like this:

```
public static IEnumerable<ApiScope> ApiScopes =>
    new ApiScope[]
    {
        new ApiScope("healthcareApiUser", "HealthCare API User"),
        new ApiScope("healthcareApiClient", "HealthCare API Client"),
    };
```

Here, we support two types of authentication scopes. These scopes will be used to support authentication for two different scenarios: client and user. Client authentication represents an unsupervised attempt to gain access to a resource, usually by another program or API. Client authentication refers to the process by which a user verifies their identity using credentials.

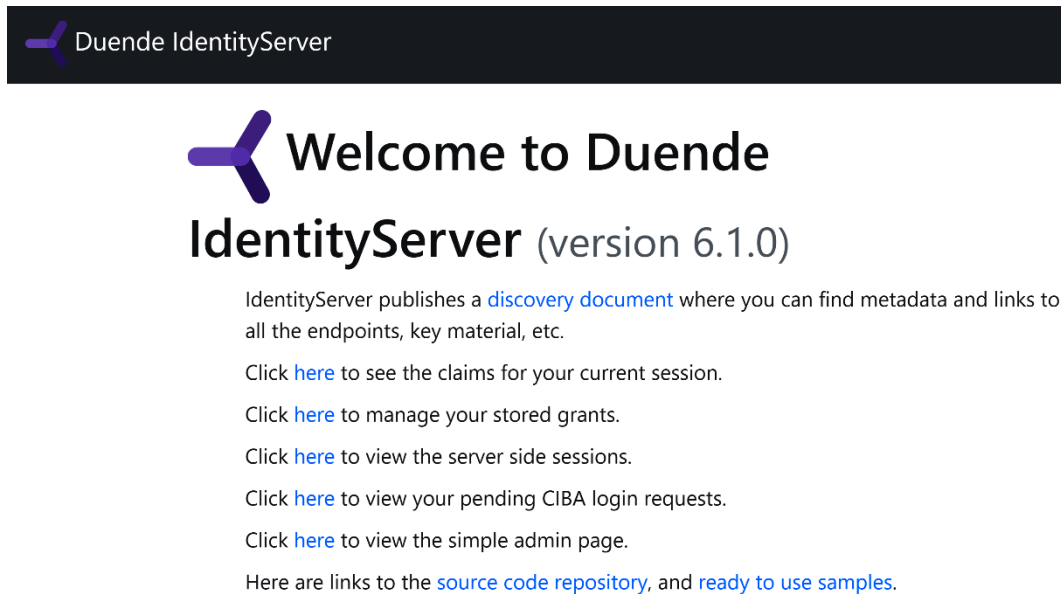
This brings us to the following configuration, which is for the clients. The term “client” may be used somewhat loosely, as any entity that attempts to gain authorization from IdentityServer is considered a client. The word “client” can also refer to a program that is trying to gain approval, such as a daemon or background service. Another scenario occurs when a user attempts to perform an operation that requires them to authenticate against IdentityServer. We extend our support to our clients in this manner:

```
public static IEnumerable<Client> Clients =>
    new Client[]
    {
        // m2m client credentials flow client
        new Client
        {
            ClientId = "m2m.client",
            ClientName = "Client Credentials Client",
            AllowedGrantTypes = GrantTypes.ClientCredentials,
            ClientSecrets = { new Secret("511536EF-F270-4058-80CA-
                                   1C89C192F69A ".Sha256()) },
            AllowedScopes = { "healthcareApiClient" }
        },
        // interactive client using code flow + pkce
        new Client
        {
            ClientId = "interactive",
            ClientSecrets = { new Secret("49C1A7E1-0C79-4A89-A3D6-
                                   A37998FB86B0".Sha256()) },
            AllowedGrantTypes = GrantTypes.Code,
            RedirectUri = { "https://localhost:5001/signin-oidc" },
            FrontChannelLogoutUri = "https://localhost:5001/signout-
                                   oidc",
            PostLogoutRedirectUri = {
                "https://localhost:5001/signout-callback-oidc" },
            AllowOfflineAccess = true,
        }
    }
```

```
AllowedScopes = { "openid", "profile",  
                  "healthcareApiUser", "roles" }  
},  
};
```

We have now defined the `ClientId` and `ClientSecret` values for our clients. By defining several clients, we can support the applications we expect to help at a more granular level, and we can define specific `AllowedScopes` and `AllowedGrantTypes` values. In this example, we have defined a client for an API, which can represent a microservice in our application that might need to authenticate with the authentication service. This type of authentication generally occurs without user interaction. We also define a web client, which could be a user-facing application. This presents a unique challenge in configuring sign-in and sign-out URLs to redirect our users during the authentication or logout flow. We also specify which scopes will be accessible via the generated token. We have added the "roles" value to the `AllowedScopes` list as we want that information to be included when a user authenticates.

We can now launch our new project using the `dotnet run` command and navigate to `https://localhost:5001` in a browser application. This will launch a page similar to the one depicted in *Figure 13.4*. This is the home landing page and shows that `IdentityServer` is up and running.



*Figure 13.4 – This landing page shows us that our IdentityServer application is running*

You can find the `TestUsers.cs` file in the `Pages` folder. We will use `alice` as both username and password for a quick test. You may proceed to use the credentials that have been provided in your instance of that file. We can then proceed to test a login operation using one of the test users that was added to the context by default, and we will be required to authenticate when attempting to access most of these links.

The most critical link to discuss is the one that leads to the discovery document. Most OAuth 2.0 and OIDC service providers have a concept of a discovery document, which outlines the built-in routes in the API, supported claims, and token types, as well as other key bits of information that make it easier for us to know and access these intricate details from `IdentityServer`. Some of the key information available in this discovery is as follows:

```
{
  "issuer": "https://localhost:5001",
  "jwks_uri": "https://localhost:5001/.well-known/openid-configuration/jwks",
  "authorization_endpoint": "https://localhost:5001/connect/authorize",
  "token_endpoint": "https://localhost:5001/connect/token",
  "userinfo_endpoint": "https://localhost:5001/connect/userinfo",
  "end_session_endpoint": "https://localhost:5001/connect/endsession",
  "check_session_iframe": "https://localhost:5001/connect/checksession",
  "revocation_endpoint": "https://localhost:5001/connect/revocation",
  "introspection_endpoint": "https://localhost:5001/connect/introspect",
  "device_authorization_endpoint":
    "https://localhost:5001/connect/deviceauthorization",
  "backchannel_authentication_endpoint":
    "https://localhost:5001/connect/ciba",
  ...
  "scopes_supported": [
    "openid",
    "profile",
    "roles",
    "healthcareApiUser",
    "healthcareApiClient",
    "offline_access"
  ],
  ...
}
```

We have a clear outline of the various endpoints now available for the different commonly accessed operations.

Next, we can test our `HealthCare.Auth SSO API` application and validate that we can retrieve a valid token. Let us attempt to recover a token using our machine client credentials. We will use an API testing tool, such as **Postman**, to send the request. *Figure 13.5* illustrates the Postman user interface and the necessary information to be added accordingly.

The screenshot shows the Postman interface for configuring an OAuth 2.0 token request. The URL bar at the top shows `https://localhost:5001/connect/token`. Below the URL bar, the **POST** method is selected, and the same URL is entered in the field. The **Authorization** tab is active, showing the **Type** as **OAuth 2.0**. The **Add authorization data to** dropdown is set to **Request Headers**. The **Configuration Options** section is expanded, showing the following fields:

- Token Name:** Token
- Grant Type:** Client Credentials
- Access Token URL:** `https://localhost:5001/connect/token`
- Client ID:** `m2m.client`
- Client Secret:** `511536EF-F270-4058-80CA-1C89C192F...`
- Scope:** `e.g. read:org`
- Client Authentication:** Send client credentials in body

At the bottom of the configuration section, there is a **Get New Access Token** button.

*Figure 13.5 – Here, we add the client ID, client secret, and token URL values in Postman to retrieve a bearer token*

Please note that if you are using Postman, ensure that *SSL verification* is disabled to complete the test successfully in development. Once we have added the required values, we proceed to click the **Get New Access Token** button. This will send a request to `IdentityServer`, which will validate the request and return a token if the information is found in the database.



Our token response automatically includes additional information, such as the token type, expiry timestamp, and scope. Our token is generated with several default data points. Since IdentityServer follows the OAuth and OIDC standards, we can be sure that we do not need to include basic claims such as `sub`, `exp`, `jti`, and `iss`, to name a few.

The values that are included as part of `scope` and `client_id` are determined by the configurations we have for each client and the information presented by the authenticating user. In this example, we are accommodating APIs that should only be accessible to authenticated users. *Figure 13.6* shows the payload of the token:

PAYLOAD: DATA

---

```
{
  "iss": "https://localhost:5001",
  "nbf": 1668216814,
  "iat": 1668216814,
  "exp": 1668220414,
  "aud": "https://localhost:5001/resources",
  "scope": [
    "healthcareApiClient"
  ],
  "client_id": "m2m.client",
  "jti": "FF86E33B7756615B609A61109BE0D4C8"
}
```

*Figure 13.6 – Our token automatically contains some claims that we would have entered manually if it were generated without IdentityServer*

Let us save our bearer token value that was returned, as we will use it in our next section. Now, let's review the changes necessary to protect an API using IdentityServer.

## Securing an API using IdentityServer

We now face the unique challenge of implementing the most effective security solution across our microservices application. We have several services that need to be secured, and based on the architecture pattern you have implemented, you might also have a gateway that is routing traffic:

- **Securing each service:** Securing each service seems simple enough, but we must bear in mind that each service has different requirements and may need to be treated as a distinct client for each request. This can lead to a maintenance nightmare when trying to manage all the scopes and clients related to each service. We also need to consider how services

will communicate, as a token will be required for service-to-service calls. One service's claims and scopes might not be sufficient for this communication. This may result in users having to authenticate multiple times when accessing different features that rely on distinct services.

- **Secured API gateway:** Securing our API gateway is the most sensible approach. Suppose we implement a gateway that all apps will communicate with. In that case, we allow the gateway to orchestrate the authentication flow for the client and then manage the token to be shared between service calls. This support can be implemented in a custom-written API gateway and is supported by most, if not all, third-party gateway service providers. This is especially useful when combined with the *Backend for Frontend* pattern.

We have already seen how we can add JWT bearer protection to our API using functionality from the Identity Core library. We can leverage some of these configurations and override the native functionality with support for IdentityServer. Let's explore how we can secure our appointment API using IdentityServer.

We have already added the necessary packages to support JWT authentication, allowing us to modify the Program.cs file and add the following configuration:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://localhost:5001/";
        options.Audience = "appointments_api";
        options.TokenValidationParameters.ValidateAudience = true;
        options.TokenValidationParameters.ValidTypes = new[] {
            "at+jwt" };
    });
```

This configuration will instruct our service to refer them to the URL in the Authority option for authentication instructions.

We can now protect our API by implementing a global authorization policy to ensure that no endpoint can be accessed without a valid bearer token issued by IdentityServer:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAuth", policy =>
    {
```

```

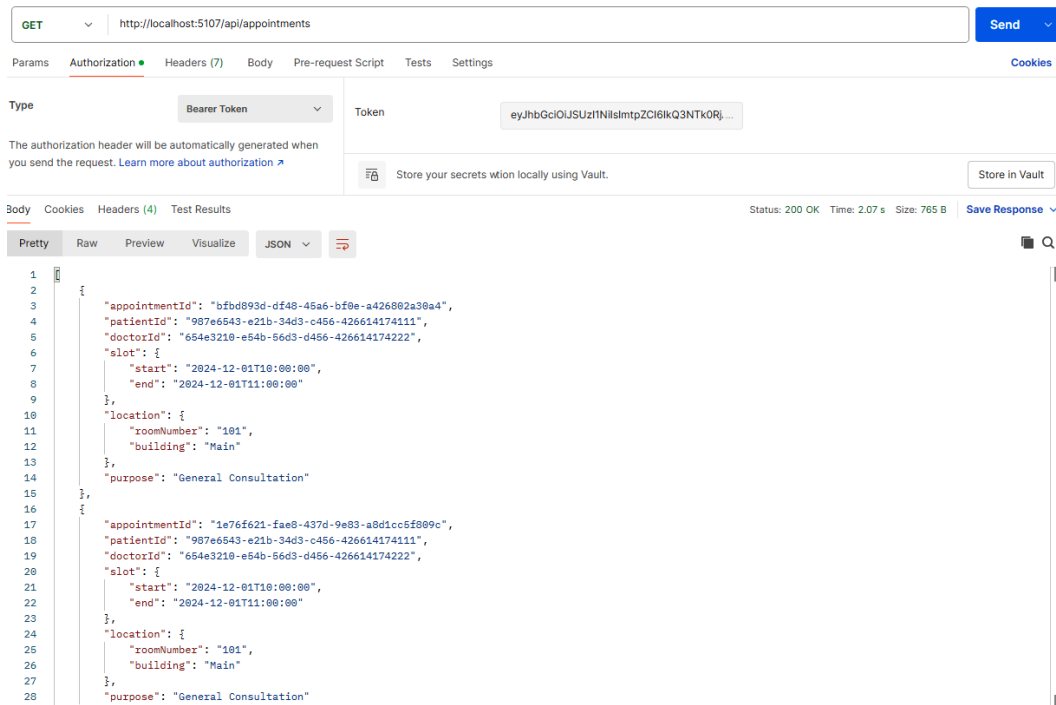
        policy.RequireAuthenticatedUser();
    });
});

```

And we modify the controller's middleware like this:

```
app.MapControllers().RequireAuthorization("RequireAuth");
```

Now, any attempt to interact with our appointments API endpoint will return a 401 Unauthorized HTTP response. The API is now expecting us to provide the bearer token in the Authorization header value. In *Figure 13.7*, we see how authorized API calls can be made to our appointment API endpoints using the bearer token retrieved in the previous section from our machine client credentials authentication.



*Figure 13.7 – Our bearer token is included in the request to our protected service, and we can comfortably access endpoints*

Now, we need to configure our API to force authentication and rely on the HealthCare.Auth service accordingly. If we reuse our Appointments API, we can make a few modifications to the Program.cs file and introduce reliance on our authentication service.

We begin by modifying the `builder.Services.AddAuthentication()` registration like this:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer("Bearer", opt =>
    {
        opt.RequireHttpsMetadata = false;

    });
```

Now that we have secured our API directly, we can explore how we can manage this new security requirement in our API gateway. Recall that we have implemented aggregation methods, and we will expect client applications to access the endpoints through the gateway.

## Securing the Ocelot API gateway with IdentityServer

Now, when we access an API endpoint protected by IdentityServer, we need to retrofit our gateway service to support authentication and forward the credentials to the target API. We start by adding the `Microsoft.AspNetCore.Authentication.JwtBearer` package using this command:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

When we set up the gateway in *Anchor 5, Implementing the API and BFF Gateway Patterns*, we added an `Authentication` section to the `ocelot.json` file. This already sets the configuration to support bearer authentication, as indicated by the `AuthenticationOptions` section:

```
{
  "DownstreamPathTemplate": "/api/appointments",
  ...
  "AuthenticationOptions": {
    "AuthenticationProviderKey": "Bearer",
    "AllowedScopes": [ "appointments.read", "appointments.write" ]
  },
  ...
},
```

This outlines that two additional scopes are required to access the appointments resources. We need to modify the `Config.cs` file in the `HealthCare.Auth` project to add the additional scopes, as follows:

```
public static class Config
{
```

```

public static IEnumerable<IdentityResource> IdentityResources => ...

public static IEnumerable<ApiScope> ApiScopes =>
    new ApiScope[]
    {
        ...
        // Add these new scopes:
        new ApiScope("appointments.read", "Read access to
            Appointments API"),
        new ApiScope("appointments.write", "Write access to
            Appointments API")
    };

public static IEnumerable<Client> Clients =>
    ...
    // Update to allow the new write scope as well:
    AllowedScopes = { "healthcareApiClient",
        "appointments.write" },
    },
    // interactive client using code flow + pkce
    new Client
    {
        ...
        AllowedScopes =
        {
            "openid",
            "profile",
            "healthcareApiUser",
            "roles",
            "appointments.read", // Add
            "appointments.write" // Add
        }
    },
    };
}

```

These changes allow the machine-to-machine client, which is the service-to-service (client credentials) client that your gateway will use when it needs to call the Appointments API, to add the `appointments.write` claim to `AllowedScopes`. Otherwise, when the gateway goes to fetch a token, that scope will be rejected, and your downstream API calls will fail with “insufficient scope.”

The interactive client represents your browser-based or MVC frontend. It utilizes the authorization code and **Proof Key for Code Exchange (PKCE)** flow, culminating in an access token that can be presented to any downstream API. By adding both `appointments.read` and `appointments.write` here, you allow signed-in users to fetch tokens that carry those claims, and thus your UI can consume read-only endpoints (to list appointments) and write endpoints (to make or modify them) without having separate client registrations.

Now, we modify our `Program.cs` file and register our authentication service to use JWT bearer authentication, as we did on the service itself:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://localhost:5001/";
        options.Audience = "appointments_api";
        options.TokenValidationParameters.ValidateAudience = true;
        options.TokenValidationParameters.ValidTypes = new[] { "at+jwt" };
    });
```

We have now secured our gateway using `IdentityServer`. This, once again, might be a better security solution for our suite of microservices, which will be accessed through the gateway, and it can help us centralize access to our services.

With this simple change, we no longer need to concern our Appointments API with the inclusion of authentication tables in its database or complex JWT bearer compilation logic. We point the service to `Authority`, which is the authentication service, and include the `Audience` value so that it can identify itself to the authentication service.

By registering JWT bearer authentication in our `Program.cs` file, we configure our gateway to validate incoming tokens issued by `IdentityServer`. This integration ensures that only requests with valid access tokens issued by the central `IdentityServer` and containing the correct audience and type are allowed to access protected endpoints.

With this configuration, a user must provide a token, such as the one retrieved previously, to make any calls to our API. If the token is missing, a 401 Unauthorized HTTP response will be returned. If a token is present but lacks the essential scopes or claims, a 403 Forbidden response will be returned.

Configuring IdentityServer is not the most challenging task, but it can become complex when attempting to account for several scenarios, configurations, and clients. There are several considerations to be made along the way.

Now let us summarize what we have learned in this chapter.

## Summary

This chapter presented a comprehensive exploration of security within the context of microservices-based architectures, with a special focus on modern authentication and authorization models implemented in .NET applications. We began by establishing the importance of security as a foundational element of application development, especially in distributed environments. As microservices proliferate across networks and teams, the attack surface increases, necessitating robust, consistent, and scalable security mechanisms.

The discussion began with a comparison between monolithic and microservices architectures in terms of security responsibilities. While monoliths can centralize authentication and authorization due to their unified deployment, microservices require each service to handle these concerns independently. This decentralization creates challenges in managing identity, enforcing consistent policies, and securing inter-service communication. Developers must protect not just the edge of the application but every internal endpoint, ensuring that authorization logic is consistent and scalable.

We then explored key security concepts, including authentication, authorization, data confidentiality, and integrity, explaining the role that each concept plays in securing an application. For example, authentication verifies who the user is, while authorization determines what the user is allowed to do. In microservices, these processes become more nuanced. Unlike traditional session-based security, stateless services rely on bearer tokens, which encapsulate claims about the authenticated entity. Bearer tokens and JWTs were introduced as solutions for stateless authentication, particularly in the context of API security. The use of bearer tokens allows services to validate identity without maintaining session state, thereby enabling scalability and statelessness. However, it also introduces complexity in token management, expiration, and revocation.

Recognizing the limitations of simple bearer tokens, primarily when used independently in each microservice, the chapter progressed toward a more sophisticated model: centralized authentication via OAuth 2.0 and OIDC. These protocols introduce the concept of access tokens, authorization flows, scopes, and claims. They also establish the role of an IdP, a centralized authority that authenticates users and issues tokens. This allows for consistent identity management, SSO, and delegated authorization across multiple services and clients.

The historical evolution of OAuth and OIDC was reviewed, highlighting their significance in modern web security. OAuth 2.0 decouples authentication from authorization and supports various grant types to facilitate machine-to-machine communication, user logins, and token refresh operations. OIDC builds upon OAuth by providing standardized identity claims and user profile information.

We then explored a practical implementation that utilized Duende IdentityServer as a standards-compliant IdP within the .NET ecosystem. Using IdentityServer, developers can implement OAuth 2.0 and OIDC within their microservices applications. We walked through the setup of IdentityServer using ASP.NET Core with Entity Framework stores, demonstrating how to configure clients, scopes, and test users. You were guided through the creation of a dedicated authentication service (`HealthCare.Auth`), its database context, and the registration of identity resources, scopes, and claims. The IdentityServer implementation was demonstrated to be flexible enough to support both machine-to-machine clients and interactive browser clients, utilizing the PKCE and code flow protocols.

We configured the Ocelot API gateway for bearer token protection, enabling it to act as a secure intermediary that authenticates tokens and enforces authorization before forwarding requests downstream. This centralized pattern simplifies the enforcement of access policies and minimizes duplication of security logic.

As we wrapped up the chapter, we emphasized the value of using a central identity authority to manage authentication and authorization in a distributed architecture. This approach mitigates the burden on individual services, promotes consistency, and ensures compliance with industry standards and regulations. After reading this chapter, you now have a foundation for implementing secure microservices and a practical understanding of how to integrate IdentityServer, bearer tokens, and the OAuth 2.0/OIDC protocols into a .NET-based microservices application.

In the next chapter, we will begin to explore the use of containers and orchestration to develop a cloud-native microservices application.



## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW



---

# Part 4

---

## Cloud Development Strategies

This final part brings the runtime story together – containers, sidecars, serverless, and full stack observability – so your services run predictably from laptop to cloud with actionable telemetry.

This part has the following chapters:

- *Anchor 2, Container Hosting and Development Patterns*
- *Anchor 2, Serverless Microservices Development*
- *Anchor 2, Observability and Monitoring with Modern Tools*
- *Anchor 2, Wrapping It All Up*



# 14

## Container Hosting and Development Patterns

As we continue to build robust and secure microservices applications, we now shift our attention from securing services to deploying and managing them. In the previous chapter, we examined strategies for protecting our microservice APIs with authentication and authorization mechanisms, rate limiting, and secure communication. These concerns are critical at the network and service boundaries. However, we must now ensure that the services themselves are hosted in environments that promote portability, resilience, and scalability. This is where containerization comes into play.

Modern microservices architectures demand more than traditional hosting methods can easily deliver. Each service may be written in a different language, rely on other runtimes, or require different infrastructure dependencies. Managing these manually across environments often leads to inconsistencies, configuration drift, and deployment friction. Containers solve these problems by providing lightweight, isolated environments for running applications with all their dependencies bundled inside.

In this chapter, we will explore container hosting as a key enabler of microservices development and the following topics:

- Containers and Docker in microservices
- Authoring Dockerfiles for ASP.NET Core apps
- Building and managing Docker images and containers
- Pushing containers to a registry
- Orchestrating services with Docker Compose and Kubernetes

By the end of this chapter, you will not only understand how to run microservices in containers but also how to build a deployment pipeline that reflects the same principles of autonomy, scalability, and efficiency that your services are designed around.

Let's start by understanding what containers are and how they are reshaping the way modern applications are developed and deployed.

## Technical requirements

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter14>.

You may also download Docker at <https://docs.docker.com/get-started/get-docker/>.

## Using containers in development

Containers are all the rage in the development space. They present us with a lightweight application hosting option that allows us to deploy our applications in a clean and repeatable manner. They are not new, but their use in more commercial and accessible spaces has been made far more popular in recent years. What, however, is a container, and why should we care about how it works? Let us review that next.

Traditionally, whenever we have applications with specific requirements for environments and software, we resort to using servers to facilitate these requirements. The problem with servers and servers per application is that they come with costs. A server machine is generally not cheap, and we must also factor in licensing and energy costs when new machines are introduced. Additionally, consider that if a machine goes down, we will need to reconfigure it to the original environmental specifications and reprovision several aspects of the original deployment.

At this point, we begin to think about virtualization. This means that we now use virtual machines for new servers and reuse the old infrastructure. This will go a long way in reducing the physical infrastructure requirements and costs, allowing us to scale a bit more easily. We can also use snapshots of the machines to maintain a quick recovery plan, on hand in the event of failure. Several virtualization solutions are available on the market, including popular options such as *VMware*, *VirtualBox*, and *Microsoft Hyper-V*. *Figure 14.1* shows a server with several virtual machines:

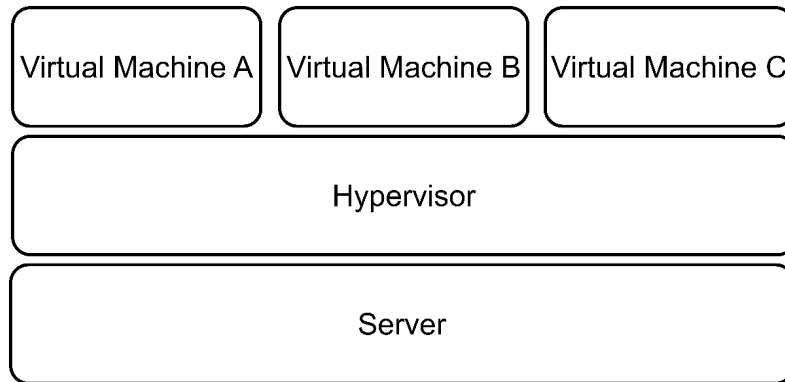


Figure 14.1 – One machine is required to support multiple virtual machines on top of a hypervisor

This visualization approach seems like the silver bullet we need, except we have found more problems with this solution, which are as follows:

- We will still need to factor in that we need powerful machines to be able to handle multiple virtual machines
- We will still have the manual maintenance tasks required to keep our environment and operating systems up to date and secure
- We must remember that there are several situations where we attempt to provide the same environment on different installations and encounter unforeseen differences each time
- We cannot always rely on the machine environment to remain consistent with each instance

Now, this brings us to the most recent solution to this problem: containers. Containers build on the concept introduced by virtual machines, allowing us to reduce the overall space and resource requirements that an application might need. The use of containers to host our applications is called **containerization**, and it is an approach in modern software development that allows us to package an application and all its dependencies, creating a repeatable, testable, and reliable package called an **image**. This image can then be deployed consistently to multiple locations.

This consistency is crucial in distinguishing the benefits of containers from those of virtual machines. We just reviewed that we cannot always be sure that each OS instance of a server or virtual machine will be the same. Containers strip away many of the variables that we contend with during deployments, providing an environment specifically tuned for the application that needs to be deployed. *Figure 14.2* shows a server with several containers and apps:

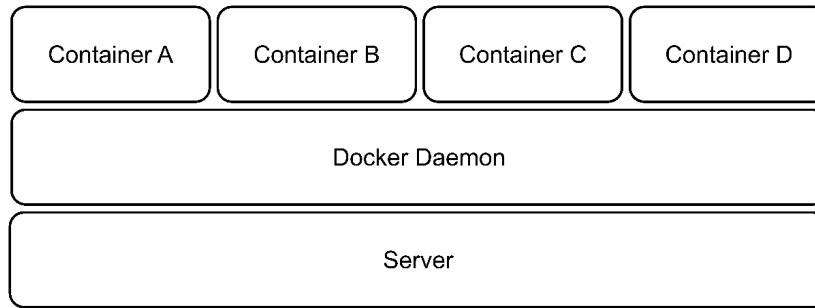


Figure 14.2 – One machine can host several container apps and make better use of its resources

Another benefit, as shown in *Figure 14.2*, is that we can now optimize the resource usage of a physical server, as we no longer need to allocate a fixed amount of RAM, CPU, and storage to entire instances of operating systems.

Containers will also provide isolation from each other on a shared operating system, so we do not need to worry about heterogeneous requirements across containers. In simpler terms, each container is a self-sustained environment that is platform- and operating system-agnostic. Using containers will make our applications more scalable and reliable. We can easily provision new instances of containers as needed and run multiple identical instances.

Containers are not new, but they are becoming increasingly popular for hosting isolated operations in consumer applications and improving efficiency for applications deployed to less powerful machines. To utilize containers, we require a container hosting solution, such as **Docker**. We will review how we can get started with Docker and containers next.

## Understanding Docker

Before we dive into how Docker works and why it's so widely adopted, it's important to note that Docker is not the only tool available for containerization. Several alternatives, including **Podman**, **containerd**, and **CRI-O**, conform to the **Open Container Initiative (OCI)** standards. These standards define a consistent format for container images and runtimes, ensuring interoperability across platforms and tools. Despite the growing ecosystem, Docker remains the most influential and developer-friendly solution, largely credited with bringing containerization into mainstream software development.

Docker is available cross-platform and free for development and open source use. It simplifies the process of packaging applications into portable containers and provides intuitive tools for building, sharing, and managing container instances. Users can interact with Docker either through its graphical interface (e.g., Docker Desktop) or via its robust **command-line interface (CLI)**.

At its core, Docker follows a client-server architecture. The Docker client, typically accessed via the CLI, communicates with the **Docker daemon (dockerd)** over a RESTful API. The daemon runs as a background service on the host machine and is responsible for managing container lifecycles, building and storing images, networking, volume provisioning, and executing containers.

In addition to containers themselves, Docker manages several key objects that support containerized workloads:

- **Networks:** Which define how containers communicate with one another and with the outside world.
- **Volumes:** Which allow containers to persist and share data across restarts.
- **Plugins:** Which extend Docker's core functionality, such as support for custom logging or secret management.

We will go into more details on containers and container images. Still, very briefly, a container is a lightweight, isolated environment designed to run a single application and all its dependencies. It is instantiated from a container image, a static blueprint that defines the filesystem, configuration, and dependencies required to run the containerized application. These images serve as reproducible snapshots of an environment and can be reused consistently across teams and deployment environments.

To facilitate image distribution, Docker offers **Docker Hub**, a widely used public container registry. Docker Hub hosts thousands of prebuilt images for standard software stacks, such as Redis, NGINX, PostgreSQL, and .NET. Developers can also use Docker Hub to publish their images, promoting collaboration and reuse. For organizations operating in cloud environments, private registries such as **Azure Container Registry (ACR)** or **Amazon Elastic Container Registry (ECR)** provide additional features, including fine-grained access control, geo-replication, and native integration with other cloud services.

With a foundational understanding of Docker's architecture and the role of container images, let's now examine how these images are defined, built, and used to create containers.

## Understanding container images

A container image is a self-contained, portable package that defines everything needed to run an application, its executable code, runtime, libraries, environment variables, and configuration files. Think of it as a *blueprint for creating containers*. When we run a container, we are essentially spinning up an instance of this image in memory, with all its pre-defined behaviors and dependencies intact.



One of the most essential characteristics of container images is that they are **immutable**. Once an image is created, it cannot be changed. If we need to update an image, let's say, by adding a new dependency or configuration, we must rebuild it. This immutability ensures consistency across environments: the container image used in development is identical to the one used in staging and production. This eliminates configuration drift and puts to rest the classic excuse: *"It works on my machine."*

Docker images are typically composed of layers, and every image begins from a foundational point, either a base image or a parent image.

A **base image** is the starting point for creating a new image. It can be empty (such as the scratch image), or it may contain only the minimal components needed to run an application. The scratch image is used when building lightweight images from scratch, often in conjunction with statically compiled binaries, such as Go applications or microservices built in .NET using **Ahead-Of-Time (AOT)** compilation.

A **parent image**, on the other hand, is an image that already contains an operating system and potentially additional runtime components. It acts as a foundation for different images. For example, the official `mcr.microsoft.com/dotnet/aspnet:10.0` image includes a minimal Linux-based OS and the ASP.NET Core runtime. You would use this as a parent image when creating a container for an ASP.NET Core web application.

Base image and parent image both refer to the foundational image from which a new container image is created. Whether starting from an empty image that contains no base image instructions or from an image that already provides an operating system and runtime components (such as `mcr.microsoft.com/dotnet/aspnet:10.0` for ASP.NET Core), both terms describe the image used as the starting point for building another image. In essence, base image and parent image mean the same thing—they serve as the blueprint for what your custom image will include.

In a more practical scenario, let's suppose we want to run a Redis cache locally using Docker. We don't need to build Redis from source or manually install the application. Instead, we can use the official Redis image published to Docker Hub, which is a complete and prebuilt parent image based on a lightweight Linux distribution such as Alpine.

For this exercise, ensure that Docker is already installed on your machine. You can download it from <https://www.docker.com> and install Docker Desktop, which includes both the engine and GUI tools. Once Docker is installed, open your terminal or command prompt and run the following command:

```
docker pull redis
```

This command tells Docker to pull the latest official Redis image from Docker Hub, a widely used public container registry that hosts thousands of open source and vendor-supported images. After pulling the image, you can create and run a Redis container like this:

```
docker run --name my-redis-container -d redis
```

This command does the following:

- `--name my-redis-container` gives your container a human-readable name
- `-d` runs the container in detached mode, allowing it to operate in the background
- `redis` specifies the image used to start the container

At this point, you have a Redis instance running on your local machine, isolated within a container, and ready to be accessed by other services or management tools, such as **RedisInsight**.

To stop the container when you're done, run the following:

```
docker stop my-redis-container
```

This illustrates the power of container images: in just a few commands, you've provisioned a fully functional instance of a third-party service, without manual configuration or dependency management.

While using third-party images (such as Redis, NGINX, or PostgreSQL) is valuable, one of Docker's most significant advantages is the ability to package and distribute applications as images. For .NET developers, this means you can build, test, and deploy your ASP.NET Core microservices consistently across all environments.

Before we dive into how to create a custom image using a Dockerfile, let's take a moment to explore the advantages and limitations of containers in microservices development. Understanding these trade-offs will help guide architectural decisions as we continue building our healthcare management system.

## Pros and cons of containers

Containerization offers a significant leap forward in how we build, package, and deploy applications. However, like any architectural decision, it comes with trade-offs that must be weighed carefully.

A key advantage of containers is their environmental consistency. Container images encapsulate not just the application code but also the exact dependencies, configurations, and runtime required for operation. This ensures that the software behaves consistently across a developer's laptop, a

CI/CD pipeline, and a production Kubernetes cluster. Since they can be created, destroyed, and replaced within seconds, this makes them a perfect match for cloud-native principles, such as auto-scaling, rolling updates, and blue/green deployments. Orchestration platforms, such as Kubernetes, leverage this trait to manage large-scale, distributed systems with precision and speed.

Also, unlike virtual machines that replicate entire operating systems and demand significant overhead, containers share the host system's OS kernel and are lightweight in both disk and memory usage. Starting a container takes seconds (or less), making them ideal for elastic workloads and rapid scaling. This also promotes better density on a host machine, allowing more services to run on fewer nodes and reducing infrastructure costs. Once built, a container image can be pushed to a registry and run on any compliant runtime, in any environment, if the host supports containers. This decouples your application from the specifics of the underlying hardware or cloud provider, enabling true multi-cloud and hybrid-cloud strategies.

The ecosystem around Docker and containerization is mature and rapidly evolving. Developers can leverage Docker Compose for local multi-container testing, Docker Desktop for GUI-based management, and various container registries such as Docker Hub, Azure Container Registry, or GitHub Container Registry for distribution.

Despite their many benefits, containers are not a silver bullet. In adopting this model, it's essential to understand where the edges and complexities lie.

All containers on a host share the same underlying OS kernel. This means that a compromised container could potentially exploit a vulnerability in the kernel to affect other containers. While isolation is strong, it is not as robust as that provided by full VMs. This is primarily a concern in multi-tenant environments, where containers from different users or business units run on the same host.

Containers shift the security model. You now must worry about the following:

- Securing the image build process
- Preventing the use of insecure or outdated base images
- Applying runtime controls (via tools such as **AppArmor**, **seccomp**, or **PodSecurityPolicies**)
- Ensuring network policies prevent lateral movement between services

Any oversight in these areas can lead to severe breaches. Using vulnerability scanners such as **Trivy** or **Anchore** and applying image signing and verification becomes critical.

Monitoring containerized environments is more complex than traditional VM-based systems. Since containers are ephemeral, standard log collection and metrics pipelines must be adapted. Logs and metrics must be aggregated centrally since containers don't persist long enough for traditional agents to be helpful. Without proper observability tooling, you may struggle to trace issues in production.

While containers offer a logical abstraction, working with persistent storage, service discovery, and network segmentation introduces new challenges. Stateful containers must be carefully managed using volume mounts or external storage systems. Network traffic must be routed correctly, often requiring overlay networks or service meshes.

Containerization has fundamentally changed the landscape of application development and deployment. It supports the very principles that microservices architectures are built upon: scalability, independence, repeatability, and automation. However, to unlock its full potential, developers must be mindful of its operational implications, especially in production environments.

In the next section, we will explore how to write a custom Dockerfile for your ASP.NET Core microservices. You'll learn how to use base images, define working directories, copy build artifacts, expose ports, and optimize the build process for clean, portable deployments.

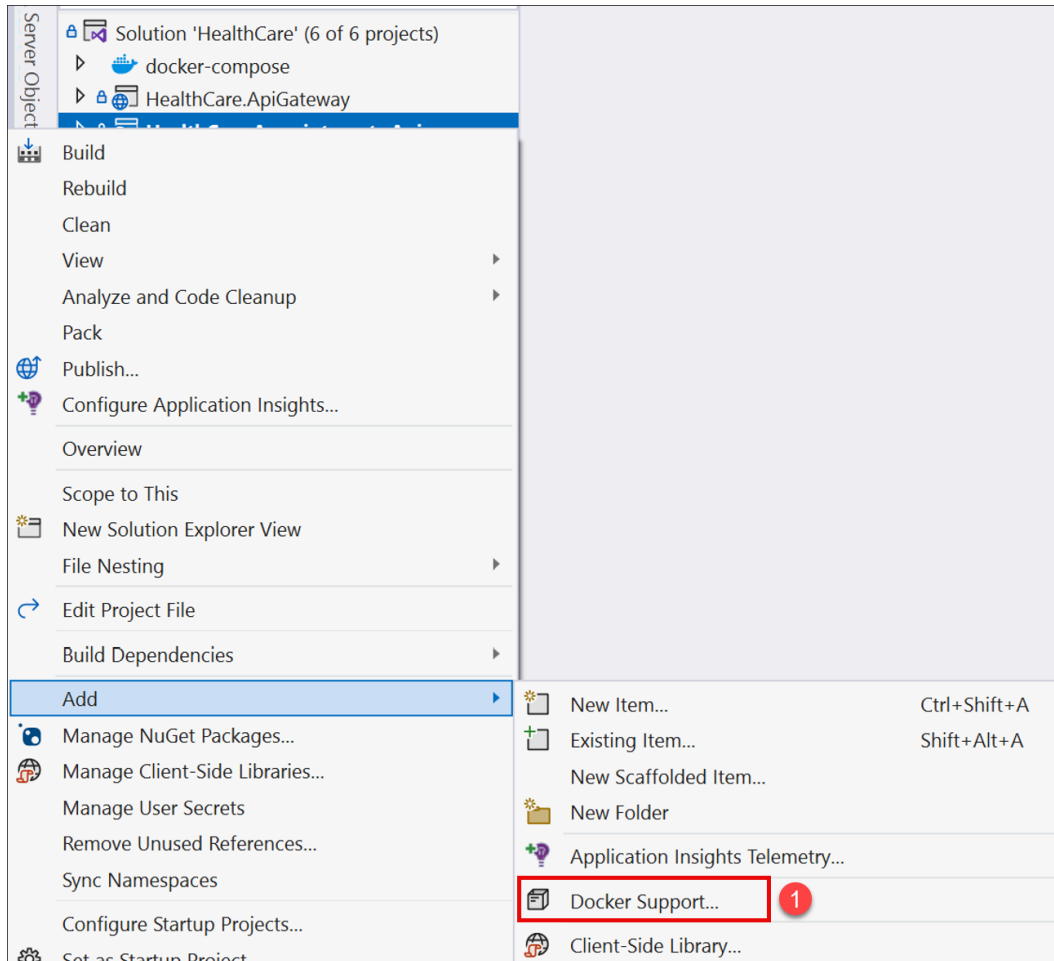
## Working with Dockerfiles and images

A **Dockerfile** is a text file that outlines the steps for building a Docker image. They use a directive-based language that is similar to **YAML (Yet Another Markup Language)**, a popular markup language commonly used for configuration files. This file usually contains the following elements:

- Base or parent image to base the new image on.
- Commands to update the operating system, additional software, and plugins as needed. Note that it is best to start with a suitable base image to minimize the ceremony of including additional plugins and addressing security concerns.
- Compiled application assets to be included in the image.
- Additional container image assets for storage and networking needs.
- Command to run the application when the container launches.

In our case, we are building a microservices-based application with several web services. This means that we need to create Dockerfiles for each web service. Since all our services are based on ASP.NET Core, we can use the following example of a Dockerfile as a base for our appointment web service and others.

To add a Dockerfile to our project, we can use **Visual Studio** and simply right-click our project in **Solution Explorer**, go to **Add**, and then click **Docker Support....** *Figure 14.3* shows the **Docker Support** option.



*Figure 14.3 – Adding Docker support in Visual Studio 2022*

In **Visual Studio Code**, you can install the official Docker extension provided by Microsoft. Once this extension is installed, you can then add Dockerfiles to your workspace by selecting the Docker : Add Docker Files to Workspace command from the command palette. It will then seek to confirm the type of application runtime that we wish to support with our container and confirm whether we need docker-compose support. Once we complete selecting our options, it will proceed to generate our files.

In either case, pay attention to the **Container OS** (Windows versus Linux) and the **Container Image Distro** option. One of the key components of containers is that they should be as lightweight as possible. The OS used (such as **Alpine** versus **Debian**) can have implications for the size of the container, so ensure that you select one that is as useful for your application's needs as it is lightweight.

Both paths will produce two new files in the target project. We receive a Dockerfile and a .dockerignore file. In the case of the appointments API service project, if we complete the preceding steps, we end up with a Dockerfile that looks like this:

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081

# This stage is used to build the service project
FROM mcr.microsoft.com/dotnet/sdk:9.0-alpine AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["AppointmentsApi.csproj", "."]
RUN dotnet restore "./AppointmentsApi.csproj"
COPY . .
WORKDIR "/src/."
RUN dotnet build "./AppointmentsApi.csproj" -c $BUILD_CONFIGURATION
    -o /app/build

# This stage is used to publish the service project to be copied to the
# final stage
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./AppointmentsApi.csproj" -c $BUILD_CONFIGURATION
    -o /app/publish /p:UseAppHost=false

# This stage is used in production or when running from VS in regular mode
# (Default when not using the Debug configuration)
FROM base AS final
```

```
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "AppointmentsApi.dll"]
```

This Dockerfile has instructions that will create an image for the target service and build the container. The instructions execute the following:

1. Use the `mcr.microsoft.com/dotnet/aspnet:v-distro` image as the base from which we will derive the rest of our new image. `v-distro` is a placeholder where `v` represents your project's .NET version and `distro` represents the OS image.
2. Define which ports we wish to expose from our container.
3. Define our content in the image by copying the contents of key files and directories that we need for the app.
4. Perform *build*, *restore*, and *publish* operations to generate the binaries. After the publish run, copy the files from the publish directory to the container space.
5. Define ENTRYPOINT as the executing binary for the project, which will launch our application.

We also get a `.dockerignore` file that outlines files and directories that should not be included in the container when it is created. Its contents look like this:

```
**/.classpath
**/.dockerignore
**/.env
**/.git
**/.gitignore
**/.project
**/.settings
**/.toolstarget
**/.vs
**/.vscode
**/*.proj.user
**/*.dbmdl
**/*.jfm
**/azds.yaml
**/bin
**/charts
**/docker-compose*
```

```
**/Dockerfile*  
**/node_modules  
**/npm-debug.log  
**/obj  
**/secrets.dev.yaml  
**/values.dev.yaml  
LICENSE  
README.md
```

This file is straightforward to understand, and modifying it is generally not required. It simply outlines the different areas of the project file composition that are not deemed necessary to transport to the container once the application is built and deployed.

Our Dockerfile serves as the starting point for our custom container, which will host the target web service. To recap how containers are built, we start with a base image, which serves as the starting point for all subsequent images. In this case, our first line refers to a base image from which we wish to derive our web service container. Note that the origin and base images of the base image we ultimately used are not visible through this process. The truth is, we don't know the hierarchy behind the `mcr.microsoft.com/dotnet/aspnet:10.0` base image. This approach enables us to utilize the various images that have contributed to our current base image, without requiring direct references to them or bloating our file with unnecessary references. We are simply making our derivative henceforth. This helps keep our container images small.

Now, let us explore how we can use this Dockerfile going forward.

## Launching a containerized application

Now that we have done this process for one of our services, we can easily repeat it for the others. By doing this, we would have completely and successfully containerized each of our web services and, by extension, our microservices application. It is noteworthy as well that Visual Studio and Visual Studio Code will always generate the best Dockerfile to suit the project type that you are working with. We can now also enjoy the new launch feature that has been introduced, which allows us to launch our new containerized web service in a Docker container while retaining real-time analytics and debugging features, just as if it were running in a standard debugging environment.



In **Solution Explorer**, we can navigate to the project folder, inspect a folder called **Properties**, and open a file named `launchSettings.json`. This is a JSON configuration file that comes as standard in every ASP.NET Core project. Unless you have a specific reason to do so, you wouldn't normally open or modify this file. It was, however, altered and given a new launch profile, which informs Visual Studio that there is a new way to launch this application for debugging. The file now looks like this.

```
"profiles": {  
  // Existing profiles  
  "Container (Dockerfile)": {  
    "commandName": "Docker",  
    "launchUrl": "{Scheme}://{ServiceHost}:{ServicePort}",  
    "environmentVariables": {  
      "ASPNETCORE_HTTPS_PORTS": "8081",  
      "ASPNETCORE_HTTP_PORTS": "8080"  
    },  
    "publishAllPorts": true,  
    "useSSL": true  
  }  
},
```

This new Docker section was created during our introduction of the Dockerfile, and it now allows us to select **Docker** as a launch option from Visual Studio. This will execute the instructions outlined in the Dockerfile, creating a new image based on the Microsoft base image, building, restoring, and publishing a web project. It will then move the files to a freshly created container and execute the application.

The only significant difference we will see in this experience is that the user interfaces in Visual Studio or Visual Studio Code will display more information about the containers, including their health, version, ports, environment variables, logs, and even the file system used by the container.

*Figure 14.4* shows the Visual Studio UI while using Docker for debugging:

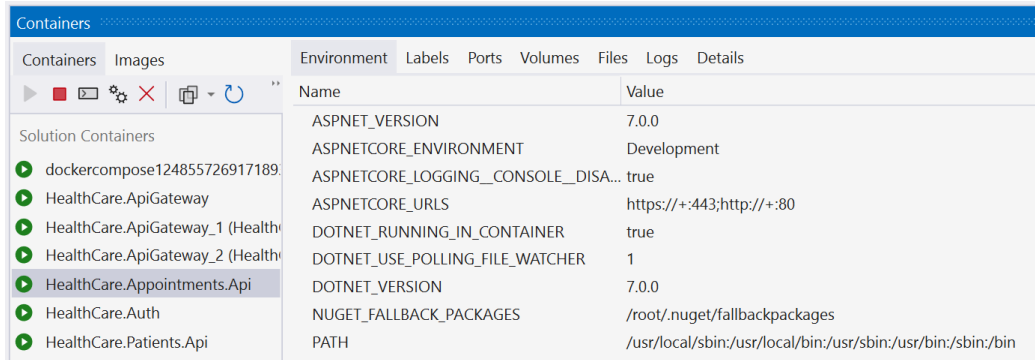


Figure 14.4 – We see Visual Studio in debug mode while containers are in use. It shows us information about the container during runtime.

While we are debugging with our Docker containers, you can notice the containers in Docker's graphical user interface, and they show which ports are available for HTTP traffic. We will use these ports to handle the configured port mapping accordingly. If you want to see the running containers using the CLI, you can run this command.

```
docker ps -a
```

It will produce a list of containers showing their names, ports, and status. Docker has several commands that help us automate our container operations. Docker also has the following commands:

- `docker run` starts or creates a container. It takes a name parameter, which allows you to specify a name that the container will be made as. If a name is not provided, a default one will be assigned.
- `docker pause` will pause a running container and suspend all services and activities.
- `docker unpause` does the opposite of `docker pause`.
- `docker restart` encapsulates the stop and start commands and will reboot a container.
- `docker stop` sends a termination signal to a container and the processes running in the container.
- `docker rm` removes a container and all data associated with the container.

The notable aspect here is that using containers has made our application extremely portable and deployable. Now, we do not need to make special configurations on a server and risk one server behaving differently from the other. We can more easily deploy the same environment through the container on any machine, and we can always expect the same outcome.

As of .NET 7, we can containerize our applications without needing a Dockerfile. We will review the steps that can be completed next.

## Using native .NET container support

With .NET, we have native support for containerization. This means that we can utilize .NET packages to incorporate container support into our application and then publish our app directly to a container, all without requiring Docker.

Container support is made available through the `Microsoft.NET.Build.Containers` package. We can add this package using the following command:

```
dotnet add package Microsoft.NET.Build.Containers
```

Now that we have added our package, we can publish our application to a container and then use Docker to run the published application. Using the CLI, we can run the following commands:

```
dotnet publish --os linux --arch x64 -c Release  
-p:PublishProfile=DefaultContainer  
docker run -it --rm -p 5010:80 healthcare-appointments-api:1.0.0
```

Now, our self-hosted container will run and listen for traffic on the configured port 5010. Now we see that we have several ways to containerize our applications, and we apply this approach to all our other services. This, however, presents a new challenge, where we may need to launch our containerized web services in a specific order or with default values and settings. For this, we need an orchestrator. We have mentioned orchestrators before, such as Kubernetes, which is an industry-leading metaphorical glove that enables Docker images (the metaphorical hand) to fit in.

Now that we have seen how to create our container images, it would be beneficial to know how to preserve them for the future and share them with other team members or the public. This is done through a container registry. We will explore this next.

## Publishing containers to a container registry

As microservices architectures grow, the ability to share, store, and distribute container images consistently becomes crucial. A **container registry** plays a key role here, acting as a centralized hub for versioned container images. This allows team members and automated systems to pull consistent images across development, testing, and production deployments.

Registries are a foundational part of any cloud-native pipeline, enabling DevOps teams to standardize environments, minimize configuration drift, and enhance deployment velocity. A registry can be public or private:

- Public registries (e.g., Docker Hub) are used to share images openly with the broader community. Developers can pull base images such as `mcr.microsoft.com/dotnet/aspnet` or contribute their own.
- Private registries provide secure, enterprise-grade storage for internal container assets.

These images are then used as base images by others and are sometimes tweaked. This is a great way to contribute to the open source collection of container images. Docker Hub is the largest public repository and community for containers and is the default source of our container images when we do a `docker pull` command.

Private repositories provide a more secure and private way to host and maintain enterprise containers. These types of registries can be remote through established registries, such as *Google Container Registry*, *Microsoft Azure Container Registry*, or *Amazon Elastic Container Registry*.

When using a private container registry, teams gain more control over the following:

- Authentication options (OAuth, Active Directory, etc.)
- Role-based access control for images
- Image vulnerability scanning and versioning
- Activity auditing and access logging

You can also host your own on-premises container registry using tools such as **Harbor**, **Nexus**, or even a self-hosted Docker registry, giving you complete control over policies and integrations. We can properly control who can upload images through stricter controls and measures, preventing unauthorized access and contributions to the registry.

We can stage a local instance of Docker Hub on a server and add configurations tailored to our organization's specific needs and policies. We can also leverage cloud-hosted container registry services. The advantage of using cloud-hosted solutions is that we can reduce the infrastructural consideration that comes with setting up a local server, as well as taking advantage of a fully managed and geo-replicated solution.

Since we have already created several images to support our application, there are times when we might need a particular application to be deployed in a consistent and repeatable manner, and we don't want to risk recreating the `Dockerfile` or `docker-compose.yml` files. In this case, we

can pull a copy of the base image, add our configurations and variables to it, and then push the image back to the registry with a new and unique name. Now, other team members, or even you in a future project, will be able to pull this new image at will and leverage the preset environment as needed. For this exercise, we need to have an account on Docker Hub.

Let us say we need to create a repeatable database image that all team members can access. We will use an example of an SQL Server image. If we need to create an image with a default database as a starting point, we can do this in a few steps. First, we pull the base image from Docker Hub with this command.

```
docker pull mcr.microsoft.com/mssql/server
```

SQL Server images tend to be large, so downloading them may take some time. We have seen this command before, where we pull the latest image version to our local machine. Do note that it is recommended that you work with a specific version in mind and not only rely on the newest image. This can assist with consistency in deployments and development environments. In this situation, you can add the version number to the name of the image, like this, to get the SQL Server 2022 image.

```
docker pull mcr.microsoft.com/mssql/server:2022-latest
```

Once downloaded, however, we can execute this command to run the container:

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=AstrongP@ssword1" -p 1434:1433 -d mcr.microsoft.com/mssql/server
```

This creates a new container based on the SQL Server image, making it available through port 1434. Port 1433 is the default port for SQL Server, so we can use a different port to avoid conflicts with other SQL instances that might be present. We also set default *sa* user credentials and accept the terms of use agreements.

Now that we have the instance of SQL Server up and running from our image, we can connect using SQL Server Management Studio or Microsoft Azure Studio and then run a script. We will keep it simple and create a database and a table:

```
CREATE DATABASE PatientsDb
GO
USE PatientsDb

CREATE TABLE Patients(
    Id int primary key identity,
```

```
    FirstName varchar(50),  
    LastName varchar(50),  
    DateofBirth datetime  
)
```

We now have two options for creating this image. The first option is to use the running container as the image. Having updated the database, let us commit the updated image to a registry. We can use the Docker UI to view the name of the running SQL Server container, or use the `docker ps` command, which lists all running containers. We then run this `docker commit` command using the name of the running SQL Server container.

```
docker commit -m "Added Patients Database" -a "Your Name" adoring_boyd  
Username/patients-db:latest
```

This creates a local copy of the image based on the current state of the container. We add a commit message to keep track of the change made, an author tag, the name of the container on which we need to base the image, and then we add our Docker Hub username and the new image's name. Now we can use the Docker UI and click the **Images** option, and see our new image created and ready for use in future projects. We can also use the `docker images` command to list all the images currently on the system. You will now see both the original SQL Server image and our recently committed image.

We have now successfully published our image to our local registry. If we want to make this image accessible on Docker Hub, we need to use the **Push to Hub** option available to us in the Docker UI. Alternatively, we can run the following command:

```
docker push Username/patients-db
```

This method is helpful if you have made changes to a running container and want to keep that exact state. However, it's less repeatable and tougher to track over time compared to building via a Dockerfile and script.

Now, let us explore the second option, where we create a new directory in our project. Let's call it `sqlserver-db` and add two files: a Dockerfile and an SQL script called `patients_init.sql`. We can populate the SQL script with the `CREATE DATABASE` script mentioned, as well as any other statements that need to be run when the container is created for the database. We can then populate our Dockerfile with the following code.

```
FROM mcr.microsoft.com/mssql/server:2022-latest  
ENV SA_PASSWORD="YourStrongP@ssword1" \  
    ACCEPT_EULA=Y \  

```

```
MSSQL_PID=Developer
COPY patients_init.sql /patients_init.sql
CMD /bin/bash -c "
/opt/mssql/bin/sqlservr &
sleep 20 &&
/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P YourStrongP@ssword1
-i /patients_init.sql &&
wait
"
```

We can then build and tag the image using the following command. The tag helps us to track the version of the image being created.

```
docker build -t healthcare/patients-db:v1 .
```

Finally, we push the newly created image to the registry with the following commands:

```
docker tag healthcare/patients-db:v1 yourdockerhub/patients-db:v1
docker push yourdockerhub/patients-db:v1
```

From now on, if we needed to add this database image to our orchestration, we could modify our `docker-compose.yml` file like this:

```
patients_sql_db:
  image: Username/patients-db
  restart: always
  ports:
    1434:1433
```

Now, we will always launch an SQL Server instance with this base database.

Now that we have a fundamental understanding of containerization and how we can preserve our images using a container registry, the next important step in building containerized applications is orchestration. This is where we organize the start sequence of the containers. We will discuss this next.

## Understanding container orchestration

Container orchestration is an automated approach to launching containers and related services. In the context of our microservices application, where we have several individual services, each one is containerized. We may also utilize a containerized cache server and other supporting services,

such as email and logging applications. We now need a way to organize our list of containers and launch them in a particular order, such that the supporting applications are available before the dependent service containers are launched.

This requirement introduces a considerable amount of complexity and can make a manual effort very difficult. Using container orchestration, we can make this operation manageable for development and operations. We now have a declarative way of defining the work that needs to be done and the order in which our containers should be launched, and with what dependencies. We will now have the following:

- **Simplified operations:** To reiterate, container orchestration significantly simplifies the recurring task of launching containers in a specific order and with predefined configurations.
- **Resiliency:** Containerization can be utilized to execute our specific operations based on container health, system load, or scaling requirements. Orchestration will manage our instances as needed and automate actions that are in the best interest of the application's stability.
- **Security:** This automated approach helps us reduce the chance of human error and ensure security in our application.

Docker Compose is the simplest form of orchestration that we can employ. Alternatives include Kubernetes and Docker Swarm, which are industry-leading options in the container orchestration space.

Docker Compose is a tool that helps us define multi-container applications. We can define a YAML file that specifies the containers to be launched, along with their dependencies, and then launch the application with a single command. The significant advantage of Docker Compose is that we can define everything about our application's stack in a single file and have it located at the root of our application's folder.

Another advantage here is that if our project is version-controlled, we can easily accommodate the evolution of this file through outside contributions, or we can share our containers and launch steps for our application with others easily.

Now, let's review the steps to add Docker Compose support to our microservices application.

## Adding Docker Compose to a project

Adding Docker Compose support to our ASP.NET Core microservices application is straightforward. In Visual Studio, we can right-click one of our service projects, go to **Add**, and select **Container Orchestration Support....** This launches a window where we can confirm our preference for



**Docker Compose** and select **OK**. We can choose either **Windows** or **Linux** as the target operating system. Either OS option works since ASP.NET Core is cross-platform. You can repeat these steps for each project that you wish to add to the orchestration.

Figure 14.5 shows the **Container Orchestrator Support...** option:

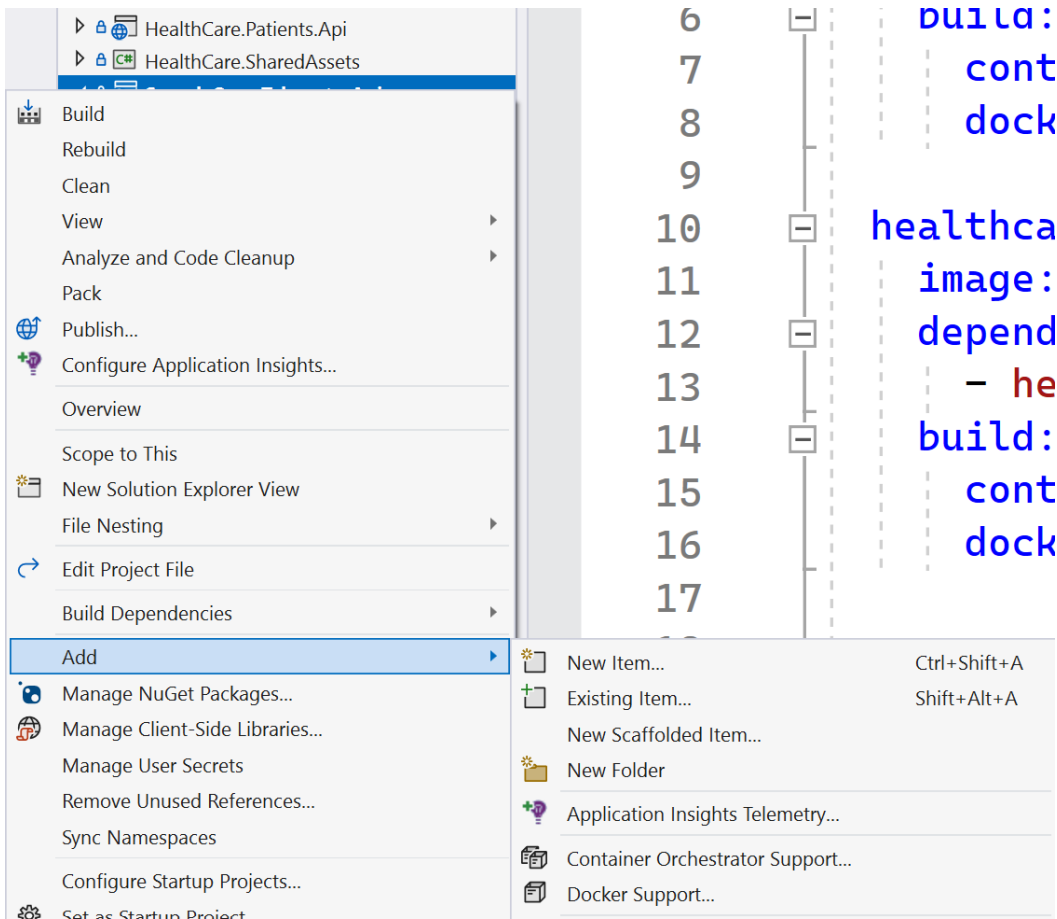


Figure 14.5 – Adding container orchestration support using Visual Studio 2022

Using Visual Studio Code, open the **Command Palette** and select **Run: Docker: Add Docker Files to Workspace**. Following the same steps required to add the Dockerfile, you will also get the option to add `docker-compose.yml` if you select any other project in the solution. Each project will be added to the orchestration accordingly. Doing this is essential when there are services that depend on others, as it makes the multi-project startup process less tedious.

If we inspect the `docker-compose.yml` file, we will see that it contains a version number and at least one service definition.

```
version: '3.4'
services:
  healthcare.patients.api:
    image: ${DOCKER_REGISTRY-}healthcarepatientsapi
    build:
      context: .
      dockerfile: HealthCare.Patients.Api/Dockerfile
```

According to our service definition, we specify the containers to be launched, state the name of the image, and provide the Dockerfile as a reference point for the image definition. Following similar steps for the other services, Visual Studio will automatically append additional services to the file accordingly. If we proceed to do this with the extra services, we will end up with a compose file like this:

```
version: '3.4'
services:
  healthcare.patients.api:
    image: ${DOCKER_REGISTRY-}healthcarepatientsapi
    build:
      context: .
      dockerfile: HealthCare.Patients.Api/Dockerfile

  healthcare.auth:
    image: ${DOCKER_REGISTRY-}healthcareauth
    build:
      context: .
      dockerfile: HealthCare.Auth/Dockerfile

  healthcare.appointments.api:
    image: ${DOCKER_REGISTRY-}healthcareappointmentsapi
    build:
      context: .
      dockerfile: HealthCare.Appointments.Api/Dockerfile

  healthcare.apigateway:
    image: ${DOCKER_REGISTRY-}healthcareapigateway
```

```
build:
  context: .
  dockerfile: HealthCare.ApiGateway/Dockerfile
```

Now we have a Docker Compose file that specifies each container that needs to be launched in our microservices application. This file can be extended to include additional containers as necessary for the entire application. If we need a Redis cache instance, we can add a command to this compose file to launch a container for Redis. It would look like this:

```
services:
  redis:
    image: "redis:alpine"
... Other services ...
```

This addition to the file will spin up a Redis cache container using the defined base image. Note that the Redis cache image composition can be extended to utilize storage volumes, and we can pass specific configuration files to our image, indicating that we wish to persist the container's information in a data volume. When we restart this image, the data from the last run will still be available.

We might also want to indicate that some containers should be available before others are started. This could be particularly useful if a container has a dependency on the Redis cache container or another service. For this, we can add another node called `depends_on`, which will allow us to specify the name of the container that should be launched before attempting to launch the other. For example, our appointment service communicates regularly with our patient service. It would be prudent for us to ensure that the patient service is launched before we attempt to launch the appointment service. We can modify the appointment container orchestration to look like this.

```
healthcare.appointments.api:
  image: ${DOCKER_REGISTRY-}healthcareappointmentsapi
  depends_on:
    - healthcare.patients.api
  build:
    context: .
    dockerfile: HealthCare.Appointments.Api/Dockerfile
```

The `docker-compose.override.yml` file is used to override or extend the base `docker-compose.yml` file for specific environments, such as development. Its main job is to layer settings on top of the base file, rather than providing an alternative to running Dockerfile commands. This lets us set up more specific configurations for each image and tailor settings for our images without having to repeat the Dockerfile commands directly.

```
version: '3.4'
services:
  healthcare.patients.api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=https://+:443;http://+:80
    ports:
      - "80"
      - "443"
    volumes:
      - ${APPDATA}/Microsoft/UserSecrets:/root/.microsoft/usersecrets:ro
      - ${APPDATA}/ASP.NET/Https:/root/.aspnet/https:ro
... Other overrides...
```

Here, we see that our API's container will be launched with specific environment variables and will have preset ports and volumes. This file can be removed if you prefer to have more explicit control over your containers; however, ensure that the relevant values are configured in the Dockerfile.

To revisit our **Backend for Frontend** pattern from a previous chapter, we outlined that we could provision multiple gateway projects and provide specific configurations for each instance. Now that we have containerization, we can remove the need for additional code projects and reuse the same project while using different configurations for each one. Let us add the following lines to the `docker-compose.yml` file and create two separate containers based on the same gateway image.

```
mobileappapi:
  image: ${DOCKER_REGISTRY-}healthcareapigateway
  build:
    context: .
    dockerfile: HealthCare.ApiGateway/Dockerfile
webappapi:
  image: ${DOCKER_REGISTRY-}healthcareapigateway
```

```
build:
  context: .
  dockerfile: HealthCare.ApiGateway/Dockerfile
```

Now that we have defined two new containers based on the exact API gateway image definition, we can add to the `docker-override.yml` file and specify the sources of the specific configuration files that should be used in either case.

```
mobilegatewaygw:
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - IdentityUrl=IDENTITY_URL
  volumes:
    - ./HealthCare.ApiGateway/mobile:/app/configuration
webhealthcaregw:
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - IdentityUrl=IDENTITY_URL
  volumes:
    - ./HealthCare.ApiGateway/web:/app/configuration
```

We can specify the volume sources as the paths to the configuration files. Still, we also took the opportunity to define environment variable values relative to configurations that may vary or need to be set when the container is being created.

In general, we will also want to consider the need for other dependencies, such as a database server, a message bus provider (such as RabbitMQ), supporting web services, and utilities, among others. All the components of your microservices application can be containerized, and the containers can make direct reference to each other as needed. The `docker-compose.yml` file allows us to outline all these containers and their variables as required, and we can easily plug in the values accordingly. With a few lines, we can get started, but there are many more avenues to explore and master.

Now we can comfortably launch our entire microservices application and all the components with one click or command. If using Visual Studio, the button to start with debugging would traditionally have cited the name of one service at a time, or we would need to enable multiple projects to be run for debugging. Now, the presence of the `docker-compose.yml` file eliminates the need for specificity and allows us to run the `compose` command to initialize all our containers in one go. Using the CLI, we can run `docker-compose up`.

Now that we have a fully containerized application, we can adjust the parameters used to launch our containers, remaining agile and scalable in how we handle application hosting and deployments. It also makes it easier to share our applications with our development team and have team members run our applications with Docker installed as the only dependency.

We have also seen that there are base images that allow us to extend and create our images. There will come a time when we have a particular application or our version of an application that we wish to preserve and reshare beyond just our project.

While Docker Compose provides a simple and effective way to orchestrate containers in development environments, it quickly reaches its limits in large-scale, production-ready systems. As microservices architectures grow in complexity, we need a more powerful and flexible solution to manage container lifecycles, networking, scaling, and fault recovery. This is where Kubernetes comes in, and we will explore this next.

## Using Kubernetes

Initially developed by Google and now maintained by the **Cloud Native Computing Foundation (CNCF)**, Kubernetes has become the industry standard for container orchestration at scale.

Unlike Docker Compose, which is designed for local development and small-scale orchestration, Kubernetes is built for production-scale workloads. It offers a declarative configuration model and a robust API to manage services with complex lifecycles, dependencies, and operational policies. It is a platform that provides the following:

- Self-healing capabilities (automatic restarts, rescheduling, and replication)
- Built-in service discovery and load balancing
- Secret and configuration management
- Automatic scaling (both vertical and horizontal)
- Rollbacks and rolling updates
- Multi-cloud and hybrid deployment support

To understand how Kubernetes orchestrates your microservices, let's briefly explore its fundamental building blocks:

- **Cluster:** A set of worker machines (nodes) managed by a control plane
- **Pod:** The smallest deployable unit in Kubernetes, which wraps one or more containers and includes shared resources such as storage and networking
- **Service:** An abstraction that exposes a set of Pods as a network service

- **Deployment:** A controller that manages Pods, ensuring the desired number of replicas and enabling rolling updates
- **Ingress:** A collection of rules that allow inbound connections to reach the cluster services

Let's walk through the steps to deploy your containerized microservices on Kubernetes. If you're using Docker Desktop, enabling Kubernetes is straightforward. It ships with a single-node Kubernetes cluster that runs inside Docker itself. As seen in *Figure 14.6*, navigate to **Settings | Kubernetes** and toggle the **Enable Kubernetes** option on. Then you click **Apply & restart**.

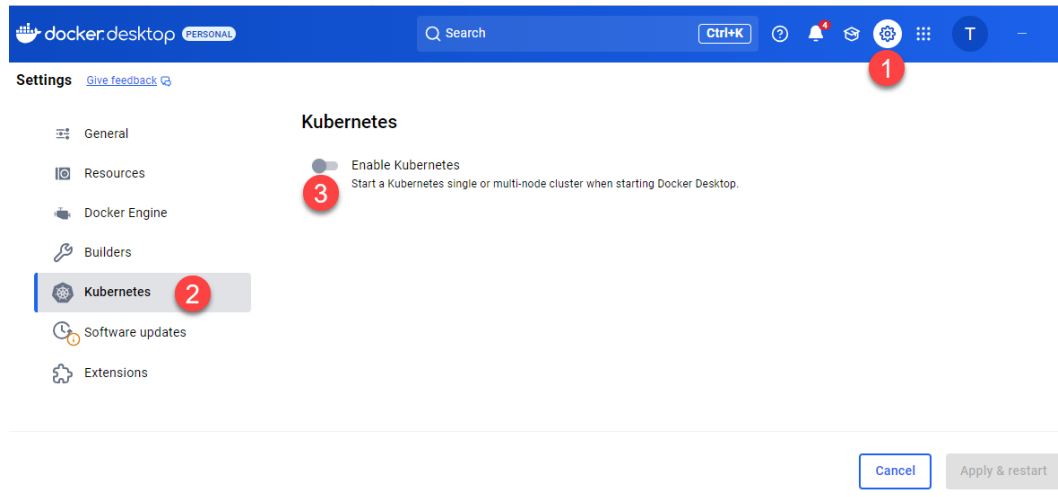


Figure 14.6 – Enabling Kubernetes via Docker Desktop

You may need to restart Docker once this has been completed. For Linux (or macOS users who prefer CLI control), **Minikube** (<https://minikube.sigs.k8s.io/docs/start/>) provides an excellent way to run Kubernetes locally. You will also need to install **kubectl** manually (<https://kubernetes.io/docs/tasks/tools/install-kubectl/>). Use the `kubectl get nodes` command to see the number of ready nodes, and you should see them (at least one) in the Ready state.

You can optionally include the Kubernetes dashboard UI using the following commands:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/
v2.7.0/aio/deploy/recommended.yaml
kubectl proxy
```

This will download and install the necessary files, then allow access at the address `http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:/proxy/`.

Now we must ensure the container image is available to Kubernetes. The deployment manifest will reference `appointments-api:1.0`, but Kubernetes can only pull it if the image is hosted in a registry.

If you are using Docker Desktop with Kubernetes enabled, Kubernetes can access images from your local Docker engine. However, for portability and real-world deployments, you should push your image to a registry such as Docker Hub, **Azure Container Registry (ACR)**, or a local private registry. Pushing it to Docker Hub requires commands like these:

```
docker build -t <dockerhub-username>/appointments-api:1.0 .
docker push <dockerhub-username>/appointments-api:1.0
```

Now that the initial setup has been completed and our image has been published, we need to create a new folder named `k8s` at the root of the solution folder. In this folder, create a new file called `appointments-api-deployment.yaml` with the following content.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: appointments-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: appointments-api
  template:
    metadata:
      labels:
        app: appointments-api
    spec:
      containers:
        - name: appointments-api
          image: appointments-api:1.0
          ports:
            - containerPort: 80
```

The file defines a Deployment resource in Kubernetes, which is responsible for managing the lifecycle of the containerized AppointmentApi microservice. This file describes the number of instances of the microservice that should be running, the container image to use, and how to configure the container environment within the Kubernetes cluster.



At the core of the deployment specification is the **Pod** template, which instructs Kubernetes to create Pods labeled with `app: appointments-api` and to run a container named `appointments-api` using the `appointments-api:1.0` image. The deployment controller maintains high availability and consistency by monitoring the application's state and ensuring that the declared number of replicas is always running. If the Pod fails due to a crash or node failure, Kubernetes will automatically attempt to restart or reschedule it.

This makes the deployment file crucial for achieving resilience and scalability in a microservices environment. Additionally, the deployment acts as the foundation for executing zero-downtime updates and rollbacks, enabling developers and DevOps engineers to manage application versioning through rolling deployments.

We also need to create `appointments-api-service.yaml`. The contents will be as follows:

```
apiVersion: v1
kind: Service
metadata:
  name: appointments-api-service
spec:
  selector:
    app: appointments-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 32002
  type: NodePort
```

Now we define a **Service** resource that exposes the Appointments API microservice running in the Kubernetes cluster. Since Pods are ephemeral and can be restarted at any time, potentially with different IP addresses, this service provides a stable network endpoint through which other components or clients can consistently reach the API, regardless of changes in the underlying Pods.

The service is set up with a selector that identifies Pods labeled with `app: appointments-api`, thereby connecting it to the Pods generated by the relevant Deployment. It opens a port on the service (port 80) and directs incoming traffic to the target port of 80, or to any port that the containerized application uses internally. This configuration allows users and other services to interact with the API through a common and predictable entry point, while maintaining a separation between the actual container setup and the external interface.

The type of service is defined as `NodePort`, meaning the service is also exposed on a high, static port on the host machine (typically between 30000 and 32767). This is particularly useful for local development and testing scenarios using **Minikube** or **Docker Desktop**, allowing developers to access the microservice directly via `localhost:<nodePort>`. The service also acts as a building block for more complex routing layers, such as Kubernetes Ingress controllers or API gateways, which require services as routing targets.

Now we can execute our orchestration by executing the following commands in the `k8s` folder.

```
kubectl apply -f appointments-api-deployment.yaml
kubectl apply -f appointments-api-service.yaml
```

After running them, we can use these commands to ensure that the Pods have been created and are running.

```
kubectl get deployments
kubectl get pods
kubectl get services
```

To access the running service, use this command to retrieve the details of the running Pod.

```
kubectl get svc appointments-api-service
```

Then you can navigate to `http://localhost:{nodePort}` to access the service.

Kubernetes is a cornerstone of modern cloud-native application development. It offers a robust, scalable, and automated platform for running containerized applications, particularly in systems designed around microservices. It provides powerful orchestration capabilities to meet enterprise-grade demands. With features such as declarative deployments, automated rollouts and rollbacks, health-driven container restarts, and self-healing infrastructure, Kubernetes enhances application delivery beyond what Docker or traditional deployment models can offer. Its seamless integration with CI/CD pipelines, cloud services, service meshes, and observability tools makes it the natural progression for teams seeking to scale both their services and their operational maturity.

That said, it is crucial to be realistic about the operational costs of Kubernetes. It is a sophisticated system with many moving parts, and managing it effectively involves a learning curve and often requires a dedicated DevOps or platform engineering team. Debugging misconfigured resources, managing YAML sprawl, and fine-tuning production settings can easily overwhelm small teams or projects that are still in their early development stages. It is not a default, but a destination, and it is essential not only because of its technical capabilities but also due to the standardization and operational maturity it provides.

When adopted with a clear understanding of its benefits and trade-offs, it becomes a robust foundation for building reliable, modern, and cloud-native microservices applications. Now that we have looked at containers and orchestration, let us summarize this chapter.

## Summary

In this chapter, we transitioned from securing our microservices to effectively deploying and managing them using containers. We began by examining the challenges that arise when deploying modern microservices, particularly when those services have varying runtime dependencies or are deployed across inconsistent environments. These challenges led us to containerization, a practice that enables us to bundle applications with all their dependencies into consistent, portable units known as container images. Through this lens, we came to understand how containers help us eliminate configuration drift, reduce overhead, and streamline software delivery pipelines.

To bring these concepts to life, we explored Docker, the de facto standard for containerization. We studied how Docker operates under the hood, how it employs a client-server architecture using the `dockerd` daemon, and how it enables us to define, build, and manage images and containers through Dockerfiles and commands. We also discussed the key components of containers, including networks, volumes, and plugins, and reviewed how to utilize both the command line and IDE tooling to add container support to our .NET microservices projects. Our practical walk-throughs demonstrated how easy it is to use Docker to pull images, run containers, and manage container lifecycles across platforms.

Once we understood the basics of containers and Dockerfiles, we explored how to create reusable container images for third-party services, such as SQL Server, as well as our own ASP.NET Core microservices. We examined how to publish these images to both public and private registries to foster collaboration, share preconfigured environments, and enable repeatable deployments across teams and environments. The benefits of container registries were evident, particularly in the context of enterprise development and DevOps automation.

With our services containerized, we shifted our focus to orchestration. We began with Docker Compose, a lightweight and developer-friendly tool for coordinating multi-container applications. We showcased how Compose can be utilized to define service dependencies, manage environment variables, mount configuration volumes, and orchestrate startup sequences. This was especially beneficial for our microservices system, which encompasses not only core APIs but also supporting services such as Redis, authentication, and gateway layers. By configuring `docker-compose.yml` and `docker-compose.override.yml`, we created a flexible, local orchestration layer that streamlines development and testing.

As our architecture became more complex, we recognized the limitations of Docker Compose and introduced Kubernetes as the industry-standard orchestration platform for production-scale workloads. We discussed Kubernetes' declarative model and its core components: clusters, Pods, Deployments, Services, and Ingress, while highlighting features such as autoscaling, self-healing, and configuration management. We then walked through the practical steps of enabling Kubernetes in Docker Desktop, writing deployment and service manifests for the Appointment API, and deploying the service using `kubectl`. These examples reinforced our understanding of how Kubernetes abstracts and automates infrastructure concerns, making it easier to manage resilient and scalable microservices.

Finally, we paused to reflect on the operational realities of adopting Kubernetes. While powerful, it is not trivial. It introduces significant complexity and requires experience to manage it effectively. We emphasized that Kubernetes should be adopted with intent and preparedness. It is not the default solution, but rather a destination for teams ready to embrace the rigor and flexibility it offers. For a new user, the process from `docker-compose` to a real-world Kubernetes setup is a massive leap, and it is crucial to be aware of the complexities involved.

By the end of this chapter, we had not only demystified containers and orchestration but also built a strong foundation for deploying microservices with confidence. We now have the tools to run our applications anywhere, scale them intelligently, and share them effortlessly with our teams and infrastructure platforms.

In the upcoming chapter, we will expand on these concepts by examining cloud-oriented development patterns that aid in creating efficient and scalable services.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





# 15

## Serverless Microservices Development

In the evolving landscape of cloud-native application development, serverless computing has emerged as a powerful paradigm for building agile, cost-efficient, and scalable microservices. As software teams strive to reduce operational overhead and increase deployment velocity, serverless architecture provides a compelling model for delivering business value with minimal infrastructure management.

In this chapter, we will examine the application of serverless computing to microservices-based systems. Serverless doesn't simply mean running code without managing servers, but it represents a shift in how we architect, execute, and operate software. From various public cloud platform offerings (such as Azure Functions and AWS Lambda) to durable workflows and message queues, we will explore the core building blocks needed to unlock the full potential of serverless in your .NET applications.

In this chapter, we will discuss the following:

- The fundamentals of serverless architecture
- Designing microservices specifically for serverless deployment
- Building serverless microservices on Microsoft Azure

By the end of this chapter, you'll understand how to design microservices that respond to events, scale automatically under load, and gracefully handle failures without requiring infrastructure provisioning or maintenance. Through practical examples, patterns, and a guided case study, you'll gain the confidence to introduce serverless elements into your distributed systems.

Let us begin by exploring the fundamentals of serverless architecture.

## Technical requirements

You will need the following tools for this chapter:

- **Visual Studio Code:** <https://code.visualstudio.com/>
- **Azure Tools:** <https://marketplace.visualstudio.com/items?itemName=ms-vscode.vscode-node-azure-pack>
- **Azure Functions Core Tools:** <https://github.com/Azure/azure-functions-core-tools>
- **Azurite:** <https://github.com/Azure/Azurite>
- **Azure Storage Explorer:** <https://www.storageexplorer.com/>

Code references used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter15>.

## Introducing serverless architecture

Software deployment has consistently trended toward minimizing infrastructure management while maximizing developer efficiency and system scalability. Serverless computing represents a significant step forward in that journey. It enables developers to focus solely on writing business logic while abstracting away the provisioning, scaling, and maintenance of servers. Before we explore the details, let's first understand what serverless computing is.

### What is serverless computing?

When designing a microservices architecture, choosing the right infrastructure model is a crucial decision. Traditionally, hosting web applications (and, by extension, microservices) involved using physical servers. These servers often supported large, monolithic applications built with frameworks such as ASP.NET, Java EE, PHP, or Ruby on Rails. Although servers have proven reliable and configurable over time, their suitability for microservices is often limited and counterproductive, especially when each microservice needs a server or set of servers.

The next level to hosting is **virtual machines (VMs)**, which are completely isolated OS-level environments that simulate hardware. Applications, including microservices, operate within guest operating systems managed separately from the host machine. This allows one physical server to support multiple VMs, offering various services. This reduces infrastructure costs.

Then we have containers. As discussed in the previous chapter, containers provide lightweight, portable environments that encapsulate an application and its dependencies. Unlike VMs, containers share the host **operating system (OS)** kernel, making them faster and more efficient. Containers, however, require managing the container orchestration layer, which can be complex and operationally intensive. They also require orchestration and infrastructure provisioning, cluster scaling, and monitoring setup, which can have a steeper learning curve.

For teams that want as few infrastructure concerns as possible, serverless computing becomes a great hosting option. **Serverless computing** is a cloud-native execution model where the cloud provider dynamically allocates machine resources. In this model, developers write code as discrete functions, often referred to as **Functions as a Service (FaaS)**. These functions can be configured to execute based on a trigger and are generally stateless, meaning they execute without full knowledge of the context. There are also stateful function options, which we will discuss further in the chapter.

The term *serverless* does not imply that servers are no longer involved. Instead, it signifies that server management responsibilities are offloaded to the cloud provider. You no longer need to worry about provisioning VMs, scaling containers, or patching operating systems. Instead, you deploy your functions, define their triggers, and allow the provider to handle the rest. Some key characteristics of serverless architectures are the following:

- **No infrastructure management:** Developers do not manage or provision servers.
- **Automatic scaling:** Functions scale individually, depending on the number of incoming requests.
- **Pay-per-use billing:** You are charged only for the actual execution time of your functions.
- **Event-driven execution:** Functions are triggered by events such as HTTP requests, message queues, file uploads, or database changes. This event-driven pattern enables loose coupling between services. For example, when a patient registers in the healthcare system, a `PatientRegistered` event can trigger functions to create a welcome notification, provision initial records, and notify a practitioner, all without the patient service needing to know the downstream details.
- **Statelessness:** Serverless functions are stateless by design. Any required state must be passed explicitly or retrieved from external storage.



There are two primary models under the serverless umbrella:

- **Functions as a Service (FaaS):** FaaS, as mentioned before, is the most common form of serverless computing. It allows developers to deploy individual functions that are invoked by specific events.

Example providers are as follows:

- **Microsoft Azure:** Azure Functions
- **Amazon Web Services (AWS):** AWS Lambda
- **Google Cloud Platform (GCP):** Google Cloud Functions

These functions are stateless and ephemeral. They are ideal for microservice endpoints, data transformation operations, or lightweight orchestrations.

- **Backend as a Service (BaaS):** BaaS refers to cloud-hosted services that provide ready-to-use backend functionalities such as authentication (e.g., Firebase Auth), file storage (e.g., Azure Blob Storage), databases (e.g., Cosmos DB), and real-time messaging (e.g., SignalR Service).

In serverless microservices architectures, BaaS components often complement FaaS by offloading everyday backend responsibilities.

The idea here is that the underlying infrastructure for hosting microservices is far more flexible than traditional hosting platforms, and both developers and the supporting infrastructure and DevOps teams benefit from this flexibility. The idea of serverless architecture is, in some ways, like what we described as cloud-native development with containers. We will compare the two next.

## Serverless versus containerized microservices

In the previous chapter, we examined **containerization** as a method for packaging microservices to ensure consistent deployment across different environments. While containers offer flexibility and portability, they still require some degree of infrastructure management, such as managing container orchestration with Kubernetes or Docker Swarm.

Serverless complements containerization by abstracting away that orchestration layer. Instead of deploying a container to a cluster, you deploy a function to a serverless runtime, and it becomes immediately available to respond to incoming events. This simplicity is a key reason why serverless is increasingly adopted for specific workloads, particularly those that are the following:

- Short-lived or intermittent
- Spiky or unpredictable in load

- Event-driven (e.g., webhooks, file uploads, or scheduled jobs)
- Part of a workflow (e.g., background processing or notification dispatching)

It's worth noting that serverless and containers are not mutually exclusive. Many modern cloud-native applications use both approaches in tandem, where containers handle long-running processes and serverless functions manage short-lived or reactive operations.

Now, let us take some time to discuss the different implementations of serverless technologies and the various providers.

## Cloud providers and serverless technologies

In our healthcare management system, we need to manage patient onboarding, appointment scheduling, medical records, real-time notifications, and integrations with external systems. Each of these areas benefits from event-driven, loosely coupled microservices that can scale independently and operate with minimal infrastructure overhead.

As mentioned before, serverless computing is widely adopted across all major cloud providers. Each offers a distinct set of services, developer experiences, pricing models, and integration patterns. While the philosophy behind the serverless paradigm remains consistent, the implementation and tooling vary significantly from one provider to another. For this section, we will focus on understanding the services present in AWS and Microsoft Azure.

Let us start with AWS. **AWS** is often chosen for its maturity, language support, and massive integration ecosystem. Launched in 2006, AWS now offers over 200 services, spanning computing and storage, as well as AI, machine learning, analytics, and developer tools. Many healthcare organizations have already adopted AWS due to its security features, HIPAA-eligible services, and architectural flexibility. Some of the most useful AWS services that could be used to build our application include the following:

- **AWS Lambda:** Amazon's FaaS offering. It enables you to run backend code in response to events such as HTTP requests, queue messages, file uploads, or schedule triggers, without provisioning or managing servers.
- **Amazon API Gateway:** A fully managed service that allows you to expose AWS Lambda functions (or other services) via HTTP(S) endpoints. It serves as a front door for serverless APIs, supporting both REST and WebSocket protocols.

- **EventBridge/SNS/SQS messaging and event bus:** This is a group of services that allow applications to support event-driven architecture and asynchronous messaging:
  - **Amazon EventBridge:** A fully managed event bus that routes events between AWS services, SaaS applications, and custom services. It allows for event-driven architecture decoupling.
  - **Amazon Simple Notification Service (SNS):** A pub/sub messaging service for broadcasting notifications to multiple subscribers, such as Lambda, SMS, email, or HTTP endpoints.
  - **Amazon Simple Queue Service (SQS):** A durable, fully managed message queuing service for decoupling and buffering communication between microservices.
- **AWS Step Functions:** A workflow orchestration service for coordinating multiple AWS services into serverless workflows. It supports branching, retry, delays, and human interactions, making it ideal for modeling appointment flows or patient onboarding.
- **Amazon CloudWatch:** The core observability and monitoring tool on AWS. It collects logs, metrics, and traces from AWS services and applications, enabling teams to create dashboards, set alarms, and analyze system behavior in real-time.
- **Serverless backends:** A collection of storage services optimized for serverless architectures:
  - **Amazon DynamoDB:** A managed NoSQL, low-latency database. It can store both structured and semi-structured data.
  - **Amazon S3:** A highly durable object store for uploading and retrieving unstructured files such as scanned documents, lab reports, or identity proofs.
  - **Amazon Aurora Serverless:** A relational database engine (MySQL/PostgreSQL-compatible) that automatically scales based on load. Ideal for transactional and relational data needs, such as audit trails or medical claims.

AWS is an industry leader in serverless service offerings, making it a great contender for hosting and managing serverless solutions that target almost any industry. Their most significant competitor in this space is Microsoft Azure, which has been a long-standing presence.

**Microsoft Azure** is a comprehensive cloud computing platform developed by Microsoft. It offers a broad range of services, including computing, storage, networking, databases, AI, analytics, DevOps, and application hosting. These features enable developers and businesses to build, deploy, and manage applications at scale across Microsoft-managed data centers worldwide. Azure supports multiple programming languages and frameworks, including .NET, Java, Python, Node.js, and Go, and provides both **Infrastructure as a Service (IaaS)** and **Platform as a Service (PaaS)**.

It is recognized for its close integration with enterprise tools, hybrid cloud solutions, and strong identity and compliance features. Some of the most useful Azure services that could be used to build our application include the following:

- **Azure Functions:** Azure's FaaS platform. It allows developers to write event-driven code triggered by HTTP requests, queue messages, timers, blob storage events, and more. Supports .NET, Node.js, Python, Java, and PowerShell.
- **Azure API Management (APIM):** A comprehensive API gateway and management service. It exposes Azure Functions and other services as RESTful APIs, enforces policies (e.g., rate limiting and validation), and secures access through keys, OAuth2, or Entra ID.
- **Azure Event Grid/Service Bus/Queue Storage:** Azure provides a collection of messaging tools that enable event-driven and asynchronous workflows:
  - **Azure Event Grid:** A fully managed event routing service that supports reactive programming by delivering events from publishers (such as Blob Storage or custom apps) to subscribers (such as Functions or Logic Apps).
  - **Azure Service Bus:** A reliable, enterprise-grade messaging service featuring queues, topics (pub/sub), sessions, and dead-lettering. Ideal for inter-microservice communication and orchestrating business events.
  - **Azure Queue Storage:** A straightforward queuing system for lightweight background tasks and retries.
- **Azure Durable Functions:** An extension of Azure Functions that enables stateful orchestrations. It is ideal for modeling workflows such as appointment bookings, patient intake, or multi-step insurance verification processes. It supports function chaining, fan-out/fan-in, and long-running tasks with persistence.
- **Azure Monitor and Application Insights:** Comprehensive tools for logging, tracing, and metric collection. Azure Monitor monitors infrastructure and app health, while Application Insights offers detailed telemetry on function performance, dependency calls, and failure diagnostics.
- **Serverless backends:** A suite of scalable, cloud-native data stores suited for microservices:
  - **Azure Cosmos DB:** A globally distributed, multi-model NoSQL database with low-latency reads and writes. Ideal for storing patient records, appointment data, or IoT health telemetry.
  - **Azure Blob Storage:** A highly scalable object storage solution for binary files such as scanned prescriptions, identity documents, and attachments.

- **Azure SQL Database (Serverless tier):** An auto-scaling relational database that pauses when inactive and resumes on demand. Ideal for storing structured, transactional data with SQL support and Entra ID integration.
- **Azure Key Vault:** A secure storage for secrets, keys, and certificates. Used to protect connection strings, service credentials, and cryptographic data, ensuring secure communication between microservices and proper environment setup.

While both AWS and Azure offer robust cloud platforms with extensive support for serverless computing, microservices, and global scalability, they differ in their philosophies, ecosystem integration, and developer experiences. AWS, as the earliest and most established cloud provider, offers a broader range of services with detailed configuration options and a global infrastructure, making it an excellent choice for teams needing maximum flexibility, especially when developing new, cloud-native applications from scratch. Its tools are highly modular but often require more cloud expertise to set up effectively. Azure, by contrast, excels in seamless integration with the Microsoft ecosystem, providing a more unified experience for developers already using .NET, C#, Visual Studio, and Microsoft 365. Azure's serverless options are built with strong support for enterprise workloads, making them especially attractive for developers working on line-of-business or regulated industry apps, such as healthcare systems.

**GCP** serves as a compelling alternative for serverless development, especially for teams focusing on simplicity, data-driven applications, and rapid prototyping. GCP's **Cloud Functions** offer lightweight, event-driven computing capabilities similar to those of AWS Lambda and Azure Functions. At the same time, **Cloud Run** provides a unique container-based serverless model that supports any language and scales down to zero instances. Developers building microservices that integrate with **Firebase**, **BigQuery**, or **machine learning APIs** often find GCP to be an intuitive and efficient environment. GCP emphasizes developer ergonomics by offering clear documentation, streamlined deployment workflows, and transparent costs.

Notable options include **Cloudflare Workers**, which bring serverless execution to the edge with sub-millisecond cold starts, and **Netlify Functions** and **Vercel Serverless Functions**, which facilitate easy frontend-backend integration in **JavaScript**, **API**, and **Markup (JAMstack)** applications. For teams with container-native workloads, open source frameworks such as **OpenFaaS**, **Knative**, and **Kubeless** offer serverless features on Kubernetes, providing portable, self-hosted alternatives to vendor-managed platforms. Each of these tools presents trade-offs among control, scalability, ecosystem integration, and operational complexity, allowing developers to choose the option that best fits their architecture and deployment needs.

Developers might choose Azure for its enterprise-ready features, quick .NET deployment, and easier onboarding for teams already invested in Microsoft technology. Ultimately, the choice often depends on ecosystem compatibility, team skills, and integration requirements. We will focus on Microsoft Azure's service offerings for the remainder of this chapter.

Now that we have explored the various technologies and platforms that support serverless architecture, let us review some of the advantages of adopting this approach when building microservices.

## Advantages of serverless computing for microservices

Serverless computing is a natural fit for microservices. As organizations adopt microservices to decompose complex domains into manageable, autonomous units, the challenges of infrastructure management, inter-service communication, and operational overhead often increase. Some of the major takeaways are as follows:

- Serverless computing directly tackles these issues by providing a model that improves both technical flexibility and business responsiveness. Unlike traditional infrastructure setups that require pre-allocated compute resources, this often leads to over-provisioning when traffic is low or under-provisioning during traffic spikes, resulting in unnecessary costs or performance issues.
- Serverless computing charges organizations based on use, so they only pay for the duration their code runs. This approach reduces costs associated with idle resources, unlike VMs or long-lasting containers, which incur fixed fees regardless of traffic.
- Unused functions incur no operational cost, making serverless ideal for microservices that experience sporadic or seasonal traffic. This pricing model empowers startups and enterprise teams alike to operate leanly while supporting highly elastic workloads.

Using our healthcare management system as a practical example for cost savings, consider that our patient registration microservice may be heavily utilized during business hours but remains idle at night. With a serverless model, you avoid paying for compute capacity during downtime, while still guaranteeing availability and performance during peak hours. A significant challenge in adopting microservices is the operational complexity that arises from managing multiple independently deployed services. Containerization is powerful, but it requires orchestration platforms such as Kubernetes, which come with a steep learning curve and associated maintenance costs. Serverless computing removes the need for managing servers, containers, and orchestration. It also offers built-in fault tolerance, patching, and high availability, all of which are managed by the cloud provider. This allows development teams to focus on delivering features and domain

logic rather than troubleshooting infrastructure issues. Now, microservices can be implemented as functions that scale independently without the need for teams to configure resource pools or pod auto-scaling policies.

By shifting focus from operations to business value, serverless accelerates the delivery pipeline and reduces the cognitive load on teams by unlocking faster development cycles, facilitating the following:

- **Rapid prototyping and iteration:** Developers can write a single function and deploy it in seconds
- **Granular deployments:** Update one function without redeploying an entire service or affecting unrelated components
- **Rich development tooling:** Azure Functions, for instance, supports .NET isolated process models, dependency injection, and local debugging

For example, let's say a new policy requires an additional step in the prescription approval process. With serverless, a team can add a `VerifyInsuranceEligibility` function that runs alongside the current flow, deployed independently within hours.

Now, let us explore some of the drawbacks and gotchas when adopting the serverless model.

## Drawbacks of serverless computing for microservices

While serverless computing offers a powerful way to build scalable, event-driven applications, it also comes with trade-offs. It's essential to understand its limitations and operational challenges in a serverless-first approach. For example, serverless functions are inherently stateless.

This design choice creates a challenge in maintaining workflow context or user session data across functions, requiring a reliance on external state stores, such as Cosmos DB or Redis, for continuity. Each persistence step adds latency, cost, and potential points of failure. To address this, it's best to store only the minimal data needed for the next step. Fortunately, most FaaS providers offer durable functions, which are designed to preserve state data without requiring extra effort from developers.

Additionally, using serverless technology means that the chosen cloud provider's ecosystems will restrict you. This results in tight coupling to platform-specific APIs and event models, making it difficult to move workloads between different cloud providers. Although we have not yet addressed the concerns of logging and observability, it is worth noting that access to logs and traces is limited to what the cloud providers expose. While most providers are generating extensive logging and telemetry, you are still limited by vendor lock-in and the available tooling to access said logs.

Another central talking point is that serverless computing technology is characterized by its **cold start latency**. This term describes the latency encountered when a serverless function is triggered following a duration of inactivity, during which the associated execution environment has been scaled down to zero. This latency arises as the platform provisions the necessary infrastructure, loads the runtime, and initializes the function before processing the request. While this is a deliberate measure to aid in cost efficiency, cold starts introduce unpredictability, which can negatively impact the user experience and system reliability.

In our healthcare application, where specific endpoints, such as patient lookup or **Electronic Medical Records (EMR)** access, are performance-critical, this can erode confidence in the system's reliability. Adopting a serverless architecture presents a significant trade-off that must be carefully evaluated. This consideration becomes particularly important when serving real-time APIs or managing critical processes within orchestrated workflows. By understanding the impact of cold starts, architects can make more informed decisions about deployment. For latency-sensitive operations, it may be prudent to utilize containerized microservices or reserved function plans. Conversely, standard serverless solutions may be suitable for background tasks, batch jobs, or workflows that occur infrequently.

Serverless computing, despite its evident benefits in terms of agility and scalability, has its drawbacks. It introduces significant challenges related to operational visibility, workflow orchestration, resource flexibility, and cost predictability. In microservices architecture, where composability, resilience, and autonomy are crucial, these limitations necessitate meticulous architectural planning and the implementation of compensatory design patterns. Before adopting serverless for all workloads, teams must evaluate whether a service will benefit from auto-scaling and event-driven execution, and whether it is inherently short-lived and stateless.

Now that we have a fundamental understanding of what serverless computing is and why we may or may not use this pattern, let's explore how to design our microservices optimally for serverless computing.

## Designing microservices with serverless in mind

Designing microservices involves more than simply dividing a monolithic application into smaller parts; it requires creating independently deployable, loosely coupled, and domain-aligned components that can evolve and scale independently. When incorporating serverless computing into this architectural framework, the fundamental design principles remain intact; however, the execution model undergoes a significant transformation. Serverless computing redefines the concepts of service boundaries, state management, latency, and operational control.



Serverless platforms, including Azure Functions, AWS Lambda, and Google Cloud Functions, offer the advantages of reduced operational overhead and fine-grained scalability. However, developing microservices for these platforms necessitates careful planning and consideration of constraints that are not present in traditional or containerized environments.

Designing serverless architecture requires more than simply migrating existing services to a new platform; it necessitates a reevaluation of function granularity, orchestration strategies, observability practices, and inter-service communication. For instance, decisions regarding whether to chain functions, orchestrate with durable workflows, or emit domain events asynchronously will directly influence the system's performance, maintainability, and user experience.

Our healthcare management system provides an exemplary scenario for discussing these considerations. Consider the patient onboarding process, which encompasses data collection, appointment scheduling, identity verification, insurance validation, and notification dispatch. Each of these steps can, and ideally should, be implemented as individual serverless functions. However, how do we maintain the integrity of business transactions? How do we handle retries, timeouts, and dependencies?

Let us begin by examining the guiding principles for modeling function granularity and service boundaries in a serverless ecosystem.

## Choosing the right size for a function

In traditional microservices, each service usually corresponds to a business capability or bounded context. In serverless architecture, this mapping is often fine-tuned, where each function typically handles a single task or step in a process.

When building serverless applications with .NET and Azure Functions, the granularity, size, and scope of functions significantly influence the system's efficiency, performance, maintainability, and cost. In serverless development, developers need to carefully balance simplicity and cohesion with latency, overhead, and observability. This section discusses how to choose the appropriate "size" for a function, ensuring that your .NET serverless architecture stays agile, cost-effective, and ready for production. Some steps to help with this are the following:

- **One function, one responsibility:** Each function should encapsulate one straightforward task, such as creating an appointment, sending a notification, or validating a payment. This aligns with the **single responsibility principle**, simplifying testing, debugging, and deployment.
- **Group based on cohesion, not convenience:** Avoid grouping unrelated tasks into a single function for convenience. Instead, group highly cohesive steps, such as validating and sanitizing input, if they always occur together.

- **Avoid function over-chaining:** Chaining too many fine-grained functions (e.g., step-by-step HTTP functions) introduces performance overhead and complicates tracing. Consider using an orchestrator for multi-step flows.
- **Use clear naming conventions:** Use function names that describe *what* they do rather than *how* they do it, such as `RegisterPatient`, `CreateAppointment`, or `SendAppointmentNotification`.

Choosing the right size for a function is crucial for performance, maintainability, and cost management in .NET serverless development. Functions should be focused, stateless, and aligned with business operations, avoiding both monolithic and overly fragmented designs.

The next step we can take is managing our dependencies.

## Dependency injection in serverless functions

Traditional ASP.NET Core applications depend greatly on built-in **dependency injection (DI)**. Serverless functions must also implement this pattern, but with added constraints due to their lifecycle and cold start behavior. DI is a core software engineering pattern that promotes loose coupling, facilitates easier testing, and enhances maintainability. It enables components to specify their dependencies via constructors or methods with those dependencies supplied from outside, avoiding manual creation within the component. In .NET, DI is seamlessly integrated into ASP.NET Core and Azure Functions, making it a fundamental aspect of development.

Some general best practices when practicing DI are further reinforced when designing serverless functions. Additional considerations include the following:

- Use the .NET isolated worker model, which fully supports ASP.NET Core DI using `IHostBuilder`, `IServiceCollection`, and `ConfigureServices`.
- Register only what you need. Heavy DI graphs increase startup time and amplify cold start latency.
- Choose service lifetimes wisely:
  - Singleton services persist across invocations (if warm), so they're suitable for shared resources such as `HttpClient`
  - Scoped services should be used per invocation if you need tenant- or request-specific context

Since we need our serverless services to be optimal, we must write our code in the most lightweight and portable manner possible. Next, we can consider situations where durability and state management are necessary. We will look at this next.

## Durable orchestration design

For workflows that span multiple steps or require retries, timeouts, or compensation logic, using serverless orchestration frameworks such as **Azure Durable Functions** is highly recommended. Durable Functions are an extension of Azure Functions that enable the orchestration of stateful workflows in a serverless environment. They allow you to write long-running processes as code, while the Azure platform manages the workflow state, event history, and execution reliability.

Traditional serverless functions are best for simple, single-purpose tasks. However, many real-world business scenarios require the following:

- Sequential execution of multiple steps
- Parallel execution of independent tasks
- Waiting for human input or external events
- Long-running operations (hours or days)
- Retry policies and error compensation

Durable Functions make this possible without needing external workflow engines or complex message orchestration systems. There are several implementation styles for durable functions, and there are reasons why we may need to use them:

- **Fan in/fan out:** This pattern is used when multiple tasks can be performed **in parallel**, and the system must wait until all tasks are complete before proceeding. The *fan-out* represents launching numerous functions concurrently, while the *fan-in* represents the aggregation of their results or simply waiting for all tasks to finish.

In our healthcare system, this pattern is ideal for operations that involve several independent but related processes that must occur concurrently, such as the following:

- After a lab result is finalized, fan out to do the following:
  - Notify the patient
  - Notify the assigned physician
  - Update the medical record system
- When onboarding a patient, use it to do the following:
  - Initialize patient portal
  - Create an **electronic health record (EHR)**
  - Generate a welcome message

Since this pattern allows parallel execution, the function will take as long as the longest task in the chain. Durable Functions ensure that even if one parallel task fails, the orchestrator can retry or perform compensatory actions without losing the overall workflow state.

- **Function chaining:** This pattern enables tasks to execute in a strict sequence, where the output of one function becomes the input to the next. Durable orchestrators handle each step, ensuring correct order and passing values between tasks.
- In our healthcare context, function chaining is helpful for the following:
  - **Patient registration workflows:**
    - Register patient details
    - Verify identity and insurance
    - Assign a primary care physician
    - Send portal activation email
  - **Billing workflows:**
    - Generate invoice
    - Submit to insurance
    - Record patient balance
    - Schedule payment reminders

This pattern enhances clarity and auditability, allowing for easy tracing of each step in the patient's journey while ensuring adherence to business rules.

- **Human interaction/external event waiting:** Many healthcare processes require a human decision or approval before continuing. The `WaitForExternalEvent` method in durable functions allows the workflow to pause until a specific event is received, without consuming compute resources.

This pattern is valuable in scenarios such as the following:

- **Prescription authorization:** The system pauses until a physician confirms a prescription, especially for controlled substances
- **Patient consent:** The orchestrator waits for the patient to accept or reject a proposed treatment plan
- **Insurance pre-authorization:** External systems (e.g., an insurer API or webhook) notify the orchestrator when a claim has been approved or denied

Using this model, workflows remain idle yet reliable, resuming instantly when the necessary input or signal is received. It removes unnecessary polling and guarantees systems only move forward when they're truly prepared.

- **Monitor pattern:** The monitor pattern enables durable functions to check a condition repeatedly, looping until the condition is satisfied or a timeout occurs. This is crucial in healthcare for ensuring timely follow-up on tasks with deadlines.

Use cases include the following:

- **Appointment confirmations:** After an appointment is booked, the system checks every hour to see whether the patient has confirmed, and cancels the slot if no response is received within 24 hours
- **Prescription pickup tracking:** Monitor pharmacy logs to confirm medication collection within a defined timeframe
- **Post-discharge compliance:** Verify that follow-up actions (e.g., submitting test results or surveys) are completed after a patient is discharged

By modeling such checks as orchestrated loops, this pattern provides **automated vigilance** over critical activities while minimizing the risk of human error or missed deadlines.

- **Asynchronous HTTP APIs:** Durable Functions also support asynchronous HTTP API patterns, enabling clients to trigger long-running workflows and receive a status endpoint to poll for completion. This is particularly useful for operations that exceed typical HTTP timeout thresholds.

Healthcare use cases include the following:

- **Patient data exports:** Aggregating and exporting all medical records, lab results, and appointments into a downloadable file for patients or auditors
- **Bulk reporting:** Generating month-end utilization summaries, billing extracts, or quality assurance audits
- **Data anonymization pipelines:** Preparing de-identified datasets for clinical research or data sharing

The general workflow of this type of function is as follows:

1. The client initiates the workflow with HTTP POST.
2. The server responds immediately with a 202Accepted response and a status check URL.

3. The workflow runs independently and updates its progress.
4. When complete, the client can download the final output or receive a completion webhook.

This pattern guarantees non-blocking client experiences and enables serverless systems to manage resource-intensive processes reliably.

Durable functions can solve several problems when we need to implement predictable workflows, but things can always go wrong. This is why we need to be able to monitor what happens during and between operations. We will review this next.

## Observability, logging, and monitoring

Serverless applications divide business logic into separate units, making observability a crucial aspect. The temporary, distributed nature of Azure Functions and Durable Functions complicates tracking failures, measuring performance, and diagnosing workflows.

Azure Functions seamlessly integrate with Application Insights, enabling the collection of real-time telemetry for function executions, failures, dependencies, and custom events. When using Durable Functions, orchestration traces, including function start, activity execution, errors, and retries, are logged with a correlation ID, allowing developers to track the entire lifecycle of long-running workflows from start to finish:

- **Traces and metrics:** Every durable orchestration execution generates custom events and logs sent to Application Insights, which can be queried with **Kusto Query Language (KQL)** for insights such as failure trends, performance issues, or usage spikes
- **Live metrics stream:** Developers can observe workflows as they happen, tracking invocation frequency, success rates, cold starts, and execution times

In healthcare systems, these practices ensure compliance, improve root cause analysis during service disruptions, and help meet operational **Service Level Agreements (SLAs)** in a complex, regulated environment. For this reason, we must ensure that our implementations are secure. We will review security next.

## Security considerations

Security is a foundational pillar in any system, and this is especially true where sensitive data and regulated workflows must be tightly protected. While serverless architecture abstracts much of the infrastructure, identity, access control, and secrets management remain the developer's responsibility.

**Azure managed identities** enable serverless functions to authenticate securely with other Azure resources without requiring credential management. For example, an Azure Function can use its identity to do the following:

- Read from **Key Vault** for secret configuration (e.g., database keys)
- Access **Azure Storage** or **Cosmos DB** using **Role-Based Access Control (RBAC)**
- Publish messages to **Service Bus** queues or topics

We may also need to store sensitive values, such as API keys, encryption keys, and certificates, which should never be hardcoded or stored in environment variables. Instead, they should be stored as follows:

- Stored in Azure Key Vault
- Accessed securely using the function's managed identity
- Cached per invocation to reduce latency

Design serverless microservices with minimal-privilege access, well-defined identity boundaries, and comprehensive auditing of all services via **Entra ID logs**. Another serious consideration is the integrations and triggers that you need to scope to support your intended workflow.

## Integration technologies and triggers

Building applications with serverless technologies requires careful design that considers event-driven execution, available integration options, and the triggers that link functions into larger workflows. When done well, serverless apps offer scalability, resilience, and cost savings with minimal operational effort. However, if overlooked, they can quickly become fragile, hard-to-maintain collections of disconnected functions.

At the core of serverless computing is the concept that code runs in response to specific events. These events can be as simple as an HTTP request or as complex as a message appearing on a queue, a timer expiring, or a file being uploaded to storage. This event-driven model makes us think about how services coordinate. Instead of traditional synchronous calls, where one service directly calls another, serverless systems often use integration services such as Azure Queue Storage, Service Bus, or Event Grid, which separate producers from consumers and support asynchronous communication.

Once you determine the integration type, you can select the trigger. Triggers are the mechanism that decides when a function runs. Bindings offer declarative links to external resources, such as queues, tables, or blob storage. Together, they hide boilerplate code for integration, enabling

developers to focus on business logic. The fact is that functions rarely live in isolation. Their power comes from integrating with other Azure services, and choosing the proper integration technology is as essential as writing the function code itself.

For example, a function with an HTTP trigger can verify a patient's registration data and then send a message to a queue. A second function with a queue trigger reacts to that message, coordinating downstream tasks such as saving data in Azure Table Storage and sending confirmation emails. This pattern of chaining triggers and bindings establishes a clear separation of concerns: the first function handles receiving and validating requests, while the second manages durable, asynchronous processing.

Designing microservices with serverless in mind requires more than just moving code into Azure Functions. It involves rethinking service granularity, carefully implementing DI to enhance reusability and testability, and adopting powerful orchestration frameworks, such as Durable Functions, to model real-world application workflows.

Building effective serverless microservices requires more than writing functional code. These services must be observable, secure, and maintainable in production. In the next section, we'll explore best practices for monitoring, tracing, and debugging serverless workflows, ensuring that the microservices we build are not only operational but also visible and diagnosable when issues occur.

Now that we have an idea of the considerations to take when planning a serverless application, let's review the process of creating an Azure Functions application.

## Building serverless microservices on Microsoft Azure

One of the main benefits of developing serverless microservices in Azure is the **pay-per-execution** model. We only pay when the function runs, and Azure automatically scales the service according to workload. This is ideal for healthcare systems, where usage can be unpredictable, spiking during open enrollment or flu season, and quieter at other times.

In this section, before we start the hands-on tutorial, we will outline the following:

- The core concepts of Azure Functions and how they fit into a microservices architecture
- The benefits and trade-offs of building serverless solutions
- The Azure services we will use to implement a complete appointment booking workflow
- The development and testing tools needed to build and run everything locally—on any operating system—without incurring Azure costs



For this example, we will create an appointment booking service using Azure Functions and various serverless technologies. Although Azure offers a powerful, globally accessible environment for running serverless workloads, deploying directly to the cloud during development is rarely the most efficient or cost-effective method. A disciplined development process that starts locally offers several unique benefits, including speed, cost savings, and improved quality.

First, local development accelerates the workflow. With tools such as Azure Functions Core Tools, Azurite (for Azure Storage simulation), and emulators for Azure Service Bus or Queue Storage, you can develop and test functions without waiting for cloud deployment or dealing with network delays. Changes can be tested instantly, creating a highly efficient feedback cycle. This speed is critical for complex workflows, durable functions, or integration points, such as message queues and storage tables, where trial and error often play a significant role. While this is ideal for local and cost-effective development, it is essential to remember that a local environment does not fully represent the actual production environment. In production, we must contend with cold-starts, virtual networks, delays, and several other situations that we cannot replicate in a local environment.

Cost efficiency is another key consideration. Although serverless functions are billed per execution and can be cost-effective at scale, frequent redeployment and repeated triggering during development can quickly increase resource use. Using local emulators means you avoid Azure costs until your application is ready for staging or production testing. This makes local development especially appealing for early-stage prototypes or lengthy development cycles.

Local tools ensure consistency across platforms. Whether you develop on Windows, macOS, or Linux, the Azure Functions Core Tools and supporting emulators behave the same on different operating systems. This allows your whole team to work in their preferred development environment without concerns about environment-specific problems. By standardizing on local tools, you remove dependencies on internet access and can work offline—something not possible if every test needs a cloud deployment.

In short, although Azure's hosting platform is the primary deployment target, local development tools form the backbone of a fast, safe, and cost-efficient serverless workflow. By utilizing emulators and cross-platform tools, you can move quickly, collaborate effectively, and be confident that what you eventually deploy to the cloud has already been tested in a realistic yet fully controlled environment.

Next, we will set up our environment using cross-platform local tools.

# Recommended local toolset for Azure Functions development

To optimize the advantages of local development before deploying on Azure, a dependable collection of free, cross-platform tools is essential. These tools emulate Azure’s key services on your local machine, providing a uniform developer experience across different operating systems. The described setup functions seamlessly on Windows, macOS, and Linux, enabling all team members to collaborate without environment-related obstacles.

Firstly, Visual Studio Code is a lightweight, extensible editor with rich integration with Azure Functions, making it ideal for cross-platform serverless development. For developing Microsoft Azure Functions, we can install the Azure Functions extension, which provides templates, debugging, and deployment features. We can also install Azure Storage, used for working with queues, tables, and blobs in the emulator. Azure Tools, as seen in *Figure 15.1*, provides all the Azure-related extensions that you may need for other forms of development.

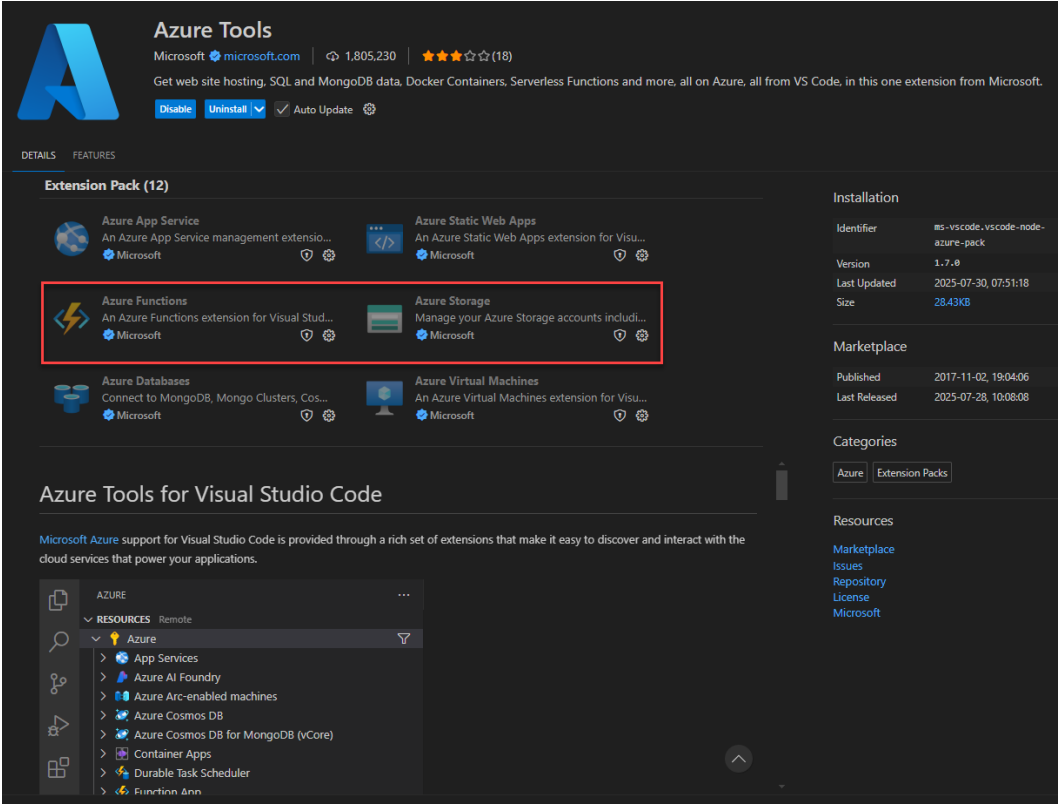


Figure 15.1 – The Azure Tools extension and all the additional extensions that will be installed

We also need to install the Azure Functions Core Tools. This runtime enables you to create, run, and debug Azure Functions locally, with complete support for triggers and bindings. This is available for all operating systems. Next, we can use Azurite (Azure Storage Emulator) to emulate Azure Storage locally. This allows us to simulate queues, tables, and blobs and test without provisioning cloud resources. Docker can be used for this setup, where the following command will create and run a container for the emulator:

```
docker run -p 10000:10000 -p 10001:10001 -p 10002:10002 \  
-v azurite_data:/data mcr.microsoft.com/azure-storage/azurite
```

To interact with the resources in the emulator, we can use Azure Storage Explorer. Now that we have configured our local environment for development and testing, let us proceed to create our serverless appointment microservice.

## Building an appointment booking service on Azure Functions

We can begin by creating a new folder and project using the following commands:

```
mkdir AppointmentBooking && cd AppointmentBooking  
func init --worker-runtime dotnet-isolated --target-framework net8.0
```

Next, we need packages to support our Azure Storage tables, queues, and durable tasks. We can install the packages with these commands in the terminal:

```
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Storage.  
Queues  
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Tables  
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.DurableTask
```

We are adding the durable task package to facilitate an orchestrated process that writes to an Azure Storage table and sends two emails. This will help us achieve reliability, fan-out/fan-in, and history with minimal code complexity.

In our project, we need to configure local settings using `local.settings.json`, which allows us to specify parameters for local development. These settings do not apply once the application is published. This file can be used to set values such as the Azurite connection string and SMTP configuration for local testing:

```
{  
  "IsEncrypted": false,  
  "Values": {
```

```
"AzureWebJobsStorage": "UseDevelopmentStorage=true",
"FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
// Durable Functions storage (uses the same Azurite account)
"AzureWebJobsStorage__serviceUri": "",
"Host": {
  "LocalHttpPort": 7071,
  "CORS": "*"
}
}
```

Let's analyze this thoroughly. This JSON file represents Azure's application settings. It shows environment variables for hosted applications. First, note that the settings are not encrypted, as indicated by the `IsEncrypted: false` line. This should be `false` for local development. In production, the settings are stored securely in Azure App Configuration or Key Vault, allowing Azure to manage encryption instead of this file.

The `Values` section holds key-value pairs of all environment variables required for the Functions runtime and your application. When running locally, these get injected into your process just like environment variables in the cloud.

`Host` provides configuration specific to the Functions host itself (the local Functions runtime). This is not about your app logic, but rather about how the Functions emulator behaves when you run the `func start` command.

The `AzureWebJobsStorage` section allows us to set `UseDevelopmentStorage=true`, which tells the Functions runtime to use Azurite, the local Azure Storage emulator. In Azure, this setting would contain a real storage account connection string.

We also need to determine the runtime of the Azure function. `FUNCTIONS_WORKER_RUNTIME` defines a `dotnet-isolated` runtime model. Azure Functions supports two models for .NET:

- **In-process model:** Functions are executed within the Azure Functions host process
- **Isolated worker model:** Functions operate in a separate process, offering improved isolation, dependency management, and full support for .NET 8

Since we're focusing on modern serverless development, `dotnet-isolated` is the recommended choice.

AzureWebJobsStorage\_\_serviceUri is an additional configuration sometimes used by Durable Functions. It allows you to override the storage URI where the orchestration state is kept. In local development, it's left blank, since Durable Functions will use Azurite. In production, this could point to a specific storage account endpoint if you want to separate orchestration data from regular queues/blobs.

Now, we can define the data contract models as `Patient.cs`, `Appointment.cs`, and `BookingRequest.cs`. Create a new folder called `Models` and create the respective files with the corresponding properties:

```
// Models/Patient.cs
public record Patient(string FirstName, string LastName,
    string Email, DateTime DateOfBirth);

// Models/Appointment.cs
public record Appointment(DateTime StartsAtUtc,
    TimeSpan Duration, string ProviderId, string? Location = null );

// Models/BookingRequest.cs
// The HTTP contract we accept and pass through the queue/orchestrator.
public record BookingRequest(Patient Patient, Appointment Appointment);
```

In the same folder, we can define table entity models that will model our Azure tables. One represents patients, and the other is for the appointments:

```
// Models/PatientTableEntity.cs
using Azure;
using Azure.Data.Tables;

public class PatientTableEntity : ITableEntity
{
    public string PartitionKey { get; set; } = "Patients";
    public string RowKey { get; set; } = Guid.NewGuid().ToString("N");
    public string FirstName { get; set; } = default!;
    public string LastName { get; set; } = default!;
    public string Email { get; set; } = default!;
    public DateTime DateOfBirth { get; set; }
    public DateTimeOffset? Timestamp { get; set; }
    public ETag ETag { get; set; }
```

```

    }
    // Models/AppointmentTableEntity.cs
    public class AppointmentTableEntity : ITableEntity
    {
        public string PartitionKey { get; set; }
        public string RowKey { get; set; } = Guid.NewGuid().ToString("N");
        public string PatientEmail { get; set; } = default!;
        public DateTime StartsAtUtc { get; set; }
        public TimeSpan Duration { get; set; }
        public string ProviderId { get; set; } = default!;
        public string? Location { get; set; }
        public DateTimeOffset? Timestamp { get; set; }
        public ETag ETag { get; set; }
    }

```

PatientTableEntity and AppointmentTableEntity require additional using statements because we are defining an entity model class for use with Azure Table Storage. We need to define PartitionKey, which Azure Table storage combines with RowKey to identify each row uniquely. The PartitionKey is a string that determines the partition where an entity is stored. All entities with the same PartitionKey are stored in the same logical partition, and a partition serves as the unit of scalability in Table storage.

Think of the table as a library, and each partition is a bookshelf. The PartitionKey is the label that says which bookshelf a book (row) belongs to. This improves indexing and performance for read/write operations. We can also create a shared helper method that can be used anywhere in the Functions application to access the Table storage:

```

// Infrastructure/TableStorage.cs
using Azure.Data.Tables;

public static class TableStorage
{
    public static TableClient GetClient(string tableName)
    {
        var conn =
            Environment.GetEnvironmentVariable("AzureWebJobsStorage")
                ?? "UseDevelopmentStorage=true";
        var service = new TableServiceClient(conn);
    }
}

```

```

        var client = service.GetTableClient(tableName);
        client.CreateIfNotExists();
        return client;
    }
}

```

As mentioned before, Azure Functions are activated via triggers. You will determine the most appropriate trigger based on the chosen communication model and the varying integration needs. For instance, if this patient onboarding function should be called synchronously, then an HTTP-triggered method might be the most appropriate. Alternatively, there may be a need to complete the onboarding asynchronously, and the function can be created to monitor a specific queue. Once a message is present, the function will execute. We do both, where we start the process by calling the HTTP-triggered function, which performs preliminary checks before placing a message on a queue for background processing.

Let us begin with the **HTTP-triggered** function, which will POST a patient, perform validation checks, and then enqueue a message if all is well:

```

// Functions/StartBookingHttp.cs
using System.Net;
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Azure.Storage.Queues;
using Azure.Storage.Queues.Models;
public class StartBookingHttp
{
    private const string QueueName = "appointments";
    private readonly QueueClient _queue;
    private bool isValid = true;

    public StartBookingHttp()
    {
        var conn =
            Environment.GetEnvironmentVariable("AzureWebJobsStorage");
        // Ensure consistent Base64 message encoding
        var options = new QueueClientOptions { MessageEncoding =
            QueueMessageEncoding.Base64 };
    }
}

```

```
_queue = new QueueClient(conn, QueueName, options);
_queue.CreateIfNotExists();
}

[Function("StartBookingHttp")]
public async Task<HttpresponseData> Run(
    [HttpTrigger(AuthorizationLevel.Anonymous, "post")]
    HttpRequestData req,
    FunctionContext ctx)
{
    BookingRequest? payload;
    try
    {
        var body = await new StreamReader(req.Body).ReadToEndAsync();
        payload = JsonSerializer.Deserialize<BookingRequest>(body,
            new JsonSerializerOptions { PropertyNameCaseInsensitive =
                true });
    }
    catch
    {
        return await BadRequest(req, "Malformed JSON.");
    }
    if (string.IsNullOrEmpty(br.Patient.FirstName))
        isValid=false;
    if (string.IsNullOrEmpty(br.Patient.LastName))
        isValid=false;
    if(!isValid) return await BadRequest(req, $"Validation failed");

    var json = JsonSerializer.Serialize(payload);
    await _queue.SendMessageAsync(json);
    var ok = req.CreateResponse(HttpStatusCode.Accepted);
    await ok.WriteStringAsync("Booking request accepted and
        queued for processing.");
    logger.LogInformation("Enqueued booking for {Email} at {Start}",
        payload!.Patient.Email, payload.Appointment.StartsAtUtc);
    return ok;
}
```



```
private static async Task<HttpresponseData> BadRequest(  
    HttpRequestData req, string msg)  
{  
    var res = req.CreateResponse(HttpStatusCode.BadRequest);  
    await res.WriteStringAsync(msg);  
    return res;  
}  
}
```

Now, let us implement the **queue-triggered** function. This function monitors the appointment queue and initiates the booking orchestrator, which performs a series of sequenced operations necessary to create an appointment:

```
// Functions/StartBookingQueue.cs  
using System.Text.Json;  
using Microsoft.Azure.Functions.Worker;  
using Microsoft.DurableTask.Client;  
  
public class StartBookingQueue  
{  
    [Function("StartBookingQueue")]  
    public async Task Run(  
        [QueueTrigger("appointments")] string message,  
        [DurableClient] DurableTaskClient client,  
        FunctionContext ctx)  
    {  
        BookingRequest? request = null;  
        try  
        {  
            request = JsonSerializer.Deserialize<BookingRequest>(message,  
                new JsonSerializerOptions {  
                    PropertyNameCaseInsensitive = true });  
        }  
        catch (Exception ex)  
        {  
            logger.LogError(ex, "Failed to deserialize booking message.");  
            return;  
        }  
    }  
}
```

```

    }

    if (request is null)
    {
        logger.LogError("Null booking request.");
        return;
    }
    var instanceId = await
        client.ScheduleNewOrchestrationInstanceAsync(
            nameof(BookingOrchestrator), request);
    logger.LogInformation("Orchestration started: {InstanceId}",
        instanceId);
}
}

```

Now, we have our orchestrator function that will perform specific actions in a particular sequence. The following durable function serves as the central workflow definition for handling patient appointment bookings:

```

// Functions/BookingOrchestrator.cs
using Microsoft.Azure.Functions.Worker;
using Microsoft.DurableTask;

public class BookingOrchestrator
{
    [Function(nameof(BookingOrchestrator))]
    public async Task Run([OrchestrationTrigger]
        TaskOrchestrationContext ctx)
    {
        var request = ctx.GetInput<BookingRequest>(!);

        // Create patient
        await ctx.CallActivityAsync(nameof(AddPatientActivity),
            request.Patient);

        // Create appointment
        await ctx.CallActivityAsync(nameof(AddAppointmentActivity),
            request);
    }
}

```

```
// Fan-out: notify patient & admin
var t1 = ctx.CallActivityAsync(nameof(SendPatientEmailActivity),
    request);
var t2 = ctx.CallActivityAsync(nameof(SendAdminEmailActivity),
    request);
await Task.WhenAll(t1, t2);
}
}
```

This function calls several individual functions, each of which completes a specific step in the workflow. `AddPatientActivity` will add the patient record to the Azure Storage table:

```
// Functions/AddPatientActivity.cs
using Azure.Data.Tables;
using Microsoft.Azure.Functions.Worker;

public class AddPatientActivity
{
    [Function(nameof(AddPatientActivity))]
    public async Task Run([ActivityTrigger] Patient patient,
        FunctionContext ctx)
    {
        var logger = ctx.GetLogger(nameof(AddPatientActivity));
        TableClient client = TableStorage.GetClient("Patients");
        var entity = new PatientTableEntity
        {
            FirstName = patient.FirstName,
            LastName = patient.LastName,
            Email = patient.Email,
            DateOfBirth = patient.DateOfBirth
        };
        await client.AddEntityAsync(entity);
        logger.LogInformation("Patient added: {Email}", patient.Email);
    }
}
```

The following function creates an appointment:

```
// Functions/AddAppointmentActivity.cs
using Azure.Data.Tables;
using Microsoft.Azure.Functions.Worker;

public class AddAppointmentActivity
{
    [Function(nameof(AddAppointmentActivity))]
    public async Task Run([ActivityTrigger] BookingRequest request,
        FunctionContext ctx)
    {
        var logger = ctx.GetLogger(nameof(AddAppointmentActivity));
        TableClient client = TableStorage.GetClient("Appointments");
        var dateKey =
            request.Appointment.StartsAtUtc.ToString("yyyyMMdd");
        var entity = new AppointmentTableEntity
        {
            PartitionKey = dateKey,
            PatientEmail = request.Patient.Email,
            StartsAtUtc = request.Appointment.StartsAtUtc,
            Duration = request.Appointment.Duration,
            ProviderId = request.Appointment.ProviderId,
            Location = request.Appointment.Location
        };

        await client.AddEntityAsync(entity);
        logger.LogInformation("Appointment added for {Email} at {Start}",
            request.Patient.Email, request.Appointment.StartsAtUtc);
    }
}
```

Now, we can execute two functions in parallel, where we send notification emails to both the patient and the admin. Since neither function depends on the other, they can be executed in a fan-out fashion. They are defined here:

```
// Functions/SendPatientEmailActivity.cs
using Microsoft.Azure.Functions.Worker;
```

```

public class SendPatientEmailActivity
{
    [Function(nameof(SendPatientEmailActivity))]
    public async Task Run([ActivityTrigger] BookingRequest request,
        FunctionContext ctx)
    {
        var logger = ctx.GetLogger(nameof(SendPatientEmailActivity));
        // Send Email Logic
    }
}

// Functions/SendAdminEmailActivity.cs
using Microsoft.Azure.Functions.Worker;
public class SendAdminEmailActivity
{
    [Function(nameof(SendAdminEmailActivity))]
    public async Task Run([ActivityTrigger] BookingRequest request,
        FunctionContext ctx)
    {
        var logger = ctx.GetLogger(nameof(SendAdminEmailActivity));
        // Send Email Logic
    }
}

```

Finally, to run everything locally, ensure that your Azurite container is running and then start the Functions host with the `func start` command. Now, we can test the HTTP-triggered function using the following cURL command. You may also opt to use a testing tool such as Postman:

```

curl -X POST http://localhost:{PORT_NUMBER}/api/StartBookingHttp \
-H "Content-Type: application/json" \
-d '{
    "patient": { "firstName":"Ava","lastName":"Nguyen","email":"ava@example.com",
    "dateOfBirth":"1992-05-16T00:00:00Z" },
    "appointment": { "startsAtUtc":"2025-08-25T14:00:00Z","duration":"01:00:00",
    "providerId":"dr-42","location":"Room 3" }
}'

```

You can validate the function flow by reviewing the logs in the console and verifying that the Azure tables are populated at the end. Now that we have explored serverless microservice development, let us summarize all that we have learned.

## Summary

This chapter examined how serverless computing revolutionizes microservices development by providing a highly agile, cost-efficient, and scalable alternative to traditional hosting models. It explained how Azure Functions and Durable Functions can be applied to real-world domains such as healthcare systems, where workflows require both efficiency and resilience. The discussion balanced the benefits of serverless architectures with their inherent challenges, offering practical strategies, patterns, and examples for .NET developers.

We emphasized that designing for serverless requires more than just migrating existing workloads into a new environment. Function granularity is a key design decision: functions should adhere to the single responsibility principle, be cohesive, and avoid excessive chaining, which can cause latency and tracing challenges. We also highlighted naming conventions and dependency injection practices, paying particular attention to the isolated worker model in Azure Functions that aligns with ASP.NET Core conventions while supporting modern .NET features.

After exploring the fundamentals of different Azure serverless technologies, we reviewed a local development setup using Visual Studio Code with Azure extensions, Azure Functions Core Tools, and the Azurite storage emulator. This allowed us to build and test serverless applications cross-platform without incurring cloud costs. This local-first workflow enhances collaboration and confidence before deployment.

Our healthcare appointment booking system served as a guiding case study. It showed how HTTP-triggered functions can gather patient input, perform validation, and enqueue requests for background processing. Queue-triggered orchestrations then coordinated activities such as patient creation, appointment scheduling, and sending email notifications to patients and administrators. Azure Storage Tables were used for data persistence, with partition keys ensuring scalability and efficient queries. The system demonstrated how combining triggers, durable orchestration, and fan-out execution creates resilient workflows.

In this chapter, serverless microservices was positioned as a powerful but nuanced paradigm. The benefits of scalability, agility, and reduced infrastructure burden were apparent; however, they also come with challenges related to latency, state persistence, and vendor lock-in. Through Durable Functions, thoughtful function granularity, and strong observability and security practices, teams can overcome these limitations. By using practical workflows, such as healthcare appointment scheduling, the chapter demonstrated how .NET developers can harness Azure Functions and Durable Functions to build resilient, event-driven systems that gracefully balance business requirements with technical constraints.

To provide a comprehensive view, it's essential to explore how serverless computing can be approached using other major cloud providers such as AWS and GCP. AWS offers AWS Lambda, which allows you to run code without provisioning or managing servers. It integrates seamlessly with other AWS services, enabling you to build a wide range of serverless applications. For more information, you can refer to <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>.

Similarly, GCP provides Cloud Functions, a lightweight, event-driven compute solution that allows you to create small, single-purpose functions that respond to cloud events without the need to manage a server or runtime environment. You can learn more about it at <https://cloud.google.com/functions/docs>.

By understanding the serverless offerings from Azure, AWS, and GCP, developers can make informed decisions on the best platform for their specific needs, ensuring they leverage the strengths of each provider to build scalable, efficient, and resilient microservices.

Now, we will move on to exploring cross-cutting concerns related to monitoring and observing microservices.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW



# 16

## Observability and Monitoring with Modern Tools

One of the biggest challenges with APIs is that we rarely receive actual feedback on what is happening in our service. We strive to design our services in a way that our HTTP responses clearly indicate the success or failure of each operation, but this is not always sufficient. The most concerning types of responses are those in the 5xx range, which provide no useful information.

As microservice architectures become more complex, understanding the internal workings of the system becomes a crucial part of maintaining healthy applications. Earlier in this book, we discussed how communication patterns, resilience mechanisms, and orchestration strategies help microservices withstand inevitable failures. However, even the most resilient system is vulnerable if developers and operators lack visibility into its inner processes. This is why observability is essential.

Monitoring is essential but only indicates when issues arise. Observability goes further by helping us understand why problems occur. It involves inferring a system's internal state by analyzing data such as logs, metrics, and traces. These three components turn a distributed system from a mysterious "black box" into a transparent, diagnosable environment. Modern .NET developers now have powerful tools to make observability not just possible but straightforward.



This chapter guides you through implementing observability in .NET microservices. We start with logging strategies, setting up structured, centralized logs that turn diagnostic data into actionable insights rather than noise. By the end of this chapter, you will be able to do the following:

- Implement structured, centralized logging across microservices
- Understand log aggregation and distributed tracing
- Use the sidecar pattern to centralize logging and telemetry
- Use .NET Aspire for microservice orchestration and monitoring

Observability is not just an add-on feature but a core architectural capability. Without it, the microservice patterns we have studied, such as sagas, API gateways, and CQRS, may fall apart due to their own complexity. With it, we gain the clarity and confidence needed to operate distributed systems at scale. In distributed microservices, monitoring alone only tells us that something is wrong. Observability enables us to delve deeper and ask *why*. To achieve this, we rely on three complementary pillars:

- **Logs:** Rich, contextual, timestamped records of discrete events
- **Metrics:** Numerical measures of system health over time (e.g., latency, error rates)
- **Traces:** End-to-end request flows across services, broken into spans

Together, these signals turn a healthcare system from a black box into a transparent environment. Imagine a patient booking an appointment: logs show request activity, metrics reveal increased latency, and traces highlight a bottleneck in the notification service. Without all three, we would struggle to pinpoint the cause.

Let's start by examining logging, which is the foundation of observability in microservices.

## Technical requirements

The code used in this chapter can be found in the project repository, which is hosted on GitHub at this URL: <https://github.com/PacktPublishing/Microservices-Design-Patterns-in-.NET---Second-Edition/tree/main/Chapter16>.

Now, let us review what it means to monitor our microservices and implement logging.

## Implementing structured logging

Logs are blocks of text that most applications produce during runtime. They are designed to be mini-reports about what is happening in the application, allowing us to track and trace errors that occur within our applications.

In a monolithic application, we typically write logs to a log file or a database. In fact, in .NET Core, we have access to powerful logging providers and third-party libraries that allow us to integrate with several log output destinations. There is no absolute best destination, and while some work better than others, it is really a matter of project preference and overall comfort.

Logging in a monolithic setup is straightforward. Keep in mind that all components of a monolith are in the same space and are directly connected. In a distributed system, however, this gets more complicated because activities occur across multiple applications. The standard approach is to create logs for each service, which can result in numerous log files, each containing parts of the information required for the application. Imagine having to check multiple log files to troubleshoot a failure that happened at 5:00 P.M. To understand what went wrong, you would need to review several sources and piece together meaningful information, which can be a challenging task.

This investigation gets even more difficult if our logs are too verbose. A verbose log reports all events that occur within the application. Even if we don't report everything, we need to be clear about what we log to reduce noise and better highlight the key events that need to be captured.

We will then need a clean way to centralize the logs that are generated across the services. A centralized database may seem like a good idea, but it can lead to resource and table locking if multiple services frequently attempt to write logs. We will review the best centralization techniques later. For now, let us focus on determining what the most valuable pieces of information are to log and the general requirements for implementing logging.

## Determining log requirements

A crucial decision when implementing logging is what to log. Do we want to log a play-by-play sequence of everything that happens in our application, or do we only want to record errors? Different systems have different requirements, and the correct choice depends on how crucial the service is to the overall running of the application.

Now that we have determined the most essential services that require closer monitoring through logs, we need to determine what information will be logged. Recall that we don't want our logs to be too chatty, but we don't want to neglect to place pertinent information in the logs. Too much information can lead to large, useless logs and high storage costs, while too little information will result in useless files. Helpful information would include, but is not limited to, the following:

- The ID of a resource that is being accessed through an API request
- The different functions being invoked during a request cycle
- A correlation ID, which is an ID that is unique to a particular operation

What we want to avoid is logging sensitive information. We do not want to log, for example, the following:

- User credentials during an authentication flow
- Payment information
- **Personally identifiable information (PII)**

Despite our best security efforts, logs remain a primary source of truth about our system, and a security breach involving sensitive information in log files could prove to be a detrimental event. There are laws and data protection laws that govern how we should store and secure our logs.

For example, the **General Data Protection Regulation (GDPR)**, enforced throughout the European Union, requires strict regulations on managing PII, such as names, email addresses, and IP addresses, including its collection, storage, and retention. GDPR also obligates organizations to allow users to request the deletion of their data, which can be nearly unfeasible if sensitive information is embedded in log files.

Similarly, the United States' **Health Insurance Portability and Accountability Act (HIPAA)** enforces strict standards for safeguarding health information, requiring logs that contain patient identifiers, treatment details, or medical histories to be anonymized or omitted. In the financial industry, the **Payment Card Industry Data Security Standard (PCI DSS)** bans storing cardholder data, such as credit card numbers or CVV codes, in logs. Additional frameworks, including the **California Consumer Privacy Act (CCPA)** and **ISO/IEC 27001**, also emphasize data minimization—collecting and keeping only what is necessary for operational needs.

Thus, it's preferable to log only IDs that can be retrieved on demand, avoiding the collection of personal or sensitive data initially. This approach facilitates adherence to relevant laws and minimizes the risk of data leaks and potential penalties in the event of a breach.

We also have the concept of **log levels**, which categorize the severity of a log message. These levels are split into *information*, *debug*, *warning*, *error*, and *critical*. There might be other categories in between, but these are the most used ones. This is what they stand for:

- **Information:** A standard log level that is used when something has happened as expected. Generally used to share information about an operation, it can help trace the sequence of operations that might have led to an error.
- **Debug:** A very informational log level that is more than we might need for everyday use. It is primarily helpful for development and helps us to track more intricate operations in the code. Production systems generally do not produce debug logs.

- **Warning:** This log level indicates that something has happened that isn't an error but isn't normal. Think of it as an amber light that suggests that some attention should be given to a situation, but it might not be mission-critical.
- **Error:** This refers to an error. This type of log entry is usually created when an exception is encountered. This can be paired with the exception and stack trace and proves to be a critical type of log to have when debugging issues.
- **Critical:** This indicates that we encountered an error that we cannot recover from. This type of log entry can be used when the application fails to start or cannot connect to a critical data source or dependency.

Log levels serve as a universal language, so we must accurately represent the situation being logged with the appropriate level. We also need to avoid misclassifying events and logging deceptive information about what has occurred in our system.

Once again, the ultimate decision on what is logged is relative to the application and the developers' and organization's needs, and we need to ensure that we are thorough enough in capturing the essential bits about the application's runtime. Now, let us review how we implement logging in .NET.

## Using the .NET logging API

.NET has a built-in logging mechanism that is baked into our application startup operation. We get an out-of-the-box logging library that produces logs on all the happenings of our application as soon as it is started. This mechanism works with several third-party logging providers, making it both extensible and powerful. With our providers, we can determine the target destinations for our logs.

We start with the **ILogger** interface. This interface is an abstraction of the logging API that ships with .NET. It is made available to us through the `Microsoft.Extensions.Logging` NuGet package. This library provides us with the necessary classes, interfaces, and functionality for application logging, as well as providers for logging to the *Console*, *Debug*, *Azure App Services Logging*, *EventSource*, and *Windows Event Log*:

- **Console:** This provider outputs logs to the console. A console window appears when debugging, and most IDEs provide a debugging console window for runtime logs.
- **Debug:** This provider writes log entries using the `System.Diagnostics.Debug` class.
- **EventSource:** A cross-platform provider that can act as an event source with the name `Microsoft-Extensions-Logging`.

- **Windows Event Log:** A Windows-only provider that sends log output to the Windows event log. It will only log *warning* messages by default, but can be configured as needed.
- **Azure App Services logging:** This writes to the App Service filesystem and/or blob storage. We can easily write logs to this provider.

To get our .NET application to begin writing logs, we inject `ILogger<T>` into the class from which logs should originate. `T` represents the name of the class that we are injecting the service into. This helps with log categorization and filtering later because the logs will automatically indicate the class name when they are generated. `ILogger<T>` is typically used by application code, which may be in several places. Because the class name is being used as a *category*, it makes it easy for us to link the log entries back to the class that produced them. In the following code, we inject `ILogger<T>` into our `AppointmentController`:

```
public class AppointmentController(ILogger<AppointmentController>
    logger, /* Other Services */) : ControllerBase
{
    /* Code */
}
```

Injecting `ILogger<T>` is standard compared to how we inject other services. The benefit of now having this logger present is that we can write logs to inform us of the activities and errors in our API. If we need to log each time a list of appointments is retrieved through an API call, we can modify our GET method like this:

```
// GET: api/Appointments
[HttpGet]
public async Task<ActionResult<IEnumerable<Appointment>>>
    GetAppointments()
{
    logger.LogInformation("Returning Appointments");
    return await _context.Appointments.ToListAsync();
}
```

With the preceding code, GET requests to this endpoint will generate a message in the debug console that launches and displays messages about the running .NET application:

```
HealthCare.Appointments.Api.Controllers.AppointmentController:
    Information: Retrieving appointments
```

Note that we can clearly see not only the source calls but also their namespace, which plays a significant role in helping us determine which exact class is producing the log. We also receive a log-level flag, allowing us to determine the severity immediately. You will also notice that there are many other default log entries that we didn't orchestrate. We can control the global levels and sources of logs that we want in our application through the `appsettings.json` file. By default, it will have the following configuration:

```
"Logging": {  
  "LogLevel": {  
    "Default": "Information",  
    "Microsoft.AspNetCore": "Warning"  
  }  
},
```

This specifies that the minimum default level that should be output to the destinations is `Information` for the default logging source. Anything related to the inner workings of our application should only be surfaced when it is a warning. If we were to modify and place them both in `Information`, then our logs would become far chattier from the get-go.

Our `LogLevel` methods have a standard layout that allows us to include additional information as needed easily. The possible parameters are as follows:

- `eventId`: A numeric value that you would associate with an action in your application. There is no predefined standard for this, and you can assign your own values based on your organization and needs. This is an optional parameter, but it can be helpful when we need to associate logs with a particular action.
- `exception`: This is most useful when we are logging an error or critical message and wish to include the details of the exception in the log. It is optional, but it is strongly recommended that you include it whenever an exception object is available, such as after catching an exception in a `try/catch` block.
- `message`: This is a straightforward text field. It is arguably the most important one as it allows us to express our thoughts on the event being logged. It is a parameterized string that enables us to use placeholders and provide variable values without needing to concatenate or interpolate the string. An example of such a text is `Logging request from {username}`. The value for `{username}` should then be provided in `messageArgs`.
- `messageArgs`: An array of objects that will bind to placeholders that are outlined in the message string. The binding will occur in the order that the parameters appear, and therefore, the values should be provided in the same order. If a value is not provided, the placeholder will be printed out in the string.

An example that uses all the parameters could look something like this:

```
public async Task<ActionResult<AppointmentDetailsDto>>
    GetAppointment(Guid id)
{
    try
    {
        var appointment = await _context.Appointments.FindAsync(id);
        if (appointment == null)
        {
            return NotFound();
        }
        // Operations
        return appointmentDto;
    }
    catch (Exception ex)
    {
        logger.LogError(100, ex, "Failure retrieving appointment
            with Id: {id}", id);
        throw;
    }
}
```

We have now added exception handling to our endpoint that retrieves the appointment record by ID. If there is an exception to any of the operations, we catch it and log it. We are using 100 as the eventId value for this operation. A more sustainable solution to improve the context and readability of the eventId values would be to use a constant class or enum that defines the values with readable and descriptive names. Here is an example:

```
public enum LogEvents
{
    // General application lifecycle
    RetrieveAppointmentFailure= 100,
    // etc...
}
```

The updated logging code would appear as follows:

```
logger.LogError(LogEvents.RetrieveAppointmentFailure, ex, "Failure
    retrieving appointment with Id: {id}", id);.
```

We are logging the exception and including a custom message with additional information to help us better contextualize the nature of the exception. We also included the ID of the record that caused the failure. Notice the {id} placeholder, which will map to the argument ID. Giving the parameters the same name is not necessary, but it does help to reduce any confusion with the value bindings.

If we want to extend the number of providers that should be used for each log message, we can configure the logging settings in the `Program.cs` file of our application. In a standard boilerplate ASP.NET Core project, we can add code that looks like this:

```
builder.Logging.ClearProviders();
builder.Logging.AddConsole()
    .AddEventLog(new EventLogSettings { SourceName = "Appointments Service" })
    .AddDebug()
    .AddEventSourceLogger();
```

We first clear any preconfigured providers and then add all the providers that we wish to support. Now, one log message will be written to several destinations. This can be a convenient way to fan out our log messages and have different monitoring methods attached to each destination. Remember that you should always be aware of the information security rules that govern your country and company, and try not to expose too much information in too many places. We can also provide specific configurations per provider by modifying the logging configuration in the `appsettings.json` file, as shown here:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Error",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Warning"
    },
    "Debug": {
      "LogLevel": {
        "Default": "Trace"
      }
    },
    "Console": {
      "LogLevel": {
        "Default": "Information"
      }
    }
  }
}
```



```

    }
  },
  "EventSource": {
    "LogLevel": {
      "Microsoft": "Information"
    }
  },
  "EventLog": {
    "LogLevel": {
      "Microsoft": "Information"
    }
  },
}
}

```

We retain our default log levels for each log source, but then provide overrides per provider. If the log source is not defined under the provider's configuration, it will retain the default behaviors. However, we reserve the right to control what type of log each provider should prioritize.

If we needed to extend support to Azure systems, we could utilize the Azure application's file storage and/or blob storage. We start by including `Microsoft.Extensions.Logging.AzureAppServices` via the NuGet package manager, and then we can configure the logging services with code like this:

```

builder.Logging.AddAzureWebAppDiagnostics();
builder.Services.Configure<AzureFileLoggerOptions>(options =>
{
    options.FileName = "azure-log-filename";
    options.FileSizeLimit = 5 * 2048;
    options.RetainedFileCountLimit = 15;
});
builder.Services.Configure<AzureBlobLoggerOptions>(options =>
{
    options.BlobName = "appLog.log";
});

```

This will configure the app to use both the filesystem and Azure Blob Storage. Based on the App Service log settings, there are some defaults that we can look for in terms of log output locations. We can similarly override the default log output by this provider by adding sections to the `appsettings.json` file with the `AzureAppServicesBlob` and `AzureAppServicesFile` aliases. We

can also define `ApplicationInsights` if we intend to use that service for our application. To support `Application Insights`, we need the `Microsoft.Extensions.Logging.ApplicationInsights` NuGet package. Azure Application Insights is a powerful log aggregation platform provided by Microsoft Azure and is an excellent choice for an Azure-hosted solution.

Several third-party frameworks extend the capabilities of the built-in logging API of .NET. We will explore how to integrate a popular logging framework, Serilog, in the next section.

## Adding Serilog

Several third-party frameworks exist that extend the logging capabilities and options available to us in our .NET applications. Popular choices include **Serilog**, **Log4Net**, **elmah.io**, and **NLog**, among others. Each one has its pros and cons, but we will explore Serilog as it best fits our needs for our microservices application.

Even though we will be using Serilog, let us review the alternatives.

- **NLog**: A solid and high-performing solution with good targets, but out of the box, its structured logging and **OpenTelemetry (OTel)**/modern cloud enrichers aren't as seamless for our needs. Additionally, the middleware and ecosystem are not optimal for our ASP.NET Core and containers baseline.
- **Log4Net**: Stable but legacy ergonomics, with slower progress on .NET 8/9 features and cloud integrations we depend on.
- **elmah.io**: Hosted options offer a great experience, but we required sink-level portability across multiple destinations and self-hosted options for specific environments.

Serilog treats logs as *events with properties*. That means every write includes machine-queryable fields (e.g., `PatientId`, `AppointmentId`, `CorrelationId`, `Route`, `ElapsedMs`). This is critical for microservices where we need to filter, join, and pivot logs across services. It also simplifies PII minimization, allowing us to log lookup IDs and omit sensitive values while still preserving rich context.

Serilog integrates with the `Microsoft.Extensions.Logging` abstraction via an adapter and can *replace* the default providers. Your code continues to depend on `ILogger<T>`, while Serilog becomes the underlying engine that handles enrichment and writes to sinks. We can begin using it with the following package installation:

```
dotnet add package Serilog.AspNetCore
```

Serilog has the concept of **sinks**. A sink is like a logging provider in concept and represents an output channel for the logs written by the framework. We need to add additional packages per sink that we wish to support. Popularly used sinks are *Console*, *File*, *Seq*, *SQL Server*, and *Azure Application Insights*, to name a few. You can get a complete list from their GitHub wiki page: <https://github.com/serilog/serilog/wiki/Provided-Sinks>.

For this exercise, we will configure Serilog to use both the file and console sinks. We will also be adding parameters to our `appsettings.json` file. We will need the following Serilog expressions extension to the base library to support parsing the JSON text into the required settings:

```
dotnet add package Serilog.Expressions
```

Now we can remove the Logging section from our `appsettings.json` file and replace it with the following JSON text:

```
"Serilog": {  
  "MinimumLevel": {  
    "Default": "Information",  
    "Override": {  
      "Microsoft": "Warning",  
      "Microsoft.Hosting.Lifetime": "Information"  
    }  
  },  
  "WriteTo": [  
    {  
      "Name": "File",  
      "Args": { "path": ".\\logs\\log-.txt", "rollingInterval": "Day" }  
    }  
  ]  
},
```

Now we have a similar structure that allows us to define logging defaults for the application. Still, we also have a `WriteTo` section that enables us to list the different channels we want to support. We have only included the settings for file writes, and we have specified the target location to be a local folder called `logs`. Files will be created daily and automatically assigned a name that is a combination of the `log-` expression and a date. This will make it easy for us to source the relevant files by day, and each log will indicate a timestamp, making it easier to review the events.

Now we can remove the `builder.Logging(...)` configuration and replace it with this:

```
builder.Host.UseSerilog((ctx, lc) => lc
    .WriteTo.Console()
    .ReadFrom.Configuration(ctx.Configuration));
```

This will initialize our logger to use the `Console` sink and read the configuration object for the `Serilog` section defined previously. This will initialize the `Console` and `File` sinks. We can now look forward to seeing text files getting created and populated daily for each microservice that has implemented the file logging configuration for `Serilog`. The code required to write the logs remains the same. We still only need to repeat the injection steps outlined for `ILogger<T>`; `Serilog` will handle the rest.

Now we have solved one issue where we are no longer blind to what is happening in our application. We can easily integrate logging into our services and review a more persistent output in the form of log files. We still, however, face the challenge of reviewing several disparate logs across different systems to properly track what may have led to a failure at some point. This is where we need to explore methods for aggregating the logs and making them visible and searchable from a single interface. We will explore this in the next section.

## Log aggregation and its uses

**Log aggregation** is the concept of capturing and consolidating logs from different sources onto a centralized platform. This is particularly useful in a distributed system where logs are generated from multiple sources, and we need a comprehensive view of all the messages to efficiently correlate and analyze the logs.

This log aggregation serves as a single source of truth for troubleshooting application performance, identifying errors, and pinpointing bottlenecks, points of failure, or vulnerabilities. Several platforms enable us to aggregate our logs, ranging from free to paid and cloud-hosted solutions. Some popular options include **Azure Application Insights**, **Seq**, **Datadog**, and **Elasticsearch, Logstash, and Kibana (ELK)**, among others. When selecting a platform, we must consider the following:

- **Efficiency:** We all love and want efficient systems. The platform we choose needs to support this narrative and make it as easy as possible to integrate logging, export log information in various formats, and quickly sift through log information. Most log aggregators enable us to author queries that can intelligently sift through the logging noise and provide more targeted data.

- **Processing power:** The platform needs to allow for comfortable throughput from several sources and be able to index, compress, and efficiently store these logs. We may not necessarily know their techniques to achieve this, but we can assess the accuracy of the indexing functions through our queries and the overall presentation of the data.
- **Real-time features:** Real-time monitoring is critical since we generally need log aggregation for the purpose of monitoring what is happening in our application. The more instantly the information is made available to us, the more quickly we can respond to a failure.
- **Scalability:** The platform needs to be capable of handling varied periods of traffic and not break when there is a sudden shift in volume. We must ensure that the system's performance does not degrade under increased load.
- **Alerting mechanisms:** Some platforms have built-in functionality to alert us to certain types of log events. Even if this is not built in, we should have integration options through **APIs** and **webhooks** that allow us to integrate with our third-party applications, where we spend most of our time.
- **Security:** Security is essential with our logging information, as we have mentioned before. An ideal platform will encrypt data both at rest and in transit. This is usually the case, but we need to confirm it. We may also need to control user access.
- **Cost:** We all love free and cheap solutions. We cannot always have the best of both worlds, but be sure that the platform offers a good return on investment, relative to the features that are being offered. Ensure that you do a proper cost-benefit analysis.

The easiest way to integrate with a log aggregation platform is through tools and packages that are specifically optimized for this type of integration. We need to employ the services of libraries that are tuned to integrate with these platforms efficiently. In the next section, we will explore how to leverage Serilog and combine it with Seq.

## Integrating with Seq

**Seq**, pronounced seek, is a sleek (see what I did there?) log aggregation developed and maintained by **Datalust**. This platform has been designed to support log messaging templates output by ASP.NET Core, Serilog, and NLog. It can also be extended to support other development frameworks as needed.

It provides us with access to a powerful dashboard featuring leading data presentation and querying capabilities. Seq can be installed on a local machine for free for individual development, but it will incur some costs as we look to use it in a more enterprise setting. It also offers a hosted solution, which removes the need for users to set it up locally.

For this activity, we will use it locally and for free on our machine. We now have two options: we can use a **Docker** image and spin up a container for the application, or install it on our local machine. It is available for Windows and Linux operating systems, so we will use the Docker option to cater to all scenarios.

We start by downloading the Docker image with this command:

```
docker pull datalust/seq
```

Now that we have the latest Seq image, we will create our container with this command:

```
docker run --name seq -d --restart unless-stopped -e ACCEPT_EULA=Y -p  
5341:80 datalust/seq:latest
```

Now we have a container that hosts an instance of Seq and can be accessed through port 5341, which is Seq's default port. We can now navigate to the URL <http://localhost:5341/#/events> to see our user interface for the aggregator. It will be empty, so now we need to integrate our API with this new logging channel.

Now that we have Seq up and running, we can modify our service to begin sending logs to this platform. We already have Serilog installed, so we can add the Seq sink to our project by adding this package:

```
dotnet add package Serilog.Sinks.Seq
```

With this new package, we can modify our `appsettings.json` Serilog section and add a new object block to the `WriteTo` section of the configuration. It will now look like this:

```
"WriteTo": [  
  {  
    // File Configuration  
  },  
  {  
    "Name": "Seq",  
    "Args": { "serverUrl": "http://localhost:5341" }  
  }  
]
```

We already have the configuration section being read at application startup, so the next time the application starts up, it will write all the default logs to our local file as expected, but also to the Seq platform.

Figure 16.1 shows the Seq interface:

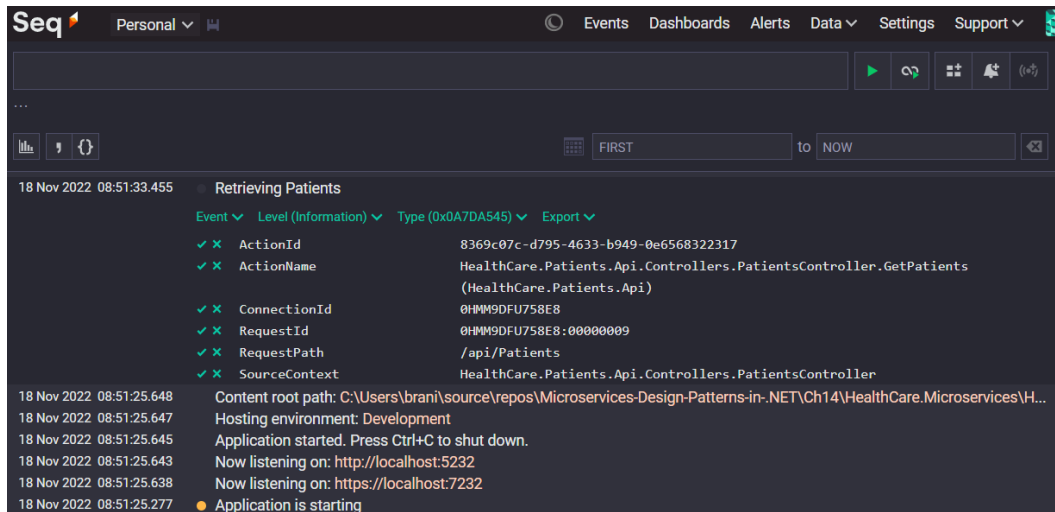


Figure 16.1 – The Seq interface receiving logs from a microservice

We see the user interface outlining the default logs that get produced at application startup. What appears in this interface is related to the logging configurations that we have added, as well as the log entries that we create as we proceed. You can also notice color-coded dots that indicate the log level for each log entry. We can click on a line and expand it to view the details of the log message.

Now, these code modifications can be made to all services that we wish to add to the log aggregation initiative, and we can use this unified platform to interrogate logs as needed. With this in place, we need to understand the concept of log tracing in a distributed setting. We will discuss this next.

## Distributed log tracing

**Distributed tracing** is a method for monitoring logs and tracing issues within a microservices application. Developers and DevOps engineers both rely on logs to track the path of a request as it traverses the various systems and checkpoints, and then attempt to identify the point of failure. The more robust the logging, the easier it will be for them to pinpoint weak points, bugs, and bottlenecks in the application.

Because microservices are designed to be autonomous and scale independently, it is common to have multiple instances of the service running simultaneously, which further complicates the request tracing process. We now need to backtrack to which instance of the service handled the request, leading to even more complex tracing situations.

Distributed tracing is a technique designed to address these problems. It refers to a diagnostic methodology behind observing requests as they make their way through distributed systems. Each trace shows the activity of an individual user in the application. In an aggregated logging system, we will end up with a collection of traces that highlight the backend service and dependencies that have the most significant impact on performance. In distributed tracing, we have three main factors that help us navigate our way.

- **Trace:** Represents an end-to-end request from user activity.
- **Span:** Represents work done by a single service in a specific time span. These combine to form a trace.
- **Tags:** Metadata about the span that helps us to categorize and contextualize the logs properly.

Each span represents a step in the entire journey of the request and is encoded with essential data related to the process carried out during the operation. This information can include things such as the following:

- The service's name and address
- Tags that can be used in queries and filters, such as the HTTP method, database host, and session ID, to name a few
- Stack traces and details error messages

.NET has evolved over the years to provide top-notch support for producing logs with these details as seamlessly as possible, through integrations with OTel. Microsoft Azure also provides an excellent distributed tracing platform in **Azure Application Insights**, a platform we mentioned earlier. Many other paid and open source solutions can support our needs for distributed tracing. For this chapter, we will use a free and simple platform called **Jaeger**. Let us explore how we can add telemetry enhancements to our services and visualize them with Jaeger.

## OTel and distributed tracing

OTel is a popular open source project that is at the helm of standardizing logging standards for distributed and cloud-native applications. It helps us generate and collect detailed logs, also known as telemetry data, which contain traces and metrics. Given that it is an open standard, we are at liberty to choose a suitable visualization and analysis tool.



To install OTEL in our ASP.NET Core application, we need to add the following packages:

```
dotnet add package OpenTelemetry.Instrumentation.AspNetCore
dotnet add package OpenTelemetry.Extensions.Hosting
```

While logs and traces provide detail, metrics give us system-wide visibility immediately. Metrics answer questions such as: *Is the appointment service healthy? Are notifications being delivered on time?* Types of metrics include the following:

- **Request throughput:** How many appointment bookings are processed per second
- **Latency:** Average response time of the patient API
- **Error rate:** Percentage of failed requests (HTTP 5xx)
- **Queue depth:** Number of pending notification messages

When evaluating observability platforms, two open source projects are often discussed together: Jaeger and Prometheus. Although they complement one another, they serve very different roles.

Jaeger is a **distributed tracing platform** created by Uber and now part of the **Cloud Native Computing Foundation (CNCF)**. It helps developers and operators track a single request as it traverses multiple services. A span represents each step in the request lifecycle, and all spans are correlated into a **trace**. It provides deep visibility into end-to-end request flows coupled with rich visualization tools. This makes it ideal for diagnosing **latency issues** and **bottlenecks** between services. It is not, however, designed for metrics collection, only traces.

Prometheus, on the other hand, is a time-series metrics platform also hosted by the CNCF. Instead of following individual requests, Prometheus collects numerical data points over time, such as request rates, error counts, and system resource usage. In our healthcare system, Prometheus might track average API response times, the number of appointments scheduled per minute, or the depth of the notifications queue. It is optimized for metrics collection and storage, with native support for alerting and integration with Grafana for dashboards. It is generally not intended for tracing individual requests, as it cannot show where a single operation failed.

Rather than being alternatives, Jaeger and Prometheus complement each other, as Prometheus might indicate that a microservice is responding slowly, whereas Jaeger indicates why the failure might occur. For this demo, we will set up to use Jaeger. It can be downloaded for free from

<https://www.jaegertracing.io/download/>, and it can be deployed as a Docker container by using the following `docker run` command:

```
docker run -d --name jaeger -p 16686:16686 -p 14250:14250 -p 4317:4317
-p 4318:4318 -e COLLECTOR_OTLP_ENABLED=true -e LOG_LEVEL=debug --restart
unless-stopped jaegertracing/all-in-one:1.49
```

Once the container is running, we can access the Jaeger UI at the address of `http://localhost:16686`. We can add application-level support for it using the following command:

```
dotnet add package OpenTelemetry.Exporter.Jaeger
```

Between these three packages, we are installing ASP.NET Core OTel support and support for exporting our telemetry data to a distributed tracing analysis platform called Jaeger. Now that we have the packages, we can make the following adjustments to our `Program.cs` file:

```
var serviceName = "appointments-service";
var otlp = new Uri("http://localhost:4318"); // use 4317 for gRPC
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService(serviceName,
        serviceVersion: "1.0.0"))
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation(o => o.RecordException = true)
        .AddHttpClientInstrumentation()
        .AddOtlpExporter(o => o.Endpoint = otlp))
    .WithMetrics(m => m
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddRuntimeInstrumentation()
        .AddOtlpExporter(o => o.Endpoint = otlp));
```

With this configuration, we are adding OTel support to our application's startup and then outlining various enrichments that we wish to include with each message that is sent to Jaeger. Note that OTel supports multiple platforms, allowing you to choose the one that best suits your needs.

Now, with this configuration, all traffic to our API endpoints will be logged with as many enrichment data points as we specified. *Figure 16.2* shows the Jaeger interface.

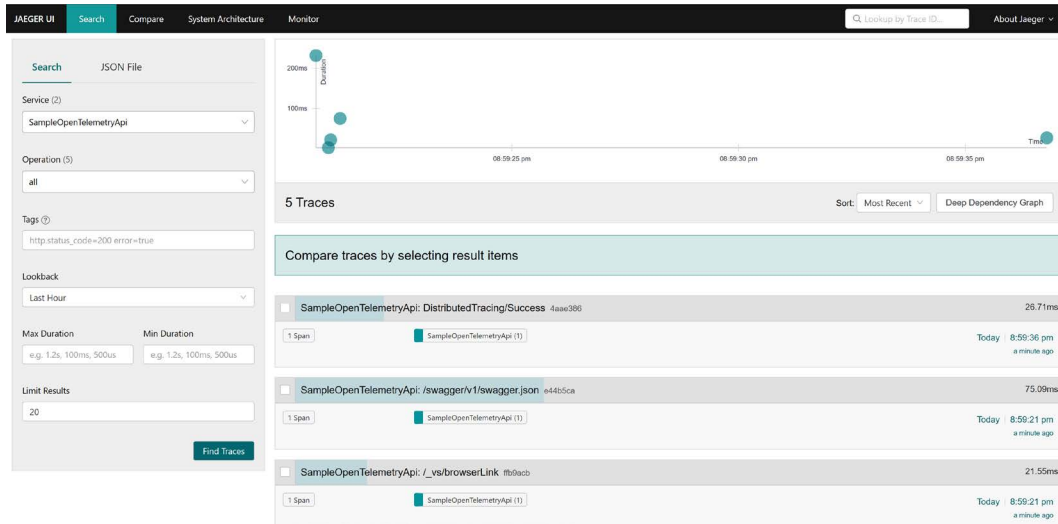


Figure 16.2 – Telemetry data that has been generated and deposited in the Jaeger aggregation platform

Jaeger is simple enough for us to get started. As depicted in *Figure 16.2*, we can view all the services that send telemetry data, filter based on the operations we need to see, and review data across a specified timeline. These are general features of distributed tracing platforms, and once again, we need to ensure that we choose one that gives us the best value for our needs.

To effectively handle and record logs in distributed systems, several resources are essential. Basic requirements include sufficient disk space, network bandwidth, and processing power to manage and transmit log data. The logging system must be resilient enough to handle high-throughput conditions, ensuring that logs are captured without causing significant delays in application processes. In cloud-native or microservices setups, a common strategy for improving and delegating logging tasks is using **sidecars**. A sidecar is a separate process or container running alongside the primary services, focusing solely on logging, monitoring, or proxy tasks. Using sidecars enables teams to centralize and standardize telemetry collection across various services, thereby avoiding disruptions to core application code while enhancing maintainability and scalability as the system expands. We will review this concept next.

## Implementing the sidecar pattern

Microservices promise autonomy, agility, and scalability, but these promises come with a cost: complexity. Each service must handle not only its domain logic but also concerns such as logging, monitoring, configuration, and communication with other services. Embedding all these responsibilities into every microservice leads to bloated code, duplication, and consistency challenges.

The **Sidecar pattern** was created to address cross-cutting concerns by enabling reusability, clear separation of duties, and increased resilience. It involves auxiliary components that operate in a separate process or container, yet run closely with the main service. The name “sidecar” is derived from motorcycles, where a small car is mounted on the side of a bike. Analogously, a sidecar process is connected to the primary service, sharing its life cycle and environment while maintaining distinct responsibilities.

In practice, a sidecar is typically implemented as a container within the same pod as the main microservice (when using Kubernetes) or as a co-located process on the same virtual machine. The service and its sidecar share resources, such as the filesystem or network namespace, allowing for close integration without adding complexity to the main application logic. Some tangible benefits of using this pattern include the following:

- **Code simplification:** Developers focus on business logic while delegating operational complexity to the sidecar
- **Reusability:** One sidecar implementation can serve multiple microservices
- **Improved observability:** Uniform logging and tracing improve debugging across distributed systems
- **Security:** Sidecars can enforce security policies consistently across all services
- **Resilience:** Sidecars can apply retries, rate limiting, and circuit breaking without code changes, aligning with resilience patterns

A service mesh essentially implements the sidecar pattern for scalable networking. Each workload, such as a pod or container, is paired with a lightweight proxy sidecar (such as Envoy) that manages all inbound and outbound traffic. A central control plane configures these proxies. This setup addresses various network concerns, including **mutual TLS (mTLS)**, authentication, authorization, retries, timeouts, circuit breaking, traffic shaping, and detailed telemetry, without requiring any modifications to your service code.

In our healthcare system, the mesh sidecars automatically generate metrics, logs, and distributed traces for each call (**appointments** → **patients** → **notifications**). They also enforce mTLS by default, with automatic key and certificate rotation and verifiable service identities such as SPIFFE. This setup helps us establish zero-trust service-to-service communication and meet HIPAA transmission security standards by encrypting data in transit and supporting precise, auditable policies. While compliance isn’t automatic, the mesh provides essential components: encrypted channels, strong service identities, policy-based access control, and consistent monitoring, all implemented via sidecars instead of app code.

Because observability and monitoring are often among the most difficult challenges in a microservices architecture, as each service runs independently, communicates over the network, and maintains its own state, the sidecar pattern becomes extremely valuable in this context because it introduces a standardized way to capture and manage observability concerns without burdening the application code.

By running a proxy or auxiliary container alongside each service, the sidecar can intercept all inbound and outbound traffic, automatically generating rich telemetry such as request rates, latencies, error codes, and distributed tracing information. This is achieved without modifying the business logic of the service itself, which means developers no longer need to manually add logging statements, correlation identifiers, or retry logic to achieve end-to-end visibility.

For example, in our healthcare management system, the appointments service may call the patients service, the notifications service, and the billing service as part of a single user action. Without a sidecar, developers would have to implement their own tracing code to correlate logs across all these calls, and any missed step could break the trace. With a sidecar such as **Envoy**, all traffic is routed through the proxy, and distributed tracing headers (such as those defined by OTel) are automatically injected and propagated.

For context, Envoy is a high-performance, open source service proxy originally built by Lyft and now part of the CNCF. It is designed for modern cloud-native architectures where service-to-service communication must be fast, secure, resilient, and observable.

Using a sidecar ensures that a complete request journey can be reconstructed across multiple services, giving operators insight into where bottlenecks or failures occur. Similarly, they can automatically collect metrics and export them to systems such as Prometheus or Azure Monitor, creating a uniform telemetry pipeline that is independent of how carefully each microservice was instrumented. It also standardizes log forwarding by capturing both service-specific logs and network-level logs, shipping them to a centralized store such as Elasticsearch or Azure Application Insights.

This uniform approach solves a critical problem of inconsistency. In many teams, microservices evolve at different speeds, often developed by other developers or squads with varied practices for logging and metrics. Without a sidecar, this results in heterogeneous telemetry data that isn't easy to aggregate and analyze. It also supports compliance and auditing needs, especially in sensitive domains such as healthcare, where administrators must not only prove that an operation succeeded but also demonstrate the precise path it took through the system.

Sidecars are especially effective when paired with service meshes, as they benefit centralized policies and orchestration while continuing to provide uniform observability, security, and resilience at the service level. By deploying a sidecar alongside each microservice, organizations gain a consistent mechanism for handling operational concerns across diverse teams and environments.

The cohesion between sidecars and service meshes simplifies the complexity of distributed architectures, making it easier to enforce standards and gather actionable insights, all while freeing developers to focus on business logic rather than infrastructure. Let us take some time to review service meshes next.

## Understanding service meshes

In short, a service mesh is to microservices what air traffic control is to aviation. It ensures that every service call is routed securely, reliably, and observably, without relying on each service to manage its own traffic rules. A service mesh handles service-to-service communication in a microservices architecture, enabling developers to delegate responsibilities such as retries, load balancing, encryption, or observability to a dedicated network layer.

It introduces several core concepts that enhance communication between microservices. Each microservice instance operates alongside a sidecar proxy, such as Envoy, deployed in the same pod or virtual machine. All incoming and outgoing traffic flows through these proxies, enabling consistent policy enforcement and comprehensive telemetry collection. The service mesh architecture is typically divided into a data plane, consisting of the sidecar proxies that handle actual service traffic, and a control plane, which centrally configures these proxies with relevant rules for routing, security, and rate limiting. This division allows for dynamic and uniform service-to-service communication.

The mesh is also equipped with service discovery and intelligent routing capabilities, maintaining awareness of all available services, their locations, and the optimal paths for intercommunication, much like a DNS system tailored to microservices. Additionally, the mesh offers deep observability, since all traffic passing through the proxies is logged, metered, and traced, granting detailed visibility into system latency, error rates, and the journey of each request. Security is also strengthened, as service meshes can enforce mTLS, ensuring all inter-service traffic remains encrypted and accessible only to authorized services.

To illustrate, take the example of our healthcare microservices system, where the appointments service might call a notification service to send reminders. In a traditional setup without a service mesh, the appointments service would be responsible for implementing its own retry logic, SSL/TLS encryption, logging, and metrics collection. With a service mesh, however, the appointments

service can issue the request, while the sidecar proxy handles all operational concerns behind the scenes. The proxy ensures the connection is encrypted using mTLS, automatically retries failed requests with an exponential backoff, and reports metrics and traces to monitoring systems such as Prometheus and Jaeger. Additionally, the service mesh can intelligently route traffic, facilitating advanced deployment strategies such as canary testing by directing requests to new versions of the notifications service.

Popular service mesh implementations include the following:

- **Istio**, which is built on Envoy and renowned for its comprehensive feature set within Kubernetes environments
- **Linkerd**, a lightweight and straightforward option also designed for Kubernetes
- **Consul Connect from HashiCorp**, which integrates seamlessly with Consul's service discovery
- Microsoft's **Open Service Mesh (OSM)**, a lightweight, Envoy-based mesh tailored for Kubernetes

Now, let us review some of the pros and cons of using service meshes and the sidecar pattern.

## Pros and cons of sidecars and service meshes

Implementing the sidecar pattern and service meshes offers significant benefits alongside notable drawbacks, which must be carefully considered in microservices architectures. The sidecar approach separates service discovery, retries, logging, and encryption concerns by running these functions in a companion process alongside the microservice. This simplifies the core code base, allowing developers to focus on domain logic while operational policies become standardized across the platform.

In a complete service mesh, this consistency benefits the entire ecosystem, supporting uniform communication, security, and observability policies. Features such as automatic mTLS for zero-trust networking, traffic routing for canary releases or A/B testing, and detailed request visibility enhance reliability, especially in regulated sectors such as healthcare, where compliance and traceability are critical. However, these advantages include trade-offs that are often underappreciated.

Each sidecar increases CPU and memory usage, which can significantly impact resources in large clusters, effectively doubling the number of containers. This adds complexity to deployment and monitoring since teams manage both services and proxies. Service meshes further complicate operations by introducing a control plane that requires reliable installation, upgrades, and maintenance, adding another layer of infrastructure that can fail or malfunction.

Troubleshooting issues also becomes more difficult, as engineers must identify whether problems originate from the service, the proxy, or the control plane configurations. At that point, we must consider whether this pattern is necessary for our implementation. The pattern introduces latency, as every request passes through at least one proxy, which can become problematic when multiple services are involved. Additionally, for smaller or simpler systems, sidecars and service meshes may be overkill. The extra cognitive load, resource demands, and operational costs could outweigh the benefits, making lightweight solutions, such as resilient HTTP clients or minimal middleware, more appropriate.

Sidecar implementations can be complicated and should not be approached lightly. Thankfully, however, tools such as **Distributed Application Runtime (Dapr)** exist to alleviate some of the attrition that the implementation might attract. Let us review this tool next.

## Sidecars with Dapr

Dapr runs as a lightweight sidecar beside each service in Kubernetes, exposing simple, uniform APIs for cross-cutting concerns such as service-to-service calls, retries, pub/sub, state, secrets, bindings, and telemetry. It is an open, portable runtime that you deploy next to each microservice.

Essentially, your code talks to the Dapr sidecar over HTTP or gRPC using standard endpoints such as `/v1.0/invoke/{app-id}/method/...` for service invocation, `/v1.0/publish/{pubsub}/{topic}` for pub/sub, or `/v1.0/state/{store}` for state. The sidecar handles service discovery, timeouts, retries with backoff, circuit breaking, mTLS between sidecars, distributed tracing headers, and integration with external systems, including Redis, Kafka, Azure Service Bus, Cosmos DB, and Key Vault, through **components** configured with YAML.

In essence, Dapr provides a curated collection of building blocks accessible through a stable API, enabling your services to concentrate on domain logic. Meanwhile, the sidecar handles resilience and connectivity. Since Dapr offers language-neutral HTTP/gRPC APIs, teams can select the optimal technology stack and programming languages for integrations without re-creating cross-cutting code for each platform.

In our healthcare system, the appointment service can prompt patients to confirm their status and publish appointment . created events to Notifications, store idempotency keys in a state store, and export traces and metrics. All these steps can be accomplished through calls to localhost on its own pod. Dapr's sidecar manages complex tasks, including name resolution, secure communication, retries, and telemetry. The operational platform team centrally manages these behaviors via Dapr's component and configuration resources, allowing application teams to maintain clean and testable controllers, handlers, and domain services.



Dapr makes life a little more straightforward, but no abstraction is without its drawbacks. Dapr adds a second container per service that consumes CPU and memory and must be upgraded and observed. Combining Dapr with a service mesh may also result in overlapping capabilities for retries, mTLS, and traffic shaping, and one must decide which layer owns which behavior. You and your team may also prefer library-based approaches (e.g., Polly for retries, MassTransit or NServiceBus for messaging, raw SDKs for cloud services) and explicit code to sidcar indirection, especially when very tight control or extreme performance tuning is required.

Introducing Dapr adds components such as control plane services, CRDs, and sidcar injection, which require operators to be proficient in upgrades and incident management. While Dapr's APIs are stable, specific components vary in maturity. Using less-tested critical parts in your cloud could introduce risks. If your system is small, static, or unlikely to grow beyond a few services, the complexity of deploying a sidcar runtime may outweigh its benefits.

Now, let us review an example of outfitting a microservice with a Dapr sidcar. First, we need to install the Dapr CLI. Instructions can be found here: <https://docs.dapr.io/getting-started/install-dapr-cli/>. Once installed, you can initialize the Dapr services with the `dapr init` command. This initializes runtime containers, which you can view using the `docker ps` command. The containers include the following:

- **Redis:** Gives you a local state and pub/sub without extra setup. You can switch to a preferred message bus later by modifying the Dapr components.
- **Zipkin:** Shows end-to-end traces in a user interface at `http://localhost:9411`.
- **Caching the sidcar image:** Makes the `dapr run` command execute instantly.

Now, let us create a new project and add Dapr support to it. For this demo, we will forego setting up a database and completing data-driven operations:

```
dotnet new webapi -n Appointments.Api.Dapr
cd Appointments.Api
dotnet add package Dapr.AspNetCore
dotnet add package Dapr.Client
```

`Dapr.Client` abstracts the sidcar's HTTP/gRPC APIs so you write expressive C# instead of raw HTTP calls. Now we will modify the `Program.cs` file to implement two endpoints in our minimal API:

```
using Dapr.Client;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDaprClient();
```

```
var app = builder.Build();
app.MapGet("/api/appointments/health", () => Results.Ok(
    new { status = "ok" }));
// Additional endpoints
app.Run();
```

`builder.Services.AddDaprClient();` registers a singleton `DaprClient` in the IoC container. With this in place, we can inject the `DaprClient` service anywhere in code as needed. We also create endpoints for health checks, notifications, and publishing events.

Now, we need to set up our components, which we do by creating the following YAML files in a folder called `components`:

```
# components/statestore.yaml
apiVersion: dapr.io/v1alpha1
kind: Component
metadata: { name: statestore }
spec:
  type: state.redis
  version: v1
  metadata:
    - { name: redisHost, value: "localhost:6379" }
    - { name: redisPassword, value: "" }

# components/pubsub.yaml
apiVersion: dapr.io/v1alpha1
kind: Component
metadata: { name: pubsub }
spec:
  type: pubsub.redis
  version: v1
  metadata:
    - { name: redisHost, value: "localhost:6379" }
    - { name: redisPassword, value: "" }
```

The YAML configuration first defines a state store using Redis as the backend. This setup is necessary as the application refers to the logical name `statestore`, while the actual physical store can be swapped simply by editing the YAML file. The metadata parameters provide the connection details for the driver, allowing you to point to a managed Redis service in production and inject

secrets as necessary. The configuration also sets up a pub/sub broker, again using Redis for development purposes. With this, any service can publish or subscribe to messages by referencing pubsub, gaining event-driven capabilities without being locked into a specific SDK, for enhanced guarantees, such as exactly once delivery or FIFO processing.

Next, we can configure support for distributed tracing with 100% sampling locally. This will enable them to be visible in Zipkin. Add the following `config.yml` file in a folder called `dapr`:

```
apiVersion: dapr.io/v1alpha1
kind: Configuration
metadata: { name: tracing }
spec:
  tracing:
    samplingRate: "1"
    zipkin:
      endpointAddress: "http://localhost:9411/api/v2/spans"
  metric:
    enabled: true
  logLevel: info
```

Dapr automatically distributes **W3C trace context** and generates spans for service calls, pub/sub, and state management. This is a standardized method for passing distributed tracing details across service boundaries, independent of language or platform. It specifies how trace identifiers, such as `traceparent` and `tracestate` HTTP headers, are attached to requests as they traverse your system. This enables effective tracing without requiring modifications to your business code. Additionally, the Zipkin endpoint guides the sidecar on where to send these spans. To test this, we can execute the Dapr sidecar using the following command:

```
dapr run --app-id appointments --app-port APP_PORT --components-path ./
components --config ./dapr/config.yml -- dotnet run
```

This single command sets up a sidecar next to your app with the correct identity, configuration, and components. The explanation for each part is as follows:

- `--app-id appointments`: Identity of your service in Dapr. Other services use this ID to invoke your service.
- `--app-port APP_PORT`: Kestrel's port. Dapr uses it to forward inbound invocation traffic.
- `--dapr-http-port/--dapr-grpc-port`: The sidecar's ports. Useful for tooling and direct API calls.

- `--resource-path`: Where the sidecar loads **component** definitions (state/pubsub).
- `--config`: The configuration resource enabling tracing/metrics.
- `-- dotnet run`: launches the API project launches your API.

Replace `APP_PORT` with the port number from the `launchsettings.json` file in your project. Once the command has been executed successfully, you can run a simple test on the health check endpoint by sending a GET request to the following URL: `http://localhost:3500/v1.0/invoke/appointments/method/api/appointments/health`.

Dapr will handle routing and route your request to the appropriate app port and running project. You can open Zipkin at `http://localhost:9411` and run a query to view spans that reflect the request.

For richer app-level spans, you can instrument with .NET OTel. This can be done by adding the following packages to the project:

```
dotnet add package OpenTelemetry.Extensions.Hosting
dotnet add package OpenTelemetry.Instrumentation.AspNetCore
dotnet add package OpenTelemetry.Instrumentation.Http
dotnet add package OpenTelemetry.Exporter.OpenTelemetryProtocol
```

Then add the following code to `Program.cs` before the `var app = builder.Build()` line:

```
var serviceName = "appointments-service";
var endpoint = "http://localhost:4318";
builder.Services.AddOpenTelemetry()
    .ConfigureResource(rb => rb
        .AddService(serviceName: serviceName, serviceVersion: "1.0.0")
        .AddAttributes(
            [
                new KeyValuePair<string, object>("deployment.environment",
                    builder.Environment.EnvironmentName)
            ]
        ))
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation(o => o.RecordException = true)
        .AddHttpClientInstrumentation()
        .AddSource("Appointments.Domain")
        .AddOtlpExporter(o => o.Endpoint = new Uri(endpoint)))
    .WithMetrics(m => m
        .AddAspNetCoreInstrumentation()
```

```
.AddHttpClientInstrumentation()  
.AddOtlpExporter(o => o.Endpoint = new Uri(endpoint)));
```

Since all Dapr components go through the sidecar, they automatically create traces and spans for service calls. This supports, though does not replace, application-level telemetry, such as domain spans or custom business metrics, which can be emitted with .NET's OTel integration.

Beyond monitoring and observability, running sidecars in production changes the operational surface area of every service, so you need platform-grade guardrails. Start with security: enforce zero-trust by keeping mTLS on between sidecars, and rotate certificates automatically. Treat every component as infrastructure as code, backed by a proper secret store (Kubernetes Secrets + CSI or Azure Key Vault) and clear rotation procedures.

In governance, production systems require strict cost control and compliance measures. Establish environment-specific sampling and retention policies for traces and logs early on, and centralize them into a single pipeline. This enables precise visibility and cost management of the retained data. Avoid including PHI/PII in spans and logs; instead, use surrogate keys and perform enrichment within secure tools.

With these governance measures in place, you can ensure the consistency and portability of the sidecar model without it becoming a hidden risk. While it's true that adding sidecars increases complexity in our microservices architecture, we've long relied on third-party solutions for monitoring and observing all application components. Microsoft has responded to this need by developing Aspire, a cloud-ready framework designed to help us orchestrate and monitor various parts of an application. We will explore this further next.

## Using .NET Aspire

.NET Aspire is a development-time orchestration and observability layer for .NET-based distributed applications. Instead of juggling multiple terminals, local containers, ad hoc scripts, and hand-wired telemetry, Aspire provides a single app model to describe your microservices, local infrastructure (such as Redis, PostgreSQL, and RabbitMQ), default health/telemetry wiring, and a built-in dashboard to monitor the entire system in real time.

For our healthcare solution and our appointments, patients, notifications, and billing services, Aspire lets us run the application one time using **AppHost**, and bring up the entire graph with sane defaults for logging, metrics, and traces, while remaining agnostic about production de-

ployment (AKS, Azure Container Apps, VMs, on-premises). It is akin to a *developer cockpit* that complements the various patterns discussed throughout this book. Aspire ships three big ideas that matter day to day:

- **Distributed application model (AppHost):** A compact .NET project that defines your entire application, including projects, container dependencies, environment variables, connection strings, health checks, and startup procedures. It serves as your single entry point for local execution.
- **Service defaults:** A lightweight, shared library (often called `ServiceDefaults`) that ensures consistent logging, OTel, health checks, resilience policies, and configuration across all services, preventing discrepancies in telemetry or retry strategies.
- **Dashboard:** A local user interface displaying services, dependencies, logs, health status, and trace timelines. It allows you to explore these elements easily during development, making the “logs, metrics, traces” story more concrete and accessible beyond just the production environment.

Aspire doesn’t replace containers or Kubernetes. Instead, it eliminates the manual tweaks of local orchestration and offers a ready-to-use, observability-focused developer experience that aligns with your earlier guidance on logging, metrics, and tracing.

Now that we have an overview of what Aspire brings to the table, let us look at writing up a microservice using it.

## Using Aspire for microservices

This step-by-step guide walks you through setting up a minimal healthcare application with two core services: **PatientService** and **AppointmentService**. You’ll see end-to-end telemetry lighting up in Aspire’s dashboard as requests flow across services.

Let us begin by installing the necessary Aspire project templates using the following command:

```
dotnet new install Aspire.ProjectTemplates
```

Then we can create a new Aspire project using the empty Aspire template:

```
dotnet new aspire -n HealthcareAspire
```

It contains two projects, AppHost and ServiceDefaults. It is intentionally minimal, allowing you to add your own services:

- AppHost is a small executable project that describes your distributed application for local development: which services exist (e.g., PatientService or AppointmentService), what infrastructure they depend on (PostgreSQL or Redis), and how those parts depend on each other. When you run AppHost, Aspire builds the topology, starts the services and containers, and opens the Aspire dashboard, allowing you to view resources, logs, traces, and metrics in one place.
- ServiceDefaults is a tiny shared class library that centralizes the cross-cutting “good stuff” every service should have: OTel (logs, traces, and metrics), health checks, service discovery, and sensible resilience defaults (e.g., Polly policies for retries/timeouts). The project exposes extension methods (conventionally AddServiceDefaults() and MapDefaultEndpoints()) that you call in each microservice’s Program.cs. This keeps your setup DRY and consistent.

Now we can add two microservices: the appointment and patient APIs. For demo purposes, these APIs will be minimal, with limited functionality. We can use the following commands to create the new projects:

```
dotnet new web -n HealthcareAspire.AppointmentsApi
dotnet sln add HealthcareAspire.PatientsApi/HealthcareAspire.
AppointmentsApi.csproj
dotnet new web -n HealthcareAspire.PatientsApi
dotnet sln add HealthcareAspire.PatientsApi/HealthcareAspire.PatientsApi.
csproj
```

We can add a sample endpoint to the appointments service’s Program.cs file:

```
var builder = WebApplication.CreateBuilder(args);
// Aspire service defaults: telemetry, health checks, discovery,
// resilience
builder.AddServiceDefaults();

//shortened for brevity

var app = builder.Build();
app.MapGet("/appointments", () => Results.Ok(new[] { new { id = 1, date =
    DateTime.UtcNow, patientId = 123 } })).WithName("GetAppointments");
```

```
// Health endpoints (/health, /alive) in Development
app.MapDefaultEndpoints();
app.Run();
```

Similarly, we will add an endpoint for the patient service:

```
var builder = WebApplication.CreateBuilder(args);
// Aspire service defaults: telemetry, health checks, discovery,
// resilience
builder.AddServiceDefaults();

// Shortened for brevity

var app = builder.Build();
app.MapGet("/patients", () => Results.Ok(new[] { new { id = 123,
    name = "Jane Doe" } })).WithName("GetPatients");

// Health endpoints (/health, /alive) in Development
app.MapDefaultEndpoints();
app.Run();
```

`app.MapDefaultEndpoints()` maps `/health` (readiness) and `/alive` (liveness). These routes are mapped by default in development; you can enable them in other environments and protect them as needed. Readiness should check dependencies (e.g., database), while liveness is a lightweight “process is up” check.

These endpoints allow Aspire (and, later, your orchestrator/load balancer) to determine whether the service is ready to handle traffic or just alive and should keep running. In Aspire templates, those endpoints are enabled by default in development, and they work with `builder.AddServiceDefaults()`, which registers the health checks and OTel infrastructure.

On the other hand, we can register the services in `ServiceDefaults` and add the new projects to `AppHost` so that Aspire can orchestrate them using these commands:

```
dotnet add HealthcareAspire.AppointmentsApi reference HealthcareAspire.
ServiceDefaults
dotnet add HealthcareAspire.PatientsApi reference HealthcareAspire.
ServiceDefaults
dotnet add HealthcareAspire.AppHost reference HealthcareAspire.
AppointmentsApi HealthcareAspire.PatientsApi
```



Finally, let us modify the `AppHost.cs` file to reference the microservices:

```
var builder = DistributedApplication.CreateBuilder(args);
var patients = builder.AddProject
    <Projects.HealthcareAspire_PatientsApi>("patientsapi");
var appointments = builder.AddProject
    <Projects.HealthcareAspire_AppointmentsApi>("appointmentsapi");
builder.Build().Run();
```

With all these adjustments made, `AppHost` will now orchestrate the launch of our microservices when the project is run. When it is launched, the following happens:

- The `AppHost` process sets up a handful of Aspire-specific environment variables (for example, the dashboard OTLP endpoint and browser/auth tokens) so the rest of the system can talk to the dashboard securely.
- `Program.cs` calls `DistributedApplication.CreateBuilder(args)` and describes resources (databases, caches, and message brokers) and projects (your APIs and workers).
- For each hosting integration you added (e.g., `AddPostgres("postgres").AddDatabase("healthcaredb"), AddRedis("cache")`), `AppHost` ensures the container image is available, creates the container, and exposes a network endpoint.
- `AppHost` builds and starts that target project as its own process, wiring in environment variables and configuration. The strongly typed `Projects.*` classes exist specifically so `AppHost` can reference your projects safely.
- `AppHost` starts the **Aspire dashboard** and opens it in your browser. The dashboard authenticates using the tokens that `AppHost` created, as it displays sensitive information (environment variables, connection strings, etc.).
- Because the microservices call `AddServiceDefaults()`, OTel is already configured. Logs, traces, and metrics from ASP.NET Core, HttpClient, EF Core, and any custom activity or meter you add stream to the dashboard automatically.
- `app.MapDefaultEndpoints()`, publishes liveness/readiness endpoints (e.g., `/alive`, `/health`). The dashboard uses these to show the current status; later, Kubernetes (or Azure Container Apps) can probe the same endpoints.

We can launch our application by executing the `dotnet run` command in the `AppHost` project folder. This launches the Aspire dashboard, as shown in *Figure 16.3*, which displays all running services, their statuses, and endpoint addresses, along with console output of all activities, structured logs, metrics, and traces.

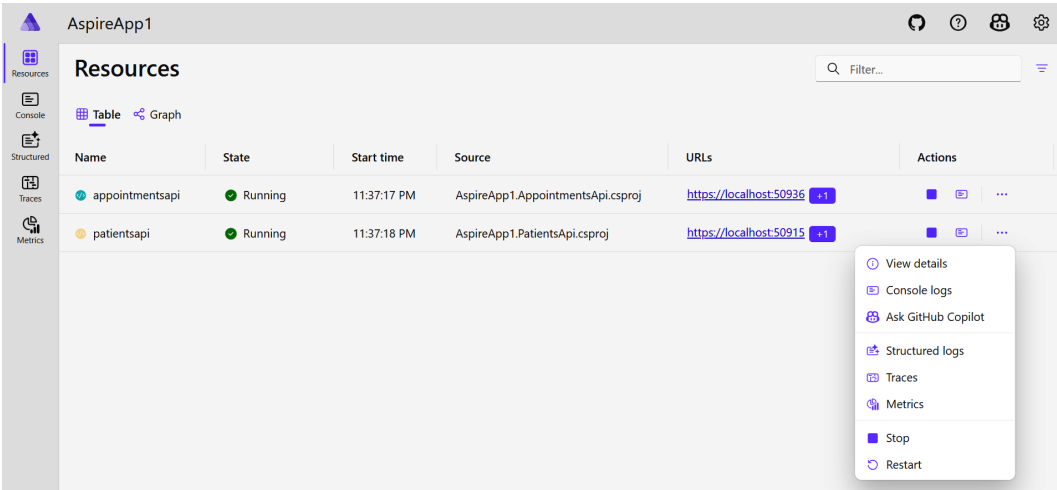


Figure 16.3 – The Aspire dashboard to monitor and manage all Aspire registered services

Figure 16.4 shows the structured log output. These logs are generated automatically during the service’s runtime.

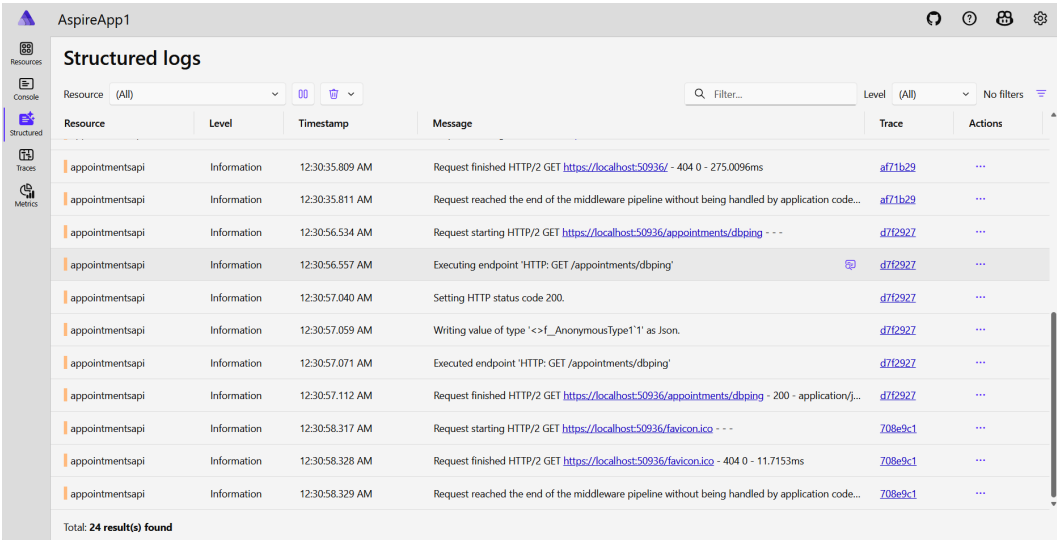


Figure 16.4 – The structured logs tab shows log output from all services that are being or-  
chestrated

Now, if we were to add, for example, PostgreSQL databases to our microservices, we could benefit from the monitoring and orchestration capabilities that Aspire affords us. We can add PostgreSQL to the AppHost project by adding the `Aspire.Hosting.Postgres` package:

```
dotnet add package Aspire.Hosting.Postgres
```

Now that PostgreSQL support has been added, we can modify the `AppHost.cs` file to have the following content:

```
// PostgreSQL server
var postgres = builder.AddPostgres("pg");

// Databases for each service
var patientsDb = postgres.AddDatabase("patientsdb");
var appointmentsDb = postgres.AddDatabase("appointmentsdb");

// Projects
var patients = builder.AddProject<Projects.AspireApp1_PatientsApi>
    ("patientsapi").WithReference(patientsDb);
var appointments = builder.AddProject<Projects.AspireApp1_AppointmentsApi>
    ("appointmentsapi").WithReference(appointmentsDb);
```

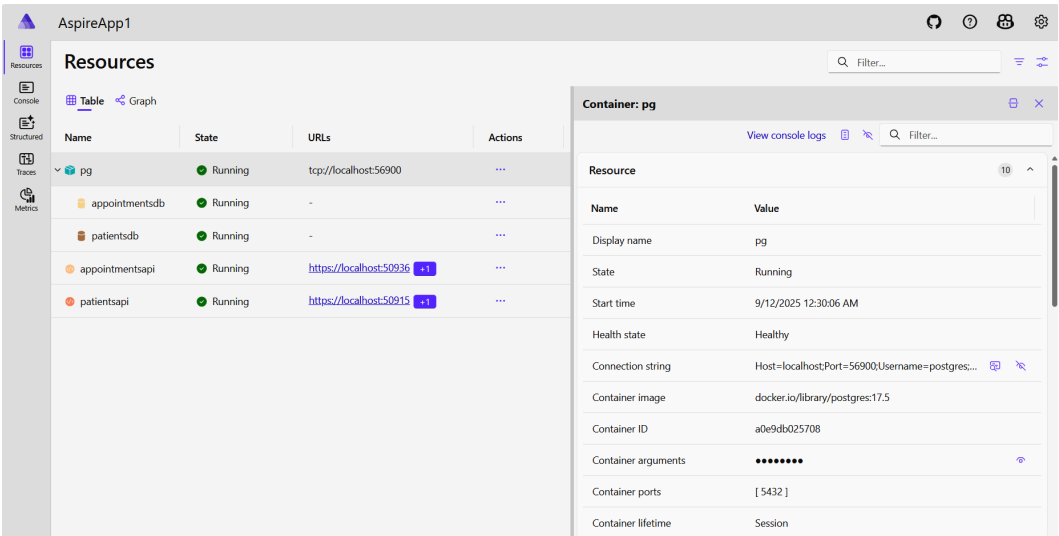
The first line declares a PostgreSQL server resource named `pg` in the Aspire application model. When the app runs, a local Postgres container (image + port binding) will be launched. Additional parameters can be added through additional methods such as `WithDataVolume()`, `WithImageTag()`, and `WithPassword()`, if we need persistence, specific versions, or password login protection.

We then go on to assign two databases hosted on the same `pg` server to the two microservices. This ensures that those databases exist on startup and builds distinct connection strings for each (same host/port/credentials, but different database names). This gives each service its own schema boundary and connection string. This supports loose coupling and the “database-per-service” pattern, while still utilizing a single Postgres instance for convenience in development. The microservice will use the resource name, `pg`, as the hostname when forming the connection string.

We modify the microservice orchestration code to add service dependencies to the databases. This means that Aspire will register the startup order for the services. When our microservices launch, they reference their respective databases, and their containers get a `ConnectionStrings:DB_NAME` entry (e.g., `ConnectionStrings:patientsdb`) along with related environment variables that

point to the associated database. AppHost automatically resolves the databases, which is helpful for startup ordering and the dashboard’s resource graph.

Finally, before you run with these new dependencies, ensure that your container-hosting platform (e.g., Docker and Podman) is installed and running. *Figure 16.5* shows an updated Aspire dashboard with both microservices and associated databases running.



*Figure 16.5 – Aspire dashboard showing all running services that AppHost orchestrates*

.NET Aspire makes the early stages of a distributed system more predictable and manageable. By using a code-first app model (AppHost) and shared ServiceDefaults, you can achieve service discovery, health checks, and OTel tracing, metrics, and logs without needing to set up any additional plumbing. Additionally, you gain a dashboard that provides a comprehensive view of the entire system, which is particularly beneficial for our distributed application workflow. When you’re ready to scale, the exact composition can be published to your chosen platform without changing your service code.

Going beyond the basics, Aspire’s integrations cover common data and infrastructure, while service discovery supports logical names, composite schemes (e.g., https+http://), and even DNS-based providers for hosted environments. You can customize telemetry with your own ActivitySource and Meter signals, adjust sampling according to your environment, and utilize the dashboard’s structured logs, traces, and metrics to identify and resolve cross-service issues in minutes instead of hours.

Aspire is built for development-time orchestration. Use it to shape, test, and observe your microservices locally, then deploy to an app platform or Kubernetes for production. With these components in place, Aspire gives your microservices the same day-one ergonomics we just demonstrated, discoverable, health-checked, and observable by default so that teams can focus on domain behavior instead of setup.

Now that we have explored several avenues for logging, monitoring, and observing our microservices applications, let us wrap up this chapter.

## Summary

This chapter made the case that healthy microservices are unmanageable without observability, and then showed, with practical .NET examples, how to make observability routine rather than reactive. It opened by distinguishing between monitoring and observability: monitoring tells you that something is wrong, while observability provides you with sufficient high-quality signals to infer why. Logs, metrics, and traces are the three complementary signals that turn distributed behavior into explainable, debuggable narratives. We established a flow where logs record salient events, metrics reveal systemic health and trendlines, and traces stitch the cross-service journey into a single timeline that highlights where time and errors accumulate.

We started by exploring logging, as it's the signal every team already has and the easiest to misuse. It's crucial to decide what to log, how to avoid chatty noise and sensitive data, and how to use log levels consistently so severity means the same thing across services. Built-in .NET logging abstractions can be extended using libraries such as Serilog to add structured events and multiple "sinks" without abandoning the `Microsoft.Extensions` pipeline. Serilog can ship logs to various destinations, such as Seq, which serves as an aggregator for structured events from multiple services into a single, searchable user interface, immediately improving the feedback loop while keeping code changes small and repeatable.

We then pivoted to distributed tracing, the signal that answers the question "where did the time go?" across service boundaries. We began by examining OTel, a standard and portable method for emitting traces and metrics. OTel contains additional details and requires more specialized tools, such as Jaeger, to visualize traces. We also distinguished between Jaeger and Prometheus, which handles time-series metrics. They complement each other rather than compete, and both can be used alongside the OTel Collector as your estate grows. The net effect is to make cross-service pathologies visible in minutes, not hours, and to replace guesswork with timelines.

Because not every cross-cutting concern belongs in application code, we looked into the sidecar pattern and service meshes as operational building blocks that standardize traffic management, telemetry, and security. The sidecar narrative explains why teams push retries, circuit breaking, mTLS, and log/trace export into a companion process, typically Envoy, so that business code remains lean while maintaining uniform telemetry. The service mesh discussion builds on this, describing the data plane (proxies) and control plane (policy and configuration), the benefits (zero-trust, advanced routing such as canaries, and pervasive traces), and the trade-offs (extra containers, CPU/memory overhead, operational complexity, and new failure modes).

When discussing sidecars, we introduced Dapr as a focused, sidecar-based runtime that exposes stable HTTP/gRPC building blocks, service invocation, pub/sub, state, and secrets, so teams can adopt common patterns without importing a full mesh. We explored how a service communicates with its local sidecar, how components are defined using YAML and swapped per environment, and how Dapr propagates W3C trace context to maintain end-to-end visibility. We weighed the pros and cons: another container per service, a control plane to manage, and overlapping capabilities with meshes or library-based approaches.

With .NET Aspire, we bring observability back into the developer workflow. This development-time orchestrator lets you define the entire application graph, including services, databases, and caches, and run it with a single command. As a result, you get logs, traces, metrics, health, and service discovery at no extra cost. AppHost serves as the code-first wiring diagram, starting containers, launching services, and injecting connection strings. It also opens the Aspire dashboard. Meanwhile, ServiceDefaults is a one-liner that enables OTel, health endpoints, resilient HttpClient defaults, and discovery for each service.

It's essential to note that Aspire doesn't replace production platforms, and the chapter explicitly clarified that "run" and "publish" are distinct concepts. However, Aspire eliminates the need for multi-terminal, ad hoc orchestration and makes observability the default, not an afterthought reserved for the end of a sprint.

Overall, the chapter provided a unified mental model and a practical toolkit for working with microservices in real-world scenarios. You learned how to make logs valuable and secure, how to centralize and query them, how to track a single request across services using traces, and how to reveal system health through metrics. You also explored how operational patterns, such as sidecars, meshes, and Dapr, can improve or hinder your development and observability efforts, and wrapped up with a developer-friendly approach, .NET Aspire, that combines orchestration and telemetry, enabling your team to iterate quickly and confidently. The underlying theme is

straightforward yet powerful: observability is not an afterthought, but rather a core architectural capability that, when embraced early on, transforms complexity into clarity, allowing the healthcare domain logic to take center stage. We are now transitioning to the final chapter of the book where we will review the major talking points and wrap up the concepts we have explored.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](https://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW



# 17

## Wrapping It All Up

As we arrive at the culmination of our journey through microservices architecture with .NET, it's time to pause and take pride in the depth and practicality of the knowledge you've gained. What began as a set of design principles has evolved into a complete architectural mindset, one that equips you to build systems that are not only modular and scalable but also intelligent, self-healing, and ready for the realities of modern software delivery.

Microservices are far more than simply dividing a monolith into smaller services. They represent a disciplined approach to crafting distributed systems that mirror the structure and agility of real-world business domains. Each service becomes an autonomous unit of value, capable of evolving independently while seamlessly collaborating with others through well-defined contracts and resilient communication patterns.

Throughout this book, you have mastered scoping and designing these services with precision, ensuring that every boundary serves a business purpose. You have learned how to orchestrate communication between them, both synchronously and asynchronously, with confidence in their reliability, and have explored proven strategies for maintaining data consistency, implementing resilience, and designing for failure so that your systems recover gracefully instead of collapsing under pressure.

You have also acquired the skills to build observability into every layer, turning complex, distributed architectures into transparent, diagnosable, and optimizable ecosystems. And as we ventured into the world of modern deployment, you discovered how to take full advantage of Kubernetes for container orchestration, serverless for event-driven scalability, and .NET Aspire for streamlined orchestration and observability in development environments.



By this point, you are no longer just familiar with microservices design; you are ready to build and deploy them in production. You have architectural insight, the implementation strategies, and the tooling expertise to turn concepts into concrete, operational systems. You can walk into any development discussion, any code review, or any cloud architecture meeting with confidence, knowing that you understand not only how to build microservices but also how to make them thrive in real-world environments.

This is the power of what you’ve learned. You now have the clarity to design distributed systems that align perfectly with business goals, the discipline to apply the right patterns at the right time, and the confidence to deliver scalable, resilient, and observable solutions that perform under pressure. What lies ahead is no longer uncertain; it is yours to shape.

This chapter consolidates the main lessons from these topics. It doesn’t aim to match the technical detail of earlier chapters but instead distills them into key principles you can apply to your projects. We will cover strategies for microservice development, revisit the challenges of data management across services, emphasize the importance of monitoring and resilience, and explore cloud deployment options. The chapter concludes with a discussion on pragmatism, exploring when microservices are appropriate and how to evolve them gradually without introducing undue complexity.

Let us begin by reviewing some of the principles behind developing microservices.

## Developing microservices

Our journey started with a clear understanding of what microservices truly are and why they matter. Early in the book, we explored how **domain-driven design (DDD)** provides the foundation for effective service boundaries. By identifying bounded contexts, we learned to shape systems around real business capabilities instead of arbitrary technical divisions. This approach transformed our healthcare management system from a collection of features into a well-coordinated ecosystem of independent, purpose-driven services.

Through examples such as the Patient Service, Appointment Service, and Notification Service, we observed how each microservice could fully own its domain—handling its own data, logic, and deployment life cycle. This separation was not just a technical task; it was a mindset change. Each service became a reflection of a business process, allowing for agility, easier maintenance, and a quicker path from idea to production.

By mastering this initial step, you established the foundation of successful microservice design: the ability to align technology choices directly with business goals. With that clarity in place, every architectural decision that followed—whether about communication patterns, data consistency, or deployment—was built on a solid, intentional foundation.

The key trait of a microservice is its independence. It should be deployable on its own and capable of evolving without requiring unrelated services to change. However, independence does not mean isolation. Services inevitably need to work together, which leads us to the communication patterns discussed in *Chapters 3 and 4*.

Synchronous communication via REST or gRPC offers immediacy. For example, the Appointment Service requests available doctors by calling the Doctors API and receives real-time data. Although this method is effective, it can create tight coupling and cause cascading failures. As a result, we also considered asynchronous patterns using RabbitMQ or Azure Service Bus. By publishing events such as AppointmentBooked, services such as Notifications or Billing can respond independently, without delaying the booking process.

From a development standpoint, microservices demand robust coding practices. In *Anchor 7, Building Resilient Microservices*, we discussed using Polly to add retries and circuit breakers. Without these measures, a temporary issue in the Patient Service could stop new appointments from being scheduled. Implementing retries and circuit breakers allows the system to handle interruptions and recover smoothly.

The key takeaway is straightforward: developing microservices involves striking a balance between independence and teamwork. We should create cohesive services, enable graceful failure, and choose communication methods that meet business needs.

Microservices are a distributed system with many moving parts and additional concerns. One of the significant concerns we addressed was how to handle data. We can review these concepts next.

## Managing data in distributed applications

One of the most challenging aspects of microservices is managing data. Unlike a monolithic system, where data typically resides in a single relational database, microservices adopt a decentralized approach to data ownership. Each service maintains its own schema and does not permit direct queries from other services.

In our healthcare management system, each microservice was designed to own its data and select the storage technology best suited to its domain. For example, the Patient Service uses PostgreSQL to store structured demographic and clinical data that benefits from relational integrity and

transactional support. Conversely, the Notification Service employs a NoSQL database, optimized for storing message templates, delivery logs, and event metadata that require flexible schemas and high write throughput. This intentional use of polyglot persistence ensures that each service operates with a data model and technology stack tailored to its specific needs, without mixing message persistence with database storage.

It's important to remember that dedicated message queues are not the same as databases; they are managed by brokers such as RabbitMQ or Azure Service Bus, or by managed queue services such as Azure Queue Storage. These components handle temporary communication between services, allowing for asynchronous event delivery and decoupled workflows. They work alongside, rather than replace, the service's own data store. Mixing the messaging layer with the persistence layer can blur responsibilities, potentially decreasing maintainability and reliability over time.

However, decentralization introduces its own challenges. Eventual consistency is one such concern—when a patient's phone number is updated, for instance, the change must also propagate to the Notification Service. To address this, we implemented the Outbox pattern, ensuring that domain events are published reliably alongside transactional database updates. Similarly, distributed transactions, such as those during appointment scheduling, cannot rely on traditional ACID guarantees across services. Instead, we applied the Saga pattern (see *Anchor 4*) to coordinate these workflows and execute compensating actions when needed.

We also examined event sourcing and CQRS (*Anchor 4*) as advanced architectural techniques. Event sourcing creates an immutable sequence of domain events that serves as a detailed audit trail, an essential feature in healthcare systems where traceability and compliance are critical. CQRS separates read and write operations, enabling each to scale independently and enhancing overall responsiveness under load.

Ultimately, data management in microservices is about balancing autonomy and consistency with performance and complexity. While there's no one-size-fits-all solution, the patterns and techniques introduced throughout this book equip you with a strong toolkit to craft data strategies that meet both technical and business goals confidently.

Although data management is complex, monitoring and maintaining resilience introduce even broader concerns. Let's examine this further.

## Implementing monitoring and service resilience

Even with thorough planning, real-world systems can face unforeseen disruptions. Networks may lag, APIs might randomly timeout, or external providers might experience brief outages. These instances do not indicate flaws in your architecture; instead, they offer chances to showcase its robustness. As outlined in *Anchor 6, Building Resilient Microservices*, resilience is essential and ensures that your services can adapt, recover, and maintain steady operation amid challenges.

Methods such as timeouts, retries, circuit breakers, bulkheads, and caching enable services to recover from temporary issues. For instance, if the Insurance API goes down during patient onboarding, our system can either retry the request after a period or place it in a durable queue for future processing.

Resilience cannot be fully achieved without observability. In *Anchor 4, Observability and Monitoring with Modern Tools*, we defined observability as the capacity to comprehend why a system behaves in a certain way. The three main components of observability are logs, metrics, and traces. Together, they transform a distributed system from a mysterious black box into a clear, diagnosable environment.

We also emphasized cross-cutting concerns, such as centralized logging and health checks. Using tools such as OpenTelemetry and Jaeger, we traced a patient's appointment request across various services. Metrics in Prometheus revealed latency spikes, and logs identified the service responsible for the delay.

Furthermore, modern frameworks such as .NET Aspire (*Anchor 3*) offer integrated orchestration and telemetry features, simplifying local development and service discovery. Aspire also eases dependency management by providing a cohesive developer experience.

The core idea is to design services that can fail gracefully and to monitor them as if they are already experiencing issues. This approach prepares your system for real-world scenarios.

## Cloud-based development patterns

Microservices thrive in the cloud environment, benefiting from its elasticity, global accessibility, and managed services, which complement their decentralized architecture. In *Chapters 14, 15, and 16*, we explored various hosting strategies, including containerized applications with Kubernetes, Azure Functions, and Aspire orchestration.

Containers provide portability and consistency. Packaging our Appointment Service into a Docker image guarantees that the same artifact runs seamlessly across development, staging, and production environments without configuration drift. Kubernetes enhances this setup by orchestrating these containers: it scales services based on demand, performs rolling updates, and automatically recovers failed Pods.

Yet Kubernetes introduces its own complexity. That is why serverless is so compelling for specific workloads. In *Anchor 4, Serverless Microservices Development*, we saw how Azure Functions enabled us to build an event-driven appointment orchestrator without requiring infrastructure provisioning. Functions scale automatically, and we only pay when they execute. Durable Functions even allowed us to model long-running workflows such as booking an appointment and sending notifications.

The message is not that one model is superior, but that modern microservices architectures often combine these approaches. Containers may host core services, Functions may handle lightweight triggers, and Aspire may streamline development workflows.

Now, let us conclude this book.

## Summary

In this book, we've explored the world of microservices, covering topics such as defining services with DDD, managing communication, addressing distributed data challenges, and ensuring resilience and observability. We also discuss securing services and deploying them using containers, Kubernetes, serverless, and the Aspire platform.

The main lesson is to be pragmatic. While microservices offer agility, scalability, and resilience, they also introduce complexity, operational challenges, and new failure risks. Not all systems require microservices, and not every microservice needs every advanced pattern. Begin with a simple approach, apply patterns only when they address actual issues, and improve gradually.

This system demonstrates a practical implementation of the patterns and practices we've examined, rather than an idealized concept. As you finish reading this book, keep in mind that the objective measure of success for microservice architecture is not its design elegance but its capacity to consistently provide business value and adapt over time. By applying these lessons with discipline and practicality, you'll be prepared to create systems that address current challenges and future opportunities.

## Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to [packtpub.com/unlock](http://packtpub.com/unlock)). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

UNLOCK NOW





# Index

## A

**access tokens** 350

security, implementing 350-353

**ACID principles** 162, 194

**ACID transactions** 91, 190

**acknowledgment patterns** 253

**action methods** 65

**ActiveMQ** 88, 96

**ADO.NET** 155, 156

**aggregate pattern** 41

relationships 46

relationships, handling 46

using 41

**aggregate root** 32, 34, 41-46

**Aggregator pattern** 114, 115, 119

aggregator service, creating 123-127

asynchronous processing 122

data transformation benefits 121

failure management benefits 121

horizontal scaling 122

implementing 119-122

optimization benefits 121

performance optimization and best practices 132

security benefits 120

sharding and partitioning 122

UI, connecting to aggregator service 128-131

**Aggregator pattern, best practices**

asynchronous processing 132

caching 132, 133

compression, using 134, 135

error handling 136

logging and monitoring 136, 137

**Ahead-of-Time (AOT) compilation** 317, 382

implementing 324, 325

**Alpine** 387

**Amazon API Gateway** 415

**Amazon Aurora Serverless** 416

**Amazon CloudWatch** 416

**Amazon DynamoDB** 198, 416

**Amazon EventBridge** 416

**Amazon S3** 416

**Amazon Simple Notification Service (SNS)** 416

**Amazon Simple Queue Service (SQS)** 88, 96

**Amazon Web Services (AWS)** 232

**Anchore** 384

**Angular** 309, 316

**Apache HTTP Server** 281

**Apache Kafka** 95



- API composition** 189
- API gateway** 274, 275
  - advantages 276-278
  - disadvantages 278
  - implementing 279, 280
  - implementing, with Ocelot 285-291
  - implementing, with reverse proxy 280, 281
  - implementing, with YARP 281-285
  - rate limiting 295
  - rate limiting, implementing 300-303
  - security 295
  - security, implementing 296-299
  - selection considerations 279
- API key authentication** 344
- API security** 341-343
  - authentication 341
  - authorization 341
  - data confidentiality and integrity 342
- AppArmor** 384
- AppHost** 474
- Application Insights** 417
- Application Programming Interfaces (APIs)** 56
  - securing, with IdentityServer 366-369
- Appointment aggregate** 43
  - example implementation 43
- AppointmentService** 475
- Aspire dashboard** 478
- ASP.NET project** 6
- asynchronous API failures**
  - handling 253, 254
- asynchronous communication** 52, 86, 87
  - consumer, implementing 101-104
  - fault-handling policies, implementing 265
  - implementing 95
  - message bus, selecting 95, 96
  - publisher, implementing 99, 100
  - RabbitMQ, installing 97, 98
- asynchronous service communications** 214
- asynchronous services**
  - eventual consistency 91, 92
  - message bus systems 89, 90
  - message queues 88, 89
  - pub-sub pattern 93, 94
  - strategies 87, 88
- authentication** 341
  - errors, reducing 262-265
- authentication system** 296
- authorization** 341
- authorization system** 296
- automated vigilance** 426
- AWS** 14
- AWS Lambda** 15, 415
- AWS Step Functions** 232, 416
- Azure** 14
- Azure AD logs** 428
- Azure API Management (APIM)** 417
- Azure Application Insights** 461
- Azure Blob Storage** 417
- Azure Container Registry (ACR)** 381, 405
- Azure Cosmos DB** 417
- Azure Durable Functions** 232, 417, 424
- Azure Event Grid** 417
- Azure Functions** 15, 417
- Azure Key Vault** 418
- Azure managed identities** 428
- Azure Monitor** 417
- Azure Queue Storage** 88
- Azure Service Bus** 96, 417

Azure SQL Database (Serverless tier) 418

Azure Storage Queues 417

## B

Backend as a Service (BaaS) 414

Backend for Frontend (BFF)  
    pattern 113, 273, 291

    challenges 291, 292

    implementing, with Ocelot 292-295

backoff concept 256

Base64 representation 347

base image 382

Basic Authentication 344

bearer token 346-349

    key-value pair 349

    limitations 353, 354

BigQuery 418

Blazor WebAssembly (Blazor WASM)  
    application 116

    UI, developing without aggregator 116-119

Blazor WebAssembly micro frontend  
    creating 317-321

Blazor WebAssembly (WASM) 314-316  
    versus JavaScript frameworks 316

Bootstrap 309

Bootstrap CSS framework 318

business logic layer 5

## C

C# 19

caching layer

    adding 257-262

Cadence 231

California Consumer Privacy Act (CCPA) 448

Camunda 232

CAP theorem 92

cardinality 196

Cascading Style Sheets (CSS) 309

Cassandra 198

central entity 32

centralized identity management 356

challenges and solutions, asynchronous  
    communication

    additional considerations 109, 110

    duplicate messages 107-109

    message ordering 105-107

choreography saga pattern 221

    building, with MassTransit and  
        RabbitMQ 226-229

    cons 230

    implementing 221-223

    pros 230

    rollback on failure 223-225

Circuit Breaker pattern 53

circuit breaker policy

    adding 256, 257

circuit breakers 252

client application, and API

    login flow 347

closed state 256

cloud-based development patterns 489, 490

Cloudflare Workers 418

Cloud Native Computing Foundation  
    (CNCF) 403, 462

Cloud Run 418

code-first approach

    advantages 205

cold start latency 421

command-line interface (CLI) 380

**command messages** 91

**Command Query Responsibility Segregation (CQRS) pattern** 139-142, 213

benefits 142, 143

best practices 152

classes, organizing 153, 154

command, creating 146-149

disadvantages 143, 144

files, organizing 153, 154

implementing 144, 145

MediatR, adding 145

naming conventions 152

query, creating 149-151

read operations, speeding up 155-157

**Command-Query Segregation (CQS) pattern** 140

**Common Language Runtime (CLR)** 19

**concurrency management** 91

**constraints** 204

**containerd** 380

**container images** 381-383

**containerization** 379, 414

**containerized application**

launching 389-391

**container orchestration** 396

**container registry** 392

**containers** 15

cons 384, 385

pros 383-385

publishing, to container registry 392-396

using, in development 378-380

**controllers** 63

**controller-view pattern** 314

**cookies** 345

**core failure scenarios, between microservices** 250, 251

asynchronous API failures 253-255

synchronous API failures 251-253

**Correlation IDs**

practical purposes 222

**Couchbase** 198

**CRI-O** 380

**cross-site request forgery (CSRF)** 296

**CRUD operations** 33, 315

**cumulative latency** 53

**custom claims** 341

## D

**Dapper** 150, 156

**data access layer** 5

**database** 5

**database architecture patterns**

database per service 189, 190

exploring 188

shared database 190-193

**database development** 193

database, rolling back 205

database technology, selecting 200-202

non-relational databases 198, 199

ORM selection 202, 203

relational databases 194-197

technique, selecting 203-207

**database development technique**

code-first 204

schema-first 204

selecting 203

**database per service** 188

- databases, best practices** 207
  - database sharding 209-213
  - eventual consistency 213, 214
  - polyglot persistence strategy 207-209
- database update command** 204
- data confidentiality and integrity** 342
- data management**
  - in distributed applications 487, 488
- data synchronization** 190
- data types** 204
- dead-letter queue (DLQ)** 266
- Debian** 387
- deduplication tokens** 253
- delivery types**
  - at-least-once delivery 94
  - at-most-once delivery 94
  - exactly once delivery 94
- denial-of-service (DoS) attack** 256, 277
- denormalization** 155
- dependency injection (DI)** 423
- Distributed Application Runtime (Dapr)** 469
- distributed denial-of-service (DDoS) attack** 280
- distributed tracing** 460, 461
- Docker** 15, 380, 381
- Docker Compose** 293, 397
  - adding 397-402
- Docker daemon (dockerd)** 381
- Dockerfile** 385
  - working with 385-389
- Docker Hub** 381
- Docker image** 459
- Docker, key objects**
  - networks 381

- plugins 381
  - volumes 381
- document database** 199
- Document Object Model (DOM)** 309
- Domain-Driven Design (DDD)** 23, 311, 486
  - benefits 28
  - drawbacks 28, 29
  - microservices, using 30
  - significance 24-27
- downstream routes** 285
- dual write problem** 254
- durable buffer** 267

## E

- Elastic Container Registry (ECR)** 381
- Elastic Database tools** 210
- Elasticsearch, Logstash, and Kibana (ELK)** 457
- elmah.io** 455
- enterprise IdPs** 356
- entities** 31
  - need for 31
  - practical uses in code 32, 33
- Entity Framework Core (EF Core)** 19, 61, 150, 202
- Envoy** 466
- error boundary components**
  - implementing 328
- event bus** 93
- event definition**
  - event metadata 94
  - event schema 93
  - event type 93
  - event versioning 94
- event messages** 91

**events 160-162**

- advantages 160, 161
- key attributes 161, 162

**event sourcing 159, 160, 189, 213**

- cons 163
- pros 163

**event sourcing, best practices 178**

- appropriate data store and ORM, selecting 180, 181
- events, designing 178, 179
- snapshots, using for performance optimization 179, 180

**event-sourcing patterns**

- domain events, implementing 171
- domain events, implementing in application 171-175
- events, using for read-only operations 175-177
- implementing 170, 171
- in microservices 162, 163

**event store**

- creating 164, 165
- event storage, scoping with non-relational database 166-168
- event storage, scoping with relational database 165, 166
- implementing, with PostgreSQL 169, 170

**EventStoreDB 180****event streaming 88****eventual**

- consistency 47, 91, 92, 190, 213, 214

**Express Workflows tier 232****F****F# 19****façade 281****fault-handling policies implementation, in asynchronous communication 265**

- outbox pattern, using 267-270
- resiliency, in message brokers and queues 266, 267

**fault-handling policies implementation, in synchronous communication 254-256**

- authentication errors, reducing 262-265
- caching layer, adding 257-262
- circuit breaker policy, adding 256, 257

**Firebase 418****First-in-First-out (FIFO) queues 88****foreign key 195****full stack developer role 193****function chaining 425****Functions as a Service (FaaS) 413, 414****G****General Data Protection Regulation (GDPR) 448****Globally Unique Identifier (GUID) value 31****Google Cloud Functions 15****Google Cloud Platform (GCP) 14, 418**

- Cloud Functions 418

**graph database 199****gRPC 53, 86**

- benefits 75

**gRPC API communication**

- gRPC service, communicating with 79-82
- implementing 74-76

**gRPC microservice**

- creating 76-79

## H

- HAProxy** 281
- Harbor** 393
- hash-based sharding** 210
- health checks** 14
- Health Insurance Portability and Accountability Act (HIPAA)** 448
- heavy API gateway** 280
- heterogenous development stack** 190
- HTTP** 86
- HTTP/1.1** 76
- HTTP/2** 76
- HTTP API communication** 54
  - HTTPClient, using 70-74
  - implementing 54, 55
  - RESTful microservice, implementing 60-68
  - RESTful microservice, testing 68-70
  - SOAP, for API development 56-60
- HTTP communication** 53
- HTTP response code** 55
- HTTP/REST** 309
- HTTP-triggered function** 436
- HyperText Markup Language (HTML)** 308

## I

- IBM DB2** 195
- idempotency key** 225
- Identity library** 350
- identity provider (IdP)** 344-356
  - enterprise IdPs 356
  - social IdPs 356
- IdentityServer** 356
  - API, securing 366-369
  - authentication flow 357

- Ocelot API gateway, securing 369-373
  - recommended implementation style 357

### **IdentityServer service**

- setting up 357

### **ILogger interface** 449

### **image** 379

### **immutable** 382

### **Infrastructure as a Service (IaaS)** 417

### **integrated development environment (IDE)** 18

### **Interface Definition Language (IDL)** 76

### **invariant** 42

### **ISO/IEC 27001** 448

## J

### **Jaeger** 461, 462

### **JavaScript** 309, 418

### **JavaScript frameworks**

- versus Blazor 316

### **JSON** 258

### **JSON Web Token (JWT)** 264, 281, 344, 346

### **jwt.io** 348

## K

### **key-value database** 199

### **key-value store** 194

### **Knative** 418

### **kubectrl**

- reference link 404

### **Kubeless** 418

### **Kubernetes**

- using 403-407

## L

**LINQ syntax** 203

**Log4Net** 455

**log aggregation** 457, 458

distributed log tracing 460, 461

integrating with Seq 458-460

**log levels** 448

**long-term support (LTS) release** 17

## M

**machine learning APIs** 418

**macrofrontend** 310

**man-in-the-middle (MITM) attacks** 296, 342

**many-to-many relationships** 196, 197

**Markup (JAMstack)** 418

**MassTransit** 226

**materialization** 203

**Material UI** 309

**MediatR** 145

features 145

**message broker** 93

**message bus systems** 89, 90

**message queues** 88

interacting with 89

**micro frontends** 307-311

and DDD 311-313

API gateways, using 321-323

communication 331

designing 314-316

developing 314-316

performance optimization 324

real-world implementations 313, 314

state management 331

user interface (UI) 308

user session state, sharing 331-335

**microservices** 8, 311

benefits 8, 13, 14

developing 486, 487

securing, challenges 343

versus monolith 9-12

**microservices designing, with serverless** 421

dependency injection, in serverless

functions 423

durable orchestration design 424-427

integration technologies and

triggers 428, 429

logging 427

monitoring 427

observability 427

right size for function, selecting 422

security considerations 427

**microservices design patterns** 15, 16

**Microsoft Azure** 416

**Minikube**

reference link 404

**models** 31

anemic domain model 34-37

rich domain model 34-37

**Model-View-Controller (MVC)**

framework 314

**modular monolith development pattern** 7

benefits 7, 8

**module federation** 325

**MongoDB** 198

**monitoring** 445

**monitoring and service resilience**

implementing 489

**monolith** 5

challenges 6

**multi-factor authentication (MFA)** 342

**mutual TLS (mTLS)** 465

**MVC** 321

**MySQL/MariaDB** 195

## **N**

**.NET** 16

benefits 16, 17

frameworks 18, 19

languages 18, 19

libraries 18, 19

version 17

**.NET 8.0 (LTS)**

reference link 17

**.NET 9 (STS)**

reference link 17

**.NET Aspire** 474

using 474, 475

using, for microservices 475-481

**.NET Core** 16

**.NET Intermediate Language (IL) interpreter**  
325

**.NET logging API** 449

**.NET MAUI** 321

**native .NET container**

using 392

**NATS** 96

**Netlify Functions** 418

**network hop** 53

**Nexus** 393

**NGINX** 281

**NLog** 455

**normalization** 195

**NoSQL databases** 198

document database 199

graph database 199

key-value database 199

**no-tracking queries** 155

**n-tier architecture** 6

## **O**

**Object Relational Mapping (ORM)** 202

**observability** 446

**Ocelot** 285

built-in features 285, 286

for API gateway implementation 286-291

for BFF pattern implementation 292-295

**Ocelot API gateway**

securing, with IdentityServer 369-373

**one-to-many relationships** 196

**one-to-one relationships** 196

**Open Authorization (OAuth) 2.0** 354, 355

grant types 355

**Open Container Initiative (OCI)** 380

**OpenFaaS** 418

**OpenID Connect (OIDC)** 354, 355

**OpenTelemetry (OTel)** 14, 455, 461

distributed tracing 461-464

**operating system (OS) kernel** 413

**Oracle Database** 195

**orchestration** 231

cons 243, 244

implementing 231-243

pros 243, 244

rolling back, on failure 234, 235

**outbox pattern** 213, 253

using 267-270

**outbox table** 253



## P

- parent image 382
- partition key 212
- PatientService 475
- Payment Card Industry Data Security Standard (PCI DSS) 448
- performance optimization,
  - micro frontends 324
  - ahead-of-time compilation,
    - implementing 324, 325
  - lazy loading components 325-327
  - resilient micro frontend, building 328-330
- personally identifiable information (PII) 342
- Platform as a Service (PaaS) 417
- Podman 380
- PodSecurityPolicies 384
- Polly 19, 252
- Polly-powered resilience policies 288
- polyglot microservices architectures 76
- polyglot persistence 188, 189, 207
  - benefits 208
  - implementation considerations 208, 209
- PostgreSQL 195
- Postman 365
- presentation layer 5
- Progressive Web Apps (PWAs) 315
- Protocol Buffers (Protobuf) 75
- proxy 81
- PublishAot 325
- publish-subscribe (pub-sub) model 88
- PublishTrimmed 325
- pub-sub pattern 93
  - event definition 93
  - event handling 95

- events, publishing 94
- events, subscribing to 94
- implementing 93, 94

## Q

- query 149
- queue-triggered function 438

## R

- RabbitMQ 88, 96, 226
  - installing 97
  - management plugin 98
- range-based sharding 210
- rate limiting
  - implementing, in API gateways 300-303
- Razor 321
- Razor Class Library (RCL) 326
- React 309, 316
- record types 38
- Redis 470
- Redis cache 258
- RedisInsight 383
- referential integrity 195
- relational databases 194
  - management systems 195
- Remote Procedure Call (RPC) 74
- replaying 162
- Representational State Transfer (REST) 56, 59
  - features 59, 60
- REST Client 68
- RESTful APIs 53

**RESTful microservice**

- implementing 60-62
- testing 68-70

**retry pattern 53****reverse proxy 280**

- for API gateway implementation 280, 281
- key functions 280

**rich domain model**

- versus anemic domain model 34-37

**role-based access control (RBAC) 341****S****saga pattern 16, 218**

- exploring 218, 219
- issues and considerations 219-221

**schema per service 188, 191****search engine optimization (SEO) 314****seccomp 384****security 340**

- implementing, in API gateways 296-299

**Seq 458****Serilog 455****serverless architecture 412**

- cloud providers and serverless technologies 415-418
- key characteristics 413
- serverless computing 413, 414
- serverless, versus containerized microservices 414, 415

**serverless architectures 15****serverless computing 15, 412, 413, 419**

- advantages 419
- drawbacks 420, 421
- for microservices 419, 420

**serverless microservices, on Microsoft Azure**

- appointment booking service, building on Azure Functions 432-442
- building 429, 430
- local toolset, for Azure Functions development 431, 432

**service contract 76****service meshes 467**

- benefits 468
- drawbacks 468

**service-oriented architecture (SOA) 309, 310**

- client 274
- database 274
- server 274

**Service resource 406****service unavailability 252****sessions 345****sharding 209**

- hash-based sharding 210
- range-based sharding 210

**sharding key 210****Shard Map Manager 210****shards 209****shared database 190-193****sidecar 464**

- with Dapr 469-474

**Sidecar pattern 464**

- benefits 465, 468
- drawbacks 468
- implementing 464-467

**SignalR 315****signed cookie 341****Simple Object Access Protocol (SOAP) 56**

- for API development 56-59

**Single-Page Application (SPA) 116**

- Single Responsibility Principle (SRP) 153
- single sign-on (SSO) 355
- SOAP 309
- social IdPs 356
- software development
  - approaches 4
- spaghetti code situation 6
- Spotify 313
- SQLite 155, 195
  - database 61
- SQL Server 195
- standard-term support (STS) release 17
- state 345
- stateless APIs 346
- structured logging
  - implementing 446, 447
  - log requirements, determining 447-449
  - .NET logging API, using 449-454
  - Serilog, adding 455-457
- stub 81
- synchronous API failures
  - handling 251-253
- synchronous communication 52, 86
  - characteristics 52
  - fault-handling policies,
    - implementing 254-256
  - inter-service operation
    - considerations 53, 54
- Syntactically Awesome Style Sheets (Sass) 309

## T

- tables per service 188, 191
- Tailwind CSS 309
- TCP handshake 54
- Temporal 231
- thick API gateway 280
- thin API gateway 280
- thin-edge component 281
- token authentication
  - implementing 357-366
- tokens 345
- transactional outbox table 213
- tree shaking 325
- Trivy 384
- TypeScript 316

## U

- upstream routes 285

## V

- value objects 37
  - using 38-41
- Vercel Serverless Functions 418
- vertical slice architecture 153
- virtual machines (VMs) 412
- Visual Basic .NET (VB.NET) 19
- Visual Studio Code (VS Code) 18, 61, 386
  - URL 18
- Visual Studio (VS) 18
  - reference link 18
- Vue 316
- Vue.js 309

## W

**W3C trace context** 472

### **web frontend development**

Cascading Style Sheets (CSS) 309

HyperText Markup Language (HTML) 308

JavaScript 309

## Y

**Yet Another Markup Language (YAML)** 385

**Yet Another Reverse Proxy (YARP)** 281

for API gateway implementation 281-285

## Z

**Zipkin** 470





packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

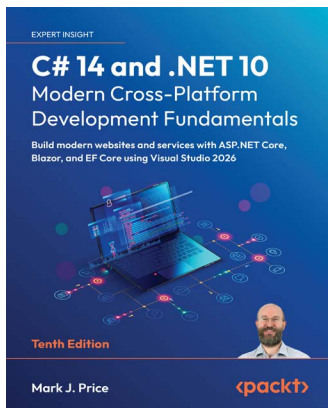
## Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At [www.packt.com](http://www.packt.com), you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

# Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



## **C# 14 and .NET 10 – Modern Cross-Platform Development Fundamentals**

Mark J. Price

ISBN: 978-1-83620-663-7

- Discover the new features in .NET 10, including the NumericOrdering option and OrderedDictionary enhancements
- Leverage the latest ASP.NET Core 10 features, such as OpenAPI 3.1 document generation, YAML format support, and populating XML doc comments
- Use native AOT publishing for faster startup and reduced memory footprint
- Build rich web user interface experiences with Blazor
- Integrate databases into your applications using EF Core 10 entity models
- Query and manipulate data using LINQ
- Develop powerful web services with Minimal APIs



## Real-World Web Development with .NET 10

Mark J. Price

ISBN: 978-1-83588-892-6

- Build web applications using ASP.NET Core MVC with well-structured, maintainable code
- Develop secure and scalable RESTful services using Web API and OData
- Implement authentication and authorization for your applications
- Test and containerize your .NET projects for smooth deployment
- Optimize application performance with caching and other techniques
- Learn how to use and implement Umbraco CMS



## Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit [authors.packtpub.com](https://authors.packtpub.com) and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

## Share your thoughts

Now you've finished *Microservices Design Patterns in .NET, Second Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

